Springer Optimization and Its Applications 170

Panos M. Pardalos
Varvara Rasskazova
Michael N. Vrahatis *Editors*

Black Box Optimization, Machine Learning, and No-Free Lunch Theorems

Springer

Springer Optimization and Its Applications

Volume 170

Series Editors
Panos M. Pardalos, *University of Florida*
My T. Thai, *University of Florida*

Honorary Editor
Ding-Zhu Du, *University of Texas at Dallas*

Advisory Editors
Roman V. Belavkin, *Middlesex University*
John R. Birge, *University of Chicago*
Sergiy Butenko, *Texas A&M University*
Vipin Kumar, *University of Minnesota*
Anna Nagurney, *University of Massachusetts Amherst*
Jun Pei, *Hefei University of Technology*
Oleg Prokopyev, *University of Pittsburgh*
Steffen Rebennack, *Karlsruhe Institute of Technology*
Mauricio Resende, *Amazon*
Tamás Terlaky, *Lehigh University*
Van Vu, *Yale University*
Michael N. Vrahatis, *University of Patras*
Guoliang Xue, *Arizona State University*
Yinyu Ye, *Stanford University*

Aims and Scope

Optimization has continued to expand in all directions at an astonishing rate. New algorithmic and theoretical techniques are continually developing and the diffusion into other disciplines is proceeding at a rapid pace, with a spot light on machine learning, artificial intelligence, and quantum computing. Our knowledge of all aspects of the field has grown even more profound. At the same time, one of the most striking trends in optimization is the constantly increasing emphasis on the interdisciplinary nature of the field. Optimization has been a basic tool in areas not limited to applied mathematics, engineering, medicine, economics, computer science, operations research, and other sciences.

The series **Springer Optimization and Its Applications (SOIA)** aims to publish state-of-the-art expository works (monographs, contributed volumes, textbooks, handbooks) that focus on theory, methods, and applications of optimization. Topics covered include, but are not limited to, nonlinear optimization, combinatorial optimization, continuous optimization, stochastic optimization, Bayesian optimization, optimal control, discrete optimization, multi-objective optimization, and more. New to the series portfolio include Works at the intersection of optimization and machine learning, artificial intelligence, and quantum computing.

Volumes from this series are indexed by Web of Science, zbMATH, Mathematical Reviews, and SCOPUS.

More information about this series at http://www.springer.com/series/7393

Panos M. Pardalos • Varvara Rasskazova
Michael N. Vrahatis
Editors

Black Box Optimization, Machine Learning, and No-Free Lunch Theorems

Springer

Editors
Panos M. Pardalos
Department of Industrial & Systems Engineering
University of Florida
Gainesville, FL, USA

Varvara Rasskazova
Moscow Aviation Institute
Moscow, Russia

Michael N. Vrahatis
Mathematics Department
University of Patras
Patras, Greece

ISSN 1931-6828 ISSN 1931-6836 (electronic)
Springer Optimization and Its Applications
ISBN 978-3-030-66514-2 ISBN 978-3-030-66515-9 (eBook)
https://doi.org/10.1007/978-3-030-66515-9

Mathematics Subject Classification: 90C90, 90C56, 68T01, 90C26

© Springer Nature Switzerland AG 2021
This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Preface

This is the second published book of the "Deucalion Summer Institute for Advanced Studies in Optimization, Mathematics, and Data Sciences," which was established in 2015 by Panos M. Pardalos at Drossato in the mountainous region Argithea of Thessaly in Central Greece. The first published book was on "Open Problems in Optimization and Data Analysis."[1] The focus of the Deucalion Summer Institute is to organize summer schools that concentrate on certain aspects of recent mathematical developments and data science. In such schools, each day is dedicated to discussions regarding one specific topic. The idea is to encourage thinking "out of the box" to generate new ideas, initiate research collaboration, and identify new research directions. The summer schools are inspired by and organized very much in the spirit of the "peripatetic school/lyceum" of Aristotle.

In general, *Machine Learning* (ML) deals with the development and assessment of algorithms that enable computer systems to learn by trial and error, that is, to improve with more "experience" their performance with respect to some task without being explicitly programmed for it. ML methods have been applied in numerous real-world problems and tasks with big success. A few examples include medical diagnosis, recommendation systems, computer vision, robotics, sentiment analysis, time-series analysis and prediction, and many more.

Almost all ML methods require some optimization process to be performed. For example, in deep learning (neural network training), we use analytical derivatives (via back-propagation) and stochastic gradient descent to minimize the loss function at hand. However, not all problems or loss functions can have analytical forms and/or gradients. These problems are usually referred to as "black-box systems," and we assume that this black box can be queried through a simulation or experimental measurements that provide a system output for specified values of inputs. In these cases, we need a different class of optimization algorithms that can handle these unique type of problems. The *Black-Box Optimization* (BBO) field aims to create optimization methods that are able to efficiently optimize black-box functions.

[1] https://www.springer.com/gp/book/9783319991412.

Many optimization algorithms claim to be superior to other methods in optimizing these black-box functions. The *No-Free Lunch Theorems*, though, prove that under a uniform distribution over induction problems, all induction algorithms perform equally. In essence, even if an algorithm performs better in one class of optimization problems, there will always be another algorithm that performs better in another class of optimization problems.

This edited volume collects 13 essays on the above topics ranging from small notes to reviews and novel theoretical results. The goal of this volume is to inform the interested readers about the latest developments, and future perspectives on optimization methods. It is intended not only for beginners to get a wide overview of optimization methods but also for more experienced readers who want to get a quick reference of important topics and methods. The works cover from mathematically rigorous methods to heuristic and evolutionary approaches in an attempt to equip the reader with different viewpoints of the same problem.

In particular, in Chapter "Learning Enabled Constrained Black-Box Optimization", Archetti et al. attempt to tackle the problem of black-box constrained optimization where both the objective function and the constraints are unknown. They provide analysis on the modelling and computational issues of a novel algorithm that is based on Support Vector Machines and Gaussian Processes.

In Chapter "Black-Box Optimization: Methods and Applications", Bajaj et al. present a review of the latest black-box optimization algorithmic developments (both for simple and constrained optimization), along with many applications in many domains (machine learning, fluid mechanics, chemical engineering, etc.).

In Chapter "Tuning Algorithms for Stochastic Black-Box Optimization: State of the Art and Future Perspectives", Bartz-Beielstein et al. provide a review of tuning algorithms for stochastic black-box optimization methods. They also discuss and compare many tuning software packages that are available.

In Chapter "Quality-Diversity Optimization: A Novel Branch of Stochastic Optimization", Chatzilygeroudis et al. provide a gentle introduction to Quality-Diversity algorithms that instead of searching for the optimum of the cost function, they try to illuminate the whole search space by providing a large set of high-performing but diverse solutions. Throughout the chapter, many applications (e.g., robotics, reinforcement learning, deep learning) of Quality-Diversity optimization are presented and discussed.

In Chapter "Multi-Objective Evolutionary Algorithms: Past, Present, and Future", Coello Coello et al. attempt to review the evolutionary algorithms that attempt to solve multi-objective optimization problems. The chapter covers works on the topic from early 1980s to recent techniques, as well as detailing possible paths for future research in the area.

In Chapter "Black-Box and Data-Driven Computation", Jin et al. present a quick note with observations on the usage of black-box optimization algorithms in studying computational complexity theory with the use of big data.

In Chapter "Mathematically Rigorous Global Optimization and Fuzzy Optimization", Ralph Baker Kearfott provides an overview of mathematically rigorous

global optimization and fuzzy optimization literature with discussions of many comparisons and applications.

In Chapter "Optimization Under Uncertainty Explains Empirical Success of Deep Learning Heuristics", Kreinovich and Kosheleva present how the success of many well-established heuristics in the machine learning literature (especially in deep learning) can be explained by optimization-under-uncertainty techniques.

In Chapter "Variable Neighborhood Programming as a Tool of Machine Learning", Mladenovic et al. propose a novel technique for automated programming inspired by the Variable Neighborhood Search algorithm. The efficacy and efficiency of the method are tested in many experiments, including symbolic regression, prediction, and classification problems.

In Chapter "Non-lattice Covering and Quantization of High Dimensional Sets", Noonan and Zhigljavsky present a novel method for constructing efficient n-point coverings of a d-dimensional cube. They also present a theoretical analysis of this method, along with practical recommendations.

In Chapter "Finding Effective SAT Partitionings Via Black-Box Optimization", Semenov et al. develop a novel method for partitioning hard instances of the Boolean satisfiability problem (SAT). The main finding of the work is that a SAT partitioning problem can be formulated as the problem of minimizing a special pseudo-Boolean black-box function.

In Chapter "The No Free Lunch Theorem: What Are its Main Implications for the Optimization Practice?", Loris Serafino discusses the practical meaning and implication of No-Free Lunch Theorems for practitioners facing real-life industrial and design optimization problems.

In Chapter "What Is Important About the No Free Lunch Theorems?", David H. Wolpert provides a review of the No-Free Lunch Theorems and explains the real-world implications of their findings. One interesting implication is that selecting hypotheses using cross-validation does not free you from the need for inductive bias, because cross-validation is no better than any other selection algorithm under a uniform prior.

Acknowledgments We would like to thank all the authors for their efforts and contributions. P. M. Pardalos was supported by a Humboldt research award (Germany) and the Paul and Heidi Brown Preeminent Professorship at the ISE, University of Florida (USA).

Gainesville, FL, USA — Panos M. Pardalos

Moscow, Russia — Varvara Rasskazova

Patras, Greece — Michael N. Vrahatis

Contents

Learning Enabled Constrained Black-Box Optimization 1
F. Archetti, A. Candelieri, B. G. Galuzzi, and R. Perego

Black-Box Optimization: Methods and Applications 35
Ishan Bajaj, Akhil Arora, and M. M. Faruque Hasan

Tuning Algorithms for Stochastic Black-Box Optimization: State of the Art and Future Perspectives 67
Thomas Bartz-Beielstein, Frederik Rehbach, and Margarita Rebolledo

Quality-Diversity Optimization: A Novel Branch of Stochastic Optimization 109
Konstantinos Chatzilygeroudis, Antoine Cully, Vassilis Vassiliades, and Jean-Baptiste Mouret

Multi-Objective Evolutionary Algorithms: Past, Present, and Future 137
Carlos A. Coello Coello, Silvia González Brambila, Josué Figueroa Gamboa, and Ma. Guadalupe Castillo Tapia

Black-Box and Data-Driven Computation 163
Rong Jin, Weili Wu, My T. Thai, and Ding-Zhu Du

Mathematically Rigorous Global Optimization and Fuzzy Optimization 169
Ralph Baker Kearfott

Optimization Under Uncertainty Explains Empirical Success of Deep Learning Heuristics 195
Vladik Kreinovich and Olga Kosheleva

Variable Neighborhood Programming as a Tool of Machine Learning 221
Nenad Mladenovic, Bassem Jarboui, Souhir Elleuch, Rustam Mussabayev, and Olga Rusetskaya

Non-lattice Covering and Quantization of High Dimensional Sets 273
Jack Noonan and Anatoly Zhigljavsky

Finding Effective SAT Partitionings Via Black-Box Optimization 319
Alexander Semenov, Oleg Zaikin, and Stepan Kochemazov

The No Free Lunch Theorem: What Are its Main Implications for the Optimization Practice? 357
Loris Serafino

What Is Important About the No Free Lunch Theorems? 373
David H. Wolpert

Learning Enabled Constrained Black-Box Optimization



F. Archetti, A. Candelieri, B. G. Galuzzi, and R. Perego

1 Introduction

Optimization methods have become ubiquitous in all branches of science, business and government well beyond the traditional areas of engineering design and operations management. This has led to the emergence of a large spectrum of optimization settings each with a specific armoury of models and algorithms.

At one end of the spectrum, the problem exhibits features like linearity or convexity, derivatives are available, evaluations of the objective function tend to be cheap and noise is negligible. These features have been leveraged into extremely effective computational methods able to handle very efficiently millions of variables and constraints.

The next area in the optimization landscape is related to the case where derivatives are not available, and evaluations of the function are still relatively cheap. This is the domain of derivative-free methods [48] and, when noise is significant, stochastic optimization [60].

Stochastic algorithms are largely used in this domain: random search [80, 82], simulated annealing, evolutionary computation [3, 35, 51] and swarm intelligence [57]. These algorithms converge, albeit slowly, to a local minimum and also have global properties driven by their random elements and their population-based structure. A "kernelization" of genetic approach, the Covariance Matrix

F. Archetti
University of Milano-Bicocca, Milan, Italy

Consorzio Milano-Ricerche, Milan, Italy

A. Candelieri (✉) · B. G. Galuzzi · R. Perego
University of Milano-Bicocca, Milan, Italy
e-mail: antonio.candelieri@unimib.it

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_1

Adaptation algorithm, uses the information collected during the generations to learn the structure of the "fitness" function [2].

These methods have been initially applied to problems with box constraints and have been recently extended to problems with black-box constraints. To mitigate the issue of slow convergence, different solutions have been proposed including an "accelerated" random search proposed in [55] and extended to problems with black-box inequality constraints.

The same tools can be applied in the related field of simulation-based optimization [4], which presents some specific challenges: convexity cannot be assumed, the evaluation of the objective function and constraints can be done through computer simulations or physical experiments and can also be very expensive, the uncertainty about the model is structural and not only observation noise and derivative observations are not to be expected. This problem is usually referred to as "black-box" optimization (BBO).

In these cases, the time requirements of the optimization process are dominated by the cost of the simulation model: a constraint on the budget available is typically active, requiring a finite horizon efficiency and not just asymptotic convergence.

These are the cases as evaluating in silico the response of a patient in a clinical trial or simulating, ab initio, the pharmacological activity of a new chemical compound [18, 19, 54] or quantifying the uncertainty in protein docking [16]. These models may take several days to run and require a significant amount of resources before one data point is obtained even relying on state-of-the-art computational methods and high-performance computing.

Despite these drawbacks, in silico modelling represents an attractive alternative to physical experiments which can be even more expensive, time consuming and may also impose some constraints due to ethical considerations. The same problems are also found in many engineering design applications specifically when finite element method or fluid-dynamic computations are involved.

BBOs have been considered first in "essentially unconstrained" conditions, where the solution was searched for within a bounded-box search space. Recently, due to methodological and application reasons, there has been an increasing interest in the generalization of black-box optimization to the constrained case (constrained black-box optimization, CBBO):

$$x^* = \arg\min_{x \in X} f(x)$$

Subject to

$$c_i(x) \leq 0 \; i = 1, \ldots, n_c.$$

In which, we assume that one or more of the functions can be computed only pointwise through input–output black boxes.

A paper of general interest about a taxonomy of constraints in simulation-based optimization is given in [21]. The mostly used approach for the optimization of computationally expensive functions is based on surrogate models (a.k.a. metamodels or

response surface) given by multivariate relatively inexpensive approximation of the black-box function: function and constraints are evaluated in an initial set of points from a space-filling design (Latin hypercube sampling [LHS], pseudo-uniform or quasi-random) to obtain initial surrogates which are used to drive the selection of new points where the objective and constraints are evaluated enabling to update the model in order to allow the sequential selection of new points.

This approximate model can be local, as in the case of DF trust-region methods, or global as originally proposed in [39, 40] and is updated as new function evaluations become available.

Surrogate-based methods, originally proposed for box-constrained problems, have been extended to nonlinear constraints in [63]. A recent wide survey about surrogate approaches for constrained BB optimization is in [4] and [61].

The key questions are the choice of the model, its updating procedure and most importantly the strategy that drives the choice of the next evaluation point. A general remark is that as simulators become more versatile and complex, their reliability might degrade generating convergence failures. This can be due to instability of the numerical scheme [67] in a fluid-dynamics solver. In addition, for example in a hydraulic simulator the input can generate pressures or flows physically impossible [74]. This situation is also frequent in the case of the training of a machine learning model in which the gradient (stochastic) descent may fail to converge.

A closely related case is when the objective function is undefined and cannot be therefore computed, outside the feasible region. In this case, we speak about partially defined objective functions [66, 69], "crash constraints" [8] or "non-computable domains" [67]. Also in this case the evaluation of the objective requires the execution of a simulation model which returns a valid output or a failure message. A solution strategy, which will be analysed in Sect. 5, splits the constrained problem into two phases: feasibility determination that is formulated as a classification problem and the optimization phase that takes place only in the feasible region.

This chapter fits into the fast-growing literature on the relation between optimization methods and machine leaning so that the two fields are now tightly interwoven. It is well known that the optimization approaches have been mainstreamed at least in ML in the last 25 years, initially in the classical formulation of SVM as a quadratic programming problem, thereafter with convex relaxations and stochastic gradients and more recently regularized optimization. These optimization tools are important in so many ML applications that one could speak of *optimization enabled machine learning* [38, 71].

Still that relation is two-way: less known, equally important and close to the focus of this chapter, is learning enabled optimization that is how "learning" can be incorporated into different optimization frameworks. Outside the focus of this chapter, one can consider, for instance, in [68], how the learning enabled paradigm can be applied to the two-stage stochastic programming problem. Even relatively common applications, like algorithmic configuration and hyperparameter optimization in Automated Machine Learning (AutoML), lead in general to black-box optimization and require learning enabled optimization methods [23].

The issue of efficiency of metaheuristics is related to a set of general results collectively known as the No Free Lunch (NFL) theorems [1] that state that one cannot find algorithms optimal for all instances in a wide class of problems.

Therefore, in order to obtain an efficient algorithm for a specific problem, it is necessary to find methods that learn from data and are able to find structure in the objective function and exploit this knowledge for data-efficient black-box optimization. The central requirements for optimization methods in this domain are:

– "sample efficiency" because the cost of observations—i.e. function evaluations—is the dominating cost and
– global properties that can leverage all the available information towards the selection of highly informative new evaluations.

To be sample-efficient, the algorithm must use the surrogate model that incorporates the knowledge from previously evaluated data and query, at a cost comparatively low with respect to evaluations of the objective function, the design space to find the new point with maximal informative value.

This can be enabled only by a learning process that is in turn centred on an issue, the so-called exploration vs. exploitation dilemma: exploration means devoting resources to learn about the structure of the problem, in particular possible solutions, while exploitation devotes resources to improve on solutions already identified in previous phase. We could also associate "pure" exploration to pure random search and exploitation with local search.

The search for the new point must strike an effective balance between the needs of exploration and exploitation. The learning process requires a model of the objective function: "All models are wrong but some are useful", wrote famously George Box [13]. Different kinds of surrogate models have been shown to be useful in CBBO. Our black-box optimizer should be "learning enabled", able to augment its dataset of models and function evaluations in order to select sequentially "high value" points.

The need to solve the exploration/exploitation dilemma is common to all surrogate-based methods: how they solve it, modelling and updating in different ways the surrogate and how the informative value of new points is computed balancing exploration and exploitation are the main differentiating drivers. Surrogate-based methods can be broadly classified into deterministic and probabilistic:

– Among deterministic ones, there are derivative-free trust-region methods, direct search methods or component-based methods like, radial basis function methods, or best subset techniques [20].
– Among probabilistic ones are prominent GP-based methods, such as kriging (Klejinen [43]; Bhosekar and Ierapetritou [11], Mehdad and Kleijnen [52]) and Bayesian Optimization [5, 40, 53, 82]. In the black-box context, only a finite sample dataset is available: locating the optimum becomes an inferential problem which leads naturally to a Bayesian framework and locating promising areas to infer, in Bayesian term, requires an a posteriori model of the objective

function. The power of the GP-based approach lies in its ability to tackle optimization problems with a limited number of function evaluations and in the mathematically principled way in which it tackles the exploration–exploitation dilemma and the connected issue of generalization.

The main contributions of this chapter are:

1. a unifying analysis of surrogated models both deterministic and probabilistic built around the different solutions to the exploration–exploitation dilemma,
2. learning enabled optimization and optimization enabled learning, ML and optimization methods are increasingly interwoven bringing up relevant contributions to the black-box optimization domain, originated also in the machine learning community and
3. analyzing a recent very challenging field in CBBO when crash constraints or partially defined functions transform the very nature of the optimization problem into a combined classification/optimization one.

Other topics strongly related to surrogate models, CBBO, are "multi-fidelity and "multi-source" optimization and "meta-learning", which have been recently gaining importance to mitigate the issue of large cost of function evaluations. We cannot deal with them, for space constraints, but we can, at least, indicate the main references.

Multi-fidelity and Multiple Sources Optimization The availability of data from physical experiments beside the output of simulation models has motivated the development of multi-source multi-fidelity models that enable the fusion of data from different sources.

This is, for instance, the case in drug design where the multiple sources are large databases of the properties of compounds, in silico simulations and chemical experiments [18, 58]. The goal is to use surrogate models of different precision[59] and integrate information from these different sources [25].

Multi-fidelity optimization has been largely motivated by several applications also in engineering design: the authors in [81] propose an optimization algorithm that guides the search for solutions on a high-fidelity model through the approximation of a level set from a low-fidelity model; an interesting application of multi-fidelity BO is presented for model inversion in hemodynamics [59] and biomedical engineering [18].

A novel approach was proposed in [30] where the CBO problem is solved in the context of multi-information sources. A GP is constructed for each source by using available evaluations and incorporating the correlation within and between information sources. This method enables to "suggest" which information source to query next and when to query it.

Multi-fidelity optimization has also been considered in for the optimization of Machine Learning algorithms [42, 44] and [68], which proposes to train the algorithm on a sub-sampled version of the whole dataset.

A comprehensive multi-fidelity framework, in the context of bandit problems, has been proposed in [42] where the objective function and its approximations are sampled from a GP.

Meta-learning The meta-learning approach to black-box optimization is closely linked to multi-fidelity: it assumes access to a set of functions that are similar to the objective but cheaper to evaluate. A DEEPMIND's report [56] explains how memory-based meta-learning of sequential strategies can offer a tool for building sample-efficient strategies that learn from the past experience. "Transfer Learning" and "Learning to Learn" are a closely related subjects considered in several recent papers. In [17], recurrent neural networks (RNNs) are trained to perform black-box optimization. Another approach is proposed in [75] to learn optimizers that are automatically adjusted to a given class of objective function in the context of BB optimization. The approach is rooted in the classical BO framework, only the acquisition function is replaced by a NN and therefore the resulting algorithm retains the generalization capabilities of the GP.

The structure of the chapter is as follows: after this introduction, Sect. 2 is devoted to deterministic surrogate models and Sect. 3 to the basic probabilistic modelling frameworks whose elements are Gaussian Processes (GPs) and the acquisition functions in GP-based optimization. Section 4 deals with the case of unknown constraints and Sect. 5 with the issue of partially defined functions that generate non-computable domains. Section 6 is dealing with test function generators available for constrained black-box optimization.

2 Constrained Black-Box Optimization

The reference constrained black-box optimization problems are

$$x^* = \arg\min_{x \in X} f(x)$$

Subject to

$$c_i(x) \leq 0 \; i = 1, 2, \ldots, n_c.$$

When one or more of the functions can be computed only pointwise through input–output black boxes, we speak of constrained black-box optimization (CBBO). In this section, we briefly outline the main deterministic approaches. The aim is not to go into detailed analysis but just give a general view, indicating the relevant references, in order to understand the underlying commonalities with the probabilistic approaches of Sects. 4 and 5.

The first approach is Automated Learning of Algebraic Models implemented in the ALAMO solver well-known and widely used solver of black-box constrained global optimization [79].

Its framework consists of a two-step approach, in which:

1. algebraic models of the objective function and the constraints are provided and
2. a global optimization method for algebraic functions is used to locate and certify a globally optimal solution (this step can be performed by any solver, for instance, BARON).

The algebraic model obtained through ALAMO (Automated Learning of Algebraic Models) represents the state-of-the-art computational methodology for learning algebraic functions from data.

First, an initial design of training points x_i, with $i = 1, \ldots, N$, is generated over the problem space $X \subset \mathbb{R}^d$, and the objective function is queried at these points, $y_i = f(x_i)$.

Then, a simple, algebraic model $\tilde{f}(x)$ is built using this initial training dataset $\{x_i, y_i\}$. This model is expressed as a combination of simple basis functions, such as polynomial, multinomial, exponential and logarithmic. The model has a sufficient complexity to represent the objective function accurately while maintaining adequate simplicity to ensure that the optimization process is tractable. The choice of the basis functions and their weights are obtained by solving a set of cardinality constrained mixed-integer quadratic programmes, in which the binary variables define the set of T active basis functions. The above optimization problem formulation is solved by increasing the value of T until the Akaike Information Criterion (AIC) (or some other criterion) is met:

$$AIC\,(S, \beta) = N \cdot \log\left(\frac{1}{N} \cdot \sum_{i=1}^{N}\left(y_i - \sum_{j \in S} \beta_j X_{ij}\right)^2\right) + 2|S| + \frac{2|S|\,(|S|+1)}{N - |S| - 1},$$

where S is a subset of the basis functions. This criterion gives a measure of the accuracy vs. the complexity of the model, which should be balanced by the surrogate model.

The obtained algebraic model is tested subsequently against the simulation results by using an adaptive sampling technique called Error Maximization Sampling (EMS) [78]. The sampling technique is based on a black-box optimization function to find points in the problem space that maximize the squared relative model error:

$$g = \max_{x \in X}\left(\frac{f(x) - \tilde{f}(x)}{f(x)}\right)^2.$$

If the sampling technique yields g values larger than a specified tolerance, the newly sampled data points are added to the training set. The surrogate models are iteratively retrained and improved until the adaptive sampling routine fails to find model inconsistencies. To solve the EMS problem, ALAMO uses the derivative-free optimization SNOBFIT [36]. When the model-refining process is terminated,

a global optimization method is used to locate the globally optimal solution for the algebraic model $\tilde{f}(x)$, provided by ALAMO. More in detail, the Branch-And-Reduce Optimization Navigator (BARON) is used, which is a computational system for the global solution of algebraic nonlinear programmes (NLPs) and mixed-integer nonlinear programmes (MINLPs), where both the objective function and the constrains must be provided explicitly in an algebraic form.

There are other surrogate-based deterministic approaches: a first example of deterministic surrogate is the CORS (Constrained Optimization with Response Surface) [62] solver. Given an initial design, a linear combination of Radial Basis Functions (RBFs) is built to model the objective and constraint, interpolating the initial points. At each iteration, the new evaluation point is obtained by optimizing the response model, satisfying some distance constraints from previously evaluated points. The point selected is evaluated and the response surface updated, consequently. This is a possible solution of the exploration (selecting points in less-explored regions) and exploitation (minimizing the current surrogate model) dilemma.

This approach is further developed in the COBRA (Constrained Optimization by RBF Approximation) solver [65]. In order to avoid frequent updates in the neighbourhood of the best observation, COBRA uses a distance-related constraint that prevents being stuck into local optima.

The same approach has been considered in [45], but linked to a "repair mechanism" for reducing the constraint violation of the solution obtained minimizing the surrogate.

Other significant references to CBBO are [12, 22, 33, 63–65, 76].

3 The Basic Probabilistic Framework

3.1 Gaussian Processes

Gaussian Processes (GPs) are a powerful non-parametric model for implementing both regression and classification. One way to interpret a GP is as a distribution over functions, with inference taking place directly in the space of functions [77]. A GP, therefore, is a collection of random variables, any finite number of which has a joint Gaussian distribution. A GP is completely specified by its mean function $\mu(x)$ and covariance function $cov\left(f(x), f(x')\right) = k(x, x')$:

$$\mu(x) = \mathbb{E}[f(x)]$$

$$cov(f(x), f(x')) = k(x, x') = \mathbb{E}\left[(f(x) - \mu(x))\left(f\left(x'\right) - \mu(x')\right)\right]$$

and will be denoted by

$$f(x) \sim GP(\mu(x), k(x, x')).$$

Usually, for notational simplicity, we will take the prior of the mean function to be zero, although this is not necessary.

A simple example of a Gaussian process can be obtained from a Bayesian linear regression model $f(x) = \phi(x)^T w$ with prior $w = \mathcal{N}(0, \sum_p)$, where $\phi(x)$ and w are p-dimensional vectors, while $\sum_p$ is a $p \times p$ matrix. More precisely, $\phi(x)$ is a function mapping the d-dimensional vector x into a p-dimensional vector.

In this case, the equations for mean and covariance become

$$\mathbb{E}[f(x)] = \phi(x)^T \mathbb{E}[w] = 0$$
$$\mathbb{E}[f(x) f(x')] = \phi(x)^T \mathbb{E}[w w^T] \phi(x') = \phi(x)^T \sum_p \phi(x').$$

Since the function values $f(x_1), \ldots, f(x_n)$ obtained at n different points $x_1, \ldots, x_n$ are jointly Gaussian, the covariance function assumes a critical role in the GP modelling, as it specifies the distribution over functions. To see this, we can draw samples from the distribution of functions evaluated at any number of points; we choose a set of input points $X_{1:n} = (x_1, \ldots, x_n)^T$, then compute the corresponding covariance matrix elementwise

$$f(X_{1:n}) \sim \mathcal{N}(\mathbf{0}, \mathrm{K}(X_{1:n}, X_{1:n}))$$

and plot the generated values as a function of the inputs, where $\mathrm{K}(X_{1:n}, X_{1:n})$ is the covariance matrix, where the entry $\mathrm{K}_{ij} = k(x_i, x_j)$.

Figure 1 displays 5 GP samples drawn from the GP prior: the covariance function used is the Squared Exponential (SE) kernel.

We are usually not primarily interested in drawing random functions from the prior but want to incorporate the knowledge about the function obtained through the evaluations performed so far.

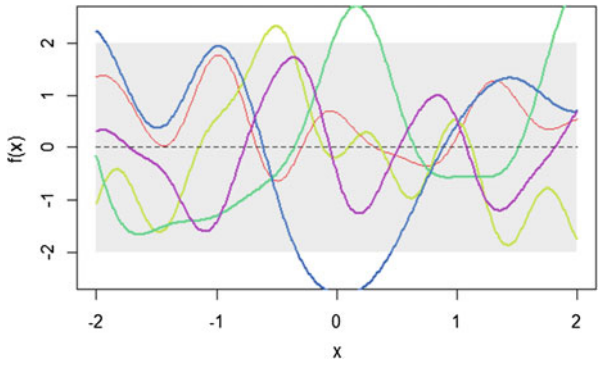


Fig. 1 Five different samples from the prior of a GP with Squared Exponential kernel as covariance function [Source: [6]]

We have usually access only to noisy function values, denoted by $y = f(x) + \varepsilon$. Assuming additive independent identically distributed Gaussian noise ε with variance λ^2, the prior on the noisy observations becomes

$$f\left(X_{1:n}\right) \sim \mathcal{N}\left(\mathbf{0}, \mathrm{K}\left(X_{1:n}, X_{1:n}\right) + \lambda^2 I\right)$$

with

$$cov\left(f(x), f(x')\right) = k(x, x') + \lambda^2 \delta_{xx'},$$

where $\delta_{xx'}$' is a Kronecker delta which is equal to 1 if and only if $x = x'$. Thus, the covariance over all the function values $y = (y_1, \ldots, y_n)$ is

$$cov(y) = \mathrm{K}(X_{1:n}, X_{1:n}) + \lambda^2 I.$$

Therefore, the predictive equations for GP regression, which are $\mu(x)$ and $\sigma^2(x)$, can be easily updated, by conditioning the joint Gaussian prior distribution on the observations:

$$\mu(x) = \mathbb{E}[f(x)|D_{1:n}, x] = \mathrm{k}(x, X_{1:n})[\mathrm{K}(X_{1:n}, X_{1:n}) + \lambda^2 I]^{-1}\mathbf{y}$$
$$\sigma^2(x) = k(x, x) - \mathrm{k}(x, X_{1:n})[\mathrm{K}(X_{1:n}, X_{1:n}) + \lambda^2 I]^{-1}\mathrm{k}(X_{1:n}, x),$$

where $D_{1:n} = (x_i, y_i)_{(i=1,..,n)}$, the vector $\mathbf{y}$ gives the value of the function evaluations performed so far and $\mathrm{k}(x, X_{1:n})$ is an n-dimensional vector with components $k_i = k(x, x_i)$. Figure 2 displays an example of 5 samples drawn at random from a GP prior and posterior, respectively. Posterior is conditioned to 6 function observations. The grey-shaded area is $\pm 2\sigma$.

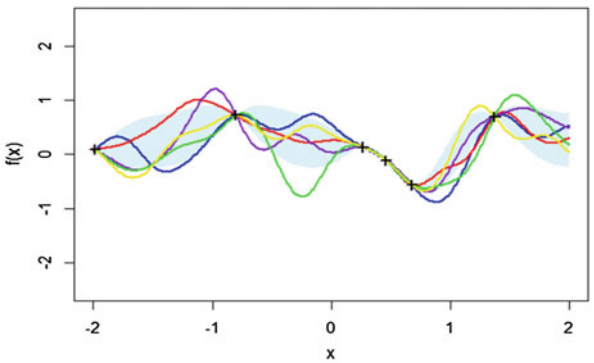


Fig. 2 Sampling from prior vs sampling from posterior (for the sake of simplicity, we consider the noise-free setting) [Source: [6]]

Sampling from posterior can be, ideally, considered as generating functions from the prior and rejecting the ones that disagree with the observations. Naturally, this strategy would not be computationally sensible.

The mean prediction can also be expressed as a linear combination of n radial basis functions, each one centred on an evaluated point. This allows to write $\mu(x)$ as

$$\mu(x) = \sum_{i=1}^{n} \alpha_i k(x, x_i),$$

where the vector $\alpha = [\mathrm{K}(X_{(1:n)}, X_{(1:n)}) + \lambda^2 I]^{-1}\mathbf{y}$ and α_i is the ith component of the vector α. When the chosen kernel is a radial basis function, the approximation, for the mean of the GP, is very close to that suggested by deterministic surrogate models previously outlined. The key difference, between them and GP-based optimization, is that GP is a "probabilistic" surrogate and specific (deterministic) surrogates, which account for the variance, can be sampled from it as shown in Fig. 2. This points to an important relation between GP-based optimization and other approaches using surrogates based on radial basis functions, such as [37, 62].

Several covariance functions have been proposed, and some of the most widely adopted will be presented in the following. Every covariance function has some hyperparameters to be set up, defining shape features of the GP, such as smoothness and amplitude. The values of these hyperparameters are usually unknown a priori and are set up depending on the observations $D_{1:n}$. Summarizing with γ the vector of the covariance's hyperparameters, their values are usually set up via marginal likelihood maximization, which is given, in the case of GP for regression, in closed form:

$$p\left(y|X_{1:n}, \gamma\right) = \int p\left(y|f, X_{1:n}\right) p\left(f|X_{1:n}\right) df.$$

The GP's hyperparameters γ appear nonlinearly in the kernel matrix K, and a closed-form solution to maximizing the marginal likelihood cannot be found in general. In practice, gradient-based optimization algorithms are adopted to find a (local) optimum of the marginal likelihood.

A covariance function is the crucial ingredient in a GP predictor, as it encodes assumptions about the function to approximate. From a slightly different viewpoint, it is clear that, in any learning process, both the supervised and the unsupervised, the notion of similarity between data points is crucial; it is a basic assumption that points that are close in x are likely to have similar target values y, and thus function evaluations that are near to a given point should be informative about the prediction at that point. Under the GP view, it is the covariance function that defines nearness or similarity.

Examples of covariance (a.k.a. kernel) functions are as follows:

Squared Exponential (SE) Kernel

$$k_{SE}(x, x') = e^{-\frac{\|x-x'\|^2}{2l^2}},$$

with l known as *characteristic length scale*. The role of this hyperparameter is to rescale any point x by $1/l$ before computing the kernel value. A large length scale implies long-range correlations, whereas a short length scale makes function values strongly correlated only if their respective inputs are very close to each other.

This kernel is infinitely differentiable, meaning that the GP is very "smooth". Note that the covariance between the outputs is written as a function of the inputs. For this particular covariance function, we see that the covariance is almost unity between variables whose corresponding inputs are very close and decreases as their distance in the input space increases.

Matérn Kernels

$$k_{Mat}(x, x') = \frac{2^{1-v}}{\Gamma(v)} \left(\frac{|x - x'|\sqrt{2v}}{l} \right)^{v} K_v \left(\frac{|x - x'|\sqrt{2v}}{l} \right),$$

with two hyperparameters v and l, and where K_v is a modified Bessel function. Note that for $v \to \infty$, we obtain the SE kernel. A GP with Matern kernel function as sample path that is $v - 1$ times differentiable.

The Matérn covariance functions become especially simple when v is half-integer: $v = p + 1/2$, where p is a non-negative integer. In this case, the covariance function is a product of an exponential and a polynomial of order p. The most widely adopted versions, specifically in the Machine Learning community, are $v = 3/2$ and $v = 5/2$.

$$k_{v=3/2}(x, x') = \left(1 + \frac{|x - x'|\sqrt{3}}{l} \right) e^{-\frac{|x-x'|\sqrt{3}}{l}}$$

$$k_{v=5/2}(x, x') = \left(1 + \frac{|x - x'|\sqrt{5}}{l} + \frac{(x - x')^2}{3l^2} \right) e^{-\frac{|x-x'|\sqrt{5}}{l}}.$$

Rational Quadratic Covariance Function

$$k_{RQ}(x, x') = \left(1 + \frac{(x - x')^2}{2\alpha l^2} \right)^{-\alpha},$$

where α and l are two hyperparameters. This kernel can be considered as an infinite sum (*scale mixture*) of SE kernels, with different characteristic length scales. Indeed, one of the most important properties of kernel functions is that a sum of kernels is a kernel. When $p = 0$, the Matern kernel becomes the Ornstein-Uhlenbeck, which encodes the assumption that the function is rough and that observations provide the information only about points that are very close to the previous observations.

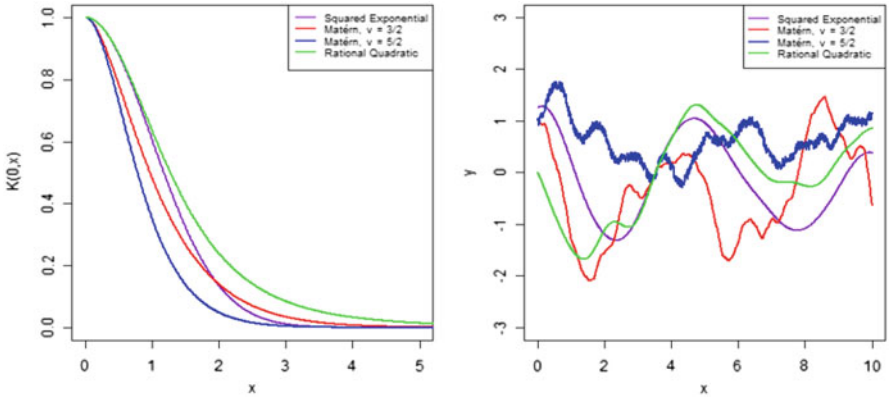


Fig. 3 Value of four different kernels with x moving away from $x' = 0$ (left) and four samples from GP prior, one for each kernel considered (right). The value of the characteristic length scale is $l = 1$ for all the four kernels; α of the RQ kernel is set to 2.25 [Source: [6]]

Figure 3 summarizes how the value of the four kernels decreases with x moving away from $x' = 0$ (on the left side) and which are possible resulting samples with different shape properties (on the right side).

3.2 GP-Based Optimization

The acquisition function is the mechanism to implement the trade-off between *exploration* and *exploitation* in BO. More precisely, any acquisition function aims to guide the search of the optimum towards points with potential low values of objective function either because the prediction of $f(x)$, based on the probabilistic surrogate model, is low or because the uncertainty, also based on the same model, is high (or both). Indeed, *exploiting* means to consider the area providing more chance to improve the current solution (with respect to the current surrogate model), while exploring means to move towards less explored regions of the search space where predictions based on the surrogate model are more uncertain, with higher variance.

This chapter is devoted to present some of the most relevant acquisition functions, from the "traditional" towards the most recent ones.

Probability of Improvement (PI) was the first acquisition function proposed in the literature [46]:

$$PI(x) = P\left(f(x) \leq f\left(x^+\right)\right) = \mathbf{\Phi}\left(\frac{f\left(x^+\right) - \mu(x)}{\sigma(x)}\right),$$

where $f(x^+)$ is the best value of the objective function observed so far, $\mu(x)$ and $\sigma(x)$ are mean and standard deviation of the probabilistic surrogate model, such as a GP, and $\mathbf{\Phi}(\cdot)$ is the normal cumulative distribution function.

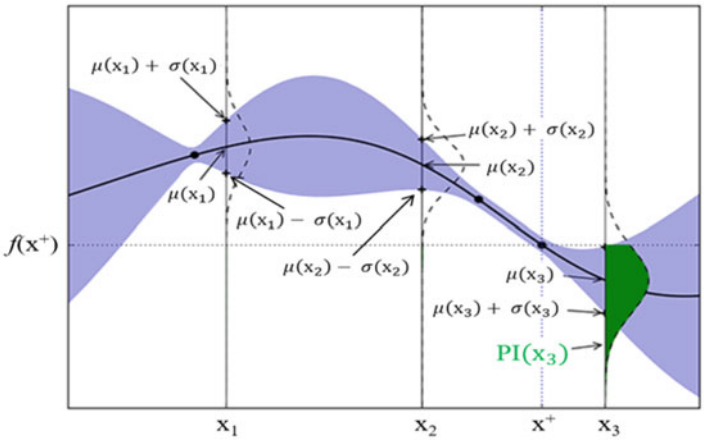


Fig. 4 A representation of probability of improvement [Source: [6]]

One of the drawbacks of PI is that it is biased towards exploitation. To mitigate this effect, one can introduce the parameter ξ that modulates the balance between exploration and exploitation. The resulting equation is

$$PI(x) = P\left(f(x) \leq f\left(x^+\right) + \xi\right) = \mathbf{\Phi}\left(\frac{f\left(x^+\right) - \mu(x) - \xi}{\sigma(x)}\right).$$

More precisely, $\xi = 0$ is towards exploitation, while $\xi > 0$ is more towards exploration (Figs. 4 and 5).

Finally, the next point to evaluate is chosen according to $x_{n+1} = \arg\max_{x \in X} PI(x)$.

However, a weak point of PI is to assign a value to a new point irrespectively of the potential magnitude of the improvement. This is the reason why the next acquisition function was proposed.

Expected Improvement (EI) was proposed initially in [53] and then made popular in [41], which measures the expectation of the improvement on $f(x)$ with respect to the predictive distribution of the probabilistic surrogate model.

$$EI(x) = \begin{cases} \left(f\left(x^+\right) - \mu(x)\right)\mathbf{\Phi}(Z) + \sigma(x)\phi(Z) \text{ if } \sigma(x) > 0 \\ 0 \text{ if } \sigma(x) = 0, \end{cases}$$

where $\phi(Z)$ and $\mathbf{\Phi}(Z)$ are the probability distribution and the cumulative distribution of the standardized normal, respectively, and

$$Z = \begin{cases} \frac{f\left(x^+\right) - \mu(x)}{\sigma(x)} \text{ if } \sigma(x) > 0 \\ 0 \text{ if } \sigma(x) = 0. \end{cases}$$

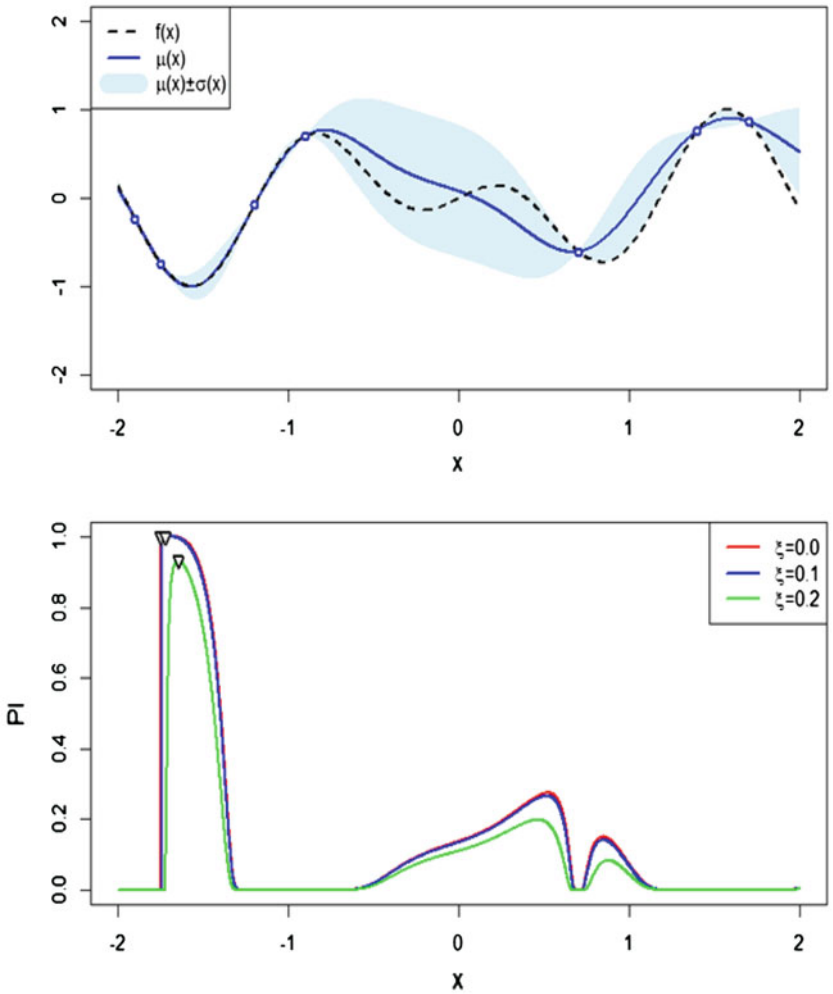


Fig. 5 GP trained depending on 7 observations (top), PI with respect to different values of ξ and max values corresponding to the next point to evaluate (bottom) [Source: [6]]

The EI is made up of 2 terms: the first is increased by decreasing the predictive mean and the second by increasing the predictive uncertainty. Thus, EI, in a sense, automatically balances, respectively, exploitation and exploration. When we want to actively manage the trade-off between exploration and exploitation, we can introduce the parameter ξ. When exploring, points associated with high uncertainty of the probabilistic surrogate model are more likely to be chosen, while when exploiting, points associated with low value of the mean of the probabilistic surrogate model are selected (Fig. 6).

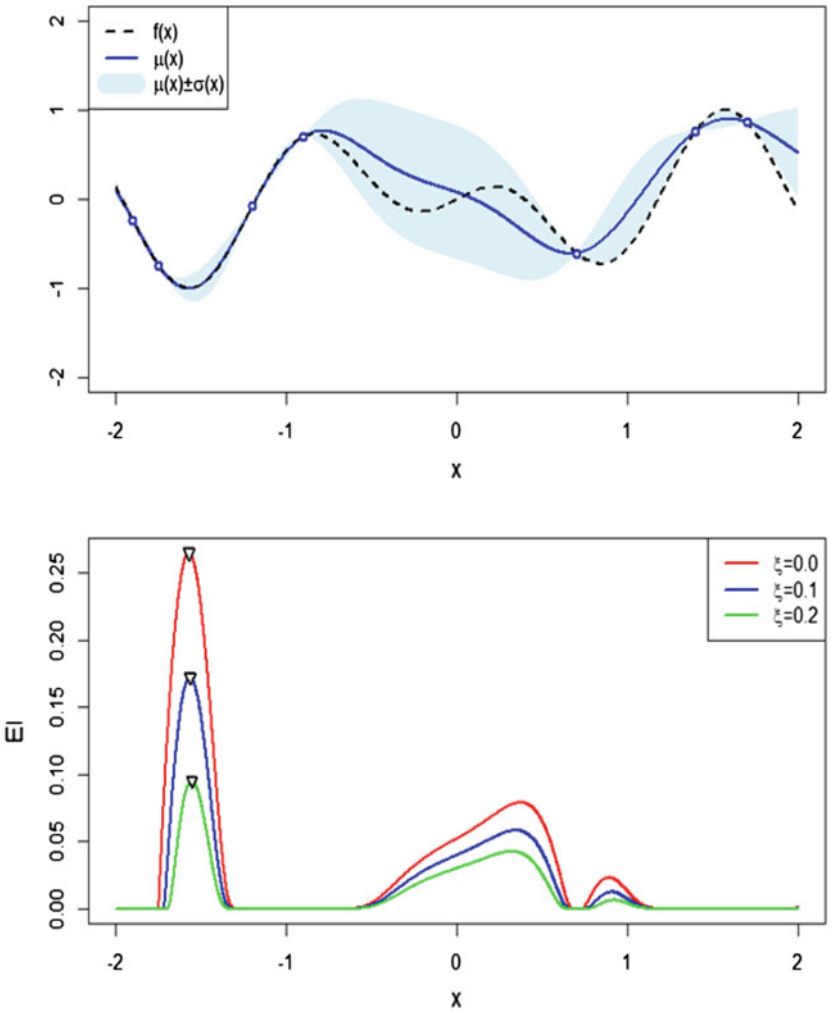


Fig. 6 GP trained depending on 7 observations (top), EI with respect to different values of ξ and max values corresponding to the next point to evaluate (bottom) [Source: [6]]

$$EI(x) = \begin{cases} \left(f\left(x^{+}\right) - \mu(x)\right) - \xi)\mathbf{\Phi}(Z) + \sigma(x)\phi(Z) \text{ if } \sigma(x) > 0 \\ 0 \text{ if } \sigma(x) = 0 \end{cases}$$

and

$$Z = \begin{cases} \frac{f\left(x^{+}\right) - \mu(x) - \xi}{\sigma(x)} \text{ if } \sigma(x) > 0 \\ 0 \text{ if } \sigma(x) = 0. \end{cases}$$

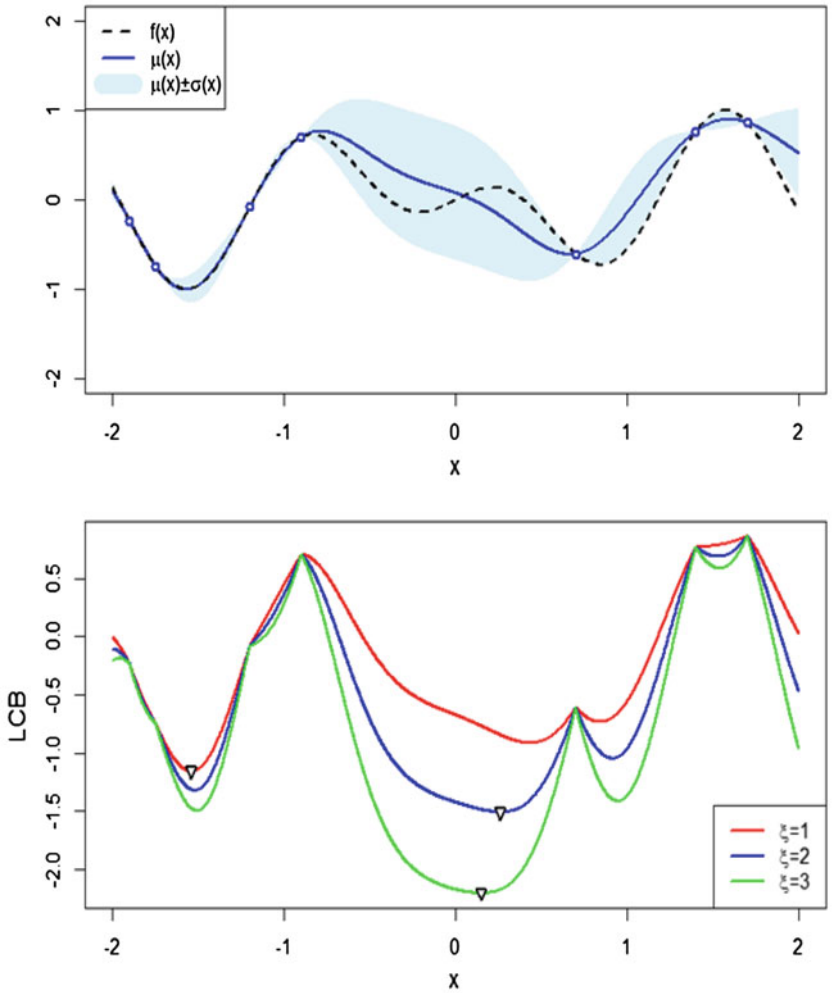


Fig. 7 GP trained depending on 7 observations (top), LCB with respect to different values of ξ and min values corresponding to the next point to evaluate (bottom). Contrary to the other acquisition function, LCB is minimized instead of maximized [Source: [6]]

Ideally, ξ should be adjusted dynamically to decrease monotonically with the function evaluations. Finally, the next point to evaluate is chosen according to

$$x_{n+1} = \arg\max_{x \in X} EI(x).$$

Confidence Bound–where upper and lower are used, respectively, for maximization and minimization problems–is an acquisition function that manage exploration–exploitation by being optimistic in the face of uncertainty, in the sense of considering the best-case scenario for a given probability value [7].

For the case of minimization, LCB is given by

$$\mathrm{LBC}(x) = \mu(x) - \xi\sigma(x),$$

where $\xi \geq 0$ is the parameter to manage the trade-off between exploration and exploitation ($\xi = 0$ is for pure exploitation; on the contrary, higher values of ξ emphasize exploration by inflating the model uncertainty). For the convergence of the sequence generated by this acquisition function, there are strong theoretical results, originated in the context of multi-armed bandit problems by Srinivas et al. [72].

Finally, the next point to evaluate is chosen according to $x_{n+1} = \arg\min_{x \in X} LBC(x)$, in the case of a minimization problem, or $x_{n+1} = \arg\max_{x \in X} UBC(x)$, in the case of a maximization problem (Fig. 7).

Most of the acquisition functions consider only the impact of the next function evaluation: in this sense, they are also called "myopic". This is a limitation addressed, e.g. in [27], where a 2-step-ahead solution is presented. Another look-ahead acquisition function has been proposed in [47], where the approximate dynamic programming is adopted to solve the problem of selecting the next candidate point to evaluate.

As shown in Fig. 7, L/UCB handles the exploration–exploitation dilemma in a structurally different way: high values of σ improve the exploration without degrading the acquisition function to a pure Random Search.

4 Constrained Bayesian Optimization

GO and BO have been considered first in "essentially unconstrained" conditions, where the solution was searched for within a bounded-box search space in $\mathbb{R}^n$.

There have been several attempts at leveraging BO framework into dealing with constrained optimization: the main problem is to propose an acquisition function for CBO.

The use of GP- and EI-based heuristics has been first proposed in [41] allowing for $c_i(x)$ to be black box and assuming their mutual independence and with the objective function. A GP is given as a prior to each constraint. If f_c^+ is the best feasible observation of f, the EI acquisition function is

$$EI\left(x|f_c^+\right) = \begin{cases} (\mu(x) - f_c^+)\boldsymbol{\Phi}(Z) + \sigma(x)\phi(Z) \text{ if } \sigma(x) > 0 \\ 0 \text{ if } \sigma(x) = 0, \end{cases}$$

where ϕ and $\boldsymbol{\Phi}$ are the probability distribution and the cumulative distribution functions, respectively, and

$$Z = \begin{cases} \frac{\mu(x) - f_c^+}{\sigma(x)} \text{ if } \sigma(x) > 0 \\ 0 \text{ if } \sigma(x) = 0. \end{cases}$$

In the presence of constraints, the formula becomes as in [34]:

$$EIC\left(x|f_c^+\right) = EI\left(x|f_c^+\right)\prod_{i=1}^{n_c}\mathbb{P}\left(c_i(x)\leq 0\right),$$

where the improvement of a candidate solution x over f is zero if x is not feasible.

When the noise in the constraints is taken into account, we may not know which observations are feasible: [49] proposed to use the best GP mean value satisfying each constraint $c_i(x)$ with a probability at least $1-\delta_i$. The works of [50] and [32] propose a different approach for handling constraints in which they are brought into the objective function via a Lagrangian. EI is no longer tractable analytically but can be evaluated numerically via Monte Carlo integration or quadrature. Relevant prior results on BO with unknown constraints proposed new acquisition functions, such as Integrated Expected Conditioned Improvement (IECI) [31]. A new general approach is offered by information-based methods that have been extended to the constrained case (e.g. Predictive Entropy Search with Constraints, PESC) in [34]: the code for PESC is included in Spearmint and available at https://github.com/HIPS/Spearmint/tree/PESC. Another approach is presented in [24], where an adaptive random search is used to approximate the feasible region in a multi-objective optimization problem. The above approaches assume that the number of constraints is known a priori, and they are statistically independent.

The assumption of independence permits to compute the probability of feasibility simply as the product of individual probability with respect to every constraint. The result is multiplied by the acquisition function whose optimization would prefer points satisfying the constraints with high probability.

A penalty approach has been considered first in [26], where a penalty is assigned directly to the acquisition function in case of infeasibility, with the aim to move away from infeasible regions. However, determining a suitable value for the penalty is not a trivial task and might imply, anyway, a loss of accuracy in the Gaussian surrogate model and sample efficiency. A penalty-based approach has been extended in [15], and also in the case, in which the function is partially defined: infeasibility is treated by assigning a fixed penalty as value of the objective function (we refer as "BO with penalty"). Other possible software tools are EGO-LS-SVM [67] and GPC with EI [10].

5 Constrained Bayesian Optimization for Partially Defined Objective Functions

In this section, the objective functions will be considered undefined in parts outside of the feasible region Ω, which is a subset of overall bounded-box search space $X \subset \mathbb{R}^d$.

The reference problem is CBBO with the important specification that the objective function is partially defined, which means that it cannot be computed outside the feasibility region. This issue arises in different context, in particular simulation optimization—in the case that the simulation software is not able to complete the simulation and provide a function evaluation—as well as in machine learning—in the case that training a specific configuration of a given algorithm results is some errors like "out-of-memory". Other terms for the same concept are known in the literature as "crash constraints", "non-computable domains" or "simulation failures". Bayesian Optimization in these contexts requires to be framed into a classification problem combined with a GP-based optimization.

In [9], a Probabilistic SVM (PSVM) is used to calculate the so-called probability of feasibility, and then the optimization scheme alternates between a global search for the optimal solution, depending on both this probability and the estimated value of the objective function—modelled through a GP—and a local refinement of the PSVM through an adaptive local sampling scheme. The most common PSVM model is based on the sigmoid function [8]. For a given sample x, the probability of belonging to the $+1$ class (i.e. "feasible") is

$$P(+1|x) = \frac{1}{1 + e^{As(x)+B}}.$$

The parameters $A(A < 0)$ and B of the sigmoid function are found by maximum likelihood. Two alternative formulations of the acquisition function are proposed:

$$\max_x EI(x)P(+1|x)$$

and

$$\max_x EI(x);$$

subject to

$$P(+1|x) \geq 0.5.$$

A closely related approach is in [67]. In this work, the feasible region is approximated by a Least-Square Support Vector Machine (LS-SVM) classifier. More in detail, the authors propose a binary classifier with two classes X^+ and X^- over the domain X, corresponding to the feasible and not-feasible domains, respectively. To predict the class of a new point x, they introduce a classification function h computed through LS-SVM. Finally, four possible acquisition functions are proposed for the computation of the new point x_{n+1} in which the Augmented Expected Improvement (AEI) is combined with the classification function h. During the BO optimization, if the new proposed point x_{n+1} results feasible $\left(x_{n+1} \in X^+\right)$, it is added to the training set and the GP is updated. Otherwise, the new point is simply considered unfeasible $\left(x_{n+1} \in X^-\right)$ for the GP model, but the training set for the classification is updated.

Another classification base is in [8]. In this work, the GP model is used to approximate both the objective function and the crash constraints, both considered as noise-free. The authors construct a binary classifier to approximate the crash constrain using a revised GP classification with a model based on the observation signs (positive if the new proposed point is feasible, and 0 otherwise).

The previous approaches use independent GPs to model the objective function and the constraints, requiring two strong assumptions: a priori knowledge about the number of constraints and the independence among objective function and all the constraints. Extending EI to account for correlations between constraints and the objective function is still, according to [50], an open challenge.

A new method to overcome the above limitations, namely SVM-CBO (Support Vector Machine-based Constrained BO), has been proposed in [14]. The approach uses Support Vector Machine (SVM) to sequentially estimate and model the unknown feasible region (Ω) within the search space (i.e. feasibility determination), without any assumption on the number of constraints as well as their independence.

SVM-CBO is organized in two phases: the first is aimed to provide a first estimate of Ω (feasibility determination), and the second is BO performed on such an estimate, only. The motivation is that we are interested in obtaining a good approximation of the overall feasible region, and not only close to the optimal solution (i.e. feasibility determination is a goal per se). Another relevant difference with [9] is that SVM-CBO uses more efficiently the available "budget" (i.e. the maximum number of function evaluations): at every iteration, of both phases 1 and 2, we perform just one function evaluation, while in the boundary refinement of [9], a given number n_p of function evaluations (namely, auxiliary samples) is performed in the neighbourhood of the next point to evaluate (namely, update region), with the aim to locally refine the boundary estimate. SVM-CBO is detailed in the following. We introduce some notation that will be used in the following:

- $D_n^{\Omega} = \{(x_i, y_i)\}_{i=1,\ldots,n}$ is the feasibility determination dataset;
- $D_l^f = \{(x_i, f(x_i))\}_{i=1,\ldots,l}$ is the function evaluation dataset, with $l \leq n$ (because $f(x)$ is partially defined on X) and where l is the number of points where it was possible to compute f out of the n queried so far,
  and x_i is the ith queried point and $y_i = \{+1, -1\}$ defines if x_i is feasible or infeasible, respectively.

Phase 1—Feasibility Determination

The first phase of the approach aims to find an estimate $\tilde{\Omega}$ of the actual feasible region Ω in M function evaluations ($\tilde{\Omega}_M = \tilde{\Omega}$). The sequence of function evaluations is determined according to an SMBO process where we defined a "feasibility acquisition function" to select the next promising point according to two different goals:

- improving the estimate of feasible region and
- discovering possible disconnected feasible regions.

To deal with the first goal, we use the distance from the boundaries of the currently estimated feasible region $\tilde{\Omega}_n$, using the following formula from the SVM classification theory:

$$d_n\left(h_n(x), x\right) = |h_n(x)| = \left| \sum_{i=1}^{n_{SV}} \alpha_i \, y_i \, k(\overline{x}_i, x) + b \right|,$$

where $h_n(x)$ is the argument of the SVM-based classification function:

$$h_n(x) = \sum_{i=1}^{n_{SV}} \alpha_i \, y_i \, k(\overline{x}_i, x) + b$$

with α_i and y_i are the Lagrangian coefficient and the "feasibility label" of the ith support vector, $\overline{x}_i$, respectively, $k(.,.)$ is the kernel function (i.e. an RBF kernel, in this study), b is the offset and n_{SV} is the number of support vectors.

The boundaries of the estimated feasible region $\tilde{\Omega}_n$ are given by $h_n(x) = 0$, which is the (nonlinear) separation hypersurface of the SVM classifier trained on the set D_n^{Ω}. The SVM-based classification function provides the estimated feasibility for any $x \in X$:

$$\tilde{y} = \text{sign}\left(h_n(x)\right) = \begin{cases} +1 \text{ if } x \in \tilde{\Omega}_n \\ -1 \text{ if } x \notin \tilde{\Omega}_n. \end{cases}$$

To deal with the second goal, we introduce the concept of "coverage of the search space", defined by

$$c_n(x) = \sum_{i=1}^{n} e^{-\frac{\|\overline{x}_i - x\|^2}{2\sigma_c^2}}.$$

So, $c_n(x)$ is a sum of n RBF functions centred on the points evaluated so far, with σ_c a parameter to set the width of the corresponding bell-shaped curve.

Finally, the feasibility acquisition function is given by the sum of $d_n(h_n(x), x)$ and $c_n(x)$, and the next promising points are identified by solving the following optimization problem:

$$x_{n+1} = \arg\min_{x \in X} \left\{ d_n\left(h_n(x), x\right) + c_n(x) \right\}.$$

Thus, we want to select the point associated with minimal coverage (i.e. max uncertainty)—exploration—and minimal distance from the boundaries of the current estimated feasible region—exploitation. This allows us to balance between improving the estimate of the feasible region and discovering possible disconnected feasible regions (in less explored areas of the search space). It is important to

highlight that, in phase 1, the optimization is performed on the overall bounded-box search space X, since in phase 1, we do not need the values of the objective function.

After the function evaluation of the new point x_{n+1}, the following information is available:

$$y_{n+1} = \begin{cases} +1 \text{ if } x_{n+1} \in \Omega\colon\ f(x_{n+1}) \text{ if defined} \\ -1 \text{ if } x_{n+1} \notin \Omega\colon\ f(x_{n+1}) \text{ if not defined,} \end{cases}$$

and the following updates are performed:

- feasibility determination dataset and estimated feasible region $\tilde{\Omega}_{n+1}$

$$D^{\Omega}_{n+1} = D^{\Omega}_{n} \cup \{(x_{n+1}, y_{n+1})\}$$
$$h_{n+1}(x) | D^{\Omega}_{n+1}$$
$$n \leftarrow n + 1,$$

- only if $x \in \Omega$, function evaluation dataset

$$D^{f}_{l+1} = D^{f}_{l} \cup \{(x_{l+1}, f(x_{l+1}))\}$$
$$l \leftarrow l + 1.$$

The SMBO process for phase 1 is repeated until $n = M$.

Phase 2—Bayesian Optimization in the Estimated Feasible Region
In this phase, a traditional BO process is performed, but with the following relevant differences:

- the search space is not box-bounded but the estimated feasible region $\tilde{\Omega}_n$ identified in phase 1,
- the surrogate model—a GP—is fitted only using the feasible solutions observed so far, D^{f}_{l}, and
- the acquisition function for phase 2—Lower Confidence Bound (LCB), in this study—is defined on $\tilde{\Omega}_n$, only.

Thus, the next point to evaluate is given by

$$x_{n+1} = \arg\min_{x \in \tilde{\Omega}_n} \{LCB_n(x) = \mu_n(x) - \beta_n \sigma_n(x)\},$$

where $\mu_n(x)$ and $\sigma_n(x)$ are the mean and the standard deviation of the current GP-based surrogate model and β_n is the inflate parameter to deal with the trade-off between exploration and exploitation for this phase. It is important to highlight that, contrary to phase 1, the acquisition function is here minimized on $\tilde{\Omega}$, only, instead of the entire bounded-box search domain X.

The point x_{n+1} is just expected to be feasible, according to $\tilde{\Omega}$, but the information on its actual feasibility is known only after having checked whether $f(x_{n+1})$ can or cannot be computed (i.e. it is defined or not in x_{n+1}). Subsequently, the feasibility determination dataset is updated as follows:

$$D_{n+1}^{\Omega} = D_n^{\Omega} \cup \{(x_{n+1}, y_{n+1})\}$$

and according to the two alternative cases:

- x_{n+1} is actually feasible: $x_{n+1} \in \Omega$, $y_{n+1} = +1$;
  the function evaluation dataset is updated as follows: $D_{l+1}^{f} = D_l^{f} \cup \{(x_{l+1}, f(x_{l+1}))\}$, with $l \leq n$ is the number of the feasible solutions with respect to all the points observed. The current estimated feasible region Ω_n can be considered accurate, and retraining of the SVM classifier can be avoided: $\tilde{\Omega}_{n+1} = \tilde{\Omega}_n$.
- x_{n+1} is actually infeasible: $x_{n+1} \notin \Omega$, $y_{n+1} = -1$;
  the estimated feasible region must be updated to reduce the risk for further infeasible evaluations

$$h_{n+1}(x) | D_{l+1}^{f} \Rightarrow \tilde{\Omega}_{n+1}.$$

The phase 2 continues until the overall available budget $n = N$ is reached.

In [14], the SVM-CBO approach has been validated on five 2D test functions for CGO and compared to "BO with penalty".

For each independent run, the initial set of solutions identified through LHS is the same for SVM-CBO and BO with penalty, in order to avoid differences in the values of the gap metric due to different initialization.

The so-called gap metric has been used to measure the improvement obtained along the SMBO process with respect to global optimum $f(x^*)$ and the initial best solution $f(x_0)$ obtained from the initialization step:

$$G_n = \frac{|f(x_0 - f(x^+)|}{|f(x_0 - f(x^*)|},$$

where $f(x^+)$ is the "best seen" up to iteration n. Gap metrics vary in the range of [0, 1]. For statistical significance, the gap metrics have been computed on 30 different runs, performed for every test function and for both SVM-CBO and BO with penalty.

The gap metric computed on the Branin constrained to two-ellipse test function with respect to the number of function evaluations excluding the LHS-based initialization is shown in Fig. 10, excluding the LHS-based initialization. The value of gap metric at iteration 0 is the best seen at the end of the initialization step (i.e. $f(x_0)$ in the gap metric formula). Each graph compares the gap metric provided by SVM-CBO and BO with penalty, respectively. Both the average and the standard deviation of the gap metric, computed on 30 independent runs for each approach,

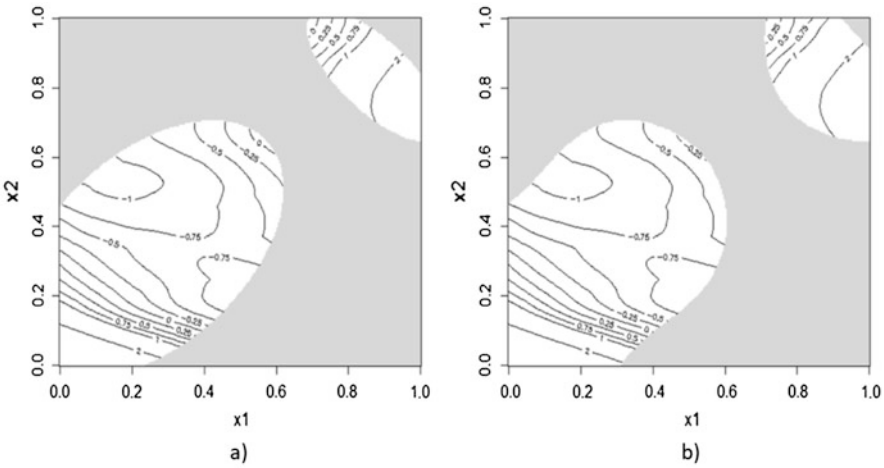


Fig. 8 Panels (**a**) and (**b**) both represent the BO-based surrogate function with LCB acquisition function. The constrained test function used is the Branin test function, and it is defined in $X = [0, 1]^2$. Also, two nonlinear constraints are defined by two ellipses on this test function by Candelieri and Archetti [14]. Panel (**a**) represents the surrogate function with real constraints that define the infeasible region (grey region). Instead, panel (**b**) represents the surrogate function with an estimation of constraints provided by SVM classifier inside of the SVM-CBO approach

are depicted. The higher effectiveness of the proposed approach is clear, even considering the variance in the performances. End of phase 1 is represented by a vertical dotted line in the charts; a significant improvement of the SVM-CBO's gap metric is observed after this phase, in every test case. It is important to remind that phase 1 of the SVM-CBO is aimed at approximating the unknown feasible region, while the optimization process only starts with phase 2. Thus, the relevant shift in the gap metric is motivated by the explicit model of the feasible region learned in phase 1. Figures 8 and 9 show a comparison on the constrained Branin function between Penalty BO and SVM-CBO regarding the optimization process performed.

6 Software for the Generation of Constrained Test Problems

In order to evaluate the performance of a constrained global optimization approach, several methods are adopted in CBBO. The first method to evaluate an optimization algorithm is to consider a specific set of constrained benchmark test functions available in the literature. An example of a set of constrained benchmark test functions is reported in [22], considering problems with different umber of dimensions and constraints.

The second validation framework is given by large-scale real-life black-box problems: the most famous example is MOTPA08 in [40], where the research space

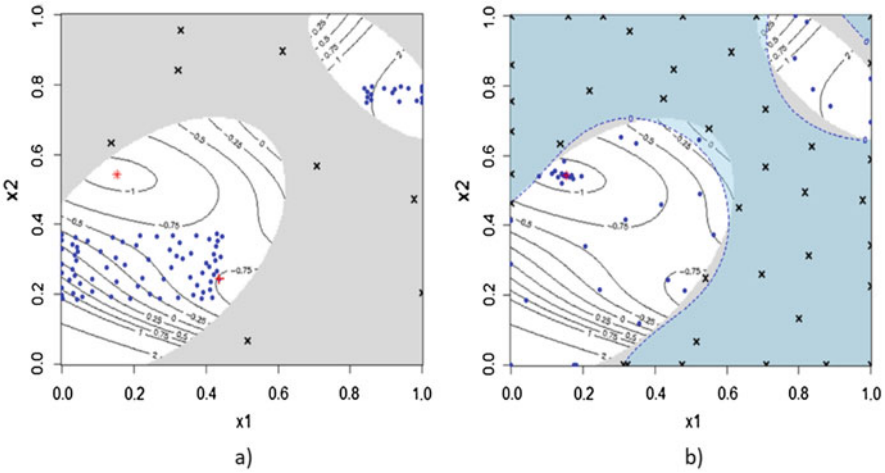


Fig. 9 Panels (**a**) and (**b**) represent the comparison between BO with penalty against SVM-CBO [14] on the test function of Fig. 8. In both panels, (**a**) and (**b**), the infeasible region (grey region) is defined by two ellipses, while the red star represents the feasible global minimizer, and the red plus represents the optimal minimum found by each approach. The black cross points represent non-computable points, and the blue points represent feasible function evaluations. In panel (**b**), it is also possible to see the estimated feasible region according to the SVM classifier. In panel (**b**), it is possible to notice that the optimal solution identified by SVM-CBO is the feasible global minimizer

is box-bounded in $X = [0, 1]^{124}$ (Material Variables, Shape Variables and others), and there are 68 inequality constraints, where $g_i(x) \leq 0 (1 <= i <= 68)$.

The third validation method can be the adoption of a test function generator for evaluation. The primary motivation of the choice of the function generator, as a method of validation, is driven by the possibility to extend the set of design parameters, such as the differentiability, the number of dimensions and local minimizers presented in generated functions, making for a flexible evaluation tool.

In the following section, it will be introduced the constrained test function generator Emmental-type GKLS [70].

6.1 Emmental-Type GKLS Generator

This generator can produce random global optimization constrained test problems with similar characteristics, nonlinear constraints, known minimizers and a parameterizable difficulty. This generator, based on a well-known GKLS generator [28], can generate three classes of test functions: non-differentiable, differentiable, and twice continuously differentiable.

$$X_{[a,b]} = \{x \in \mathbb{R}^d : a_i \leq x_i \leq b_i, 1 \leq i \leq d\}, a, b \in \mathbb{R}^d;$$

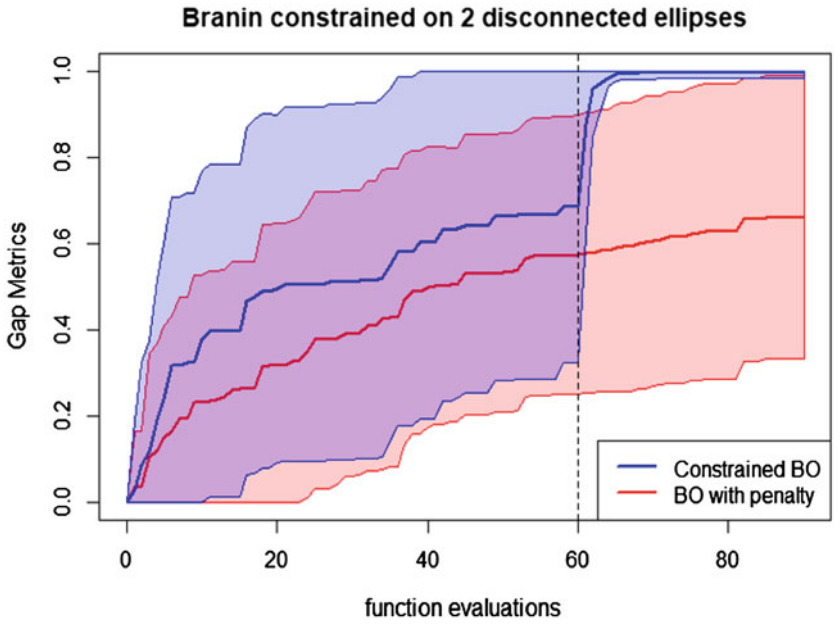


Fig. 10 The figure represents the gap metric values over 30 independent runs of SVM-CBO and BO with penalty on constrained Branin test function constrained to two ellipses. The main remark in this result comparison is the relevant shift, in terms of gap metric, that the SVM-CBO approach encountered when passed in its second phase, pointed by vertical black dashed line, that means the optimization phase

then, in order to make the test functions multiextremal, with m local minima, they are successively distorted through a set of polynomial transformations. The radius of attraction of every local minimizer can be controlled through the generator's parameter ρ. The feasible region is defined through a set of nonlinear constraints: basically "balls" with a radius that can be controlled by the generator's parameter r. The area inside each "ball" is considered infeasible, and four different types of constraints can be used. Moreover, it is also possible to set the number of active constraints on the constrained global optimizer. Figure 11 shows some examples of a test function generated through Emmental-type GKLS, with different constraint settings.

There is also another constrained function generator, namely "Global Constrained optimization problem Generator" (GCGen) [29], which is available online as Open Source at GitHub (https://github.com/UNN-ITMM-Software/GCGen). GCGen is a generator of constrained test functions, based on a modified GKLS function generator for generating several test functions. This function generator, in addition to the traditional GKLS input (e.g. the global minimizer value, the radius of attraction region of the global minimizer and others), can control the size of feasible domain with respect to the whole domain of parameters (dimensions of the problem) and also can set the global minimizer out of the feasible domain. The position of

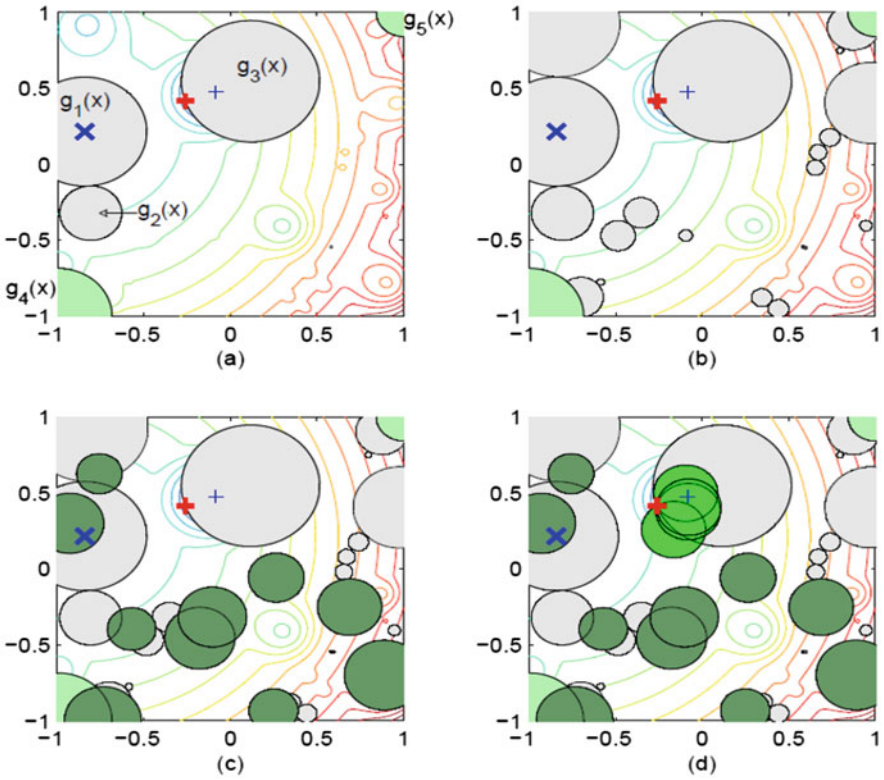


Fig. 11 Four examples of constrained objective functions generated through Emmental-type GKLS [70]. The objective function is the same, the set of constraints is different. The blue crosses are the global minimizers of the basis paraboloid functions, while the blue pluses are the global minimizers of the unconstrained problem. The red pluses are the global minimizers of the constrained problem. The infeasible regions are coloured according to the associated constraint type (grey for the first type, light green for the second, dark green for the third and bright green for the fourth one)

global minimizer is conditioned by some active constraints that are able to shift the global minimizer by the original position, produced by GKLS generator, to the boundary of the feasible domain. To shift the global minimizer, the computational cost increases linearly with increasing the dimensionality of the test function.

7 Conclusions

This chapter has analysed the growing relevance of constrained black-box optimization problems, where features like hidden constraints, uncertainty and the unavailability of a priori information, like linearity, convexity or derivative, make

classical optimization methods largely unsuitable. A dominant approach for CBBO is based on surrogate models, which are updated on the basis of new function evaluations at points suggested by the model itself. After a synthetic analysis of deterministic surrogates, this chapter focuses on probabilistic models that, at least in the authors' opinion, handle the process of learning from observations in a more natural way ensuring a better management of the exploration–exploitation dilemma and resulting in sample efficient algorithms. An emerging issue is when the function cannot be computed outside the feasible domain, a.k.a. "crush constraints" or "non-computable" domains.

A closely related problem is "safe" optimization where additional constraints are on the upper or lower bound of the function value whose violation is physically impossible or unsafe [73].

The above problems will become increasingly relevant in optimization landscape arising, for instance, from Automated Machine Learning, in particular neural architecture search for deep networks, where the training process could fail to converge for a given hyperparameter configuration, and reinforcement learning where some policy roll-outs result in failure behaviours causing premature termination. In these as well as many other cases, penalty-based approaches do not offer an effective solution and recently proposed methods combining classification to learn the feasible region and optimization taking place in the interior of the approximate feasible region offer better solution strategies. The main focus of this chapter is to analyse their modelling and computational issues and demonstrate the sample efficiency of the resulting algorithms.

References

1. Adam, S.P., Alexandropoulus S.A.N., Pardalos, P., Vrahatis, M.: No free lunch theorem: A review. In: Approximation and Optimization, pp. 57–82. Springer, Berlin (2019)
2. Akimoto, Y., Auger, A., Hansen, N.: CMA-ES and advanced adaptation mechanisms. In: Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion, pp. 533–562. ACM, New York (2016)
3. Alexandropoulos, S.A.N., Aridas, C.K., Kotsiantis, S.B., Vrahatis, M.N.: Multi-objective evolutionary optimization algorithms for machine learning: A recent survey. In: Approximation and Optimization, pp. 35–55. Springer, Cham (2019)
4. Amaran, S., Sahinidis, N.V., Sharda, B., Bury, S.J.: Simulation optimization: a review of algorithms and applications. Ann. Operat. Res. **240**(1), 351–380 (2016)
5. Archetti, F., Betro, B.: A probabilistic algorithm for global optimization. Calcolo **16**(3), 335–343 (1979)
6. Archetti, F., Candelieri, A.: Bayesian Optimization and Data Science. SpringerBriefs in Optimization. Springer International Publishing, New York (2019)
7. Auer, P. (2002). Using confidence bounds for exploitation-exploration trade-offs. Journal of Machine Learning Research, 3(Nov), 397–422
8. Bachoc, F., Helbert, C., Picheny, V.: Gaussian process optimization with failures: classification and convergence proof. J. Global Optim. **78**, 483–506 (2019). hal.archives-ouvertes.fr
9. Basudhar, A., Dribusch, C., Lacaze, S., Missoum, S.: Constrained efficient global optimization with support vector machines. Struct. Multidiscipl. Optim. **46**(2), 201–221 (2012)

10. Bernardo, J., Bayarri, M., Berger, J., Dawid, A., Heckerman, D., Smith, A., West, M.: Optimization under unknown constraints. Bayesian Stat. **9**(9), 229 (2011)
11. Bhosekar, A., Ierapetritou, M.: Advances in surrogate based modeling, feasibility analysis, and optimization: a review. Comput. Chem. Eng. **108**, 250–267 (2018)
12. Bouhlel, A.M., Bartoli, N., Regis, R.G., Otsmane, A., Morlier, J.: Efficient global optimization for high-dimensional constrained problems by using the Kriging models combined with the partial least squares method. Eng. Optim. **50**(12), 2038–2053 (2018)
13. Box G. E. P.; Draper, N. R. (2007), Response Surfaces, Mixtures, and Ridge Analyses, John Wiley & Sons. pg. 414
14. Candelieri, A., Archetti, F.: Sequential model-based optimization with black-box constraints: Feasibility determination via machine learning. In: AIP Conference Proceedings, p. 020010 (2019)
15. Candelieri, A., Perego, R., Archetti, F.: Bayesian optimization of pump operations in water distribution systems. J. Global Optim. **71**, 1–23 (2018)
16. Cao, Y., Shen, Y.: Bayesian active learning for optimization and uncertainty quantification in protein docking (2019). Preprint arXiv:1902.00067
17. Chen, Y., Hoffman, M. W., Colmenarejo, S. G., Denil, M., Lillicrap, T.P., de Freitas, N.: Learning to learn for global optimization of black box functions (2016). Preprint arXiv:1611.03824
18. Costabal, F.S., Perdikaris, P., Kuhl, E., Hurtado, D.E.: Multi-fidelity classification using Gaussian processes: accelerating the prediction of large-scale computational models (2019). Preprint arXiv:1905.03406
19. Costabal, F.S., Yao, J., Sher, A., Kuhl, E.: Predicting critical drug concentrations and torsadogenic risk using a multiscale exposure-response simulator. Progress Biophys. Molecular Biolo. **144**, 61–76 (2019)
20. Cozad, A., Sahinidis, N.V., Miller, D.C.: Learning surrogate models for simulation-Řbased optimization. AIChE J. **60**(6), 2211–2227 (2014)
21. Digabel, S.L., Wild, S.M.: A taxonomy of constraints in simulation-based optimization (2015). Preprint arXiv:1505.07881
22. Dong, H., Song, B., Dong, Z., Wang, P.: SCGOSR: surrogate-based constrained global optimization using space reduction. Appl. Soft Comput. **65**, 462–477 (2018)
23. Eggensperger, K., Lindauer, M., Hutter, F.: Pitfalls and best practices in algorithm configuration. J. Artif. Intell. Res. **64**, 861–893 (2019)
24. Feliot, P., Bect, J., Vazquez, E.: A Bayesian approach to constrained single-and multi-objective optimization. J. Global Optim. **67**(1–2), 97–133 (2017)
25. Frazier, P.I., Powell, W.B., Dayanik, S.: A knowledge-gradient policy for sequential information collection. SIAM J. Control Optim. **47**(5), 2410–2439 (2008)
26. Gardner, J.R., Kusner, M.J., Xu, Z.E., Weinberger, K.Q., Cunningham, J.P.: Bayesian optimization with inequality constraints. In: International Conference on Machine Learning, pp. 937–945 (2014)
27. Garnett, R., Osborne, M.A., Roberts, S.J.: Sequential Bayesian prediction in the presence of changepoints. In: Proceedings of the 26th Annual International Conference on Machine Learning, pp. 345–352. ACM, New York (2009)
28. Gaviano, M., Kvasov, D.E., Lera, D., Sergeyev, Ya.D.: Algorithm 829: Software for generation of classes of test functions with known local and global minima for global optimization. ACM Trans. Math. Softw. **29**(4), 469–480 (2003)
29. Gergel, V., Barkalov, K., Lebedev, I., Rachinskaya, M., Sysoyev, A.: A flexible generator of constrained global optimization test problems. In: AIP Conference Proceedings, vol. 2070, no. 1, p. 020009. AIP Publishing, College Park (2019)
30. Ghoreishi, S.F., Allaire, D.: Multi-information source constrained Bayesian optimization. Struct. Multidiscip. Optim. **59**, 1–15 (2018)
31. Gramacy R.B., Lee, H.K.: Optimization under unknown constraints. Bayesian Stat. **9**(9), 229 (2011)

32. Gramacy, R.B., Gray, G.A., Le Digabel, S., Lee, H.K., Ranjan, P., Wells, G., Wild, S.M.: Modeling an augmented Lagrangian for blackbox constrained optimization. Technometrics **58**(1), 1–11 (2016)
33. Grishagin, V., Israfilov, R.: Multidimensional constrained global optimization in domains with computable boundaries. In: CEUR Workshop Proceedings. Vol. 1513: Proceedings of the 1st Ural Workshop on Parallel, Distributed, and Cloud Computing for Young Scientists (Ural-PDC 2015).—Yekaterinburg, 2015 (2015)
34. Hernández-Lobato, J.M., Gelbart, M.A., Hoffman, M.W., Adams, R.P., Ghahramani, Z.: Predictive entropy search for Bayesian optimization with unknown constraints. In: 32nd International Conference on Machine Learning, ICML 2015, pp. 1699–1707. International Machine Learning Society (IMLS) (2015)
35. Hu, W., Fathi, M., Pardalos, P.M.: A multi-objective evolutionary algorithm based on decomposition and constraint programming for the multi-objective team orienteering problem with time windows. Appl. Soft Comput. **73**, 383–393 (2018)
36. Huyer, W., Neumaier, A.: SNOBFIT-stable noisy optimization by branch and fit. ACM Trans. Math. Softw. **35**(2), 200 (2006)
37. Ilievski, I., Akhtar, T., Feng, J., Shoemaker, C.A.: Efficient hyperparameter optimization for deep learning algorithms using deterministic RBF surrogates. In: Thirty-First AAAI Conference on Artificial Intelligence (2017)
38. Jain, P., Kar, P.: Non-convex optimization for machine learning. Found. Trends Mach. Learn. **10**(3–4), 142–336 (2017)
39. Jones, D.R.: A taxonomy of global optimization methods based on response surfaces. J. Global Optim. **21**(4), 345–383 (2001)
40. Jones, D.R.: Large-scale multi-disciplinary mass optimization in the auto industry. In: MOPTA 2008 Conference (2008)
41. Jones, D.R., Schonlau, M., Welch, W.J.: Efficient global optimization of expensive black-box functions. J. Global Optim. **13**(4), 455–492 (1998)
42. Kandasamy, K., Dasarathy, G., Schneider, J., Poczos, B.: Multi-fidelity Bayesian optimisation with continuous approximations. In Proceedings of the 34th International Conference on Machine Learning, vol. 70, pp. 1799–1808 (2017). https://JMLR.org
43. Kleijnen J.P.C.: Kriging: Methods and Applications. CentER Discussion Paper Series No. 2017-047 (2017)
44. Klein, A., Falkner, S., Springenberg, J.T., Hutter, F.: Learning curve prediction with Bayesian neural networks. In: Published as a Conference Paper at ICLR 2017 (2016)
45. Koch, P., Bagheri, S., Konen, W., Foussette, C., Krause, P., Bäck, T.: A new repair method for constrained optimization. In: Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation, pp. 273–280. ACM, New York (2015)
46. Kushner, H.J.: A new method of locating the maximum point of an arbitrary multipeak curve in the presence of noise. J. Basic Eng. **86**(1), 97–106 (1964)
47. Lam, R., Willcox, K., Wolpert, D.H.: Bayesian optimization with a finite budget: An approximate dynamic programming approach. In: Advances in Neural Information Processing Systems, pp. 883–891 (2016)
48. Larson, J., Menickelly, M., Wild, S.M.: Derivative-free optimization methods (2019). Preprint arXiv:1904.11585
49. Letham, B., Karrer, B., Ottoni, G., Bakshy, E. (2017). Constrained Bayesian optimization with noisy experiments. arXiv preprint arXiv:1706.07094.
50. Letham, B., Karrer, B., Ottoni, G., Bakshy, E.: Constrained Bayesian optimization with noisy experiments. Bayesian Anal. **14**(2), 495–519 (2019)
51. Martì, R., Pardalos, P.M., Resende, M.G. (Eds.): Handbook of Heuristics. Springer, Berlin (2018)
52. Mehdad, E., Kleijnen, J.P.: Efficient global optimisation for black-box simulation via sequential intrinsic Kriging. J. Oper. Res. Soc. **69**(11), 1725–1737 (2018)

53. Mockus, J., Tiesis, V., Zilinskas, A.: The application of Bayesian methods for seeking the extremum. Towards global optimization, 2 (117–129), 2, Dixon, L.C.W., Szego, G.P. (eds.) (1978)
54. Moreno, J.D., Zhu, Z.I., Yang, P.C., Bankston, J.R., Jeng, M.T., Kang, C., Wang, L., Bayer, J.D., Christini, D.J., Trayanova, N.A., Ripplinger, C.M., Kass, R.S., Clancy, C.E.: A computational model to predict the effects of class I anti-arrhythmic drugs on ventricular rhythms. Sci. Transl. Med. **3**(98), 98ra83 (2011)
55. Nuñez, L., Regis, R.G., Varela, K.: Accelerated random search for constrained global optimization assisted by radial basis function surrogates. J. Comput. Appl. Math. **340**, 276–295 (2018)
56. Ortega, P.A., Wang, J.X., Rowland, M., Genewein, T., Kurth-Nelson, Z., Pascanu, R., et al.: Meta-learning of Sequential Strategies (2019). Preprint arXiv:1905.03030
57. Parsopoulus K.E., Vrahatis, M.N.: Particle Swarm Optimization and Intelligence: Advances and Applications (2010). https://doi.org/10.4018/978-1-61520-666-7
58. Peherstorfer, B., Willcox, K., Gunzburger, M.: Survey of multifidelity methods in uncertainty propagation, inference, and optimization. SIAM Rev. **60**(3), 550–591 (2018)
59. Perdikaris, P., Venturi, D., Karniadakis, G.E.: Multifidelity information fusion algorithms for high-dimensional systems and massive data sets. SIAM J. Sci. Comput. **38**(4), B521–B538 (2016)
60. Powell, W.B.: A unified framework for stochastic optimization. Euro. J. Oper. Res. **275**(3), 795–821 (2019)
61. Regis, R.G.: A Survey of Surrogate Approaches for Expensive Constrained Black-Box Optimization. In: Le Thi, H., Le, H., Pham Dinh, T. (eds.) Optimization of Complex Systems: Theory, Models, Algorithms and Applications, pp. 37–47. WCGO 2019. Springer, Cham (2020)
62. Regis, R.G., Shoemaker, C.A.: Constrained global optimization of expensive black box functions using radial basis functions, J. Global Optim. **31**, 153–171 (2005)
63. Regis, R.G., Shoemaker, C.A.: A stochastic radial basis function method for the global optimization of expensive functions. INFORMS J. Comput. **19**(4), 497–509 (2007)
64. Regis, R.G., Shoemaker, C.A.: Parallel radial basis function methods for the global optimization of expensive functions. Eur. J. Oper. Res. **182**(2), 514–535 (2007)
65. Regis, R.G., Shoemaker, C.A.: Combining radial basis function surrogates and dynamic coordinate search in high-dimensional expensive black-box optimization. Eng. Optim. **45**(5), 529–555 (2013)
66. Rudenko, L.I.: Objective functional approximation in a partially defined optimization problem. J. Math. Sci. **72**(5), 3359–3363 (1994)
67. Sacher, M., Duvigneau, R., Le Maitre, O., Durand, M., Berrini, E., Hauville, F., Astolfi, J.A.: A classification approach to efficient global optimization in presence of non-computable domains. Struct. Multidiscip. Optim. **58**(4), 1537–1557 (2018)
68. Sen, S., Deng, Y.: Learning enabled optimization: Towards a fusion of statistical learning and stochastic programming. INFORMS Journal on Optimization (2018)
69. Sergeyev, Y.D., Kvasov, D.E., Khalaf, F.M.: A one-dimensional local tuning algorithm for solving GO problems with partially defined constraints. Optim. Lett. **1**(1), 85–99 (2007)
70. Sergeyev, Y.D., Kvasov, D.E., Mukhametzhanov, M.S.: Emmental-type GKLS-based multiextremal smooth test problems with non-linear constraints. In: International Conference on Learning and Intelligent Optimization, pp. 383–388. Springer, Cham (2017)
71. Sra, S., Nowozin, S., Wright, S.J. (Eds.): Optimization for Machine Learning. Mit Press, Cambridge (2012)
72. Srinivas, N., Krause, A., Kakade, S. M., & Seeger, M. W. (2012). Information-theoretic regret bounds for gaussian process optimization in the bandit setting. IEEE Transactions on Information Theory, **58**(5), 3250–3265
73. Sui, Y., Gotovos, A., Burdick, J., & Krause, A. (2015, June). Safe exploration for optimization with Gaussian processes. In International Conference on Machine Learning (pp. 997–1005). PMLR

74. Tsai, Y.A., Pedrielli, G., Mathesen, L., Zabinsky, Z.B., Huang, H., Candelieri, A., Perego, R.: Stochastic optimization for feasibility determination: An application to water pump operation in water distribution network. In: Proceedings of the 2018 Winter Simulation Conference, pp. 1945–1956. IEEE Press, New York (2018)
75. Volpp, M., Fr´ohlich, L., Doerr, A., Hutter, F., Daniel, C.: Meta-Learning Acquisition Functions for Bayesian Optimization (2019). Preprint arXiv:1904.02642
76. Wang, Y., Shoemaker, C.A.: A General Stochastic Algorithmic Framework for Minimizing Expensive Black Box Objective Functions Based on Surrogate Models and Sensitivity Analysis (2014). Preprint arXiv:1410.6271
77. Williams, C.K., Rasmussen, C.E.: Gaussian Processes for Machine Learning, vol. 2, No. 3, p. 4. MIT Press, Cambridge (2006)
78. Wilson, Z.T., Sahinidis, N.V.: The ALAMO approach to machine learning. Comput. Chem. Eng. **106**, 785–795 (2017)
79. Wilson, Z.T., Sahinidis, N.V.: The ALAMO approach to machine learning. Comput. Chem. Eng. **106**, 785–795 (2017)
80. Zabinsky, Z.B.: Stochastic Adaptive Search for Global Optimization, vol. 72. Springer Science & Business Media, Berlin (2013)
81. Zhang, Z., Buisson, M., Ferrand, P., Henner, M.: Databases coupling for morphed-mesh simulations and application on fan optimal design. In: World Congress on Global Optimization, pp. 981–990. Springer, Cham (2019)
82. Žilinskas, A., Zhigljavsky, A.: Stochastic global optimization: a review on the occasion of 25 years of Informatica. Informatica **27**(2), 229–256 (2016)

Black-Box Optimization: Methods and Applications



Ishan Bajaj, Akhil Arora, and M. M. Faruque Hasan

1 Introduction

Black-box optimization (BBO) refers to a class of problems, where the objective and constraint function values are available as outputs of a computer simulation, legacy codes, or physical experimentation. The BBO problems considered in this chapter can be represented as follows:

$$\begin{aligned} \min_{x} \quad & f(x, y) \\ s.t. \quad & g_i(x, y) \leq 0 \quad \forall i \in \mathbb{P} := \{1, \ldots, p\} \\ & y = d(x) \\ & x \in [x^L, x^U], \end{aligned} \tag{1}$$

where $x \in \mathbb{R}^n$ are the independent variables, $y \in \mathbb{R}^q$ are the dependent variables, $f(x, y)$ and $g(x, y)$ denote the objective function and the constraints, respectively, and $d(x)$ represents a complex model whose analytical form is unknown. An example of a BBO problem is illustrated in Fig. 1.

I. Bajaj
Department of Chemical and Biological Engineering, University of Wisconsin-Madison, Madison, WI, USA
e-mail: ibajaj@wisc.edu

A. Arora · M. M. F. Hasan (✉)
Artie McFerrin Department of Chemical Engineering, Texas A&M University, College Station, TX, USA
e-mail: akhilarora619@tamu.edu; hasan@tamu.edu

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_2

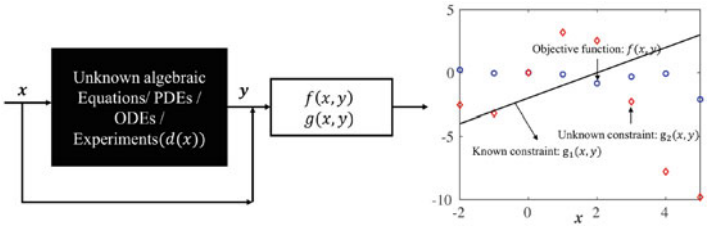


Fig. 1 Illustration of a BBO problem. The values of y are obtained by fixing x, which are then used to compute black-box objective function (blue circles) and the unknown constraints (red diamonds). The known constraints (black solid line) are functions of only decision variables

In the literature, BBO is also referred as derivative-free optimization (DFO), gradient-free optimization, simulation-based optimization, and zeroth-order optimization. However, BBO and DFO are the most widely used terms. Audet and Hare [14] listed some differences between the two terms based on their experience. They defined DFO as the "mathematical study of optimization algorithms that do not use derivatives," while BBO is defined as the "study of design and analysis of algorithms that assume the objective and/or constraint functions are given by blackboxes." Thus, DFO focuses on methods that can be proven to converge, while BBO is a larger field that includes heuristic methods as well. However, these distinctions have not yet been widely adopted, and the two terms are used interchangeably. We will be using the term BBO throughout the chapter.

The common characteristics of BBO problems are:

- the analytical forms of some or all of the functions are explicitly unavailable,
- the function values are obtained by solving computationally expensive model,
- some or all of the functions can be non-differentiable and non-convex, and
- the presence of hidden constraints and their violation lead to simulation failure.

Due to aforementioned characteristics, classical gradient-based approaches that rely on either automatic differentiation or finite difference to estimate the derivatives are inapplicable to BBO problems. Automatic differentiation fails when the analytical functional forms are unknown. Applying finite difference to an expensive computer simulation can be prohibitive since it requires at least $n + 1$ evaluations, (n represents the problem dimension) to approximate the gradient at a point. To this end, BBO algorithms strive to find a solution in a computationally efficient manner by using only function values.

Despite the challenges, BBO methods are increasingly being applied to solve real-world problems in a variety of fields, including engineering, medicine, physics, operations research, and machine learning. However, it has been observed that BBO methods are sometimes applied due to their ease of use rather than necessity. In general, gradient-based methods provide better performance in terms of accuracy, speed, and problem size. Thus, it is important to carefully examine a problem before deciding to use a BBO method.

The remaining chapter is organized as follows. An overview of several BBO algorithms is provided in Sect. 2. An indicative list of available BBO solvers is provided in Sect. 3. Section 4 presents few application areas where BBO methods have been applied in recent times. We also briefly discuss several remaining challenges in the area in Sect. 5.

2 Overview of BBO Methods

In this section, we provide an overview of different BBO methods for solving unconstrained black-box problems. The methods are divided into four categories—direct search, model based, heuristic, and hybrid. Direct search methods work by examining a collection of points to determine a candidate point, whereas model-based methods attempt to approximate the original functions by surrogate models to guide future iterations. Both direct search and model-based methods have proven convergence results. Heuristic methods are designed to find an approximate solution quickly and do not guarantee finding the exact solution. Hybrid methods combine different strategies to improve computational time or solution quality. The convergence properties of hybrid methods depend on the underlying methods used.

2.1 Direct Search Methods

In this section, we present three direct search algorithms designed for black-box problems. We begin our discussion with simplex search methods in Sect. 2.1.1. In Sect. 2.1.2, the coordinate search is presented. Generalized pattern search (GPS) and mesh adaptive direct search (MADS) are described in Sects. 2.1.3 and 2.1.4, respectively.

2.1.1 Simplex Search

The simplex search method was originally proposed by John Nelder and Roger Mead in 1965 [158], and it has become an enormously popular direct search method for multidimensional unconstrained minimization. The algorithm begins by choosing $n + 1$ affinely independent points to construct a simplex. Function evaluation is performed at each of the simplex vertices and ranked based on the function values. A candidate point is obtained by reflecting the point corresponding to the largest function value through the hyperplane defined by the remaining n points. Based on the function value obtained at the reflected point, the simplex at the next iteration is obtained by performing reflection, expansion, and contraction operations to the simplex at the current iteration. The algorithm stops when the

standard error of the function values at the simplex vertices is below a pre-specified tolerance.

Despite its widespread use and fairly good performance on practical problems, the algorithm does not provide any guarantees of convergence to a critical point. Using a non-convex function with two variables, Woods [201] showed that the algorithm converges to a non-stationary point. McKinnon [148] further showed that the algorithm fails to converge to a stationary point for a certain class of convex problems. However, minor modifications to the algorithm have been proposed that improves on the convergence properties [34, 124, 192].

2.1.2 Coordinate Search

The CS algorithm starts from an initial point and searches for a point in the neighborhood to reduce the objective function value. Thus, the CS algorithm focuses on finding a local minimum. At iteration k, a finite set of points is generated near the current point x_k by adding terms of the form $\Delta_k d$, where $d \in D_k$. D_k are called the poll directions, and for the CS algorithm, $D_k = \{\pm e_i : i = 1, 2, \ldots, n\}$, where e_i is the coordinate vector. If an improvement is obtained at any of the points, the iteration is deemed successful, the current point is updated, and Δ_k remains the same. Otherwise, the iteration is unsuccessful, the current point remains the same, and Δ_k is reduced. The algorithm stops when Δ_k is less than the user-defined tolerance. The algorithm is proven to converge to a critical point if the original function is continuously differentiable and has bounded level sets [14].

Several variants of the CS algorithm have been developed to potentially improve its performance. One change that could be done is that instead of evaluating objective function at all the $2n$ points (defined as complete polling), stop evaluation once an improved objective function value (defined as opportunistic polling) is obtained relative to the current point. The order in which function evaluations are performed while using opportunistic polling also affects the performance of the algorithm.

2.1.3 Generalized Pattern Search

The GPS algorithm operates similarly to the CS algorithm, but it provides more flexibility in choosing the polling directions D and also includes a search step that allows to possibly escape a local minimum [29]. Thus, the goal of poll step is to explore locally, and the search step aims to allow global exploration. Instead of fixing the poll directions to coordinate vectors for all the iterations as done in the coordinate search, they can change with iterations in GPS. The only requirement on the polling direction is that it should form a positive spanning set. A set D is said to form a positive spanning set for the domain Ω if

$$x = \sum_{i=1}^{|D|} \lambda_i d_i,$$

where $x \in \Omega$, $d_i \in D$, and $\lambda_i \geq 0$. To perform the search step, the notion of a mesh is defined, which can be interpreted as enumerable discretization of variable space. The mesh M_k of coarseness Δ_k generated by D and centered at the current solution x_k is defined as follows:

$$M_k := \{x_k + \Delta_k Dy \quad : \quad y \in N^p\}.$$

At iteration k of the GPS algorithm, the search step is applied, which is based on selecting a subset of candidate points of the mesh M_k to perform function evaluations. If an improved objective function is obtained during the search step, Δ_k is increased, and x_{k+1} is updated to the point corresponding to the least function value. Otherwise, go to the polling step. If an improved objective function value is obtained, increase Δ_k; otherwise, it is decreased, and x_{k+1} is set to x_k.

Similar to the CS algorithm, GPS also allows to incorporate opportunistic strategies in search and poll steps. Particularly, the search step allows opportunities to escape local minima. Thus, global heuristic methods such as genetic algorithms, tabu search, VNS, and simulated annealing can be applied in the search step, which enforces the search points to lie on the mesh. Due to the flexibility of the algorithm, various extensions have been proposed in the literature (e.g., [2, 4, 50]).

Several authors have also studied the convergence of the algorithm based on different assumptions [10, 54, 190]. In general, the algorithm will converge to a first-order critical point if the function is continuously differentiable, and it may converge to non-stationary point if the problem is nonsmooth.

2.1.4 Mesh Adaptive Direct Search

The MADS algorithm [12] is an extension of the GPS algorithm and generalizes it by allowing an infinite set of polling directions. Instead of using the same parameter for mesh size and polling, a new parameter called the frame size (Δ_k^f) is defined. The frame of size Δ_k^f centered at x_k is defined by

$$F_k := \{x \in M_k \quad : \quad ||x - x_k||_\infty \leq \Delta_k^f\}.$$

The MADS algorithm works similarly to the GPS algorithm with an exception to how the poll step is generated. In the MADS algorithm, any poll step that lies with the frame is acceptable. At every iteration, the two parameters controlling the size of mesh and frame need to satisfy $0 < \Delta_k \leq \Delta_k^f$. Recent variants of the MADS algorithm have focused on making it more efficient for noisy functions [16],

reducing the number of evaluations [15], and extending the convergence proofs to discontinuous functions [196].

2.2 Model-Based Methods

BBO algorithms are developed based on the assumption that the derivatives are unavailable. However, in many cases, the black-box function is smooth in nature, and it may be advantageous to approximate derivatives in order to accelerate convergence. To this end, model-based methods are based on approximating a black-box function by a surrogate model and using its derivatives to guide future iterations.

2.2.1 Model-Based Trust Region

In a trust-region method [46], a model is typically constructed in the neighborhood of a point (x_k) at iteration k such that the model is considered to be an adequate representation of the original function. This region is called a trust region and defined as follows:

$$B(x_k, \Delta_k) = \{x \in \mathbb{R}^n : ||x - x_k|| \leq \Delta_k\},$$

where x_k is the trust region's center and Δ_k is its size.

When the first- and second-order derivative information is available, a quadratic model (f^r) is constructed by incorporating that information. A new candidate point ($\bar{x}$) is obtained by checking if it sufficiently reduces the model and lies within the trust region. Since the derivative information is available, a reduction in the objective function is guaranteed. Based on the quality of the candidate point, the trust region's size is either kept constant or increased or decreased, and the candidate point is either accepted or rejected as the trust region's center. Specifically, actual reduction achieved by the candidate point is compared with the predicted reduction by defining ρ_k as follows:

$$\rho_k = \frac{f(x_k) - f(\bar{x})}{f^r(x_k) - f^r(\bar{x})}.$$

Given the parameters $0 \leq \eta_0 \leq \eta_1 < 1$, $0 < \gamma_0 < 1$, and $\gamma_1 > 1$, the trust region's size is updated as follows:

$$\Delta_{k+1} = \begin{cases} \gamma_1 \Delta_k, & \text{if } \rho_k \geq \eta_1 \\ \Delta_k, & \text{if } \eta_0 \leq \rho_k < \eta_1 \\ \gamma_0 \Delta_k, & \text{if } \rho_k < \eta_0. \end{cases}$$

The trust region's center is updated as follows:

$$x_{k+1} = \begin{cases} \bar{x} & \text{if } \rho_k \geq \eta_0 \\ x_k, & \text{if } \rho_k < \eta_0. \end{cases}$$

The algorithm stops when the norm of the gradient is less than a pre-specified tolerance, i.e., $||\nabla f|| \leq \epsilon_1$. Due to the absence of derivative information, trust-region methods for black-box problems rely on data-driven models to find descent directions and convergence criteria. Thus, to ensure that the algorithm converges to a true optimal point, the model needs to be sufficiently accurate. The authors in [48] formalized the concept of accuracy by defining fully linear and fully quadratic models. Interpolating, underdetermined, and regression polynomial models and radial basis functions have been shown to satisfy fully linear property [125]. A model is said to be fully linear if f and f^r are continuously differentiable and the following relation holds:

$$||f(x) - f^r(x)|| \leq \kappa_f \Delta^2 \quad \forall x \in B(x_k, \Delta),$$
$$||\nabla f(x) - \nabla f^r(x)|| \leq \kappa_d \Delta \quad \forall x \in B(x_k, \Delta),$$

where the constants κ_f and $\kappa_d > 0$ are dependent on the Lipschitz constants of f and f^r and a constant Λ that measures the poisedness of the sample points [48]. Sample points are said to be well-poised if they are well dispersed in a region of interest. Note that poisedness is independent of the function values. The most common way to quantify poisedness is based on Lagrange polynomials. For more details of how poisedness is ensured in the algorithm, the reader is referred to the book by Conn et al. [49] and articles by Powell [166, 168]. The above relationship ensures that the gradient of the function and the function itself can be approximated well if the trust region's size is small and the samples are well-poised. Thus, if the current point is not already optimal, it is possible to find a descent direction by making the trust region smaller and samples well-poised. The update rule for trust region's center remains the same, but the criteria for updating its size are modified as follows:

$$\Delta_{k+1} = \begin{cases} \gamma_1 \Delta_k, & \text{if } \rho_k \geq \eta_1 \\ \Delta_k, & \text{if } \rho_k < \eta_1 \text{ and } f^r \text{ is not fully linear} \\ \gamma_0 \Delta_k, & \text{if } \rho_k < \eta_1 \text{ and } f^r \text{ is fully linear.} \end{cases}$$

The algorithm stops when the gradient of the surrogate model and the trust region's size both are less than the respective pre-specified tolerances, i.e., $||\nabla f^r|| \leq \epsilon_1$ and $\Delta_k \leq \epsilon_2$.

2.2.2 Projection-Based Methods

Projection-based methods are often employed to develop reduced-order models by projecting the actual model or the governing equations onto a suitably chosen low-dimensional subspace. There exist many projection-based reduction techniques (e.g., proper orthogonal decomposition) for models with known equations. However, black-box optimization poses more difficult challenges because of the purely data-driven nature of the problem. To this end, Bajaj and Hasan [20] proposed UNIPOPT (UNIvariate Projection-based OPTimization), which is a framework based on a projection of samples onto a univariate space defined by a linear combination of the decision variables. The projection results in a point-to-set map of the sample space. A univariate function exists on this map such that its optimum is also the optimum of the original multidimensional black-box problem. This so-called *lower envelope* can be approximated using a combination of sensitivity information (prediction step) and a trust region-based method (correction step). Once the lower envelope is approximated, it is then modeled as a single-variable function whose global minima also correspond to the global minima of the original problem.

2.3 Heuristic Methods

2.3.1 DIRECT

Among the hybrid methods, DIRECT is a Lipschitzian-based algorithm, introduced by Jones et al. [111] in 1993, for global optimization subject to simple variable bounds without requiring the need of obtaining gradient information. The algorithm is guaranteed to converge to the global optima if the objective function is continuous, or at least continuous in the vicinity of the global optimum [27]. The algorithm is well-suited for BBO applications as computing a Lipschitz constant is impossible if a closed-form expression of the objective function is unknown.

Jones et al. introduced a new perspective of viewing the Lipschitz constant as a weighting parameter to determine the emphasis on global versus local search. Traditional Lipschitzian-based methodologies assume a high value of Lipschitz constant such that its value is greater than the maximum rate of change of objective function. On the other hand, the DIRECT algorithm performs multiple searches with different values of Lipschitz constants such that the algorithm considers both local and global levels, thereby leading to faster convergence.

The DIRECT algorithm is suited for high-dimensional problems. When a multivariate objective function involves n variables with simple bounds, traditional Lipschitzian algorithms consider the search space to be an n-dimensional hyperrectangle in Euclidean space. The whole space is divided into smaller hyperrectangles, and function evaluations are performed at vertices. During the initialization stage, this requires performing 2^n function evaluations at each of the vertices.

However, the DIRECT algorithm addresses this complexity by only requiring evaluation at the midpoints instead of vertices. To ensure that sampling is performed at center points, during mesh refinement, the interval is divided into thirds and the function evaluations are then performed at center points of left and right thirds. The initial center point just becomes the center point of the middle interval where the objective function is known already. The algorithm does not utilize the Lipschitz constant for determining which hyperrectangles to sample next. Instead, the algorithm computes a set of potential hyperrectangles with the lowest objective value and the largest expected decrease in objective function value. For termination, as the Lipschitz constant is unknown, the number of algorithm iterations is pre-specified.

2.3.2 Multilevel Coordinate Search

The multilevel coordinate search (MCS) algorithm is inspired from the DIRECT algorithm, and similar to DIRECT, MCS recursively splits hyperrectangles and is guaranteed to converge to the global minimum if the function is continuous in its neighborhood. There are several limitations of DIRECT. For instance, the algorithm cannot handle infinite bounds. Moreover, since the sampling is done at center points, the algorithm takes longer time to converge if the solution lies at the boundary.

To counter these limitations, MCS extends the DIRECT algorithm by incorporating a multilevel approach and allowing more irregular splitting procedure. Similar to DIRECT, MCS performs both global and local searches and balances these two searches using a multilevel approach. However, unlike DIRECT, MCS splits the box in a single coordinate. The information gained through already sampled points is utilized for determining the splitting coordinate and position.

To keep a track of the number of times a box is processed, the algorithm assigns a *level* $s \in \{0, 1, \ldots, s_{max}\}$ to each of the boxes, wherein s indicates the number of times a box has been processed. $s = 0$ means that the box can be ignored as it has already been split, and $s = s_{max}$ indicates that the box is too small to be split further. MCS performs global search by splitting boxes with low levels, i.e., the boxes that are not searched extensively. To perform a local search, at a given level, the box with the lowest objective function is selected. The local search constructs a quadratic surrogate model and performs a line search by determining promising search direction thereby leading to fast convergence. The boxes are processed and split until the value of their level reaches s_{max}.

If a box has been processed and split numerous times, MCS splits along the coordinate with lowest levels, i.e., split by rank. However, if the boxes have not been split many times, MCS selects the most promising coordinate with the highest expected gain and splits along it, i.e., split by expected gain. As a result of this scheme, MCS utilizes the s_{max} value to specify the depth of local search thereby balancing the trade-off between global and local searches.

2.3.3 Hit-and-Run algorithms

Hit-and-run algorithms are stochastic global search methods independently developed by Boneh and Golan [28] and Smith [186]. In hit-and-run algorithms, a set of random candidate points are generated using a two-step methodology. Firstly, a random direction vector d is selected with the help of a uniform distribution over the unit hypersphere. Next, given this search direction, a random step s is generated such that $x + ds$ lies within the feasible region. The incumbent solution is compared with the randomly generated sample, and the incumbent solution is updated if an improvement in objective is realized. With extensive sampling, the points obtained through hit-and-run methods asymptotically converge to a uniform distribution [187].

Telgen et al. [24] modified the original algorithm by postulating coordinate directions, i.e., for an n-dimensional problem, the direction is selected among the $2n$ coordinate vectors, and the step length is chosen randomly on the search line set. There also exist other methods on extending hit-and-run methods such that the direction and/or step size are obtained using non-uniform distributions thereby leading to faster algorithm convergence [23, 117, 118].

2.3.4 Simulated Annealing

The simulated annealing algorithm is based on an analogy between optimization and statistical mechanics. Motivated by metropolis algorithm, which is used for numerically simulating the annealing of solids, Kirkpatrick et al. [120] developed the simulated annealing optimization algorithm for combinatorial optimization problems.

Unlike several other optimization algorithms, simulated annealing allows accepting a solution with worse objective value compared to the incumbent to avoid getting trapped in a local minimum. Such an uphill move is decided probabilistically with the help of an acceptance function, $f^{sa} = \exp(\delta/T)$ [150]. This function contains a parameter T, which is analogous to temperature in physical annealing. In the beginning of the algorithm, lower T values are set such that the uphill moves are accepted with a higher probability to escape local optimum. With time, T is decreased gradually thereby leading to lower probability of uphill moves. The algorithm has two loops wherein T is varied in the first loop, and the number of neighborhood moves is selected and performed in the second loop. Even though simulated annealing was initially proposed for combinatorial optimization problems, it has been extended to continuous problems.

Theoretically, the simulated annealing algorithm has been shown to be modeled using Markov chains with unique stationary distribution, which is independent of initial state, provided it is possible to move between any two states with a non-zero probability [58]. Here, when considering the physical analogy, the stationary distribution corresponds to the Boltzmann distribution thereby asymptotically converging

to globally optimal solutions. However, it is not guaranteed that the global solution will be attained in a finite number of iterations.

2.3.5 Genetic Algorithm

First proposed by Holland [97], genetic algorithm (GA) is a global heuristic search method that mimics the natural evolution process to find approximate solutions for optimization problems. GA is a class of evolutionary algorithms in which the underlying operations are motivated by evolutionary biology. Specifically, the evolutionary concepts such as inheritance, selection, mutation, and crossover are performed on a solution vector (analogous to chromosomes), and the resulting solutions are checked for their quality (analogous to fitness value). GA is based on survival of the fittest, i.e., high-quality solutions and their subsequent offsprings are selected for further consideration. Due to the ease with which it can be applied and its ability to handle complex optimization problems well, GA has been extensively applied on a variety of applications including combinatorial optimization [185], machine learning [75], chemometrics [127], electromagnetics [200], and operations management [18]. The advantages of GA-based methodologies are that they are not affected by discontinuous response surface, are good for multi-modal problems, and are suitable for high-dimensional problems.

A typical GA methodology is as follows. Initially, a set of individual solutions is randomly generated to cover the entire range of search space extensively. The user can also specify initial solutions (i.e., seeding) where the likelihood of finding good solutions is high. The next step consists of the selection stage, where the solution quality of the sample points is utilized for determining the solutions that will be selected for breeding to generate solutions of the future generation. Several methods, e.g., roulette wheel selection and tournament selection, exist for selecting individual solutions for breeding. To prevent premature convergence and balance local versus global search, a few solutions of poor quality can also be selected to ensure diversity. Next, the reproduction stage occurs where the selected solutions from the previous step are mated through crossover and/or mutation. More than two solutions (parents) can also be utilized for obtaining candidate solutions as they have been shown to be of higher quality [59]. As chromosomes with high fitness values are carried forward, their offsprings are expected to have better fitness value compared to previous generations. The algorithm is continued until some pre-specified end condition is met. Typical termination conditions often include minimum solution quality, maximum number of generations, solution improvement criterion, or a mix of these criteria.

2.3.6 Particle Swarm Optimization

Particle swarm optimization (PSO) is a population-based global search algorithm developed by Kennedy and Eberhart, which is motivated by the cooperative social

behavior of animals such as birds and bees [119]. PSO aims to find the optimal solution by leveraging a swarm of particles where each particle is a candidate solution. In the search space, the particles cooperate with one another to reach the best solution. Each particle has position and velocity attributes, which are updated at every step of the algorithm. At the next step, the particle velocity is determined by previous velocity, best solution position for both the particle and the entire swarm, and acceleration coefficients. The future position is calculated for a given particle using current position and computed velocity. For optimization applications, PSO has been shown to be competitive when compared to other global search heuristic algorithms such as genetic algorithms [184, 207]. In the literature, several extensions of PSO have been proposed to improve its performance for high-dimensional functions [40, 62, 132], better algorithm convergence [63, 123, 141, 149] to avoid local optimum [109], and constrained optimization problems [56].

2.3.7 Surrogate Management Framework

The surrogate management framework (SMF) is developed by Booker and coworkers for optimization problems with expensive simulations, where inexpensive approximations are utilized for guiding search direction in pattern search methods [29]. The key idea behind SMF is that surrogate models are leveraged for accelerating pattern search methods while preserving robust convergence guarantees offered by them. Similar to pattern search methods, SMF consists of *search* and *poll* steps. The search step utilizes the surrogate model to generate promising candidate points for improving objective value while balancing global versus local search. The poll step leads to algorithm convergence, wherein the algorithm checks whether the current best solution is a mesh local optimizer. Within the SMF workflow, the literature exists where pattern search methods have been replaced with others methods such as trust-region methods [80].

2.3.8 Branch and Fit

Proposed by Huyer and Neumaier [103], SNOBFIT is the Matlab implementation of the branch-and-fit algorithm, which stands for Stable Noisy Optimization by Branch and Fit. The algorithm has been developed for solving bound-constrained optimization problems with expensive objective function. SNOBFIT performs function evaluations both near local optimum where the search is guided by fitted quadratic models and in unexplored regions of the input space by recursively partitioning search space. Based on existing function evaluations, the algorithm constructs approximated function models locally, which are then minimized to obtain the user-defined number of candidate points. In case of insufficient number of points for model fitting, random sampling is performed for generating new candidate points. Depending on the methodology used for generating candidate points, they are classified into one of the five classes. The points within classes 1 to 3 are

generated considering the local aspect, whereas the points belonging to classes 4 and 5 indicate the global aspect and are positioned in unexplored regions of search space. Within SNOBFIT, soft constraints are handled by a penalty-based approach, and to handle hidden constraints, a fictitious function value is assigned. SNOBFIT does not provide any theoretical guarantees for reaching global minimum within a finite number of function evaluations.

2.4 Hybrid Methods

To overcome the limitations of different methods, hybrid algorithms have been developed to combine the advantages of traditional optimization methods in terms of run time, convergence, or the number of function evaluations. There are three major ways of combining different algorithms to develop hybrid algorithms. The first class of methods is when one algorithm is run to obtain an initial point for the second algorithm, which is then used for the rest of the optimization. The second class consists of simultaneously running optimization algorithms that often share function evaluation data or intermediate obtained solutions. The third class consists of combining optimization methods on the iteration level wherein one algorithm controls the outer loop, whereas the other method is sequentially called in the inner loop.

In the literature, there have been several hybrid algorithms that have been developed by modifying the original DIRECT algorithm. DIRECT usually gets close to the global solution quickly, but it can be slow in converging to the final solution. Jones et al. [110] reported that the performance of DIRECT can be improved by using a local optimization routine. As a result, there have been several research works incorporating local search methods in the DIRECT workflow. Carter et al. [37] proposed a hybrid algorithm combining DIRECT and implicit filtering methods for a gas pipeline optimization problem wherein the global search characteristic of DIRECT was leveraged for generating a feasible initial point for implicit filtering. Wachowiak et al. [197, 198] combined DIRECT with generalized pattern search and multidirectional search methods for local refinement to improve the algorithm performance for multiprocessor systems. They observed a significant improvement in speed and accuracy for a medical image registration problem. Griffin et al. [83] combined DIRECT and generating set search (GSS) algorithms for parallel hybrid optimization and observed that the hybrid algorithm has significantly better performance in terms of CPU run time than DIRECT without any compromise on the solution quality. Hemker et al. [95] introduced an inner loop to the DIRECT algorithm, based on a surrogate-based optimization framework, to take advantage of both local and global methods. Liuzzi et al. [136] leveraged BBO-based local strategies to enhance the efficiency of DIRECT-type algorithms for bound-constrained global optimization problems.

Metaheuristic methods have also been combined with deterministic methods to develop more efficient hybrid optimization schemes [57, 89, 90, 162, 174, 189].

Yehui et al. [204] developed a hybrid algorithm by combining the pattern search method with genetic algorithm for unconstrained optimization. Here, during the search phase of the pattern search algorithm, genetic algorithm was leveraged for producing candidate trial points. Payne et al. [162] also combined pattern search and genetic algorithm to locate heavy atoms within simulated crystals. Vaz et al. [194, 195] incorporated particle swarm optimization in the search phase of pattern search method for global optimization of a function with simple variable bounds. Griffin et al. [81] combined genetic algorithm and pattern search methods to solve mixed-integer nonlinear optimization problems wherein genetic algorithm facilitated the global search aspect, and pattern search methods were called for the local search. Martelli et al. [146] developed a hybrid algorithm for nonsmooth black-box problems with several nonlinear constraints by combining three different algorithms, i.e., particle swarm, GSS, and COMPLEX.

2.5 Extension to Constrained Problems

Several derivative-based techniques in the deterministic constrained optimization literature have been extended to derivative-free counterparts for BBO. This section presents some of the approaches developed.

2.5.1 Penalty Method

Penalty-based methods are one of the most popular ways for handling constraints in BBO. In these algorithms, penalty functions based on constraint violation are incorporated in the objective function in such a way that minimizing constraint violation leads to better objective values. The constraint violation is multiplied by a penalty parameter, and the value of penalty parameter can either be increased iteratively or can be fixed as is the case in exact penalty-based methods. There exist different types of penalty functions, e.g., quadratic penalty and log-barrier penalty functions. Those methods that consider both feasible and infeasible solutions, and where the optimal solution can be approached from the infeasible region, are called exterior methods. The interior methods, on the other hand, impose the optimization to stay within the feasible region and approach the optimal solution from within the feasible region.

In the context of applying penalty methods for constrained BBO, Griffin et al. [82] studied the application of a variety of penalty functions using GSS with missing derivative information of objective function or constraints. Audet and Dennis [13] proposed a progressive barrier-based penalty method with GPS for a wide class of constrained BBO problems. In their approach, constraint violations were summed together into a single function, and a threshold was imposed on the constraint violation function which reduces progressively with the number of iterations. Liuzzi et al. [137] extended sequential penalty approach to BBO with

continuously differentiable objective function and constraints. Fasano et al. [66] developed a line search-based approach wherein an exact penalty function was utilized to solve BBO problems with nonsmooth objective function and constraints. Gratton et al. [79] combined the use of merit function and extreme barrier for handling relaxable and unrelaxable constraints, respectively, for BBO problems subject to variable bounds. In addition to these developments, there also exists a vast literature on penalty-based constraint handling techniques for particle swarm optimization [152], genetic algorithm [153], and other nature-inspired algorithms [122, 151].

2.5.2 Augmented Lagrangian

Augmented Lagrangian methods are similar to penalty-based methods. However, in these methods, an additional term is added to the objective function which mimics a Lagrange multiplier. Therefore, penalty parameters as well as Lagrange multipliers need to be computed and updated during the course of an iteration. The advantage of using augmented Lagrangian is that the penalty parameters are not required to have very high values due to the addition of Lagrange multipliers, and the algorithm converges to the optimal solution even with fixed values of penalty parameters.

Traditional pattern search methods require derivative information to estimate search directions. To get rid of the requirement of gradient information, Lewis et al. [130] extended the bound-constrained augmented Lagrangian method, originally developed by Conn et al. [45], for BBO problems by proposing a stopping criterion on mesh size such that the convergence guarantees remain intact. Kolda et al. [121] used a similar strategy of replacing a derivative-based stopping criterion in original GSS with a derivative-free stopping criterion for developing a GSS-based augmented Lagrangian algorithm for BBO problems with linear constraints. Lewis et al. [131] used an augmented Lagrangian-based method based on GSS for solving nonlinear programming problems with linear constraints. Gramacy et al. [78] developed a hybrid optimization approach combining Gaussian process surrogates, expected improvement heuristics, and augmented Lagrangian methods for handling constraints in complex BBO problems.

2.5.3 Filter Method

Filter methods were first introduced for sequential quadratic programming (SQP) methods for constrained nonlinear optimization problems [69]. Relatively recently, these methods have been proposed as an alternative to penalty methods for handling constraints in BBO [70]. Unlike penalty-based methods that combine objective function and constraints into a unified function, filter-based methods recast original constrained optimization problem as a biobjective optimization problem wherein the algorithm focuses on minimizing both objective and constraint violation values. A solution dominates another solution only when both the values of objective

function and constraint violation are improved. The major advantage of the filter method is that there is no requirement of specifying penalty parameters or estimating Lagrange multipliers. Audet and Dennis [11] developed a BBO algorithm based on a filter pattern search method for general constrained problems. Abramson et al. [3] presented a filter-based BBO algorithm for mixed-variable optimization problems with continuous and categorical variables. Motivated by the SQP filter method of Fletcher et al. [69], Eason et al. [55] developed a trust region-based filter method for constrained BBO problems.

2.5.4 Surrogate Modeling

Surrogate-assisted BBO has been leveraged for constrained optimization problems wherein individual surrogate models are developed for each constraint and objective function [167, 176]. This strategy is especially versatile when the approximated constraint models are used individually, as this preserves information regarding individual constraints. Overall, the objective of surrogate-assisted constrained BBO is to obtain feasible candidate points with maximum expected improvement in objective value.

Zhou et al. [208] presented a surrogate-assisted genetic algorithm for bound-constrained nonlinear problems that leverages hierarchical global and local surrogates based on Gaussian process and radial basis models, respectively. Regis et al. [177] developed a surrogate-assisted evolutionary programming algorithm for constrained BBO problems with several inequality constraints and applied it successfully on high-dimensional problems. The developed algorithm was shown to have superior performance over penalty-based evolutionary programming algorithms. They further developed surrogate-assisted algorithms for BBO problems with infeasible starting point using radial basis surrogates [178]. Conn et al. [47] proposed a hybrid method with integrated direct search algorithm and quadratic surrogate models and demonstrated that inclusion of surrogate models improved the performance of direct search. More recently, Bajaj et al. [21] developed a two-phase strategy for constrained black-box and gray-box optimization problems. The first phase involves finding feasible point through minimizing a smooth constraint violation function (feasibility phase). The second phase improves the objective in the feasible region using the solution of the feasibility phase as starting point (optimization phase). Surrogate models are developed using input–output data in a trust-region framework. The strategy does not require feasible initial points and handles hard constraints via a novel optimization-based constrained sampling scheme.

3 BBO Solvers

There are many software packages that are available in the public domain for solving BBO problems. For example, adaptive simulated annealing (ASA) is a C implementation of the simulated annealing algorithm for unconstrained BBO problems [104, 105]. TOMLAB is a Matlab package within which several BBO algorithms can be accessed including genetic algorithm (GENO), DIRECT (glcDirect and glcCluster), scatter search (OQNLP), multistart (MULTIMIN), ant colony optimization (MIDACO), among others [98, 99]. Nonlinear Optimization with MADs (NOMAD) is a C++ software that implements the mesh adaptive direct search for complex BBO problems [12, 126]. To handle constraints, NOMAD offers several constraint-handling techniques including progressive barrier, filter, and extreme barrier. UNIPOPT is a projection-based solver for multidimensional box-constrained black-box problems [20]. The method projects a multidimensional black-box function onto an auxiliary univariate space, which leads to a point-to-set map. The lower envelope of the map contains the global minimum. A sensitivity theorem is employed to predict the points on the lower envelope, and a trust region-based algorithm is used to correct the predictions. A model-based algorithm is used to optimize the univariate lower envelope such that its minimum corresponds to that of the original problem. MCS is a Matlab implementation of the multilevel coordinate search method for bound-constrained global BBO [102]. DFO is a Fortran package developed by Conn and Scheinberg for small-scale black-box optimization problems with expensive simulations and noise [44]. Covariance matrix adaptation evolutionary strategy (CMA-ES) is an evolutionary algorithm with implementations in several programming languages including C, C++, Fortran, JAVA, Python, and Matlab [88]. NEWUOA is Fortran software for unconstrained optimization with simple variable bounds that employs quadratic models for function approximation [169, 170]. Bound Optimization BY Quadratic Approximation (BOBYQA) is a Fortran package that extends the NEWUOA method and implements a trust region-based algorithm, developed by Powell et al. [171], for finding the minimum of a black-box function with box constraints. PSwarm combines pattern search and particle swarm optimization for linearly and bound-constrained BBO problems, with implementations in both C and Matlab [194, 195]. The solver can also be interfaced with AMPL, Python, and R through an API. Stable Noisy Optimization by Branch and FIT (SNOBFIT) is a Matlab package for noisy BBO problems with bounded continuous variables, where additional soft constraints can be handled by a penalty approach [103]. Design Analysis Kit for Optimization and Terascale Applications (DAKOTA) is an extensive C++ library that implements numerous gradient and nongradient-based optimization algorithms [60]. For BBO, different algorithms are present in DAKOTA including pattern search, DIRECT, NOMAD, evolutionary algorithms, efficient global optimization, and SOLIS-WETS. Hybrid Optimization Parallel Search PACKage (HOPSPACK) is a C++ pattern search implementation for linearly and nonlinearly constrained BBO problems with continuous and integer variables and leverages parallel processing through MPI or multithreading [163].

PDFO (Powell's Derivative-Free Optimization solvers) is a cross-platform package, developed by Ragonneau and Zhang [173], in Matlab and Python that implements several algorithms proposed by Powell including UOBYQA [168], NEWUOA [169], BOBYQA [171], and LINCOA [172].

4 Recent Applications

4.1 Automatic Machine Learning

There has been a growing interest in BBO strategies for automating machine learning workflows with minimal human intervention. The major application consists of hyperparameter optimization for ML models, i.e., obtaining optimal learning rates, neural network architecture, model selection, etc. Here, the ML model is considered as a black-box function, and the objective consists of optimizing single or multiple criteria of performance [25]. Moreover, more complex black-box functions can be considered in the form of a ML pipeline wherein several steps could occur including data pre-processing, dimensionality reduction, model selection, model training, cross-validation, and post-processing. The strategies have been developed to address several types of hyperparameters that exist in ML models, such as continuous and categorical parameters with hierarchical dependencies. Several of the developed methodologies are based on model-based optimization where the surrogate model for the objective function could be a deep neural network, random forest model, or a Gaussian process.

Multi-fidelity techniques have also become popular for implementing BBO-based optimization routines in hyperparameter optimization workflows where the black-box function is expensive, and it is not possible to obtain several function evaluations exactly [113–115, 182]. To address this, relatively course approximations are computed with lesser time and computational resources. For instance, a single run of a machine learning pipeline with model training and cross-validation can take several hours to days even with GPU resources, thereby limiting the number of function evaluations that can be performed. To counter this, the model can be trained with fewer epochs as a way of obtaining cheap approximations. In spirit, this is analogous to physics-based solvers where a partial differential equation model can be solved with a course discretization, as opposed to a finely discretized grid, to bring down the simulation time.

More recently, Google developed an internal service called Google Vizier incorporating several black-box optimization methodologies [76]. The service is extensively used for optimizing ML models, parameter tuning, and providing key functionalities to Google's ML HyperTune subsystem. Even though several of the algorithms implemented already have their open-source implementations, the service standardizes the workflow for applying different BBO methodologies thereby resulting in a faster set-up and prototyping time.

4.2 Optimization Solvers

In addition to tuning hyperparameters for machine learning methods, BBO is finding additional uses in tuning optimization solver options. Tuning the solver options fall under the broader area of algorithm-based parameter tuning where BBO techniques have found an extensive use [9, 17]. Traditionally, users rely on default solver parameters as it remains largely unclear what would constitute an optimal combination of solver options. However, there exist a large number of combinations of possible solver options that significantly affect solver performance thereby necessitating the need of obtaining good solver options.

Hutter et al. [101] used a BBO-based strategy for optimally configuring solver options for efficiently solving mixed integer programming (MIP) problems. Here, the solver is considered as a black box, the objective function consists of improving solver performance, and the inputs consist of different solver options. Multiple MIP solvers, including CPLEX, GUROBI, and LPSOLVE, were configured via BBO techniques, and at least a speedup of 2.3 was observed. Notably, in case of CPLEX, a speedup by a factor of 52 was realized compared to the default CPLEX tuning tool. Haöglund et al. [87] leveraged surrogate models based on radial basis functions for computing optimal parameter settings for crew optimization problems. Optimal solver options were obtained automatically to reduce solver time and improve solution quality. Recently, Liu et al. [134] used BBO for optimizing the solver performance of deterministic global optimization solver, BARON, for nonlinear programming (NLP) and mixed-integer nonlinear programming (MINLP) class of problems.

4.3 Fluid Mechanics

BBO finds extensive applications in complex fluid mechanics problems that involve solving time-dependent Navier–Stokes equations. Such problems are commonly found in several fields including chemical engineering, aerospace engineering, mechanical engineering, architectural engineering, and biological engineering.

To counter the limitations of gradient-based methods, Lehnhäuser et al. [128] used a trust region-based BBO strategy for optimizing shapes of fluid flow domains for several test cases including pressure-drop minimization, lift-force maximization, and heat-transfer optimization. Marsden et al. [143–145] optimized the shape of an airfoil trailing-edge for reducing noise in constrained unsteady laminar flow problems. Marsden et al. [142] further coupled a BBO optimization algorithm with blood flow simulations for optimization of cardiovascular geometries.

Forrester et al. [73] developed global surrogate models wherein CFD simulations were performed for constructing metamodels. Instead of performing full-scale simulations, a novel strategy of leveraging partially converged simulation data was proposed which was observed to outperform traditional strategies. Tran et

al. [191] devised a BBO optimization strategy where two different Gaussian processes-based surrogate models were utilized for representing objective function and unknown constraints, respectively, and the overall algorithm was applied for optimizing a multi-phase CFD model for optimizing the design of a centrifugal slurry pump impeller. More recently, Park et al. [161] leveraged BBO for performing multiobjective optimization for optimizing the operation of a stirred-tank chemical reactor that was represented through CFD simulations.

In addition to aforementioned works, there are several other applications incorporating BBO strategies to optimize CFD-based systems including chemical reactors [41, 112, 156, 180, 193], nanotubes [1, 61, 165, 181], heat transfer [38, 71, 77], natural and mechanical ventilation [42, 116, 139, 140], and HVAC systems [138, 202, 203, 206].

4.4 Oilfield Development and Operations

BBO strategies have found an extensive use in optimizing several oilfield development and operational problems including well placement, well type and drilling decision-making, production optimization, and net present value maximization. This is because a large number of oilfield optimization problems incorporate expensive objective functions that require calling high-fidelity reservoir models. In addition, physical and economic constraints on oilfields, along with highly nonlinear reservoir dynamics, often lead to nonlinear constrained optimization problems. Such cases have motivated the use of BBO strategies, which are preferred due to ease of implementation, no requirement of gradient information, and parallel computing.

Artus et al. [7] leveraged genetic algorithm for optimizing the type and location of an unconventional well under geological uncertainty. A similar problem of determining optimal well type and location was addressed by Onwunalu et al. [159], and they concluded that particle swarm optimization outperformed genetic algorithm on an average. Ciaurri et al. [43] utilized several BBO techniques, including generalized pattern search, genetic algorithm, and particle swarm optimization, for optimizing three key optimization problems in oilfield management, and it was observed that generalized pattern search was one of the most suitable methodologies for such optimization problems. Isebor et al. [106, 107] and Humphries et al. [100] used particle swarm optimization and mesh adaptive direct search methods for simultaneously solving well location and control optimization problems. In addition, an improved hybrid strategy was developed using the two algorithms wherein particle swarm optimization was applied for the well location problem, and mesh adaptive direct search was used for well control problem.

Asadollahi et al. [8] investigated several BBO and gradient-based methods for optimizing production of the Brugge field. It was observed that gradient-based methods have several issues thereby causing limited performance, and BBO methods have better performance in terms of objective value and computational burden. Foroud et al. [72] also performed a similar study of evaluating different

optimization methodologies while treating the Brugge field simulator as a black box. In their study, guided pattern search was found to be the most efficient algorithm for oil production optimization. In addition to the aforementioned studies, there is a vast literature on the application of BBO methodologies for the optimization of oilfield development and operation problems (see [22, 26, 52, 65, 85, 86, 133, 183, 205]).

4.5 Chemical and Biochemical Engineering

In chemical and biomedical engineering, BBO has been extensively used for numerous applications including heat exchanger network design [53, 146, 147, 160, 164, 175], process flowsheet optimization [31, 35, 39, 67, 84, 96, 129, 157], supply chain management [93, 199], carbon capture [30, 92], natural gas purification [68, 155], biofuel production [51, 64, 74], parameter estimation [154, 179, 188], and process intensification [5, 6]. To counter the limitations of traditional optimization strategies for complex chemical engineering problems, which are typically represented by mixed-integer nonlinear programs, Gross et al. [84] used evolutionary algorithms for simultaneously optimizing process network topology and operating conditions. Grossmann et al. [35, 36] and Boukouvala et al. [31, 32] developed Kriging-based interpolating metamodels for optimizing constrained and noisy black-box problems commonly found in process flowsheet optimization. Hasan and coworkers developed methodologies for the design and optimization of many dynamic and intensified chemical process systems with applications to carbon capture, natural gas separation, methanol production, hydrogen generation, and acid gas removal [5, 6, 33, 68, 92, 94, 108, 135]. Lewin et al. [129] and Dipama et al. [53] leveraged genetic algorithm for simultaneous synthesis and optimization of heat exchanger networks. In biochemical engineering, there have been several works wherein BBO has been utilized for biofuel production from biomass [51, 64, 74].

5 Open Problems and Future Research Directions

While we have seen major strides made in recent years in the areas of derivative-free and black-box optimization, there are still several important open questions that need to be resolved. The size and scale of the black-box problems that can be solved using current methods are small. Many solvers fail when the dimension of a black-box problem is high. The model-based approaches suffer from a large number of parameters that need to be fitted. This requires them to use a large number of samples or simulations before converging to a solution. This is prohibitive if the simulations are computationally very expensive. We also do not have efficient methods for solving black-box problems involving discrete decisions. Another key challenge is the lack of global optimization methods for non-convex black-box problems. Direct search methods (such as PSWARM, MCS,

and CMA-ES) are sometimes successful in achieving global optimal solutions, but they do not provide theoretical guarantee. Model-based methods often converge to local minima because of their limitation in exploring the entire search space using finite samples. Although it is difficult to provide a deterministic guarantee on the solution optimality for black-box problems, some attempts have been made in this area. A notable example is the work of Bajaj and Hasan [19] who have recently proposed a deterministic global optimization algorithm based on a vertex polyhedral edge-concave underestimator [91] that only requires information on the upper bound of the diagonal element of the Hessian of a black-box function. With some physical intuition and information on the problem at hand (e.g., mass and energy conservation laws, physically attainable regions, etc.), one can exploit such an approach to obtain solutions that are better than many local solutions.

References

1. Abdollahi, A., Shams, M.: Optimization of heat transfer enhancement of nanofluid in a channel with winglet vortex generator. Appl. Thermal Eng. **91**, 1116–1126 (2015)
2. Abramson, M.A., Audet, C., Dennis, J.E.: Generalized pattern searches with derivative information. Math. Program. **100**(1), 3–25 (2004)
3. Abramson, M.A., Audet, C., Dennis Jr, J.: Filter pattern search algorithms for mixed variable constrained optimization problems. Tech. rep., Air Force Inst of Tech Wright-Patterson AFB OH (2004)
4. Abramson, M.A., Audet, C., Dennis Jr, J.E., Digabel, S.L.: OrthoMADS: a deterministic MADS instance with orthogonal directions. SIAM J. Optim. **20**(2), 948–966 (2009)
5. Arora, A., Bajaj, I., Iyer, S.S., Hasan, M.F.: Optimal synthesis of periodic sorption enhanced reaction processes with application to hydrogen production. Comput. Chem. Eng. **115**, 89–111 (2018)
6. Arora, A., Iyer, S.S., Bajaj, I., Hasan, M.F.: Optimal methanol production via sorption-enhanced reaction process. Ind. Eng. Chem. Res. **57**(42), 14143–14161 (2018)
7. Artus, V., Durlofsky, L.J., Onwunalu, J., Aziz, K.: Optimization of nonconventional wells under uncertainty using statistical proxies. Comput. Geosci. **10**(4), 389–404 (2006)
8. Asadollahi, M., Nævdal, G., Dadashpour, M., Kleppe, J.: Production optimization using derivative free methods applied to Brugge field case. J. Pet. Sci. Eng. **114**, 22–37 (2014)
9. Audet, C., Dang, K.C., Orban, D.: Optimization of algorithms with OPAL. Math. Program. Comput. **6**(3), 233–254 (2014)
10. Audet, C., Dennis Jr, J.E.: Analysis of generalized pattern searches. SIAM J. Optim. **13**(3), 889–903 (2002)
11. Audet, C., Dennis Jr, J.E.: A pattern search filter method for nonlinear programming without derivatives. SIAM J. Optim. **14**(4), 980–1010 (2004)
12. Audet, C., Dennis Jr, J.E.: Mesh adaptive direct search algorithms for constrained optimization. SIAM J. Optim. **17**(1), 188–217 (2006)
13. Audet, C., Dennis Jr, J.E.: A progressive barrier for derivative-free nonlinear programming. SIAM J. Optim. **20**(1), 445–472 (2009)
14. Audet, C., Hare, W.: Derivative-Free and Blackbox Optimization. Springer, Berlin (2017)
15. Audet, C., Ianni, A., Le Digabel, S., Tribes, C.: Reducing the number of function evaluations in mesh adaptive direct search algorithms. SIAM J. Optim. **24**(2), 621–642 (2014)
16. Audet, C., Ihaddadene, A., Le Digabel, S., Tribes, C.: Robust optimization of noisy blackbox problems using the mesh adaptive direct search algorithm. Optim. Lett. **12**(4), 675–689 (2018)

17. Audet, C., Orban, D.: Finding optimal algorithmic parameters using derivative-free optimization. SIAM J. Optim. **17**(3), 642–664 (2006)
18. Aytug, H., Khouja, M., Vergara, F.: Use of genetic algorithms to solve production and operations management problems: a review. Int. J. Prod. Res. **41**(17), 3955–4009 (2003)
19. Bajaj, I., Hasan, M.F.: Deterministic global derivative-free optimization of black-box problems with bounded hessian. Optim. Lett. **14**, 1011–1026 (2020)
20. Bajaj, I., Hasan, M.F.: UNIPOPT: Univariate projection-based optimization without derivatives. Comput. Chem. Eng. **127**, 71–87 (2019)
21. Bajaj, I., Iyer, S.S., Hasan, M.F.: A trust region-based two phase algorithm for constrained black-box and grey-box optimization with infeasible initial point. Comput. Chem. Eng. **116**, 306–321 (2018)
22. Bangerth, W., Klie, H., Wheeler, M.F., Stoffa, P.L., Sen, M.K.: On optimization algorithms for the reservoir oil well placement problem. Comput. Geosci. **10**(3), 303–319 (2006)
23. Bélisle, C.J., Romeijn, H.E., Smith, R.L.: Hit-and-run algorithms for generating multivariate distributions. Math. Oper. Res. **18**(2), 255–266 (1993)
24. Berbee, H., Boender, C., Ran, A.R., Scheffer, C., Smith, R.L., Telgen, J.: Hit-and-run algorithms for the identification of nonredundant linear inequalities. Math. Program. **37**(2), 184–207 (1987)
25. Bischl, B., Richter, J., Bossek, J., Horn, D., Thomas, J., Lang, M.: mlrMBO: a modular framework for model-based optimization of expensive black-box functions (2017). arXiv preprint arXiv:1703.03373
26. Bittencourt, A.C., Horne, R.N., et al.: Reservoir development and design optimization. In: SPE Annual Technical Conference and Exhibition. Society of Petroleum Engineers (1997)
27. Björkman, M., Holmström, K.: Global optimization using the direct algorithm in Matlab (1999)
28. Boneh, A., Golan, A.: Constraints redundancy and feasible region boundedness by random feasible point generator (RFPG). In: Third European Congress on Operations Research (EURO III), Amsterdam (1979)
29. Booker, A.J., Dennis, J.E., Frank, P.D., Serafini, D.B., Torczon, V., Trosset, M.W.: A rigorous framework for optimization of expensive functions by surrogates. Struct. Optim. **17**(1), 1–13 (1999)
30. Boukouvala, F., Hasan, M.F., Floudas, C.A.: Global optimization of general constrained grey-box models: new method and its application to constrained PDEs for pressure swing adsorption. J. Global Optim. **67**(1–2), 3–42 (2017)
31. Boukouvala, F., Ierapetritou, M.G.: Surrogate-based optimization of expensive flowsheet modeling for continuous pharmaceutical manufacturing. J. Pharm. Innov. **8**(2), 131–145 (2013)
32. Boukouvala, F., Ierapetritou, M.G.: Derivative-free optimization for expensive constrained problems using a novel expected improvement objective function. AIChE J. **60**(7), 2462–2474 (2014)
33. Boukouvala, F., Misener, R., Floudas, C.A.: Global optimization advances in mixed-integer nonlinear programming, MINLP, and constrained derivative-free optimization, CDFO. Eur. J. Oper. Res. **252**(3), 701–727 (2016)
34. Bűrmen, Á., Puhan, J., Tuma, T.: Grid restrained Nelder-Mead algorithm. Comput. Optim. Appl. **34**(3), 359–375 (2006)
35. Caballero, J.A., Grossmann, I.E.: An algorithm for the use of surrogate models in modular flowsheet optimization. AIChE J. **54**(10), 2633–2650 (2008)
36. Caballero, J.A., Grossmann, I.E.: Rigorous flowsheet optimization using process simulators and surrogate models. In: Computer Aided Chemical Engineering, vol. 25, pp. 551–556. Elsevier, Amsterdam (2008)
37. Carter, R., Gablonsky, J., Patrick, A., Kelley, C.T., Eslinger, O.: Algorithms for noisy problems in gas transmission pipeline optimization. Optim. Eng. **2**(2), 139–157 (2001)
38. Cavazzuti, M., Corticelli, M.A.: Optimization of heat exchanger enhanced surfaces through multiobjective genetic algorithms. Numer. Heat Transf. A Appl. **54**(6), 603–624 (2008)

39. Chambers, M., Mount-Campbell, C.: Process optimization via neural network metamodeling. Int. J. Prod. Econ. **79**(2), 93–100 (2002)
40. Chan, F.T., Kumar, V., Mishra, N.: A CMPSO algorithm based approach to solve the multi-plant supply chain problem. Swarm Intell. Focus Ant Particle Swarm Optim. **532**, 54879954 (2007)
41. Chen, M., Wang, J., Zhao, S., Xu, C., Feng, L.: Optimization of dual-impeller configurations in a gas–liquid stirred tank based on computational fluid dynamics and multiobjective evolutionary algorithm. Ind. Eng. Chem. Res. **55**(33), 9054–9063 (2016)
42. Chen, Q.: Ventilation performance prediction for buildings: a method overview and recent applications. Build. Environ. **44**(4), 848–858 (2009)
43. Ciaurri, D.E., Mukerji, T., Durlofsky, L.J.: Derivative-free optimization for oil field operations. In: Computational Optimization and Applications in Engineering and Industry, pp. 19–55. Springer, Berlin (2011)
44. Conn, A., Scheinberg, K., Toint, P.: Manual for Fortran Software Package DFO v1. 2 (2000)
45. Conn, A.R., Gould, N.I., Toint, P.: A globally convergent augmented Lagrangian algorithm for optimization with general constraints and simple bounds. SIAM J. Numer. Anal. **28**(2), 545–572 (1991)
46. Conn, A.R., Gould, N.I.M., Toint, P.L.: Trust Region Methods, vol. 1. SIAM (2000)
47. Conn, A.R., Le Digabel, S.: Use of quadratic models with mesh-adaptive direct search for constrained black box optimization. Optim. Methods Softw. **28**(1), 139–158 (2013)
48. Conn, A.R., Scheinberg, K., Vicente, L.N.: Geometry of interpolation sets in derivative free optimization. Math. Program. **111**(1–2), 141–172 (2008)
49. Conn, A.R., Scheinberg, K., Vicente, L.N.: Introduction to derivative-free optimization, vol. 8. SIAM (2009)
50. Custódio, A.L., Vicente, L.N.: Using sampling and simplex derivatives in pattern search methods. SIAM J. Optim. **18**(2), 537–555 (2007)
51. Dhingra, S., Bhushan, G., Dubey, K.K.: Development of a combined approach for improvement and optimization of Karanja biodiesel using response surface methodology and genetic algorithm. Front. Energy **7**(4), 495–505 (2013)
52. Ding, S., Jiang, H., Li, J., Tang, G.: Optimization of well placement by combination of a modified particle swarm optimization algorithm and quality map method. Comput. Geosci. **18**(5), 747–762 (2014)
53. Dipama, J., Teyssedou, A., Sorin, M.: Synthesis of heat exchanger networks using genetic algorithms. Appl. Thermal Eng. **28**(14–15), 1763–1773 (2008)
54. Dolan, E.D., Lewis, R.M., Torczon, V.: On the local convergence of pattern search. SIAM J. Optim. **14**(2), 567–583 (2003)
55. Eason, J.P., Biegler, L.T.: A trust region filter method for glass box/black box optimization. AIChE J. **62**(9), 3124–3136 (2016)
56. Eberhard, P., Sedlaczek, K.: Using augmented Lagrangian particle swarm optimization for constrained problems in engineering. In: Advanced Design of Mechanical Systems: From Analysis to Optimization, pp. 253–271. Springer, Berlin (2009)
57. Egea, J.A., Martí, R., Banga, J.R.: An evolutionary method for complex-process optimization. Comput. Oper. Res. **37**(2), 315–324 (2010)
58. Eglese, R.W.: Simulated annealing: a tool for operational research. Eur. J. Oper. Res. **46**(3), 271–281 (1990)
59. Eiben, A.E., Raue, P.E., Ruttkay, Z.: Genetic algorithms with multi-parent recombination. In: International Conference on Parallel Problem Solving from Nature, pp. 78–87. Springer, Berlin (1994)
60. Eldred, M.S., Giunta, A.A., van Bloemen Waanders, B.G., Wojtkiewicz, S.F., Hart, W.E., Alleva, M.P.: DAKOTA, a multilevel parallel object-oriented framework for design optimization, parameter estimation, uncertainty quantification, and sensitivity analysis. Tech. rep., CiteSeer (2006)
61. Esfe, M.H., Hajmohammad, H., Moradi, R., Arani, A.A.A.: Multi-objective optimization of cost and thermal performance of double walled carbon nanotubes/water nanofluids by NSGA-II using response surface method. Appl. Thermal Eng. **112**, 1648–1657 (2017)

62. Esmin, A.A., Coelho, R.A., Matwin, S.: A review on particle swarm optimization algorithm and its variants to clustering high-dimensional data. Artif. Intell. Rev. **44**(1), 23–45 (2015)
63. Esmin, A.A., Lambert-Torres, G., De Souza, A.Z.: A hybrid particle swarm optimization applied to loss power minimization. IEEE Trans. Power Syst. **20**(2), 859–866 (2005)
64. Fahmi, I., Cremaschi, S.: Process synthesis of biodiesel production plant using artificial neural networks as the surrogate models. Comput. Chem. Eng. **46**, 105–123 (2012)
65. Farshi, M.M.: Improving genetic algorithms for optimum well placement. Ph.D. thesis, Stanford University Stanford, CA (2008)
66. Fasano, G., Liuzzi, G., Lucidi, S., Rinaldi, F.: A linesearch-based derivative-free approach for nonsmooth constrained optimization. SIAM J. Optim. **24**(3), 959–992 (2014)
67. Fernandes, F.A.: Optimization of Fischer–Tropsch synthesis using neural networks. Chem. Eng. Technol. Ind. Chem. Plant Equip. Process Eng. Biotechnol. **29**(4), 449–453 (2006)
68. First, E.L., Hasan, M.F., Floudas, C.A.: Discovery of novel zeolites for natural gas purification through combined material screening and process optimization. AIChE J. **60**(5), 1767–1785 (2014)
69. Fletcher, R., Gould, N.I., Leyffer, S., Toint, P.L., Wächter, A.: Global convergence of a trust-region SQP-filter algorithm for general nonlinear programming. SIAM J. Optim. **13**(3), 635–659 (2002)
70. Fletcher, R., Leyffer, S., Toint, P.L., et al.: A brief history of filter methods. Preprint ANL/MCS-P1372-0906, Argonne National Laboratory, Mathematics and Computer Science Division, vol. 36 (2006)
71. Foli, K., Okabe, T., Olhofer, M., Jin, Y., Sendhoff, B.: Optimization of micro heat exchanger: CFD, analytical approach and multi-objective evolutionary algorithms. Int. J. Heat Mass Transf. **49**(5–6), 1090–1099 (2006)
72. Foroud, T., Baradaran, A., Seifi, A.: A comparative evaluation of global search algorithms in black box optimization of oil production: a case study on Brugge field. J. Pet. Sci. Eng. **167**, 131–151 (2018)
73. Forrester, A.I., Bressloff, N.W., Keane, A.J.: Optimization using surrogate models and partially converged computational fluid dynamics simulations. Proc. R. Soc. A Math. Phys. Eng. Sci. **462**(2071), 2177–2204 (2006)
74. Gassner, M., Maréchal, F.: Methodology for the optimal thermo-economic, multi-objective design of thermochemical fuel production from biomass. Comput. Chem. Eng. **33**(3), 769–781 (2009)
75. Goldberg, D.E., Holland, J.H.: Genetic algorithms and machine learning (1988)
76. Golovin, D., Solnik, B., Moitra, S., Kochanski, G., Karro, J., Sculley, D.: Google Vizier: a service for black-box optimization. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1487–1495 (2017)
77. Gosselin, L., Tye-Gingras, M., Mathieu-Potvin, F.: Review of utilization of genetic algorithms in heat transfer problems. Int. J. Heat Mass Transf. **52**(9–10), 2169–2188 (2009)
78. Gramacy, R.B., Gray, G.A., Le Digabel, S., Lee, H.K., Ranjan, P., Wells, G., Wild, S.M.: Modeling an augmented Lagrangian for blackbox constrained optimization. Technometrics **58**(1), 1–11 (2016)
79. Gratton, S., Vicente, L.N.: A merit function approach for direct search. SIAM J. Optim. **24**(4), 1980–1998 (2014)
80. Gratton, S., Vicente, L.N.: A surrogate management framework using rigorous trust-region steps. Optim. Methods Softw. **29**(1), 10–23 (2014)
81. Griffin, J.D., Fowler, K.R., Gray, G.A., Hemker, T., Parno, M.D.: Derivative-free optimization via evolutionary algorithms guiding local search. Sandia National Laboratories, Albuquerque, NM, Tech. Rep. SAND2010-3023J (2010)
82. Griffin, J.D., Kolda, T.G.: Nonlinearly-constrained optimization using asynchronous parallel generating set search. Tech. rep., Sandia National Laboratories (2007)
83. Griffin, J.D., Kolda, T.G.: Asynchronous parallel hybrid optimization combining DIRECT and GSS. Optim. Methods Softw. **25**(5), 797–817 (2010)

84. Gross, B., Roosen, P.: Total process optimization in chemical engineering with evolutionary algorithms. Comput. Chem. Eng. **22**, S229–S236 (1998)
85. Güyagüler, B., Horne, R.N., Rogers, L., Rosenzweig, J.J., et al.: Optimization of well placement in a gulf of Mexico waterflooding project. SPE Reserv. Eval. Eng. **5**(03), 229–236 (2002)
86. Guyaguler, B., Horne, R.N., et al.: Uncertainty assessment of well placement optimization. In: SPE Annual Technical Conference and Exhibition. Society of Petroleum Engineers (2001)
87. Häglund, S.: A surrogate-based parameter tuning heuristic for Carmen crew optimizers. Master's thesis, Chalmers University of Technology (2010)
88. Hansen, N.: The CMA evolution strategy: a comparing review. In: Towards a New Evolutionary Computation, pp. 75–102. Springer, Berlin (2006)
89. Hart, W.E.: Evolutionary pattern search algorithms for unconstrained and linearly constrained optimization. IEEE Trans. Evol. Comput. **5**(4), 388–397 (2001)
90. Hart, W.E., Hunter, K.O.: A performance analysis of evolutionary pattern search with generalized mutation steps. In: Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406), vol. 1, pp. 672–679. IEEE, Piscataway (1999)
91. Hasan, M.F.: An edge-concave underestimator for the global optimization of twice-differentiable nonconvex problems. J. Global Optim. **71**(4), 735–752 (2018)
92. Hasan, M.F., Baliban, R.C., Elia, J.A., Floudas, C.A.: Modeling, simulation, and optimization of postcombustion CO_2 capture for variable feed concentration and flow rate. 2. pressure swing adsorption and vacuum swing adsorption processes. Ind. Eng. Chem. Res. **51**(48), 15665–15682 (2012)
93. Hasan, M.F., Boukouvala, F., First, E.L., Floudas, C.A.: Nationwide, regional, and statewide CO_2 capture, utilization, and sequestration supply chain network optimization. Ind. Eng. Chem. Res. **53**(18), 7489–7506 (2014)
94. Hasan, M.F., First, E.L., Floudas, C.A.: Cost-effective CO_2 capture based on in silico screening of zeolites and process optimization. Phys. Chem. Chem. Phys. **15**(40), 17601–17618 (2013)
95. Hemker, T., Werner, C.: Direct using local search on surrogates. Pac. J. Optim. **7**(3), 443–466 (2011)
96. Henao, C.A., Maravelias, C.T.: Surrogate-based superstructure optimization framework. AIChE J. **57**(5), 1216–1232 (2011)
97. Holland, J.H.: Adaptation in Natural and Artificial Systems. Society for Industrial and Applied Mathematics, Philadelphia (1976)
98. Holmström, K.: The TOMLAB optimization environment in Matlab (1999)
99. Holmstrom, K., Goran, A., Edvall, M.: User's guide for Tomlab 4.0. 6. Tomlab Optimization, Sweden (2003)
100. Humphries, T.D., Haynes, R.D.: Joint optimization of well placement and control for nonconventional well types. J. Pet. Sci. Eng. **126**, 242–253 (2015)
101. Hutter, F., Hoos, H.H., Leyton-Brown, K.: Automated configuration of mixed integer programming solvers. In: International Conference on Integration of Artificial Intelligence (AI) and Operations Research (OR) Techniques in Constraint Programming, pp. 186–202. Springer, Berlin (2010)
102. Huyer, W., Neumaier, A.: Global optimization by multilevel coordinate search. J. Global Optim. **14**(4), 331–355 (1999)
103. Huyer, W., Neumaier, A.: SNOBFIT—stable noisy optimization by branch and fit. ACM Trans. Math. Softw. **35**(2), 1–25 (2008)
104. Ingber, L.: Adaptive simulated annealing (ASA): lessons learned (2000). arXiv preprint cs/0001018
105. Ingber, L., et al.: Adaptive Simulated Annealing (ASA). Global Optimization C-code. Caltech Alumni Association, Pasadena (1993)
106. Isebor, O.J., Durlofsky, L.J., Ciaurri, D.E.: A derivative-free methodology with local and global search for the constrained joint optimization of well locations and controls. Comput. Geosci. **18**(3–4), 463–482 (2014)

107. Isebor, O.J., Echeverría Ciaurri, D., Durlofsky, L.J., et al.: Generalized field-development optimization with derivative-free procedures. SPE J. **19**(5), 891–908 (2014)
108. Iyer, S.S., Bajaj, I., Balasubramanian, P., Hasan, M.F.: Integrated carbon capture and conversion to produce syngas: novel process design, intensification, and optimization. Ind. Eng. Chem. Res. **56**(30), 8622–8648 (2017)
109. Jie, J., Zeng, J., Han, C.: Self-organization particle swarm optimization based on information feedback. In: International Conference on Natural Computation, pp. 913–922. Springer, Berlin (2006)
110. Jones, D.R.: Direct global optimization algorithm. Encyclopedia Optim. **1**(1), 431–440 (2009)
111. Jones, D.R., Perttunen, C.D., Stuckman, B.E.: Lipschitzian optimization without the Lipschitz constant. J. Optimization Theory Appl. **79**(1), 157–181 (1993)
112. Jung, I., Kshetrimayum, K.S., Park, S., Na, J., Lee, Y., An, J., Park, S., Lee, C.J., Han, C.: Computational fluid dynamics based optimal design of guiding channel geometry in u-type coolant layer manifold of large-scale microchannel Fischer–Tropsch reactor. Ind. Eng. Chem. Res. **55**(2), 505–515 (2016)
113. Kandasamy, K., Dasarathy, G., Oliva, J., Schneider, J., Poczos, B.: Multi-fidelity Gaussian process bandit optimisation. J. Artif. Intell. Res. **66**, 151–196 (2019)
114. Kandasamy, K., Dasarathy, G., Oliva, J.B., Schneider, J., Póczos, B.: Gaussian process bandit optimisation with multi-fidelity evaluations. In: Advances in Neural Information Processing Systems, pp. 992–1000 (2016)
115. Kandasamy, K., Dasarathy, G., Schneider, J., Póczos, B.: Multi-fidelity Bayesian optimisation with continuous approximations. In: Proceedings of the 34th International Conference on Machine Learning, vol. 70, pp. 1799–1808 (2017). JMLR.org
116. Kato, S., Lee, J.H.: Optimization of hybrid air-conditioning system with natural ventilation by GA and CFD. In: 25th AIVC Conference, Ventilation and Retrofitting (2004)
117. Kaufman, D.E., Smith, R.L.: Optimal direction choice for hit-and-run sampling. Tech. rep. (1991)
118. Kaufman, D.E., Smith, R.L.: Direction choice for accelerated convergence in hit-and-run sampling. Oper. Res. **46**(1), 84–95 (1998)
119. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: Proceedings of the IEEE International Conference on Neural Networks, vol. 4, pp. 1942–1948 (1995)
120. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983)
121. Kolda, T.G., Lewis, R.M., Torczon, V., et al.: A generating set direct search augmented Lagrangian algorithm for optimization with a combination of general and linear constraints, vol. 6. Sandia National Laboratories (2006)
122. Kramer, O.: A review of constraint-handling techniques for evolution strategies. Appl. Comput. Intell. Soft Comput. **2010**, 185063 (2010)
123. Krink, T., VesterstrOm, J.S., Riget, J.: Particle swarm optimisation with spatial particle extension. In: Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No. 02TH8600), vol. 2, pp. 1474–1479. IEEE, Piscataway (2002)
124. Lagarias, J.C., Reeds, J.A., Wright, M.H., Wright, P.E.: Convergence properties of the Nelder–Mead simplex method in low dimensions. SIAM J. Optim. **9**(1), 112–147 (1998)
125. Larson, J., Menickelly, M., Wild, S.M.: Derivative-free optimization methods. Acta Numer. **28**, 287–404 (2019)
126. Le Digabel, S.: Algorithm 909: NOMAD: nonlinear optimization with the MADS algorithm. ACM Trans. Math. Softw. **37**(4), 1–15 (2011)
127. Leardi, R.: Genetic algorithms in chemometrics and chemistry: a review. J. Chemometrics **15**(7), 559–569 (2001)
128. Lehnhäuser, T., Schäfer, M.: A numerical approach for shape optimization of fluid flow domains. Comput. Methods Appl. Mech. Eng. **194**(50–52), 5221–5241 (2005)
129. Lewin, D.R., Wang, H., Shalev, O.: A generalized method for HEN synthesis using stochastic optimization—I. General framework and MER optimal synthesis. Comput. Chem. Eng. **22**(10), 1503–1513 (1998)

130. Lewis, R.M., Torczon, V.: A globally convergent augmented Lagrangian pattern search algorithm for optimization with general constraints and simple bounds. SIAM J. Optim. **12**(4), 1075–1089 (2002)
131. Lewis, R.M., Torczon, V.: A direct search approach to nonlinear programming problems using an augmented Lagrangian method with explicit treatment of linear constraints. Technical Report of the College of William and Mary pp. 1–25 (2010)
132. Li, H.Q., Li, L.: A novel hybrid particle swarm optimization algorithm combined with harmony search for high dimensional optimization problems. In: The 2007 International Conference on Intelligent Pervasive Computing (IPC 2007), pp. 94–97. IEEE, Piscataway (2007)
133. Litvak, M.L., Gane, B.R., Williams, G., Mansfield, M., Angert, P.F., Macdonald, C.J., McMurray, L.S., Skinner, R.C., Walker, G.J., et al.: Field development optimization technology. In: SPE Reservoir Simulation Symposium. Society of Petroleum Engineers, Richardson (2007)
134. Liu, J., Ploskas, N., Sahinidis, N.V.: Tuning BARON using derivative-free optimization algorithms. J. Global Optim. **74**(4), 611–637 (2019)
135. Liu, T., First, E.L., Hasan, M.F., Floudas, C.A.: A multi-scale approach for the discovery of zeolites for hydrogen sulfide removal. Comput. Chem. Eng. **91**, 206–218 (2016)
136. Liuzzi, G., Lucidi, S., Piccialli, V.: Exploiting derivative-free local searches in direct-type algorithms for global optimization. Comput. Optim. Appl. **65**(2), 449–475 (2016)
137. Liuzzi, G., Lucidi, S., Sciandrone, M.: Sequential penalty derivative-free methods for nonlinear constrained optimization. SIAM J. Optim. **20**(5), 2614–2635 (2010)
138. Macek, K., Rojicek, J., Kontes, G., Rovas, D.: Black-box optimisation for buildings and its enhancement by advanced communication infrastructure. Adv. Distrib. Comput. Artif. Intell. J. **2013**, 53–64 (2013)
139. Malkawi, A.M., Srinivasan, R.S., Yi, Y.K., Choudhary, R.: Performance-based design evolution: the use of genetic algorithms and CFD. In: Eighth International IBPSA. Eindhoven, Netherlands pp. 793–798 (2003)
140. Malkawi, A.M., Srinivasan, R.S., Yun, K.Y., Choudhary, R.: Decision support and design evolution: integrating genetic algorithms, CFD and visualization. Autom. Constr. **14**(1), 33–44 (2005)
141. Marinakis, Y., Marinaki, M., Matsatsinis, N.: A hybrid bumble bees mating optimization-grasp algorithm for clustering. In: International Conference on Hybrid Artificial Intelligence Systems, pp. 549–556. Springer, Berlin (2009)
142. Marsden, A.L., Feinstein, J.A., Taylor, C.A.: A computational framework for derivative-free optimization of cardiovascular geometries. Comput. Methods Appl. Mech. Eng. **197**(21–24), 1890–1905 (2008)
143. Marsden, A.L., Wang, M., Dennis, J., Moin, P.: Trailing-edge noise reduction using derivative-free optimization and large-eddy simulation. J. Fluid Mech. **572**, 13–36 (2007)
144. Marsden, A.L., Wang, M., Dennis, J.E., Moin, P.: Optimal aeroacoustic shape design using the surrogate management framework. Optim. Eng. **5**(2), 235–262 (2004)
145. Marsden, A.L., Wang, M., Dennis Jr, J.E., Moin, P.: Suppression of vortex-shedding noise via derivative-free shape optimization. Phys. fluids **16**(10), L83–L86 (2004)
146. Martelli, E., Amaldi, E.: PGS-COM: a hybrid method for constrained non-smooth black-box optimization problems: brief review, novel algorithm and comparative evaluation. Comput. Chem. Eng. **63**, 108–139 (2014)
147. Martelli, E., Amaldi, E., Consonni, S.: Numerical optimization of heat recovery steam cycles: mathematical model, two-stage algorithm and applications. Comput. Chem. Eng. **35**(12), 2799–2823 (2011)
148. McKinnon, K.I.: Convergence of the Nelder–Mead simplex method to a nonstationary point. SIAM J. Optim. **9**(1), 148–158 (1998)
149. Meissner, M., Schmuker, M., Schneider, G.: Optimized particle swarm optimization (OPSO) and its application to artificial neural network training. BMC Bioinform. **7**(1), 125 (2006)

150. Metropolis, N., Rosenbluth, A.W., Rosenbluth, M.N., Teller, A.H., Teller, E.: Equation of state calculations by fast computing machines. J. Chem. Phys. **21**(6), 1087–1092 (1953)
151. Mezura-Montes, E., Coello, C.A.C.: Constraint-handling in nature-inspired numerical optimization: past, present and future. Swarm Evol. Comput. **1**(4), 173–194 (2011)
152. Mezura-Montes, E., Flores-Mendoza, J.I.: Improved particle swarm optimization in constrained numerical search spaces. In: Nature-Inspired Algorithms for Optimisation, pp. 299–332. Springer, Berlin (2009)
153. Michalewicz, Z., Schoenauer, M.: Evolutionary algorithms for constrained parameter optimization problems. Evol. Comput. **4**(1), 1–32 (1996)
154. Moles, C.G., Mendes, P., Banga, J.R.: Parameter estimation in biochemical pathways: a comparison of global optimization methods. Genome Res. **13**(11), 2467–2474 (2003)
155. Morin, A., Wahl, P.E., Mølnvik, M.: Using evolutionary search to optimise the energy consumption for natural gas liquefaction. Chem. Eng. Res. Design **89**(11), 2428–2441 (2011)
156. Na, J., Kshetrimayum, K.S., Lee, U., Han, C.: Multi-objective optimization of microchannel reactor for Fischer–Tropsch synthesis using computational fluid dynamics and genetic algorithm. Chem. Eng. J. **313**, 1521–1534 (2017)
157. Nascimento, C.A.O., Giudici, R., Guardani, R.: Neural network based approach for optimization of industrial chemical processes. Comput. Chem. Eng. **24**(9–10), 2303–2314 (2000)
158. Nelder, J.A., Mead, R.: A simplex method for function minimization. Comput. J. **7**(4), 308–313 (1965)
159. Onwunalu, J.E., Durlofsky, L.J.: Application of a particle swarm optimization algorithm for determining optimum well location and type. Comput. Geosci. **14**(1), 183–198 (2010)
160. Pariyani, A., Gupta, A., Ghosh, P.: Design of heat exchanger networks using randomized algorithm. Comput. Chem. Eng. **30**(6–7), 1046–1053 (2006)
161. Park, S., Na, J., Kim, M., Lee, J.M.: Multi-objective Bayesian optimization of chemical reactor design using computational fluid dynamics. Comput. Chem. Eng. **119**, 25–37 (2018)
162. Payne, J.L., Eppstein, M.J.: A hybrid genetic algorithm with pattern search for finding heavy atoms in protein crystals. In: Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation, pp. 377–384 (2005)
163. Plantenga, T.D.: Hopspack 2.0 user manual. Sandia National Laboratories Technical Report Sandia National Laboratories Technical Report SAND2009-6265 (2009)
164. Ploskas, N., Laughman, C., Raghunathan, A.U., Sahinidis, N.V.: Optimization of circuitry arrangements for heat exchangers using derivative-free optimization. Chem. Eng. Res. Design **131**, 16–28 (2018)
165. Pourfattah, F., Sabzpooshani, M., Bayer, Ö., Toghraie, D., Asadi, A.: On the optimization of a vertical twisted tape arrangement in a channel subjected to MWCNT–water nanofluid by coupling numerical simulation and genetic algorithm. J. Thermal Anal. Calorim. (2020). https://doi.org/10.1007/s10973-020-09490-5
166. Powell, M.: On the Lagrange functions of quadratic models that are defined by interpolation. Optim. Methods Softw. **16**(1–4), 289–309 (2001)
167. Powell, M.J.: A direct search optimization method that models the objective and constraint functions by linear interpolation. In: Advances in Optimization and Numerical Analysis, pp. 51–67. Springer, Berlin (1994)
168. Powell, M.J.: UOBYQA: unconstrained optimization by quadratic approximation. Math. Program. **92**(3), 555–582 (2002)
169. Powell, M.J.: The NEWUOA software for unconstrained optimization without derivatives. In: Large-Scale Nonlinear Optimization, pp. 255–297. Springer, Berlin (2006)
170. Powell, M.J.: Developments of NEWUOA for minimization without derivatives. IMA J. Numer. Anal. **28**(4), 649–664 (2008)
171. Powell, M.J.: The BOBYQA algorithm for bound constrained optimization without derivatives. Cambridge NA Report NA2009/06, University of Cambridge, Cambridge pp. 26–46 (2009)
172. Powell, M.J.: On fast trust region methods for quadratic models with linear constraints. Math. Program. Comput. **7**(3), 237–267 (2015)

173. Ragonneau, T.M., Zhang, Z.: PDFO: cross-platform interfaces for Powell's derivative-free optimization solvers (2020). https://www.pdfo.net/
174. Raidl, G.R.: A unified view on hybrid metaheuristics. In: International Workshop on Hybrid Metaheuristics, pp. 1–12. Springer, Berlin (2006)
175. Ravagnani, M., Silva, A., Arroyo, P., Constantino, A.: Heat exchanger network synthesis and optimisation using genetic algorithm. Appl. Thermal Eng. **25**(7), 1003–1017 (2005)
176. Regis, R.G.: Stochastic radial basis function algorithms for large-scale optimization involving expensive black-box objective and constraint functions. Comput. Oper. Res. **38**(5), 837–853 (2011)
177. Regis, R.G.: Evolutionary programming for high-dimensional constrained expensive black-box optimization using radial basis functions. IEEE Trans. Evol. Comput. **18**(3), 326–347 (2013)
178. Regis, R.G.: Constrained optimization by radial basis function interpolation for high-dimensional expensive black-box problems with infeasible initial points. Eng. Optim. **46**(2), 218–243 (2014)
179. Rodriguez-Fernandez, M., Egea, J.A., Banga, J.R.: Novel metaheuristic for parameter estimation in nonlinear dynamic biological systems. BMC Bioinform. **7**(1), 483 (2006)
180. Rößger, P., Richter, A.: Performance of different optimization concepts for reactive flow systems based on combined CFD and response surface methods. Comput. Chem. Eng. **108**, 232–239 (2018)
181. Safikhani, H., Abbassi, A., Khalkhali, A., Kalteh, M.: Multi-objective optimization of nanofluid flow in flat tubes using CFD, artificial neural networks and genetic algorithms. Adv. Powder Technol. **25**(5), 1608–1617 (2014)
182. Sen, R., Kandasamy, K., Shakkottai, S.: Multi-fidelity black-box optimization with hierarchical partitions. In: International Conference on Machine Learning, pp. 4538–4547 (2018)
183. Siavashi, M., Doranehgard, M.H.: Particle swarm optimization of thermal enhanced oil recovery from oilfields with temperature control. Appl. Thermal Eng. **123**, 658–669 (2017)
184. Silva, A., Neves, A., Costa, E.: Chasing the swarm: a predator prey approach to function optimisation. InL Proceedings of Mendal, pp. 5–7 (2002)
185. Sivanandam, S., Deepa, S.: Genetic algorithms. In: Introduction to Genetic Algorithms, pp. 15–37. Springer, Berlin (2008)
186. Smith, R.L.: Monte Carlo procedures for generating random feasible solutions to mathematical programs. In: A Bulletin of the ORSA/TIMS Joint National Meeting, Washington, DC, vol. 101 (1980)
187. Smith, R.L.: Efficient Monte Carlo procedures for generating points uniformly distributed over bounded regions. Oper. Res. **32**(6), 1296–1308 (1984)
188. Srivastava, R., Rawlings, J.B.: Parameter estimation in stochastic chemical kinetic models using derivative free optimization and bootstrapping. Comput. Chem. Eng. **63**, 152–158 (2014)
189. Talbi, E.G.: A taxonomy of hybrid metaheuristics. J. Heuristics **8**(5), 541–564 (2002)
190. Torczon, V.: On the convergence of pattern search algorithms. SIAM J. Optim. **7**(1), 1–25 (1997)
191. Tran, A., Sun, J., Furlan, J.M., Pagalthivarthi, K.V., Visintainer, R.J., Wang, Y.: pBO-2GP-3B: a batch parallel known/unknown constrained Bayesian optimization with feasibility classification and its applications in computational fluid dynamics. Comput. Methods Appl. Mech. Eng. **347**, 827–852 (2019)
192. Tseng, P.: Fortified-descent simplicial search method: a general approach. SIAM J. Optim. **10**(1), 269–288 (1999)
193. Uebel, K., Rößger, P., Prüfert, U., Richter, A., Meyer, B.: CFD-based multi-objective optimization of a quench reactor design. Fuel Proces. Technol. **149**, 290–304 (2016)
194. Vaz, A.I.F., Vicente, L.N.: A particle swarm pattern search method for bound constrained global optimization. J. Global Optim. **39**(2), 197–219 (2007)
195. Vaz, A.I.F., Vicente, L.N.: PSwarm: a hybrid solver for linearly constrained global derivative-free optimization. Optim. Methods Softw. **24**(4–5), 669–685 (2009)

196. Vicente, L.N., Custódio, A.: Analysis of direct searches for discontinuous functions. Math. Program. **133**(1–2), 299–325 (2012)
197. Wachowiak, M.P., Peters, T.M.: Parallel optimization approaches for medical image registration. In: International Conference on Medical Image Computing and Computer-Assisted Intervention, pp. 781–788. Springer, Berlin (2004)
198. Wachowiak, M.P., Peters, T.M.: Combining global and local parallel optimization for medical image registration. In: Medical Imaging 2005: Image Processing, vol. 5747, pp. 1189–1200. International Society for Optics and Photonics (2005)
199. Wan, X., Pekny, J.F., Reklaitis, G.V.: Simulation-based optimization with surrogate models—application to supply chain management. Comput. Chem. Eng. **29**(6), 1317–1328 (2005)
200. Weile, D.S., Michielssen, E.: Genetic algorithm optimization applied to electromagnetics: a review. IEEE Trans. Antennas Propag. **45**(3), 343–353 (1997)
201. Woods, D.J.: An interactive approach for solving multi-objective optimization problems. Tech. rep. (1985)
202. Wright, J., Zhang, Y., Angelov, P., Hanby, V., Buswell, R.: Evolutionary synthesis of HVAC system configurations: algorithm development (RP-1049). HVAC&R Res. **14**(1), 33–55 (2008)
203. Wright, J.A., Loosemore, H.A., Farmani, R.: Optimization of building thermal design and control by multi-criterion genetic algorithm. Energy Build. **34**(9), 959–972 (2002)
204. Yehui, P., Zhenhai, L.: A derivative-free algorithm for unconstrained optimization. Appl. Math. A J. Chin. Univ. **20**(4), 491–498 (2005)
205. Yeten, B., Durlofsky, L., Aziz, K.: Optimization of nonconventional well type. Location Trajectory, SPE **77565**, 14 (2002)
206. Zhang, Y., Wright, J.A., Hanby, V.I.: Energy aspects of HVAC system configurations—problem definition and test cases. HVAC&R Res. **12**(S3), 871–888 (2006)
207. Zhao, B., Guo, C., Cao, Y.: A multiagent-based particle swarm optimization approach for optimal reactive power dispatch. IEEE Trans. Power Syst. **20**(2), 1070–1078 (2005)
208. Zhou, Z., Ong, Y.S., Nair, P.B., Keane, A.J., Lum, K.Y.: Combining global and local surrogate models to accelerate evolutionary optimization. IEEE Trans. Syst. Man Cybern. C (Appl. Rev.) **37**(1), 66–76 (2006)

Tuning Algorithms for Stochastic Black-Box Optimization: State of the Art and Future Perspectives



Thomas Bartz-Beielstein, Frederik Rehbach, and Margarita Rebolledo

1 Introduction

Stochastic optimization algorithms such as Evolutionary algorithms (EAs) or Particle swarm optimization (PSO) are popular solvers for real-world optimization problems. In many cases, these real-world problems belong to the class of black-box optimization problems. Because the performance of an optimization algorithm depends on its parameterization, tuning algorithms were introduced. To avoid confusion, we will use the term *tuning algorithm* for the tuning procedures and the term *optimization algorithm* for the underlying stochastic optimization algorithm, e.g., PSO. The term *environment* (or framework) will be used for a method (or a process) and a suite of tools supporting that method.

Every optimization algorithm requires a set of parameters to be specified in order to obtain an executable version. These parameters will be referred to as *algorithm parameters*. Algorithm parameters have a large influence on the performance of the algorithm in a given problem. Hence, the choice of the algorithm parameter values can be understood as an optimization problem in itself. According to a selected performance metric, the best algorithm parameter set is searched for in order to obtain an optimal solution.

Internal tuning, also known as parameter control, refers to the adaptation of the algorithm parameter values *during* the optimization run [46]. Internal tuning requires the determination of the method and triggers used to define when and where the change of value will happen. It can be implemented in two different ways:

T. Bartz-Beielstein (✉) · F. Rehbach · M. Rebolledo
TH Köln, Institute for Data Science, Engineering, and Analytics, Köln, Germany
e-mail: thomas.bartz-beielstein@th-koeln.de; frederik.rehbach@th-koeln.de; margarita.rebolledo@th-koeln.de

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_3

- Deterministically, i.e., the parameter is modified according to some fixed rule that does not consider the current state of the algorithm. Reducing the mutation rate in Evolution strategies (ESs) every ten generation is one simple example.
- Adaptively, i.e., feedback information from the algorithm is used to decide when the change should be made. The 1/5th rule, which modifies the step size based on the success rate of the ES, is a famous example of self-adaptive parameter control [100].

For the remainder of this chapter, we will use the term "internal tuning" when referring to parameter control, i.e., parameter changes during the optimization run.

Parameter tuning is an approach where the algorithm parameter values are specified *before* the algorithm is run on the actual problem instance. The corresponding parameters will be referred to as *external* algorithm parameters. Note that the *initial* values of internal algorithm parameters, e.g., the initial step size in ESs, are external algorithm parameters.

There are different techniques to solve the parameter tuning problem. In the most simple case, the values of the parameters can be selected following configurations that had shown good performance on similar problems. A more structured but also more complex approach to parameter tuning is achieved by implementing specialized tuning algorithms. These tuning algorithms usually take into account the interaction effects between the different parameters and the impact that their values have on the evaluation metric. One drawback of parameter tuning is that it ignores the algorithm dynamic and adaptive process.

Although tuning focuses on the external algorithm parameters only, it is considered important. Algorithm tuning can be used to

- avoid wrong parameter settings (this is the main goal from our perspective),
- improve the existing algorithms, i.e., to obtain an algorithm instance with high performance, given a performance measure or a combination of measures (on one or more problems),
- help select the best algorithm for working with a real-world problem,
- show the value of a novel algorithm, when compared to a more classical method,
- compare a new version of optimization software with earlier releases,
- evaluate the performance of an optimization algorithm when different parameter settings are used, and
- obtain an algorithm instance that is robust to changes in problem specification or random effects during execution.

Tuning is based on the experimental analysis of algorithms. De Jong's [37] Ph.D. thesis is a milestone in the experimental analysis of Genetic algorithms (GAs). He evaluated several combinations of GA parameters on a set of five test functions: parabola (sphere), Rosenbrock (banana), step function, noise function, and Shekel. The recommended parameter settings from this study have been adopted by many researchers. Since they have been used so often, they are considered as "standard settings" for algorithm tuning and comparison [57].

Because the analysis of stochastic optimization algorithms is very complex, some researchers focused their analysis and tuning experiments on a limited set of parameters only. In these so-called facet wise studies, interaction effects were ignored and main effects were analyzed only [81]. For example, Goldberg and Deb [50] analyzed selection, Mühlenbein [91] and Bäck [8] investigated mutation, and Goldberg et al. [51] considered population sizes in GAs. Much existing work deals with relatively small numbers of numerical (often continuous) parameters; see, e.g., [6, 33]. Good examples of systematic studies, which consider several parameters, are presented in [12, 69–71, 83].

Benchmarking is related to tuning because both procedures use similar components, e.g., test functions and performance metrics. While tuning is usually applied to one algorithm, benchmarking requires at least two algorithms. We claim that benchmarking is not feasible, if the algorithms are not tuned. As this is not always practical, it is important to emphasize that tuning parameters can have a major impact on the performance of an algorithm; therefore, it is not appropriate to tune the parameters of some methods while leaving other methods at their default settings. An accurate approach for benchmarking of optimization algorithms requires a pre-processing step in which a tuning method is employed to find the suitable parameter settings for all the optimization algorithms [56]. Beiranvand et al. [19] systematically review the benchmarking process of optimization algorithms and discuss the challenges of fair comparison. They provide suggestions for the comparison process and highlight the pitfalls to avoid when evaluating the performance of optimization algorithms.

Selecting the most appropriate algorithm out of a set of available algorithms when attempting to solve black-box optimization problems is a related task because algorithm selection requires expert knowledge of search algorithm efficacy and skills in algorithm engineering and statistics. Muñoz et al. [92] present a survey of methods for algorithm selection in the black-box continuous optimization domain.

During the last years, several tuning software packages for stochastic optimization algorithms were developed. Well-established software tools are IRACE, Iterated local search in parameter configuration space (ParamILS), GGA, and Sequential parameter optimization toolbox (SPOT) (and its derivates such as SMAC). Several tuners, e.g., BONESA [111] or OPAL [7], were developed but are not maintained anymore.

This chapter presents a comprehensible introduction to tuning and gives hands-on advice on how to select a suitable tuner for one specific algorithm–problem combination. It is structured as follows. Section 2 discusses strategical issues and defines eight key topics for tuning, namely, optimization algorithms, test problems, experimental setup, performance metrics, reporting, parallelization, tuning methods, and software. Test functions, which play a central role in tuning, are considered in Sect. 3. Statistical considerations as well as performance metrics are discussed in Sect. 4. Since runtime is an important issue, Sect. 5 discusses parallelization. Tuning algorithms are discussed in Sect. 6. Available software is presented in Sect. 7. Section 8 identifies key challenges and proposes future research directions. Section 9 presents a summary and an outlook.

2 Tuning: Strategies

2.1 Key Topics

Before tuning is performed, the selection of a tuning strategy is highly recommended because tuning can be very time-consuming. A tuning strategy should consider the following key elements.

(K-1) *Optimization algorithms* with their corresponding algorithm parameters. For example, GAs require the specification of a population size, say N. Basically, there are two types of algorithm parameters: numerical (e.g., real-valued) and categorical (e.g., integer-valued). Optimization algorithms will be discussed in Sect. 2.2.

(K-2) *Optimization problems* require the specification of associated optimization parameters such as problem dimension d. An example is shown in Table 2. One specific realization of a test problem will be referred to as a problem instance. Real-world application instances, as well as artificial test functions, are used. Optimization problems will be discussed in Sect. 3.

(K-3) *Experimental setup*, e.g., number of repeats. Since we consider stochastic optimization algorithms, we must run them multiple times to establish the statistical distribution of the solution as function of the design parameters to demonstrate their effectiveness. This topic will be discussed in Sect. 4.2.

(K-4) *Performance metrics* describe a dimension of the algorithm performance such as solution quality and time or space usage. They will be discussed in Sect. 4.3.

(K-5) *Reporting and Visualization*. Besides tabular output, graphical tools for comparison are useful. Tools from Exploratory data analysis (EDA) are highly recommended [12, 17, 113]. This topic will be discussed in Sect. 4.4.

(K-6) *Parallelization*. Due to the often very high evaluation times in tuning processes and the increasing level of parallelization in modern computing clusters, serially running tuners lack efficiency. Parallelization will be discussed in Sect. 5.

(K-7) *Tuning algorithms* (tuners) are applied to the optimization algorithm. They can be classified in several manners. For example, Eiben and Smit [45] used the search effort perspective and assigned tuning methods to four categories: sampling methods, model-based methods, screening methods, and meta-evolutionary algorithms. Model-based tuning approaches differ in whether or not explicit models (the so-called response surfaces) are used. We will consider the interface perspective, i.e., we will consider manual, automatic, and interactive tuning algorithms in Sect. 6.

(K-8) Tuning software will be discussed in Sect. 7.

McGeoch has published a series of papers that discuss these key elements. She surveys issues arising in the design, development, and execution of computational experiments to study algorithms [84]. An algorithm is viewed here as an abstract

model of an implemented program: experiments are performed to study the model, and new insights about the model can be applied to predict program performance. Issues related to choosing performance metrics, planning experiments, developing software tools, running tests, and analyzing data are considered. McGeoch [85] discusses theoretical questions and motivations, combined with empirical research methods, in order to produce insights into algorithm performance. McGeoch [86] covers several important topics in experimental research and algorithm tuning.

In a highly recommendable paper, Rardin and Uzsoy [99] focus on the methodological issues that must be confronted by researchers undertaking experimental evaluations of algorithms, including experimental design, sources of test instances, performance metrics, analysis, pitfalls to avoid, and presentation of results. Bartz-Beielstein et al. [17] discuss methodological contributions on different scenarios of experimental analysis, e.g., important issues in the experimental analysis of algorithms. They discuss the experimental cycle of algorithm development, algorithm performance in terms of solution quality, runtime, and other measures and collect advanced methods from experimental design for configuring and tuning algorithms on a specific class of instances with the goal of using the least amount of experimentation. Eiben and Smit [45] present a conceptual framework for parameter tuning, provide a survey of tuning methods, and discuss related methodological issues. The authors establish a taxonomy to categorize tuning methods.

Tuning approaches can be categorized as sampling methods, model-based methods, screening methods, and meta-evolutionary algorithms [45]. Model-based tuning approaches, which are considered the most efficient method, differ in whether or not explicit models (the so-called response surfaces) are used to describe the dependence of target algorithm performance on parameter settings. There has been a substantial amount of work on both model-free and model-based approaches. Some notable model-free approaches include F-Race by Birattari et al. [21] and ParamILS by Hutter et al. [64]. State-of-the-art model-based approaches use Gaussian stochastic processes (also known as Kriging models) to fit a response surface model. Design and analysis of computer experiments (DACE) is one particularly popular and widely studied version of Gaussian stochastic process models [103]. Combining such a predictive model with sequential decisions about the most promising next design point (often based on a so-called Expected improvement (EI) criterion) gives rise to a sequential optimization approach. An influential contribution in this field was the Efficient global optimization (EGO) procedure by Jones et al. [72], which addressed the optimization of deterministic black-box functions. In the context of parameter optimization, EGO could be used to optimize deterministic algorithms with continuous parameters on a single problem instance. Two independent lines of work extended EGO to noisy functions, which in the context of parameter optimization, allow the consideration of randomized algorithms: the Sequential parameter optimization (SPO) procedure by Bartz-Beielstein et al. [15] and the Sequential kriging optimization (SKO) algorithm by Huang et al. [62].

2.2 Stochastic Optimization Algorithms

In the remainder of this chapter, we will focus on non-deterministic (stochastic) algorithms, e.g., EAs or PSO. Kennedy and Eberhart [74] describe PSO. Schwefel [109] is the reference work on ES. Eiben and Smith [44] present a nice introduction to Evolutionary computing (EC). Bartz-Beielstein et al. [18] present a comprehensive introduction to EAs. The current state of EAs is described in [110]. Jin et al. [68] present an overview of data-driven evolutionary optimization and provide a taxonomy of different data-driven evolutionary optimization problems.

Definition 1 (Black-Box Optimization Problem) Let $x \in \mathrm{U}$, $f : \mathrm{U} \to \mathrm{V}$, and find

$$\min_x f(x),$$

where we can only evaluate $f(x)$ for any $x \in \mathbb{U}$.

Our discussion includes stochastic (noisy) functions, i.e.,

$$y = f(x) + \epsilon,$$

where ϵ denotes some random variable.

Definition 2 (Iterative Optimization Algorithm) Let $\mathbb{S}$ denote the set of internal (strategy) parameters and $\mathbb{P}$ the set of external (algorithm) parameters. An iterative optimization algorithm A uses (besides initialization methods and termination criteria) in every iteration i the update rule

$$a : (x_i, \sigma_i, \theta) \mapsto (x_{i+1}, \sigma_{i+1})$$

with $x_i \in \mathbb{U}$, $\sigma_i \in \mathbb{S}$, and $\theta \in \mathbb{P}$ to generate new solutions.

We included internal control parameters σ_i, which may be modified during an iteration, in Definition 2 to make the difference between internal tuning and parameter tuning explicit [45]. Internal tuning handles the internal algorithm parameters $\sigma_i \in \mathbb{S}$, whereas parameter tuning focuses on the external parameters $\theta \in \mathbb{P}$, which do not depend on the iteration i.

Definition 3 (Stochastic Optimization Algorithm) Stochastic optimization algorithms are iterative optimization methods that use random variables. Stochastic optimization methods include methods with random iterates. They are applicable to deterministic and stochastic problems.

Example 1 ESs [108] can be considered as stochastic optimization algorithms because the mutation rate σ_i can be implemented as a random variable.

2.3 Algorithm Tuning

Algorithm tuning can be formulated as follows.

Definition 4 (Simple Tuning Problem) Find an algorithm configuration $\theta^* \in \mathbb{P}$, such that the performance of the optimization algorithm is optimized for a given black-box optimization problem $f : \mathbb{U} \to \mathbb{V}$ according to some performance metric $Q_f : \mathbb{P} \to \mathbb{K}$, i.e., let $\theta \in \mathbb{P}$, $t_f : \mathbb{P} \to \mathbb{P}$, and find

$$\theta^* = \min_{\theta \in \mathbb{P}} Q_f(\theta).$$

Usually, $\mathbb{K} = \mathbb{R}$ or $\mathbb{N}$. For example, the performance metric (quality function) Q_f can determine the best-ever function value, where f denotes the Rosenbrock function.

Note, Definition 4 is based on one tuning problem and considers one optimization algorithm and one optimization problem. More complex scenarios are possible, e.g., tuning one algorithm on several problem instances. These scenarios are described in [13, 31]. Figure 1 illustrates the simple tuning problem.

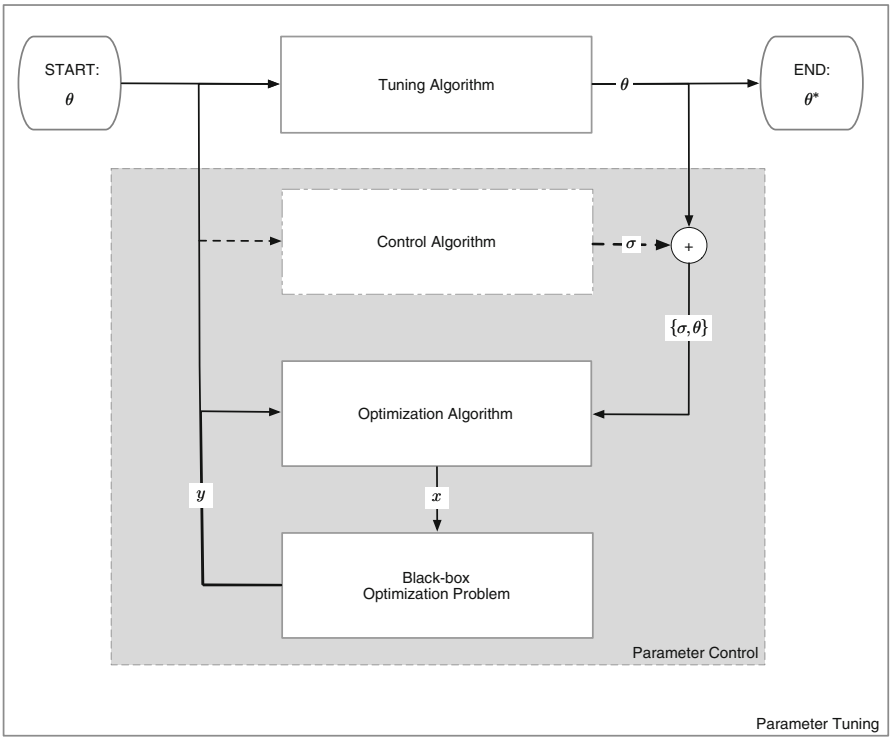


Fig. 1 Schematic illustration of internal tuning (*gray shaded box*) and parameter tuning components and symbols used in this paper. The vector of algorithm parameters is denoted by θ

In principle, optimization algorithms themselves can serve as tuning algorithms. Tuning algorithms are specialized optimization algorithms that implement certain components, which simplify the tuning process and provide tools for analyzing results.

2.4 Example: Grefenstette's Study of Control Parameters for Genetic Algorithms

To illustrate these key points of algorithm tuning from Sect. 2.1, we will use Grefenstette's [53] seminal study as an illustrative example.

Example 2 Grefenstette [53] attempts to determine the optimal control parameters for GAs. He uses a metalevel GA to search the parameterized space of GAs. This setup results in a two-level adaptive system model, which is the cornerstone of many tuning approaches. He described experiments, which were designed to search the space of GAs defined by six algorithm parameters, namely, population size, crossover rate, mutation rate, generation gap, scaling window, and selection strategy. A GA was used as a *metalevel* GA for tuning, i.e., tuning and optimization algorithms are similar. He mentions that "metalevel GA could similarly search any other space of parameterized optimization procedures." The parameterization of these six parameters, i.e., $\theta = (N, C, M, G, W, S)$, is shown in Table 1.

Grefenstette used a combination of numerical and categorical values. The numerical values were discretized, i.e., instead of analyzing generation gap values from the infinite interval [0.3; 1.0], the finite set $\{0.3, 0.4, 0.5, \ldots, 0.9, 1.0\}$ was used. The Cartesian product of these settings results in $2^{18} = 262{,}144$ combinations. The metalevel GA, which was used as the tuning algorithm, optimized an 18-bit vector. Each 18-bit vector represented one GA algorithm parameter configuration. In order to compare the GA configurations, five optimization functions with different characteristics were chosen. His test set, which is summarized in Table 2, included discontinuous, multidimensional, and noisy functions.

To measure the performance of one algorithm configuration, Grefenstette uses two metrics, which were introduced by De Jong [37]. De Jong used online and offline performance measures: *online performance* considers the performance of

Table 1 Algorithm parameters. There are 2^{18} combinations

Name	Symbol	Range	Increment	Type
Population size	N	10–160	10	Real
Crossover rate	C	0.25–1	0.05	Real
Mutation rate	M	0–1	Eight values, exponentially increasing	Real
Generation gap	G	0.3–1.0	0.1	Real
Scaling window	W	Three modes		Categorical
Selection strategy	S	Two strategies		Categorical

Table 2 Function set from [53]. There are five optimization problems

Name	Symbol	Dimension d
Parabola (sphere)	f_1	3
Rosenbrock	f_2	2
Step	f_3	5
Quartic with noise	f_4	30
Shekel's foxholes	f_5	2

the algorithm at a certain time step t, whereas *offline performance* considers the best performance achieved in the time interval $[0, t]$. Grefenstette [53] considers these measures to analyze GAs. In addition to the GA runs, a random search is performed. This allows normalization of the performance results by calculating the quotient of the performance of the GA runs and random search on the same problem set. A quotient smaller than one indicates that the GA performs better than random search.

Grefenstette [53] uses the metalevel GA to evaluate online and offline performances separately. These experiments were conducted as follows:

Step 1 (a) Each GA was run with a budget of $n_p = 5000$ function evaluations on the five test functions f_1 to f_5 from Table 1.
(b) The metalevel GA (tuning algorithm) used a budget of $n_t = 1000$ algorithm runs. It started with a default configuration and a population size $N = 50$.

Step 2 (a) Each of the best 20 GAs from the experiments from Step 1 was evaluated five times on the test functions using different random number seeds.
(b) The GA with the best performance in the last step was the winner.

Plots of the average online performance versus the number of generations were used to visualize the results. Based on the experimental results, [53] draws conclusions, e.g., "The higher mutation rate also helps prevent premature convergence to local optima." He compared his results to results from previous studies, e.g., to results from [37]. He concluded that "mutation rates above 0.05 are generally harmful with respect to online performance. . . ." Furthermore, he discussed interactions between algorithm parameters, e.g., in "small populations [. . .] good online performance is associated with either a high crossover rate combined with a low mutation rate or a low crossover rate combined with a high mutation rate." However, no additional experiments to validate these conclusions were performed.

Finally, he performed an additional experiment using an optimization problem that was not included in the experimental tuning environment. Five runs of the tuned algorithm on an image processing problem were performed. These additional experiments were conducted to demonstrate that tuning results can be generalized to other optimization problems or even other optimization domains.

2.5 No Free Lunch Theorems

An interesting implication arises from the No free lunch (NFL) theorems [115], which can be summarized as follows: *tuning an optimization algorithm for one class of problem is likely to make it perform more poorly for other problems.* Or, interpreted otherwise: *unless adequate restrictions are placed on the class of problems one is attempting to solve, there is no single algorithm that will outperform all other algorithms.*

For a recent analysis of the NFLs theorems, the reader is referred to [1], who discuss main implications as well as restrictions. De Jong [38] points out two frequently encountered misconceptions about NFL results:

> The first misconception is that NFL results are often interpreted as applying only to algorithms that do not self-adapt during a problem-solving run. So, for example, I might imagine a two-stage algorithm that begins by first figuring out what kind of problem it is being asked to solve and then choosing the appropriate algorithm (or parameter settings) from its repertoire, thus avoiding NFL constraints. While these approaches can extend the range of problems for which an algorithm is effective, it is clear that the ability of the first stage to identify problem types and select appropriate algorithms is itself an algorithm that is subject to NFL results. A second misconception is the interpretation of NFL results as leaving us in a hopeless state unable to say anything general about the algorithms we develop.

De Jong [38] states that NFL results "present us with a challenge to define more carefully the classes of problems we are trying to solve and the algorithms we are developing to solve them. In particular, NFL results provide the context for a meaningful discussion parameter setting in optimization algorithms, including: What algorithms parameters are useful for improving performance?"

Haftka [54] argues in a similar manner. He claims that the NFL theorems provide an important limitation of global optimization algorithms and suggests that when we propose a new or improved global optimization algorithm, it should be targeted toward a particular application or set of applications rather than tested against a fixed set of problems.

2.6 Tuning for Deterministic Algorithms

Algorithm parameter tuning is also important for deterministic algorithms. Because it has as a long tradition in this field, it is worth looking at relevant publications and compares established key elements from the deterministic domain to their implementations in the stochastic domain.

Miele et al. [87] compare different versions of gradient algorithms. Their comparison is based on the cumulative number of iterations for convergence. Buckley [27] explores the design of software for the evaluation of algorithms. Bussieck et al. [28] introduce PAVER, an environment for the automated performance analysis of

benchmarking data. Last but not least, the CPLEX automatic tuning tool should be mentioned in this context [66].

The reader is referred to McGeoch's [86] guidebook, which discusses tuning for deterministic algorithms. It is accompanied by the Open laboratory for experiments on algorithms (AlgLab) web site.

3 Test Sets

3.1 Test Functions

Artificial test functions are cheap functions with known properties, which imitate the behavior of real-world problems. They have many benefits such as known optima and a well-understood landscape. Large scale experiments can be run due to their inexpensiveness per evaluation. Obviously, these test functions can only mimic the complexity of real-world optimization problems up to a certain point.

A very popular set of test functions in the field of GAs was defined by De Jong [37]. This set contained instances of continuous, discontinuous, convex and non-convex, unimodal and multimodal, quadratic and non-quadratic, low- and high-dimensional, as well as noisy and deterministic test functions. Due to the limited computation capabilities at this time and for reasons of comparability, each test function was restricted to a bounded subspace of $\mathbb{R}^n$. Hillstrom [59] proposed a set of test functions for unconstrained nonlinear optimization. He simulated problems encountered in practice by employing a repertoire of problems representing various topographies (descending curved valleys, saddle points, ridges, etc.), dimensions, degrees of nonlinearity (e.g., linear to exponential) and minima, addressing them from various randomly generated initial approximations to the solution and recording their performances in the form of statistical summaries. In a similar manner, a set of 35 test functions for unconstrained optimization was proposed by More et al. [90]. Lenard and Minkoff [79] describe a procedure for randomly generating positive definite quadratic programming test problems. The test problems are constructed in the form of linear least-squares problems subject to linear constraints. Floudas et al. [47] present a large collection of mathematical optimization problems where most of them model a practical application, in particular from computational chemistry. The main emphasis is to find a global solution in case of a non-convex Nonlinear programming (NLP), not a local one. The widely used CUTE test problem collection by Bongartz et al. [23] contains problems for mathematical optimization, where the following categories are covered: quadratic programming problems, quadratically constrained problems, univariate polynomial problems, bilinear problems, biconvex and difference of convex function problems, generalized geometric programming, twice continuously differentiable NLPs, bilevel programming problems, complementary problems, semidefinite programming problems, mixed-integer nonlinear problems, combinatorial optimization problems, nonlinear systems of equations,

and dynamic optimization problems. Andrei [3] presents a collection of unconstrained optimization test functions. The purpose of this collection is to give to the optimization community a large number of general test functions to be used in testing the unconstrained optimization algorithms and comparisons studies. For each function, its algebraic expression and the standard initial point are given. Some of the test functions are from the CUTE collection established by Bongartz et al. [23], and others are More et al. [90], Himmelblau [60] or from some other papers or technical reports.

In recent years, parametrizable test functions and test problem generators became popular. Liu and Zhang [80] propose a test problem generator, by means of neural networks nonlinear function approximation, that provides test problems, with many predetermined local minima and a global minimum, to evaluate nonlinear programming algorithms that are designed to solve the problem globally. Addis and Locatelli [2] propose a new class of test functions for unconstrained global optimization problems. The class depends on some parameters through which the difficulty of the test problems can be controlled.

Daniels et al. [36] introduce a suite of Computational fluid dynamics (CFD) based test functions. Instead of trying to mimic real-world behavior, they make real-world functions available for benchmarking. They created a set of currently three computationally expensive real-world problems taken from different fields of engineering. The task in all three of them is the shape optimization. Each evaluation of the functions requires a CFD simulation.

3.2 Application Domains

3.2.1 Tuning in Industry

Many complex and expensive optimization problems can be found in industry. Parameter tuning offers a good solution to improve an algorithms' performance in said problems while minimizing the required effort. Just to name a few, examples in the water, energy, metallurgy, automotive, and information technology industries are presented.

3.2.2 Energy

A common challenge in the energy sector is cleaning exhaust gases from pollutants before they are released into the environment. Electrostatic precipitators (ESPs) are large scale gas cleaning devices. They are widely applied to the separation of solid particles from gas streams in industrial processes such as coal-fired power plants. Because they are large metal structures that can reach dimensions of over $30\,\text{m} \times 30\,\text{m} \times 50\,\text{m}$, their construction results in very high material costs. Increasing the efficiency of the separator would allow for a smaller build size and thus result in

huge money savings. The ESPs' efficiency is largely affected by the gas distribution in the so-called separation zones, where the incoming particles are ionized. Due to their electrical charge, the ionized particles stick to collecting electrodes inside of the ESP and are thus separated from the gas stream. A well-configured ESP should have a nearly homogeneous gas distribution throughout all of the separation zones. A poorly configured one will have areas with fast gas streams as well as areas where the gas is too slow or even flowing backwards in the device. Fast gas streams reduce the chance of the particles being successfully ionized. Thus, a lot of the particles can escape through the separation zones, largely reducing the ESP's separation efficiency. In areas where the gas is too slow, the filter remains mostly unused as there are only few particles that have to be collected. In order to ensure a homogeneous gas flow, a Gas distribution system (GDS) is employed in the entrance area of the ESP. The ESP consists of multiple exchangeable metal plates that can be fitted into several slots. Finding the correct configuration of metal plates for each slot of the ESP reveals a high-dimensional discrete optimization task. Schagen et al. [107] employ a surrogate-assisted evolutionary algorithm that was tuned with SPOT in order to solve the task.

3.2.3 Water Industry

Challenging optimization problems can also be found in the water resources community. The design of Water distribution systems (WDSs) poses a complex multiobjective optimization problem. Conflicting objectives such as construction costs, water quality, or network resilience generate a considerable computational overhead. Known optimal solutions are compiled in Pareto fronts to guide engineers to possible design alternatives for WDSs, but the generation of the fronts is a tedious process. Zheng et al. [117] use a NLP and Self-adaptative multiobjective differential evolution (SAMODE) hybrid to aid in the design of WDSs. SAMODE's mutation weighting factor and crossover rate parameters are part of the evolution and thus automatically tuned. The design of WDSs by means of NLP-SAMODE was tested using two objectives: network cost minimization and network reliability maximization. The approach was tested in three WDSs with 34, 164, and 315 decision variables each. NLP-SAMODE achieved superior Pareto fronts when compared with Non-dominated sorting genetic algorithm II (NSGA-II) and SAMODE methods.

3.2.4 Steel Industry

In metalworking, the process to reduce and homogenize the thickness of a metal piece is known as rolling. This is a complex process requiring several physical models to describe the thermal behavior of the furnace or the movement of the rolling mills. These models rule the configuration settings of the machine and alter the speed and quality at which the metal pieces are produced. Jung et al.

[73] describe an approach to optimize the hot rolling mill flow parameters using surrogate-based optimization. The flow parameters are important to determine the roll force and change in value and number depending on the material being worked in the mill. Given the high number of different materials and the lack of direct measures from the rolling process, the flow parameters are difficult to optimize. Traditionally, these parameters are determined by experimentation, but this method is time-consuming and cannot adapt quickly to new materials. Using data from an aluminium hot mill, the author implemented a model-based optimization using SPOT that solved the problem of flow parameter determination and minimization of roll force. The SPOT approach used Kriging as the surrogate model with expected as infill criterion (infill criteria will be discussed in Sect. 5.3.1). To compare the best-found local optima, the downhill simplex algorithm was used [93]. Empirical results showed that the optimization approach followed closely the shape of already known flow parameters. This demonstrated the possibility to predict flow parameters without conducting expensive laboratory measurements.

3.2.5 Automotive

One example from the automotive industry is the tuning of the throttle-valve controller. The throttle valve controls the amount of air flowing into the car's engine. The flow has a direct impact on the engine power and the amount of harmful exhaust gases. Neumann-Brosig et al. [95] use Bayesian optimization (BO) to implement an automatic tuning approach for the throttle-valve controller. BO combines prior information, or expert information, with experimental information. For every new iteration, the knowledge of the system is updated and new sampling points are selected to maximize the information gain. To avoid the complex and more expensive approach of learning a surrogate model for the controller, the authors decided to model the cost function instead, which results in a model-free approach. Gaussian process (GP) with the Matern52 kernel was used to model the cost function. BO uses this model together with an infill criterion (acquisition function) to select new parameter configurations. Entropy search and EI were both tested as infill criteria. The new parameter set is selected in such a way that the acquisition function is maximized. The GP model is then updated with the new selected values. This is repeated until the stopping criterion is reached. For this case, BO had a maximum budget of ten iterations. After experimentation, it was clear that BO outperformed manual tuning.

3.2.6 Information Technology

As a final example, an application from the information technology domain is presented. Hutter et al. [63] use the parameter optimization tool ParamILS [64] in the development of the high-performance Boolean satisfiability problem (SAT) solver SPEAR. ParamILS is a modified hill-climbing local search process. Its

variant FocusedILS will be explained in more detail in Sect. 7. SPEAR has 26 tunable parameters and is used to optimize two sets of problems: hardware bounded model checking (BMC) and software verification (SWV), each with 754 and 604 instances, respectively. Each time, only half of the instances were used to train the solver, and the other half is used for testing purposes. The FocusedILS variant was used to tune the parameter configuration for both problems on the training set. Results on the test set demonstrated highly improved performance of SPEAR when using the tuned parameter configuration in comparison to default settings. The tuned version of SPEAR also had large speedups. The SWV problem had a 500 speedup factor when compared to the default setting. A smaller but still significant speedup factor of 4.5 was observed for the BMC.

4 Statistical Considerations

4.1 Experimental Setup

The following example [17] illustrates fundamental statistical considerations, which are relevant for the tuning procedure.

Example 3 Can the following be considered good practice? The authors of a journal article used 200,000 function evaluations as the termination criterion and performed 50 runs for each algorithm. The test suite contained ten objective functions. For the comparison of two algorithms, population sizes were set to 20 and 200. They used a crossover rate of 0.1 in algorithm A and 1.0 in B. The final conclusion from their experimental study reads: "Algorithm A outperforms B significantly on test problem f_6 to f_{10}."

Problems related to this experimental study may not be obvious at first sight but consider the following questions: *Why did the authors use 200,000 function evaluations to generate their results? Would results differ if only 100,000 function evaluations were used?*

Solutions for this question are discussed by Cohen [32]. He presents a comprehensive discussion of the so-called floor and ceiling effects. A ceiling effect occurs when results cluster at the optimum, i.e., the problem is too easy. Floor effects occur if the problem is too hard and no algorithm is able to find a good solution. The selection of an adequate test problem, which avoids floor and ceiling effects, is crucial in algorithm tuning. Based on ideas from [40, 61] develops statistical tools, the so-called Run length distributions (RLDs),, which can be used to detect floor and ceiling effects.

In order to perform tuning experiments, we need tools to determine an adequate number of function evaluations, thoughtfully chosen number of repeats, suitable parameter settings for comparison, as well as suitable parameter settings to get

working algorithms. The Design of experiments (DOE) methodology provides tools for handling these problems [77, 88].

4.2 Design of Experiments

Algorithm parameters come in different types. Their type influences how the search space is explored. Perhaps the simplest and most common type is the real parameter, representing a finite real number that can assume any value in a given interval of $\mathbb{R}$. An example of such a parameter is the mutation rate, M, in GAs, see Table 1. Other parameters may be integer, i.e., assume one of a number of allowed values in $\mathbb{N}$, e.g., the population size, N. Categorical parameters typically represent the selection of certain subroutines, e.g., recombination operators in GA. These are parameters on which no particular order is naturally imposed. They do not fit in the integer category. The selection strategy, S, is an example. Binary parameters can be considered as special cases of categorical parameters.

The DOE methodology provides tools for the selection of algorithm parameter combinations as well as problem parameter settings such as starting points and computational budgets. It has an important impact on experimental research in computer science, which lays the foundation for tuning. Most influential is Kleijnen's work [75, 77]. It is highly recommended to combine the DOE methodology with tools from EDA [113].

Barton [11] presents views of algorithm performance that can aid in algorithm design and revision. Based on DOE, they provide a useful framework for performing and presenting results and identifying useful test functions and measures of performance. How DOE can be used for tuning EAs is demonstrated by Bartz-Beielstein [12]. In a similar manner, Ridge [101] presents in his Ph.D. thesis DOE methodologies for tuning the performance of algorithms. Ridge and Kudenko [102] provide a tutorial on using a DOE approach for tuning the parameters that affect algorithm performance. Coy et al. [33] use DOE to determine effective settings for parameters found in heuristics. Bartz-Beielstein et al. [15] used DOE, Kriging and computational statistic methods to investigate the interactions among optimization problems, algorithms, and environments. The technique is applied to the parameterization of PSOs and EAs. An elevator supervisory group control system is introduced as a test case to provide intuition regarding the performance of the proposed approach in highly complex real-world problems. Doerr and Wagner [39] discuss the sensitivity of parameter control mechanism with respect to their initialization. Campelo and Takahashi [29] discuss the sample size estimation for power and accuracy in the experimental comparison of algorithms. They present a methodology for defining the required sample sizes for designing experiments with desired statistical properties for the comparison of two methods on a given problem class.

4.3 Measuring Performance

The performance of an optimization algorithm, say A, is typically measured on the basis of a number of specific metrics after the algorithm has been run on a black-box optimization problem. The number of iterations required to reach the vicinity of an optimum is one simple metric. External factors, such as the required CPU time and the amount of memory, or the speedup compared to a benchmark in a parallel computing setting, can be used as performance measures, too. Combinations of these metrics are possible, too.

A few measures are omnipresent in the stochastic optimization literature, possibly under different names, but with the same meaning. These are the Mean best fitness (MBF), the Average number of evaluations to solution (AES),, the Success rate (SR), and the *best-of-n* or *best ever*. The following paragraphs summarize the discussion from [14].

While the MBF is easy to apply, it has two shortcomings. Firstly, by taking the mean one loses all information about the result distribution, rendering a stable algorithm with mediocre performance similar to an unstable one coming up with very good solutions often, but also failing sometimes. Secondly, especially in situations where the global optimum is unknown, interpreting the obtained differences is difficult. Does an improvement of, e.g., 1% mean much, or not? In many practical settings, attaining a good local optimum itself is the hardest step and one would not be interested in infinitesimal improvements.

The latter situation may be better served by SRs or the AES as a performance measure. However, one has to set up meaningful quality values to avoid floor and ceiling effects. The AES holds more information as it measures the "time factor," whereas the SR may be easier to understand. However, the AES is ambiguous if the desired quality is not always reached.

The best-of-n value is most interesting in a design problem, as only the final best solution will be implemented, regardless of the number of runs n to get there. However, this measure strongly depends on n.

Whatever measure is used, one should be aware that it establishes only one specific cut through the algorithm output space. Every single measure can be misleading, e.g., if algorithm A_1 is good for short runs but algorithm A_2 usually overtakes it on long runs. However, defining an ordering of the tested algorithm designs cannot be avoided so that one has to select a specific performance measure. The tuning process should improve the algorithm performance according to the given measure; thus, a strong influence of this measure on the resulting best algorithm design is highly likely. Whenever the standard measures do not deliver what is desired, one may be creative and design a measure that does, as suggested by Rardin and Uzsoy [99].

We conclude this section with examples of best practice. Box [24] evaluates the performances of eight methods for unconstrained optimization using a set of test problems with up to twenty variables. To measure the performance, comparisons of methods were carried out on the basis of the number of equivalent function

evaluations because "for many real problems the time involved in the computation of the function is vastly in excess of that required to organize the search." The performance measure is implemented as follows (the desired optimum is 0). For each starting point, where nine different starting points are considered, the number of equivalent function evaluations necessary to reduce the function to the values $1,\ 0.1,\ 0.01$, and 0.00001 is recorded. Eason and Fenton [43] compare 17 numerical optimization methods by plotting their convergence characteristics when applied to design problems and test functions. Several ranking schemes are used to determine the most general, efficient, inexpensive, and convenient methods. Eason [42] shows that common performance measures are intuitively machine-independent, which encourage such use. Unfortunately, the relative performance of optimization codes does depend on the computer and compiler used for testing, and this dependence is evident regardless of the performance measure. Dolan and Moré [40] propose performance profiles, i.e., distribution functions for a performance metric, as a tool for benchmarking and comparing optimization software. Moré and Wild [89] propose data profiles as a tool for analyzing the performance of derivative-free optimization solvers when there are constraints on the computational budget. They use performance and data profiles, together with a convergence test that measures the decrease in function value, to analyze the performance of three solvers on sets of smooth, noisy, and piecewise-smooth problems. Their results provide estimates for the performance difference between these solvers and show that on these problems, the model-based solver tested performs better than the two direct search solvers tested. Bartz-Beielstein [12] compared several performance metrics. Domes et al. [41] developed an optimization test environment, i.e., an interface to efficiently test different optimization solvers. It is designed as a tool for both developers of solver software and practitioners who just look for the best solver for their specific problem class. It enables users to choose and compare diverse solver routines, organize, and solve large test problem sets, select interactively subsets of test problem sets, and perform a statistical analysis of the results. Gould and Scott [52] discuss performance profiles, which have become a popular and widely used tool for benchmarking and evaluating the performance of several solvers when run on a large test set. They use data from a real application as well as a simple artificial example to illustrate that caution should be exercised when trying to interpret performance profiles to assess the relative performance of the solvers. They claim that:

> if performance profiles are used to compare more than two solvers (and Dolan and More state that "performance profiles are most useful in comparing several solvers"), we can determine which solver has the highest probability $p_i(f)$ of being within a factor f of the best solver for f in a chosen interval, but we cannot necessarily assess the performance of one solver relative to another that is not the best.

4.4 Reporting Results

After conducting tuning experiments, reporting results should consider the following topics:

1. Replicability: One must be able to repeat the experiment, at least to a high degree of similarity.
2. Results: What is the outcome? Does it correspond to expectation? Are established performance differences statistically significant and scientifically meaningful?
3. Interpretation: What do the results mean regarding the algorithms and the problems? Is it possible to generalize? Where do the observed effects originate from?

The following publications give useful recommendations for writing reports: Crowder et al. [35] provide a summary of important points that should be considered when writing reports. Jackson et al. [67] present updated guidelines for the reporting of the results of experimentation to reduce some common errors and to clarify some issues for all concerned. They reconsider the guidelines from [35] and offer extensions and modifications as deemed necessary. Barr and Hickman [9] examine the appropriateness of several performance metrics and explore the effects of testing variability, machine influences, testing biasses, as well as the effects of tuning parameters. Barr et al. [10] discuss the design of computational experiments to test heuristic methods and provide reporting guidelines for such experimentation. The goal is to promote thoughtful, well-planned, and extensive testing of heuristics, full disclosure of experimental conditions, and integrity in and reproducibility of the reported results. Bartz-Beielstein and Preuss [14] propose organizing the presentation of experiments into seven parts: general research question (reason for tuning), pre-experimental planning, concrete task, experimental setup (problem and algorithm design), results (visualization), observations (unexpected results), and discussion. Beiranvand et al. [19] discuss various methods of reporting benchmarking results.

5 Parallelization

5.1 Overview

Most tuning tasks are very time-consuming. Coupled with the ever-increasing amount of CPU cores per machine, parallelization is a valid method of choice for achieving real-time speedups. This is especially true for cases in which the algorithm that is supposed to be tuned runs only on a single core (serial evaluation). But still holds in many cases where the algorithm uses multiple cores, just not all the cores that are available on a given system. Often, the tuner is allowed to use lots of resources on a system with a high level of parallelization such as a computing

cluster. Just so that the tuned algorithm can run with little resources on a machine with less computing power, an embedded setup, or in a restricted small runtime.

Before we continue with an overview of parallelization techniques, we should define our aim for speedup more precisely. Barr and Hickman [9] give an overview of different definitions of speedup through parallelization. Here, p denotes the amount of evaluations that can be done in parallel. They discuss which measures are most reasonable by evaluating a survey of experts in the field of parallelization research. Based on this work, we define *speedup* by a reduction in measured wall time that was required to solve a fixed task. This definition explicitly ignores that the actual CPU time, which is summed up over all involved CPU cores, can drastically increase, as long as the wall time is reduced.

In the following, we will use Barr and Hickman's [9] definition of Relative speedup (RS), which is defined as follows:

$$\mathrm{RS}(p) = \frac{\text{Time to solve a problem on one processor}}{\text{Time to solve same problem on } p \text{ processors}}.$$

This section will only consider parallelization that is achieved by running the optimization algorithm, which is being tuned, in parallel. We do not aim at parallelizing the code of the tuner itself. Therefore, we adapt Barr's definition from the parallelization factor p from processor cores to the amount of algorithm runs that can be done on a system in parallel. To clarify this definition in more detail, we will take a look at an example.

Example 4 Imagine we are trying to tune an EA that usually runs on four cores because it generates four new individuals per generation. The tuner is running on a cluster with 32 available cores. Then, the tuner can be parallelized by proposing $p = 8$ new parameter sets at each time step. Each of these parameter sets is run in parallel with the given algorithm that itself further parallelizes by using four cores.

Lastly, we will assume for the remainder of this section that the quality assessment of a given parameter set on an algorithm is computationally much more expensive than the runtime of the tuner. Therefore, the tuner will only add an insignificant portion of runtime on top of the evaluation time required for the algorithm tests. Thus, if, for example, a tuner proposes two trial parameter sets per iteration, then evaluating these in parallel should closely half the overall computation time of the tuning process.

5.2 Simplistic Approaches

A simple form of potential speedup through parallelization can be achieved without even changing the tuner or the general procedure. Assuming a given tuner only runs serially and has some level of stochasticity in its search approach, it will yield different results in each run. This can be exploited by running the same tuner

multiple times in parallel. Given that the machine has more CPU cores available than the algorithm requires, this parallelization can be done at virtually no additional cost or time. Just selecting the best final result of the parallel runs will yield potentially better results.

If the choice for a specific tuner is not already made and parallelization is possible on the given system, then choosing population-based approaches is favorable. These approaches, such as EAs, are intrinsically already parallel as the individuals in a given population can be evaluated without a specific order, independent of each other. Population-based approaches should especially be favored when the available level of parallel evaluations is high ($p > 10$). When population-based methods are applied in parallel, then the population size should be fixed to a multiple of p. Other population sizes will result in efficiency losses due to some individuals being left over, that have to be evaluated without using all available processing cores.

5.3 Parallelization in Surrogate Model-Based Optimization

For smaller values of p, the sole efficiency bonus of using more cores through a population-based approach might not outvalue other benefits gained when using a more efficient tuning method. Surrogate model-based optimization (SMBO) is state of the art when it comes to optimization efficiency with very expensive to evaluate functions. In fact, most available tuners and also most of the tuners that are discussed in Sect. 7 are model-based. Therefore, they make use of some sort of a fitted surrogate model in order to guide the tuning process and spent less budget in poor regions of the search space.

Yet, achieving parallelization in SMBO is not straightforward. Standard SMBO only proposes a single new candidate solution at each iteration. Thus, parallelizing SMBO requires an internal change of the tuner itself. The most efficient way of doing so is still an open field of research. Therefore, the rest of this section gives an overview of the existing approaches that can be implemented into tuning procedures.

A highly recommendable survey of more existing methodologies in the general field of parallelization is given in [55]. The rest of this section starts with some general approaches for parallelizing SMBO. Section 5.3.1 presents methods that exploit an uncertainty estimation of the given surrogate. Lastly, Sect. 5.3.2 explains Surrogate-assisted algorithms (SAA)s.

One of the most straightforward approaches to parallelize SMBO is to use multiple surrogate models. Each of the models will then be scanned, searching for their specific optimum. These optima can then be evaluated in parallel. Once all parallel evaluations are completed, the models will be refitted on all existing data. Sadly, this process is not easily scalable. For each additional parallel evaluation, a further surrogate model has to be implemented into the tuning approach.

The approach can be further parallelized by adding additional infill criteria. An infill criterion defines some quality metric by which a candidate solution is considered to be the optimum of a surrogate model. This could, for example, simply

be the candidate solution with the best-predicted objective function value. Often these infill criteria contain information such as the model's uncertainty at a given point or the distance to already known candidate solutions. By choosing multiple infill criteria and evaluating each criterion on each surrogate, many candidate solutions can be generated at each iteration of the tuner.

5.3.1 Uncertainty-Based Methods

Many parallelization concepts in the field of SMBO use Kriging's internal uncertainty estimate. A very popular form of global optimization with the Kriging uncertainty is EGO [72]. In EGO, the so-called EI is used as infill criterion, which is defined through a combination of the model's uncertainty at a specific candidate solution and the predicted improvement of the given solution over the best-known solution so far. The main reason for its popularity is that it delivers a balance between exploration and exploitation. Three popular approaches that make use of EI are given in the following.

Sóbester et al. [112] apply the same EI criterion as used in EGO. However, they do not search for the single best candidate with the highest EI on the surrogate. Instead, they apply niching-like algorithms in the search for the q best local optima of the EI function. Each of these local optima is then evaluated in parallel. Drawbacks of this approach include that a given function might not have q distinct local optima in the EI search space. Additionally, local optima can be arbitrarily bad in the global view.

Ginsbourger et al. [49] propose the so-called q-EI criterion. This criterion estimates the combined EI of a set of multiple candidate solutions, thus giving a natural extension to the original single-point EI. They first optimized this criterion by brute force but later published an exact formula for calculating the q-EI for a small number (less than ten) of candidate solutions. They state that the criterion is costly to compute and optimize. Finding a set of good design points can also quickly lead to a very high-dimensional problem. Consider a ten-dimensional problem that shall be parallelized to only $q = 8$, this still results in an 80-dimensional search problem.

Bischl et al. [22] consider exploration and exploitation as two distinct objectives in a bi-objective optimization problem. Each Pareto-optimal point is, therefore, a good compromise between the two metrics. Thus, parallelization is achieved by gradually varying this compromise and virtually as many points as desired can be chosen from the Pareto front.

For models other than Kriging, this intuitive balance between exploration and exploitation is not available as they are missing an internal uncertainty estimator. Yet, there are multiple proposed methods that use a distance or density measure to known candidate solutions as a replacement for the missing uncertainty estimate. Therefore, the adapted concept is to sample at points that are both estimated to be good and not already densely sampled. Again this idea can be adapted for

parallelization similarly to the uncertainty-based methods, for example, by treating the distance as another objective or by searching for multiple local optima.

5.3.2 Surrogate-Assisted Algorithms

The class of SAAs forms the compromise between population-based methods and SMBO approaches. In contrast to SMBO, surrogate-assisted methods do not search for points on a surrogate by sampling an infill criterion. Instead, they generate new candidate solutions internally and then only use the surrogate as a quality indicator for the proposed candidate solutions before actually evaluating them. One example for the class of SAAs is surrogate-assisted EAs. In these, the EA generates new offspring at each generation, but instead of evaluating all offsprings, the surrogate can be used as a filter to sort out unsuited individuals. By doing so, the algorithms maintain their ability to parallelize through a population of candidate solutions but at the same time keep their efficiency improvements that are achieved through the guidance of surrogates.

6 Tuning Approaches

6.1 Overview

Existing parameter tuning approaches can be separated into two classes: external, and internal tuning. External tuning is defined by tuners that *wrap around* an existing algorithm and its parameters. Therefore, an external tuner will run the existing algorithm multiple times with different parameter settings in order to determine a good parameter configuration. We furthermore split external tuners into three subcategories: manual, automatic, and interactive tuning. Manual tuning simply defines the act of a specialist trying a few parameter sets. These sets are based on the experimenter's domain knowledge in a rather subjective manner. Automatic tuners are started with some basic user input and then automatically search for a good parameter setting without any user intervention. Interactive tuners enable the user to change settings and stop/start the tuning process as desired.

Internal tuners follow a very different approach to the three classes of external ones. Instead of changing parameters over multiple runs of an algorithm, internal tuners are an intrinsic part of the algorithm itself. They can change algorithm parameters during runtime, in a single run of that algorithm. The following sections give a more in-depth view into each of the discussed categories.

6.2 Manual Tuning

For many years, tuning was more or less ignored as an integral part of the optimization process. In order to find a set of algorithm parameters that works well, the go-to method often was manual tuning. A common approach is to select a couple of initial parameter configurations based on conventions, personal experience, ad-hoc choices, or "standard" settings, e.g., settings determined by De Jong [37]. Experiments are executed and based on personal assessment. The developer decides if new experiments with a different set of parameter configurations should be performed. Often the field specialist follows a one-parameter-at-a-time approach [76]. This step is repeated for each parameter of the algorithm, testing a few different parameter settings. The one-parameter-at-a-time completely ignores any interactions between the parameters. Thus, if the problem of tuning the given parameters is not completely separable, then an optimal solution can never be found.

There are several additional drawbacks to the manual tuning approach. Firstly, the selection of candidate solutions is strongly biased by the personal experience of the developer. Diversity of the candidate solutions and reproducibility of the experiments are hard to sustain. Secondly, manual tuning is time-consuming and requires constant human input.

6.3 Automatic Tuning

Automatic tuning approaches are tools, which are once set up, possibly with some tuning parameters, and then start an automatic external tuning process. Therefore, the approach will try multiple parameter settings for a given algorithm and makes decisions based on the final outcome of an optimization run of the algorithm. Automatic tuners are meant to be "start once and receive results" tools, which do not require any additional interaction. The remainder of this paragraph provides an overview of the existing research and methodologies for automatic tuners.

Audet and Orban [6] present a general framework for identifying locally optimal algorithm parameters. Algorithm parameters are treated as decision variables in a problem for which no derivative knowledge or existence is assumed. A derivative-free method for optimization seeks to minimize some measure of performance of the algorithm being fine-tuned. This measure is treated as a black box and may be chosen by the user. Nell et al. [94] develop a formal description of meta-algorithmic problems and use it as the basis for an automated algorithm analysis and design framework called the High-performance Algorithm Laboratory. They demonstrate this approach by conducting a sequence of increasingly complex analysis and design tasks on solvers for SAT and mixed-integer programming problems. Parejo et al. [96] perform a comparative study of metaheuristic optimization frameworks. As criteria for comparison, a set of 271 features grouped in 30 characteristics and 6 areas has been selected. A metric has been defined for each feature so that the

scores obtained by a framework are averaged within each group of features, leading to a final average score for each framework. Out of 33 frameworks, ten have been selected from the literature using well-defined filtering criteria, and the results of the comparison are analyzed with the aim of identifying improvement areas and gaps in specific frameworks and the whole set. The authors claim that "a significant lack of support has been found for hyper-heuristics, and parallel and distributed computing capabilities." Vodopija et al. [114] consider the tuning problem as a single stochastic problem for which both the spatial location and performance of the optimal parameter vector are uncertain. A direct implication, of this alternative stance, is that every parameter vector is sampled only once. In their proposed approach, the spatial and performance uncertainties of the optimal parameter vector are resolved by the spatial clustering of candidate parameter vectors in the meta-design space. In a series of numerical experiments, considering 16 test problems, they show that their approach, Efficient Sequential Parameter Optimization (ESPO), outperforms both F-Race and SPOT, especially for tuning under restricted budgets. Pavón et al. [97] define an automatic tuning system using Bayesian case-based reasoning (CBR) able to adapt to different instances of the same problem and find optimal parameter configuration for each problem instance. CBR uses Bayesian networks (BN) to model the specific problem instance and proposes new parameter configurations. Yeguas et al. [116] use this approach to automatically tune EAs for highly expensive geometric constraint solving problems. Empirical results showed performance improvements in the EAs when compared with ANOVA parameter settings. Audet and Orban [6] present a derivative-free method for identifying locally optimal algorithm parameters. Nell et al. [94] develop a formal description of meta-algorithmic problems and use it as the basis for an automated algorithm analysis and design framework called the High-performance algorithm laboratory (HAL). They demonstrate the working principles of HAL by conducting a sequence of increasingly complex analysis and design tasks on solvers for SAT and mixed-integer programming problems. Parejo et al. [96] perform a comparative study of metaheuristic optimization frameworks. As criteria for comparison, a set of 271 features grouped in 30 characteristics and six areas was selected. A metric was defined for each feature so that the scores obtained by a framework are averaged within each group of features, leading to a final average score for each framework. Out of 33 frameworks, ten have been selected from the literature using well-defined filtering criteria, and the results of the comparison are analyzed with the aim of identifying improvement areas and gaps in specific frameworks and the whole set. Vodopija et al. [114] propose Efficient sequential parameter optimisation (ESPO), which considers the tuning problem as a single stochastic problem for which both the spatial location and performance of the optimal parameter vector are uncertain. As a direct consequence of ESPO, every parameter vector is sampled only once. The spatial and performance uncertainties of the optimal parameter vector are resolved by the spatial clustering of candidate parameter vectors in the meta-design space.

The major disadvantage of these automatic tuning methods is that they require a considerable computational budget because they usually try many possible

settings to find an appropriate one. Nonetheless, in recent years some studies have specifically focused on automatic tuning of parameters in optimization solvers.

6.4 Interactive Tuning

In contrast to automatic tuning approaches, interactive approaches are meant to be started and stopped as the user desires. They can theoretically be used in the same way as an automatic tuner by starting them and waiting until the budget is depleted. However, they provide some additional information to the user during the tuning process itself and enable the user to adapt settings based on that knowledge. For example, interactive tuners provide interactive visualization tools of the current algorithm parameter search space, giving the user a deeper parameter understanding by visualizing interactions and not only reporting the final performance. Furthermore, interactive tuners provide the chance to check for configuration mistakes in the tuning parameters during the runtime. Imagine a very long-running tuning process that was started with a model-based tuner, and the selected modeling algorithm does not fit the search landscape, which results in a poor fit of the true fitness landscape. An automatic tuner can only provide two choices in such a situation (if it is even possible to notice before the budget is fully depleted):

1. The tuner is stopped, reconfigured, and started from scratch. This leaves the tuner with less budget for the second attempt.
2. The tuner continues with the same settings, likely providing an inferior solution.

Interactive tuners circumvent configuration issues by giving the user the chance to pause processes and change the tuner configurations in the tuning process without requiring a restart. Whenever something seems odd to an expert or the tuner seems to be stuck, the parameters are easily adapted, and the system continues with new settings. Misconfigurations are thus much less of an issue in interactive systems. Worst-case scenarios in which a tuner uses up all of its budget without finding a promising algorithm configuration occur seldom.

An example implementation of an interactive tuner can be found in the current R implementation of SPOT. Additionally, to all the above-described mechanics, SPOT comes with a Graphical user interface (GUI), the so-called spotGUI. The GUI can be used as an easy setup helper for the tuning process. SPOT can be configured, started, and stopped in the GUI while providing interactive plots of the current algorithm parameter search space. The GUI makes quick changes to all possible settings in SPOT viable resulting in an easy to use playground for finding good settings for the tuner while checking on the tuning process.

6.5 Internal Tuning

We define the term *internal tuner* to be any algorithm that can change its parameters during a running optimization. Harik and Lobo [57] first proposed fully parameterless EAs, which are EAs that do not have parameters that are changeable by the user, but instead they are updated during the optimization. Internal tuning approaches excel on problem classes, where a change of parameters is required during runtime. Internal tuning lies not in the main focus of this chapter, and we will therefore only shortly cover the key aspects of internal tuning mechanisms. Kramer [78] presents an overview of self-adaptive schemes for EAs, which provides a good starting point and includes a review of the historic development.

It is important to note that internal tuning is inherently different from external tuning. Internal and external tuning can be combined. For example, it is possible to set some parameters through external and some other parameters of an algorithm via internal tuning. Furthermore, it can make sense to use external tuning to set good initial parameter values that will then be internally changed over time via internal tuning. A direct comparison of internal and external tuning approaches is therefore not viable as one does not exclude the other.

7 Tuning Software

7.1 Overview

There are several tuning algorithms available that tackle the problem of parameter tuning. Well-established tuners are SPOT [16] and its derivatives, e.g., Sequential model-based algorithm configuration (SMAC), which was developed by Hutter et al. [65]. Furthermore, we have to mention Iterated racing procedure (IRACE), which was introduced by Lopez-Ibanez et al. [82], and ParamILS, which was proposed by Hutter et al. [64]. In addition to this top dogs, we will present Gender-based genetic algorithm (GGA) [4] as an alternative approach. Without the aim of doing an extensive survey, we describe the main features of some of the most known automatic parameter tuning software. An overview of the main features of the presented tuning software can be found in Table 3.

7.2 IRACE

Based on the F-Race proposed by Birattari et al. [20], the IRACE was introduced in [82]. Racing, in this scope, refers to a model selection process. A set of candidates is evaluated at a minimum of T^{first} iterations. In the end, a statistical test is performed and the candidates with the worst performance are discarded. A new round

with T^{each} evaluations, with $T^{each} < T^{first}$, is run on the remaining candidates. The statistical test is again performed and more candidates are discarded. This is repeated until a predefined budget is reached. IRACE consists of three steps: initialize the set of candidate solutions by sampling from a given distribution, select the best candidates by racing, and update the sampling distribution to generate more possible candidate solutions.

During parameter sampling, the conditional relationships between parameters are taken into account by sampling them following the order of the dependency graph of conditions. Once the candidate solutions are selected, the iterated racing starts. The Friedman statistical test is applied to determine which candidates should be eliminated. Other tests such as t-test are also available. The new candidates are sampled by updating the sampling distribution and making it biased toward the current best candidate. The surviving candidates are tested against the new sampled candidates. This is repeated until a predefined budget is reached. To introduce more diversity and avoid a premature convergence, IRACE incorporates a soft-restart feature. When the candidate solutions are too similar, the sampling distribution is reinitialized.

To avoid losing good candidates due to a bad problem instance evaluation, IRACE introduces elitist racing. If a new best candidate is found after T^{new} iterations, the old best candidate can only be discarded after its evaluation is worse than the new best by a number of T^{new} iterations.

7.3 SPOT

The SPOT software relies on SPO methodology which was introduced in [16]. The most recent version of SPOT is freely available as an R package and comes with a graphical user interface: the spotGUI. It provides a graphical configuration possibility for SPOT as well as interactive graphing and analysis features. Every configuration done in the GUI can be exported directly into R code. SPO is based on surrogate model-based optimization. This makes it especially efficient for tuning tasks as these are usually computationally expensive to evaluate. The general framework of SPOT is meant to be a modular toolbox giving the user a free choice to switch around parts of the tuning process. For this purpose, SPOT provides a wide variety of modeling techniques, model optimization algorithms, design generators, noise handling techniques, and much more. Additionally to the options that are directly provided through the SPOT package, the user can easily add his own methods into the tuning structure. For example, if another modeling technique shall be used, it can sometimes be directly applied or in a worst case easily be implemented with the help of a small wrapper.

7.4 SMAC

Similar to the approach used by SPOT, SMAC uses four steps: Initialize, FitModel, SelectConfigurations, and Intensify [65]. Initialize runs once the algorithm to be tuned (target) with its default configuration. FitModel contains the definition of the model used in SMBO. The SelectConfiguration component selects a set of candidate configurations to test. In Intensify, the last component controls how many evaluations are performed for each candidate configuration and determines when a configuration should be set as the current best result. SMAC selects candidates following a model recommendation. In the same manner as SPOT, recent versions of SMAC are able to deal with categorical algorithm parameters. This was possible by using random forest as the model driving the FitModel component as discussed in [15]. SelectConfiguration uses the model's predictive distribution to compute the expected positive improvement over the current best configuration. The selected configurations are extended by adding randomly sampled configurations to lower the bias of the configuration set. Lastly, the last component, Intensify, aggregates the predictions across the different instances of the problem.

7.5 ParamILS

ParamILS [64] presents an iterative approach to parameter tuning. The ILS method generates an array of local optima by inserting a perturbation each time a local optimum is found. ParamILS is a stochastic local search algorithm that uses one-exchange neighborhood to search the parameter space. This means that only one parameter is changed at a time when iterating through the optimization loop.

A function needs to be defined to determine when a set of parameter values is better than the other. The most simple approach is to take the parameter values with the lower cost function after N runs of the target algorithm. This approach is referred to as BasicILS. The value of N remains fixed for all the evaluations of all candidate solutions in BasicILS. Another variant of ParamILS with varying value for N is FocusedILS. Here, the necessary runs to estimate the cost function of each candidate solution can differ. To be able to compare the performance of two parameter configurations, θ_1 and θ_2, tested each N_1 and N_2 times, respectively, with $N_1 < N_2$, the concept of dominance is used. A candidate solution θ_1 is dominant iff the performance of the target algorithm with θ_1 in the first N_1 runs is as good as or better as the performance on all runs of θ_2. The determination for the best current candidate is carried out by adding runs to the candidate solutions until one demonstrates dominance over the other. Once a configuration is picked as best, a user-determined number of extra runs are performed to assure that many runs with good parameters are available. This method is guaranteed to converge when $N \rightarrow \infty$ and the cost function is static.

Another feature of ParamILS is the adaptive capping of candidate solution runs. This tackles the problem of wasting resources on a candidate θ_i worse than

previously tested ones. The basis of the adaptive capping lays in comparing the execution time of the candidate solutions. During the selection of the new best candidate, the required time for the first N runs of the best solution is set as the lower bound. When a new candidate cannot dominate the current best on the given runtime, the remaining runs are dropped.

Bringing all together, ParamILS search the parameter space by initializing the algorithm using a mix of random and default parameters. Determines the next best parameter configuration. Uses a fixed number of random moves to perturbate the current best candidate solutions. Restart the search at random with given restart probability and stop once the stopping criterion is met.

7.6 GGA

GGA [4] is a model-based optimization algorithm that uses a model-free genetic algorithm framework. Its core idea lays on using two population partitions, one containing the candidate solutions to evaluate the target algorithm (competitive), the other (non-competitive) containing other diverse candidate solutions used to increase exploration capabilities. The crossover operator is a hybrid between multi-point and uniform crossover. The crossover happens taking one individual from each of the population partitions. Mutation is performed by simple random replacement, and the selection mechanism is tournament.

To lower the cost of testing individuals directly on the target algorithm, GGA introduces the use of a specialized surrogate model for genetic algorithms. The surrogate model is introduced during the crossover operator to define new "genetically engineer" offsprings. Here, the surrogate model chooses what combination of parents will yield the fittest offsprings. A slightly modified version of random forest is used as the surrogate model. To reduce the computational effort, the model is not expected to produce a detailed approximation of the target algorithm but only to identify the areas in which high performances are to be expected given the possible offsprings. To avoid losing too much population diversity as a result of random forest guided genetical engineering, a percentage of the population is replaced with randomly generated individuals. GGA also includes the idea of sexual selection. Here, the surrogate model indicates which of the individuals in the non-competitive population should be chosen for mating. The decision is based on the "attractiveness" and increases the chances of fitter offsprings. Further details of GGA are discussed in [5].

7.7 Usability and Availability of Tuning Software

This paragraph describes important features of tuning algorithms. We consider issues such as open-source, programming languages and interfaces, operating

systems, graphical user interfaces, reporting facilities, and usability. All of the above-presented tuning software are open source. And some of them as of today are still being regularly updated. The results of this comparison are summarized in Table 3.

IRACE is based on the statistical programming language R, making it easily accessible for all the system platforms that support R [98]. At the time of writing, the IRACE team is still regularly updating the R package. The use of the method is straightforward, and it reports all of the collected information together with the set of statistically best configurations in a single file. Some visualization functions are already included, and with some familiarity with R, it is easy to expand them according to personal needs.

SPOT is similarly to IRACE programmed in R. It is regularly updated on R's package platform CRAN but also available on Github with more recent development versions. SPOT is extended with a GUI, the spotGUI. The GUI can be started with a single command in R, but it can also be hosted as a Webservice. This makes it a viable choice in teaching or industry, where a tuner shall be applied without any knowledge of R. The developers also host a spotGUI Android app as a webview to the most recent spotGUI Version. Aside from the GUI, SPOT's main functionalities can all be accessed with a single command in R. Additional commands can be used to visualize the results or do further reporting and analysis. The report includes the set of statistically best configurations. Furthermore, a surrogate model that is fitted to the results of the tuning run is returned, providing more in-depth process variable understanding.

SMAC implementation is developed in Java and Python. The Java version of the tuner has, at the time of writing, not been updated for some years. However, the Python version found on Github is still active. Being a Python implementation, it can be used without restriction in most operating systems. There is enough documentation to assist in the installation and use of SMAC so that basic knowledge of Python is enough to run the software. The results are stored in several log files, and the best-found configuration is given. To the best of our knowledge, there is no visualization function included in the SMAC implementation

Table 3 Overview on most commonly required features for tuning algorithms and their support in the given tuning software. A checkmark (✓) indicates the feature is already included. The circle (◯) indicates the feature is stated to be possible to be implemented with minimal effort

Features\Software	IRACE	SMAC	ParamILS	GGA	SPOT	Manual
Supports numerical variables	✓	✓	✓	✓	✓	✓
Supports categorical variables	✓	✓	✓	✓	✓	✓
Interactive execution					✓	✓
Multi-instance problems		✓				
Parallelization	✓		◯	✓	◯	
Model-based		✓		✓	✓	
Visualization	✓				✓	
Noise handling	✓				✓	
Graphical user interface					✓	

ParamILS is written using Ruby and can be downloaded in a ready to use Linux executable. The use on other platforms is possible by downloading the source code and doing minor edits. The source code includes some examples with different tuning scenarios. The interaction with the tuner is done mostly via text files. As in SMAC, the output of the optimization loop is saved in log files together with the best-found configuration, and no function is included to help in the visualization of the results.

GGA is written in C using the libxml2 library. Currently, there is a working version for Unix platforms with no plans to extend it to windows. The configuration for the tuner is done through an XML file. As with ParamILS and SMAC, there are no visualization tools included in the source code.

7.8 Example: SPOT

This section illustrates a typical workflow of applying parameter tuning software to an algorithm. We will use the R package SPOT, yet the general workflow will be similar to other tuners. Consider we want to optimize a function with some type of algorithm. The performance of this algorithm depends on a set of configuration parameters. In this case, performance might be defined through any sort of metric as previously described in Sect. 4.3. Through tuning, we want to maximize the performance of the algorithm. For the sake of this example, we will consider a minimization problem. We want to achieve the best result after a fixed amount of function evaluations.

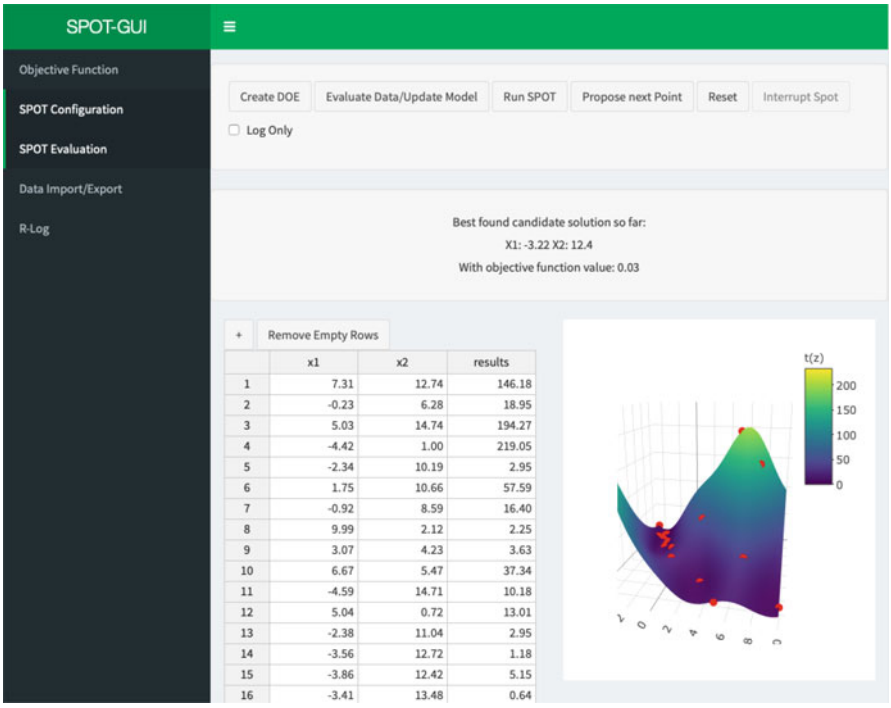
Setting up the tuning process in SPOT requires some basic initial information. First of all, SPOT requires a function that can measure the performance of the algorithm based on any proposed parameter set. Thus, we pass a function to SPOT that accepts a parameter set and runs the algorithm. The function returns the best-found objective function that the algorithm achieved with the given budget. Next, SPOT needs a definition of the given search space. It requires the lower and upper bounds for each configurable parameter of the algorithm. If some of the parameters are not continuous, this change in type, e.g., to a categorical variable, has to be specified as well. This setup for SPOT can be done in a single line of R code but also in just a few clicks in SPOT's graphical user interface (spotGUI).

Since the algorithm in this example is stochastic, it will yield noisy results. Therefore, the noise handling option in SPOT should be activated. It is possible to configure SPOT just with simple repeats, but we will configure it to use Optimal computational budget allocation (OCBA) [30] in this example. In this way, SPOT will try to assign repeats only in promising regions of the search space. Regions, where only bad samples exist so far, are unlikely to improve enough even after multiple samples. Thus, spending budget there should be avoided.

With this basic setup, SPOT is ready to start. Yet, if the user desires to do so, he is free to change more of SPOT's default settings. For example, the user could modify the initial design generation process. The default setup consists of a Latin hypercube design (LHD) with a relatively small budget. We recommend starting

SPOT with such a small budget to fully make use of its interactivity during the tuning process. By starting with a small portion of the total available budget, the user is able to continuously check if the tuning process is working correctly. Especially, the choices for the internal modeling technique and the model optimizer can largely affect SPOTs efficiency. Wrong configurations can easily be discovered by looking at the interactive model plots. Bad model fits are often clearly visible in these plots. Due to the structure of SPOT, it can be restarted multiple times, each time adding a little more budget, without any performance drawbacks.

A screenshot of the tuning process in the spotGUI is shown in Fig. 2. After the initial design is evaluated, SPOT will build a first surrogate model on the acquired data. The user is free to extract and analyze the model or use SPOTs internal tools or the spotGUI for this purpose. At each iteration, the next design point will be created by applying the configured model optimizer to the surrogate. The candidate



	x1	x2	results
1	7.31	12.74	146.18
2	-0.23	6.28	18.95
3	5.03	14.74	194.27
4	-4.42	1.00	219.05
5	-2.34	10.19	2.95
6	1.75	10.66	57.59
7	-0.92	8.59	16.40
8	9.99	2.12	2.25
9	3.07	4.23	3.63
10	6.67	5.47	37.34
11	-4.59	14.71	10.18
12	5.04	0.72	13.01
13	-2.38	11.04	2.95
14	-3.56	12.72	1.18
15	-3.86	12.42	5.15
16	-3.41	13.48	0.64

Fig. 2 Screenshot of the spotGUI software during the tuning process. Using buttons on the top, the user can execute commands in SPOT. From left to right, it is possible to create a DOE and evaluate given candidate solutions. The "Run SPOT" button starts the SPO procedure and uses the configured budget for tuning. With "Propose next Point," a single iteration of SPO is executed, creating a new design point. The tuning process can be reset, and SPOT can be interrupted during the tuning procedure. The spotGUI output can be seen on the lower half of the image. An overview of all candidate solutions is available with an interactive 3D plot of the modeled parameter landscape. Red dots in the landscape mark the evaluated candidates

solution that maximizes the configured infill criterion (default is the candidate with the best-predicted value) will be chosen for evaluation as a parameter set for the algorithm.

After each iteration, it is possible to change the surrogate model, optimizer, and even all the general settings in SPOT. Especially in the spotGUI, this results in a very intuitive and exploratory approach as a change in model or optimizer is done with just a few clicks and the new model is fitted and plotted for the user to explore. It is often possible to simply see which model provides a superior fit for a given landscape by just selecting each model once and comparing plots of their response surfaces.

After all the budget for evaluating the algorithm is spent, SPOT returns a lot of information for an a-posteriori analysis. In addition to the best-found configuration, all candidate solutions and their qualities are returned. The last fitted surrogate model is given to the user to enable further process understanding. The model can yield additional information on variable importance or robustness of the proposed candidate solutions. Finally, the user can implement the best-found parameter set for his algorithm.

8 Research Directions and Open Problems

As mentioned in Sect. 5, the parallelization of expensive optimization problems, or more specifically tuning problems, is a broad field of ongoing research, see, e.g., [34]. Here, we see the necessity for more in-depth research for budget allocation techniques in two main regions. Firstly, since many tuning problems are stochastic, resampling techniques like OCBA are required. Yet, these techniques have to be combined with the existing techniques for parallelization in order to optimally spend a given budget in a parallel manner. The same research question is open in the area of multi-fidelity optimization [48]. Multi-fidelity optimization is applied when a cheaper to evaluate but often less accurate second objective function exists. For example, consider the tuning of an algorithm that will be used to optimize parameters of a very long-running computer simulation. Now, the field specialists have a second computer simulation that is less accurate but fast to evaluate. The specialists expect the structure of the search space to have similar landscape features. If this assumption holds, then algorithm parameters that work well on the cheap simulation should also hold for the expensive one. Multi-fidelity optimization can be applied in order to transfer information gain of good parameters from one tuning problem to another. Again parallel sampling on the higher- and the lower-fidelity problem at the same time will reduce the required wall time for the tuning process. Lastly, since a large portion of the existing research work is concentrated around the uncertainty estimation in Kriging, we see room for more work on non-Kriging based approaches for parallelization.

The use of BO and its application on parameter tuning is also a topic of big interest. BO is a non-deterministic iterative optimization method that combines

available prior and past information of the problem. The use of Bayesian approaches is not new in the literature, and implementations in industry can be found in Sect. 3.2.1. However, it implies a different stochastic test and analysis than the more popular statistical (frequentist) tests. Thus, both methods cannot be directly compared. This change in methodology is one of the reasons why there are not yet many Bayesian approaches in the industry. A guideline for a detailed comparison between the performance and final results between Bayesian and frequentist methods is still needed. Additionally, Bayesian methods generally had a high computational complexity, and for this reason, it is also of interest to study how they can be combined with other methods to obtain a better and more efficient performance.

One issue that currently remains less clear and should be tackled in the future is how to tune if runs take a very long time, so that only a few of them are affordable? This especially applies to many real-world applications, where tuning would probably greatly help.

Some researchers and practitioners consider the parameterless optimization algorithm, which does not require any tuning, as the ultimate goal. Although struggling with parameter settings can be tedious, we claim that parameters can be advantageous because they are tools for understanding the algorithm and its performance, they help us to integrate domain knowledge, and they are valuable tool for adapting the algorithm to one specific problem instance. Finally, implications from the NFL, as discussed in Sect. 2.5, should be considered. De Jong [38] correctly states that NFLs theorems "serve as both a cautionary note to optimization practitioners to be careful about generalizing their results and as a source of encouragement to use parameter tuning as a mechanism for adapting algorithms to particular classes of problems."

9 Summary and Outlook

The process of improving the performance and robustness of an algorithm is considered as tuning. This chapter discusses strategical issues. We identified the key topics that are necessary for tuning, namely, the optimization algorithm and its parameters, the optimization problem with corresponding parameters, the tuning algorithm, performance metrics to measure performance, experimental setup using DOE, parallelization techniques, as well as reporting and visualization tools. Grefenstette's seminal study was used to exemplify these topics. We discussed prominent tuning approaches and related software tools. Additionally, we have named some still open problems, which should be subject of future research.

To summarize our observations and discussions from the previous sections, we will reconsider Grefentette's approach (see Sect. 2.4) from today's point of view. The following observations can be made:

- The experimental design could be enhanced. Using DOE, techniques are strongly recommended. Kleijnen [77] can be used as a starting point.
- Formulating scientific statements, e.g., "increasing the population size results in an improved performance on this specific test problem," which are broken down to statistical hypotheses, is highly recommended. Bartz-Beielstein [12] give useful hints. This approach is useful for validating the conclusions such as "... good online performance is associated with either a high crossover rate combined with a low mutation rate. ..."
- Maybe additional algorithm runs are required to obtain significant results. Tools for calculating the number of algorithm runs and repeats should be used, e.g., the article from [29] is useful. This is important to avoid floor or ceiling effects.
- Sensitivity analysis as presented by Saltelli et al. [105] might lead to interesting observations, especially, when interactions occur. The practical guide to sensitivity analysis is a good starting point [104].
- Grefenstette does not use any parallelization because the test problem instances are relatively simple and the number of algorithm runs is moderate.
- Performing the final evaluation on an unseen test problem is highly recommended. Unfortunately, it is not an established technique in the optimization community. The situation is different in the machine learning community, where data sets are partitioned into training, validation, and test data subsets [58].
- Modern tuning software packages provide tools to visualize the results. Interactive graphics, which visualize the parameter space, are highly recommended.

In general, two main research directions are relevant. The first direction, automatic tuning, focuses on the result. It is based on machine learning tools and generates big data. Here, only results count. The tuning process is considered as a black box. Racing approaches such as IRACE can be mentioned in this context.

The second direction, interactive tuning, tries to understand the behavior of the algorithm. It is based on statistics and generates small, smart data. Similar to microscopes in biology, tuning tools are used in the interactive approach as datascopes. Here, the journey is the reward. Model-based approaches such as SPO are representatives of this approach. They have a long tradition in experimentation and statistics and are based on classical response surface methodologies [25]. By integrating state-of-the-art modeling techniques, e.g., GP models and DACE, see [106], and tree-based methods, see [26], they provide powerful tools for the tuning and analysis of stochastic optimization algorithms.

Both directions are equally important. Some methods are interchangeable, and thus both directions can learn and benefit from each other.

Acknowledgment This work was supported by OWOS (FKZ: 005-1703-0011).

References

1. Adam, S.P., Alexandropoulos, S.-A.N., Pardalos, P.M., Vrahatis, M.N.: No Free Lunch Theorem: A Review, pp. 57–82. Springer International Publishing, Cham (2019). ISBN 978-3-030-12767-1, https://doi.org/10.1007/978-3-030-12767-1_5
2. Addis, B., Locatelli, M.: A new class of test functions for global optimization. J. Glob. Optim. **38**(3), 479–501 (2007) ISSN 0925-5001; 1573-2916/e
3. Andrei, N.: An unconstrained optimization test functions collection. Adv. Model. Optim. **10**(1), 147–161 (2008) ISSN 1841-4311/e
4. Ansótegui, C., Sellmann, M., Tierney, K.: A gender-based genetic algorithm for the automatic configuration of algorithms. In: Proceedings of Principles and Practice of Constraint Programming-CP 2009: 15th International Conference, CP 2009 Lisbon, 20–24 Sept 2009, p. 142. Springer, Berlin (2009)
5. Ansótegui, C., Malitsky, Y., Samulowitz, H., Sellmann, M., Tierney, K.: Model-based genetic algorithms for algorithm configuration. In: Twenty-Fourth International Joint Conference on Artificial Intelligence (2015)
6. Audet, C., Orban, D.: Finding optimal algorithmic parameters using derivative-free optimization. SIAM J. Optim. **17**(3), 642–664 (2006). ISSN 1052-6234; 1095-7189/e
7. Audet, C., Dang, K.-C., Orban, D.: Optimization of algorithms with OPAL. Math. Program. Comput. **6**(3), 233–254 (2014). ISSN 1867-2949; 1867-2957/e
8. Bäck, T.: Evolutionary Algorithms in Theory and Practice. Oxford University Press, New York (1996)
9. Barr, R., Hickman, B.: Reporting computational experiments with parallel algorithms: issues, measures, and experts' opinions. ORSA J. Comput. **5**(1), 2–18 (1993)
10. Barr, R., Golden, B., Kelly, J., Rescende, M., Stewart, W.: Designing and reporting on computational experiments with heuristic methods. J. Heuristics **1**(1), 9–32 (1995)
11. Barton, R.R.: Testing strategies for simulation optimization. In: Proceedings of the 19th Conference on Winter Simulation, WSC '87, pp. 391–401. ACM, New York (1987). ISBN 0-911801-32-4, http://doi.acm.org/10.1145/318371.318618
12. Bartz-Beielstein, T.: Experimental Research in Evolutionary Computation—The New Experimentalism. Natural Computing Series. Springer, Berlin (2006). ISBN 3-540-32026-1, http://dx.doi.org/10.1007/3-540-32027-X
13. Bartz-Beielstein, T.: How to create generalizable results. In: Kacprzyk, J., Pedrycz, W. (eds.) Springer Handbook of Computational Intelligence, pp. 1127–1142. Springer, Berlin (2015). ISBN 978-3-662-43504-5, http://dx.doi.org/10.1007/978-3-662-43505-2_56
14. Bartz-Beielstein, T., Preuss, M.: The future of experimental research. In: Bartz-Beielstein, T., Chiarandini, M., Paquete, L., Preuss, M. (eds.) Experimental Methods for the Analysis of Optimization Algorithms, pp. 17–46. Springer, Berlin (2010)
15. Bartz-Beielstein, T., Parsopoulos, K.E., Vrahatis, M.N.: Design and analysis of optimization algorithms using computational statistics. Appl. Numer. Anal. Comput. Math. **1**(2), 413–433 (2004)
16. Bartz-Beielstein, T., Lasarczyk, C., Preuss, M.: Sequential parameter optimization. In: McKay, B., et al. (eds.) Proceedings 2005 Congress on Evolutionary Computation (CEC'05), Edinburgh, pp. 773–780. IEEE Press, Piscataway (2005). ISBN 0-7803-9363-5, https://doi.org/10.1109/CEC.2005.1554761
17. Bartz-Beielstein, T., Chiarandini, M., Paquete, L., Preuss, M. (eds.): Experimental Methods for the Analysis of Optimization Algorithms. Springer, Berlin (2010). ISBN 978-3-642-02537-2, https://doi.org/10.1007/978-3-642-02538-9, http://www.springer.com/978-3-642-02537-2
18. Bartz-Beielstein, T., Branke, J., Mehnen, J., Mersmann, O.: Evolutionary algorithms. Wiley Interdiscip. Rev. Data Min. Knowl. Discov. **4**(3), 178–195 (2014). ISSN 1942-4795. https://doi.org/10.1002/widm.1124

19. Beiranvand, V., Hare, W., Lucet, Y.: Best practices for comparing optimization algorithms. Optim. Eng. **18**(4), 815–848 (2017)
20. Birattari, M., Stützle, T., Paquete, L., Varrentrapp, K.: A racing algorithm for configuring metaheuristics. In: Proceedings of the 4th Annual Conference on Genetic and Evolutionary Computation, GECCO'02, pp. 11–18. Morgan Kaufmann, San Francisco (2002). ISBN 1-55860-878-8, http://dl.acm.org/citation.cfm?id=2955491.2955494
21. Birattari, M., Yuan, Z., Balaprakash, P., Stützle, T.: Iterated F-race an overview. Technical report (2009)
22. Bischl, B., Wessing, S., Bauer, N., Friedrichs, K., Weihs, C.: MOI-MBO: multiobjective infill for parallel model-based optimization. In: International Conference on Learning and Intelligent Optimization, pp. 173–186. Springer, Berlin (2014)
23. Bongartz, I., Conn, A.R., Gould, N., Toint, P.L.: CUTE: constrained and unconstrained testing environment. ACM Trans. Math. Softw. **21**(1), 123–160 (1995). ISSN 0098-3500; 1557-7295/e
24. Box, M.J.: A comparison of several current optimization methods, and the use of transformations in constrained problems. Comput. J. **9**, 67–77 (1966). ISSN 0010-4620; 1460-2067/e
25. Box, G.E.P., Wilson, K.B.: On the experimental attainment of optimum conditions. J. R. Stat. Soc. Series B Methodol. **13**(1), 1–45 (1951). http://www.jstor.org/stable/2983966
26. Breiman, L.: Stacked regression. Mach. Learn. **24**, 49–64 (1996)
27. Buckley, A.G.: Algorithm 709: testing algorithm implementations. ACM Trans. Math. Softw. **18**(4), 375–391 (1992). ISSN 0098-3500, http://doi.acm.org/10.1145/138351.138378
28. Bussieck, M.R., Dirkse, S.P., Vigerske, S.: PAVER 2.0: an open source environment for automated performance analysis of benchmarking data. J. Glob. Optim. **59**(2–3), 259–275 (2014). ISSN 0925-5001; 1573-2916/e
29. Campelo, F., Takahashi, F.: Sample size estimation for power and accuracy in the experimental comparison of algorithms. J. Heuristics **25**(2), 305–338 (2019). ISSN 1572-9397, https://doi.org/10.1007/s10732-018-9396-7
30. Chen, C.H.: An effective approach to smartly allocate computing budget for discrete event simulation. In: Proceedings of the 34th IEEE Conference on Decision and Control, pp. 2598–2605 (1995)
31. Chiarandini, M., Goegebeur, Y.: Mixed models for the analysis of optimization algorithms. In: Bartz-Beielstein, T., Chiarandini, M., Paquete, L., Preuss, M. (eds.) Experimental Methods for the Analysis of Optimization Algorithms, pp. 225–264. Springer, Berlin (2010). ISBN 978-3-642-02537-2, https://doi.org/10.1007/978-3-642-02538-9, http://bib.mathematics.dk/preprint.php?id=DMF-2009-07-001
32. Cohen, P.R.: Empirical Methods for Artificial Intelligence. MIT Press, Cambridge (1995)
33. Coy, S.P., Golden, B.L., Runger, G.C., Wasil, E.A.: Using experimental design to find effective parameter settings for heuristics. J. Heuristics **7**(1), 77–97 (2000)
34. Crainic, T.: Parallel Metaheuristics and Cooperative Search, pp. 419–451. Springer International Publishing, Cham (2019). ISBN 978-3-319-91086-4, https://doi.org/10.1007/978-3-319-91086-4_13
35. Crowder, H.P., Dembo, R.S., Mulvey, J.M.: On reporting computational experiments with mathematical software. ACM Trans. Math. Softw. **5**(2), 193–203 (1979)
36. Daniels, S.J., Rahat, A.A., Everson, R.M., Tabor, G.R., Fieldsend, J.E.: A suite of computationally expensive shape optimisation problems using computational fluid dynamics. In: International Conference on Parallel Problem Solving from Nature, pp. 296–307. Springer, Berlin (2018)
37. De Jong, K.A.: An analysis of the behavior of a class of genetic adaptive systems. PhD thesis, University of Michigan (1975)
38. De Jong, K.: Parameter Setting in EAs: A 30 Year Perspective, pp. 1–18. Springer, Berlin (2007). ISBN 978-3-540-69432-8, https://doi.org/10.1007/978-3-540-69432-8_1

39. Doerr, C., Wagner, M.: Sensitivity of parameter control mechanisms with respect to their initialization. In: International Conference on Parallel Problem Solving from Nature (PPSN 2018), Coimbra. Lecture Notes in Computer Science, vol. 11102, pp. 360–372, Sept 2018. https://doi.org/10.1007/978-3-319-99259-4_29, https://hal.sorbonne-universite.fr/hal-01921055
40. Dolan, E.D., Moré, J.J.: Benchmarking optimization software with performance profiles. Math. Program. **91**(2), 201–213 (2002). http://link.springer.com/10.1007/s101070100263
41. Domes, F., Fuchs, M., Schichl, H., Neumaier, A.: The optimization test environment. Optim. Eng. **15**(2), 443–468 (2014). ISSN 1389-4420; 1573-2924/e
42. Eason, E.D.: Evidence of fundamental difficulties in nonlinear optimization code comparisons. In: Mulvey, J.M. (ed.) Evaluating Mathematical Programming Techniques, pp. 60–71. Springer, Berlin (1982). ISBN 978-3-642-95406-1
43. Eason, E., Fenton, R.: A comparison of numerical optimization methods for engineering design. J. Eng. Ind. **96**(1), 196–200 (1974)
44. Eiben, A.E., Smith, J.E.: Introduction to Evolutionary Computing. Springer, Berlin (2003). ISBN 3-540-40184-9, http://www.worldcat.org/title/introduction-to-evolutionary-computing/oclc/52559217
45. Eiben, A.E., Smit, S.K.: Parameter tuning for configuring and analyzing evolutionary algorithms. Swarm Evol. Comput. **1**(1), 19–31 (2011). https://doi.org/10.1016/j.swevo.2011.02.001, http://www.sciencedirect.com/science/article/pii/S2210650211000022
46. Eiben, A.E., Hinterding, R., Michalewicz, Z.: Parameter control in evolutionary algorithms. IEEE Trans. Evol. Comput. **3**(2), 124–141 (1999). citeseer.nj.nec.com/eiben00parameter.html
47. Floudas, C.A., Pardalos, P.M., Adjiman, C.S., Esposito, W.R., Gümüş, Z.H., Harding, S.T., Klepeis, J.L., Meyer, C.A., Schweiger, C.A.: Handbook of Test Problems in Local and Global Optimization, vol. 33. Kluwer Academic, Dordrecht (1999). ISBN 0-7923-5801-5/hbk
48. Forrester, A., Sóbester, A., Keane, A.: Multi-fidelity optimization via surrogate modelling. Proc. R. Soc. A Math. Phys. Eng. Sci. **463**(2088), 3251–3269 (2007). https://doi.org/10.1098/rspa.2007.1900
49. Ginsbourger, D., Le Riche, R., Carraro, L.: Kriging is well-suited to parallelize optimization. In: Computational Intelligence in Expensive Optimization Problems, pp. 131–162. Springer, Berlin (2010)
50. Goldberg, D.E., Deb, K.: A comparative analysis of selection schemes used in genetic algorithms. Foundations of Genetic Algorithms, vol. 1, pp. 69–93. Elsevier, Amsterdam (1991). https://doi.org/10.1016/B978-0-08-050684-5.50008-2, http://www.sciencedirect.com/science/article/pii/B9780080506845500082
51. Goldberg, D.E., Deb, K., Clark, J.H.: Genetic algorithms, noise, and the sizing of populations. Complex Syst. **6**, 333 (1992)
52. Gould, N., Scott, J.: A note on performance profiles for benchmarking software. ACM Trans. Math. Softw. **43**(2), 5 (2016). ISSN 0098-3500; 1557-7295/e, Id/No 15
53. Grefenstette, J.: Optimization of control parameters for genetic algorithms. IEEE Trans. Syst. Man Cybern. **16**(1), 122–128 (1986). ISSN 0018-9472, https://doi.org/10.1109/TSMC.1986.289288
54. Haftka, R.T.: Requirements for papers focusing on new or improved global optimization algorithms. Struct. Multidiscipl. Optim. **54**(1), 1–1 (2016). ISSN 1615-1488, https://doi.org/10.1007/s00158-016-1491-5
55. Haftka, R.T., Villanueva, D., Chaudhuri, A.: Parallel surrogate-assisted global optimization with expensive functions—a survey. Struct. Multidiscipl. Optim. **54**(1), 3–13 (2016). ISSN 1615-1488, https://doi.org/10.1007/s00158-016-1432-3
56. Hare, W., Wang, Y.: Fairer benchmarking of optimization algorithms via derivative free optimization. Technical report, Optimization-online (2010)
57. Harik, G.R., Lobo, F.G.: A parameter-less genetic algorithm. In: Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation - Volume 1, GECCO'99, pp. 258–265. Morgan Kaufmann, San Francisco (1999). ISBN 1-55860-611-4, http://dl.acm.org/citation.cfm?id=2933923.2933949

58. Hastie, T., Tibshirani, R., Friedman, J.: The Elements of Statistical Learning. Springer, Berlin (2001)
59. Hillstrom, K.E.: A simulation test approach to the evaluation of nonlinear optimization algorithms. ACM Trans. Math. Softw. **3**(4), 305–315 (1977). http://doi.acm.org/10.1145/355759.355760
60. Himmelblau, D.M.: Applied Nonlinear Programming. McGraw-Hill, New York (1972)
61. Hoos, H.H., Stützle, T.: Stochastic Local Search—Foundations and Applications. Elsevier, Amsterdam (2005)
62. Huang, D., Allen, T.T., Notz, W.I., Zeng, N.: Global optimization of stochastic black-box systems via sequential Kriging meta-models. J. Glob. Optim. **34**(3), 441–466 (2006)
63. Hutter, F., Babic, D., Hoos, H.H., Hu, A.J.: Boosting verification by automatic tuning of decision procedures. In: Proceedings of the Formal Methods in Computer Aided Design, FMCAD '07, pp. 27–34. IEEE Computer Society, Washington (2007). ISBN 0-7695-3023-0, https://doi.org/10.1109/FMCAD.2007.13
64. Hutter, F., Hoos, H.H., Leyton-Brown, K., Stützle, T.: ParamILS: an automatic algorithm configuration framework. Technical report (2009)
65. Hutter, F., Hoos, H., Leyton-Brown, K.: Sequential model-based optimization for general algorithm configuration. In: Learning and Intelligent Optimization, pp. 507–523 (2011). https://maanvs03.gm.fh-koeln.de/webstore/Classified.d/Hutt11a.d/Hutt11a.pdf
66. IBM Corporation: CPLEX's automatic tuning tool. Technical report, IBM (2014)
67. Jackson, R.H.F., Boggs, P.T., Nash, S.G., Powell, S.: Guidelines for reporting results of computational experiments. Report of the ad hoc committee. Math. Program. **49**(1), 413–425 (1990). ISSN 1436-4646, https://doi.org/10.1007/BF01588801
68. Jin, Y., Wang, H., Chugh, T., Guo, D., Miettinen, K.: Data-driven evolutionary optimization: an overview and case studies. IEEE Trans. Evol. Comput. **23**(3), 442–458 (2019). ISSN 1089-778X, https://doi.org/10.1109/TEVC.2018.2869001
69. Johnson, D.S., Aragon, C.R., McGeoch, L.A., Schevon, C.: Optimization by simulated annealing: an experimental evaluation. Part I, graph partitioning. Oper. Res. **37**(6), 865–892 (1989)
70. Johnson, D.S., Aragon, C.R., McGeoch, L.A., Schevon, C.: Optimization by simulated annealing: an experimental evaluation. Part II, graph coloring and number partitioning. Oper. Res. **39**(3), 378–406 (1991)
71. Johnson, D.S., McGeoch, L., Rothberg, E.: Asymptotic experimental analysis for the Held-Karp traveling salesman bound. In: Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, vol. 81, pp. 341–350 (1996)
72. Jones, D.R., Schonlau, M., Welch, W.J.: Efficient global optimization of expensive black-box functions. J. Glob. Optim. **13**(4), 455–492 (1998)
73. Jung, C., Zaefferer, M., Bartz-Beielstein, T., Rudolph, G.: Metamodel-based optimization of hot rolling processes in the metal industry. Int. J. Adv. Manuf. Technol. 1–15 (2016). ISSN 1433-3015, https://doi.org/10.1007/s00170-016-9386-6
74. Kennedy, J., Eberhart, R.C.: Particle swarm optimization. In: Proceedings IEEE International Conference on Neural Networks, pp. 1942–1948. IEEE, Piscataway (1995)
75. Kleijnen, J.P.C.: Statistical Tools for Simulation Practitioners. Marcel Dekker, New York (1987)
76. Kleijnen, J.P.C.: Design and Analysis of Simulation Experiments. Springer, New York (2008)
77. Kleijnen, J.P.C.: Design and Analysis of Simulation Experiments. International Series in Operations Research and Management Science. Springer International Publishing, New York (2015). ISBN 978-3-319-18087-8, https://books.google.de/books?id=Fq4YCgAAQBAJ
78. Kramer, O.: Evolutionary self-adaptation: a survey of operators and strategy parameters. Evol. Intell. **3**(2), 51–65 (2010). https://maanvs03.gm.fh-koeln.de/webstore/Classified.d/Kram10a.d/Kram10a.pdf
79. Lenard, M.L., Minkoff, M.: Randomly generated test problems for positive definite quadratic programming. ACM Trans. Math. Softw. **10**(1), 86–96 (1984). ISSN 0098-3500, http://doi.acm.org/10.1145/356068.356075

80. Liu, D., Zhang, X.: Test problem generator by neural network for algorithms that try solving nonlinear programming problems globally. J. Glob. Optim. **16**(3), 229–243 (2000). ISSN 0925-5001; 1573-2916/e
81. Lobo, F.G., Lima, C.F., Michalewicz, Z. (eds.): Parameter Setting in Evolutionary Algorithms. Studies in Computational Intelligence, vol. 54. Springer, Berlin (2007). ISBN 978-3-540-69431-1
82. Lopez-Ibanez, M., Dubois-Lacoste, J., Stützle, T., Birattari, M.: The irace package, iterated race for automatic algorithm configuration. Technical Report 2011-004, IRIDIA (2011)
83. McGeoch, C.C.: Experimental Analysis of Algorithms. PhD thesis, Carnegie Mellon University, Pittsburgh (1986)
84. McGeoch, C.C.: Toward an experimental method for algorithm simulation. INFORMS J. Comput. **8**(1), 1–15 (1996)
85. McGeoch, C.C.: Experimental algorithmics. Commun. ACM **50**(11), 27–31 (2007). ISSN 0001-0782, http://doi.acm.org/10.1145/1297797.1297818
86. McGeoch, C.C.: A Guide to Experimental Algorithmics, 1st edn. Cambridge University Press, New York (2012). ISBN 0521173019, 9780521173018
87. Miele, A., Tietze, J., Levy, A.: Comparison of several gradient algorithms for mathematical programming problems. Technical report, Rice University (1972)
88. Montgomery, D.C.: Design and Analysis of Experiments, 5th edn. Wiley, New York (2001)
89. Moré, J.J., Wild, S.M.: Benchmarking derivative-free optimization algorithms. SIAM J. Optim. **20**(1), 172–191 (2009). ISSN 1052-6234; 1095-7189/e
90. More, J.J., Garbow, B.S., Hillstrom, K.E.: Testing unconstrained optimization software. ACM Trans. Math. Softw. **7**(1), 17–41 (1981)
91. Mühlenbein, H.: How genetic algorithms really work : I. Mutation and hill climbing. In: Proc. 2nd Int. Conf. on Parallel Problem Solving from Nature. Elsevier, Amsterdam (1992). https://ci.nii.ac.jp/naid/10022158367/en/
92. Muñoz, M.A., Sun, Y., Kirley, M., Halgamuge, S.K.: Algorithm selection for black-box continuous optimization problems: a survey on methods and challenges. Inf. Sci. **317**, 224–245 (2015). ISSN 0020-0255, https://doi.org/10.1016/j.ins.2015.05.010, http://www.sciencedirect.com/science/article/pii/S0020025515003680
93. Nelder, J.A., Mead, R.: A simplex method for function minimization. Comput. J. **7**(4), 308–313 (1965)
94. Nell, C., Fawcett, C., Hoos, H.H., Leyton-Brown, K.: Hal: a framework for the automated analysis and design of high-performance algorithms. In: Coello, C.A.C. (ed.) Learning and Intelligent Optimization, pp. 600–615. Springer, Berlin (2011). ISBN 978-3-642-25566-3
95. Neumann-Brosig, M., Marco, A., Schwarzmann, D., Trimpe, S.: Data-efficient auto-tuning with Bayesian optimization: an industrial control study (2018). CoRR, abs/1812.06325, http://arxiv.org/abs/1812.06325
96. Parejo, J.A., Ruiz-Cortés, A., Lozano, S., Fernandez, P.: Metaheuristic optimization frameworks: a survey and benchmarking. Soft Comput. **16**(3), 527–561 (2012). ISSN 1433-7479, https://doi.org/10.1007/s00500-011-0754-8
97. Pavón, R., Díaz, F., Laza, R., Luzón, V.: Automatic parameter tuning with a Bayesian case-based reasoning system. A case of study. Expert Syst. Appl. **36**(2, Part 2), 3407–3420 (2009). ISSN 0957-4174, https://doi.org/10.1016/j.eswa.2008.02.044, http://www.sciencedirect.com/science/article/pii/S0957417408001292
98. R Core Team: R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing, Vienna (2018). https://www.R-project.org
99. Rardin, R., Uzsoy, R.: Experimental evaluation of heuristic optimization algorithms: a tutorial. J. Heuristics **7**(3), 261–304 (2001)
100. Rechenberg, I.: Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution. PhD thesis, Department of Process Engineering, Technical University of Berlin (1971)
101. Ridge, E.: Design of experiments for the tuning of optimisation algorithms. PhD thesis, The University of York (2007)

102. Ridge, E., Kudenko, D.: Tuning an Algorithm Using Design of Experiments, pp. 265–286. Springer, Berlin (2010). ISBN 978-3-642-02538-9, https://doi.org/10.1007/978-3-642-02538-9_11
103. Sacks, J., Welch, W.J., Mitchell, T.J., Wynn, H.P.: Design and analysis of computer experiments. Stat. Sci. **4**(4), 409–435 (1989)
104. Saltelli, A., Tarantola, S., Campolongo, F., Ratto, M.: Sensitivity Analysis in Practice. Wiley, New York (2004). ISBN 978-0-470-87095-2, https://doi.org/10.1002/0470870958
105. Saltelli, A., Ratto, M., Andres, T., Campolongo, F., Cariboni, J., Gatelli, D., Saisana, M., Tarantola, S.: Global Sensitivity Analysis. Wiley, New York (2008)
106. Santner, T.J., Williams, B.J., Notz, W.I.: The Design and Analysis of Computer Experiments. Springer, Berlin (2003)
107. Schagen, A., Rehbach, F., Bartz-Beielstein, T.: Model-based evolutionary algorithm for optimization of gas distribution systems in power plant electrostatic precipitators. Int. J. Gener. Storage Electricity Heat **9**, 65–72 (2018)
108. Schwefel, H.-P.: Evolutionsstrategie und numerische Optimierung. PhD thesis, Technische Universität Berlin, Fachbereich Verfahrenstechnik, Berlin (1975)
109. Schwefel, H.P.: Evolution and Optimum Seeking. Sixth-Generation Computer Technology. Wiley, New York (1995)
110. Sloss, A.N., Gustafson, S.: 2019 Evolutionary Algorithms Review (2019). http://arxiv.org/abs/1906.08870
111. Smit, S.K., Eiben, A.E.: Multi-problem parameter tuning using BONESA. In: Hao, J.K., Legrand, P., Collet, P., Monmarché, N., Lutton, E., Schoenauer, M. (eds.) Artificial Evolution, 10th International Conference Evolution Artificielle, pp. 222–233. Springer, Berlin (2011)
112. Sóbester, A., Leary, S.J., Keane, A.J.: A parallel updating scheme for approximating and optimizing high fidelity computer simulations. Struct. Multidiscipl. Optim. **27**(5), 371–383 (2004)
113. Tukey, J.W.: Exploratory Data Analysis. Addison-Wesley, Reading (1977)
114. Vodopija, A., Stork, J., Bartz-Beielstein, T., Filipič, B.: Model-based multiobjective optimization of elevator group control. In: Filipič, B., Bartz-Beielstein, T. (eds.) International Conference on High-Performance Optimization in Industry, HPOI 2018, Ljubljana, pp. 43–46, Oct 2018
115. Wolpert, D.H., Macready, W.G.: No free lunch theorems for optimization. IEEE Trans. Evol. Comput. **1**(1), 67–82 (1997)
116. Yeguas, E., Luzón, M., Pavón, R., Laza, R., Arroyo, G., Díaz, F.: Automatic parameter tuning for evolutionary algorithms using a Bayesian case-based reasoning system. Appl. Soft Comput. **18**, 185–195 (2014). ISSN 1568-4946, https://doi.org/10.1016/j.asoc.2014.01.032, http://www.sciencedirect.com/science/article/pii/S1568494614000519
117. Zheng, F., Simpson, A.R., Zecchin, A.C.: An efficient hybrid approach for multiobjective optimization of water distribution systems. Water Resourc. Res. **50**(5), 3650–3671 (2014). https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1002/2013WR014143

Quality-Diversity Optimization: A Novel Branch of Stochastic Optimization



Konstantinos Chatzilygeroudis, Antoine Cully, Vassilis Vassiliades, and Jean-Baptiste Mouret

1 Introduction

Optimization has countless uses in engineering, from designing mechanical parts [71] to controlling robots [26, 52]. In an ideal situation, the user of an optimization algorithm would write the cost function that corresponds to the problem at hand, select the right optimization algorithm, and get *the* perfect solution. Unfortunately, many real-world optimization problems are not easily captured by a simple cost function, which is why it is often required to add new terms to the cost functions and/or tune the parameters until the optimization algorithm finally reaches an acceptable solution. Consequently, optimization in engineering is often an iterative process that requires many runs of the optimizers and many changes of the cost function.

Moreover, even after fine-tuning the cost function, the optimized solution is rarely used as the final, optimal solution. In practice, optimization is most often used at the beginning of the design process to explore various options and examine

K. Chatzilygeroudis (✉)
Computer Technology Institute & Press "Diophantus" (CTI), Patras, Greece
e-mail: kchatzil@cti.gr

A. Cully
Adaptive & Intelligent Robotics Lab, Imperial College London, London, UK
e-mail: a.cully@imperial.ac.uk

V. Vassiliades
CYENS Centre of Excellence, Nicosia, Cyprus
e-mail: v.vassiliades@cyens.org.cy

J.-B. Mouret
Inria, CNRS, Université de Lorraine, LORIA, Nancy, France
e-mail: jean-baptiste.mouret@inria.fr

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_4

the trade-offs that are inherent to the domain [6]. This calls for algorithms that are designed as exploration tools more than as pure optimization tools.

Quality-Diversity (QD) optimization algorithms address this challenge: instead of searching for the optimum of the cost function, they provide a large set of high-performing solutions (typically a few thousands) that differ according to a few user-defined features of interest. The user can then pick the high-performing solutions that they deem as the most interesting according to their own knowledge, like esthetics or easiness of manufacturing. He/she can also use the resulting set to better understand how features of solutions influence performance. For instance, one of the features might have no influence on the cost function, or another one might need to be below a given threshold to attain acceptable performance.

For instance, QD algorithms have been used to find gait parameters for a 6-legged robot [10, 16, 23]. In that task, instead of finding parameters to go forward, a QD algorithm can find *in a single run* parameters so that the robot can reach any point in its vicinity (e.g., walking forward, backward, left, etc.). For each direction, the algorithm will find a different set of high-performing parameters; however, close directions are likely to have similar solutions, which means that it may be more efficient to optimize for all the directions simultaneously. In particular, if a set of parameters is tested and makes the robot turn right, it will be useless to go forward, but will be a promising solution to turn right. In addition, these parameters may be good "stepping stones" to go forward (or backward, or left, etc.), that is, a useful intermediate step in the optimization process.

Numerous QD algorithms have been proposed during recent years, which are mostly based on the principles of evolutionary algorithms. This chapter gives an overview of these algorithms and introduces several recent ideas to increase the data-efficiency, scale to high-dimensional spaces. Application examples are discussed all along the chapter.

2 Problem Formulation

We assume that the objective function returns both the fitness value $f_{\boldsymbol{\theta}}$ and a behavioral descriptor (or a feature vector) $\boldsymbol{b}_{\boldsymbol{\theta}}$ [55]:

$$f_{\boldsymbol{\theta}}, \boldsymbol{b}_{\boldsymbol{\theta}} \leftarrow f(\boldsymbol{\theta}) \tag{1}$$

The behavioral descriptor (BD) typically describes *how* the solution solves the problem, while the fitness value $f_{\boldsymbol{\theta}}$ quantifies *how well* it solves it. For example, the BD can be the curvature of a 3D design, its volume, or the trajectory of a robot, while the fitness values would be the aerodynamic drag, the energy consumption, or the distance to the target state.

Without loss of generality, we assume hereafter that the fitness function is maximized. Let us define by $\mathcal{B}$ the feature space, the goal in QD optimization is to find for each point $\boldsymbol{b} \in \mathcal{B}$ the parameters, $\boldsymbol{\theta}$, with the maximum fitness value:

Table 1 An example of discretized feature descriptor

BD		Fitness	Parameter values			
82	33	−52	−94	87	96	13
53	10	−95	85	−87	−90	−83
⋮		⋮	⋮			
−62	95	−86	−60	−30	−44	42

$$\forall \boldsymbol{b} \in \mathcal{B} \quad \boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}}$$
$$\text{s.t.} \quad \boldsymbol{b} = \boldsymbol{b}_{\boldsymbol{\theta}} \tag{2}$$

For instance, if the feature descriptors are discretized, the outcome of a QD algorithm can be a table. See Table 1 for an example:

When the BD is two-dimensional, this result is usually displayed as a colored image or heatmap.

At first sight, QD algorithms look like multitask optimization [55], that is, solving an optimization problem for each combination of features. However, first $\mathcal{B}$ can be continuous, which would mean an infinite number of problems; second, we do not know the BD before calling the fitness function, which explains why the QD problem can be viewed as a set of optimizations *constrained* by each BD.

The central hypothesis of QD algorithms is that solving this set of problems is likely to be faster if they are all solved together than by independent constrained optimizations. Intuitively, it is indeed likely that high-performing solutions for close feature descriptors will be close, therefore sharing information between the optimizations can be beneficial. In addition, independent constrained optimization would be especially wasteful in a black-box optimization context because a candidate solution that has not the right features would be discarded, whereas it could be useful for a different feature combination.

2.1 Collections of Solutions

The outcome of QD optimization is a set of solutions. This set of solutions, also called "collection," "archive," or "map," is expanded, improved, and refined during the optimization process. Each point in this collection represents a different "solution type" or "species." In practice, two solutions with similar behavioral descriptor will be considered to be of the same solution type and will compete together to be maintained in the collection. Necessarily, the notion of similarity is defined by using a hyper-parameter that sets the tolerance used to determine when two descriptors are different or similar. This hyper-parameter defines a sort of "resolution" in the behavioral descriptor space: only one solution will occupy a certain region of the space.

The simplest way to implement this segmentation of the BD space is by discretizing it into a grid, in which each cell of the grid corresponds to one type of solution (i.e., to one BD location). This approach is used by the MAP-Elites algorithm [53], one of the most used QD algorithm. In this case, the collection is a grid (or multidimensional array) and the goal of the algorithm is to fill every cell of that grid with the best possible solution. However, it is possible to avoid this discretization and replace it with distance thresholds or local density estimates (Sect. 3).

2.2 How Do We Measure the Performance of a QD Algorithm?

The overall performance of a QD algorithm is defined by the quality of the produced collection of solutions according to two criteria:

1. the performance of the solution found for each type of solutions (how much we have optimized).
2. the coverage of the behavior space (how much of the feature space is covered);

The first criterion (performance) is obvious to compute: depending on the application or use-case, we can compute the mean, median, or the sum of the individual fitness values in the collection. The second criteria (coverage) can be more challenging to evaluate when the behavior/feature space is not discretized. If we are in low-dimensional spaces, we can discretize the behavior space arbitrarily and compute the percentage of the bins filled by the algorithms [66]. Alternatively, if we are operating in a high-dimensional space, we can resort to density metrics, like the average distance of the k-nearest neighbors. A third option is to define a distance threshold between behavioral descriptions, and then compute a similar filling percentage as in the low-dimensional space case.

3 Optimizing a Collection of Solutions

In this section, we begin with the introduction of *Multidimensional Archive of Phenotypic Elites* (MAP-Elites) [53], and then present a modern categorization of QD algorithms [15] that makes it to define many variants of QD algorithms, including more advanced algorithms.

3.1 MAP-Elites

MAP-Elites takes inspiration from evolutionary algorithms: at each iteration, MAP-Elites alters copies of solutions that are already in the grid to form new solutions

(see Algorithm 1). The alterations are done with mutation and cross-over operators like in traditional evolutionary algorithms. The new solutions are evaluated and then potentially added to the cell corresponding to their BD. If the cell is empty, the solution is added to the grid. Otherwise, only the best solution is kept in the cell.

Algorithm 1 MAP-Elites algorithm

```
1: procedure MAP-ELITES([n_1, ..., n_d])
2:     A ⟵ create_empty_archive([n_1, ..., n_d])
3:     for i = 1 → G do                          ▷ Initialization: G random θ
4:         θ = random_solution()
5:         ADD_TO_ARCHIVE(θ, A)
6:     for i = 1 → I do                          ▷ Main loop, I iterations
7:         θ = selection(A)
8:         θ' = variation(θ)
9:         ADD_TO_ARCHIVE(θ', A)
10:    return A
11: procedure ADD_TO_ARCHIVE(θ, A)
12:    (p, b) ⟵ evaluate(θ)
13:    c ⟵ get_cell_index(b)
14:    if A(c) = null or A(c).p < p then
15:        A(c) ⟵ p, θ
```

MAP-Elites has been successfully employed in many domains. For instance, it has been used to produce: behavioral repertoires that enable robots to adapt to damage in a matter of minutes [17, 74], perform complex tasks [23], or even adapt to damage while completing their tasks [10]; morphological designs for walking "soft robots," as well as behaviors for a robotic arm [53]; neural networks that drive simulated robots through mazes [65]; images that "fool" deep neural networks [56]; "innovation engines" able to generate images that resemble natural objects [57]; and 3-D-printable objects by leveraging feedback from neural networks trained on 2-D images [48].

The main strength of MAP-Elites is its simplicity to understand and to implement. However, depending on the domain, it is sometimes difficult to discretize the feature/behavior space (for instance, in high dimensions or when the bounds are highly irregular). It is also possible to bias the selection in different way: the uniform selection among the elites of the original gives surprisingly good results but several ideas have been investigated to select parents in other ways that can make the convergence faster.

In the following section, we present a modern categorization of QD algorithms that make it possible to describe most QD algorithms within the same algorithmic framework.

3.2 A Unified Framework

Cully and Demiris [15] proposed a unified formulation of QD algorithms that see all QD algorithms as an instantiation of a single high-level algorithm (Algorithm 2). In this formulation, the main axes of variation of all QD algorithms are: (1) the type of container, that is, how the data are gathered and ordered into a collection, (2) the type of the selection operator, that is, how the solutions are selected to be altered in the next generation, and (3) the type of scores that are being computed in order for the container and the selection operator to work.

In particular, after a random initialization, the execution of a QD algorithm based on this framework follows four steps that are repeated:

- the selection operator produces a new set of individuals that will be altered in order to form the new batch of evaluations,
- the individuals are evaluated and their performance and BD are recorded,
- each of these individuals is then potentially added to the container, according to the solutions already in the collection,
- finally, several scores, like the novelty, the local competition, or the curiosity score, are updated.

These four steps repeat until a stopping criterion is reached (typically, a maximum number of iterations) and the algorithm outputs the collection stored in the container. In the following sections, we will detail different variants of the containers, the selection operators, and of the most widely used scores.

Algorithm 2 QD Optimization algorithm (I iterations)

$\mathcal{A} \leftarrow \emptyset$ ▷ *Creation of an empty container.*
for iter $= 1 \rightarrow I$ **do** ▷ *The main loop repeats during I iterations.*
  if iter $==$ 1 **then** ▷ *Initialization.*
    $\mathcal{P}_{\text{parents}} \leftarrow$ random() ▷ *The first* 2 *batches of individuals are generated randomly.*
    $\mathcal{P}_{\text{offspring}} \leftarrow$ random()
  else ▷ *The next controllers are generated using the container and/or the previous batch.*
    $\mathcal{P}_{\text{parents}} \leftarrow$ selection($\mathcal{A}$, $\mathcal{P}_{\text{offspring}}$) ▷ *Selection of a batch of individuals from the container and/or the previous batch.*
    $\mathcal{P}_{\text{offspring}} \leftarrow$ variation($\mathcal{P}_{\text{parents}}$) ▷ *Creation of a randomly modified copy of $\mathcal{P}_{\text{parents}}$ (mutation and/or crossover).*
  for each $\boldsymbol{\theta} \in \mathcal{P}_{\text{offspring}}$ **do**
    $\{f_{\boldsymbol{\theta}}, \boldsymbol{b}_{\boldsymbol{\theta}}\} \leftarrow f(\boldsymbol{\theta})$ ▷ *Evaluation of the individual and recording of its descriptor and performance.*
    if ADD_TO_CONTAINER($\boldsymbol{\theta}$, $\mathcal{A}$) **then** ▷ *"*ADD_TO_CONTAINER*" returns true if the individual has been added to the container.*
      UPDATE_SCORES(parent($\boldsymbol{\theta}$), Reward, $\mathcal{A}$) ▷ *The parent might get a reward.*
    else
      UPDATE_SCORES(parent($\boldsymbol{\theta}$), -Penalty, $\mathcal{A}$) ▷ *Otherwise, it might get a penalty.*
  UPDATE_CONTAINER($\mathcal{A}$) ▷ *Update of the attributes of all the individuals in the container (e.g. novelty score).*
return $\mathcal{A}$

3.2.1 Containers

The main purpose of a container is to gather all the solutions found so far into an ordered collection, in which only the best and most diverse solutions are kept.

One of the most popular container types in the literature is the *N-dimensional grid structure*, which is the one that MAP-Elites is using. In this container, we simply discretize the behavior space in a grid, where each cell of the grid corresponds to one type of solution. Originally, the MAP-Elites grid was built with only one solution per cell. Of course, one can imagine having more individuals per cell (e.g., [66] uses two individuals) in order to perform more complicated computations (e.g., for multi-objective optimization or noisy optimization [27, 39]). In high-dimensional behavior spaces, it is possible to use a Centroidal Voronoi Tessellation to define cells of identical volume regardless of the dimension (see Sect. 5.2).

An alternative container type is the *distanced-based archive*. In this type of containers, the solutions are kept in an unstructured array by using their behavior descriptor and the Euclidean distance. In essence, the user specifies a threshold parameter and a new individual is added to the archive (a) if it is "far away" from all other solutions in the archive (its Euclidean distance greater than the user-defined threshold), or (b) if it is better than its closest(s) neighbor(s). In contrast with the grid container presented previously, the descriptor space here is not discretized and the structure of the collection autonomously emerges from the encountered solutions. However, in practice this container type requires a slightly more sophisticated maintenance mechanism to avoid the "erosion effect" that may progressively remove solutions that are far from the rest of the collection in favor of solutions that are slightly closer but with a higher value, slowing down the overall optimization process.

3.2.2 Selection Operators

One of the keys for the success of QD algorithms, but also evolutionary algorithms in general, is the selection operators. The selection operators aim at answering the following question: given the current collection (or population), how do we sample or generate new individuals to be evaluated?

The most naive way of generating solutions is by *randomly sampling the parameter space*, that is, by not using the current collection. This is unlikely to be effective as it makes the QD algorithms identical to random search.

The original MAP-Elites implementation *randomly samples solutions from the ones already in the container*. This is a very simple strategy to implement and very computational efficient. However, one of its main drawbacks is that the selection pressure decreases as the number of solutions in the collection increases (the chance for a solution to be selected being inversely proportional to the number of solutions in the collection), which is likely to be ineffective with large collections.

Another interesting way of selecting new individuals is adopting *a score-based weighting for the random sampling*. In this way, we can insert more weight on more "interesting" individuals based on a specific score, and bias the selection pressure towards desired behaviors or more "interesting" individuals. In QD optimization, we aim at devising scores that will improve the collection, that is, to either discover new behaviors or optimize the already discovered ones (see Sect. 3.2.3).

So far, all the defined selection operators are operating on the stored container. In this case, the population of the QD algorithm is the container itself that *evolves* and improves over time. One can imagine having *multiple populations that evolve in parallel*. For example, Novelty Search with Local Competition (NSLC) [47] uses two distinct populations, one as a container storing information about novelty (they call it *novelty archive*) and one more traditional population storing information about performance and using it to perform the selection operation.

3.2.3 Population Scores

Traditionally, evolutionary algorithms consider only the *fitness value* (quality) to take decisions. Most of these approaches will struggle to identify diverse solutions, and might even fail at finding the global optimum. On the contrary, QD algorithms consider additional quantities in an attempt to better explore the behavior space (diversity).

One of the most widely used scores is the *novelty score*. The novelty score attempts to put higher values to solutions that are more different than other solutions, and thus forcing the algorithm to keep a diverse set of solutions instead of many similar ones. The most common formulation for the novelty score is the average distance of the k-nearest neighbors in the behavior space. This technique is introduced by the novelty search algorithms [46, 47].

Another idea is to reward individuals that produce offsprings that are novel enough or better than the individuals already in the container. In this way, the algorithm will prefer to sample individuals that will most likely produce new cells in the container or replace already occupied cells. The *curiosity score* attempts to do this by modeling the probability of an individual to generate offsprings that will be added to the container. One practical implementation of the curiosity score (see Algorithm 2) is to begin with zero curiosity score for all individuals, and then each time that one of their offsprings gets added to the container, their curiosity score increases, whereas it decreases each time their offsprings do not enter the container.

Finally, Go-Explore [24, 25] introduced the concept of attempting to *expand the frontier of the search* by promoting the selection of newly discovered individuals. The intuition is that a newly discovered individual will most likely contain an interesting behavior that can potentially lead to novel regions of the search space. The authors practically implement this idea by introducing a score that is based on visit and selection counters.

3.3 Considerations of Quality-Diversity Optimization

Results from [15] indicate that the usage of curiosity score (see Sect. 3.2.3) in a QD algorithm seems to be an effective way of keep exploring interesting regions of the behavior space while also locally optimizing the solutions. The performance of QD algorithms with the curiosity score both in distanced-based and grid-based containers was consistently better than many other variants [15]. In particular, this type of algorithms were able to learn behavior repertoires for a six-legged robot to walk in any direction or walk in a straight line in many different ways. These results showcase that relying on individuals with a high-propensity to generate individuals that are added to the collection is a promising selection heuristic.

Another quite interesting takeaway is the fact that using the uniform random selection operator (over the container), MAP-Elites' selection operator, is very effective both when used in distance-based and grid-based containers (the original MAP-Elites implementation) [15, 17, 53]. This showcases the strength of selecting candidates to reproduce from the elites (contained in the archive), rather than randomly generating new ones.

Additionally, Novelty Search with Local Competition [47] is less effective than curiosity-based QD instances and QD instances with the MAP-Elites' selection operator [15]. This finding advocates that evolving the whole collection of solutions is more effective than splitting it in multiple populations with different characteristics and functionalities.

Finally, the distance-based containers produce containers with smaller numbers of individuals. This of course depends on the choice of the threshold for the distance comparisons, but also highlights the need for better structure of the containers as it is often not very easy to tune these parameters. In Sect. 5.2, we discuss different types of containers to handle some of these limitations.

4 Origins and Related Work

4.1 Searching for Diverse Behaviors

Quality-Diversity algorithms mainly originate from the desire to evolve multiple and diverse behaviors in the evolutionary robotics community [46, 47, 54]. In particular, they build on top of Novelty Search (NS) [46], and Novelty Search with Local Competition (NSLC) [47] algorithms, which propose to search for novel solutions instead of high-performing ones. This proved to be particularly interesting to escape from deceptive regions in the search space (local optima) and reach eventually high-performing solutions [54]. Searching for novel solutions is done by rewarding solutions that are different from the previously encountered ones thanks to the *novelty score* (described in Sect. 3.2.3). This score is implemented in practice by maintaining an archive, called the novelty archive, of the previously encountered

solutions, which is then used to compute the average distance in the BD space between the contained solutions and any newly generated one. The archive is mainly used as a tool to compute the novelty score, while the actual outcome of the algorithms is their final population (like most evolutionary algorithms). It is also important to note that the archive was designed to quantify the coverage of the BD space (and thus compute the novelty score), not to produce a collection of high-performing and diverse solution.

Cully and Mouret proposed in the Behavioral-Repertoire Evolution algorithm (BR-Evolution) [16] to consider the novelty archive of NSLC as the outcome of the algorithm by introducing mechanisms to only keep the best-performing solutions by replacing them when a better one is found. This idea of building a large collection of solutions that produce different behaviors while maximizing the performance of each type of behaviors is one of the very first instances of Quality-Diversity algorithms as known today. At the same time, Mouret et al. designed a simple algorithm to plot a figure showing the distribution of high-performing solutions over a given feature space (illuminating the fitness landscape) [11]. Surprisingly, this simple algorithm, named later MAP-Elites [53] was in practice very effective in evolving behavioral repertoires like the BR-Evolution algorithm [17]. Shortly after, the concept of generating a collection of diverse and high-performing solutions has been formalized and named Quality-Diversity [65, 66].

4.2 Connections to Multimodal Optimization

Traditionally, the focus of optimization was to find a single, globally optimal solution of the objective function (Fig. 1a). It is often the case, however, that (1) the objective function is highly nonlinear, which might cause even sophisticated gradient-free algorithms, such as evolution strategies, to converge to local optima [67], and (2) the user would like from the optimization algorithm to return all local optima in order to choose between them (for example, this could be the case in engineering or design problems [45, 70]). Multimodal optimization (MMO) algorithms (e.g., see [18, 33, 35, 51, 62, 63, 68, 72, 80]) seek to address these issues, by employing various diversity maintenance techniques, also known as *niching* methods, with the aim to return multiple solutions that correspond to the peaks of the search space (Fig. 1b).

Niching has a long history in evolutionary computation. One of the earliest attempts was the *preselection* method [9] in which an offspring replaces its least fit parent, if it has higher fitness. *Crowding* [19] was the first to propose the use of distances (to the best of our knowledge). *Fitness sharing* [33] assumes that fitness is a precious resource that is shared among neighboring individuals; therefore, it reduces the fitness of individuals in densely populated regions. *Clearing* [62] is a technique that has the same inspiration as sharing, but rather than lowering the fitness, it removes less fit individuals from the neighborhoods. In *restricted tournament selection* [35] an offspring competes with its closest individual in a

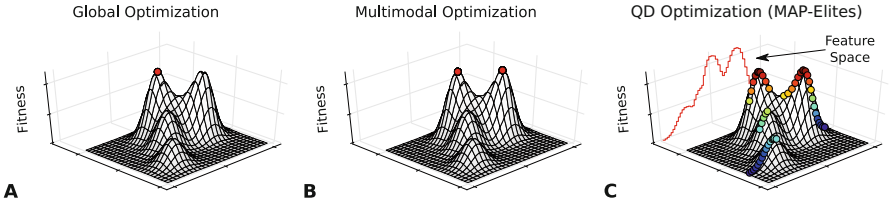


Fig. 1 Difference between global, multimodal, and QD optimization. The goal of global optimization algorithms is to find a single global optimum of the objective function (**a**). Multimodal optimization (MMO) algorithms aim to return multiple optima (**b**). QD optimization algorithms, such as MAP-Elites (**c**), discover significantly more solutions, each one being the highest-performing of a local neighborhood defined in some *feature space* of interest. For finding the solutions of the function illustrated, we used (**a**) Covariance Matrix Adaptation Evolution Strategies [34], (**b**) Restricted Tournament Selection [35], and (**c**) MAP-Elites [53]. Figure adapted from [78]

randomly selected sample of the population. Clustering [80] and multi-objective optimization [21] were also proposed (among others) as a way to maintain diversity. In addition, niching techniques have been proposed for various nature-inspired optimization algorithms, such as particle swarm optimization [3]. A comprehensive survey is outside the scope of this chapter; however, the interested reader can refer to [18, 63].

In order to better highlight the similarities and differences between global and multimodal optimization (assuming maximization) let us formally define their objectives. Global optimization can be written as

$$\boldsymbol{\theta}_g^* = \arg\max_{\boldsymbol{\theta} \in \Theta} f(\boldsymbol{\theta}) \tag{3}$$

or in set-builder notation:

$$\left\{ \boldsymbol{\theta}_g^* \in \Theta : f(\boldsymbol{\theta}_g^*) > f(\boldsymbol{\theta}), \forall \boldsymbol{\theta} \in \Theta \right\} \tag{4}$$

where Θ is the parameter space. In other words, global optimization algorithms aim to return a set that contains a single solution which is the globally optimal one. MMO can typically be expressed as

$$\left\{ \boldsymbol{\theta}_l^* \in \Theta : f(\boldsymbol{\theta}_l^*) > f(\boldsymbol{\theta}), \forall \boldsymbol{\theta} \in \Theta, d(\boldsymbol{\theta}, \boldsymbol{\theta}_l^*) < \epsilon, \epsilon > 0 \right\} \tag{5}$$

where d is a distance function; in other words, MMO aims to return the set of all locally optimal solutions, typically in parameter space.

Quality-Diversity optimization on the other hand aims at optimizing a different objective function that returns both a fitness value and a *behavior descriptor* (see Sect. 2 and Eqs. (1), (2)). Alternatively, it can be expressed as

$$\left\{ \boldsymbol{\theta}_{QD}^* \in \Theta : f(\boldsymbol{\theta}_{QD}^*) > f(\boldsymbol{\theta}), \forall \boldsymbol{\theta} \in \Theta, d(b(\boldsymbol{\theta}), b(\boldsymbol{\theta}_{QD}^*)) < \epsilon, \epsilon > 0 \right\} \tag{6}$$

where $f : \Theta \to \mathbb{R}$, and $b : \Theta \to \mathcal{B}$. Observing Eq. (6), we can easily identify that the main difference of QD from multimodal optimization is in the focus on finding the optimal parameters for each point in the behavior space, as well as returning many more points than optima (Fig. 1c).

It is clear by now that QD algorithms attempt to solve a different problem than more traditional optimization techniques. It should also be obvious that it might be possible to use off-the-shelf (local or global) optimizers with restart procedures (or in parallel) to solve the multimodal optimization problem, however, not the QD problem. For example, if we instantiate parallel hill climbers from uniformly spread random initial points, they might return the optima, however (1) some of them might return the same solution, and (2) the whole set of solutions will most probably *not* be as diverse as the one returned by QD optimization algorithms. Two questions arise now: (1) Can we use multimodal optimization algorithms to solve the QD problem? (2) Can we use QD algorithms to solve the multimodal optimization problem?

Although there are not many works in the literature that investigate these questions, it has been shown that some multimodal optimization algorithms can perform as well as QD algorithms if set to compare distances in behavior space [76], while others fail at doing so. In particular, the clearing method [62] was able to solve QD problems, whereas Restricted Tournament Selection [35] was not able to do so, mainly because of its strong focus on performance (rather than diversity) which makes it even lose certain local optima (Fig. 1b). This showcases the need for algorithms that attempt to solve the specific QD problem, and that the QD problem cannot be generically solved by other types of optimization methods. There are, however, a lot of intuitions to be taken from the multimodal optimization literature to improve QD algorithms.

Additionally, it is shown that QD algorithms can also work in high-dimensional parameter space [77, 78], and thus QD algorithms can be used to solve multimodal optimization problems. Typically, QD algorithms return many more solutions than the optima of the underlying search space, therefore, finding the optima in the returned set of solutions could potentially be done using some filtering technique (such as the nearest better clustering heuristic [64]).

4.3 Connections to Multitask Optimization

As introduced in Sect. 2, QD algorithms assume that the fitness function $f(\boldsymbol{\theta})$ returns both the fitness value f_θ and a behavioral descriptor $\boldsymbol{b}_\theta$:

$$f_\theta, \boldsymbol{b}_\theta \leftarrow f(\boldsymbol{\theta}) \tag{7}$$

By contrast, multitask optimization considers a fitness function that is parameterized by a task descriptor $\boldsymbol{\tau}$ and returns the fitness value:

$$f_{\theta,\tau} \leftarrow f(\boldsymbol{\theta}, \boldsymbol{\tau}) \tag{8}$$

The task descriptor might describe, for example, the morphology of a robot or the features of a game level; it is typically a vector of numbers that describes the parameters of the task.

The overall objective is to find, for each task $\boldsymbol{\tau}$, the solution $\boldsymbol{\theta}_{\tau}^{*}$ with the maximum fitness:

$$\forall \boldsymbol{\tau} \in T, \boldsymbol{\theta}_{\tau}^{*} = \text{argmax}_{\boldsymbol{\theta}} \big(f(\boldsymbol{\theta}, \boldsymbol{\tau}) \big) \tag{9}$$

To our knowledge, only a few algorithms have been proposed for multitask optimization, mainly in the framework of Bayesian optimization [61]. However, the MAP-Elites algorithm was recently extended to solve multitask optimization problems [55]. The general idea is to select the task $\boldsymbol{\tau}$ in the neighborhood of the parents, selected using the standard MAP-Elites procedure (uniform selection from the archive). The results show that Multitask-MAP-Elites can outperform independent optimizations with the CMA-ES algorithm [34], especially in hard optimization problems, most probably because it can achieve a more global search by looking at all the tasks simultaneously [55].

5 Current Topics

5.1 Expensive Objective Functions

QD algorithms are designed for non-convex, black-box functions with many peaks. As such, they typically assume that the objective function can be queried millions of times. For instance, MAP-Elites is typically given a budget of 20 million evaluations to find about 15,000 effective gaits for a 6-legged robot, with 36 parameters that define the gait [10, 17]. Unfortunately, many interesting engineering problems, for example aerodynamics optimization, require simulating each candidate solution for minutes or even hours: in these engineering problems, calling the objective function millions of times is not possible, even when parallelizing on large clusters and multicore computers.

This challenge is common for all black-box optimization algorithms, if not for all the optimization algorithms. In such cases, the traditional approach is to learn a *surrogate model* of the objective function [4, 59], that is a data-driven approximation of the objective function that can be used in lieu of the objective function.

The most popular theoretical framework for surrogate-based optimization is currently "Bayesian optimization" [7, 69]. Most instantiations model the objective function with Gaussian processes [79] because (1) they are designed to give uncertainty estimates and (2) their explicit smoothness assumption allows them to make accurate predictions when little data is available. Once the model is defined, an *acquisition function* is used to select the next solution to evaluate

on the expensive objective function; this function typically uses the uncertainty estimates to balance exploration—trying candidates in uncertain regions—and exploitation—trying candidates that are in the most promising regions according to the approximate model. In essence, Bayesian optimization loops over three steps: (1) finding the optimum of the acquisition function, which is a non-convex optimization problem with a "cheap" objective function, (2) evaluating the selected solution on the expensive function, and (3) updating the model with the new point. The model is usually initialized with a few candidates that are randomly chosen and evaluated on the expensive function.

The concepts of Bayesian optimization are easy to transfer to QD algorithm: the same models and the same model/optimization loop can be used. The only difference lies in the acquisition function (and how it is optimized), since the algorithm is not trying to find the optimum of a function anymore.

In the first experiments with surrogate-based QD algorithms, Gaier et al. [29–31] took inspiration from the Upper Confidence Bound (UCB) [12, 73], which is a simple but successful acquisition function in Bayesian optimization [7, 69]:

$$\text{UCB}(\boldsymbol{\theta}) = \mu(\boldsymbol{\theta}) + \beta\sigma(\boldsymbol{\theta}) \tag{10}$$

where $\mu(\boldsymbol{\theta})$ is the mean prediction according to the Gaussian processes (the surrogate model), $\sigma(\boldsymbol{\theta})$ is the prediction of the variance (which represents the uncertainty here). Intuitively, the optima of the UCB function are regions that are predicted as having a high predicted value ($\mu(\boldsymbol{\theta})$) and a high uncertainty ($\sigma(\boldsymbol{\theta})$). β tunes the exploration-exploitation trade-offs (a large β will favor candidates that have a high uncertainty, a small β the candidates that have the highest predictions).

In Bayesian optimization, the algorithm would optimize the UCB according to the model and evaluate the optimum solution on the true objective function. However, in QD algorithms there is no single, most promising solution. Instead, Gaier et al. used MAP-Elites with the UCB as the objective function. This gives an "acquisition map" instead of a maximum of an acquisition function, that is, MAP-Elites outputs the candidate with best UCB value for each bin. In the spirit of MAP-Elites, and because no bin is considered more important than the others, Gaier et al. select candidates to be evaluated on the expensive function uniformly from this "acquisition map."

The resulting algorithm, called Surrogate-Assisted Illumination (SAIL), has been evaluated on the optimization of airfoils (minimizing drags for a given lift) [29–31]. The results show that with the same number of evaluations required by CMA-ES to find a near-optimal solution in a single bin (without a surrogate), SAIL finds solutions of similar quality in every bin (625 bins in these experiments); in addition, when CMA-ES is used with a new surrogate for each bin, it requires an order of magnitude more evaluations than SAIL. Gaier et al. also showed promising results in a more complex three-dimensional aerodynamic optimization in [31]. In these experiments, the authors assumed that the bin was known from the values of a candidate solution (which is true in the design problem that they explored), but

they suggest that the mean prediction of a second GP can be used to compute the coordinates of the bin when creating the acquisition map (the variance of this prediction would be ignored).

In recent work, Kent and Branke proposed a different acquisition function that captures in a single value the expected improvement to the whole map, called the Expected Joint Improvement of Elites (EJIE) [41]. Their starting point is the Expected Improvement (EI) acquisition function, which is another popular acquisition function in Bayesian optimization [37]. However, the expected improvement is defined relative to the best known value, which does not make sense in a QD algorithm. Instead, Kent and Branke propose to compute the expected improvement for each niche, then sum all the values, so that the acquisition function is maximum for points that are likely to provide large improvements to the objective value of one or more niches. In addition, they note that it is usually not possible to know the cell from the values of the candidate solution (a hypothesis that was made in the experiments with the SAIL algorithm): a second Gaussian process is needed to predict the bin for each candidate. This second process is integrated in the Expected Joint Improvement of Elites (EJIE) by weighting the expected improvement by the probability for a candidate to be in a given cell:

$$\text{EJIE}(\boldsymbol{\theta}) = \sum_{i=1}^{C} P(\boldsymbol{\theta} \in c_i) EI_{c_i}(\boldsymbol{\theta}) \tag{11}$$

where i is the bin index, $P(\boldsymbol{\theta} \in c_i)$ is the probability of $\boldsymbol{\theta}$ to be in bin c_i (computed with a Gaussian process that models the features), and $EI_{c_i}(\boldsymbol{\theta})$ is the expected improvement for the bin c_i (computed with a Gaussian process that models the objective function). For now, the EJIE acquisition gave promising results for a one-dimensional benchmark problem [41] and more work is needed to compare it to SAIL.

An interesting side-effect of modeling the objective function with a surrogate model is that it becomes easy to increase the resolution of the map by running a QD algorithm using only the models, which can usually be achieved in minutes.

5.2 High-Dimensional Feature Space

The grid-based container approach of MAP-Elites has various benefits, such as conceptual simplicity, easy implementation, and it can form the basis for both quantitative QD metrics (e.g., number of filled bins, or the QD score [65]), as well as qualitative evaluation (by visual inspection of a 2D map). For creating the grid, the user needs to provide a number of discretization intervals per feature dimension. This, however, has the drawback of not scaling to high-dimensional feature spaces, as the number of bins increases exponentially with the feature dimensions. For instance, for a 50-dimensional space and 2 discretizations per dimension, MAP-Elites would create an empty matrix of 1.13×10^{15} cells that requires 4 petabytes of

memory (assuming 4 bytes for the pointer of each cell). This motivates the question whether grid-based containers can be used in high-dimensional feature spaces.

Vassiliades et al. [78] proposed an extension of MAP-Elites that addresses its dimensionality limitation using a tool from computational geometry known as a Centroidal Voronoi Tessellation (CVT) [22]. The key insight is that MAP-Elites defines the feature space using a bounding box that contains well-spread rectangular regions (Fig. 2 left). A similar partitioning can be achieved using a CVT with the important difference that the number of regions is explicitly controlled, while the resulting regions have a convex polygonal shape (Fig. 2 right).

The resulting CVT-MAP-Elites algorithm [78], requires a first, offline step (before QD optimization) to compute an *approximate* CVT based on a user-provided number k which defines the capacity of the container. The approximate CVT computation typically involves creating a dataset of $K >> k$ uniformly distributed random points in the bounded feature space, and using the k-means clustering algorithm [50] on this dataset to find k centroids [38] (Algorithm 3). If K is large enough, the k centroids become well spread in the bounding volume. Typically as the number of dimensions increases so should K; however, the approximate nature of the algorithm, as well as the large number of solutions requested (k) by QD algorithms makes the algorithm perform well. For instance, for finding 10,000 effective gaits for a 6-legged robot, Vassiliades et al. [78] used as feature space, subsets of the 36 parameters that define the gait [17] (i.e., 12, 24, and 36), and demonstrated that CVT-MAP-Elites has the same performance irrespective of the dimensionality, in contrast to MAP-Elites.

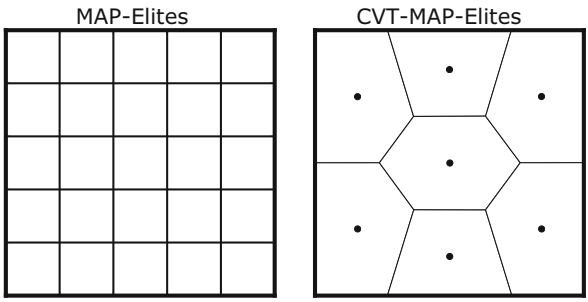


Fig. 2 MAP-Elites (left) partitions the feature space using a number of bins (25 in this example), k, that is computed by the number of discretization intervals per dimension requested by the user (i.e., $k = \prod_{i=1}^{d} n_i$, where d is the dimensionality of the feature space and n_i is the discretization interval of dimension i). CVT-MAP-Elites (right) has explicit control over the number of bins (7 in this example) irrespective of the dimensionality of the feature space. Figure taken from [78]

Algorithm 3 CVT approximation

1: **procedure** CVT(k, K)
2: $D \longleftarrow$ sample_points(K) ▷ K random samples
3: $C \longleftarrow$ kmeans(D, k) ▷ cluster dataset D using k centroids
4: **return** centroids C

It is important to note that special care needs to be taken when the high-dimensional feature space is defined by sequences. Vassiliades et al. [78] report experiments with a simulated maze-navigating mobile robot, and feature spaces of up to 1000 dimensions composed by trajectories of the robot's (x, y) location. In order for CVT-MAP-Elites to work, it needs representative centroids, and sampling from a uniform distribution (Algorithm 3 line 2) would not work, as it assumes that the various dimensions are independent. For these experiments, the authors [78] used knowledge about the robot's physical constraints (i.e., it can move at most 2 units and it cannot exceed the bounds of the maze) in order to effectively sample random trajectories that cover the space well.

Another related issue (that can also apply to lower dimensional feature spaces) is when the bounds of the various feature space dimensions are not known *a priori*. If wrong values are used, the performance of (CVT-)MAP-Elites might deteriorate as it will not be able to fill the grid (the size of the bins would either be too small or too big). A natural way of dealing with this issue is to let the feature descriptors of the sampled solutions define these bounds. Variants of MAP-Elites and CVT-MAP-Elites have explored this direction [77] and allow for the expansion of the bounding box that defines the feature space.

Archive-based containers can naturally be used when the feature space is high-dimensional, as they calculate distances between the sampled solutions. However, algorithms that use such containers, for example, NSLC [47], often come with various parameters that need to be tweaked for the task at hand. An approach called Cluster-Elites [77], that blends ideas from CVT-MAP-Elites and archive-based containers, allows the generation of centroids in non-convex spaces based on the sampled solutions.

5.3 Learning the Behavior Descriptor

Defining the behavioral descriptor space remains a challenging and crucial aspect in QD algorithms, as this decision determines the shape of the produced collections. It often requires a certain level of expertise or prior knowledge on the task at hand to know what are the features that the solutions can exhibit to define the corresponding behavioral descriptor space. However, in certain situations, this prior knowledge is not available or a user might target specific subsets of the possible solutions according to some conditions that are not easy to define in practice. Several pieces of work have proposed solutions to make easier and more automatic the definition of the behavioral descriptor using learning algorithms.

Sometimes the range of possible or desired types of solutions is known in advance, but there is no easy way to programmatically encode this knowledge into a low-dimensional BD definition. This is, for instance, the case when one wishes to generate solutions resembling a set of examples that has been obtained from a

different process. A solution to address this problem is to collect all the possible features of each example into a dataset which is then used to train a dimensionality reduction algorithm, such as an auto-encoder or principal component analysis. The outcome of this training is a low-dimensional latent space that captures the relations and similarities of the different examples and that can serve as a behavioral descriptor space. The encoder in the case of the auto-encoder (or the projector for the PCA) algorithm can project any new solution in the descriptor space. With this learned behavioral descriptor space, a QD algorithm produces a collection of solution that covers the latent space and that maximize a fitness function. The fitness function can be totally unrelated to the dimensionality reduction algorithm, or it can be defined as the reconstruction error of the algorithm, to encourage the QD algorithm to generate solutions that look similar to the example set. A schematic illustration of this approach is provided in Fig. 3a.

For example, this approach has been used to teach a robot to execute trajectories that resemble hand-written digits. Finding a way to characterize an arbitrary trajectory into a low-dimensional behavioral descriptor capturing the diversity of hand-written digit is particularly challenging. To side-step this challenge, Cully

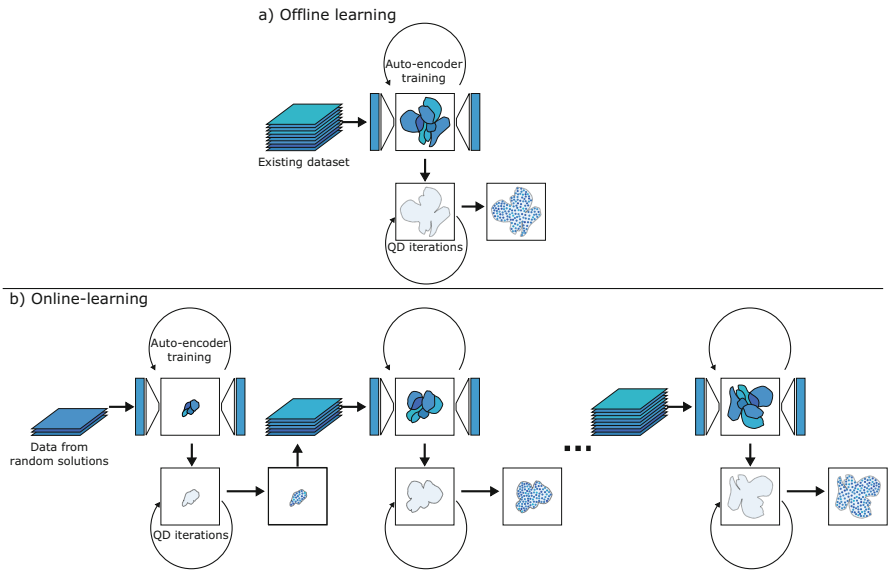


Fig. 3 Different approaches to learning the behavioral descriptor. (**a**) An existing dataset can be used to train an auto-encoder (offline) and define a latent representation of the data. This latent space is then used as the behavioral descriptor space and a QD algorithm can build a collection of solutions filling this latent space. (**b**) Alternatively, the learning of the behavioral descriptor can be achieved during the QD optimization process. Starting from randomly generated solutions, the auto-encoder is trained and the produced latent space used a behavioral descriptor. Then, the several QD steps are executed to generate new solutions in the latent space, which increase the amount of data available to train the auto-encoder. The training of the auto-encoder is then extended and the process can repeat until convergence of both the QD algorithm and the auto-encoder

and Demiris [14] employed an existing dataset of hand-written digits (the well-known MNIST dataset [44]) and trained an auto-encoder to find a low-dimensional latent space that captures the most prominent features of the different digits. This latent space can then be used as the behavioral descriptor space and QD algorithms produce a collection of solutions that covers the space of the learned features. As a result, this enabled a robotic arm to learn how to draw digits without having to manually define what are the main features of the different digits.

Instead of using an existing dataset, an alternative is to directly use the solutions generated by the QD algorithm. In this case, the randomly generated samples used in the initialization of the QD algorithm are used to train the dimensionality reduction algorithm. The corresponding latent space definition becomes the behavioral descriptor space for the execution of a few steps of the QD process. During these steps, the number of solutions in the collection grows, which has the direct effect of accumulating new samples that can be used to extend the training of the dimensionality reduction algorithm and thus change the definition of the latent space. The latent space is then redefined to better include the new solutions that have been discovered and redistribute them in the latent space according to their high-level features. This process can be repeated periodically to enable both the QD algorithm and the auto-encoder to converge simultaneously. A schematic illustration of this approach is provided in Fig. 3b.

The AURORA algorithm (AUtonomous RObots that Realise their Abilities, [13]) uses this approach to enable robots to discover large collections of diverse skills without any prior knowledge on the capabilities of the robots. After each solution evaluation, the trajectories of objects in the environment are recorded and collected for the training of the auto-encoder. This resulted in a collection of solutions to interact differently with objects in the robot's environment. This concept has been extended in the TAXONS [60] algorithm by using raw images from cameras placed on top of the robot and to discover how to move or interact in the environment. A similar concept has also been used in the DeLeNoX algorithm [49] to evolve a diversity of spacecraft sprite for video games using the Novelty-Search algorithm.

5.4 Improving Variation Operators

The state-of-the-art black-box optimization algorithms exploit the distribution of the highest-performing candidate solutions [36]; most notably, CMA-ES [34] computes a covariance matrix to sample the next population in the most promising direction. By contrast, the first QD algorithms used simple Gaussian variation (mutation) to generate new solutions, because they were focused on the selective pressure. However, the success of modern black-box optimization algorithms suggests that there is much to be improved by exploiting the distribution of the best candidate solutions in the variation operators.

Vassiliades and Mouret investigated how the genotypes of the archives of MAP-Elites are distributed in several benchmark tasks [75]: while the elites are well spread in the feature space (by construction), they occupy a specific volume in the genotype space, that they called the "Elite Hypervolume." This should not be surprising because high-performing solutions often share "similar recipes." This echoes the high number of genes that are shared by species that live in very different niches, that is, that are elites of their ecological niche: for example, fruit flies and humans share about 60% of their genes [1] (which correspond to neurons, muscle cells, etc.).

A straightforward way to exploit this hypervolume is to use the classic cross-over operator from evolutionary computation. The general idea is that a good variation operator is an operator that is likely to produce a high-performing solution from one or several existing high-performing solutions. In a QD algorithm, this means that we want to select one or several elites, which are in the current approximation of the elite hyper-volume, and create a solution in the elite hypervolume. If the hypervolume is convex, then any weighted average of elites will be in the hypervolume (Fig. 4); if the volume is not convex, it might be locally convex and a "blend" of elites is still more likely to be in the elite hypervolume than not.

As a consequence, cross-over is surprisingly effective in QD algorithm, whereas its utility is more controversial in black-box optimization. For instance, evolutionary strategies like CMA-ES [34] do not use it all. Classic cross-over operators like SBX often work well [20], but Vassiliades and Mouret proposed a simplified cross-over that gives good results in many QD problems, called "directional variation". Given two random elites $\boldsymbol{\theta}_i^{(t)}$ and $\boldsymbol{\theta}_j^{(t)}$, a new candidate solution $\boldsymbol{\theta}_i^{(t+1)}$ is generated by:

$$\boldsymbol{\theta}_i^{(t+1)} = \boldsymbol{\theta}_i^{(t)} + \sigma_1 \mathcal{N}(0, \mathbf{I}) + \sigma_2 \left(\boldsymbol{\theta}_j^{(t)} - \boldsymbol{\theta}_i^{(t)} \right) \mathcal{N}(0, 1) \tag{12}$$

where σ_1 controls the variance of the isotropic Gaussian distribution and σ_2 controls the magnitude of the perturbation of $\boldsymbol{\theta}_i^{(t)}$ along the direction of correlation with $\boldsymbol{\theta}_j^{(t)}$.

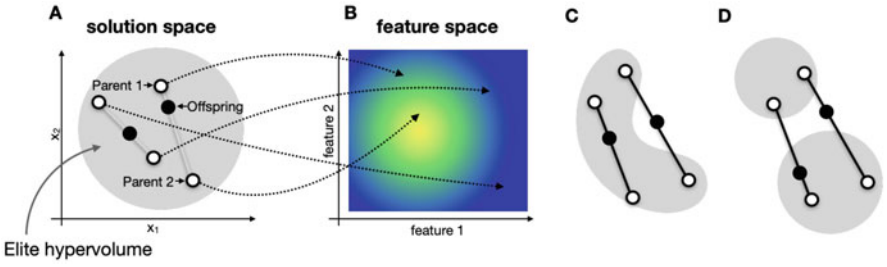


Fig. 4 Concept of the elite hypervolume [75]. (**a**)–(**b**) The elites of the feature space (**b**) occupy a very specific volume in the search space (**a**), here represented with a circle. If this volume is convex, then blending two elites creates a new elite. (**c**–**d**) If the elite hypervolume is not convex, then the blending is not guaranteed to generate a new elite, but it can still happen

During the same time, an independent study by Nordmoen et al. [58] investigated how static and dynamic mutation rates affect the exploration vs exploitation ability of MAP-Elites in a quadruped locomotion task. They tested three dynamic mutation schemes, the isotropic self-adaptation scheme of evolution strategies [5], one based on simulated annealing [43], and another based on the QD coverage metric and found them to have either similar or better QD performance than the static mutation rates. Interestingly, the study by Vassiliades and Mouret [75] also compared isotropic self-adaptation and found it to be less effective than their directional variation operator.

While cross-over is a straightforward way to generate individuals that are likely to be in the elite hypervolume, it might sometimes be possible to fit a distribution so that we can directly sample from it. Since this volume can have any shape, modeling it with a simple Gaussian distribution is unlikely to be successful; instead, Gaier et al. [32] proposed to use a Variational Auto-Encoder (VAE) [42] that is learned from the genotype of the current archive of a QD algorithm. By learning the "latent space" of the data, the algorithm learns the nonlinear "recipes" that define the high-performing solutions, so that more elites can be produced.

However, if the algorithm uses a Variational Auto-Encoder as a variation operator, it cannot generate candidates that are not in the current approximation of the hypervolume. In other words, when the QD algorithm is running, it can apply the current "current recipe" but not discover a "better recipe" for elites. It is therefore important to balance exploration—trying candidates outside of the current distribution—and exploitation—learning and using the current recipe. To tackle this issue, Gaier et al. implemented a multi-armed bandit algorithm [2, 40] that tunes the probability of using the VAE and the probability of using other variation operators like Gaussian mutation and directional variation. As a result, the VAE is used only when it helps.

Gaier et al. tested this approach, called Data-Driven Encoding (DDE), in tasks up to 1000-dimensional [32]. The results show that the VAE allows MAP-Elites to be used in this kind of high-dimensional tasks, although more tasks need to be investigated in the future. Interestingly, the latent space that is learned by the VAE is a representation of the elite hypervolume that can be reused for future optimization of tasks from the same distribution. For instance, it can be leveraged to quickly recompute a higher-dimensional map by using the latent space as the genotype space, or to fine-tune for a specific bin using a black-box optimizer like CMA-ES.

The combination of the uniform selection operator with standard mutation and cross-over operators, which is used for instance in MAP-Elites, is a simple, yet very effective mechanism to produce new solutions. One of its strengths is that it is not biased toward any specific aspect of the optimization process. However, as a direct consequence, the exploration of the search space might be slower than alternative approaches. In particular, we can note that the selective pressure (i.e., the probability of a solution to be selected) induced by this mechanism is inversely proportional to the number of solutions in the collection, which can make this approach less appropriate for the generation of large collections.

Instead of such an unbiased selection mechanism, Fontaine et al. [28] proposed in the Covariance Matrix Adaptation MAP-Elites (CMA-ME) algorithm, the concept of *emitters* that are independent processes in charge of the generation of new solutions. They defined three types of emitters, each based on the CMA-ES algorithm [34] to drive the exploration according to a specific intrinsic motivation. More precisely, they introduced the *Optimizing emitters* that use CMA-ES to sample solutions with higher performance, the *Random Direction emitters* that favor solutions that are as far as possible along a randomly generated direction in the BD space, and the *Improvement emitters* that reward solutions that are added to the collection (similarly to the curiosity score detailed above). These different emitters enable the user to specialize the search process for instance to accelerate the coverage of the behavioral space, or searching the nearest local optimum.

5.5 Noisy Functions

One of the main current challenges in QD algorithm is noisy domains, in which the evaluation of the fitness and the behavioral descriptor values are subject to noise. This noise mainly perturbs the addition of solutions in the collection: a noisy BD measure might place a solution in a wrong cell (or region of the BD space), while a noisy fitness value might over-estimate the quality of the solution and lead to its undeserved preservation in the collection. A simple approach to overcome this challenge is to replicate the evaluation of each solution multiple times (e.g., 100 times) to collect statistics, such as the median for the fitness and the geometric median for the BD, to ensure a robust optimization process. However, this severely deteriorates the data-efficiency of the algorithm. To mitigate this problem and offer a solution that is both robust the noise and more data-efficient, Justesen et al. [39] proposed to use the concept of *adaptive sampling* [8] to allocate the evaluation budget only on solutions that are promising while avoiding the re-evaluations of solutions that are unlikely to be competitive. The results show that this approach is particularly effective when only the fitness function is noisy, as the noise on the BD creates *drifting elites* (solutions that drift to different cells and leaving behind them an empty cell). An alternative approach is proposed by Flageat et al. [27] with the Deep-Grid MAP-Elites (DG-MAP-Elites), in which the grid of MAP-Elites is extended to store multiple solutions per cell, up to a fixed number (e.g., 50). This subpopulation of solutions in each cell can be seen as the depth of the grid. Additionally, the selection and addition mechanisms have been changed to select with a higher probability high-performing solutions within each subpopulation, while any new solution is added to the collection by replacing a randomly selected existing one. This avoids maintaining illegitimate solutions and forces the subpopulations to contain solutions that are likely to produce offspring that land in the same cell, thus improving the robustness of the BD.

6 Conclusion

Quality-Diversity Optimization is a novel branch of stochastic optimization that deals with a special kind of objective functions that return not only a fitness value, but also a behavior descriptor (or a feature vector). The goal of this type of optimization is to collect solutions in a container (or archive) so that they cover as much as possible the behavioral space, and they are locally optimized. We presented a short review of the history of QD optimization, and the main current topics under consideration in the community focusing more on the ones that we believe are the most promising directions to follow. Finally, we presented throughout the chapter many successful applications of QD algorithms on numerous fields, and gave an overview of the current limitations.

In particular, QD algorithms are very effective at (a) optimizing very sparse or non-convex functions, (b) illuminating the search space according to user-defined or learned features, and (c) producing locally optimized repertoires of behaviors (e.g. robots walking in every direction). We hope that readers of this chapter will now have an additional tool in their optimization toolkit, which will allow them to solve new problems or visualize their search spaces.

References

1. Adams, M.D., Celniker, S.E., Holt, R.A., Evans, C.A., Gocayne, J.D., Amanatides, P.G., Scherer, S.E., Li, P.W., Hoskins, R.A., Galle, R.F., et al.: The genome sequence of Drosophila melanogaster. Science **287**(5461), 2185–2195 (2000)
2. Auer, P., Cesa-Bianchi, N., Fischer, P.: Finite-Time Analysis of the Multiarmed Bandit Problem. Springer, Berlin (2002)
3. Barrera, J., Coello, C.A.C.: A review of particle swarm optimization methods used for multimodal optimization. In: Innovations in Swarm Intelligence, pp. 9–37. Springer, Berlin (2009)
4. Bartz-Beielstein, T., Zaefferer, M.: Model-based methods for continuous and discrete global optimization. Appl. Soft Comput. **55**, 154–167 (2017)
5. Beyer, H.-G., Schwefel, H.-P.: Evolution strategies–a comprehensive introduction. Nat. Comput. **1**(1), 3–52 (2002)
6. Bradner, E., Iorio, F., Davis, M.: Parameters tell the design story: ideation and abstraction in design optimization. In: Simulation Series (2014)
7. Brochu, E., Cora, V.M., De Freitas, N.: A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning (2010). Preprint, arXiv:1012.2599
8. Cantú-Paz, E.: Adaptive sampling for noisy problems. In: Genetic and Evolutionary Computation Conference, pp. 947–958. Springer, Berlin (2004)
9. Cavicchio, D.J.: Adaptive search using simulated evolution. PhD thesis, University of Michigan, Ann Arbor, MI (1970)
10. Chatzilygeroudis, K., Vassiliades, V., Mouret, J.-B.: Reset-free trial-and-error learning for robot damage recovery. Rob. Auton. Syst. **100**, 236–250 (2018)
11. Clune, J., Mouret, J.-B., Lipson, H.: The evolutionary origins of modularity. Proc. R. Soc. B Biol. Sci. **280**(1755), 20122863 (2013)

12. Cox, D.D., John, S.: A statistical method for global optimization. In: International Conference on Systems, Man, and Cybernetics, pp. 1241–1246. IEEE, Piscataway (1992)
13. Cully, A.: Autonomous skill discovery with quality-diversity and unsupervised descriptors. In: Proceedings of the Genetic and Evolutionary Computation Conference, pp. 81–89. ACM, New York (2019)
14. Cully, A., Demiris, Y.: Hierarchical behavioral repertoires with unsupervised descriptors. In: Proceedings of the Genetic and Evolutionary Computation Conference (2018)
15. Cully, A., Demiris, Y.: Quality and diversity optimization: a unifying modular framework. IEEE Trans. Evol. Comput. **22**(2), 245–259 (2018)
16. Cully, A., Mouret, J.-B.: Behavioral repertoire learning in robotics. In: Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation, pp. 175–182. ACM, New York (2013)
17. Cully, A., Clune, J., Tarapore, D., Mouret, J.-B.: Robots that can adapt like animals. Nature **521**(7553), 503–507 (2015)
18. Das, S., Maity, S., Qu, B.-Y., Suganthan, P.N.: Real-parameter evolutionary multimodal optimization—a survey of the state-of-the-art. Swarm Evol. Comput. **1**, 71–88 (2011)
19. De Jong, K.A.: Analysis of the behavior of a class of genetic adaptive systems. PhD thesis, University of Michigan, Ann Arbor, MI (1975)
20. Deb, K., Beyer, H.-G.: Self-adaptive genetic algorithms with simulated binary crossover. Evol. Comput. **9**(2), 197–221 (2001)
21. Deb, K., Saha, A.: Finding multiple solutions for multimodal optimization problems using a multi-objective evolutionary approach. In: Proceedings of the 12th Annual Conference on Genetic and Evolutionary Computation, pp. 447–454 (2010)
22. Du, Q., Faber, V., Gunzburger, M.: Centroidal Voronoi tessellations: applications and algorithms. SIAM Rev. **41**, 637–676 (1999)
23. Duarte, M., Gomes, J., Oliveira, S.M., Christensen, A.L.: Evolution of repertoire-based control for robots with complex locomotor systems. IEEE Trans. Evol. Comput. **22**(2), 314–328 (2018)
24. Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K.O., Clune, J.: Go-explore: a new approach for hard-exploration problems (2019). Preprint, arXiv:1901.10995
25. Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K.O., Clune, J.: First return then explore (2020). Preprint, arXiv:2004.12919
26. Escande, A., Mansard, N., Wieber, P.-B.: Hierarchical quadratic programming: fast online humanoid-robot motion generation. Int. J. Robot. Res. **33**(7), 1006–1028 (2014)
27. Flageat, M., Cully, A.: Fast and stable map-elites in noisy domains using deep grids. In: Proceeding of the Alife Conference (2020)
28. Fontaine, M.C., Togelius, J., Nikolaidis, S., Hoover, A.K.: Covariance matrix adaptation for the rapid illumination of behavior space. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion (2020)
29. Gaier, A., Asteroth, A., Mouret, J.-B.: Aerodynamic design exploration through surrogate-assisted illumination. In: 18th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference, pp. 3330 (2017)
30. Gaier, A., Asteroth, A., Mouret, J.-B.: Data-efficient exploration, optimization, and modeling of diverse designs through surrogate-assisted illumination. In: Proceedings of the Genetic and Evolutionary Computation Conference, pp. 99–106. ACM, New York (2017)
31. Gaier, A., Asteroth, A., Mouret, J.-B.: Data-efficient design exploration through surrogate-assisted illumination. Evol. Comput. **26**, 1–30 (2018)
32. Gaier, A., Asteroth, A., Mouret, J.-B.: Discovering representations for black-box optimization. In: Proceedings of the Genetic and Evolutionary Computation Conference (GECCO), vol. 11 (2020)
33. Goldberg, D.E., Richardson, J., et al.: Genetic algorithms with sharing for multimodal function optimization. In: Genetic Algorithms and Their Applications: Proceedings of the Second International Conference on Genetic Algorithms, pp. 41–49. Lawrence Erlbaum, Hillsdale (1987)

34. Hansen, N., Ostermeier, A.: Completely derandomized self-adaptation in evolution strategies. Evol. Comput. **9**(2), 159–195 (2001)
35. Harik, G.R.: Finding multimodal solutions using restricted tournament selection. In: Proceedings of the 6th International Conference on Genetic Algorithms, pp. 24–31. Morgan Kaufmann, San Francisco (1995)
36. Hauschild, M., Pelikan, M.: An introduction and survey of estimation of distribution algorithms. Swarm Evol. Comput. **1**(3), 111–128 (2011)
37. Jones, D.R., Schonlau, M., Welch, W.J.: Efficient global optimization of expensive black-box functions. J. Glob. Optim. **13**(4), 455–492 (1998)
38. Ju, L., Du, Q., Gunzburger, M.: Probabilistic methods for centroidal Voronoi tessellations and their parallel implementations. Parallel Comput. **28**(10), 1477–1500 (2002)
39. Justesen, N., Risi, S., Mouret, J.-B.: Map-elites for noisy domains by adaptive sampling. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion, pp. 121–122. ACM, New York (2019)
40. Karafotias, G., Hoogendoorn, M., Eiben, Á.E.: Parameter control in evolutionary algorithms: Trends and challenges. IEEE Trans. Evol. Comput. **19**(2), 167–187 (2014)
41. Kent, P., Branke, J.: Bop-elites, a Bayesian optimisation algorithm for quality-diversity search (2020). Preprint, arXiv:2005.04320
42. Kingma, D.P., Welling, M.: Auto-encoding variational Bayes. In: Bengio, Y., LeCun, Y. (eds.) International Conference on Learning Representation (ICLR) (2014)
43. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983)
44. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proc. IEEE **86**(11), 2278–2324 (1998)
45. Lee, C.-G., Cho, D.-H., Jung, H.-K.: Niching genetic algorithm with restricted competition selection for multimodal function optimization. IEEE Trans. Magn. **35**(3), 1722–1725 (1999)
46. Lehman, J., Stanley, K.O.: Abandoning objectives: evolution through the search for novelty alone. Evol. Comput. **19**(2), 189–223 (2011)
47. Lehman, J., Stanley, K.O.: Evolving a diversity of virtual creatures through novelty search and local competition. In: Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation, pp. 211–218. ACM, New York (2011)
48. Lehman, J., Risi, S., Clune, J.: Creative generation of 3D objects with deep learning and innovation engines. In: Proceedings of the 7th International Conference on Computational Creativity (2016)
49. Liapis, A., Martınez, H.P., Togelius, J., Yannakakis, G.N.: Transforming exploratory creativity with DeLeNoX. In: Proceedings of the Fourth International Conference on Computational Creativity, pp. 56–63. AAAI Press, Palo Alto (2013)
50. MacQueen, J.: Some methods for classification and analysis of multivariate observations. In: Proc. 5th Berkeley Symp. on Math. Statist. and Prob., vol. 1, pp. 281–297. Univ. of Calif. Press, Berkeley (1967)
51. Mahfoud, S.: Niching methods for genetic algorithms. PhD thesis, University of Illinois at Urbana-Champaign, Urbana, IL (1995)
52. Mayne, D.Q., Rawlings, J.B., Rao, C.V., Scokaert, P.O.: Constrained model predictive control: stability and optimality. Automatica **36**(6), 789–814 (2000)
53. Mouret, J.-B., Clune, J.: Illuminating search spaces by mapping elites (2015). Preprint, arXiv:1504.04909
54. Mouret, J.-B., Doncieux, S.: Encouraging behavioral diversity in evolutionary robotics: an empirical study. Evol. Comput. **20**(1), 91–133 (2012)
55. Mouret, J.-B., Maguire, G.: Quality diversity for multi-task optimization. In: Proceedings of the Genetic and Evolutionary Computation Conference. ACM, New York (2020)
56. Nguyen, A., Yosinski, J., Clune, J.: Deep neural networks are easily fooled: high confidence predictions for unrecognizable images. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 427–436 (2015)

57. Nguyen, A.M., Yosinski, J., Clune, J.: Innovation engines: automated creativity and improved stochastic optimization via deep learning. In: Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation, pp. 959–966. ACM, New York (2015)
58. Nordmoen, J., Samuelsen, E., Ellefsen, K.O., Glette, K.: Dynamic mutation in map-elites for robotic repertoire generation. In: Artificial Life Conference Proceedings, pp. 598–605. MIT Press, Cambridge (2018)
59. Ong, Y.S., Nair, P.B., Keane, A.J.: Evolutionary optimization of computationally expensive problems via surrogate modeling. AIAA J. **41**(4), 687–696 (2003)
60. Paolo, G., Laflaquiere, A., Coninx, A., Doncieux, S.: Unsupervised learning and exploration of reachable outcome space. Algorithms **24**, 25 (2019)
61. Pearce, M., Branke, J.: Continuous multi-task bayesian optimisation with correlation. Eur. J. Oper. Res. **270**(3), 1074–1085 (2018)
62. Pétrowski, A.: A clearing procedure as a niching method for genetic algorithms. In: Proceedings of IEEE International Conference on Evolutionary Computation, pp. 798–803. IEEE, Piscataway (1996)
63. Preuss, M.: Multimodal Optimization by Means of Evolutionary Algorithms. Springer, Berlin (2015)
64. Preuss, M., Schönemann, L., Emmerich, M.: Counteracting genetic drift and disruptive recombination in ($\mu \dot{+} \lambda$)-EA on multimodal fitness landscapes. In: Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation, pp. 865–872 (2005)
65. Pugh, J.K., Soros, L., Szerlip, P.A., Stanley, K.O.: Confronting the challenge of quality diversity. In: Proceedings of the 2015 on Genetic and Evolutionary Computation Conference, pp. 967–974. ACM, New York (2015)
66. Pugh, J.K., Soros, L.B., Stanley, K.O.: Quality diversity: a new frontier for evolutionary computation. Front. Robot. AI **3**, 40 (2016)
67. Rudolph, G.: Self-adaptive mutations may lead to premature convergence. IEEE Trans. Evol. Comput. **5**(4), 410–414 (2001)
68. Sareni, B., Krahenbuhl, L.: Fitness sharing and niching methods revisited. IEEE Trans. Evol. Comput. **2**, 97–106 (1998)
69. Shahriari, B., Swersky, K., Wang, Z., Adams, R.P., De Freitas, N.: Taking the human out of the loop: a review of bayesian optimization. Proc. IEEE **104**(1), 148–175 (2015)
70. Shir, O., Emmerich, M., Bäck, T., Vrakking, M.: Conceptual designs in laser pulse shaping obtained by niching in evolution strategies. In: EUROGEN 2007 (2007)
71. Sigmund, O.: A 99 line topology optimization code written in matlab. Struct. Multidiscipl. Optim. **21**(2), 120–127 (2001)
72. Singh, G., Deb, K.: Comparison of multi-modal optimization algorithms based on evolutionary algorithms. In: Proceedings of the 8th Annual Conference on Genetic and Evolutionary Computation, pp. 1305–1312 (2006)
73. Srinivas, N., Krause, A., Kakade, S., Seeger, M.: Gaussian process optimization in the bandit setting: no regret and experimental design. In: Proceedings of the 27th International Conference on International Conference on Machine Learning, pp. 1015–1022 (2010)
74. Tarapore, D., Clune, J., Cully, A., Mouret, J.-B.: How do different encodings influence the performance of the map-elites algorithm? In: Genetic and Evolutionary Computation Conference (2016)
75. Vassiliades, V., Mouret, J.-B.: Discovering the elite hypervolume by leveraging interspecies correlation. In: Proceedings of the Genetic and Evolutionary Computation Conference (2018)
76. Vassiliades, V., Chatzilygeroudis, K., Mouret, J.-B.: Comparing multimodal optimization and illumination. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion, pp. 97–98. ACM, New York (2017)
77. Vassiliades, V., Chatzilygeroudis, K., Mouret, J.-B.: A comparison of illumination algorithms in unbounded spaces. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion, pp. 1578–1581. ACM, New York (2017)

78. Vassiliades, V., Chatzilygeroudis, K., Mouret, J.-B.: Using centroidal Voronoi tessellations to scale up the multidimensional archive of phenotypic elites algorithm. IEEE Trans. Evol. Comput. **22**(4), 623–630 (2018)
79. Williams, C.K., Rasmussen, C.E.: Gaussian Processes for Machine Learning, vol. 2. MIT Press, Cambridge (2006)
80. Yin, X., Germay, N.: A fast genetic algorithm with sharing scheme using cluster analysis methods in multimodal function optimization. In: Artificial Neural Nets and Genetic Algorithms, pp. 450–457. Springer, Berlin (1993)

Multi-Objective Evolutionary Algorithms: Past, Present, and Future



Carlos A. Coello Coello, Silvia González Brambila, Josué Figueroa Gamboa, and Ma. Guadalupe Castillo Tapia

1 Introduction

The idea of using techniques based on the emulation of the mechanism of natural selection to solve optimization problems can be traced back to the 1960s when the three main techniques based on this notion were developed: genetic algorithms [61], evolution strategies [136], and evolutionary programming [41]. These approaches, which are now collectively denominated "evolutionary algorithms," have been very effective for solving single-objective optimization problems [42, 49, 137].

The solution of problems having two or more objectives (which are normally in conflict with each other) has attracted a considerable interest in recent years. The solution of the so-called *multi-objective optimization problems* (MOPs) consists of a set of solutions representing the best possible trade-offs among the objectives. Such solutions, defined in decision variable space constitute the so-called *Pareto optimal set*, and their corresponding objective function values form the so-called *Pareto front*.

Although a variety of mathematical programming techniques to solve MOPs have been developed since the 1970s [104], such techniques present several limitations, from which two of the most important are that these algorithms are

C. A. Coello Coello (✉)
Departamento de Computación, CINVESTAV-IPN, Mexico City, Mexico
e-mail: ccoello@cs.cinvestav.mx

S. G. Brambila · J. F. Gamboa
Departamento de Sistemas, UAM Azcapotzalco, México City, Mexico
e-mail: sgb@azc.uam.mx; jfgo@azc.uam.mx

M. G. C. Tapia
Departamento de Administración, UAM Azcapotzalco, México City, Mexico
e-mail: mgct@correo.uam.mx

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_5

normally very susceptible to the shape or continuity of the Pareto front and that they tend to generate a single element of the Pareto optimal set per run. Additionally, in some real-world MOPs, the objective functions are not provided in algebraic form, but are the output of a black-box software (which, for example, runs a simulation to obtain an objective function value). This severely limits the applicability of mathematical programming techniques.

Evolutionary algorithms seem particularly suitable for solving multi-objective optimization problems because they deal simultaneously with a set of possible solutions (the so-called population) which allows them to obtain several members of the Pareto optimal set in a single run of the algorithm, instead of having to perform a series of separate runs as in the case of the traditional mathematical programming techniques. Additionally, evolutionary algorithms are less susceptible to the shape or continuity of the Pareto front (e.g., they can easily deal with discontinuous and concave Pareto fronts), whereas these two issues are a real concern for mathematical programming techniques.

The potential of evolutionary algorithms for solving MOPs was first pointed out by Rosenberg in the 1960s [125], but the first actual implementation of a multi-objective evolutionary algorithm (MOEA) was produced until the mid-1980s by David Schaffer [133, 134]. Nevertheless, it was until the mid-1990s that MOEAs started to attract serious attention from researchers. Nowadays, it is possible to find applications of MOEAs in practically all areas of knowledge.[1]

The contents of this chapter are organized as follows. Some basic concepts required to make of this a self-contained chapter are provided in Sect. 2. Section 3 describes the main algorithmic paradigms (as well as some representative MOEAs belonging to each of them) developed from 1984 up to the early 2000s. In Sect. 4, the most popular MOEAs developed from the mid-2000s to date are briefly described. Some representative applications of these MOEAs are also provided in this section. Then, in Sect. 5, some possible paths for future research in this area are briefly described. Finally, our conclusions are provided in Sect. 6.

2 Basic Concepts

We are interested in solving problems of the type:[2]

$$\text{minimize } \mathbf{f}(\mathbf{x}) := [f_1(\mathbf{x}), f_2(\mathbf{x}), \ldots, f_k(\mathbf{x})] \tag{1}$$

subject to

[1] The first author maintains the EMOO repository, which currently contains over 12,400 bibliographic references related to evolutionary multi-objective optimization. The EMOO repository is located at: https://emoo.cs.cinvestav.mx.

[2] Without loss of generality, we will assume only minimization problems.

$$g_i(\mathbf{x}) \leq 0 \quad i = 1, 2, \ldots, m \tag{2}$$

$$h_i(\mathbf{x}) = 0 \quad i = 1, 2, \ldots, p \tag{3}$$

where $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$ is the vector of decision variables, $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, \ldots, k$ are the objective functions, and $g_i, h_j : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, \ldots, m$, $j = 1, \ldots, p$ are the constraint functions of the problem.

To describe the concept of optimality in which we are interested, we will introduce next a few definitions.

Definition 1 Given two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^k$, we say that $\mathbf{x} \leq \mathbf{y}$ if $x_i \leq y_i$ for $i = 1, \ldots, k$, and that $\mathbf{x}$ **dominates** $\mathbf{y}$ (denoted by $\mathbf{x} \prec \mathbf{y}$) if $\mathbf{x} \leq \mathbf{y}$ and $\mathbf{x} \neq \mathbf{y}$.

Definition 2 We say that a vector of decision variables $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^n$ is **nondominated** with respect to $\mathcal{X}$, if there does not exist another $\mathbf{x}' \in \mathcal{X}$ such that $\mathbf{f}(\mathbf{x}') \prec \mathbf{f}(\mathbf{x})$.

Definition 3 We say that a vector of decision variables $\mathbf{x}^* \in \mathcal{F} \subset \mathbb{R}^n$ ($\mathcal{F}$ is the feasible region) is **Pareto optimal** if it is nondominated with respect to $\mathcal{F}$.

Definition 4 The **Pareto Optimal Set** $\mathcal{P}^*$ is defined by

$$\mathcal{P}^* = \{\mathbf{x} \in \mathcal{F} | \mathbf{x} \text{ is Pareto optimal}\}$$

Definition 5 The **Pareto Front** $\mathcal{PF}^*$ is defined by

$$\mathcal{PF}^* = \{\mathbf{f}(\mathbf{x}) \in \mathbb{R}^k | \mathbf{x} \in \mathcal{P}^*\}$$

We thus wish to determine the Pareto optimal set from the set $\mathcal{F}$ of all the decision variable vectors that satisfy (2) and (3). Note however that in practice, not all the Pareto optimal set is normally desirable (e.g., it may not be desirable to have different solutions that map to the same values in objective function space) or achievable.

3 The Past

As indicated before, the earliest attempt to use evolutionary algorithms for solving multi-objective optimization problems dates back to Richard S. Rosenberg's PhD thesis [125] in which he suggested to use multiple properties (i.e., nearness to certain specified chemical composition) in his simulation of the genetics and chemistry of a population of single-celled organisms. Although his model considered two properties (i.e., two objectives), he transformed one of them into a constraint and dealt with a constrained single-objective optimization problem. The first actual implementation of a multi-objective evolutionary algorithm (MOEA) was developed

by David Schaffer in his PhD thesis [133]. His approach, called *Vector Evaluated Genetic Algorithm* (VEGA) [134] will be briefly described next.

In this section, we will review the approaches proposed in the period from 1984 to the early 2000s. This period is divided in three parts each of which corresponds to a different algorithmic paradigm: (1) Non-Elitist Non-Pareto Approaches, (2) Non-Elitist Pareto-based Approaches, and (3) Elitist Pareto-based Approaches. Some representative algorithms within each of these three groups are briefly described next.

3.1 Non-Elitist Non-Pareto Approaches

These are the oldest MOEAs and are characterized for not incorporating elitism and for having selection mechanisms that do not incorporate the notion of Pareto optimality. Here, we will briefly review the following approaches:

- Linear aggregating functions
- Vector Evaluated Genetic Algorithm (VEGA)
- Lexicographic ordering
- Target-vector approaches

3.1.1 Linear Aggregating Functions

The most straightforward way of transforming a vector optimization problem into a scalar optimization problem is through the use of a linear combination of all the objectives (e.g., using addition). These techniques are called "aggregating functions" because they combine (or *aggregate*) all the objectives into a single one. This is indeed the oldest mathematical programming method developed for solving multi-objective problems, and it can be derived from the Kuhn–Tucker conditions for nondominated solutions [82].

The most typical linear aggregating function is the following:

$$\min \sum_{i=1}^{k} w_i f_i(\mathbf{x}) \tag{4}$$

where $w_i \geq 0$ are the weighting coefficients representing the relative importance of the k objective functions of our problem (the objectives need to be normalized). It is usually assumed that

$$\sum_{i=1}^{k} w_i = 1 \tag{5}$$

In order to generate different elements of the Pareto optimal set, the weights must be varied and this is in fact the most common way in which linear aggregating functions have been incorporated into evolutionary algorithms for solving multi-objective problems (see for example [35]). The main problem of linear aggregating functions is that they cannot generate non-convex portions of the Pareto front, regardless of the weights that we adopt [29]. Nevertheless, some clever proposals were made in the early 2000s to overcome this limitation (see for example [74]).

3.1.2 Vector Evaluated Genetic Algorithm (VEGA)

This is the first actual implementation of an evolutionary multi-objective optimization technique, which, as indicated before, was developed by David Schaffer [133, 134] in the mid-1980s. The *Vector Evaluated Genetic Algorithm* (VEGA) basically consisted of a simple genetic algorithm (GA) with a modified selection mechanism. At each generation, a number of sub-populations (as many as the number of objectives of the problem) were generated by performing proportional selection according to each objective function in turn. Thus, for a problem with k objectives, k sub-populations of size N/k each would be generated (assuming a total population size of N). These sub-populations would then be shuffled together to obtain a new population of size N, on which the GA would apply the crossover and mutation operators in the usual way. VEGA has several limitations. For example, Schaffer realized that the solutions generated by his system were locally nondominated but not necessarily globally nondominated. Also, he noted that producing individuals which were the best in one objective is not a good idea in multi-objective optimization (in fact, this sort of selection mechanism opposes the notion of Pareto optimality). Nevertheless, the selection mechanism of VEGA has been adopted by some researchers (see for example [19]) and some other population-based selection schemes which combine VEGA with linear aggregating functions have been adopted by other researchers (see for example [70]).

3.1.3 Lexicographic Ordering

In this method, the user is asked to rank the objectives in order of importance. The optimum solution is then obtained by minimizing the objective functions, starting with the most important one and proceeding according to the assigned order of importance of the objectives, but maintaining the best solutions previously produced.

Fourman [44] was the first to suggest a selection scheme based on lexicographic ordering for a MOEA. In a first version of his algorithm, objectives are assigned different priorities by the user and each pair of individuals are compared according to the objective with the highest priority. If this resulted in a tie, the objective with the second highest priority was used, and so on. In another version of this algorithm (that apparently worked quite well), an objective is randomly selected

at each run. Several other variations of lexicographic ordering have been adopted by other authors (see for example [45, 111]), but this sort of approach is clearly not suitable for complex multi-objective problems or even for (not so complex) problems having more than two objectives [23].

3.1.4 Target-Vector Approaches

This category encompasses methods in which we have to define a set of goals (or targets) that we wish to achieve for each objective function under consideration. The MOEA in this case will try to minimize the difference between the current solution generated and the vector of desirable goals (different metrics can be used for this purpose). Although target-vector approaches can be considered as another aggregating approach, these techniques can generate (under certain conditions) concave portions of the Pareto front, whereas approaches based on simple linear aggregating functions cannot.

The most popular techniques included here are hybrids of MOEAs with: Goal Programming [30, 129, 149], Goal Attainment [150, 155], and the min-max algorithm [21, 54]. These techniques are relatively simple to implement and are very efficient (computationally speaking). However, their main disadvantage is the difficulty to define the desired goals. Additionally, some of them can generate nondominated solutions only under certain conditions [23].

3.2 Non-Elitist Pareto-Based Approaches

Goldberg discussed the main drawbacks of VEGA in his seminal book on genetic algorithms [49] and proposed an approach to solve multi-objective optimization problems which incorporated the concept of Pareto optimality (this approach is now known as Pareto ranking) and also suggested the use of a mechanism to block the selection mechanism so that a diverse set of solutions could be generated in a single run of a MOEA (he suggested fitness sharing for this sake [50]). Such a mechanism is known today as *density estimator* and is a standard procedure in modern MOEAs [23].

Early Pareto-based MOEAs relied on variations of Goldberg's proposal and adopted relatively simple density estimators. Here, we will briefly review the following MOEAs:

- Multi-Objective Genetic Algorithm (MOGA)
- Nondominated Sorting Genetic Algorithm (NSGA)
- Niched-Pareto Genetic Algorithm (NPGA)

3.2.1 Multi-Objective Genetic Algorithm (MOGA)

It was proposed by Fonseca and Fleming in 1993 [43]. In MOGA, the rank of a certain individual corresponds to the number of individuals in the population by which it is dominated. However, it adopts a clever Pareto ranking scheme which classifies all individuals in a single pass and assigns fitness to each of them based on their ranks. All nondominated individuals are assigned the same fitness value and all dominated individuals are assigned a fitness value that decreases proportionally to the number of individuals that dominate them (as more individuals dominate a certain solution, its fitness value becomes lower).

MOGA was used by an important number of researchers, particularly in automatic control (see for example [97, 113]).

3.2.2 Nondominated Sorting Genetic Algorithm (NSGA)

This algorithm was proposed by Srinivas and Deb in the mid-1990s and it was the first MOEA published in a specialized journal [139]. NSGA is based on the creation of several layers of classifications of the individuals (this procedure is now called *nondominated sorting*) as suggested by Goldberg [49]. Before selection is performed, the population is ranked on the basis of Pareto optimality: all nondominated individuals are classified into one category or layer (using a dummy fitness value, which is proportional to the population size). The density estimator in this case is fitness sharing (which is applied on the dummy fitness values). Once a group of individuals has been classified, then such a group is ignored and another layer of nondominated individuals is considered. This process is repeated until all the individuals in the population had been classified.

Several applications of NSGA were developed in the 1990s and early 2000s (see for example [8, 96, 148]).

3.2.3 Niched-Pareto Genetic Algorithm (NPGA)

It was proposed by Horn et al. in the mid-1990s [62]. NPGA uses binary tournament selection based on Pareto dominance. Thus, two individuals are randomly chosen and compared against a subset from the entire population (typically, the number of individuals in this set is of around 10% of the total population size). If one of them is dominated (by the individuals randomly chosen from the population) and the other is not, then the nondominated individual wins the tournament. Otherwise, (i.e., when both competitors are either dominated or nondominated), the result of the tournament is decided through fitness sharing [50].

NPGA was not as popular as NSGA or MOGA, but there are some applications of this MOEA reported in the literature (see for example [118, 156]).

3.3 Elitist Pareto-Based Approaches

MOEAs developed in the late 1990s started to incorporate the notion of elitism. In the context of evolutionary multi-objective optimization, elitism refers to retaining the nondominated solutions generated by a MOEA. The most popular mechanism for implementing elitism is through the use of an external archive (also called *secondary population*) which may or may not intervene in the selection process. This external archive stores the nondominated solutions generated by a MOEA and is normally bounded and pruned once it is full. This is done for two main reasons: (1) to facilitate direct comparisons among different elitist MOEAs and (2) to dilute the selection process (when the external archive participates in the selection process) and/or to avoid storing an excessively large number of solutions.

Elitism is a very important mechanism in MOEAs, because it is required to (theoretically) guarantee convergence [127].

It is worth noting that elitism can also be introduced through the use of a $(\mu + \lambda)$-selection in which parents compete with their children and those which are nondominated (and possibly comply with some additional criterion such as providing a better distribution of solutions) are selected for the following generation. This is the elitist mechanism adopted by NSGA-II [33].

The most representative elitist Pareto-based approaches developed in the late 1990s and early 2000s which will be briefly described here are the following:

- The Strength Pareto Evolutionary Algorithm (SPEA)
- The Pareto Archived Evolution Strategy (PAES)
- The Nondominated Sorting Genetic Algorithm-II (NSGA-II)

It is also worth indicating that alternative density estimators were proposed with these MOEAs as will be indicated next.

3.3.1 The Strength Pareto Evolutionary Algorithm (SPEA)

It was introduced by Zitzler and Thiele in the late 1990s [161]. This approach was conceived as a way of integrating different MOEAs. SPEA uses an external archive that contains the nondominated solutions previously generated. At each generation, nondominated individuals are copied to the external archive. For each individual in this external set, a strength value is computed. This strength is similar to the ranking value of MOGA, since it is proportional to the number of solutions to which a certain individual dominates. The fitness of each member of the current population is computed according to the strengths of all external nondominated solutions that dominate it. Additionally, a clustering technique called "average linkage method" [110] is used as the density estimator.

SPEA has been used in a variety of applications (see for example [3, 103]).

In 2001, a revised version of this algorithm (called SPEA2) was proposed [163]. This approach has three main differences with respect to its original version: (1)

it incorporates a fine-grained fitness assignment strategy which takes into account for each individual the number of individuals that dominate it and the number of individuals by which it is dominated; (2) it uses a nearest neighbor density estimation technique which guides the search more efficiently, and (3) it has an enhanced archive truncation method that guarantees the preservation of boundary solutions. SPEA2 has also been widely applied (see for example [114, 147]).

3.3.2 The Pareto Archived Evolution Strategy (PAES)

This MOEA was introduced by Knowles and Corne [80] and it consists of a (1+1) evolution strategy (i.e., a single parent that generates a single offspring) in combination with an external archive that records the nondominated solutions previously found. This archive is used as a reference set against which each mutated individual is compared. An interesting aspect of this algorithm is the procedure used to maintain diversity which consists of a crowding procedure that divides objective function space in a recursive manner. Each solution is placed in a certain grid location based on the values of its objectives (which are used as its "coordinates" or "geographical location"). A map of such grid is maintained, indicating the number of solutions that reside in each grid location. Since the procedure is adaptive, no extra parameters are required (except for the number of divisions of the objective function space). This sort of density estimator (i.e., the so-called adaptive grid) is a very nice idea, but unfortunately, it does not scale property when increasing the number of objectives [20].

There are a few applications of PAES reported in the specialized literature (see for example [1, 122]).

3.3.3 The Nondominated Sorting Genetic Algorithm-II (NSGA-II)

Deb et al. [32, 33] proposed a revised version of NSGA [139], called NSGA-II. This approach is more efficient (computationally speaking), it uses elitism and a crowded comparison operator that keeps diversity without specifying any additional parameters (it is based on how close are the neighbors of a solution). This algorithm is, in fact, quite different from the original NSGA, since even its nondominated sorting process is done in a more efficient way.

As indicated before, NSGA-II does not use an external memory but adopts instead an elitist mechanism that consists of combining the best parents with the best offspring obtained (i.e., a $(\mu + \lambda)$-selection).

NSGA-II has been the most popular MOEA developed so far. This is mainly because of its efficiency, efficacy and because of the availability of its source code in the public domain (in several versions). However, its crowded comparison operator does not scale properly as the number of objectives increases [83], which motivated a number of variations of this algorithm including the NSGA-III [31], which is discussed in a further section.

There are many applications of NSGA-II reported in the literature (see for example [27, 56, 90]).

4 The Present

In addition to the MOEAs briefly discussed in the previous section, many others were developed, but few of them were adopted by researchers different from their developers (see for example [20, 25, 26, 140]).

Nevertheless, for over 10 years, Pareto-based MOEAs remained as the most popular approaches in the specialized literature. In 2004, a different type of algorithmic design was proposed, although it remained underdeveloped for several years: indicator-based selection.[3] The core idea of this sort of MOEA was introduced in the **Indicator-Based Evolutionary Algorithm** (IBEA) [162], which consists of an algorithmic framework that allows the incorporation of any performance indicator into the selection mechanism of a MOEA. IBEA was originally tested with the hypervolume [160] and the binary ϵ indicator [162].

The limitations of Pareto-based selection for dealing with problems having 4 or more objectives (the so-called *many-objective optimization problems*) motivated researchers to look for alternative approaches. Indicator-based selection became an attractive option because these schemes can properly deal with any number of objectives. Much of the early interest in this area was motivated by the introduction of the **S Metric Selection Evolutionary Multiobjective Algorithm** (SMS-EMOA) [36]. SMS-EMOA randomly generates an initial population and then produces a single solution per iteration (i.e., it uses a steady-state selection scheme) adopting the crossover and mutation operators from NSGA-II. Then, it applies nondominated sorting (as in NSGA-II). When the last nondominated front has more than one solution, SMS-EMOA uses hypervolume [160] to decide which solution should be removed.

Beume et al. [7] proposed an improved version of SMS-EMOA in which the hypervolume contribution is not used when, in the nondominated sorting process, we obtain more than one front (i.e., the hypervolume is used as a density estimator in this case). When this happens, they use the number of solutions that dominate to a certain individual (i.e., the solution that is dominated by the largest number of solutions is removed). This makes SMS-EMOA a bit more efficient. However, since this MOEA relies on the use of exact hypervolume contributions, it becomes too computationally expensive as we increase the number of objectives [6]. SMS-EMOA started a trend for designing indicator-based MOEAs (several of which rely on the hypervolume indicator) although it is worth indicating that in such approaches, the performance indicator has been mostly used as a density estimator

[3] It is worth indicating that indicator-based archiving was introduced earlier (see [78, 79]).

(see for example [66]). The use of "pure" indicator-based selection mechanisms has been very rare in the specialized literature (see for example [101]).

Researchers quickly realized that the efficacy and efficiency of indicator-based MOEAs rely on the adopted performance indicator. So far, the only performance indicator that is known to have the mathematical properties to guarantee convergence (from a theoretical point of view) is the hypervolume (i.e., it is a Pareto compliant performance indicator [164]). The hypervolume (which is also known as the $\mathcal{S}$ metric or the Lebesgue Measure) of a set of solutions measures the size of the portion of objective space that is dominated by those solutions collectively. As indicated before, one of its main advantages is its mathematical properties, since it has been proved that the maximization of this performance measure is equivalent to finding the Pareto optimal set [40]. Additionally, empirical studies have shown that (for a certain number of points previously determined) maximizing the hypervolume does produce subsets of the true Pareto front [36, 78]. As a performance indicator, the hypervolume assesses both convergence and, to a certain extent, also the spread of solutions along the Pareto front (although without necessarily enforcing a uniform distribution of solutions). Nevertheless, there are several practical issues regarding the use of the hypervolume. First, the computation of this performance indicator depends on a reference point, which can influence the results in a significant manner. Some people have proposed to use the worst objective function values in the current population, but this requires scaling the objectives. Nevertheless, the most serious limitation of the hypervolume is its high computational cost. The best algorithms known to compute hypervolume have a polynomial complexity on the number of points used, but such complexity grows exponentially on the number of objectives [6]. This has motivated a significant amount of research related to the development of sophisticated algorithms that can reduce the computational cost of computing the hypervolume and the hypervolume contributions, which is what we need for a hypervolume-based MOEA[4] (see for example [52, 72, 84]). Today, most researchers believe that it is not possible to overcome the high computational cost of computing exact hypervolume contributions. An obvious alternative to deal with this issue is to approximate the actual hypervolume contributions. This is the approach adopted by the **Hypervolume Estimation Algorithm for Multi-Objective Optimization** (HyPE) [4] in which Monte Carlo simulations are adopted to approximate exact hypervolume values. In spite of the fact that HyPE can efficiently solve multi-objective problems having a very large number of objectives, its results are not as competitive as when using exact hypervolume contributions.

Another alternative is to use a different performance indicator whose computation is relatively inexpensive. Unfortunately, the hypervolume is the only unary indicator which is known to be Pareto compliant [164], which makes less

[4] See:
http://ls11-www.cs.uni-dortmund.de/rudolph/hypervolume/start
http://people.mpi-inf.mpg.de/~tfried/HYP/
http://iridia.ulb.ac.be/~manuel/hypervolume

attractive the use of other performance indicators. Nevertheless, there are some other performance indicators which are weakly Pareto compliant, such as $R2$ [11] and the Inverted Generational Distance plus ($IGD+$) [69]. Although several efficient and effective indicator-based MOEAs have been proposed around these performance indicators (see for example [12, 57, 86, 94, 95]), their use has remained relatively scarce until now. Another interesting idea that has been only scarcely explored is the combination of performance indicators in order to take advantage of their strengths and compensate for their limitations (see for example [37]).

In 2007, a different sort of approach was proposed, quickly attracting a lot of interest: the **Multi-Objective Evolutionary Algorithm based on Decomposition** (MOEA/D) [157]. The idea of using decomposition (or scalarization) methods was originally proposed in mathematical programming in the late 1990s [28] and it consists in transforming a multi-objective optimization problem into several single-objective optimization problems which are then solved to generate the non-dominated solutions of the original problem. Unlike linear aggregating functions, the use of scalarization (or decomposition) methods allows the generation of non-convex portions of the Pareto front and works even in disconnected Pareto fronts. MOEA/D presents an important advantage with respect to methods proposed in the mathematical programming literature (such as Normal Boundary Intersection (NBI) [28]): it uses neighborhood search to solve simultaneously all the single-objective optimization problems generated from the transformation. Additionally, MOEA/D is not only effective and efficient, but can also be used for solving problems with more than 3 objectives although in such cases it will require higher population sizes (however, the population size needs to be increased linearly with respect to the number of objectives).

Decomposition-based MOEAs became fashionable at around 2010 and have remained as an active research area since then [131]. In fact, this sort of approach influenced the development of the **Nondominated Sorting Genetic Algorithm-III** (NSGA-III[5]) [31] which adopts both decomposition and reference points to deal with many-objective optimization problems. However, it was recently found that decomposition-based MOEAs do not work properly with certain Pareto front geometries [71]. This has motivated a lot of research that aims to overcome this limitation.

4.1 Some Applications

In recent years, a significant number of applications of MOEAs have been reported in the literature [22].

[5]NSGA-III was designed to solve many-objective optimization problems and its use is relatively popular today.

Roughly, we can classify applications in three large groups: (1) engineering, (2) industrial, and (3) scientific. Some specific areas within each of these groups are indicated next. We will start with the engineering applications, which are, by far, the most popular in the literature. This is not surprising because engineering disciplines normally have problems with better known and understood mathematical models which facilitate the use of MOEAs. A sample of engineering applications is the following:

- Electrical engineering [51, 88]
- Hydraulic engineering [93, 146]
- Structural engineering [60, 67]
- Aeronautical engineering [115, 152]
- Robotics [39, 73]
- Automatic Control [63, 145]
- Telecommunications [9, 64]
- Civil engineering [75, 85]
- Transport engineering [24, 76]

A sample of industrial applications of MOEAs is the following:

- Design and manufacture [77, 92]
- Scheduling [126, 154]
- Management [121, 138]

Finally, we have a variety of scientific applications:

- Chemistry [17, 38]
- Physics [46, 109]
- Medicine [102, 128]
- Bioinformatics [34, 124]
- Computer science [55, 89]

5 The Future

A large number of MOEAs have been developed since 1984 (most of them, during this century), but few of them have become popular among practitioners. This raises a relevant question: where is the research on MOEAs leading to? In other words, can we design new MOEAs that can become popular? This is indeed an interesting question, although people working on the development of MOEAs evidently consider that there is room for new (sometimes highly specialized) MOEAs.

If we look at specific domains, it is easier to justify the development of particular MOEAs to tackle them. Let us consider the following examples:

- **Large-Scale Multi-Objective Optimization:** It refers to solving multi-objective problems with more than 100 decision variables (something not unusual in real-

world applications). Little work has been done in this area, and cooperative coevolutionary approaches (which are popular in single-objective large-scale optimization) have been the most popular choice (see for example [91, 105, 159]). However, there is still plenty of research to be done in this area. For example, appropriate test suites for large-scale multi-objective optimization are required (see for example [18]).

- **Expensive Objective Functions:** The design of parallel MOEAs seems as the most obvious choice for dealing with expensive objective functions (see for example [10]). However, basic research on parallel MOEAs has remained scarce and most of the current papers on this topic focus either on applications [24, 123] or on straightforward parallelizations of well-known MOEAs (see for example [153]). Many other topics remained to be explored, including the development of asynchronous parallel MOEAs [130], the study of theoretical aspects of parallel MOEAs [108], and the proper use of modern architectures such as Graphical Processing Units (GPUs) for designing MOEAs [116]. Another alternative to deal with expensive objective functions is the use of surrogate methods. When using surrogates, an empirical model that approximates the real problem is built through the use of information gathered from actual objective function evaluations [112]. Then, the empirical model (on which evaluating the fitness function is computationally inexpensive) is used to predict new (promising) solutions [2, 100]. Although frequently used in engineering applications, surrogate methods can normally be adopted only in problems of low dimensionality, which is an important limitation when dealing with real-world problems. Additionally, surrogate models tend to lack robustness which is also an important issue in optimization problems. Nevertheless, there has been recent research oriented towards overcoming the scalability and robustness limitations of surrogate methods (see for example [117, 151]).
- **Many-Objective Optimization:** Developing MOEAs for properly solving multi-objective problems having more than 3 objectives is indeed a hot research topic nowadays. In spite of the existence of a number of indicator-based MOEAs and decomposition-based MOEAs that were explicitly designed for many-objective optimization, a number of other approaches are also possible. For example, we can use alternative ranking schemes (different from nondominated sorting) (see for example [47]), machine learning techniques (as in MONEDA [99]), or approaches such as the two-archive MOEA, which uses one archive for convergence and another for diversity [120]. It is also possible to use dimensionality reduction techniques which identify redundant objectives (i.e., objectives that can be removed without changing the dominance relation induced by the original objective set) and remove them so that the actual dimensionality of the problem can be reduced (see for example [13, 132]). Additionally, several other topics related to many-objective optimization still require further research. Two good examples are visualization techniques [141] and density estimators [59] for problems having a large number of objectives. Another relevant topic is the solution of large-scale many-objective problems (see for example [16, 158]).

Nevertheless, a more profound and complex question is the following: *is it possible to design MOEAs in a different way?* This is a question of great relevance because if it is no longer possible to produce new algorithmic proposals, this entire research area may stagnate and even disappear.

Clearly, it is not trivial to produce a selection mechanism that does not belong to any of the paradigms that we revised in this chapter (i.e., Pareto-based, decomposition-based or indicator-based), but this is indeed possible. For example, Molinet Berenguer and Coello Coello [5, 107], proposed an approach that transforms a multi-objective optimization problem into a linear assignment problem using a set of weight vectors uniformly scattered. Uniform design is adopted to obtain the set of weights, and the Kuhn–Munkres (Hungarian) algorithm [81] is used to solve the resulting assignment problem. This approach was found to perform quite well (and at a low computational cost) even in many-objective optimization problems. This approach constitutes an intriguing new family of MOEAs, since it does not belong to any of the three types of schemes previously described. But designing a new type of MOEA is not enough. It is perhaps more challenging (and certainly more difficult) that such an approach becomes popular.

In addition to the proposal of new algorithmic paradigms, many other approaches are possible. For example, it is possible to combine components of MOEAs under a single framework that allows to exploit their advantages. This is the basic idea of **Borg** [53], which adopts ϵ-dominance, a measure of convergence speed called ϵ progress, an adaptive population size, multiple recombination operators, and a steady-state selection mechanism. Related to this sort of approach is the notion of being able to automatically design MOEAs for particular applications/domains, which is something that has been suggested by researchers from automated parameter tuning for single-objective evolutionary algorithms (see for example [65]).

A related idea is the design of hyper-heuristics for multi-objective optimization. A hyper-heuristic is a search method or learning mechanism for selecting or generating heuristics to solve complex search problems [14]. Hyper-heuristics are high-level approaches that operate on a search space of heuristics or on a pool of heuristics instead of the search space of the problem at hand. They constitute an interesting choice to provide more general search methodologies, since simple heuristics tend to work well on certain types of problems, but can perform very poorly on other classes of problems or even in slight variations of a certain class in which they perform well. Although the ideas behind hyper-heuristics can be traced back to the early 1960s in single-objective optimization, their potential had not been explored in multi-objective optimization until relatively recently. Several multi-objective hyper-heuristics have been proposed for combinatorial problems (see for example [15, 98, 142]) but they are still rare in continuous multi-objective optimization (see for example [58, 143, 144]).

Another interesting path for future research in this area is to gain a deeper understanding of the limitations of current MOEAs. For example, knowing that some scalarizing functions offer advantages over others is very useful to design good decomposition-based and even indicator-based MOEAs (see for example [119]). It is also important to design new mechanisms (e.g., operators, encodings, etc.) for

MOEAs aimed for particular real-world problems (e.g., variable length encodings, expensive objective functions, uncertainty, etc.). See for example [87].

Other evolutionary computation areas can also be brought to this field to enrich the design of MOEAs. One example is coevolutionary approaches, which have been used so far mainly for large-scale multi-objective optimization, but could have more applications in this area (e.g., they could be used to solve dynamic multi-objective optimization problems [48]). Clearly, the potential of coevolutionary schemes has been only scarcely explored in multi-objective optimization (see [106]).

6 Conclusions

This chapter has provided a review of the research on the development of multi-objective evolutionary algorithms that has been conducted since their inception in 1984 to date. In addition to providing short descriptions of the main algorithmic proposals, several ideas for future research in the area have been provided.

This overview has shown that the design of MOEAs has been a very active research area, which still has a wide variety of topics to be explored. Clearly, evolutionary multi-objective optimization is still a very promising research area which should remain very active for several more years. However, it is important to work in a diverse set of topics in order to avoid focusing only on the work by analogy (for example, producing more small variants of existing MOEAs).

Additionally, many fundamental topics still remain unexplored, thus offering great research opportunities for those interested in tackling them. For example, we are lacking theoretical studies related to the limitations of current MOEAs, which are fundamental for the development of the area. An interesting example of the importance of this topic is the study conducted by Schütze et al. [135] in which the actual source of difficulty in many-objective problems was analyzed. This study concluded that adding more objectives to a multi-objective problem does not necessarily makes it harder. According to this study (which has been largely ignored by several researchers working on many-objective optimization), the difficulty of many-objective problems is really associated with the intersection of the descent cones of the objectives (these descent cones are obtained with the combination of the gradients of each objective). This was somehow corroborated by an empirical study conducted by Ishibuchi et al. [68] in which it was shown that NSGA-II could properly solve many-objective knapsack problems in which the objectives were highly correlated. Clearly, the study of Schütze et al. [135] could had re-directed the research conducted in many-objective optimization, if researchers working in this area had taken it into account.

The main goal of this chapter is to serve as an introductory guide to those interested in tackling some of the many challenges that this research area still has to offer during the next few years. These days, such topics are not trivial to identify within the vast volume of references available. This highlights the importance of

providing a highly compressed overview of the research that has been conducted during 35 years on MOEAs. Hopefully, this chapter will serve to that purpose.

Acknowledgments The first author acknowledges support from CONACyT project no. 2016-01-1920 (*Investigación en Fronteras de la Ciencia 2016*) and from a project from the 2018 SEP-Cinvestav Fund (application no. 4).

References

1. Alcayde, A., Banos, R., Gil, C., Montoya, F.G., Moreno-Garcia, J., Gomez, J.: Annealing-tabu PAES: a multi-objective hybrid meta-heuristic. Optimization **60**(12), 1473–1491 (2011)
2. Alves Ribeiro, V.H., Reynoso-Meza, G.: Multi-objective support vector machines ensemble generation for water quality monitoring. In: 2018 IEEE Congress on Evolutionary Computation (CEC'2018), pp. 608–613. IEEE Press, Rio de Janeiro (2018). ISBN: 978-1-5090-6017-7
3. Amirahmadi, A., Dastfan, A., Rafiei, M.: Optimal controller design for single-phase PFC rectifiers using SPEA multi-objective optimization. J. Power Electron. **12**(1), 104–112 (2012)
4. Bader, J., Zitzler, E.: HypE: An algorithm for fast hypervolume-based many-objective optimization. Evol. Comput. **19**(1), 45–76 (2011)
5. Berenguer, J.A.M., Coello Coello, C.A.: Evolutionary many-objective optimization based on Kuhn–Munkres' algorithm. In: Gaspar-Cunha, A.., Antunes, C.H., Coello Coello, C. (eds.) 8th International Conference on Evolutionary Multi-Criterion Optimization, EMO 2015. Springer. Lecture Notes in Computer Science, vol. 9019, pp. 3–17, Guimarães, Portugal (2015)
6. Beume, N., Fonseca, C.M., Lopez-Ibanez, M., Paquete, L., Vahrenhold, J.: On the complexity of computing the hypervolume indicator. IEEE Trans. Evol. Comput. **13**(5), 1075–1082 (2009)
7. Beume, N., Naujoks, B., Emmerich, M.: SMS-EMOA: Multiobjective selection based on dominated hypervolume. Eur. J. Oper. Res. **181**(3), 1653–1669 (2007)
8. Blumel, A.L., Hughes, E.J., White, B.A.: Fuzzy autopilot design using a multiobjective evolutionary algorithm. In: 2000 IEEE Congress on Evolutionary Computation, vol. 1, pp. 54–61. IEEE Service Center, Piscataway (2000)
9. Bora, T.C., Lebensztajn, L., Coelho, L.D.S.: Non-dominated sorting genetic algorithm based on reinforcement learning to optimization of broad-band reflector antennas satellite. IEEE Trans. Magn. **48**(2), 767–770 (2012)
10. Bouter, A., Alderliesten, T., Bel, A., Witteveen, C., Bosman, P.A.N.: Large-scale parallelization of partial evaluations in evolutionary algorithms for real-world problems. In: 2018 Genetic and Evolutionary Computation Conference (GECCO'2018), pp. 1199–1206. ACM Press, Kyoto, (2018). ISBN: 978-1-4503-5618-3
11. Brockhoff, D., Wagner, T., Trautmann, H.: On the properties of the $R2$ indicator. In: 2012 Genetic and Evolutionary Computation Conference (GECCO'2012), pp. 465–472. ACM Press, Philadelphia (2012). ISBN: 978-1-4503-1177-9
12. Brockhoff, D., Wagner, T., Trautmann, H.: R2 indicator-based multiobjective search. Evol. Comput. **23**(3), 369–395 (2015)
13. Brockhoff, D., Zitzler, E.: Objective reduction in evolutionary multiobjective optimization: theory and applications. Evol. Comput. **17**(2), 135–166 (2009)
14. Burke, E.K., Gendreau, M., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., Qu, R.: Hyper-heuristics: a survey of the state of the art. J. Oper. Res. Soc. **64**(12), 1695–1724 (2013)
15. Burke, E.K., Landa Silva, J.D., Soubeiga, E.: Multi-objective hyper-heuristic approaches for space allocation and timetabling. In: Ibaraki, T., Nonobe, K., Yagiura, M. (eds.) Metaheuristics: Progress as Real Problem Solvers, Selected Papers from the Fifth Metaheuristics International Conference (MIC 2003), pp. 129–158. Springer, Berlin (2005)

16. Cao, B., Zhao, J., Lv, Z., Liu, X., Yang, S., Kang, X., Kang, K.: Distributed parallel particle swarm optimization for multi-objective and many-objective large-scale optimization. IEEE Access **5**, 8214–8221 (2017)
17. Chen, X., Du, W., Qian, F.: Multi-objective differential evolution with ranking-based mutation operator and its application in chemical process optimization. Chemom. Intell. Lab. Syst. **136**, 85–96 (2014)
18. Cheng, R., Jin, Y., Olhofer, M., Sendhoff, B.: Test problems for large-scale multiobjective and many-objective optimization. IEEE Trans. Cybern. **47**(12), 4108–4121 (2017)
19. Coello Coello, C.A.: Treating constraints as objectives for single-objective evolutionary optimization. Eng. Optim. **32**(3), 275–308 (2000)
20. Coello Coello, C.A.: A short tutorial on evolutionary multiobjective optimization. In: Zitzler, E., Deb, K., Thiele, L., Coello, C.A.C., Corne, D. (eds.) First International Conference on Evolutionary Multi-Criterion Optimization. Lecture Notes in Computer Science No. 1993, pp. 21–40. Springer, Berlin (2001)
21. Coello Coello, C.A., Christiansen, A.D.: Two new GA-based methods for multiobjective optimization. Civil Eng. Syst. **15**(3), 207–243 (1998)
22. Coello Coello, C.A., Lamont, G.B. (eds.): Applications of Multi-Objective Evolutionary Algorithms. World Scientific, Singapore (2004). ISBN 981-256-106-4
23. Coello Coello, C.A., Lamont, G.B., Van Veldhuizen, D.A.: Evolutionary Algorithms for Solving Multi-Objective Problems, 2nd edn. Springer, New York (2007). ISBN 978-0-387-33254-3
24. Cooper, I.M., John, M.P., Lewis, R., Mumford, C.L., Olden, A.: Optimising large scale public transport network design problems using mixed-mode parallel multi-objective evolutionary algorithms. In: 2014 IEEE Congress on Evolutionary Computation (CEC'2014), pp. 2841–2848. IEEE Press, Beijing (2014). ISBN 978-1-4799-1488-3
25. Corne, D.W., Knowles, J.D., Oates, M.J.: The Pareto envelope-based selection algorithm for multiobjective optimization. In: Schoenauer, M., Deb, K., Rudolph, G., Yao, X., Lutton, E., Merelo, J.J., Schwefel, H.P. (eds.) Proceedings of the Parallel Problem Solving from Nature VI Conference. Springer. Lecture Notes in Computer Science No. 1917, pp. 839–848, Paris (2000)
26. Corne, D.W., Jerram, N.R., Knowles, J.D., Oates, M.J.: PESA-II: region-based selection in evolutionary multiobjective optimization. In: Spector, L., Goodman, E.D., Wu, A., Langdon, W., Voigt, H.M., Gen, M. Sen, S., Dorigo, M., Pezeshk, S., Garzon, M.H., Burke, E. (eds.) Proceedings of the Genetic and Evolutionary Computation Conference (GECCO'2001), pp. 283–290. Morgan Kaufmann Publishers, San Francisco (2001)
27. Dai, L., Zhang, P., Wang, Y., Jiang, D., Dai, H., Mao, J., Wang, M.: Multi-objective optimization of cascade reservoirs using NSGA-II: a case study of the three Gorges-Gezhouba Cascade reservoirs in the Middle Yangtze River, China. Hum. Ecol. Risk Assess. **23**(4), 814–835 (2017)
28. Das, D., Patvardhan, C.: New multi-objective stochastic search technique for economic load dispatch. IEEE Proc. Gener. Transm. Distrib. **145**(6), 747–752 (1998)
29. Das, I., Dennis, J.: A closer look at drawbacks of minimizing weighted sums of objectives for Pareto set generation in multicriteria optimization problems. Struct. Optim. **14**(1), 63–69 (1997)
30. Deb, K.: Solving goal programming problems using multi-objective genetic algorithms. In: 1999 Congress on Evolutionary Computation, pp. 77–84. IEEE Service Center, Washington (1999)
31. Deb, K., Jain, H.: An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. IEEE Trans. Evol. Comput. **18**(4), 577–601 (2014)
32. Deb, K., Agrawal, S., Pratap, A., Meyarivan, T.: A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II. In: Schoenauer, M., Deb, K., Rudolph,

G., Yao, X., Lutton, E., Merelo, J.J., Schwefel, H.P. (eds.) Proceedings of the Parallel Problem Solving from Nature VI Conference. Springer. Lecture Notes in Computer Science No. 1917, pp. 849–858, Paris (2000)
33. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A Fast and Elitist Multiobjective Genetic Algorithm: NSGA–II. IEEE Trans. Evol. Comput. **6**(2), 182–197 (2002)
34. Dos Santos, B.C., Neri Nobre, C., Zárate, L.E.: Multi-objective genetic algorithm for feature selection in a protein function prediction context. In: 2018 IEEE Congress on Evolutionary Computation (CEC'2018), pp. 2267–2274. IEEE Press, Rio de Janeiro (2018). ISBN: 978-1-5090-6017-7
35. Eklund, N.H.W.: Multiobjective visible spectrum optimization: a genetic algorithm approach. Ph.D. thesis, Rensselaer Polytechnic Institute, Troy, New York (2002)
36. Emmerich, M., Beume, N., Naujoks, B.: An EMO algorithm using the hypervolume measure as selection criterion. In: Coello Coello, C.A., Hernández Aguirre, A., Zitzler, E. (eds.) Evolutionary Multi-Criterion Optimization. Third International Conference, EMO 2005. Springer. Lecture Notes in Computer Science, vol. 3410, pp. 62–76. Guanajuato, México (2005)
37. Falcón-Cardona, J.G., Coello Coello, C.A.: A multi-objective evolutionary hyper-heuristic based on multiple indicator-based density estimators. In: 2018 Genetic and Evolutionary Computation Conference (GECCO'2018), pp. 633–640. ACM Press, Kyoto (2018). ISBN: 978-1-4503-5618-3
38. Fan, Q., Wang, W., Yan, X.: Multi-objective differential evolution with performance-metric-based self-adaptive mutation operator for chemical and qbiochemical dynamic optimization problems. Appl. Soft Comput. **59**, 33–44 (2017)
39. Fang, Y., Liu, Q., Li, M., Laili, Y., Duc Truong, P.: Evolutionary many-objective optimization for mixed-model disassembly line balancing with multi-robotic workstations. Eur. J. Oper. Res. **276**(1), 160–174 (2019)
40. Fleischer, M.: The measure of Pareto optima. Applications to multi-objective metaheuristics. In: Fonseca, C.M., Fleming, P.J., Zitzler, E., Deb, K., Thiele, L. (eds.) Evolutionary Multi-Criterion Optimization. Second International Conference, EMO 2003. Lecture Notes in Computer Science, vol. 2632, pp. 519–533. Springer, Faro (2003)
41. Fogel, L.J.: Artificial Intelligence through Simulated Evolution. John Wiley, New York (1966)
42. Fogel, L.J.: Artificial Intelligence Through Simulated Evolution. Forty Years of Evolutionary Programming. Wiley, New York (1999)
43. Fonseca, C.M., Fleming, P.J.: Genetic algorithms for multiobjective optimization: formulation, discussion and generalization. In: Forrest, S. (ed.) Proceedings of the Fifth International Conference on Genetic Algorithms, pp. 416–423. University of Illinois at Urbana-Champaign, Morgan Kauffman Publishers, San Mateo, California (1993)
44. Fourman, M.P.: Compaction of symbolic layout using genetic algorithms. In: Genetic Algorithms and their Applications: Proceedings of the First International Conference on Genetic Algorithms, pp. 141–153. Lawrence Erlbaum (1985)
45. Gacôgne, L.: Research of Pareto Set by Genetic Algorithm, Application to Multicriteria Optimization of Fuzzy Controller. In: Fifth European Congress on Intelligent Techniques and Soft Computing EUFIT'97, pp. 837–845. Aachen (1997)
46. Gagin, A., Allen, A.J., Levin, I.: Combined fitting of small- and wide-angle X-ray total scattering data from nanoparticles: benefits and issues. J. Appl. Crystallogr. **47**, 619–629 (2014)
47. Garza Fabre, M., Toscano Pulido, G., Coello Coello, C.A.: Ranking methods for many-objective problems. In: Aguirre, A.H., Borja, R.M., García, C.A.R. (eds.) MICAI 2009: Advances in Artificial Intelligence. 8th Mexican International Conference on Artificial Intelligence, pp. 633–645. Springer. Lecture Notes in Artificial Intelligence, vol. 5845. Guanajuato, México (2009)
48. Goh, C.K., Tan, K.C.: A competitive-cooperative coevolutionary paradigm for dynamic multiobjective optimization. IEEE Trans. Evol. Comput. **13**(1), 103–127 (2009)

49. Goldberg, D.E.: Genetic Algorithms in Search, Optimization and Machine Learning. Addison-Wesley Publishing Company, Reading (1989)
50. Goldberg, D.E., Richardson, J.: Genetic algorithms with sharing for multimodal function optimization. In: Genetic Algorithms and their Applications: Proceedings of the Second International Conference on Genetic Algorithms, pp. 41–49. Lawrence Erlbaum, Massachusetts (1987). ISBN 0-8058-0158-8
51. Golshan, A., Ghodsiyeh, D., Izman, S.: Multi-objective optimization of wire electrical discharge machining process using evolutionary computation method: effect of cutting variation. Proc. Inst. Mech. Eng. B J. Eng. Manuf. **229**(1), 75–85 (2015)
52. Guerreiro, A.P., Fonseca, C.M.: Computing and Updating Hypervolume Contributions in up to Four Dimensions. IEEE Trans. Evol. Comput. **22**(3), 449–463 (2018)
53. Hadka, D., Reed, P.: Borg: an auto-adaptive many-objective evolutionary computing framework. Evol. Comput. **21**(2), 231–259 (2013)
54. Hajela, P., Lin, C.Y.: Genetic search strategies in multicriterion optimal design. Struct. Optim. **4**, 99–107 (1992)
55. Harel, M., Matalon-Eisenstadt, E., Moshaiov, A.: Solving multi-objective games using a-priori auxiliary criteria. In: 2017 IEEE Congress on Evolutionary Computation (CEC'2017), pp. 1428–1435. IEEE Press, San Sebastián (2017). ISBN 978-1-5090-4601-0
56. Hemmat Esfe, M., Razi, P., Hajmohammad, M.H., Rostamian, S.H., Sarsam, W.S., Arani, A.A.A., Dahari, M.: Optimization, modeling and accurate prediction of thermal conductivity and dynamic viscosity of stabilized ethylene glycol and water mixture Al_2O_3 nanofluids by NSGA-II using ANN. Int. Commun. Heat Mass Transf. **82**, 154–160 (2017)
57. Hernández Gómez, R., Coello Coello, C.A.: Improved metaheuristic based on the $r2$ indicator for many-objective optimization. In: 2015 Genetic and Evolutionary Computation Conference (GECCO 2015), pp. 679–686. ACM Press, Madrid (2015). ISBN 978-1-4503-3472-3
58. Hernández Gómez, R., Coello Coello, C.A.: A hyper-heuristic of scalarizing functions. In: 2017 Genetic and Evolutionary Computation Conference (GECCO'2017), pp. 577–584. ACM Press, Berlin (2017). ISBN 978-1-4503-4920-8
59. Hernández Gómez, R., Coello Coello, C.A., Alba Torres, E.: A multi-objective evolutionary algorithm based on parallel coordinates. In: 2016 Genetic and Evolutionary Computation Conference (GECCO'2016), pp. 565–572. ACM Press, Denver (2016). ISBN 978-1-4503-4206-3
60. Ho-Huu, V., Hartjes, S., Visser, H.G., Curran, R.: An improved MOEA/D algorithm for bi-objective optimization problems with complex Pareto fronts and its application to structural optimization. Expert Syst. Appl. **92**, 430–446 (2018)
61. Holland, J.H.: Outline for a logical theory of adaptive systems. J. Assoc. Comput. Mach. **9**, 297–314 (1962)
62. Horn, J., Nafpliotis, N., Goldberg, D.E.: A Niched Pareto genetic algorithm for multiobjective optimization. In: Proceedings of the First IEEE Conference on Evolutionary Computation, IEEE World Congress on Computational Intelligence, vol. 1, pp. 82–87. IEEE Service Center, Piscataway (1994)
63. Hu, H., Xu, L., Goodman, E.D., Zeng, S.: NSGA-II-based nonlinear PID controller tuning of greenhouse climate for reducing costs and improving performances. Neural Comput. Appl. **24**(3–4), 927–936 (2014)
64. Huang, B., Buckley, B., Kechadi, T.M.: Multi-objective feature selection by using NSGA-II for customer churn prediction in telecommunications. Expert Syst. Appl. **37**(5), 3638–3646 (2010)
65. Hutter, F., Hoos, H.H., Leyton-Brown, K., Stützle, T.: ParamILS: an automatic algorithm configuration framework. J. Artif. Intell. Res. **36**, 267–306 (2009)
66. Igel, C., Hansen, N., Roth, S.: Covariance matrix adaptation for multi-objective optimization. Evol. Comput. **15**(1), 1–28 (2007)
67. Ikeya, K., Shimoda, M., Shi, J.X.: Multi-objective free-form optimization for shape and thickness of shell structures with composite materials. Compos. Struct. **135**, 262–275 (2016)

68. Ishibuchi, H., Akedo, N., Nojima, Y.: Behavior of multiobjective evolutionary algorithms on many-objective knapsack problems. IEEE Trans. Evol. Comput. **19**(2), 264–283 (2015)
69. Ishibuchi, H., Masuda, H., Tanigaki, Y., Nojima, Y.: Modified distance calculation in generational distance and inverted generational distance. In: Gaspar-Cunha, A., Antunes, C.H., Coello Coello, C. (eds.) Eighth International Conference on Evolutionary Multi-Criterion Optimization, EMO 2015. Lecture Notes in Computer Science, vol. 9019, pp. 110–125. Springer Guimarães (2015)
70. Ishibuchi, H., Murata, T.: Multi-objective genetic local search algorithm. In: Fukuda, T., Furuhashi, T. (eds.) Proceedings of the 1996 International Conference on Evolutionary Computation, pp. 119–124. IEEE, Nagoya (1996)
71. Ishibuchi, H., Setoguchi, Y., Masuda, H., Nojima, Y.: Performance of decomposition-based many-objective algorithms strongly depends on Pareto front shapes. IEEE Trans. Evol. Comput. **21**(2), 169–190 (2017)
72. Jaszkiewicz, A.: Improved quick hypervolume algorithm. Comput. Oper. Res. **90**, 72–83 (2018)
73. Jiang, M., Huang, Z., Jiang, G., Shi, M., Zeng, X.: Motion generation of multi-legged robot in complex terrains by using estimation of distribution algorithm. In: 2017 IEEE Symposium Series on Computational Intelligence (SSCI'2017), pp. 111–116. IEEE Press, Honolulu (2017). ISBN: 978-1-5386-2727-3
74. Jin, Y., Okabe, T., Sendhoff, B.: Dynamic weighted aggregation for evolutionary multi-objective optimization: why does it work and how? In: Spector, L., Goodman, E.D., Wu, A., Langdon, W., Voigt, H.M., Gen, M., Sen, S., Dorigo, M., Pezeshk, S., Garzon, M.H., Burke, E. (eds.) Proceedings of the Genetic and Evolutionary Computation Conference (GECCO'2001), pp. 1042–1049. Morgan Kaufmann Publishers, San Francisco (2001)
75. Karakostas, S.M.: Land-use planning via enhanced multi-objective evolutionary algorithms: optimizing the land value of major greenfield initiatives. J. Land Use Sci. **11**(5), 595–617 (2016)
76. Karakostas, S.M.: Bridging the gap between multi-objective optimization and spatial planning: a new post-processing methodology capturing the optimum allocation of land uses against established transportation infrastructure. Trans. Plan. Technol. **40**(3), 305–326 (2017)
77. Kim, N., Bhalerao, I., Han, D., Yang, C., Lee, H.: Improving surface roughness of additively manufactured parts using a photopolymerization model and multi-objective particle swarm optimization. Appl. Sci. Basel **9**(1), 151 (2019). Article Number:151
78. Knowles, J., Corne, D.: Properties of an adaptive archiving algorithm for storing nondominated vectors. IEEE Trans. Evol. Comput. **7**(2), 100–116 (2003)
79. Knowles, J.D.: Local-Search and Hybrid Evolutionary Algorithms for Pareto Optimization. Ph.D. thesis, The University of Reading, Department of Computer Science, Reading, UK (2002)
80. Knowles, J.D., Corne, D.W.: Approximating the nondominated front using the Pareto archived evolution strategy. Evol. Comput. **8**(2), 149–172 (2000)
81. Kuhn, H.W.: The Hungarian method for the assignment problem. Naval Res. Logist. Q. **2**(1–2), 83–97 (1955). https://doi.org/10.1002/nav.3800020109
82. Kuhn, H.W., Tucker, A.W.: Nonlinear programming. In: Neyman, J. (ed.) Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability, pp. 481–492. University of California Press, Berkeley (1951)
83. Kukkonen, S., Deb, K.: Improved pruning of non-dominated solutions based on crowding distance for bi-objective optimization problems. In: 2006 IEEE Congress on Evolutionary Computation (CEC'2006), pp. 1164–1171. IEEE, Vancouver (2006)
84. Lacour, R., Klamroth, K., Fonseca, C.M.: A box decomposition algorithm to compute the hypervolume indicator. Comput. Oper. Res. **79**, 347–360 (2017)
85. Lepš, M.: Single and multi-objective optimization in civil engineering. In: Annicchiarico, W., Périaux, J., Cerrolaza, M., Winter, G. (eds.) Evolutionary Algorithms and Intelligent Tools in Engineering Optimization, pp. 322–342. WIT Press, CIMNE Barcelona, Southampton, Boston (2005). ISBN 1-84564-038-1

86. Li, F., Cheng, R., Liu, J., Jin, Y.: A two-stage r2 indicator based evolutionary algorithm for many-objective optimization. Appl. Soft Comput. **67**, 245–260 (2018)
87. Li, H., Deb, K.: Challenges for evolutionary multiobjective optimization algorithms in solving variable-length problems. In: 2017 IEEE Congress on Evolutionary Computation (CEC'2017), pp. 2217–2224. IEEE Press, San Sebastián (2017). ISBN 978-1-5090-4601-0
88. Li, Z., Zheng, L.: Integrated design of active suspension parameters for solving negative vibration effects of switched reluctance-in-wheel motor electrical vehicles based on multi-objective particle swarm optimization. J. Vibr. Control **25**(3), 639–654 (2019)
89. Lopez-Herrejon, R.E., Ferrer, J., Chicano, F., Egyed, A., Alba, E.: Comparative analysis of classical multi-objective evolutionary algorithms and seeding strategies for pairwise testing of software product lines. In: 2014 IEEE Congress on Evolutionary Computation (CEC'2014), pp. 387–396. IEEE Press, Beijing (2014). ISBN 978-1-4799-1488-3
90. Lotfan, S., Ghiasi, R.A., Fallah, M., Sadeghi, M.H.: ANN-based modeling and reducing dual-fuel engine's challenging emissions by multi-objective evolutionary algorithm NSGA-II. Appl. Energy **175**, 91–99 (2016)
91. Ma, X., Liu, F., Qi, Y., Wang, X., Li, L., Jiao, L., Yin, M., Gong, M.: A multiobjective evolutionary algorithm based on decision variable analyses for multiobjective optimization problems with large-scale variables. IEEE Trans. Evol. Comput. **20**(2), 275–298 (2016)
92. Ma, Y., Zuo, X., Huang, X., Gu, F., Wang, C., Zhao, X.: A MOEA/D based approach for hospital department layout design. In: 2016 IEEE Congress on Evolutionary Computation (CEC'2016), pp. 793–798. IEEE Press, Vancouver (2016). ISBN 978-1-5090-0623-9
93. Makaremi, Y., Haghighi, A., Ghafouri, H.R.: Optimization of pump scheduling program in water supply systems using a self-adaptive NSGA-II; a review of theory to real application. Water Resour. Manag. **31**(4), 1283–1304 (2017)
94. Manoatl Lopez, E., Coello Coello, C.A.: IGD^+-EMOA: a multi-objective evolutionary algorithm based on IGD^+. In: 2016 IEEE Congress on Evolutionary Computation (CEC'2016), pp. 999–1006. IEEE Press, Vancouver (2016). ISBN 978-1-5090-0623-9
95. Manoatl Lopez, E., Coello Coello, C.A.: An improved version of a reference-based multi-objective evolutionary algorithm based on IGD+. In: 2018 Genetic and Evolutionary Computation Conference (GECCO'2018), pp. 713–720. ACM Press, Kyoto (2018). ISBN: 978-1-4503-5618-3
96. Marco, N., Lanteri, S., Desideri, J.A., Périaux, J.: A Parallel genetic algorithm for multi-objective optimization in computational fluid dynamics. In: Miettinen, K., Mäkelä, M.M., Neittaanmäki, P., Périaux, J. (eds.) Evolutionary Algorithms in Engineering and Computer Science, chap. 22, pp. 445–456. Wiley, Chichester (1999)
97. Marcu, T., Ferariu, L., Frank, P.M.: Genetic evolving of dynamic neural networks with application to process fault diagnosis. In: Procedings of the EUCA/IFAC/IEEE European Control Conference ECC'99. CD-ROM, F-1046,1, Karlsruhe (1999)
98. Mariani, T., Guizzo, G., Vergilio, S.R., Pozo, A.T.: Grammatical evolution for the multi-objective integration and test order problem. In: 2016 Genetic and Evolutionary Computation Conference (GECCO'2016), pp. 1069–1076. ACM Press, Denver (2016). ISBN 978-1-4503-4206-3
99. Martí, L., García, J., Berlanga, A., Molina, J.M.: Introducing MONEDA: scalable multiobjective optimization with a neural estimation of distribution algorithm. In: 2008 Genetic and Evolutionary Computation Conference (GECCO'2008), pp. 689–696. ACM Press, Atlanta (2008). ISBN 978-1-60558-131-6
100. Mazumdar, A., Chugh, T., Miettinen, K., nez, M.L.I.: On dealing with uncertainties from kriging models in offline data-driven evolutionary multiobjective optimization. In: Evolutionary Multi-Criterion Optimization, Tenth International Conference, EMO 2019, pp. 463–474. Springer. Lecture Notes in Computer Science, vol. 11411, East Lansing (2019). ISBN: 978-3-030-12597-4
101. Menchaca-Mendez, A., Coello Coello, C.A.: An alternative hypervolume-based selection mechanism for multi-objective evolutionary algorithms. Soft Comput. **21**(4), 861–884 (2017)

102. Mendes Guimarães, M., Cruzeiro Martins, F.V.: A multiobjective approach applying in a Brazilian emergency medical service. In: 2018 IEEE Congress on Evolutionary Computation (CEC'2018), pp. 1605–1612. IEEE Press, Rio de Janeiro (2018). ISBN: 978-1-5090-6017-7
103. Mendoza, F., Bernal-Agustin, J.L., Navarro, J.A.D.: NSGA and SPEA applied to multiobjective design of power distribution systems. IEEE Trans. Power Syst. **21**(4), 1938–1945 (2006)
104. Miettinen, K.M.: Nonlinear Multiobjective Optimization. Kluwer Academic Publishers, Boston (1999)
105. Miguel Antonio, L., Coello Coello, C.A.: Decomposition-based approach for solving large scale multi-objective problems. In: Handl, J., Hart, E., Lewis, P.R., López-Ibáñez, M., Ochoa, G., Paechter, B. (eds.) 14th International Conference on Parallel Problem Solving from Nature—PPSN XIV, pp. 525–534. Springer. Lecture Notes in Computer Science, vol. 9921, Edinburgh (2016). ISBN 978-3-319-45822-9
106. Miguel Antonio, L., Coello Coello, C.A.: Coevolutionary multiobjective evolutionary algorithms: survey of the state-of-the-art. IEEE Trans. Evol. Comput. **22**(6), 851–865 (2018)
107. Miguel Antonio, L., Molinet Berenguer, J.A., Coello Coello, C.A.: Evolutionary many-objective optimization based on linear assignment problem transformations. Soft Comput. **22**(16), 5491–5512 (2018)
108. Mishra, S., Coello Coello, C.A.: Parallelism in divide-and-conquer non-dominated sorting: a theoretical study considering the PRAM-CREW model. J. Heuristics **25**(3), 455–483 (2019)
109. Moghadasi, A.H., Heydari, H., Farhadi, M.: Pareto Optimality for the Design of SMES Solenoid Coils Verified by Magnetic Field Analysis. IEEE Trans. Appl. Supercond. **21**(1), 13–20 (2011)
110. Morse, J.: Reducing the size of the nondominated set: pruning by clustering. Comput. Oper. Res. **7**(1–2), 55–66 (1980)
111. Moudani, W.E., Cosenza, C.A.N., de Coligny, M., Mora-Camino, F.: A bi-criterion approach for the airlines crew rostering problem. In: Zitzler, E., Deb, K., Thiele, L., Coello Coello, C.A., Corne, D. (eds.) First International Conference on Evolutionary Multi-Criterion Optimization. Lecture Notes in Computer Science, vol. 1993, pp. 486–500. Springer, Berlin (2001)
112. Muller, J.: SOCEMO: surrogate optimization of computationally expensive multiobjective problems. Informs J. Comput. **29**(4), 581–596 (2017)
113. Narayanan, S., Azarm, S.: On improving multiobjective genetic algorithms for design optimization. Struct. Optim. **18**, 146–155 (1999)
114. Lopez-Ibanez, M.L.I., Prasad, T.D., Paechter, B.: Multi-objective optimisation of the pump scheduling problem using SPEA2. In: 2005 IEEE Congress on Evolutionary Computation (CEC'2005), vol. 1, pp. 435–442. IEEE Service Center, Edinburgh (2005)
115. Arias-Montano, A.A.M., Coello Coello, C.A., Mezura-Montes, E.: Multi-objective evolutionary algorithms in aeronautical and aerospace engineering. IEEE Trans. Evol. Comput. **16**(5), 662–694 (2012)
116. Ortega, G., Filatovas, E., Garzon, E.M., Casado, L.G.: Non-dominated sorting procedure for pareto dominance ranking on multicore CPU and/or GPU. J. Global Optim. **69**(3), 607–627 (2017)
117. Palar, P.S., Shimoyama, K.: Multiple metamodels for robustness estimation in multi-objective robust optimization. In: Evolutionary Multi-Criterion Optimization, Ninth International Conference, EMO 2017, pp. 469–483. Springer. Lecture Notes in Computer Science, vol. 10173, Münster (2017). ISBN 978-3-319-54156-3
118. Peng, Y., Xue, S., Li, M.: An improved multi-objective optimization algorithm based on NPGA for cloud task scheduling. Int. J. Grid Distrib. Comput. **9**(4), 161–176 (2016)
119. Pescador-Rojas, M., Hernández Gómez, R., Montero, E., Rojas-Morales, N., Riff, M.C., Coello Coello, C.A.: An overview of weighted and unconstrained scalarizing functions. In: Trautmann, H., Rudolph, G., Klamroth, K., Schütze, O., Wiecek, M. Jin, Y., Grimme, C. (eds.) Ninth International Conference on Evolutionary Multi-Criterion Optimization, EMO 2017, pp. 499–513. Springer. Lecture Notes in Computer Science, vol. 10173, Münster (2017). ISBN 978-3-319-54156-3

120. Praditwong, K., Yao, X.: How well do multi-objective evolutionary algorithms scale to large problems. In: 2007 IEEE Congress on Evolutionary Computation (CEC'2007), pp. 3959–3966. IEEE Press, Singapore (2007)
121. Quintana, D., Denysiuk, R., Garcia-Rodriguez, S., Gaspar-Cunha, A.: Portfolio implementation risk management using evolutionary multiobjective optimization. Appl. Sci. Basel **7**(10), 1079 (2017). Article Number: 1079
122. Rabiee, M., Zandieh, M., Ramezani, P.: Bi-objective partial flexible job shop scheduling problem: NSGA-II, NRGA, MOGA and PAES approaches. Int. J. Prod. Res. **50**(24), 7327–7342 (2012)
123. Roberge, V., Tarbouchi, M., Labonte, G.: Comparison of parallel genetic algorithm and particle swarm optimization for real-time UAV path planning. IEEE Trans. Ind. Inform. **9**(1), 132–141 (2013)
124. Rocha, G.K., dos Santos, K.B., Angelo, J.S., Custódio, F.L., Barbosa, H.J.C., Dardenne, L.E.: Inserting co-evolution information from contact maps into a multiobjective genetic algorithm for protein structure prediction. In: 2018 IEEE Congress on Evolutionary Computation (CEC'2018), pp. 957–964. IEEE Press, Rio de Janeiro (2018). ISBN: 978-1-5090-6017-7
125. Rosenberg, R.S.: Simulation of genetic populations with biochemical properties. Ph.D. thesis, University of Michigan, Ann Arbor, Michigan (1967)
126. Rubaiee, S., Yildirim, M.B.: An energy-aware multiobjective ant colony algorithm to minimize total completion time and energy cost on a single-machine preemptive scheduling. Comput. Ind. Eng. **127**, 240–252 (2019)
127. Rudolph, G., Agapie, A.: Convergence properties of some multi-objective evolutionary algorithms. In: Proceedings of the 2000 Conference on Evolutionary Computation, vol. 2, pp. 1010–1016. IEEE Press, Piscataway (2000)
128. Sadowski, K.L., van der Meer, M.C., Hoang Luong, N., Alderliesten, T., Thierens, D., van der Laarse, R., Niatsetski, Y., Bel, A., Bosman, P.A.N.: Exploring trade-offs between target coverage, healthy tissue sparing, and the placement of catheters in HDR brachytherapy for prostate cancer using a novel multi-objective model-based mixed-integer evolutionary algorithm. In: 2017 Genetic and Evolutionary Computation Conference (GECCO'2017), pp. 1224–1231. ACM Press, Berlin (2017). ISBN 978-1-4503-4920-8
129. Sandgren, E.: Multicriteria design optimization by goal programming. In: Adeli, H. (ed.) Advances in Design Optimization, chap. 23, pp. 225–265. Chapman & Hall, London (1994)
130. Sanhueza, C., Jiménez, F., Berretta, R., Moscato, P.: PasMoQAP: a parallel asynchronous memetic algorithm for solving the multi-objective quadratic assignment problem. In: 2017 IEEE Congress on Evolutionary Computation (CEC'2017), pp. 1103–1110. IEEE Press, San Sebastián (2017). ISBN 978-1-5090-4601-0
131. Santiago, A., Huacuja, H.J.F., Dorronsoro, B., Pecero, J.E., Santillan, C.G., Barbosa, J.J.G., Monterrubio, J.C.S.: A survey of decomposition methods for multi-objective optimization. In: Castillo, O., Melin, P., Pedrycz, W., Kacprzyk, J. (eds.) Recent Advances on Hybrid Approaches for Designing Intelligent Systems, pp. 453–465. Springer, Berlin (2014). ISBN 978-3-319-05170-3
132. Saxena, D.K., ao A. Duro, J., Tiwari, A., Deb, K., Zhang, Q.: Objective reduction in many-objective optimization: linear and nonlinear algorithms. IEEE Trans. Evol. Comput. **17**(1), 77–99 (2013)
133. Schaffer, J.D.: Multiple Objective Optimization with Vector Evaluated Genetic Algorithms. Ph.D. thesis, Vanderbilt University, Nashville, Tennessee, USA (1984)
134. Schaffer, J.D.: Multiple objective optimization with vector evaluated genetic algorithms. In: Genetic Algorithms and their Applications: Proceedings of the First International Conference on Genetic Algorithms, pp. 93–100. Lawrence Erlbaum, London (1985)
135. Schütze, O., Lara, A., Coello Coello, C.A.: On the influence of the number of objectives on the hardness of a multiobjective optimization problem. IEEE Trans. Evol. Comput. **15**(4), 444–455 (2011)
136. Schwefel, H.P.: Kybernetische evolution als strategie der experimentellen forschung inder strömungstechnik. Dipl.-Ing. thesis (1965) (in German)

137. Schwefel, H.P.: Numerical Optimization of Computer Models. Wiley, Chichester (1981)
138. Song, J., Yang, Y., Wu, J., Wu, J., Sun, X., Lin, J.: Adaptive surrogate model based multiobjective optimization for coastal aquifer management. J. Hydrol. **561**, 98–111 (2018)
139. Srinivas, N., Deb, K.: Multiobjective optimization using nondominated sorting in genetic algorithms. Evol. Comput. **2**(3), 221–248 (1994)
140. Toscano Pulido, G., Coello Coello, C.A.: The micro genetic algorithm 2: towards online adaptation in evolutionary multiobjective optimization. In: Fonseca, C.M., Fleming, P.J., Zitzler, E., Deb, K., Thiele, L. (eds.) Second International Conference on Evolutionary Multi-Criterion Optimization, EMO 2003, pp. 252–266. Springer. Lecture Notes in Computer Science, vol. 2632, Faro (2003)
141. Tušar, T., Filipič, B.: Visualization of Pareto front approximations in evolutionary multiobjective optimization: a critical review and the prosection method. IEEE Trans. Evol. Comput. **19**(2), 225–245 (2015)
142. Vazquez-Rodriguez, J.A., Petrovic, S.: A new dispatching rule based genetic algorithm for the multi-objective job shop problem. J. Heuristics **16**(6), 771–793 (2010)
143. Vrugt, J.A., Robinson, B.A.: Improved evolutionary optimization from genetically adaptive multimethod search. Proc. Nat. Acad. Sci. U.S.A. **104**(3), 708–711 (2007)
144. Walker, D.J., Keedwell, E.: Multi-objective optimisation with a sequence-based selection hyper-heuristic. In: Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion, pp. 81–82. ACM Press, New York (2016)
145. Wang, L., Li, L.P.: Fixed-structure h-infinity controller synthesis based on differential evolution with level comparison. IEEE Trans. Evol. Comput. **15**(1), 120–129 (2011)
146. Wang, Q., Guidolin, M., Savic, D., Kapelan, Z.: Two-objective design of benchmark problems of a water distribution system via MOEAs: towards the best-known approximation of the true Pareto front. J. Water Resour. Plan. Manag. **141**(3), 04014060 (2015)
147. Wang, S., Hua, D., Zhang, Z., Li, M., Yao, K., Wen, Z.: Robust controller design for main steam pressure based on SPEA2. In: Huang, D.S., Gan, Y., Premaratne, P., Han, K. (eds.) Bio-Inspired Computing and Applications, Seventh International Conference on Intelligent Computing, ICIC 2011, pp. 176–182. Springer. Lecture Notes in Computer Science, vol. 6840, Zhengzhou (2012)
148. Weile, D.S., Michielssen, E.: Integer coded Pareto genetic algorithm design of constrained antenna arrays. Electron. Lett. **32**(19), 1744–1745 (1996)
149. Wienke, P.B., Lucasius, C., Kateman, G.: Multicriteria target optimization of analytical procedures using a genetic algorithm. Anal. Chim. Acta **265**(2), 211–225 (1992)
150. Wilson, P.B., Macleod, M.D.: Low implementation cost IIR digital filter design using genetic algorithms. In: IEE/IEEE Workshop on Natural Algorithms in Signal Processing, pp. 4/1–4/8. Chelmsford (1993)
151. Yang, D., Sun, Y., di Stefano, D., Turrin, M., Sariyildiz, S.: Impacts of problem scale and sampling strategy on surrogate model accuracy. An application of surrogate-based optimization in building design. In: 2016 IEEE Congress on Evolutionary Computation (CEC'2016), pp. 4199–4207. IEEE Press, Vancouver (2016). ISBN 978-1-5090-0623-6
152. Yang, W., Chen, Y., He, R., Chang, Z., Chen, Y.: The bi-objective active-scan agile earth observation satellite scheduling problem: modeling and solution approach. In: 2018 IEEE Congress on Evolutionary Computation (CEC'2018), pp. 1083–1090. IEEE Press, Rio de Janeiro (2018). ISBN: 978-1-5090-6017-7
153. Ye, C.J., Huang, M.X.: Multi-objective optimal power flow considering transient stability based on parallel NSGA-II. IEEE Trans. Power Syst. **30**(2), 857–866 (2015)
154. Ye, X., Liu, S., Yin, Y., Jin, Y.: User-oriented many-objective cloud workflow scheduling based on an improved knee point driven evolutionary algorithm. Knowl. Based Syst. **135**, 113–124 (2017)
155. Zebulum, R.S., Pacheco, M.A., Vellasco, M.: A multi-objective optimisation methodology applied to the synthesis of low-power operational amplifiers. In: Cheuri, I.J., dos Reis Filho, C.A. (eds.) Proceedings of the XIII International Conference in Microelectronics and Packaging, vol. 1, pp. 264–271. Curitiba (1998)

156. Zhang, C., Chen, Y., Shi, M., Peterson, G.: Optimization of heat pipe with axial "Omega"-shaped micro grooves based on a niched Pareto genetic algorithm (NPGA). Appl. Thermal Eng. **29**(16), 3340–3345 (2009)
157. Zhang, Q., Li, H.: MOEA/D: a multiobjective evolutionary algorithm based on decomposition. IEEE Trans. Evol. Comput. **11**(6), 712–731 (2007)
158. Zhang, X., Tian, Y., Cheng, R., Jin, Y.: A decision variable clustering-based evolutionary algorithm for large-scale many-objective optimization. IEEE Trans. Evol. Comput. **22**(1), 97–112 (2018)
159. Zille, H., Ishibuchi, H., Mostaghim, S., Nojima, Y.: A framework for large-scale multiobjective optimization based on problem transformation. IEEE Trans. Evol. Comput. **22**(2), 260–275 (2018)
160. Zitzler, E.: Evolutionary Algorithms for Multiobjective Optimization: Methods and Applications. Ph.D. thesis, Swiss Federal Institute of Technology (ETH), Zurich, Switzerland (1999)
161. Zitzler, E., Deb, K., Thiele, L.: Comparison of multiobjective evolutionary algorithms on test functions of different difficulty. In: Wu, A.S. (ed.) Proceedings of the 1999 Genetic and Evolutionary Computation Conference. Workshop Program, pp. 121–122. Orlando, Florida (1999)
162. Zitzler, E., Künzli, S.: Indicator-based selection in multiobjective search. In: X.Y. et al. (ed.) Parallel Problem Solving from Nature—PPSN VIII, pp. 832–842. Springer. Lecture Notes in Computer Science, vol. 3242, Birmingham, UK (2004)
163. Zitzler, E., Laumanns, M., Thiele, L.: SPEA2: improving the strength pareto evolutionary algorithm. In: Giannakoglou, K., Tsahalis, D., Periaux, J., Papailou, P., Fogarty, T. (eds.) EUROGEN 2001. Evolutionary Methods for Design, Optimization and Control with Applications to Industrial Problems, pp. 95–100. Athens, Greece (2001)
164. Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C.M., da Fonseca, V.G.: Performance assessment of multiobjective optimizers: an analysis and review. IEEE Trans. Evol. Comput. **7**(2), 117–132 (2003)

Black-Box and Data-Driven Computation



Rong Jin, Weili Wu, My T. Thai, and Ding-Zhu Du

1 Introduction

Recently, "data-driven" is a trendy terminology in computational study. In data-driven computation, there exists a black box that contains a big amount of data, e.g., solutions for a certain problem. When the computation needs to solve a problem, its solution can be obtained from the black box by a data mining method, such as a machine learning method, instead of computing from the scratch. Today, this frame of computation is available due to the study of big data such that the technology for big data management and mining is already successfully established. The current data-driven computation is an application of such technologies.

It is not the first time that the black box appears in studying computation. However, previously, the black box plays a different role; it usually represents something unable to compute or hard to compute. Now, it represents something already known. What makes this change of its role? In machine learning, it is made by data training, that is, when the black box accumulates a large enough amount of data through preprocessing, the black box can be used for solving a problem with a machine learning method. In this short article, we would like to present some observations based on this idea.

There are many concepts and notations in computational complexity theory, which will be used in this article. Readers can find these concepts and notations in [1].

R. Jin · W. Wu · D.-Z. Du (✉)
Department of Computer Science, University of Texas at Dallas, Richardson, TX, USA
e-mail: rong.jin@utdallas.edu; weiliwu@utdallas.edu; dzdu@utdallas.edu

M. T. Thai
CISE Department, University of Florida, Gainesville, FL, USA
e-mail: mythai@cise.ufl.edu

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_6

2 Black Box and Oracle

An oracle in a Turing machine is a black box, which is like a subroutine in a computer program. Usually, the oracle is represented by a language or a function. If it is represented by a language A, then the oracle would be able to give a solution of the membership problem about A, that is, given an input string x, the oracle can tell whether x does belong to A or not. If an oracle is represented by a function f, then it can compute the function f, that is, given an input string x, the oracle returns a string $f(x)$. The oracle is considered as a black box because we do not care how the oracle obtains the output from the input and, hence, computational process inside of the oracle is black.

When analyzing the running time complexity and the space usage of an oracle Turing machine, only the computation outside the oracle is considered. In this way, fixed a language oracle A, a sequence of computational complexity classes P^A, NP^A, $PSPACE^A$, ..., called *relativized* classes, can be defined similarly to the well-known classes P, NP, $PSPACE$, ... The first interesting result on relativized classes came from Baker et al. [2] as follows.

Theorem 1 *There exists a language oracle A such that $P^A = NP^A$. There also exists a language oracle B such that $P^B \neq NP^B$.*

What is the significance of this result? To explain it, let us consider the following hierarchy theorem [1].

Theorem 2 (Time Hierarchy Theorem) *If t_2 is a fully time-constructible function, $t_2(n) \geq t_1(n) \geq n$, and*

$$\lim_{n\to\infty} \frac{t_1(n)\log(t_1(n))}{t_2(n)} = 0,$$

then $DTIME(t_2(n)) \setminus DTIME(t_1(n)) \neq \emptyset$.

This theorem is proven using the diagonalization argument and is an important tool for separating complexity classes. However, Theorem 1 indicates that the time hierarchy theorem cannot succeed to separate classes P and NP. The reason is as follows: With the same argument, the time hierarchy theorem for relativized complexity classes can also be established.

Theorem 3 *If t_2 is a fully time-constructible function, $t_2(n) \geq t_1(n) \geq n$, and*

$$\lim_{n\to\infty} \frac{t_1(n)\log(t_1(n))}{t_2(n)} = 0,$$

then $DTIME^C(t_2(n)) \setminus DTIME^C(t_1(n)) \neq \emptyset$, where C is any language oracle.

If the time hierarchy theorem can solve the $P - NP$ problem, then following the same logic pattern, Theorem 3 would be able to derive either $P^C = NP^C$ or

$P^C \neq NP^C$. However, by Theorem 1, in case $C = A$, $P^C = NP^C$, and in case $C = B$, $P^C \neq NP^C$, a contradiction.

Actually, there exist many variations of diagonalization argument in the recursive function theory. Theorem 1 indicates that they are all not powerful enough to solve the $P - NP$ problem.

There exist many efforts made following the work of Baker et al. [2]. Among them, it is worth mentioning those about relativized polynomial-time hierarchy class HP^A. Is there a language oracle A such that $PH^A \neq PSPACE^A$? This problem was identified as a hard problem and was solved in two steps.

In the first step, [3] established a relationship between this problem and the circuit complexity of parity function

$$p(x_1, x_2, \ldots, x_n) = x_1 \oplus x_2 \oplus \cdots \oplus x_n,$$

where $\oplus$ is the exclusive-or operation, i.e.,

$$x \oplus y = \begin{cases} 1 & \text{if there is exactly one 1 in } \{x, y\}; \\ 0 & \text{otherwise.} \end{cases}$$

The language defined by the parity function is

$$P = \{x_1 x_2 \cdots x_n \mid p(x_1, x_2, \ldots, x_n) = 1\}.$$

They showed that if the language P does not belong to class AC^0, then there exists a language oracle A such that $PH^A \neq PSPACE^A$.

In the second step, Yao [4] showed that the language P does not belong to AC^0. This result was one of the two big results appearing in 1985 about circuit complexity. Yao also claimed that he proved an existence of a language oracle A such that $\Sigma_k^{p,A} \neq \Sigma_{k+1}^{p,A}$ for any natural number k. However, his proof did not get a chance to be published due to Hastad's work. Hastad [5] simplified Yao's proof in [4] and, meanwhile, published his simplified proof for Yao's claim.

Later, Ko [6–9] showed a sequence of results about relativized classes about polynomial-time hierarchy, e.g., for any integer $k \geq 1$, there exists a language oracle A such that $PSPACE^A \neq PH^A = \Sigma_k^{p,A} \neq \Sigma_{k-1}^{p,A}$ [6].

3 Reduction

An important part of computational complexity theory is to study the complete problem in each complexity class, and the completeness is usually established through certain reduction. The reduction can be seen as an application of black box. For example, initially, the NP-complete problem was established through the polynomial-time Turing reduction, also called Cook-reduction today. For two

languages A and B, A is said to be Cook-reducible to B if the membership problem of A can be solved in polynomial time with a black box B. A problem in NP is called an NP-complete problem B if every problem A in NP is Cook-reducible to B.

The complete problem in each complexity class is the hardest problem in the class, related to certain type of computational problems. For example, the NP-complete problem is hardest with respect to nondeterministic polynomial-time computation. The P-complete problem is hardest with respect to efficient parallel computation. Therefore, the reduction is used for establishing the hardness of problems in the past.

Data-driven computation brings a new role for reduction, which is used not only for establishing the hardness of a problem, but also for providing solutions to some problems. For example, if a black box collects a large amount of data about solutions of an NP-complete problem, then all problems in NP would get "efficient" (in certain sense) solutions through the polynomial-time reduction. What new issues would this new role of reduction introduce? Let us present some of our observations in the next section.

4 Data-Driven Computation

When the black box is opened by data-driven computation, the following new issues may need to be considered.

Choice of Black Box First, note that due to current technology on data management, a black box may accumulate a very large amount of data; however, the quantity is still finite and cannot be infinitely large. Therefore, it cannot contain solutions of a problem for all inputs. This means that the data-driven algorithm aims mainly at practical solutions, not at theoretical. In a practical point of view, "polynomial-time" is not enough to describe the efficiency. For example, nobody would like to implement an algorithm with a running time of $O(n^{100})$ to solve a practical problem. Possibly, an algorithm with the running time at most $O(n^3)$ may be more practical. A question is motivated from this consideration: Is there an NP-complete problem A such that for any problem B in class NP, B can be reduced to A via a reduction with the running time at most $O(n^k)$ for a fixed number k? Following theorem gives us a negative answer.

Theorem 4 *For any integer $k \geq 1$, there is no NP-complete problem A such that for every problem B in NP, there exists a reduction, running in time at most $O(n^k)$, from B to A.*

Proof Consider an NP-complete problem A. Note that

$$\lim_{n\to\infty} \frac{n^k \log n^k}{n^{2k}} = 0.$$

By Theorem 3, there exists a language $B \in DTIME^A(n^{2k}) \setminus DTIME^A(n^k)$. This means that B can be Cook-reducible to A. However, the reduction cannot be running in time $O(n^k)$. Therefore, A cannot be the NP-complete problem satisfying the condition in the theorem. □

This theorem indicates that there does not exist an NP-complete problem that can be used for solving all problems in NP practically through reduction and using data-driven method. Therefore, we have to make a proper choice of black box for each small class of the real-world problems.

Speed Up Reduction There already exist many reductions in the literature for the proof of NP-completeness. In the past, these reductions only need to have the polynomial running time. Under the new setting of its new roles, we may want to speed up the reduction methods. Is it always possible to speed them up? Following negative answer is a corollary of Theorem 4.

Corollary 1 *For any integer $k \geq 1$ and any NP-complete problem A, there exists a problem in NP such that any reduction from B to A cannot run within time $O(n^k)$.*

Multiple Black Boxes A computer program may contain more than one subroutine. When data-driven technique is used, it is possible to use more than one black box. In the study of computational complexity theory for discrete problems, no effort has been made on more than one oracle. However, with the similar way, a time hierarchy theorem may be established regarding to more than one language oracle, by which following result can be proven with a similar argument to the proof of Theorem 4.

Theorem 5 *For any integer $k \geq 1$, there do not exist a finite number of NP-complete languages $A_1, A_2, \ldots, A_h$ such that the membership problem of every language B in NP can be solved by a $O(n^k)$-algorithm with language oracles $A_1, A_2, \ldots, A_h$.*

It is worth mentioning that more than one black box may be found in the information-based complexity theory for continuous problems [10]. Therefore, we may get some research ideas from there.

Complexity Issues To build a model for the data-driven computation, we have to face a computation with mixed of uniform and nonuniform computations, that is, carry out uniform computation outside the black box while implementing nonuniform computation inside the black box. This may bring to new issues in the study of computational complexity theory.

Acknowledgments This work is supported in part by NSF under grants 1747818, 1907472, and 1908594.

References

1. Du, D.-Z., Ko, K.-I.: Theory of Computational Complexity, 2nd edn. John Wiley and Sons, New York (2014)
2. Baker, T., Gill, J., Solovay, R.: Relativizations of the P = PNP question. SIAM J. Comput. **4**, 431–442 (1975)
3. Furst, M., Saxe, J., Sipser, M.: Parity, circuits, and the polynomial time hierarchy. Math. Syst. Theory **17**, 13–27 (1984)
4. Yao, A.: Separating the polynomial time hierarchy by oracle. In: Proceedings of 26th IEEE Symposium on Foundation of Computer Science, pp. 1–10 (1985)
5. Hastad, J.T.: Almost optimal lower bounds for small depth circuits. In: Proceedings of 18th ACM Symposium on Theory of Computing, pp. 6–20 (1986)
6. Ko, K.-I.: Relativized polynomial time hierarchies having exactly K levels. In: STOC '88: Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing, pp. 245–253 (1988)
7. Ko, K.-I.: Separating and collapsing results on the relativized probabilistic polynomial-time hierarchy. J. ACM **37**(2), 415–438 (1990)
8. Ko, K.-I.: A note on separating the relativized polynomial time hierarchy by immune sets. RAIRO Theor. Inf. Appl. **24**, 229–240 (1990)
9. Ko, K.-I.: Separating the low and high hierarchies by oracles. Inf. Comput. **90**(2), 156–177 (1991)
10. Traub, J.F., Werschulz, A.G.: Complexity and Information. Oxford University Press, Oxford (1998)

Mathematically Rigorous Global Optimization and Fuzzy Optimization



A Brief Comparison of Paradigms, Methods, Similarities, and Differences

Ralph Baker Kearfott

1 Introduction

Mathematical optimization in general is a huge field, with numerous experts working in each of the plethora of application areas. There are various ways for categorizing the applications, methods, and techniques. For our view, we identify the following areas in which computer methods for optimization are applied:

1. Computation of parameters in a scientific model, where the underlying physical laws are assumed to be well-known and highly accurate.
2. Computation of an optimal engineering design, where the underlying physical laws are assumed to be exact, but where there may be uncertainties in measurements of known quantities.
3. Estimation of parameters in statistical models, where a fixed population is known, a subset of that population is well-defined, and we want to estimate the proportion of the whole population in the sub-population.
4. Decision processes in the social and managerial sciences.
5. Learning in expert systems.

We have put these application areas roughly in the order of decreasing "hardness" of the underlying science and formulas. In addition to particular algorithmic techniques that take advantage of problem structure in specific applications in each of these areas, these five areas admit differing underlying guiding paradigms. We give the following three such contrasting philosophies.

For scientific and engineering computing in the hard sciences (fields 1 and 2), a philosophy has been that equations would give the results (outputs) exactly,

R. B. Kearfott (✉)
University of Louisiana at Lafayette, Lafayette, LA, USA
e-mail: rbk@louisiana.edu; rbk@lusfiber.net

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_7

provided that known quantities (inputs) are known exactly. In traditional floating point computation, best guesses for the inputs are made, the floating computations are done, and the result (output) of the floating point computations is taken in lieu of the exact answer. With the advent of modern computers, numerical analysts have recognized that small inaccuracies in the model, errors or uncertainties in the inputs, and errors introduced during the computation may have important effects on the result (output) and thus should be taken into account. Various ways of estimating or bounding the effects of these uncertainties and errors are being investigated, and such handling this kind of uncertainty has been the subject of a number of prestigious conferences. Here, we focus on interval analysis for handling this kind of uncertainty.

Statistical computations (field 3) contrast only slightly with scientific and engineering computations. In statistical computations, there are not deterministic equations that give a unique output (optimum) given unique inputs (parameters in the objective and constraints), but the underlying laws of probability, as well as assumptions about the entire population and the sub-population, can be stated precisely. Thus, we end up with a statistical model whose optimum has certain statistical properties, assuming that the model and population assumptions are correct. In this sense, statistical computations are the same as computations stemming from fields 1 and 2. Uncertainty enters into statistical computations in a similar way.

In recent decades, there has been a push to develop models for some problems in the managerial sciences that are similar to scientific and engineering models or statistical models. Linear models (linear programming) have had phenomenal success in managed environments where inputs and model assumptions are precisely known; such environments include military or government procurement and supply, or optimizing operations in a large corporation.

Equation-based models have also been applied to more fundamentally complicated situations in management and economics, such as stock market prediction and portfolio management, in free-market economies. However, certain simplifying assumptions, such as market efficiency (all participants immediately have full current information) and each player acting in their own best interest (e.g., the assumption that pure altruism and ethics are not factors), have been shown to not be universally valid. Furthermore, nonlinearities due to many players acting according to the same model are not well-understood. Thus, even if current inputs are precisely known and the objective is precisely defined, the "physical law" paradigm for analyzing such situations is debatable.

A different type of managerial or every-day decision process is where an unambiguous quantitative statement of the inputs and goals is not well-defined. An example of such a specification might be design of a customer waiting room at a service center, where the manager does not want an unneeded expense for an excessively large room but also does not want the customers to feel too crowded; that is, the room should not be too small. Here, "too small" is subjective and depends on individual customers, but there are sizes that most people would agree are "too small" for the anticipated number of customers, and the sizes most people would agree are "more than large enough." This is not the same as well-defined inputs but

with uncertainty in their measurements. Also, there is no defined subset of "small rooms" among an existing set of "all rooms" for which a statistical estimate of "smallness" can be made. This is *ambiguity*, rather than *uncertainty* or *randomness*. Nonetheless, people routinely make decisions based on this type of ambiguity. A goal of fuzzy set theory is to model and automate decision making based on this kind of ambiguity.

In what follows, we review interval methods for handling uncertainty, and fuzzy set theory and technology for handling ambiguity. The basic ideas, fundamental equations, scope of applicability, and references for interval analysis appear in Sect. 2, while fuzzy set technology is similarly treated in Sect. 3.

We introduce our notation and basic ideas for branch and bound algorithms in Sect. 4. Ubiquitous in mixed integer programming, branch and bound algorithms are a mainstay in interval arithmetic-based global optimization, are almost unavoidable in general deterministic algorithms for nonconvex problems, and are also used in a significant number of algorithms based on the fuzzy paradigm.

In Sect. 5, we relate some branch and bound techniques to interval computations, while we do the same for fuzzy technology[1] in Sect. 6. There is a huge amount of published and on-going scholarly work in both interval analysis and fuzzy set theory and applications. Here, rather than trying to be comprehensive, we restrict the references to a few well-known introductions and some of the work most familiar to us. We apologize to those we have left out and would like to hear from them. We also realize that we have not comprehensively mentioned all subtleties and terminologies associated with these two areas; please refer to the references we give as starting points.

We contrast the two areas in a concluding section.

2 Interval Analysis: Fundamentals and Philosophy

For interval arithmetic, the underlying assumptions are:

- The inputs are well-defined quantities, but only known to within a certain level of uncertainty, such as when we have measurements with known error bars.
- The output is exactly defined, and
  - either the output can be computed exactly or
  - an exact output can be computed, but with known error bars in the model equations, provided that the arithmetic used in producing the output from the inputs is exact.

[1] Our interval details are more comprehensive, since fuzzy technology is such a large field, and since our primary expertise lies in interval analysis.

– If the computer arithmetic is not exact, the computer can provide mathematically rigorous bounds on the roundoff error resulting from an arithmetic operation.[2]

2.1 Overview

The idea behind interval arithmetic is to encompass both measurement errors in the input and roundoff errors in the computation, to obtain the range of all possible outputs. Specifically, the logical definition of interval arithmetic is, for $\odot \in \{+, -, \times, \div\}$,

$$\boldsymbol{x} \odot \boldsymbol{y} = \{x \odot y \mid x \in \boldsymbol{x} \text{ and } y \in \boldsymbol{y}\}, \tag{1}$$

for intervals $\boldsymbol{x} = [\underline{x}, \overline{x}]$ and $\boldsymbol{y} = [\underline{x}, \overline{x}]$; interval evaluation of a univariate operation or function is defined as

$$\boldsymbol{f}(\boldsymbol{x}) = \{f(x) \mid x \in \boldsymbol{x}\}. \tag{2}$$

Computer implementations are practical because

- it is usually sufficient to relax (1) to

$$\boldsymbol{x} \odot \boldsymbol{y} \supseteq \{x \odot y \mid x \in \boldsymbol{x} \text{ and } y \in \boldsymbol{y}\}, \tag{3}$$

  and to relax (2) to

$$\boldsymbol{f}(\boldsymbol{x}) \supseteq \{f(x) \mid x \in \boldsymbol{x}\}, \tag{4}$$

  Provided that the containments in (3) and (4) are not too loose.
- The definition (1) corresponds to the operational definitions

$$\left.\begin{aligned}
\boldsymbol{x} + \boldsymbol{y} &= [\underline{x} + \underline{y}, \overline{x} + \overline{y}], \\
\boldsymbol{x} - \boldsymbol{y} &= [\underline{x} - \overline{y}, \overline{x} - \underline{y}], \\
\boldsymbol{x} \times \boldsymbol{y} &= [\min\{\underline{x}\underline{y}, \underline{x}\overline{y}, \overline{x}\underline{y}, \overline{x}\overline{y}\}, \max\{\underline{x}\underline{y}, \underline{x}\overline{y}, \overline{x}\underline{y}, \overline{x}\overline{y}\}] \\
\frac{1}{\boldsymbol{x}} &= [\frac{1}{\overline{x}}, \frac{1}{\underline{x}}] \qquad \text{if } \underline{x} > 0 \text{ or } \overline{x} < 0 \\
\boldsymbol{x} \div \boldsymbol{y} &= \boldsymbol{x} \times \frac{1}{\boldsymbol{y}}
\end{aligned}\right\} \tag{5}$$

[2] Typically, through control of the rounding mode, as specified in the IEEE 754 standard for floating point arithmetic [19].

- On a computer with control of rounding modes, such as those adhering to the IEEE 754 Standard for Floating Point Arithmetic,[3] the operational definitions (5) may be relaxed in a mathematically rigorous way, to (3), by rounding the lower end point of the result down and the upper end point of the result up. Similarly, using such directed rounding, known error bounds, and common techniques for evaluating standard functions, (4) may be implemented in a mathematically rigorous way with floating point arithmetic.

These basic ideas make interval arithmetic highly attractive. However, there are subtleties that can cause problems for the naive, such as interval dependency [63], the wrapping effect when solving differential equations,[4] and the clustering effect in global optimization[5] (see [12, 13, 29] and subsequent work of other authors).

Furthermore, the above definitions leave some ambiguity concerning operational definition and semantic interpretation in cases such as $[10, 100]/[-1, 0]$, $[10, 100]/[-1, 2]$, or $[-1, 1]/[-1, 1]$. Various alternatives have been proposed and extensively discussed; consensus has resulted in IEEE 1788–2015, the IEEE standard for interval arithmetic [18].

2.2 Interval Logic

Logical computations figure prominently in implementations of branch and bound algorithms, and traditional, interval, and fuzzy logic differ significantly.

Logic is often based on sets. In such a traditional logic, we define "true" (**T**), "false" (**F**), the binary logical operators "and" ($\wedge$) and "or" ($\vee$) , and the unary logical operator "not" ($\neg$):

Definition 1 (often-taught traditional logic based on sets) Let r and s be objects, and let $\mathcal{T}$ be the set of true objects for which a property is true. Furthermore, denote the truth value of an object r by $x = \mathfrak{T}(r)$.

- Truth values: $\mathfrak{T}(r) = \mathbf{T}$ if $r \in \mathcal{T}$ and $\mathfrak{T}(r) = \mathbf{F}$ if $r \notin \mathcal{T}$.
- The "and" operator: $\mathfrak{T}(r \wedge s) = \mathbf{T}$ if $r \in \mathcal{T}$ and $s \in \mathcal{T}$, and $\mathfrak{T}(r \wedge s) = \mathbf{F}$ otherwise.
- The "or" operator: $\mathfrak{T}(r \vee s) = \mathbf{T}$ if $r \in \mathcal{T}$ or $s \in \mathcal{T}$ and is $\mathfrak{T}(r \vee s) = \mathbf{F}$ otherwise.
- "not" operator: $\mathfrak{T}(\neg r) = \mathbf{T}$ if and only if $r \notin \mathcal{T}$.

[3] Most desktops and laptops with appropriate programming language support, and some supercomputers, adhere to this standard.

[4] First observed in [39, pp. 90–93] and treated by many since then.

[5] The clustering effect is actually in common to most basic branch and bound algorithms, whether or not interval technology is used. However, its cause is closely related to the interval dependency problem.

Table 1 Truth tables for standard logic

$\wedge$		
r/s	**T**	**F**
T	**T**	**F**
F	**F**	**F**

$\vee$		
r/s	**T**	**F**
T	**T**	**T**
F	**T**	**F**

$\neg$	
r	$\neg(r)$
T	**F**
F	**T**

Once logical values $\mathtt{r}$ have been assigned to objects r, boolean expressions, analogous to arithmetic expressions, may be formed from the logical values $\mathtt{r}$, with operators $\wedge$ and $\vee$ corresponding to multiplication and addition, and operator $\neg$ corresponding to negation; this leads to well-known *truth tables* defining these operations. We see truth tables for traditional logic in Table 1; such a truth table would be useful, for example, in searching through individual constraints to determine feasibility of an entire system of constraints. In the context of numerical computation, and branch and bound algorithms for optimization in particular, we may without loss of generality think of the logical statements r and s as numerical values and the set $\mathcal{T}$ as $[0, \infty)$, so we have

Definition 2 (Traditional Logic in a Simplified Numerical Context)

- Truth values: $\mathfrak{T}(r) = \mathbf{T}$ if $r \geq 0$. and $\mathfrak{T}(r) = \mathbf{F}$ if $r < 0$.
- The "and" operator: $\mathfrak{T}(r \wedge s) = \mathbf{T}$ if $r \geq 0$ and $s \geq 0$, $\mathfrak{T}(r \wedge s) = \mathbf{F}$ otherwise.
- The "or" operator: $\mathfrak{T}(r \vee s) = \mathbf{T}$ if $r \geq 0$ or $\boldsymbol{s} \geq 0$, and $\mathfrak{T}(r \vee s) = \mathbf{F}$ otherwise.
- The "not" operator: $\mathfrak{T}(\neg r) = \mathbf{T}$ if $r < 0$, and $\mathfrak{T}(\neg r) = \mathbf{F}$ otherwise.

In the context of Definition 2, the values r and s are known only to lie within sets that are intervals, that is, $r \in \boldsymbol{r} = [\underline{r}, \overline{r}]$ and $s \in \boldsymbol{s} = [\underline{s}, \overline{s}]$. In this context, what does it mean for $\boldsymbol{r}$ to be true, that is, for $\boldsymbol{r} \geq 0$? Addressing this question reveals various subtleties associated with defining logic based on intervals, when the underlying philosophy is making mathematically rigorous statements. These subtleties are codified, for example, in [18, Table 10.7]. In short, the following three cases can occur:

1. $\overline{r} < 0$, and it is certain that r cannot be nonnegative;
2. $\underline{r} \geq 0$, and it is certain that r is nonnegative;
3. $\underline{r} < 0$, and $\overline{r} \geq 0$, and r may or may not be nonnegative.

This leads to the following three-valued logic with values **T**, **F**, and **U** ("unknown").

Definition 3 (interval set-based logic in simplified form)

- Truth values:
  - $\mathfrak{T}(\boldsymbol{r}) = \mathbf{T}$ if $\overline{r} \geq 0$.
  - $\mathfrak{T}(\boldsymbol{r}) = \mathbf{F}$ if $\underline{r} < 0$.
  - $\mathfrak{T}(\boldsymbol{r}) = \mathbf{U}$ otherwise.

Table 2 Truth tables for interval logic

$\wedge$			
r/s	**T**	**F**	**U**
T	**T**	**F**	**U**
F	**F**	**F**	**F**
U	**U**	**F**	**U**

$\vee$			
r/s	**T**	**F**	**U**
T	**T**	**T**	**T**
F	**T**	**F**	**U**
U	**T**	**U**	**U**

$\neg$	
r	$\neg(r)$
T	**U**[a]
F	**T**
U	**U**

[a] This can be made more precise if it is known whether or not r corresponds to $\boldsymbol{r}$ with $0 \notin \boldsymbol{r}$

- The "and" operator: $\mathfrak{T}(\boldsymbol{r} \wedge \boldsymbol{s}) = \mathfrak{T}(\boldsymbol{r} \cap \boldsymbol{s})$.
- The "or" operator: Let $\boldsymbol{t} = \boldsymbol{r} \cup \boldsymbol{s}$. $\mathfrak{T}(\boldsymbol{r} \vee \boldsymbol{s}) = \mathfrak{T}(\boldsymbol{r} \cup \boldsymbol{s})$.
- The "not" operator:
  - $\mathfrak{T}(\neg \boldsymbol{r}) = \mathbf{T}$ if $\underline{r} < 0$.
  - $\mathfrak{T}(\neg \boldsymbol{r}) = \mathbf{F}$ if $\overline{r} > 0$.
  - $\mathfrak{T}(\neg \boldsymbol{r}) = \mathbf{U}$ if $0 \in \boldsymbol{r}$.

Notably, interval logical expressions differ from traditional logical expressions in the sense that $\mathfrak{T}(\neg \boldsymbol{r}) \neq (\neg \mathfrak{T}(\boldsymbol{r}))$. This is important in implementations of branch and bound algorithms where the reported results are meant to be mathematically rigorous. Truth tables for our simplified interval logic appear in Table 2.

2.3 Extensions

There is significant literature and practice on improving interval arithmetic with similar alternatives and extensions of it. Among these are:

- affine arithmetic [50, 54] to better bound expressions with almost-linear subexpressions;
- Taylor arithmetic to produce notably tighter bounds in various contexts; see, for example, [9, 11, 35] and especially [8] if available;
- random interval arithmetic, which discards mathematical rigor but has advantages if certainty is not needed [60];
- algebraic completion of the set of intervals[6] [6];
- others, with theoretical, practical, and algorithmic goals.

[6] But with complicated interpretation of results in general settings.

2.4 History and References

The ideas underlying interval arithmetic are sufficiently basic that they have been independently discovered, in various contexts, various times, as evidenced in [14, 55, 61, 64], etc.; based on an entry in an 1896 periodical for beginning high school pedagogy, Siegfried Rump even claims that interval arithmetic was common knowledge in Germany before the turn of the twentieth century.

The most complete beginning of interval arithmetic in the twentieth century is generally recognized to be Ramon Moore's dissertation [39], and subsequent monograph [40]. Outlines of most current applications to interval arithmetic occur in Moore's dissertation. Moore published a 1979 revision [41]. For an elementary introduction to interval arithmetic, we recommend our 2009 update [42]. Another common reference text is Alefeld and Herzberger's 1972 book [3] and 1983 translation into English [4].

3 Fuzzy Sets: Fundamentals and Philosophy

Amid a huge amount of literature, software manuals, technical descriptions, and mathematical analysis of algorithms and variants, the foundation and basic philosophy of fuzzy technology continues to be well-described by its inventor Lofteh Zadeh's seminal article [65], while we also rely on [33] for additional insight. We suggest the reader new to the subject turn to these for additional reading concerning what follows here.

In contrast to interval analysis, meant to make traditional scientific measurement and computation mathematically rigorous, fuzzy set theory and associated areas are meant to allow computers to process ambiguous natural language and to make decisions in the way humans do in the absence of scientific measurements or models. Fuzzy set theory, fuzzy logic, and applications are characterized less by underlying assumptions than lack thereof.[7]

The following example is amenable to processing with fuzzy technology.

Example 1 Take a college departmental tenure committee's use of student evaluation of instruction data. The committee wants to grant tenure only to professors with "good" (or sufficiently popular) ratings. However, it is *subjective* what sufficiently good means. If the students rate the professor on a scale of 1 to 5, some committee members might think a rating of 3 or greater is sufficiently good, while others might think the rating is sufficiently good only if it is greater than or equal to 4. If $\mathcal{S}$ represents the set of sufficiently good ratings, a *membership function* is used to describe $\mathcal{S}$ as a fuzzy set.

[7] This is not to say that mathematical implications of particular procedures are not highly analyzed.

Fuzzy sets are defined via membership functions. In some ways, membership functions are like characteristic functions, and in some ways, they are like statistical distributions, with notable differences.

Definition 4 (Modified from [65]) Let $\mathbb{X}$ be a domain space (a set of objects, words, numbers, etc.; discrete or continuous), and let $x \in \mathbb{X}$. The *degree of membership* of $x \in \mathbb{X}$ in a set $\mathcal{A} \in \mathbb{X}$ is defined by a *fuzzy membership function*

$$\mu_A : \mathbb{X} \to [0, 1].$$

The general membership function has few mathematical restrictions: Different authors have placed various additional restrictions on membership functions, to facilitate analysis in various applications.

Definition 5 If $\mathcal{A} \in \mathbb{X}$ is as in Definition 4, the pair $(\mathcal{A}, \mu_A)$ is termed a *fuzzy set* $\mathfrak{A}$.

In problem formulations and computations, particular quantities are assumed to be fuzzy or not.

Definition 6 A quantity x is called *fuzzy value* $\tilde{x}$ if it is assumed to belong to some fuzzy set $\mathfrak{X}$ with degree of membership $\mu_X(x)$. Well-defined quantities, that is, those that are not fuzzy, are called *crisp* quantities.

The interval arithmetic analog of a fuzzy set is a nondegenerate interval, [8] whereas a crisp value corresponds to a point, i.e., a degenerate interval.

Any membership function $\mu(t)$ for Example 1 would reasonably be like a cumulative statistical distribution with support $[1, 5]$, since it would seem reasonable for it to increase from 0 to 1 as t increases from 1 to 5. However, while statistical distributions are *derived* based on assumptions and principles, fuzzy membership functions are typically *designed* according to a subjective view of the application, ease of computation, and experience with what gives acceptable results.[9]

Membership functions can also be similar to probability density functions:

Example 2 (A Classic Example) In Example 1, suppose the department chair would like to commend faculty whose evaluations are above average and to take other action whose evaluations are below average. Some might think a rating of 2 would qualify as average, and some might think a rating of 4 would qualify as average, but all might agree a rating of 3 would be average.

In Example 2, a well-designed membership function $\mu(t)$ might have $\mu(3) = 1$, $\mu(1) = 0$, and $\mu(5) = 0$ and be increasing on $[1, 3]$ and decreasing on $[3, 5]$.

[8] An interval $\mathbb{X}$ corresponds to a fuzzy set $\mathfrak{A}$ as in Definition 5 with $\mathcal{A} = \mathbb{X}$ and $\mu_A(x) = 1$ if $x \in \mathbb{X}$, $\mu_A(x) = 0$ otherwise.

[9] However, data, and even statistics, can sometimes be used in the design of membership functions.

However, unlike a probability density function, in general it would not be necessary to have $\int_1^5 \mu(t)dt \neq 1$.

Example 3 Suppose we wish to describe a quantity that we know varies within the interval [1, 2], and we know that every value in the interval [1, 2] is taken on by that quantity, but we know nothing else about the distribution of values of the quantity within [1, 2]. An appropriate membership function for the set of values of the quantity takes on might then be $\mu(t) = \chi_{[1,2]}(t)$, where $\chi_{1.2]}$ is the characteristic function of [1, 2].

Example 3 highlights one relationship between interval computation and fuzzy sets.

To do computations with fuzzy sets and to make decisions, degrees of truth are defined, and the user of fuzzy set technology determines an acceptable degree of truth, defined through *alpha cuts*:

Definition 7 The set

$$\mathcal{S}_\alpha = \{x \mid \mu(x) \geq \alpha\} \quad \text{for some } \alpha \in [0, 1] \tag{6}$$

is called an *alpha-cut* for the fuzzy set $\mathfrak{S}$. Here, we will call a particular value α_0 the *degree of truth*[10] of the elements of $\mathcal{S}_{\alpha_0}$.

The degree of truth is like a probability, but, in some contexts, we call degrees of truth *possibilities*.

For $\mathbb{X}$ an interval subset of $\mathbb{R}$ and under reasonable assumptions on $\mu(t)$, such as μ be unimodal or monotonically increasing, $\mathcal{S}_\alpha$ is an interval for each α; in Example 1, $\mathcal{S}_\alpha$ is an interval subset of [1, 5] for each $\alpha \in [0, 1]$.

Decisions based on fuzzy computations are made according to what degree of truth α_0 is acceptable.

Example 4 Suppose, in Example 2, the possibilities are modeled by the membership function $\mu(t) = -\frac{1}{4}(t-1)(t-5)$, and the department head decides to accept an evaluation as average if it has a possibility $\alpha \geq 0.75$. Then $\mathcal{S}_{0.75} = [2, 4]$; see Fig. 1.

An important aspect of computations involving fuzzy sets involves functions of variables from a fuzzy set.

Theorem 1 *(well-known; see for example [33, Section 3.4] for an explanation) Suppose the interval $\mathcal{X}_\alpha$ is an α-cut for a real quantity x, and suppose a dependency between a quantity y and the quantity $x \in \mathcal{X}$ can be expressed with a computable expression f, i.e., $y = f(x)$, where $\mathcal{X}$ corresponds to a fuzzy set $\mathfrak{X}$. Then, the range $\boldsymbol{f}(\mathcal{X}_\alpha)$ of f over $\mathcal{X}_\alpha$ is the corresponding α-cut for the value $f(x)$.*

[10]The terms "degree of truth" and "degree of belief" are common in the literature concerning handling uncertain knowledge. Also, "belief functions," like membership functions, are defined. See [53], for example. We do not guarantee that our definition of "degree of truth" is the same as that of all others.

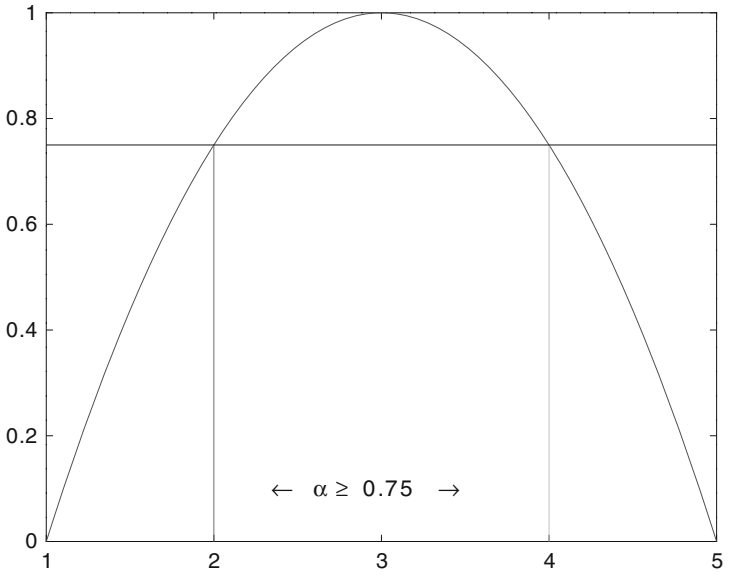


Fig. 1 The α-cut $\alpha \geq 0.75$ for Example 4

Thus, interval arithmetic can be used for bounding α-cuts of functions of variables, and the corresponding membership function for a function f of a fuzzy variable can be approximated by subdividing [0, 1] and using interval arithmetic to evaluate f over each sub-interval. See, for example, [33].

Intersection and union of fuzzy sets can be defined in terms of the membership functions, as introduced in Zadeh's seminal work [65].

Definition 8 (a classical alternative for union and intersection of fuzzy sets) If $\mathcal{S}$ is a fuzzy set with membership function μ_S and $\mathcal{T}$ is a fuzzy set with membership function μ_T, the membership function for the intersection $\mathcal{S} \cap \mathcal{T}$ may be defined as

$$T(\mathcal{S}, \mathcal{T})(x) = \mu_{S \cap T}(x) = \inf \{\mu_S(x), \mu_T(x)\}, \tag{7}$$

and the membership function for the union may be defined as

$$S(\mathcal{S}, \mathcal{T})(x) = \mu_{S \cup T}(x) = \sup \{\mu_S(x), \mu_T(x)\}. \tag{8}$$

Other, somewhat similar definitions of intersection $T(\mathcal{S}, \mathcal{T})(x)$ and union $S(\mathcal{S}, \mathcal{T})(x)$ of fuzzy sets are also defined and used; the operators $T(\mathcal{S}, \mathcal{T})$ and $S(\mathcal{S}, \mathcal{T})$, obeying certain common properties, are called *T-norms* and *S-norms*. Relations between different T- and S-norms have been analyzed. Which particular S- or T-norm is used in an application can depend on subjective or design considerations.

3.1 Fuzzy Logic

Fuzzy logic is based on fuzzy union and fuzzy intersection. Here, a common definition of the membership function for "not-$\mathcal{S}$" is

$$\mu_{\neg S}(x) = 1 - \mu_S(x). \tag{9}$$

If (8), (7), and (9) are used to define truth values in fuzzy logic, then, for each degree of truth α_0 other than $\alpha_0 = 0.5$, the truth tables for the fuzzy logic are the same as the truth tables for traditional logic (Table 1), whereas, for $\alpha_0 = 0.5$, $\neg s$ is **T** when s is **T**.

3.2 A Brief History

Like interval arithmetic, basic concepts in fuzzy set theory have deep roots, going back to Bernoulli and Lambert (see [53, Section 2]). Also, like Ray Moore in his dissertation and subsequent book, Lofti Zadeh clearly defines the philosophy, principles, and underlying mathematics of fuzzy sets in [65]. Since Zadeh's seminal paper, fuzzy information technology has become ubiquitous throughout computer science and technology. There are numerous conferences held worldwide each year on the subject, there are various software packages, there are numerous devices controlled with fuzzy technology, etc. Examples of recent conference proceedings are [7, 27], and [59].

4 The Branch and Bound Framework: Some Definitions and Details

Here, we focus on the general nonconvex optimization problem, which we pose with this notation:

$$\begin{aligned} &\text{minimize } \varphi(x) \\ &\text{subject to } c_i(x) = 0, \;\; i = 1, \ldots, m_1, \\ &\qquad\qquad g_j(x) \leq 0, \;\; j = 1, \ldots, m_2, \\ &\text{where } \varphi : \mathcal{X} \subseteq \mathbb{R}^n \to \mathbb{R} \\ &\text{and each } c_i, g_j : \mathcal{X} \to \mathbb{R}, \text{ with } x = (x_1, \ldots, x_n) \in \mathcal{X}. \end{aligned} \tag{10}$$

The subset of $\mathbb{R}^n$ satisfying the constraints is called the *feasible set*. Here, some of the constraints $g_i \leq 0$ may be of the simple form $x_i \leq a_i$ or $\boldsymbol{x}_i \geq b_i$; these *bound constraints* are often handled separately with efficient techniques.

An important general concept in branch and bound algorithms for global optimization is that of *relaxations*:

Definition 9 A *relaxation* of the global optimization problem (10) is a related problem, where φ is replaced by some related function $\breve{\varphi}$, each set of constraints is replaced by a related set of constraints, such that, if φ^* is the global optimum to the original problem (10) and $\varphi^\dagger$ is the global optimum to the relaxed problem, then $\varphi^\dagger \leq \varphi^*$.

We will say more about relaxations in Sect. 5.

In branch and bound (B&B) methods, an initial domain is adaptively subdivided, and each sub-domain is analyzed. The general structure is outlined in Algorithm 1.

Data: An initial region $\mathcal{D}^{(0)}$, the objective φ,
the constraints C, a domain stopping tolerance ε_d, and a limit M on the maximum number of regions to be allowed to be processed.
Result: Set the Boolean variable `OK`:
• Set `OK=true`, set the best upper bound $\overline{\varphi}$ for the global optimum, set the list $\mathcal{C}$ within which all optimizing points must lie, if the algorithm completed with less than M regions considered.
• Set `OK=false` if the algorithm could not complete.

1 Initialize the list $\mathcal{L}$ of regions to be processed to contain $\mathcal{D}^{(0)}$;
2 Determine an upper bound $\overline{\varphi}$ on the global optimum;
3 $i \leftarrow 1$;
4 **while** $\mathcal{L} \neq \emptyset$ **do**
5 $i \leftarrow i + 1$;
6 **if** $i > M$ **then return** *OK=false*;
7 Remove a region $\mathcal{D}$ from $\mathcal{L}$;
8 **Bound:** Determine if $\mathcal{D}$ is not infeasible, and if it is not proven to be infeasible, determine a lower bound $\underline{\varphi}$ on φ over the feasible part of $\mathcal{D}$;
9 **if** $\mathcal{D}$ *is infeasible or* $\underline{\varphi} > \overline{\varphi}$ **then** return to Step 7;
10 **Reduce:** (Possibly) eliminate portions of $\mathcal{D}$ through various efficient techniques, without subdividing.;
11 **Improve upper bound:** Possibly compute a better upper bound $\overline{\varphi}$;
12 **if** *a scaled diameter diam of* $\mathcal{D}$ *satisfies* $diam(\mathcal{D}) < \varepsilon_d$ **then**
13 Store $\mathcal{D}$ in $\mathcal{C}$;
14 Return to Step 7;
15 **else**
16 **Branch:** Split $\mathcal{D}$ into two or more sub-regions whose union is $\mathcal{D}$;
17 Put each of the sub-regions into $\mathcal{L}$;
18 Return to Step 7;
19 **end**
20 **end**
21 **return** *OK=true,* $\overline{\varphi}$*, and* $\mathcal{C}$ *(possibly empty)*;

Algorithm 1: General branch and bound structure

B&B algorithms represent a deterministic framework, that is, a framework within which, should the algorithm complete and use exact arithmetic, the actual global

optimum (and, depending on the algorithm, possibly one or possibly all globally optimizing points) will be found. Neumaier calls such algorithms *complete*; see [44] for a perspective on this.

A common choice for the regions $\mathcal{D}$ in Algorithm 1 is rectangular paralellepipeds

$$\boldsymbol{x} = (\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n) = ([\underline{x}_1, \overline{x}_1], [\underline{x}_2, \overline{x}_2], \ldots, [\underline{x}_n, \overline{x}_n]), \tag{11}$$

that is, the set

$$\left\{ x = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n \;\mid\; x_i \in [\underline{x}_i, \overline{x}_i] \text{ for } 1 \leq i \leq n \right\},$$

otherwise known as a *box* or *interval vector*. Subdivision of boxes is easily implemented, typically, but not always, by bisection in a scaled version of the widest coordinate:

$$\boldsymbol{x} \longrightarrow \{\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}\}, \tag{12}$$

where

$$\boldsymbol{x}^{(1)} = \left(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{i-1}, [(\underline{x}_i + \overline{x}_i)/2, \overline{x}_i], \boldsymbol{x}_{i+1}, \ldots \boldsymbol{x}_n\right)$$

and

$$\boldsymbol{x}^{(2)} = \left(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{i-1}, [\underline{x}_i, (\underline{x}_i + \overline{x}_i)/2], \boldsymbol{x}_{i+1}, \ldots \boldsymbol{x}_n\right)$$

for the selected coordinate direction i. Besides allowing simple management of the subdivision process, using boxes $\boldsymbol{x}$ for the regions $\mathcal{D}$ can provide an alternative way of handling bound constraints $\underline{x} \leq x_i \leq \overline{x}$, provided that sufficient care is taken in algorithm design, implementation, and interpretation of results.

Besides boxes for the regions $\mathcal{D}$, *simplexes* have appeared in some of the literature and implementations. An n-simplex is defined geometrically as the convex hull of $n+1$ points $x^{(i)}$, $0 \leq i \leq n$ in $\mathbb{R}^m$, for $m \geq n$; we define the canonical simplex in $\mathbb{R}^n$ as having $x^{(0)} = (0, \ldots, 0) \in \mathbb{R}^n$ and $x^{(i)}$ the i-th coordinate vector e_i for $1 \leq i \leq n$. There are various disadvantages to using simplexes in branch and bound methods, such as lack of simple correspondence to bounds on the variables, complicated geometries resulting from subdivision processes, and the fact that the volume of the canonical n-simplex is only $\frac{1}{n!}$. However, some have reported success with simplexes in B&B algorithms. For instance, Paulavičius and Žilinskas [45, 46, 66] report successes on general problems; however, their methods use statistical estimates of Lipschitz constants and so are not complete in the sense of Neumaier [44].

Nonetheless, use of simplexes can be advantageous in complete algorithms for several classes of problems. This is because of two alternative characterizations of a simplex in terms of sets of constraints. The set of $n+1$ inequality constraints

$$\sum_{i=1}^{n} x_i \leq 1, \quad x_i \geq 0 \text{ for } 1 \leq i \leq n \tag{13}$$

defines the canonical n-simplex. Alternatively, the set of constraints

$$\sum_{i=0}^{n} x_i = 1, \quad x_i \geq 0 \text{ for } 0 \leq i \leq n \tag{14}$$

defines an n simplex in $\mathbb{R}^{n+1}$. Serendipitously, a number of practical problems have such sets of constraints, so their feasible set is a subset of a simplex. An initial investigation with this in mind is [21]. An investigation into B&B algorithms based on simplexes, as well as classes of constraint sets easily converted to simplicial constraint sets (13) or (14), is currently underway [34].

We will say more about bounding techniques (Step 8 of Algorithm 1) and reducing techniques (Step 10 of Algorithm 1) in Sect. 5.

5 Interval Technology: Some Details

Interval-based methods for global optimization are generally strongly deterministic in the sense that they provide mathematically rigorous bounds on the global optimum, global optimizers, or both. They are largely based on branch and bound methods, in which an initial domain is recursively subdivided (branching) to obtain better and better bounds on the range of the objective function. In addition to using interval arithmetic to bound the range of the objective and constraints (if present) over each sub-region, there are various interval-based acceleration processes, such as constraint propagation and interval Newton methods. Books on interval-based global optimization include Hansen and Walster's 1983 book [17] as well as their 2003 update [16], Ratschek and Rokne [47], our monograph [20, 24] (with a special emphasis on applications and constraint propagation), and others.

Implementations of interval-based branch and bound algorithms vary in efficiency, depending on the application and how the problem is formulated. For example, only rigorous and tight bounds on the objective may be desired, bounds on a global optimizer may be desired, or bounds on all global optimizers may be desired, it may be required to prove that there are points satisfying all equality constraints, or relaxations of the equality constraints may be permissible, etc.

Interval-based optimization and system solving is having an increasing number of successes in applications but is not a panacea. This is largely due to interval

dependency in the evaluation of expressions[11] and the wrapping effect[12] when integrating systems of differential equations. These problems can often be avoided with astute utilization of problem-specific features.

Here, we outline a few common interval analysis tools for B&B algorithms.

5.1 Interval Newton Methods

Interval Newton methods and the related Krawczyk method are a class of methods that are derived from the traditional Newton–Raphson method in several ways.

Definition 10 Suppose $F : D \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^n$, suppose $\boldsymbol{x} \in D$ is an interval n-vector, and suppose that $\boldsymbol{F}'(\boldsymbol{x})$ is an interval extension of the Jacobian matrix[13] of F over $\boldsymbol{x}$ (obtained, for example, by evaluating each component with interval arithmetic), and $\check{x} \in \boldsymbol{x}$. Then a *multivariate interval Newton operator* F is any mapping $\boldsymbol{N}(F, \boldsymbol{x}, \check{x})$ from the set of ordered pairs $(\boldsymbol{x}, \check{x})$ of interval n-vectors $\boldsymbol{x}$ and point n-vectors $\check{x}$ to the set of interval n-vectors, such that

$$\tilde{\boldsymbol{x}} \leftarrow \boldsymbol{N}(F, \boldsymbol{x}, \check{x}) = \check{x} + \boldsymbol{v}, \tag{15}$$

where $\boldsymbol{v} \in \mathbb{IR}^n$ is any box that bounds the solution set to the linear interval system

$$\boldsymbol{F}'(\boldsymbol{x})v = -F(\check{x}). \tag{16}$$

Interval Newton methods can verify nonexistence of optimizers in regions $\mathcal{D}$ in Algorithm 1, allowing quick rejection of such regions without extensive subdivision. In contrast to other techniques, such as examination of the feasibility of each constraint individually, interval Newton methods implicitly take advantage of the coupling between constraints and the objective. Furthermore, interval Newton methods can reduce the size of a region $\mathcal{D}$ in an often quadratically convergent iteration process, much more efficiently than if $\mathcal{D}$ would need to be subdivided. They can also sometimes prove existence of critical points of the optimization problem within a specific region $\mathcal{D}$.

This theorem summarizes some of the relevant properties

Theorem 2 *Suppose $F : D \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^n$ represents a system of nonlinear equations $F(x) = 0$, suppose F has continuous first-order partial derivatives, and suppose $\boldsymbol{N}(F, \boldsymbol{x}, \check{x})$ is the image under an interval Newton method of the box $\boldsymbol{x}$.*

[11] The results of (1) and (2) are sharp to within measurement and rounding errors but are sometimes increasingly un-sharp when operations are combined.

[12] See [42], etc.

[13] For details and generalizations, see a text on interval analysis, such as [4, 43], or our work [24].

Then any solutions $x^* \in \boldsymbol{x}$ *of* $F(x) = 0$ *must also lie in* $\boldsymbol{N}(F, \boldsymbol{x}, \check{x})$. *In particular, if* $\boldsymbol{N}(F, \boldsymbol{x}, \check{x}) \cap \boldsymbol{x} = \emptyset$, *there can be no solutions of* F *in* $\boldsymbol{x}$.

Typically.[14] if $\boldsymbol{N}(F, \boldsymbol{x}, \check{x}) \subseteq \boldsymbol{x}$, then iteration of (15) will result in the widths of the components of $\boldsymbol{x}$ converging quadratically to narrow intervals that contain a solution to $F = 0$. Some analysis of this may be found in our work [1, Section 8.4], while more analyses can be found in numerous other works, including [43].

Disadvantages of interval Newton methods include their limited scope of applicability and the large amount of linear algebra computations. In particular, the interval Newton equation (15) can be iterated to progressively narrow the bounds on a system of nonlinear equations. Interval Newton methods may not work well if the system of equations is ill-conditioned at a solution, although they do have some applicability in such cases and also when the region $\mathcal{D}$ is larger than a typical region of attraction of Newton's method (for example, see [22] and [31]). In branch and bound algorithms for global optimization, the system of equations can be either a system of equality constraints or the Kuhn–Tucker or Fritz John equations, and those can be ill-conditioned or singular at solutions, for some fairly common situations, as for example, demonstrated in [32].

Regarding linear algebra computations, interval Newton methods are generally effective only if a preconditioner matrix Y is computed, so (15) becomes

$$Y\boldsymbol{F}'(\boldsymbol{x})v = -YF(\check{x}), \tag{17}$$

and the solution set of the preconditioned system is bounded. A common preconditioner, used in various interval Newton formulations, is for Y to be the inverse of the matrix of midpoints of the interval Jacobian matrix $\boldsymbol{F}'(\boldsymbol{x})$, while other preconditioners involve solving a linear programming problem to obtain each row of the point matrix Y (see [22, 31]). Either method involves more computations than the classical floating point Newton method, and the advantages of using an interval Newton method should be weighed against the smaller cost of simpler and less costly techniques within the steps of a B&B algorithm.

5.2 Constraint Propagation

Constraint propagation, an entire sub-field of computer science, is strongly tied to interval arithmetic when continuous variables appear in the problem formulation. Within the context of general nonconvex global optimization algorithms, it seldom leads to a precise globally optimal solution, but it can be a relatively inexpensive way of reducing the size of regions $\mathcal{D}$ in Step 10 (the "reduce" step) of Algorithm 1.

[14] Under mild conditions on the interval extension of F' and for most ways of computing $\boldsymbol{v}$.

At its simplest, constraint propagation involves parsing the expressions defining the objective and constraints into a list of elementary or canonical operations, evaluating the resulting intermediate expressions, then inverting the canonical operations to obtain sharper values on the variables. We give the following very simple example for the benefit of readers new to the subject.

Example 5

$$\begin{array}{l} \text{minimize } \varphi(x) = x_1 x_2 \\ \text{subject to } g(x) = 1 - x_1 x_2 \leq 0 \text{ and } x_i \in [0,1],\ i = 1,2. \end{array} \tag{18}$$

The variables x_1 and x_2, as well as the intermediate results of the computation and the function and constraint values, can be represented with the following list[15]:

$$\begin{array}{rcl} v_1 & = & x_1, \\ v_2 & = & x_2; \\ v_3 & \leftarrow & v_1 v_2, \\ v_4 & \leftarrow & 1 - v_3; \\ \varphi & = & v_3, \\ g & = & v_4 \leq 0. \end{array} \tag{19}$$

We initialize v_1 to $[0,1]$ and v_2 to $[0,1]$, then do a "forward evaluation" of the list (19), obtaining $v_3 \leftarrow [0,1]$ and $v_4 \leftarrow [0,1]$. The condition $v_4 \leq 0$ gives $v_4 \in (-\infty, 0] \cap [0,1] = [0,0]$. We then solve $v_4 = 1 - v_3$ for v_3, giving $v_3 \in [1,1]$. Using this value for v_3 and solving for v_1 in $v_3 = v_1 v_2$ then give

$$v_1 \leftarrow ([1,1]/[0,1]) \cap [0,1] = [1,\infty) \cap [0,1] = [1,1].$$

We similarly get $v_2 \leftarrow [1,1]$. In this case, a few elementary operations of constraint propagation give $(x_1, x_2) = (1,1)$ as the only solution.

In addition to using constraints, as in Example 5, upper bounds on the objective from Step 11 of Algorithm 1 can also be used with the back propagation we have just illustrated, to narrow the bounds on possible minimizing solutions.

A notable historical example of interval constraint propagation in general global optimization software is in work of Pascal van Hentenryck [57, 58]. Van Hentenryck's work is one of many examples, such as [38] or even our own early work [23]. Such interval constraint propagation is found in much current specialized and general global optimization software; a well-known current commercial such package is BARON [51, 52].

[15] Various researchers refer to similar lists as *code lists* (Rall), *directed acyclic graphs* (DAGs, Neumaier), etc. Although, for a given problem, such lists are not unique, they may be generated automatically with computer language compilers or by other means.

The book chapter [10] contains a somewhat recent review of constraint propagation processes. The COPROD series of workshops [62] is held every year, in conjunction either with one of the meetings on interval analysis or on fuzzy technology.

5.3 Relaxations

If $\mathcal{D}$ is a box, an interval evaluation of φ over $\mathcal{D}$ provides an easy lower bound on φ in the bounding step 8 of a branch and bound process (Algorithm 1). However, such an evaluation can be crude, since it does not take account of the constraints at all. For example, many problems, such as minimax and ℓ_1 data fitting, are formulated so the objective consists of a single slack variable, and all of the problem information is in the constraints. Interval Newton methods may not be useful in such situations (see [32], for instance). Furthermore, constraint propagation as explained in Sect. 5.2 does not take full advantage of coupling between the constraints and so may not be so useful either for certain problems. This is where relaxations step in.

Relaxations as in Definition 9 do not in general need interval arithmetic, but interval computations are frequently either central to the relaxation or are a part of the relaxation. In particular, we generally replace Problem 10, augmented with constraints describing the current region $\mathcal{D}$ under consideration, by a relaxation that is easier to solve than Problem 10; the optimum of the relaxation then provides a lower bound[16] on φ over $\mathcal{D}$.

Generally, relaxations are formed by replacing the objective φ by an objective $\tilde{\varphi}$ such that $\tilde{\varphi}(x) \leq \varphi(x)$ for all $x \in \mathcal{D}$, and replacing the feasible set $\mathcal{F}$ (defined as the portion of $\mathcal{D}$ satisfying the constraints) by a set $\tilde{\mathcal{F}}$ such that $\mathcal{F} \subseteq \tilde{\mathcal{F}}$. $\tilde{\mathcal{F}}$ is generally defined by modifying the inequality constraints. For example, $g(x) \leq 0$ can be "relaxed" by replacing g by $\tilde{g}$, where $\tilde{g}(x) \leq g(x)$ for all $x \in \mathcal{D}$; an equality constraint $c(x) = 0$ may be relaxed by first replacing it by two inequality constraints $c(x) \leq 0$ and $-c(x) \leq 0$, then relaxing these.

A classical relaxation for convex univariate φ or g is to replace g (or φ) by a tangent line approximation ℓ: $\ell(x_i) = ax_i + b$ at some point $x^{(0)} = (x_1^{(0)}, \ldots, x_i^{(0)}, \ldots, x_n^{(0)}) \in \mathcal{D}$. This can be done at multiple points $x^{(0)} \in \mathcal{D}$; the more the points, the tighter the enclosure $\tilde{\mathcal{F}}$ is to $\mathcal{F}$. Nonconvex univariate functions g can be relaxed by finding Lipschitz constants for g over $\mathcal{D}$, and replacing g by the affine lower bound implied by the Lipschitz constant. Mathematically rigorous Lipschitz constants can be computed with interval evaluations of g' over $\mathcal{D}$. However, we cannot tighten the enclosing set $\tilde{\mathcal{F}}$ by adding additional constraints

[16] To obtain a possibly better upper bound on the global optimum, we replace φ in Problem 10 by $-\varphi$ and then form and solve a relaxation of the resulting problem. However, for mathematical rigor, computing an upper bound on φ is more complicated than computing a lower bound over $\mathcal{D}$, since the region $\mathcal{D}$ may not contain a feasible point.

$\ell \leq 0$ corresponding to such nonconvex g without subdividing $\mathcal{D}$; if we could, we might be able to use this procedure to prove P = NP.

The parsing procedure illustrated in Sect. 5.2 with Example 5 and formulas (19) can be used to decompose the objective and each constraint into constraints depending on only one or two variables. The resulting constraints can then be relaxed with linear functions as we have just described.[17] The resulting relaxation of the overall problem (10) is then a sparse linear program whose size depends on the number of operations. Such relaxations are called *McCormick Relaxations* and, to our knowledge, first appeared in [36, 37]. Various researchers have studied such relaxations; one of our publications on the subject is [26, 30]. In [28], we[18] use the ideas to automatically analyze the difficulty (in terms of amount of nonconvexity, in a certain sense) of a B&B method to solve a problem.

Other researchers have improved upon McCormick relaxations for certain problems by defining relaxations for multiple combinations of operations, more than two variables, and by using more general functions (other than linear[19]) in the relaxation. C. Floudas' group and their α-BB algorithm (see [5] and subsequent work) come to mind. In α-BB, the effects of Hessian matrices over domains $\mathcal{D}$ are bounded using interval arithmetic; see [2, 5], etc.

5.4 Interval Arithmetic Software

Numerous software packages are available for interval arithmetic and for interval-based global optimization; we refer the reader to the aforementioned general references. One of the most well-known current interval arithmetic packages is Siegfried Rump's Matlab toolbox INTLAB [48, 49]. Our own GlobSol software[25], based on ideas in [24], subsequent developments in constraint propagation, and our own ACM Transactions on Mathematical Software algorithms, have been much cited in comparison with newer packages.

6 Fuzzy Technology: A Few Details

Since Zadeh's seminal work, a large number of papers and reports concerning underlying philosophy and technical details of optimization in a fuzzy context have appeared. One review, containing various references, is [56]; we recommend this review for further reading.

[17] Two-variable relaxations correspond to multiplication; the literature describes various relaxations to these.

[18] Another group previously published similar ideas, with a slightly different perspective; see [15].

[19] Notably, quadratics, since quadratic programs have been extensively studied.

Fuzzy optimization problems can be classified in various ways. One way is according to where the fuzziness occurs:

1. *fuzzy domain, fuzzy range, crisp objective*: The domain $\mathcal{X}$ of the objective φ and the constraints c_i and g_i is a fuzzy set $\mathfrak{X}$, and the range $\mathcal{Y} = \{y \mid y = \varphi(x) \text{ for } x \in \mathcal{X}\}$ is also a fuzzy set $\mathfrak{Y}$. However, the objective φ itself is assumed to be well-defined.
2. *crisp domain, fuzzy range*: The domain $\mathcal{X}$ is a subset of a usual real vector space, but the range Y is a fuzzy set $\mathfrak{Y}$.
3. *fuzzy domain, crisp range*: The domain $\mathcal{X}$ is a fuzzy set $\mathfrak{x}$, but the range Y is a usual real vector space, and φ is well-defined for crisp (real) vectors x.
4. *fuzzy function*: Whether or not the domain or range are fuzzy, the φ or the g_i or c_i may be defined in a fuzzy way.

The concept of a fuzzy function (Type 4) corresponds to an interval-valued objective φ, such as a polynomial φ with interval coefficients, in which the values of a function at a point are intervals. Similarly, if the values of some of the c_i or g_i at points are intervals, this can be construed to define a fuzzy domain. However, the concept of a fuzzy range does not otherwise seem to have an analog in interval optimization.

Optimization based on fuzzy sets typically proceeds by somehow reformulating the problem as a nonfuzzy (crisp) optimization problem. For instance, let us consider the case where the objective and constraints are real valued, as in the global optimization problem (10), but when both objective and constraints defining the feasible set are fuzzy. Then, minimizing φ can be interpreted to mean that there is $x \in \mathcal{X}$ at which $\varphi(x)$ is sufficiently likely to be judged minimum and at which the degree of membership in the feasible set is acceptably high. To describe this situation, we make the following definitions.

Definition 11 Let $\mathfrak{X} = \{\mathcal{X}, \mu_X\}$ be a fuzzy set, and let f be a function as in Theorem 1. Define the fuzzy set $\mathfrak{X}^{\{f=0\}} = \{\mathcal{X}^{\{f=0\}}, \mu_X^{\{f=0\}}\}$ by

$$\mathcal{X}^{\{f=0\}} = \{x \in \mathcal{X} \mid f(x) = 0\},$$

and define $\mu_X^{\{f=0\}} : \mathcal{X} \to [0, 1]$ to be a membership function[20] of x in $\mathcal{X}^{\{f=0\}}$. Similarly define $\mathfrak{X}^{\{f \le 0\}}$.

Definition 12 Let $\mathfrak{X}$ be as in Definition 11, and let φ be an objective function as in (10). Define $\mu_X^{\{\min \varphi\}} : \mathcal{X} \to [0, 1]$ to be a membership function for the set $\operatorname{argmin}_{x \in \mathcal{X}} \varphi(x)$.

In this context, with this notation, and with the feasible set defined by constraints c_i, $1 \le i \le m_1$, and g_j, $1 \le j \le m_2$ of (10), one crisp optimization problem formulation is

[20] Derived from μ_X and f or designed some other way, to take account of perceived goodness of values of φ.

$$\max \min_{\mathcal{X}} \left\{ \mu_X^{\{\min \varphi\}}(x); \quad \mu_X^{\{c_i=0\}}(x) \ , 1 \le i \le m_1; \quad \mu_X^{\{g_i \le 0\}}(x) \ , 1 \le j \le m_2 \right\}. \tag{20}$$

Solution of the crisp problem (20) can proceed, e.g., via a specialized interval-based branch and bound algorithm, by nonrigorous techniques, or by local methods such as descent methods.

For a somewhat simpler example of a reformulation, consider a problem of our Type 3 classification that is unconstrained (no c_i and no g_i), and suppose we have predetermined an acceptable degree of membership α_0. We may then reformulate the problem as

$$\min_{x \in \mathcal{X}} \mu_X^{\{\min \varphi\}}(x) \ \text{ such that } \mu_X^{\{\min \varphi\}}(x) \ge \alpha_0. \tag{21}$$

Objectives (20) and (21) are merely examples of formulations of crisp objectives corresponding to fuzzy optimization problems. In contrast to what has been developed in interval optimization, techniques for fuzzy optimization consist more of a guiding way of thinking than prescriptions for deriving algorithms.

A common problem with fuzzy optimization is that crisp reformulations often do not have isolated minimizing points, especially when the range is a fuzzy set (that is, when the values in the range of f are known only to be within a degree of membership). For example, suppose we required only that $\mu_X^{\{\min \varphi\}}(x) \ge \alpha_0$ in (21), and not that $\mu_X^{\{\min \varphi\}}(x)$ also be minimized. Problem (21) then becomes a constraint satisfaction problem that usually would have an open region of solutions.

It is impossible here to cover all of the techniques in the literature for fuzzy technology, especially when dealing with specific applications. A recent proceedings is [27].

7 Conclusions

Fuzzy sets and interval analysis have an outwardly similar history in the twentieth century. Techniques and tools for implementing and using interval computations and fuzzy computations are similar, and implementations of fuzzy logic use interval arithmetic. However, the underlying premises are very different, based on the innate difference between measurement uncertainty and ambiguity in human thought and language. Properties of interval arithmetic are largely deduced, while properties of particular fuzzy logic systems are largely designed. While interval arithmetic strives to guarantee the range of all possible outcomes, fuzzy sets and fuzzy logic strive to automate ambiguous human perception and decision processes in a way that the outcome seems reasonable to humans.

We are currently investigating combining fuzzy sets in interval global optimization: fuzzy sets may be a way to control heuristics involved in the branching and fathoming processes.

References

1. Ackleh, A.S., Allen, E.J., Kearfott, R.B., Seshaiyer, P.: Classical and Modern Numerical Analysis: Theory, Methods, and Practice. Taylor and Francis, Boca Raton (2009)
2. Adjiman, C.S., Dallwig, S., Floudas, C.A., Neumaier, A.: A global optimization method, αBB, for general twice-differentiable constrained NLPs. I. Theoretical advances. Comput. Chem. Eng. **22**(9), 1137–1158 (1998)
3. Alefeld, G., Herzberger, J.: Nullstelleneinschließung mit dem Newton-Verfahren ohne Invertierung von Intervallmatrizen. (German) [Including zeros of nonlinear equations by the Newton method without inverting interval matrices]. Numer. Math. **19**(1), 56–64 (1972)
4. Alefeld, G., Herzberger, J.: Introduction to Interval Computations. Academic Press, New York (1983). Transl. by Jon G. Rokne from the original German 'Einführung in die Intervallrechnung'
5. Androulakis, I.P., Maranas, C.D., Floudas, C.A.: αbb: a global optimization method for general constrained nonconvex problems. J. Global Optim. **7**(4), 337–363 (1995). https://doi.org/10.1007/BF01099647
6. Anguelov, R.: The algebraic structure of spaces of intervals: Contribution of Svetoslav Markov to interval analysis and its applications. BIOMATH **2** (2014). https://doi.org/10.11145/j.biomath.2013.09.257
7. Barreto, G.A., Coelho, R. (eds.): Fuzzy Information Processing—37th Conference of the North American Fuzzy Information Processing Society, NAFIPS 2018, Fortaleza, Brazil, July 4–6, 2018, Proceedings, *Communications in Computer and Information Science*, vol. 831. Springer, Berlin (2018). https://doi.org/10.1007/978-3-319-95312-0
8. Berz, M., Makino, K.: Taylor models web page (2020). https://bt.pa.msu.edu/index_TaylorModels.htm. Accessed 30 March 2020
9. Berz, M., Makino, K., Kim, Y.K.: Long-term stability of the Tevatron by verified global optimization. Nucl. Instrum. Methods Phys. Res., Sect. A **558**(1), 1–10 (2006). https://doi.org/10.1016/j.nima.2005.11.035. http://www.sciencedirect.com/science/article/pii/S0168900205020383. Proceedings of the 8th International Computational Accelerator Physics Conference
10. Bessiere, C.: Chapter 3 - constraint propagation. In: Rossi, F., van Beek, P., Walsh, T. (eds.) Handbook of Constraint Programming. Foundations of Artificial Intelligence, vol. 2, pp. 29–83. Elsevier, Amsterdam (2006). https://doi.org/10.1016/S1574-6526(06)80007-6. http://www.sciencedirect.com/science/article/pii/S1574652606800076
11. Corliss, G.F., Rall, L.B.: Bounding derivative ranges. In: Pardalos, P.M., Floudas, C.A. (eds.) Encyclopedia of Optimization. Kluwer, Dordrecht (1999)
12. Du, K.: Cluster problem in global optimization using interval arithmetic. Ph.D. thesis, University of Southwestern Louisiana (1994)
13. Du, K., Kearfott, R.B.: The cluster problem in global optimization: the univariate case. Comput. Suppl. **9**, 117–127 (1992)
14. Dwyer, P.S.: Matrix inversion with the square root method. Technometrics **6**, 197–213 (1964)
15. Epperly, T.G.W., Pistikopoulos, E.N.: A reduced space branch and bound algorithm for global optimization. J. Global Optim. **11**(3), 287–311 (1997)
16. Hansen, E., Walster, G.W.: Global Optimization Using Interval Analysis. Marcel Dekker, New York (2003)
17. Hansen, E.R.: Global Optimization Using Interval Analysis. Marcel Dekker, New York (1983)
18. IEEE: 1788-2015—IEEE Standard for Interval Arithmetic. IEEE Computer Society Press, 1109 Spring Street, Suite 300, Silver Spring (2015). https://doi.org/10.1109/IEEESTD.2015.7140721. http://ieeexplore.ieee.org/servlet/opac?punumber=7140719. Approved 11 June 2015 by IEEE-SA Standards Board. http://ieeexplore.ieee.org/servlet/opac?punumber=7140719
19. IEEE Task P754: IEEE 754-2008, Standard for Floating-Point Arithmetic. IEEE, New York, NY, USA (2008). https://doi.org/10.1109/IEEESTD.2008.4610935. http://en.wikipedia.org/wiki/IEEE_754-2008; http://ieeexplore.ieee.org/servlet/opac?punumber=4610933

20. Jaulin, L., Keiffer, M., Didrit, O., Walter, E.: Applied Interval Analysis. SIAM, Philadelphia (2001)
21. Karhbet, S., Kearfott, R.B.: Range bounds of functions over simplices, for branch and bound algorithms. Reliab. Comput. **25**, 53–73 (2017). Special volume containing refereed papers from SCAN 2016, guest editors Vladik Kreinovich and Warwick Tucker
22. Kearfott, R.B.: Preconditioners for the interval Gauss–Seidel method. SIAM J. Numer. Anal. **27**(3), 804–822 (1990). https://doi.org/10.1137/0727047
23. Kearfott, R.B.: Decomposition of arithmetic expressions to improve the behavior of interval iteration for nonlinear systems. Computing **47**(2), 169–191 (1991)
24. Kearfott, R.B.: Rigorous Global Search: Continuous Problems. No. 13 in Nonconvex optimization and its applications. Kluwer Academic Publishers, Dordrecht (1996)
25. Kearfott, R.B.: GlobSol User Guide. Optim. Methods Softw. **24**(4–5), 687–708 (2009)
26. Kearfott, R.B.: Erratum: Validated linear relaxations and preprocessing: some experiments. SIAM J. Optim. **21**(1), 415–416 (2011)
27. Kearfott, R.B., Batyrshin, I.Z., Reformat, M., Ceberio, M., Kreinovich, V. (eds.): Fuzzy Techniques: Theory and Applications - Proceedings of the 2019 Joint World Congress of the International Fuzzy Systems Association and the Annual Conference of the North American Fuzzy Information Processing Society IFSA/NAFIPS'2019 (Lafayette, Louisiana, USA, June 18–21, 2019). Advances in Intelligent Systems and Computing, vol. 1000. Springer, Berlin (2019). https://doi.org/10.1007/978-3-030-21920-8
28. Kearfott, R.B., Castille, J.M., Tyagi, G.: Assessment of a non-adaptive deterministic global optimization algorithm for problems with low-dimensional non-convex subspaces. Optim. Methods Softw. **29**(2), 430–441 (2014). https://doi.org/10.1080/10556788.2013.780058
29. Kearfott, R.B., Du, K.: The cluster problem in multivariate global optimization. J. Global Optim. **5**, 253–265 (1994)
30. Kearfott, R.B., Hongthong, S.: Validated linear relaxations and preprocessing: some experiments. SIAM J. Optim. **16**(2), 418–433 (2005). https://doi.org/10.1137/030602186. http://epubs.siam.org/sam-bin/dbq/article/60218
31. Kearfott, R.B., Hu, C.Y., Novoa III, M.: A review of preconditioners for the interval Gauss–Seidel method. Interval Comput. **1**(1), 59–85 (1991). http://interval.louisiana.edu/reliable-computing-journal/1991/interval-computations-1991-1-pp-59-85.pdf
32. Kearfott, R.B., Muniswamy, S., Wang, Y., Li, X., Wang, Q.: On smooth reformulations and direct non-smooth computations in global optimization for minimax problems. J. Global Optim. **57**(4), 1091–1111 (2013)
33. Kreinovich, V.: Relations between interval and soft computing. In: Hu, C., Kearfott, R.B., de Korvin, A. (eds.) Knowledge Processing with Interval and Soft Computing, Advanced Information and Knowledge Processing, pp. 75–97. Springer, Berlin (2008). https://doi.org/10.1007/BFb0085718
34. Liu, D.: A Bernstein-polynomial-based branch-and-bound algorithm for polynomial optimization over simplexes. Ph.D. thesis, Department of Mathematics, University of Louisiana, Lafayette, LA 70504-1010 USA (2021). (work in progress)
35. Makino, K., Berz, M.: Efficient control of the dependency problem based on Taylor model methods. Reliab. Comput. **5**(1), 3–12 (1999)
36. McCormick, G.P.: Converting general nonlinear programming problems to separable nonlinear programming problems. Tech. Rep. T-267, George Washington University, Washington (1972)
37. McCormick, G.P.: Computability of global solutions to factorable nonconvex programs. Math. Prog. **10**(2), 147–175 (1976)
38. Messine, F.: Deterministic global optimization using interval constraint propagation techniques. RAIRO-Oper. Res. **38**(4), 277–293 (2004). https://doi.org/10.1051/ro:2004026
39. Moore, R.E.: Interval arithmetic and automatic error analysis in digital computing. Ph.D. Dissertation, Department of Mathematics, Stanford University, Stanford, CA, USA (1962). http://interval.louisiana.edu/Moores_early_papers/disert.pdf. Also published as Applied Mathematics and Statistics Laboratories Technical Report No. 25
40. Moore, R.E.: Interval analysis. Prentice-Hall, Upper Saddle River (1966)

41. Moore, R.E.: Methods and Applications of Interval Analysis. SIAM, Philadelphia (1979)
42. Moore, R.E., Kearfott, R.B., Cloud, M.J.: Introduction to Interval Analysis. SIAM, Philadelphia (2009). http://www.loc.gov/catdir/enhancements/fy0906/2008042348-b.html; http://www.loc.gov/catdir/enhancements/fy0906/2008042348-d.html; http://www.loc.gov/catdir/enhancements/fy0906/2008042348-t.html
43. Neumaier, A.: Interval Methods for Systems of Equations. Encyclopedia of Mathematics and Its Applications, vol. 37. Cambridge University Press, Cambridge (1990)
44. Neumaier, A.: Complete search in continuous global optimization and constraint satisfaction. In: Iserles, A. (ed.) Acta Numerica 2004, pp. 271–369. Cambridge University Press, Cambridge (2004)
45. Paulavčius, R., Žilinskas, J.: Simplicial Global Optimization. Springer, Berlin (2014). https://doi.org/10.1007/978-1-4614-9093-7
46. Paulavičius, R., Žilinskas, J.: Simplicial global optimization. J. Global Optim. **60**(4), 801–802 (2014). http://EconPapers.repec.org/RePEc:spr:jglopt:v:60:y:2014:i:4:p:801-802
47. Ratschek, H., Rokne, J.G.: New Computer Methods for Global Optimization. Wiley, New York (1988)
48. Rump, S.M.: INTLAB–INTerval LABoratory. In: Csendes, T. (ed.) Developments in Reliable Computing: Papers Presented at the International Symposium on Scientific Computing, Computer Arithmetic, and Validated Numerics, SCAN-98, in Szeged, Hungary, Reliable Computing, vol. 5(3), pp. 77–104. Kluwer Academic Publishers Group, Norwell and Dordrecht, (1999). http://www.ti3.tu-harburg.de/rump/intlab/
49. Rump, S.M.: INTLAB—INTerval LABoratory (1999–2020). http://www.ti3.tu-harburg.de/rump/intlab/
50. Rump, S.M., Kashiwagi, M.: Implementation and improvements of affine arithmetic. Nonlinear Theory and Its Applications, IEICE **6**(3), 341–359 (2015). https://doi.org/10.1587/nolta.6.341
51. Sahinidis, N.V.: BARON: A general purpose global optimization software package. J. Global Optim. **8**(2), 201–205 (1996)
52. Sahinidis, N.V.: BARON Wikipedia page (2020). https://en.wikipedia.org/wiki/BARON. Accessed 16 March 2020
53. Smets, P.: The degree of belief in a fuzzy event. Inf. Sci. **25**, 1–19 (1981)
54. Stolfi, J., de Figueiredo, L.H.: An introduction to affine arithmetic. TEMA (São Carlos) **4**(3), 297–312 (2003). https://doi.org/10.5540/tema.2003.04.03.0297. https://tema.sbmac.org.br/tema/article/view/352
55. Sunaga, T.: Theory of interval algebra and its application to numerical analysis. RAAG Mem. **2**, 29–46 (1958). http://www.cs.utep.edu/interval-comp/sunaga.pdf
56. Tang, J.F., Wang, D.W., Fung, R.Y.K., Yung, K.L.: Understanding of fuzzy optimization: Theories and methods. J. Syst. Sci. Complex. **17**(1), 117 (2004). http://123.57.41.99/jweb_xtkxyfzx/EN/abstract/article_11437.shtml
57. Van Hentenryck, P.: Constraint Satisfaction in Logic Programming. MIT Press, Cambridge (1989)
58. Van Hentenryck, P., McAllester, D., Kapur, D.: Solving polynomial systems using a branch and prune approach. SIAM J. Numer. Anal. **34**(2), 797–827 (1997). https://doi.org/10.1137/S0036142995281504
59. Vellasco, M., Estevez, P. (eds.): 2018 IEEE International Conference on Fuzzy Systems, FUZZ-IEEE 2018, Rio de Janeiro, Brazil, July 8–13, 2018. IEEE, Piscataway (2018). http://ieeexplore.ieee.org/xpl/mostRecentIssue.jsp?punumber=8466242
60. Žilinskas, J., Bogle, D.: A survey of methods for the estimation ranges of functions using interval arithmetic. In: Models and Algorithms for Global Optimization: Essays Dedicated to Antanas Žilinskas on the Occasion of His 60th Birthday, pp. 97–108 (2007). https://doi.org/10.1007/978-0-387-36721-7_6
61. Warmus, M.M.: Approximations and inequalities in the calculus of approximations. classification of approximate numbers. Bull. Acad. Polon. Sci. Ser. Sci. Math. Astronom. Phys. **9**, 241–245 (1961). http://www.ippt.gov.pl/~zkulpa/quaphys/warmus.html

62. Wikipedia: COPROD web page (2020). http://coprod.constraintsolving.com/. Accessed 17 March 2020
63. Wikipedia: Interval arithmetic Wikipedia page (2020). https://en.wikipedia.org/wiki/Interval_arithmetic. Accessed 30 March 2020
64. Young, R.C.: The algebra of many-valued quantities. Math. Ann. **104**, 260–290 (1931)
65. Zadeh, L.A.: Fuzzy sets. Inf. Control **8**, 338–353 (1965)
66. Žilinskas, J.: Branch and bound with simplicial partitions for global optimization. Math. Model. Anal. **13**(1), 145–159 (2008). https://doi.org/10.3846/1392-6292.2008.13.145-159

Optimization Under Uncertainty Explains Empirical Success of Deep Learning Heuristics



Vladik Kreinovich and Olga Kosheleva

1 Formulation of the Problem

Need for Machine Learning One of the main objectives of science and engineering is to

- predict the future state of the world and
- come up with devices and strategies that would make this future state better.

In some practical situations, we know how the state changes with time. For example, in meteorology, we know the partial differential equations that describe the atmospheric processes, and we can use these equations to predict tomorrow's weather. In such situations, prediction becomes largely a purely computational problem.

In many other situations, however, we do not know the equation describing the system's dynamics. In such situations, we need to learn this dynamics from data. The corresponding techniques are known as *machine learning*.

Neural Networks: A Brief Reminder Not only computers can learn, we humans can learn, and many animals can learn. So, to make machines learn, a reasonable idea is to emulate how we humans learn. All our mental activities—including learning—are performed by special interconnected cells called *neurons*. Thus, a reasonable idea is to have a network of devices simulating such neurons; such a network is known as an *artificial neural network*, or simply a *neural network*, for short.

V. Kreinovich (✉) · O. Kosheleva
University of Texas at El Paso, El Paso, TX, USA
e-mail: olgak@utep.edu; vladik@utep.edu

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_8

Each biological neuron takes several input signals $x_1, \ldots, x_n$ and transforms them into an output. In the first approximation, signals are first linearly combined, with appropriate weights $w_0, w_1, \ldots, w_n$. Then, some nonlinear transformation $s_0(z)$ – known as the *activation function*—is applied to the resulting linear combination $w_0 + w_1 \cdot x_1 + \ldots + w_n \cdot x_n$: $y = s_0(w_0 + w_1 \cdot x_1 + \ldots + w_n \cdot x_n)$. (This activation function is usually non-decreasing.) In most neural network models, this is exactly how each neuron operates.

Signals from some neurons become inputs to other neurons, etc. First, neurons get output from the problem. The set of such neurons is usually called the *first* (*input*) layer. The outputs of these neurons are processed by other neurons—which form the *second layer*, etc., until we get the last (*output*) layer that will eventually generate the desired prediction.

Such neural networks have indeed turned out to be efficient machine learning tools; see, e.g., [4, 8, 16, 17].

Deep Neural Networks: Main Idea In many cases, artificial neural networks are not yet as good as well-trained human experts. One of the main reasons for this is that while our brain uses billions of neurons, the computer systems do not yet have an ability to incorporate that many neurons. With the progress in computing hardware, however, we are able to incorporate more and more neurons and thus, hopefully, get better and better performance.

In principle, we have several possible ways to add neurons to a network:

- we can place more neurons in each layer, or
- we can form new layers, or
- we can do both.

Researchers tried all three options and found out that the best results are achieved if we add new layers—i.e., if we consider "deep" neural networks, with a large number of layers. This is the main idea behind *deep learning* and *deep neural networks*; see, e.g., [8].

This can easily be explained (see, e.g., [12]). Indeed, for the same number of variables, we want to get more accurate approximations. For a given number of variables and given accuracy, we get N possible combinations. If all combinations correspond to different functions, we can implement N functions. However, if some combinations lead to the same function, we can implement fewer different functions.

If we have K neurons in a layer, then each of $K!$ permutations of these neurons retains the resulting function; see, e.g., [3, 9]. Thus, instead of N functions, we only implement

$$\frac{N}{K!} \ll N \text{ functions.}$$

So, to increase approximation accuracy, we need to minimize the number K of neurons in each layer.

To get a good accuracy, we need many parameters, thus many neurons. Since each layer has few neurons, we thus need many layers.

Deep Learning Became a Success The main idea behind deep learning may be reasonable, but by itself, this idea does not necessarily lead to successful learning. Many improving ideas have been proposed, some worked, some did not. As a result of all this trial-and-error experimentation, at present, deep learning is the most efficient machine learning tool.

Let us briefly enumerate the main ideas that made deep learning a success.

First Idea: What Activation Function $s_0(z)$ Should We Use? Originally, artificial neural networks used *sigmoid function* $s_0(z) = \dfrac{1}{1+\exp(-z)}$. The main reason for their use was that this is how most biological neurons work.

This activation function worked well for traditional neurons, but, somewhat surprisingly, for multi-layer deep network, it did not work at all. Several alternatives have been tried—until it turned out that the *rectified linear* activation function $s_0(z) = \max(0, z)$ works the best. Why is not fully clear. There is some understanding of why this new activation function is better than the original sigmoid function (see, e.g., [8]), but it is not clear why namely this activation function—and not many similar ones—works the best.

And, to add to the suspense, in some cases, it is efficient to add a layer of sigmoid neurons to the network consisting of rectified linear neurons. Why sigmoid and not anything else?

Second Idea: Need for Pooling If we use a neural network to process an image or a signal, we quickly realize that the corresponding input contains too many numerical values, and processing all the them makes computations last forever. For example, an average-quality image has 1,000,000 pixels—so, we need 1,000,000 real numbers to describe this image. This requires too much of computation time.

To decrease the size of the input and thus make computations feasible, it is desirable to "pool" several inputs—e.g., inputs corresponding to nearby locations—into a single input value. Such "pooling" is ubiquitous in signal processing—e.g., to get a more accurate measurement result, we often measure the same quantity several times and take an average. Based on this experience, one would expect that for neural networks as well, the average pooling should work the best. Surprisingly, it turns out that in most situations, maximum pooling works much better. Why?

Third and Fourth Ideas: Softmax Instead of Maximum and Geometric Mean Instead of the Usual Average At several stages, during the learning process, we may get somewhat different results. For example, we can get different results at different stages of the training or—if we want to speed up the learning process and run several stages in parallel—different results produced by several subsystems. In such situations, there are two possible ways to handle with this multiplicity:

- the first way is to select the most promising result—i.e., the result for which the estimated quality is the best;

- the second way is, instead of *selecting* one of the results, to *combine* them, hoping that such a combination will avoid random perturbations that make all the results imperfect.

Both ways are used in machine learning, but each one with a twist:

- Instead of selecting the best result, i.e., the result x with the largest value of the corresponding objective function $J(x)$, we select one of the results with some probability. This probability increases with $J(x)$ but remains non-zero even for much smaller values $J(x)$. In other words, instead of "hard maximum," we use "soft maximum" (softmax, for short). Empirically, the most efficient softmax is when the probability of selecting x is proportional to $\exp(\beta \cdot J(x))$ for some $\beta > 0$. A natural question is: why?
- For combination ("averaging"), instead of a seemingly natural arithmetic average, it turns out that the geometric mean often works much better. Why?

What We Do in This Paper From the theoretical viewpoint, we have a challenge: for deep learning to become a success, four ideas are needed (and several others as well, we just mentioned the main ones). However, it is not clear why these particular heuristics were successful, and others, seemingly more natural and promising ones, did not work as well.

In this chapter, we provide a theoretical explanation for these empirical successes. It turns out that all these successes can be explained if we apply optimization under uncertainty; see also [10, 14].

2 Why Rectified Linear Neurons Are Efficient: A Theoretical Explanation

Question: Reminder Let us start with the very first question: which activation function $s_0(z)$ should we use? To be more precise, what is the *optimal* choice of the activation function? Which functions are the best according to some optimality criterion?

To answer this question, let us first recall what we mean by an optimality criterion in the first place. This recall will be useful when answering all other questions as well, so we will formulate it in general terms.

What Do We Mean by Optimality Criterion? In general, what do we mean by an optimality criterion, i.e., by a criterion that allows us to select one of many possible alternatives? In many cases, we have a well-defined objective function $F(a)$—i.e., we have a numerical value $F(a)$ attached to each alternative a. We then select the alternative a for which this value is—depending on what we want—either the largest or the smallest.

For example, when we look for the shortest path,

- we assign, to each path a, its length $F(a)$, and

- we select the path for which this length is the smallest possible.

When we look for an algorithm for solving problems of given size, often

- we assign, to each algorithm a, the worst-case computation time $F(a)$ on all inputs of this size, and
- we select the algorithm a for which this worst-case time $F(a)$ is the smallest possible.

However, an optimality criterion can be more complicated. For example, we may have several different shortest paths a for a car to go from one city location to another. In this case, it may be reasonable to select, among these shortest paths, a path a along which the overall exposure to pollution $G(a)$ is the smallest. The resulting optimality criterion can no longer be described by a single objective function, and it is more complicated: we prefer a to a' if

- either $F(a) < F(a')$ or
- $F(a) = F(a')$ and $G(a) < G(a')$.

Similarly, if we have two different algorithms a with the same worst-case computation time $F(a)$, we may want to select, among them, the one for which the average computation time $G(a)$ is the smallest possible. In this case too, we prefer a to a' if

- either $F(a) < F(a')$ or
- $F(a) = F(a')$ and $G(a) < G(a')$.

The optimality criterion can be even more complicated. However, no matter how many different objective functions we use, we do need to have a way to compare different alternatives. Thus, we can define a general optimality criterion as a relation $\preceq$ on the set of all possible alternatives, so that $a \preceq a'$ means that the alternative a' is better (or of the same quality) than the alternative a.

Naturally, each alternative has the same quality as itself, so we should have $a \preceq a$. Such relations are called *reflexive*. Also, if a' is better than or of the same quality as a and a'' is better than or of the same quality as a', then, clearly, a'' should be better than or of the same quality as a. Relations with this property are known as *transitive*. So, we arrive at the following definition.

Definition 1 Let a set A be given; its elements will be called *alternatives*.

- We say that a relation $\preceq$ on the set A is *reflexive* if $a \preceq a$ for all a.
- We say that a relation $\preceq$ is *transitive* if $a \preceq a'$ and $a' \preceq a''$ imply that $a \preceq a''$.
- By an *optimality criterion* in the set A, we mean a reflexive transitive relation $\preceq$ on this set.

What Is an Optimal Alternative? In these terms, an alternative a_{opt} is *optimal* if it is better (or of the same quality) than all other alternatives.

Definition 2 We say that an alternative a_{opt} is *optimal* with respect to the optimality criterion $\preceq$ on the set A if $a \preceq a_{\text{opt}}$ for all $a \in A$.

The Optimality Criterion Must Be Useful We want an optimality criterion to be useful, i.e., we want to use it to select an alternative. Thus, there should be at least one alternative that is optimal according to this criterion.

Definition 3 We say that an optimality criterion $\preceq$ is *useful* if there exists at least one optimal alternative.

When Is the Optimality Criterion Final? What if several different alternatives are optimal according to the given criterion? In this case, we can use this non-uniqueness to optimize something else.

For example, if, on a given class of benchmarks, neurons that use several different activation functions have the same average approximation error, we can select, among them, the function with the smallest computational complexity.

This way, instead of the original optimality criterion, we, in effect, use a new criterion according to which s_0' is better than s_0 if

- either it has the smaller average approximation error or
- it has the same average approximation error and the smaller computational complexity.

If, based on this modified criterion, we still have several different activation functions that are equally good, we can use this non-uniqueness to optimize something else, e.g., worse-case approximation accuracy, etc.

Thus, every time the optimality criterion selects several equally good alternatives, we, in effect, replace it with a modified criterion and keep modifying it until *finally* we get a criterion for which only one alternative is optimal. So, we arrive at the following definition.

Definition 4 We say that a useful optimality criterion is *final* if there exists exactly one alternative that is optimal with respect to this optimality criterion.

Natural Symmetries: General Description In many practical situations, we have transformations that do not affect the physics of the situation.

For example, when we measure a physical signal, the resulting numerical value depends on what measuring unit we use in this measurement. When we measure the height in meters, the person's height is 1.7. However, if we measure the same height in centimeters, we get a different numerical value: 170.

In general, if, instead of the original measuring unit, we use a different unit that is λ times smaller than the previous one, then all the numerical values get multiplied by λ: $x \to \lambda \cdot x$. For example, if we replace meters by centimeters, all numerical values get multiplied by $\lambda = 100$.

If we use a different unit, then numerical values change, but the physical situation remains the same. So, it is reasonable to expect that this change should not affect which alternatives—e.g., which activation functions—should be better and which should be worse.

Other possible transformations come from the fact that for many physical quantities such as time, the choice of a starting point is also arbitrary—we can select

a moment of time which is t_0 moments earlier, in which case each numerical value t is replaced by $t + t_0$.

In all such cases, we have a class of corresponding "natural" transformations. Clearly, if two transformation are natural, then their composition—when we first apply the first one and then the second one—should also be natural. Similarly, the inverse should be natural. The class of all transformations with this property is known as a *transformation group*.

Definition 5 Let a set A be given.

- A *transformation* is a reversible function from A to A.
- A set G of transformations is called a *transformation group* if it satisfies the following two properties:
  - if $g \in G$, then the inverse transformation g^{-1} should also belong to G, and
  - if f and g belong to G, then their composition $f \circ g$ also belongs to G.
- Transformations from the group G are called *symmetries.*

For natural transformations, the relation $a \preceq a'$ should not change if we apply the same symmetry to both a and a'. This leads to the following definition.

Definition 6 We say that the optimality criterion $\preceq$ is *invariant* with respect to the transformation group G (or simply, G-*invariant*, for short) if for all $a, a' \in A$, and for all $g \in G$, we have

$$a \preceq a' \Leftrightarrow g(a) \preceq g(a').$$

Main Lemma It turns out that for invariant final optimality criteria, the optimal alternative is also invariant, in some reasonable sense.

Definition 7 We say that an alternative $a_0 \in A$ is *invariant* with respect to a transformation group G (or, for short, G-*invariant*) if $g(a_0) = a_0$ for all $g \in G$.

Proposition 1 *For each G-invariant final optimality criterion, the optimal alternative a_{opt} is also G-invariant.*

Comments

- It is important to emphasize that our result is *not* based on selecting a *single* optimality criterion: it holds for *all* optimality criteria that satisfy reasonable properties—such as being final and being G-invariant.
- For reader's convenience, all the proofs are placed in the special (last) section of this paper.

Natural Symmetries of an Activation Function We claim that for an activation function, it is important to take into account that we can change the measuring unit and thus get different numerical values for describing the same quantity.

In the neural networks, inputs are usually normalized, so, at first glance, there seems to be no need to such rescaling $x \to \lambda \cdot x$. However, normalization parameters may change if we get new data.

For example, often, normalization means that the range of possible values of some positive quantity is linearly rescaled to the interval $[0, 1]$—by dividing all inputs by the largest possible value of the corresponding quantity. When we add more data points, we may get values that are somewhat larger than the largest of the previously observed value. In this case, the normalization based on the enlarged data set leads to rescaling of all previously normalized values—i.e., in effect, to a change in the measuring unit.

It is therefore reasonable to require that the quality of an activation function does not depend on the choice of the measuring unit. Let us describe this property in precise terms.

Scaling-Invariance: Towards a Precise Description Suppose that in some selected units, the activation function has the form $s(x)$. If we replace the original measuring unit by a new unit that is λ times larger than the original one, then the value x in the new units is equivalent to $\lambda \cdot x$ in the old units. If we apply the old-unit activation function to this amount, we get the output of $s(\lambda \cdot x)$ of old units—which is equivalent to $\lambda^{-1} \cdot s(\lambda \cdot x)$ new units.

Thus, after the change in units, the transformation described, in the original units, by an activation function $s(x)$ is described, in the new units, by a modified activation function $\lambda^{-1} \cdot s(\lambda \cdot x)$. So, the above requirement takes the following form.

Definition 8

- For each $\lambda > 0$, by a λ-*scaling* T_λ, we mean a transformation from the original function $s(x)$ to a new function $(T_\lambda(s))(x) \stackrel{\text{def}}{=} \lambda^{-1} \cdot s(\lambda \cdot x)$.
- We say that an optimality criterion or an alternative is *scale-invariant* if they are invariant with respect to all λ-scalings.

Now, we are ready to formulate our first result.

Proposition 2 *If a function $s_0(x)$ is optimal with respect to some final scale-invariant optimality criterion, then it has the following form:*

- $s_0(x) = c_+ \cdot x$ *for* $x \geq 0$ *and*
- $s_0(x) = c_- \cdot x$ *for* $x < 0$.

Comment One can easily check that each such function has the form

$$s_0(x) = c_- \cdot x + (c_+ - c_-) \cdot \max(x, 0).$$

Thus, if $c_+ \neq c_-$, i.e., if the corresponding activation function is not linear, then the class of functions represented by s_0-neural networks coincides with the class of functions represented by rectified linear neural networks.

So, *we have a theoretical justification for the success of rectified linear activation functions.*

Historical Comment This result first appeared in [6].

3 Why Sigmoid Activation Functions

Scalings, Shifts, What Else? In our explanation of success of rectified linear activation functions, we used scale-invariance. To explain why it is efficient to use the sigmoid activation functions for some layers, let us analyze what other types of invariance we can have. We have already mentioned that we can also have *shift-invariance* $t \to t + t_0$. Together, shifts and scalings form a group of all linear transformations.

Since we want to be able to represent transformations in a computer, and in a computer, we can only store finitely many parameter values, the desired group should depend only on a finite number of parameters—i.e., it should be *finite-dimensional*. It turns out that the only finite-dimensional group that strictly contains the group of all linear transformations is the group of all fractionally linear transformations $g(x) = \dfrac{A \cdot x + B}{C \cdot x + D}$. The proof of this result can be found, e.g., in [13, 15, 18] (a more general result was formulated by Norbert Wiener, the father of cybernetics, in [19]). Under assumptions of smoothness, the proof is given in the proofs section.

We Should Consider Families of Functions If we have a reasonable transformation g and $x \to y = s_0(x)$ is a reasonable activation function, then after rescaling $y \to g(y)$, we shall also get a reasonable activation function. Thus, with the original function $s_0(z)$, all the functions $g(s_0(z))$ should also be reasonable. From this viewpoint, instead of considering the original activation functions, it takes more sense to consider *families* of such functions $\{g(s_0(x))\}_{g \in G}$. Out of all such families, we should select the optimal one.

In addition to changing the result y of neural activity, we can also rescale the inputs x—e.g., by shift, which corresponds to changing the starting point for x. It is reasonable to require that the relative quality of different families of activation functions should not depend on what starting point we use. Let us describe this in precise terms.

Definition 9 By a *family* of activation functions, we mean the family of all the functions of the type $\dfrac{A \cdot s_0(x) + B}{C \cdot s_0(x) + D}$, where $s_0(x)$ is a given smooth non-decreasing function and A, B, C, and D are arbitrary constants.

Definition 10 By a *shift*, we mean a transformation $s_0(x) \to s_0(x + x_0)$ for some x_0.

Proposition 3 *For every shift-invariant final optimality criterion on the set of all families of activation functions, the optimal family corresponds to* $s_0(x) = x$, $s_0(x) = \exp(\beta \cdot x)$, *or to the sigmoid function* $s_0(x) = \dfrac{1}{1 + c \cdot \exp(-\beta \cdot x)}$.

Comment The sigmoid function can be reduced to its standard form by an appropriate rescaling of x: from x to $x' = \beta \cdot x - \ln(c)$. The cases $s_0(x) = x$

and $s_0(x) = \exp(\beta \cdot x)$ are actually limit cases of the sigmoid. Thus, the above result indeed explains the empirical optimality of the sigmoid activation function.

Historical Comment This result was previously mentioned in [12, 13, 15, 18].

4 Selection of Poolings

What Is Pooling: Towards a Precise Definition We start with m values $a_1, \ldots, a_m$, and we want to generate a single value a that represents all these values.

In the case of arithmetic average, pooling means that we select a value a for which $a_1 + \ldots + a_m = a + \ldots + a$ (m times). In general, pooling means that we select some combination operation $*$ and we then select the value a for which $a_1 * \ldots * a_m = a * \ldots * a$ (m times). For example, if, as a combination operation, we select $\max(a, b)$, then the corresponding condition

$$\max(a_1, \ldots, a_n) = \max(a, \ldots, a) = a$$

describes the max-pooling.

From this viewpoint, selecting pooling means selecting an appropriate combination operation. Thus, selecting the optimal pooling means selecting the optimal combination operation.

Natural Properties of a Combination Operation The combination operation transforms two non-negative values—such as intensity of an image at a given location—into a single non-negative value. The result of applying this operation should not depend on the order in which we combine the values. Thus, we should have $a * b = b * a$ (commutativity) and $a * (b * c) = (a * b) * c$ (associativity).

Definition 11 By a *combination operation*, we mean a commutative, associative operation $a * b$ that transforms two non-negative real numbers a and b into a non-negative real number $a * b$.

Scale-Invariance As we have mentioned earlier, numerical values of a physical quantity depend on the choice of a measuring unit. It is therefore reasonable to require that the preference relation should not change if we simply change the measuring unit. Let us describe this requirement in precise terms. If, in the original units, we had the operation $a * b$, then, in the new units, the operation will take the following form:

- first, we transform the values a and b into the new units, so we get $a' = \lambda \cdot a$ and $b' = \lambda \cdot b$;
- then, we combine the new numerical values, getting $(\lambda \cdot a) * (\lambda \cdot b)$; and
- finally, we rescale the result to the original units, getting $R_\lambda(*)$ defined as

$$a R_\lambda(*) b \stackrel{\text{def}}{=} \lambda^{-1} \cdot ((\lambda \cdot a) * (\lambda \cdot b)).$$

It therefore makes sense to require that if $* \preceq *'$, then for every $\lambda > 0$, we get $R_\lambda(*) \preceq R_\lambda(*')$.

Definition 12 We say that an optimality criterion on the set of all combination operations is *scale-invariant* if, for all $\lambda > 0$, $* \preceq *'$ implies $R_\lambda(*) \preceq R_\lambda(*')$, where $a R_\lambda(*) b \stackrel{\text{def}}{=} \lambda^{-1} \cdot ((\lambda \cdot a) * (\lambda \cdot b))$.

Shift-Invariance The numerical values also change if we change the starting point for measurements. For example, when measuring intensity, we can measure the actual intensity of an image, or we can take into account that there is always some noise $a_0 > 0$ and use the noise-only level a_0 as the new starting point. In this case, instead of each original value a, we get a new numerical value $a' = a - a_0$ for describing the same physical quantity.

If we apply the combination operation in the new units, then, in the old units, we get a slightly different result; namely,

- first, we transform the values a and b into the new units, so we get $a' = a - a_0$ and $b' = b - a_0$;
- then, we combine the new numerical values, getting $(a - a_0) * (b - a_0)$; and
- finally, we rescale the result to the original units, getting $S_{a_0}(*)$ defined as

$$a S_{a_0}(*) b \stackrel{\text{def}}{=} (a - a_0) * (b - a_0) + a_0.$$

It makes sense to require that the preference relation should not change if we simply change the starting point. So if $* \preceq *'$, then, for every a_0, we get $S_{a_0}(*) \preceq S_{a_0}(*')$.

Definition 13 We say that an optimality criterion is *shift-invariant* if, for all a_0, $* \preceq *'$ implies $S_{a_0}(*) \preceq S_{a_0}(*')$, where $a S_{a_0}(*) b \stackrel{\text{def}}{=} ((a - a_0) * (b - a_0)) + a_0$.

Weak Version of Shift-Invariance Alternatively, we can have a weaker version of this "shift-invariance" if we require that shifts in a and b imply a possibly different shift in $a * b$, i.e., if we shift both a and b by a_0, then the value $a * b$ is shifted by some value $f(a_0)$, which is, in general, different from a_0.

Definition 14 We say that an optimality criterion is *weakly shift-invariant* if, for every a_0, there exists a value $f(a_0)$ such that $* \preceq *'$ implies $W_{a_0}(*) \preceq W_{a_0}(*')$, where

$$a W_{a_0}(*) b \stackrel{\text{def}}{=} ((a - a_0) * (b - a_0)) + f(a_0).$$

Now, we are ready to formulation our results.

Proposition 4 *For every final, scale- and shift-invariant optimality criterion, the optimal combination operation has one of the following two forms:* $a * b = \min(a, b)$ *or* $a * b = \max(a, b)$.

Discussion Since the max combination operation corresponds to max-pooling, *this result explains why max-pooling is empirically the best combination operation*.

Proposition 5 *For every final, scale-invariant and weakly shift-invariant optimality criterion, the optimal combination operation has one of the following four forms:* $a * b = 0$, $a * b = \min(a, b)$, $a * b = \max(a, b)$, *or* $a * b = a + b$.

Discussion Since the addition combination operation corresponds to average-based pooling, *this result explains why max-pooling and average-pooling are empirically the best combination operations*.

Historical Comment This result first appeared in [5].

5 Why Softmax

Formulation of the Problem We want to describe how the probability $p(a)$ of selecting an alternative a depends on the value $J(a)$) of the corresponding objective function. In other words, we want to find a non-decreasing function $f(x)$ for which the probability $p(a)$ is proportional to $f(J(a))$, i.e., for which $p(a) = c \cdot f(J(a))$. Since the probabilities add up to 1, we have $\sum\limits_{a \in A} p(a) = c \cdot \sum\limits_{a \in A} f(J(a)) = 1$; hence, $c = \dfrac{1}{\sum\limits_{a \in A} f(J(a))}$ and

$$p(a) = \frac{f(J(a))}{\sum\limits_{a' \in A} f(J(a'))}.$$

One can easily see that if we multiply all the values of the function $f(x)$ by the same constant, the probabilities remain the same. Since we are interested only in the probabilities, this means that instead of selecting a single function $f(x)$, we should select a *family* of functions $\{c \cdot f(x)\}_c$, where a function $f(x)$ is given, and c takes all possible positive values.

Definition 15 By a *family*, we mean a family of functions $\{c \cdot f(x)\}_c$, where $f(x)$ is a given non-decreasing function, and c takes all possible positive values.

What Are Natural Symmetries Here? The main purpose of selecting an objective function is to decide which alternative is better and which is worse. If we add a constant to all the values of the objective function, this does not change which ones are better and which ones are worse. Thus, it is reasonable to require that the optimality criterion on the class of all the families should not change if we simply add a constant x_0 to all the values x, i.e., replace each function $f(x)$ with $f(x + x_0)$.

Definition 16 For each x_0, by a x_0-*shift* of a family $\{c \cdot f(x)\}_c$, we mean the family $\{c \cdot f(x + x_0)\}_c$.

Proposition 6 *For each final shift-invariant optimality criterion on the set of all families, the optimal family corresponds to* $f(x) = \exp(\beta \cdot x)$ *for some* $\beta \geq 0$.

Discussion This result explains why this particular version of softmax has been most empirically successful.

Historical Comment This result, in effect, appeared in [11, 12, 15].

6 Which Averaging Should We Choose

Discussion Averaging is similar to pooling, with the following main difference:

- in pooling, we combine the original measurement result, with possibly a lot of noise, so noise-related shifts make sense;
- in contrast, in averaging, we combine the results of processing, where the noise has been largely eliminated; so, shift no longer makes sense, only scaling makes sense.

What does make sense here is monotonicity: if we increase one or both numbers, the result should increase—or at least stay the same.

Definition 17 We say that a combination operation $a * b$ is *monotonic* if whenever $a \leq a'$ and $b \leq b'$, then $a * b \leq a' * b'$.

Proposition 7 *For every final scale-invariant optimality criterion on the set of all monotonic combination operations, the optimal combination operation has one of the following forms:* $a * b = 0$, $a * b = \min(a, b)$, $a * b = \max(a, b)$, *and* $a * b = (a^\alpha + b^\alpha)^{1/\alpha}$ *for some* α.

Discussion What are the "averaging" operations corresponding to these optimal combination operations?

For $a * b = 0$, the property $v_1 * \ldots * v_m = v * \ldots * v$ is satisfied for any possible v, so this combination operation does not lead to any "averaging" at all.

For $a * b = \min(a, b)$, the condition $v_1 * \ldots * v_m = v * \ldots * v$ leads to

$$v = \min(v_1, \ldots, v_m).$$

For $a * b = \max(a, b)$, the condition $v_1 * \ldots * v_m = v * \ldots * v$ leads to

$$v = \max(v_1, \ldots, v_m).$$

As we have mentioned, this "averaging" operation is actually sometimes used in deep learning – namely, in pooling.

Finally, for the combination operation $a * b = (a^\alpha + b^\alpha)^{1/\alpha}$, the condition $v_1 * \ldots * v_m = v * \ldots * v$ leads to

$$v = \left(\frac{v_1^\alpha + \ldots + v_m^\alpha}{m}\right)^{1/\alpha}.$$

For $\alpha = 1$, we get arithmetic average, and for $\alpha \to 0$, we get the geometric mean—the combination operation that turned out to be empirically the best for deep learning-related dropout training.

Indeed, in this case, the condition $v_1 * \ldots * v_m = v * \ldots * v$ takes the form

$$(v_1^\alpha + \ldots + v_m^\alpha)^{1/\alpha} = (v^\alpha + \ldots + v^\alpha)^{1/\alpha},$$

which is equivalent to

$$v_1^\alpha + \ldots + v_m^\alpha = m \cdot v^\alpha.$$

For every real value a, we have

$$a^\alpha = (\exp(\ln(a))^\alpha = \exp(\alpha \cdot \ln(a)).$$

For small x, $\exp(x) \approx 1 + x$, so $a^\alpha \approx 1 + \alpha \cdot \ln(a)$. Thus, the above condition leads to

$$(1 + \alpha \cdot \ln(v_1)) + \ldots + (1 + \alpha \cdot \ln(v_m)) = m \cdot (1 + \alpha \cdot \ln(v)),$$

i.e., to

$$m + \alpha \cdot (\ln(v_1) + \ldots + \ln(v_m)) = m + m \cdot \alpha \cdot \ln(v),$$

and thus to

$$\ln(v) = \frac{\ln(v_1) + \ldots + \ln(v_m)}{m} = \frac{\ln(v_1 \cdot \ldots \cdot v_m)}{m},$$

hence to $v = \sqrt[m]{v_1 \cdot \ldots \cdot v_m}$.

So, we indeed have a 1D family that contains combination operations efficiently used in deep learning:

- the arithmetic average that naturally comes from the use of the least-squares optimality criterion and
- the geometric mean, empirically the best combination operation for deep learning-related dropout training.

Historical Comment This result first appeared in [7]; it is based on a theorem proven in [2].

7 Proofs

Proof of Proposition 1 Let a_{opt} be the optimal alternative, and let g be any transformation from the group G. We want to prove that $g(a_{\text{opt}}) = a_{\text{opt}}$. To prove this equality, let us prove that the alternative $g(a_{\text{opt}})$ is optimal. Then, the desired equality will follow from the fact that the optimality criterion is final—and thus, there is only one optimal alternative.

To prove that the alternative $g(a_{\text{opt}})$ is optimal, we need to prove that $a \preceq g(a_{\text{opt}})$ for all a. Due to G-invariance of the optimality criterion, this condition is equivalent to $g^{-1}(a) \preceq a_{\text{opt}}$—which is, of course, always true, since a_{opt} is optimal. Thus, $g(a_{\text{opt}})$ is also optimal, and hence $g(a_{\text{opt}}) = a_{\text{opt}}$ for all $g \in G$. The proposition is proven.

Proof of Proposition 2 According to Proposition 1, the optimal activation function $s_0(x)$ should be scale-invariant. In other words, for all x and all $\lambda > 0$, we have $\lambda^{-1} \cdot s_0(\lambda \cdot x) = s_0(x)$, and thus

$$s_0(\lambda \cdot x) = \lambda \cdot s_0(x).$$

Let us show that this property leads to the desired form of the activation function.

Every input x is either equal to 0 or positive or negative. Let us consider these three cases one by one.

1°. Let us first consider the case of $x = 0$.

For $x = 0$ and $\lambda = 2$, scale-invariance means that if $y = s_0(0)$, then $2y = s_0(0)$. Thus, $2y = y$, and hence $y = s_0(0) = 0$.

2°. Let us now consider the case of positive values x.

Let us denote $c_+ \stackrel{\text{def}}{=} s_0(1)$. Then, by using scale-invariance with

- x instead of λ,
- 1 instead of x, and
- c_+ instead of $s_0(1)$,

we conclude that for all $x > 0$, we have $s_0(x) = x \cdot c_+$.

For positive values x, the desired equality is proven.

3°. To complete the proof of this result, we need to prove it for negative inputs x.

Let us denote $c_- \stackrel{\text{def}}{=} -s_0(-1)$. In this case, $s_0(-1) = -c$. Thus, for every $x < 0$, by using scale-invariance with

- $\lambda = |x|$,
- $x = -1$, and
- $s_0(-1) = -c_-$,

we conclude that

$$s_0(x) = s_0(|x| \cdot (-1)) = |x| \cdot s_0(-1) = |x| \cdot (-c_-) = c_- \cdot x.$$

The proposition is proven.

Proof That Every Transformation from a Finite-Dimensional Group Containing All Linear Transformations Is Fractionally Linear Every transformation is a composition of infinitesimal ones

$$x \to x + \varepsilon \cdot f(x),$$

for infinitely small ε. So, it is enough to consider infinitesimal transformations.

The class of the corresponding functions $f(x)$ is known as a *Lie algebra* A of the corresponding transformation group. Infinitesimal linear transformations correspond to

$$f(x) = a + b \cdot x,$$

so all linear functions are in A. In particular, $1 \in A$ and $x \in A$.

For any λ, the product $\varepsilon \cdot \lambda$ is also infinitesimal, so we get

$$x \to x + (\varepsilon \cdot \lambda) \cdot f(x) = x \to x + \varepsilon \cdot (\lambda \cdot f(x)).$$

So, if $f(x) \in A$, then $\lambda \cdot f(x) \in A$.

If we first apply $f(x)$, then, $g(x)$, we get

$$x \to (x + \varepsilon \cdot f(x)) + \varepsilon \cdot g(x + \varepsilon \cdot f(x)) =$$

$$x + \varepsilon \cdot (f(x) + g(x)) + o(\varepsilon).$$

Thus, if $f(x) \in A$ and $g(x) \in A$, then $f(x) + g(x) \in A$. So, A is a linear space.

In general, for the composition, we get

$$x \to (x + \varepsilon_1 \cdot f(x)) + \varepsilon_2 \cdot g(x_1 + \varepsilon_1 \cdot f(x)) =$$

$$x + \varepsilon_1 \cdot f(x) + \varepsilon_2 \cdot g(x) + \varepsilon_1 \cdot \varepsilon_2 \cdot g'(x) \cdot f(x) + \text{ quadratic terms}.$$

If we then apply the inverses to $x \to x + \varepsilon_1 \cdot f(x)$ and $x \to x + \varepsilon_2 \cdot g(x)$, the linear terms disappear, and we get

$$x \to x + \varepsilon_1 \cdot \varepsilon_2 \cdot \{f, g\}(x),$$

where

$$\{f, g\} \stackrel{\text{def}}{=} f'(x) \cdot g(x) - f(x) \cdot g'(x).$$

Thus, if $f(x) \in A$ and $g(x) \in A$, then $\{f, g\}(x) \in A$. The expression $\{f, g\}$ is known as the *Poisson bracket*.

Let us expand any function $f(x)$ in Taylor series:

$$f(x) = a_0 + a_1 \cdot x + \ldots$$

If k is the first non-zero term in this expansion, we get

$$f(x) = a_k \cdot x^k + a_{k+1} \cdot x^{k+1} + a_{k+2} \cdot x^{k+2} + \ldots$$

For every λ, the algebra A also contains

$$\lambda^{-k} \cdot f(\lambda \cdot x) = a_k \cdot x^k + \lambda \cdot a_{k+1} \cdot x^{k+1} + \lambda^2 \cdot a_{k+2} \cdot x^{k+2} + \ldots$$

In the limit $\lambda \to 0$, we get $a_k \cdot x^k \in A$, and hence $x^k \in A$.

Thus,

$$f(x) - a_k \cdot x^k = a_{k+1} \cdot x^{k+1} + \ldots \in A.$$

We can similarly conclude that A contains all the terms x^n for which $a_n \neq 0$ in the original Taylor expansion.

Since $g(x) = 1 \in A$, for each $f \in A$, we have

$$\{f, 1\} = f'(x) \cdot 1 + f(x) \cdot q' = f'(x) \in A.$$

Thus, for each k, if $x^k \in A$, we have

$$(x^k)' = k \cdot x^{k-1} \in A,$$

and hence $x^{k-1} \in A$, etc. So, if $x^k \in A$, all smaller powers are in A too. In particular, this means that if $x^k \in A$ for some $k \geq 3$, then we have $x^3 \in A$ and $x^2 \in A$; thus,

$$\{x^3, x^2\} = (x^3)' \cdot x^2 - x^3 \cdot (x^2)' = 3 \cdot x^2 \cdot x^2 - x^3 \cdot 2 \cdot x = x^4 \in A.$$

In general, once $x^k \in A$ for $k \geq 3$, we get

$$\{x^k, x^2\} = (x^k)' \cdot x^2 - x^k \cdot (x^2)' = k \cdot x^{k-1} \cdot x^2 - x^k \cdot 2 \cdot x =$$

$$(k-2) \cdot x^{k+1} \in A, \text{ hence } x^{k+1} \in A.$$

So, by induction, $x^k \in A$ for all k. Thus, A is infinite-dimensional—which contradicts to our assumption that A is finite-dimensional.

So, we cannot have Taylor terms of power $k \geq 3$; therefore, we have

$$x \to x + \varepsilon \cdot (a_0 + a_1 \cdot x + a_2 \cdot x^2).$$

This corresponds to an infinitesimal fractional linear transformation

$$x \to \frac{\varepsilon \cdot A + (1 + \varepsilon \cdot B) \cdot x}{1 + \varepsilon \cdot D \cdot x} =$$

$$(\varepsilon \cdot A + (1 + \varepsilon \cdot B) \cdot x) \cdot (1 - \varepsilon \cdot D \cdot x) + o(\varepsilon) =$$

$$x + \varepsilon \cdot (A + (B - D) \cdot x - D \cdot x^2).$$

So, to match, we need

$$A = a_0, \quad D = -a_2, \text{ and } B = a_1 - a_2.$$

We concluded that every infinitesimal transformation is fractionally linear. Every transformation is a composition of infinitesimal ones. Composition of fractional linear transformations is fractional linear. Thus, all transformations are fractional linear.

Proof of Proposition 3 Due to Proposition 1, we can conclude that the optimal family is also shift-invariance, i.e., for each c, the function $s_0(x - c)$ also belongs to the optimal family. Thus, we conclude that for every x, there exist values A, B, C, and D (which depend on c) for which

$$s_0(x - c) = \frac{A(c) \cdot s_0(x) + B(c)}{C(c) \cdot s_0(x) + D(c)}.$$

For $c = 0$, we get $A(0) = D(0) = 1$ and $B(0) = C(0) = 0$. Differentiating the above equation by c and taking $c = 0$, we get a differential equation for $s_0(x)$:

$$-\frac{ds_0}{dx} = (A'(0) \cdot s_0(x) + B'(0)) - s_0(x) \cdot (C'(0) \cdot s_0(x) + D'(0)).$$

So,

$$\frac{ds_0}{C'(0) \cdot s_0^2 + (A'(0) - C'(0)) \cdot s_0 + B'(0)} = -dx.$$

Integrating and taking into account that the activation function must be non-decreasing, we indeed get the desired expressions (after an appropriate linear rescaling of $s_0(x)$). The proposition is proven.

Proof of Proposition 4

1°. Let $a * b$ be the optimal combination operation. Due to Proposition 1, this operation is itself scale-invariant and shift-invariant. Let us prove that it has one of the two forms described in the formulation of the Proposition.

For every pair (a, b), we can have three different cases: $a = b$, $a < b$, and $a > b$. Let us consider them one by one.

2°. Let us first consider the case when $a = b$.

Let us denote $v \stackrel{\text{def}}{=} 1 * 1$. From scale-invariance with $\lambda = 2$ and from $1 * 1 = v$, we get $2 * 2 = 2v$. From shift-invariance with $s = 1$ and from $1 * 1 = v$, we get $2 * 2 = v + 1$. Thus, $2v = v + 1$, and hence $v = 1$, and $1 * 1 = 1$.

For $a > 0$, by applying scale-invariance with $\lambda = a$ to the formula $1 * 1 = 1$, we get $a * a = a$.

For $a = 0$, if we denote $c \stackrel{\text{def}}{=} 0 * 0$, then, by applying shift-invariance with $s = 1$ to $0 * 0 = c$, we get $1 * 1 = c + 1$. Since we already know that $1 * 1 = 1$, this means that $c + 1 = 1$ and thus, that $c = 0$, i.e., $0 * 0 = 0$.

So, for all $a \geq 0$, we have $a * a = a$. In this case, $\min(a, a) = \max(a, a) = a$, so we have $a * a = \min(a, a)$ and $a * a = \max(a, a)$.

3°. Let us consider the case when $a < b$. In this case, $b - a > 0$.

Let us denote $t \stackrel{\text{def}}{=} 0 * 1$. By applying scale-invariance with $\lambda = b - a > 0$ to the formula $0 * 1 = t$, we conclude that

$$0 * (b - a) = (b - a) \cdot t. \tag{1}$$

Now, by applying shift-invariance with $s = a$ to the formula (1), we conclude that

$$a * b = (b - a) \cdot t + a. \tag{2}$$

To find possible values of t, let us take into account that the combination operation should be associative. This means, in particular, that for all possible triples a, b, and c for which we have $a < b < c$, we must have

$$a * (b * c) = (a * b) * c. \tag{3}$$

Since $b < c$, by the formula (2), we have

$$b * c = (c - b) * t + b.$$

Since $t \geq 0$, we have $b * c \geq b$, and thus $a < b * c$. So, to compute $a * (b * c)$, we can also use the formula (2) and get

$$a*(b*c) = (b*c-a)\cdot t+a = ((c-b)\cdot t+b)\cdot t+a = c\cdot t^2+b\cdot(t-t^2)+a. \tag{4}$$

Let us restrict ourselves to the case when $a * b < c$. In this case, the general formula (2) implies that

$$(a * b) * c = (c - a * b) \cdot t + a * b = (c - ((b - a) \cdot t + a)) \cdot t + (b - a) \cdot t + a,$$

so

$$(a * b) * c = c \cdot t + b \cdot (t - t^2) + a \cdot (1 - t)^2. \tag{5}$$

Due to associativity, formulas (4) and (5) must coincide for all a, b, and c for which $a < b < c$ and $c > a * b$. Since these two linear expressions must be equal for all sufficiently large values of c, the coefficients at c must be equal, i.e., we must have $t = t^2$. From $t = t^2$, we conclude that $t - t^2 = t \cdot (1 - t) = 0$, so either $t = 0$ or $1 - t = 0$ (in which case $t = 1$).

If $t = 0$, then the formula (2) has the form $a * b = a$, i.e., since $a < b$, the form $a * b = \min(a, b)$.

If $t = 1$, then the formula (2) has the form $a * b = (b - a) + a = b$, i.e., since $a < b$, the form $a * b = \max(a, b)$.

4°. If $a > b$, then, by commutativity, we have $a * b = b * a$, where now $b < a$. So,

- if $t = 0$, then, due to Part 3 of this proof, we have $b * a = \min(b, a)$; since $a * b = b * a$ and since clearly $\min(a, b) = \min(b, a)$, we can conclude that $a * b = \min(a, b)$ for $a > b$ as well;
- if $t = 1$, then, due to Part 3 of this proof, we have $b * a = \max(b, a)$; since $a * b = b * a$ and since clearly $\max(a, b) = \max(b, a)$, we can conclude that $a * b = \max(a, b)$ for $a > b$ as well.

So, we have either $a * b = \min(a, b)$ for all a and b or $a * b = \max(a, b)$ for all a and b. The proposition is proven.

Proof of Proposition 5

1°. Let $a * b$ be the optimal combination operation. Due to Proposition 1, this operation is scale-invariant and weakly shift-invariant—which means that $a * b = c$ implies $(a + s) * (b + s) = c + f(s)$. Let us prove that the optimal operation $*$ has one of the above four forms.

1°. Let us first prove that $0 * 0 = 0$.

Indeed, let s denote $0 * 0$. Due to scale-invariance, $0 * 0 = s$ implies that $(2 \cdot 0) * (2 \cdot 0) = 2s$, i.e., $0 * 0 = 2s$. So, we have $s = 2s$, and hence $s = 0$ and $0 * 0 = 0$.

2°. Similarly, if we denote $v \stackrel{\text{def}}{=} 1 * 1$, then, due to scale-invariance with $\lambda = a$, $1 * 1 = v$ implies that $a * a = v \cdot a$ for all a.

On the other hand, due to weak shift-invariance with $a_0 = a$, $0 * 0 = 0$ implies that $a * a = f(a)$. Thus, we conclude that $f(a) = v \cdot a$.

2°. Let us now consider the case when $a < b$ and, thus, $b - a > 0$.

Let us denote $t \stackrel{\text{def}}{=} 0 * 1$. From scale-invariance with $\lambda = b - a$ and from $0 * 1 = t \geq 0$, we get $0 * (b - a) = t \cdot (b - a)$. From weak shift-invariance with $a_0 = a$, we get $a * b = t \cdot (b - a) + v \cdot a$, i.e.,

$$a * b = t \cdot b + (v - t) \cdot a. \tag{6}$$

Similarly to the proof of Proposition 1, to find possible values of t, let us take into account that the combination operation should be associative. This means, in particular, that for all possible triples a, b, and c for which we have $a < b < c$, we must have

$$a * (b * c) = (a * b) * c.$$

Since $b < c$, by the formula (6), we have

$$b * c = t \cdot c + (v - t) \cdot b.$$

3°. We know that $t \geq 0$. This means that we have either $t > 0$ or $t = 0$.

4°. Let us first consider the case when $t > 0$. In this case, for sufficiently large c, we have $b * c > a$. So, by applying the formula (6) to a and $b * c$, we conclude that

$$a * (b * c) = t \cdot (b * c) + (v - t) \cdot a = t^2 \cdot c + t \cdot (v - t) \cdot b + (v - t) \cdot a. \quad (7)$$

For sufficient large c, we also have $a * b < c$. In this case, the general formula (6) implies that

$$(a * b) * c = (t \cdot b + (v - t) \cdot a) * c = t \cdot c + t \cdot (v - t) \cdot b + (v - t)^2 \cdot a. \quad (8)$$

Due to associativity, formulas (7) and (8) must coincide for all a, b, and c for which $a < b < c$, $c > a * b$, and $b * c > a$. Since these two linear expressions must be equal for all sufficiently large values of c, the coefficients at c must be equal, i.e., we must have $t = t^2$.

From $t = t^2$, we conclude that $t - t^2 = t \cdot (1 - t) = 0$. Since we assumed that $t > 0$, we must have $t - 1 = 0$, i.e., $t = 1$.

The coefficients at a must also coincide, so we must have $v - t = (v - t)^2$; hence, either $v - t = 0$ or $v - t = 1$. In the first case, the formula (6) becomes $a * b = b$, i.e., $a * b = \max(a, b)$ for all $a \leq b$. Since the operation $*$ is commutative, this equality is also true for $b \leq a$ and is, thus, true for all a and b.

In the second case, the formula (6) becomes $a * b = a + b$ for all $a \leq b$. Due to commutativity, this formula holds for all a and b.

5°. Let us now consider the case when $t = 0$. In this case, the formula (6) takes the form $a * b = (v - t) \cdot a$.

Here, $a * b \geq 0$, and thus $v - t \geq 0$. If $v - t = 0$, this implies that $a * b = 0$ for all $a \leq b$ and thus, due to commutativity, for all a and b.

Let us now consider the remaining case when $v - t > 0$. In this case, if $a < b < c$, then, for sufficiently large c, we have $a * b < c$, and hence

$$(a * b) * c = (v - t) \cdot (a * b) = (v - t) \cdot ((v - t) \cdot a) = (v - t)^2 \cdot a.$$

On the other hand, here, $b * c = (v - t) \cdot b$. So, for sufficiently large b, we have $(v - t) \cdot b > a$, and thus

$$a * (b * c) = (v - t) \cdot a.$$

Due to associativity, we have $(v-t)^2 \cdot a = (v-t) \cdot a$, and hence $(v-t)^2 = v-t$ and, since $v - t > 0$, we have $v - t = 1$. In this case, the formula (6) takes the form $a * b = a = \min(a, b)$ for all $a \le b$. Thus, due to commutativity, we have $a * b = \min(a, b)$ for all a and b.

We have thus shown that the combination operation indeed has one of the four forms. Proposition 5 is therefore proven.

Proof of Proposition 6 Due to Proposition 1, the optimal family should be shift-invariant. This means, in particular, that if we take the function $f(x)$ from the original optimal family and shift it, then the resulting function $f(x+x_0)$ should also belong to the same optimal family, i.e., we should have $f(x + x_0) = c(x_0) \cdot f(x)$ for some c depending on x_0.

It is known that the only non-decreasing solutions to this functional equation are functions $f(x) = \text{const} \cdot \exp(\beta \cdot x)$ for some $\beta > 0$; see, e.g., [1]. The proposition is proven.

Proof of Proposition 7 Due to Proposition 1, the optimal combination operation should be scale-invariant, i.e., we should have $(\lambda \cdot a) * (\lambda \cdot b) = \lambda \cdot (a * b)$ for all $\lambda > 0$, a, and b. To avoid confusion, let us denote $a * b$ by $f(a, b)$.

1°. Depending on whether the value $f(1, 1)$ is equal to 1 or not, we have two possible cases: $f(1, 1) = 1$ and $f(1, 1) \ne 1$. Let us consider these two cases one by one.

2°. Let us first consider the case when $f(1, 1) = 1$. In this case, the value $f(0, 1)$ can be either equal to 0 or different from 0. Let us consider both subcases.

2.1°. Let us first consider the first subcase, when $f(0, 1) = 0$.

In this case, for every $b > 0$, scale-invariance with $\lambda = b$ implies that

$$f(b \cdot 0, b \cdot 1) = b \cdot 0,$$

i.e., $f(0, b) = 0$. By taking $b \to 0$ and using continuity, we also get $f(0, 0) = 0$. Thus, $f(0, b) = 0$ for all b.

By commutativity, we have $f(a, 0) = 0$ for all a. So, to fully describe the operation $f(a, b)$, it is sufficient to consider the cases when $a > 0$ and $b > 0$.

2.1.1°. Let us prove, by contradiction, that in this subcase, we have $f(1, a) \le 1$ for all a.

Indeed, let us assume that for some a, we have $b \stackrel{\text{def}}{=} f(1, a) > 1$. Then, due to associativity and $f(1, 1) = 1$, we have $f(1, b) = f(1, f(1, a)) = f(f(1, 1), a) = f(1, a) = b$.

Due to scale-invariance with $\lambda = b$, the equality $f(1, b) = b$ implies that $f(b, b^2) = b^2$. Thus, $f(1, b^2) = f(1, f(b, b^2)) = f(f(1, b), b^2) = f(b, b^2) = b^2$.

Similarly, from $f(1, b^2) = b^2$, we conclude that for $b^4 = (b^2)^2$, we have $f(1, b^4) = b^4$ and, in general, that $f(1, b^{2^n}) = b^{2^n}$ for every n.

Scale-invariance with $\lambda = b^{-2^n}$ implies that $f(b^{-2^n}, 1) = 1$. In the limit $n \to \infty$, we get $f(0, 1) = 1$, which contradicts to our assumption that $f(0, 1) = 0$. This contradiction shows that, indeed, $f(1, a) \le 1$.

2.1.2°. For $a \ge 1$, monotonicity implies $1 = f(1, 1) \le f(1, a)$, so $f(1, a) \le 1$ implies that $f(1, a) = 1$.

Now, for any a' and b' for which $0 < a' \le b'$, if we denote $r \stackrel{\text{def}}{=} \dfrac{b'}{a'} \ge 1$, then scale-invariance with $\lambda = a'$ implies that $a' \cdot f(1, r) = f(a' \cdot 1, a' \cdot r) = f(a', b')$. Here, $f(1, r) = 1$, and thus $f(a', b') = a' \cdot 1 = a'$, i.e., $f(a', b') = \min(a', b')$. Due to commutativity, the same formula also holds when $a' \ge b'$. So, in this case, $f(a, b) = \min(a, b)$ for all a and b.

2.2°. Let us now consider the second subcase of the first case, when $f(0, 1) > 0$.

2.2.1°. Let us first show that in this subcase, we have $f(0, 0) = 0$.

Indeed, scale-invariance with $\lambda = 2$ implies that from $f(0, 0) = a$, we can conclude that $f(2 \cdot 0, 2 \cdot 0) = f(0, 0) = 2 \cdot a$. Thus, $a = 2 \cdot a$, and hence $a = 0$. The statement is proven.

2.2.2°. Let us now prove that in this subcase, $f(0, 1) = 1$.

Indeed, in this case, for $a \stackrel{\text{def}}{=} f(0, 1)$, we have, due to $f(0, 0) = 0$ and associativity, that $f(0, a) = f(0, f(0, 1)) = f(f(0, 0), 1) = f(0, 1) = a$. Here, $a > 0$, so by applying scale-invariance with $\lambda = a^{-1}$, we conclude that $f(0, 1) = 1$.

2.2.3°. Let us now prove that for every $a \le b$, we have $f(a, b) = b$. So, due to commutativity, we have $f(a, b) = \max(a, b)$ for all a and b.

Indeed, from $f(1, 1) = 1$ and $f(0, 1) = 1$, due to scale-invariance with $\lambda = b$, we conclude that $f(0, b) = b$ and $f(b, b) = b$. Due to monotonicity, $0 \le a \le b$ implies that $b = f(0, b) \le f(a, b) \le f(b, b) = b$, and thus $f(a, b) = b$. The statement is proven.

3°. Let us now consider the remaining case when $f(1, 1) \ne 1$.

3.1°. Let us denote $v(k) \stackrel{\text{def}}{=} f(1, f(\ldots, 1)\ldots)$ (k times). Then, due to associativity, for every m and n, the value $v(m \cdot n) = f(1, f(\ldots, 1)\ldots)$ ($m \cdot n$ times) can be represented as

$$f(f(1, f(\ldots, 1)\ldots), \ldots, f(1, f(\ldots, 1)\ldots)),$$

where we divide the 1s into m groups with n 1s in each. For each group, we have $f(1, f(\ldots, 1)\ldots) = v(n)$. Thus, $v(m \cdot n) = f(v(n), f(\ldots, v(n))\ldots)$ (m times).

We know that $f(1, f(\ldots, 1)\ldots)$ (m times) $= v(m)$. Thus, by using scale-invariance with $\lambda = v(n)$, we conclude that $v(m \cdot n) = v(m) \cdot v(n)$, i.e., the function $v(n)$ is multiplicative. In particular, this means that for every number p and for every positive integer n, we have $v(p^n) = (v(p))^n$.

3.2°. If $v(2) = f(1,1) > 1$, then by monotonicity, we get $v(3) = f(1, v(2)) \geq f(1,1) = v(2)$ and, in general, $v(n+1) \geq v(n)$. Thus, in this case, the sequence $v(n)$ is (non-strictly) increasing.

Similarly, if $v(2) = f(1,1) < 1$, then we get $v(3) \leq v(2)$ and, in general, $v(n+1) \leq v(n)$, i.e., in this case, the sequence $v(n)$ is strictly decreasing.

Let us consider these two cases one by one.

3.2.1°. Let us first consider the case when the sequence $v(n)$ is increasing. In this case, for every three integers m, n, and p, if $2^m \leq p^n$, then $v(2^m) \leq v(p^n)$, i.e., $(v(2))^m \leq (v(p))^n$.

For all m, n, and p, the inequality $2^m \leq p^n$ is equivalent to $m \cdot \ln(2) \leq n \cdot \ln(p)$, i.e., to $\dfrac{m}{n} \leq \dfrac{\ln(p)}{\ln(2)}$. Similarly, the inequality $(v(2))^m \geq (v(p))^n$ is equivalent to $\dfrac{m}{n} \leq \dfrac{\ln(v(p))}{\ln(v(2))}$. Thus, the above conclusion "if $2^m \leq p^n$ then $(v(2))^m \leq (v(p))^n$" takes the following form:

$$\text{for every rational number } \frac{m}{n}, \text{ if } \frac{m}{n} \leq \frac{\ln(p)}{\ln(2)}, \text{ then } \frac{m}{n} \leq \frac{\ln(v(p))}{\ln(v(2))}.$$

Similarly, for all m', n', and p, if $p^{n'} \leq 2^{m'}$, then $v(p^{n'}) \leq v(2^{m'})$, i.e., $(v(p))^{n'} \leq (v(2))^{m'}$. The inequality $p^{n'} \leq 2^{m'}$ is equivalent to $n' \cdot \ln(p) \leq m' \cdot \ln(2)$, i.e., to $\dfrac{\ln(p)}{\ln(2)} \leq \dfrac{m'}{n'}$. Also, the inequality $(v(p))^{n'} \leq (v(2))^{m'}$ is equivalent to $\dfrac{\ln(v(p))}{\ln(v(2))} \leq \dfrac{m'}{n'}$. Thus, the conclusion "if $p^{n'} \leq 2^{m'}$ then $(v(p))^{n'} \leq (v(2))^{m'}$" takes the following form:

$$\text{for every rational number } \frac{m'}{n'}, \text{ if } \frac{\ln(p)}{\ln(2)} \leq \frac{m'}{n'} \text{ then } \frac{\ln(v(p))}{\ln(v(2))} \leq \frac{m'}{n'}.$$

Let us denote $\gamma \stackrel{\text{def}}{=} \dfrac{\ln(p)}{\ln(2)}$ and $\beta \stackrel{\text{def}}{=} \dfrac{\ln(v(p))}{\ln(v(2))}$. For every $\varepsilon > 0$, there exist rational numbers $\dfrac{m}{n}$ and $\dfrac{m'}{n'}$ for which $\gamma - \varepsilon \leq \dfrac{m}{n} \leq \gamma \leq \dfrac{m'}{n'} \leq \gamma + \varepsilon$. For these numbers, the above two properties imply that $\dfrac{m}{n} \leq \beta$ and $\beta \leq \dfrac{m'}{n'}$ and, thus, that $\gamma - \varepsilon \leq \beta \leq \gamma + \varepsilon$, i.e., $|\gamma - \beta| \leq \varepsilon$. This is true for all $\varepsilon > 0$, so we conclude that $\beta = \gamma$, i.e., $\dfrac{\ln(v(p))}{\ln(v(2))} = \gamma$. Hence, $\ln(v(p)) = \gamma \cdot \ln(p)$, and thus $v(p) = p^\gamma$ for all integers p.

3.2.2°. We can reach a similar conclusion $v(p) = p^\gamma$ when the sequence $v(n)$ is decreasing and $v(2) < 1$ and a conclusion that $v(p) = 0$ if $v(2) = 0$.

3.3°. By definition of $v(n)$, we have $f(v(m), v(m')) = v(m+m')$. Thus, we have $f(m^\gamma, (m')^\gamma) = (m+m')^\gamma$. By using scale-invariance with $\lambda = n^{-\gamma}$, we get $f\left(\frac{m^\gamma}{n^\gamma}, \frac{(m')^\gamma}{n^\gamma}\right) = \frac{(m+m')^\gamma}{n^\gamma}$. Thus, for $a = \frac{m^\gamma}{n^\gamma}$ and $b = \frac{(m')^\gamma}{n^\gamma}$, we get $f(a,b) = (a^\alpha + b^\alpha)^{1/\alpha}$, where $\alpha \stackrel{\text{def}}{=} 1/\gamma$.

Rational numbers $r = \frac{m}{n}$ are everywhere dense on the real line, and hence the values r^γ are also everywhere dense, i.e., every real number can be approximated, with any given accuracy, by such numbers. Thus, continuity implies that $f(a,b) = (a^\alpha + b^\alpha)^{1/\alpha}$ for every two real numbers a and b.

The proposition is proven.

Acknowledgments This work was supported in part by the National Science Foundation via grants 1623190 (A Model of Change for Preparing a New Generation for Professional Practice in Computer Science) and HRD-1242122 (Cyber-ShARE Center of Excellence).

References

1. Aczel, J., Dhombres, J.: Functional Equations in Several Variables. Cambridge University Press, New York (2008)
2. Autchariyapanitkul, K., Kosheleva, O., Kreinovich, V., Sriboonchitta, S.: Quantum econometrics: how to explain its quantitative successes and how the resulting formulas are related to scale invariance, entropy, and fuzziness. In: Huynh, V.-N., Inuiguchi, M., Tran, D.-H., Denoeux, Th. (eds.) Proceedings of the International Symposium on Integrated Uncertainty in Knowledge Modelling and Decision Making IUKM'2018, Hanoi, Vietnam, March 13–15, 2018
3. Baral, C., Fuentes, O., Kreinovich, V.: Why deep neural networks: a possible theoretical explanation. In: Ceberio, M., Kreinovich, V. (eds.) Constraint Programming and Decision Making: Theory and Applications, pp. 1–6. Springer Verlag, Berlin (2018)
4. Bishop, C.M.: Pattern Recognition and Machine Learning. Springer, New York (2006)
5. Farhan, A., Kosheleva, O., Kreinovich, V.: Why max and average poolings are optimal in convolutional neural networks. In: Proceedings of the Seventh International Symposium on Integrated Uncertainty in Knowledge Modelling and Decision Making IUKM'2019, Nara, Japan, March 27–29 (2019)
6. Fuentes, O., Parra, J., Anthony, E., Kreinovich, V.: Why rectified linear neurons are efficient: a possible theoretical explanations. In: Kosheleva, O., Shary, S., Xiang, G., Zapatrin, R. (eds.) Beyond Traditional Probabilistic Data Processing Techniques: Interval, Fuzzy, etc. Methods and Their Applications, Springer, Cham (2019)
7. Gholamy, A., Parra, J., Kreinovich, V., Fuentes, O., Anthony, E.: How to best apply deep neural networks in geosciences: towards optimal 'averaging' in dropout training. In: Watada, J., Tan, S.C., Vasant, P., Padmanabhan, E., Jain, L.C. (eds.). Smart Unconventional Modelling, Simulation and Optimization for Geosciences and Petroleum Engineering, pp. 15–26. Springer, Berlin (2019)
8. Goodfellow, I., Bengio, Y., Courville, A.: Deep Leaning. MIT Press, Cambridge (2016)
9. Kainen, P.C., Kurkova, V., Kreinovich, V., Sirisaengtaksin, O.: Uniqueness of network parameterization and faster learning. Neural Parallel Sci. Comput. **2**, 459–466 (1994)
10. Kosheleva, O., Kreinovich, V.: Why deep learning methods use KL divergence instead of least squares: a possible pedagogical explanation. Math. Struct. Model. **46**, 102–106 (2018)

11. Kreinovich, V.: Group-theoretic approach to intractable problems. Lecture Notes in Computer Science. Springer, Berlin, vol. 417, pp. 112–121 (1990)
12. Kreinovich, V.: From traditional neural networks to deep learning: towards mathematical foundations of empirical successes. In: Shahbazova, S.N., Kacprzyk, J., Balas, V.E., Kreinovich, V. (eds.) Proceedings of the World Conference on Soft Computing, Baku, Azerbaijan, May 29–31 (2018)
13. Kreinovich, V., Quintana, C.: Neural networks: what non-linearity to choose? In: Proceedings of the Fourth University of New Brunswick Artificial Intelligence Workshop, pp. 627–637. Fredericton, New Brunswick (1991)
14. Muela, G., Servin, C., Kreinovich, V.: How to make machine learning robust against adversarial inputs. Math. Struct. Model. **42**, 127–130 (2017)
15. Nguyen, H.T., Kreinovich, V.: Applications of Continuous Mathematics to Computer Science. Kluwer, Dordrecht (1997)
16. Parra, J., Fuentes, O., Anthony, E., Kreinovich, V.: Prediction of volcanic eruptions: case study of rare events in chaotic systems with delay. In: Proceedings of the IEEE Conference on Systems, Man, and Cybernetics SMC'2017, Banff, Canada, October 5–8, pp. 351–356 (2017)
17. Parra, J., Fuentes, O., Anthony, E., Kreinovich, V.: Use of machine learning to analyze and—hopefully—predict volcano activity. Acta Politech. Hung. **14**(3), 209–221 (2017)
18. Sirisaengtaksin, O., Kreinovich, V., Nguyen, H.T.: Sigmoid neurons are the safest against additive errors. In: Proceedings of the First International Conference on Neural, Parallel, and Scientific Computations, Atlanta, GA, May 28–31, vol. 1, pp. 419–423 (1995)
19. Wiener, N.: Cybernetics: Or Control and Communication in the Animal and the Machine. MIT Press, Cambridge (1948)

Variable Neighborhood Programming as a Tool of Machine Learning



Nenad Mladenovic, Bassem Jarboui, Souhir Elleuch, Rustam Mussabayev, and Olga Rusetskaya

1 Introduction

In the fields of artificial intelligence (AI) and machine learning (ML), there is a fast-growing interest in developing techniques to solve problems automatically. The term automatic programming (AP) indicates that programs are automatically generated without human intervention. The solution to an AP problem is a program that is usually represented by the tree that we will call an AP tree. Such a tree has different types of nodes, i.e., some nodes present operations and some variables and constants. An illustrative example will be given later.

Genetic Programming (GP) is a well-known technique in AP. It is based on Genetic Algorithm (GA) operators (Koza in 1992 [52]), performed at AP tree. GP is used in many AI fields, such as symbolic regression [9, 24, 53], forecasting [6, 12], data mining [16], and classification [13]. There are many GP based methods that

N. Mladenovic (✉)
Department of Industrial and Systems Engineering, Research Center on Digital Supply Chain and Operations Management, Khalifa University, Abu Dhabi, UAE
e-mail: Nenad.mladenovic@ku.ac.ai

B. Jarboui
Higher Colleges of Technology, Abu Dhabi, UAE

S. Elleuch
Department of Management Information Systems, College of Business and Economics, Qassim University, Buraydah, Saudi Arabia

R. Mussabayev
Institute of Information and Computational Technologies, Almata, Kazakhstan

O. Rusetskaya
Leontief Centre, Saint Petersburg State University of Economics, Saint Petersburg, Russia
e-mail: olga@leontief.ru

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_9

include local searches [12, 57]. Good results are obtained by using such methods for solving the symbolic regression problem [57] and energy consumption forecasting problem [12]. In [70], Automatic Programming via Iterated Local Search (APRILS) is proposed, where the local search is presented by a selection of random solutions obtained by mutation.

Recently, a new AP method, called Variable Neighborhood Programming (VNP), was introduced by Elleuch et al. [19]. It is based on the Variable Neighborhood Search (VNS) metaheuristic [63] aimed to solving global optimization problems. VNP method is different from GP since it does not use selection, mutation, crossing-over, etc., operators, used in Genetic Algorithms and GP.

The topic of this chapter is a brief introduction to the VNP methodology, one possible tool in AI and ML. In the next section, we briefly give the basic ideas of VNS metaheuristic for solving global optimization problems and some of its variants that are after adapted for the VNP. In Sect. 3, we consider an application of VNP for solving the symbolic regression problem and compare the results with state-of-the-art techniques, such as GP. Section 4 includes the implementation of the VNP methodology to health care. We are searching for the formula of life expectancy of Russian citizens divided into 83 different geographical regions, as a function of 4 economic infrastructure parameters. For that purpose, we collected a large data set and divided it into training and testing groups [20]. Section 5 consists of another practical and important problem that can be solved by ML techniques: preventive maintenance planning of railway infrastructure. Our reduced VNP method is illustrated in the case study announced by the Rail Application Section within INFORMS.

2 Variable Neighborhood Search

Variable Neighborhood Search (VNS) is a metaheuristic proposed some 25 years ago [63]. It is based upon the idea of a systematic change of neighborhood both in a descent phase to find a local optimum and in a perturbation phase to get out of the corresponding valley. Originally designed for approximate solution of combinatorial optimization problems, it was extended to address mixed-integer programs, nonlinear programs, and mixed-integer nonlinear programs. In addition, VNS has been used as a tool for automated or computer-assisted graph theory and many other fields.

Applications are rapidly increasing in number and pertain to many fields: location theory, cluster analysis, scheduling, vehicle routing, network design, lot sizing, artificial intelligence, engineering, pooling problems, biology, phylogeny, reliability, geometry, telecommunication design, etc. References are too numerous to be listed here, but let us mention some recent ones [66, 72] and the VNS chapter in the handbook, where many applications can be found in [31].

A deterministic optimization problem may be formulated as

$$\min\{f(x)|x \in X, X \subseteq \mathcal{S}\}, \tag{1}$$

where $\mathcal{S}$, X, x, and f denote the *solution space*, the *feasible set*, a *feasible solution*, and a real-valued *objective function*, respectively. If $\mathcal{S}$ is a finite but large set, a *combinatorial optimization* problem is defined. If $\mathcal{S} = \mathbb{R}^n$, we refer to *continuous optimization*. A solution $x^* \in X$ is *optimal* if

$$f(x^*) \leq f(x), \ \forall x \in X.$$

An *exact algorithm* for problem (1), if one exists, finds an optimal solution x^*, together with the proof of its optimality, or shows that there is no feasible solution, i.e., $X = \emptyset$, or the solution is unbounded.

Let us denote $\mathcal{N}_k$, $(k = 1, \ldots, k_{\max})$, a finite set of neighborhood structures, and $\mathcal{N}_k(x)$ the set of solutions in the kth neighborhood of x. An *optimal solution* x_{opt} (or global minimum) is a feasible solution where a minimum is reached. We call $x' \in X$ a *local minimum* of (1) with respect to $\mathcal{N}_k$ (w.r.t. $\mathcal{N}_k$ for short), if there is no solution $x \in \mathcal{N}_k(x') \subseteq X$ such that $f(x) < f(x')$. Metaheuristics (based on local search procedures) try to continue the search by other means after finding the first local minimum. VNS is based on three simple facts:

Fact 1 A local minimum w.r.t. one neighborhood structure is not necessarily so for another. □

Fact 2 A global minimum is a local minimum w.r.t. all possible neighborhood structures. □

Fact 3 For many problems, local minima w.r.t. one or several $\mathcal{N}_k$ are relatively close to each other. □

Neighborhood Change We first examine in Algorithm 1 the solution move and neighborhood change function that will be used within a VNS framework. Function `NeighborhoodChange()` compares the incumbent value $f(x)$ with the new value $f(x')$ obtained from the kth neighborhood (line 1).

```
Function NeighborhoodChange (x, x', k)
1  if f(x') < f(x) then
2  |   x ← x' // Make a move
3  |   k ← 1 // Initial neighborhood
   else
4  |   k ← k + 1 // Next neighborhood

return x, k
```

Algorithm 1: Neighborhood change

If an improvement is obtained, the incumbent is updated (line 2) and k is returned to its initial value (line 3). Otherwise, the next neighborhood is considered (line 4).

In order to solve (1) by using several neighborhoods, facts 1 to 3 can be used in three different ways: (i) deterministic; (ii) stochastic; and (iii) both deterministic and stochastic. Below we discuss just the simplest variants, i.e., Reduced Variable Neighborhood Search and Basic Variable Neighborhood Search.

The **Reduced VNS** (RVNS) method is obtained when a random point is selected from $\mathcal{N}_k(x)$, and no descent is attempted from this point. Rather, the value of the new point is compared with that of the incumbent, and an update takes place in the case of improvement. We also assume that a stopping condition has been chosen such as the maximum CPU time allowed $t_{\max}$, or the maximum number of iterations between two improvements. To simplify the description of the algorithms, we always use $t_{\max}$ below. Therefore, RVNS (Algorithm 2) uses two parameters: $t_{\max}$ and $k_{\max}$.

Function RVNS(x, k_{max}, t_{max})
1 **repeat**
2 $k \leftarrow 1$
3 **repeat**
4 $x' \leftarrow$ Shake(x, k)
5 $x, k \leftarrow$ NeighborhoodChange (x, x', k)
until $k = k_{max}$
6 $t \leftarrow$ CpuTime ()
until $t > t_{max}$

return x

Algorithm 2: Reduced VNS

Shaking The function Shake in line 4 generates a point x' at random from the kth neighborhood of x, i.e., $x' \in \mathcal{N}_k(x)$. It is given in Algorithm 3, where it is assumed that the points from $\mathcal{N}_k(x)$ are numbered as $\{x^1, \ldots, x^{|\mathcal{N}_k(x)|}\}$.

Function Shake(x, k)
1 $w \leftarrow \lfloor 1+\text{Rand}(0, 1) \times |\mathcal{N}_k(x)| \rfloor$
2 $x' \leftarrow x^w$
return x'

Algorithm 3: Shaking function

RVNS is useful for very large instances for which local search is costly. It can be used as well for finding initial solutions for large problems before decomposition. It has been observed that the best value for the parameter k_{max} is often 2 or 3. In addition, a maximum number of iterations between two improvements is typically used as the stopping condition.

The **Basic VNS** (BVNS) method [63] combines deterministic and stochastic changes of the neighborhood. The deterministic part is represented by a local search heuristic (see Fig. 1).

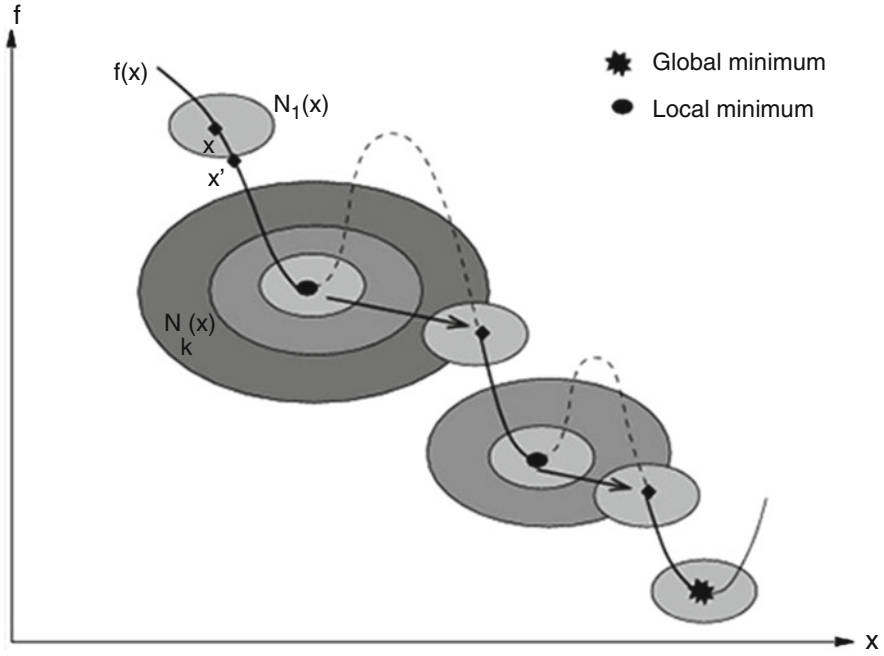


Fig. 1 Basic VNS

The stochastic phase of BVNS (see Algorithm 4) is represented by the random selection of a point x' from the kth neighborhood of the shake operation. Note that point x' is generated at random in Step 5 in order to avoid cycling, which might occur with a deterministic rule.

Function BVNS(x, k_{max}, t_{max})
1 $t \leftarrow 0$
2 **while** $t < t_{max}$ **do**
3 $k \leftarrow 1$
4 **repeat**
5 $x' \leftarrow$ Shake (x, k) // Shaking
6 $x'' \leftarrow$ LocalSearch(x') // Local search
7 $x, k \leftarrow$ NeighborhoodChange(x, x'', k) // Change neighborhood
until $k = k_{max}$
8 $t \leftarrow$ CpuTime ()
return x

Algorithm 4: Basic VNS

3 Variable Neighborhood Programming

Automatic programming (AP) is an area of artificial intelligence and machine learning. Its task consists in searching automatically for the best program or model of a given problem. AP methods evolve programs. A program can be presented in many ways: tree, line of codes, and a set of rules. In this paper, a program is coded with a directed tree structure. Several types of trees were adopted in the literature. According to Koza, a program is a hierarchical combination of functions and terminals of undefined shape and dynamically varying in size [52]. The first node of the tree is called the root. If a node is connected by another node, then it is called a parent node and the connected node is a child. Typically, a syntax tree is expressed in a prefix notation (Lisp) and it is composed of internal nodes including functions $(+, \cdot, \sin, \max, \log, if \ldots)$ and external nodes, also called terminals, containing problem variables and constants [53]. Let us introduce the following sets:

$F = \{f_1, f_2, \ldots, f_n\} = \{+, \cdot, \sin, \log, \max, \min, if, \ldots\}$ denote the set of operators, also called functional set, and let
$E = X \cup C$ denote the terminal set, where
$X = \{x_1, x_2, \ldots, x_{k'}\}$ is a set of the problem variables, and
$C = \{c_1, c_2, \ldots, c_{k''}\}$ is a set of constants;
$k = k' + k''$ denotes the cardinality of the set E.

We have to note that functional nodes can include binary and unary operations and an operator can be present in more than one node in the same tree. Functional and terminal sets are fixed according to the present problem. A tree can include the same terminal or functional value more than once. Therefore, a program or a solution is presented as a tree with different types of nodes. As we have mentioned, the individual program is presented as a tree for the first time in [52]. Such an AP tree differs from the classical tree on the network since AP tree has different types of nodes.

3.1 Solution Presentation

Example 1 We now present some basic facts of VNP metaheuristic using a simple example (for more details, see [19, 20]). Let us consider the following formula as a solution for some problems, whose quality we are able to evaluate

$$\max\left\{\frac{3}{x_1 + (-2)}; 1.8 - x_2\right\}. \tag{2}$$

In Fig. 2a, the current solution (5) is presented as a tree, with different node types.

There are nine nodes in the tree. Functional set is presented by four circular nodes ($n = 4$), $\{\max, /, -, +\}$; there are $k' = 2$ variables and $k'' = 3$ coefficients,

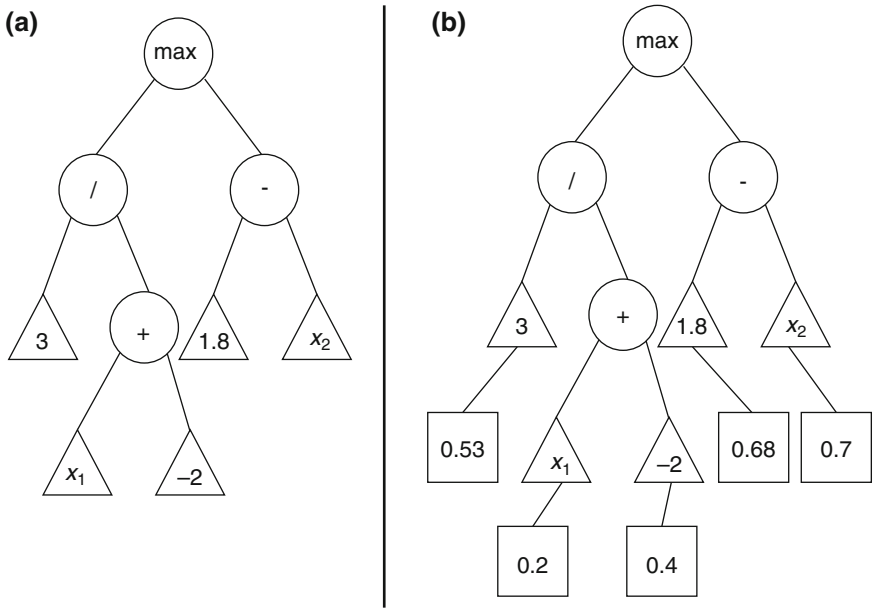


Fig. 2 (**a**) Solution presentation in GP; (**b**) Solution representation in VNP

and therefore, $k = 5$. They are presented as triangles in Fig. 2. The transformation of the tree to expression (5) is obtained by reading the tree from left to right (see, e.g., [53]).

VNP uses the presentation of the solution as a tree as well. However, a new extended solution presentation introduces the new parameter set

$$P = \{\alpha_1, \alpha_2, \ldots, \alpha_k\}.$$

Each parameter is assigned to each terminal (triangle) node of the tree. It allows the adjustment of the terminal set values by multiplying variables with the corresponding coefficients. This extension is proposed to overcome the non-flexibility problem of the traditional representation of the tree. In fact, in the old model, it could be seen as a special case of a new one, where all α_i coefficients are equal to 1. Therefore, an extended solution representation by adding coefficients to variables is proposed. In this way, a small correction of the current solution is allowed, rather than changing the sets F and E in searching for the better program or formula. Each terminal node is attached to its own parameter value. Their values are generally taken from the interval $[-1, 1]$. Note that this domain can be changed according to the additional experiments. However, our experience shows that the convergence speed is higher for the interval $[-1, 1]$. Thus, the addition of these coefficients may simplify the research process and can reduce the size of the resulted tree. Obviously,

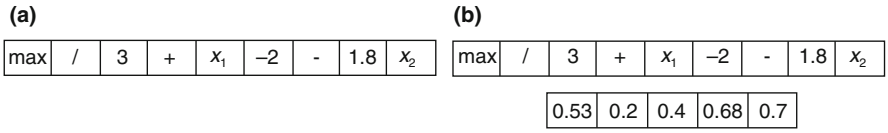


Fig. 3 Computer solution recording (**a**) GP and (**b**) VNP

the parameter set and the terminal set have the same cardinality k as set E. Figure 2b represents maximization problem (5) in a new way.

For the sake of clarity, in this section, we give this new solution presentation using the same example. Figure 2b illustrates the following expression:

$$\max\left\{\left(\frac{0.53\times 3}{0.2\times x_1+0.4\times(-2)}\right);0.68\times 1.8-0.7\times x_2\right\}, \tag{3}$$

where $F = \{\max, /, +, -\}$, $E = \{x_1, x_2\} \cup \{3, -2, 1.8\}$ and $P = \{0.53, 0.2, 0.4, 0.68, 0.7\}$. Expression (5) can also be written in a *prefix* notation [53]:

$$\max(/(3+(x_1-2))(-(1.8\,x_2)). \tag{4}$$

The tree structure of expression (5) can be presented in the computer memory as a list whose number of elements equals to the number of nodes (see Fig. 3a). The tree of expression (3) needs two lists to be saved in the computer, as given in (Fig. 3b). The second list is reserved to save coefficients. So, its size is equal to the number of terminal nodes.

3.2 Neighborhood Structures

In [19], a list of 9 possible neighborhood structures is proposed for solving AP problems by using VNP. In this paper, we use just 3 of them to solve the maintenance planning of railway infrastructure problems. Again, we will illustrate them in our example. In order to present more capabilities of the VNP approach, let us also assume that we are solving a symbolic regression problem, i.e., we are searching for the formula that approximates some data in the best possible way. In that case, we have to change variables, functional, and parameter sets.

1. **Changing a node value operator** (N_1). This neighborhood structure conserves the skeleton of the tree and changes only the values of a functional or a terminal node. Each node can obtain many values from its corresponding set. Let S be the current solution, and its neighbor S' differs from S by just a single node. Figure 4 shows a possible move within this neighborhood structure.

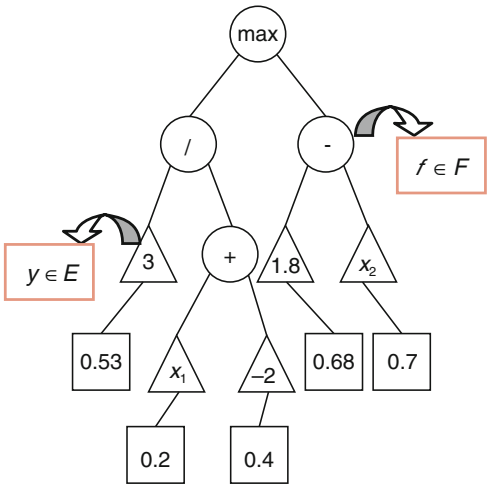


Fig. 4 Neighborhood structure N_1: changing a node value

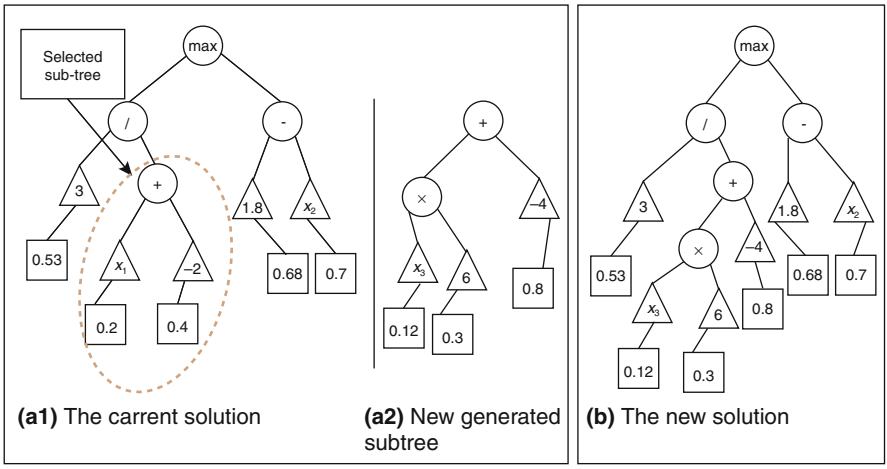


Fig. 5 Neighborhood structure N_2: swap

2. **Swap operator** (N_2). In this operator, first we choose a node from the current tree at random and we generate a new subtree, as presented in Fig. 5a1 and a2. Then, we attach it in the place of the subtree, corresponding to the selected node. In this move, we must respect the constraint related to the maximum tree size. More details are given in Fig. 5.
3. **Changing a parameter value operator** (N_3). In the previous neighborhood structures, we have just considered the tree form and its node values. Here, we focus on the parameter optimization. Therefore, we keep the position and the value of nodes to search the neighbors in the parametric space. Figure 6 illustrates the details. The change from one value to another is at random.

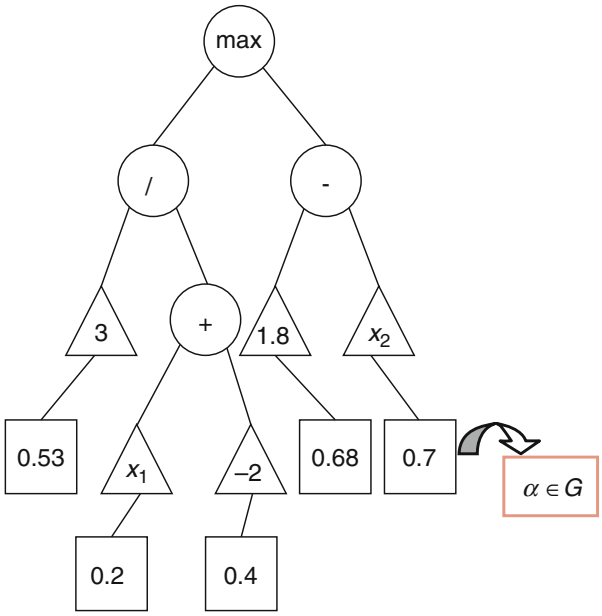


Fig. 6 Neighborhood structure N_3: changing a parameter value

3.3 Elementary Tree Transformation in Automatic Programming

Another neighborhood structure that could be used in AP tree is adaptation of the well-known neighborhood structure from the regular graph theory: Elementary Tree Transformation (ETT). We devote to it a separate subsection since we believe it could be used not only within VNP but also in other AP methods as GP. Its steps we will present again using an example.

Example 2 Figure 7 illustrates the expression of

$$\max\left\{\frac{3}{x_3 * 2.4}; x_1 - x_2\right\}, \tag{5}$$

which can be presented as an AP tree with the functional set $F = \{f_1, \ldots, f_4\} = \{\max, +, -, *\}$, and the terminal set $E = X \cup C$, where $X = \{x_1, x_2, x_3\}$ is the set of the problem variables, and $C = \{c_1, c_2\} = \{3, 2.4\}$ denotes the set of constants; $k = 3 + 2 = 5$ is the cardinality of E. This expression can also be written in a *prefix* notation [53]

$$\max(/(3 * (x_3 2.4))(-(x_1 x_2)).$$

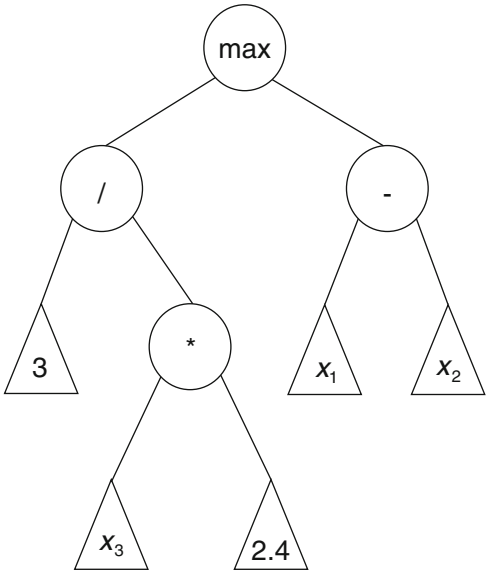


Fig. 7 Solution presentation as AP tree

This notation facilitates to extract the relationship between expressions and their corresponding trees.

3.3.1 ETT in the Tree of an Undirected Graph

ETT is a transformation technique of a tree extracted from an undirected graph $G(V, E)$, where V is a node set and E is an edge set. Let $T(V, A)$ denote any spanning tree of G, where $A \subseteq E$ is the edge set of T, ETT transforms a tree T into a tree T' in two steps ($T' = \text{ETT}(T)$) [64]:

1. add an edge a to T such that it belongs to E, but not to A ($a \in E \setminus A$);
2. detect the unit cycle obtained, and remove any edge (different to the new one) from that cycle to get a subgraph T'. The following obvious statement holds.

Proposition 1.1 *The resulting subgraph T', obtained from T by steps (1) and (2), is a spanning tree.*

All possible edge additions, followed by all possible removals from the corresponding cycles, constitute the neighborhood of T, i.e., the set of trees $N(T) = \{T' \mid T' = \text{ETT}(T)\}$. When the solution of a combinatorial optimization problem is a spanning tree having n nodes, the definition of a neighborhood $N(T)$ allows us to establish a local search procedure with respect to ETT neighborhood structure:

1. find a tree $T^* \in N(T)$ with the best value of the objective function;

2. if a better solution is obtained, then $T \leftarrow T^*$, and return to (1); otherwise, stop with T^* as the local optimum.

Proposition 1.2 *The cardinality of $N(T)$ is less than $\frac{n(n-1)(n-3)}{2}$.*

Proof If we assume that the initial graph is complete, then the number of possible edges to be added to T is $\frac{n(n-1)}{2} - n = \frac{n(n-3)}{2}$, since spanning tree has n edges. The largest cycle to remove an edge from has $n-1$ edges. After multiplying these two numbers to get maximum possible cardinality, we get the result from above. □

3.3.2 ETT in AP Tree

Here, we try to answer the question whether ETT is possible in the AP tree. First, we consider the example of the small tree illustrated in Fig. 8.

Let us first denote an AP tree rooted in r as $T(r, F, H, E, d)$, where F is the set of functional nodes, and H is the set of terminal nodes. The set E includes all

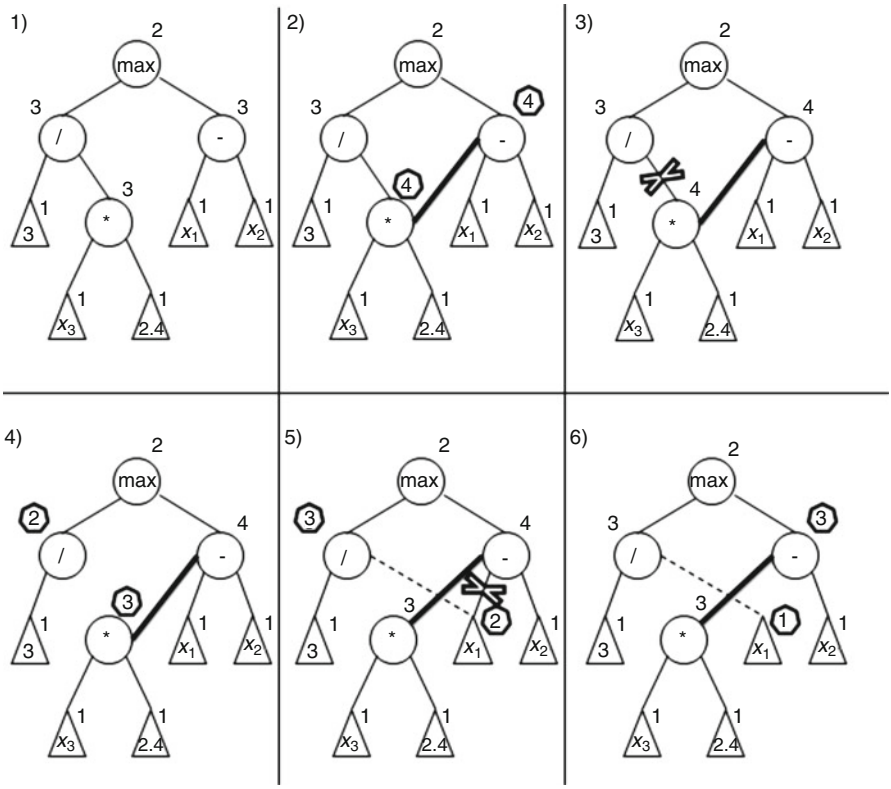


Fig. 8 An example of ETT move in AP tree: from the expression $T = \max\left\{\frac{3}{x_3 * 2.4}; x_1 - x_2\right\}$, the neighboring $T' = \max\left\{\frac{3}{x_1}; x_3 * 2.4 - x_2\right\}$ is obtained

edges of the tree T, and array $d = (d_1, \ldots, d_n)$ denotes the node degrees of T. The degree d_i gives the number of edges incident to the node i. For example, in a binary tree, the degree of a functional node is 3 if the functional node represents a binary operation; otherwise, for unary operation, it is 2. The degree of a terminal node is equal to 1 since it has no child. We first give pseudocode for the local search AP-ETT (see Algorithm 5) and then give the description of its steps.

1: $Terminate \leftarrow false$;
2: **for all** node $i \in F$ **do**
3: **for all** node $j \in F \cup V \setminus \{i, parent(i), child(i)\}$ **do**
4: $E' \leftarrow E \cup \{(i, j)\}$; // A cycle is obtained
5: $d_i \leftarrow d_i + 1$; $d_j \leftarrow d_j + 1$;
6: **if** $j \in F$ **then**
7: **// case 1**
8: $E' \leftarrow E' \setminus \{(x, b)\}\ /x \in \{i, j\},\ b \in \{parent(i), parent(j)\}$;
9: $d_b \leftarrow d_b - 1$;
10: **if** $x = i$ **then**
11: $d_i \leftarrow d_i - 1$; $E' \leftarrow E' \cup \{(b, child(j))\}$;
12: $d_b \leftarrow d_b + 1$; $d_{child(j)} \leftarrow d_{child(j)} + 1$;
13: $E' \leftarrow E' \setminus \{(j, child(j))\}$;
14: $d_j \leftarrow d_j - 1$; $d_{child(j)} \leftarrow d_{child(j)} - 1$;
15: **else**
16: $d_j \leftarrow d_j - 1$; $E' \leftarrow E' \cup \{(b, child(i))\}$;
17: $d_b \leftarrow d_b + 1$; $d_{child(i)} \leftarrow d_{child(i)} + 1$;
18: $E' \leftarrow E' \setminus \{(i, child(i))\}$;
19: $d_i \leftarrow d_i - 1$; $d_{child(i)} \leftarrow d_{child(i)} - 1$;
20: **end if**
21: **else**
22: **// case 2**
23: $b \leftarrow parent(j)$; $E \leftarrow E' \setminus \{(j, b\}$;
24: $d_j \leftarrow d_j - 1$; $d_b \leftarrow d_b - 1$;
25: $E' \leftarrow E' \cup \{(b, child(i))\}$;
26: $d_b \leftarrow d_b + 1$; $d_{child(i)} \leftarrow d_{child(i)} + 1$;
27: $E' \leftarrow E' \setminus \{(i, child(i))\}$;
28: $d_i \leftarrow d_i - 1$; $d_{child(i)} \leftarrow d_{child(i)} - 1$;
29: **end if**
30: $T' \leftarrow T(r, F, H, E', d)$;
31: **if** $f(T(r, F, H, E', d)) < f(T(r, F, H, E, d))$ **then**
32: $E \leftarrow E'$; $Terminate \leftarrow true$; **Break**;
33: **end if**
34: **end for**
35: **if** Terminate=true **then**
36: **Break**;
37: **end if**
38: **end for**

Algorithm 5: AP-ETT($T(r, F, H, E, d)$)

Here, we give description of Algorithm 5 for each pair $(i, j), i \in F \cup V$ (lines 1 and 2). In order to underline the similarity with classical ETT, we classify all commands from Algorithm 5 into four groups as follows:

1. **Add an edge (lines 4, 5):** Add (i, j) to the current tree T, where neither i nor j is a root node ($i, j \neq r$); in addition, at least i or j must be a functional node; update degrees d of the tree. In this step, a cycle is appeared in the current tree.
2. **Drop an edge (lines 8, 9, 23, 24):** There are two cases: If both i and j are functional nodes (case 1 from Algorithm 5), then two edges can be deleted to remove the cycle, which are $(i, parent(i))$, $(j, parent(j))$; update degrees d. If one of the nodes is a terminal node (let it be j, for example) (case 2 from Algorithm 5), then drop the edge $(j, parent(j))$; update degrees d.
3. **Add an edge (lines 11, 12, 16, 17, 25, 26):** If $(i, parent(i))$ is deleted, then add the edge (parent(i),child(j)); update d. If $(j, parent(j))$ is deleted, then add the edge (parent(j),child(i)); update d.
4. **Drop an edge (lines 12, 13, 18, 19, 27, 28):** Drop an edge from the tree T to keep the degree of each node feasible and update d. For example, if we look at step 5 of the Fig. 8, we can notice that the functional node "−" and the terminal node "x_1" have a degree greater than their original degree. Therefore, we have to delete the edge between "−" and "x_1."

The resulting neighboring tree is kept if its fitness is better than the original solution (Algorithm 5: Lines from 31 to 36). Figure 8 illustrates the steps presented in Algorithm 5 at the example from Fig. 7, when the first added edge connects two functional nodes (Case 1 from Algorithm 5). Note that in Fig. 8 the Plot (2) represents step "Add an edge" explained earlier. Plots (3) and (4) delete an edge, while Plots (5) and (6) perform additional add and drop edges, respectively. We now follow steps from Algorithm 5 on example from Figs. 7 and 8.

Algorithm 5 on example 1:

- $Terminate = false$.
- $i \in F = \{-, /, *\}$: for example, we take $i =' -'$, the degree of functional nodes is equal to 3. Therefore, $d_i = 3$.
- $j \in F \cup V \setminus \{i, parent(i), child(i)\}$: for example, $j =' *' \; d_j = 3$.
- $E' \leftarrow E \cup \{(i, j)\}$: we add the edge to the current tree connecting the node $'-'$ and the node $'*'$ (Fig. 8: step 2). Therefore, a cycle connecting the nodes $'-', '*', '/', \; and' max'$ is obtained.
- We increase the degrees of the nodes $'-'$ and $'*'$: $d_i = 3+1 = 4, d_j = 3+1 = 4$ (Fig. 8: step 2).
- Or $(j =' *')$, then $j \in F$. Therefore, it is a functional node and we are in case 1 of the algorithm.
- $E' \leftarrow E' \setminus \{(x, b)\} \; /x \in \{i, j\}, \; b \in \{parent(i), parent(j)\}$: We can delete the edge $(i, parent(i))$ that is $('-', max)$ or the edge $(j, parent(j))$, which is $('*', '/')$. We will delete, for example, the edge $('*', '/')$ (Fig. 8: step 3). So, $x = j$ and $b = parent(j)$ and we are in line 15 of Algorithm 5.
- We decrease the degree of the node $parent(j)$: $d_{parent(j)} = 3 - 1 = 2$ and the degree of the node j: $d_j = 4 - 1 = 3$ (Fig. 8: step 4).
- $E' \leftarrow E' \cup \{(b, child(i))\}$: We add the edge $(parent(j), child(i)$, which can be ('/','x_1') or ('/','x_2'). Let us add the edge ('/','x_1') (Fig. 8: step 5).

- Now, we increase the degree of $child(i)$, which is "x_1" and the degree of $parent(j)$, which is '/': $d_{parent(j)} = 2 + 1 = 3$, $d_{child(i)} = 1 + 1 = 2$ (Fig. 8: step 5).
- $E' \leftarrow E' \setminus \{(i, child(i))\}$: Finally, we delete the edge ('−','x_1'), and we update degrees: $d_i = 4 - 1 = 3$, $d_{child(i)} = 2 - 1 = 1$ (Fig. 8: step 6). We notice that the resulting tree $T(r, F, H, E', d)$ conserves the same degree of each node as in the original tree $T(r, F, H, E, d)$.
- If the fitness of the new tree $T(r, F, H, E', d)$ is better than the fitness of $T(r, F, H, E, d)$, then $Terminate = true$.
- The algorithm is stopped when we find the first better tree (the first improvement local search) or when exploring all the neighborhood structures.

From the example above, it can be observed that we made two edge additions and two edge deletions. The second add–drop pair is made to keep the degree of functional nodes equal to three, which is the necessary condition for any AP tree. Indeed, all operators in set F are binary and need 2 child nodes. From Algorithm 5, we can conclude that the following proposition holds.

Proposition 1.3 *Following the steps of Algorithm 5, the resulting AP subgraph T', obtained from the AP tree T, is an AP tree as well.*

Proof By adding an edge to the AP tree T, we get a unique cycle. Then, by dropping an edge from that cycle, we get another tree. However, this new tree is not a feasible AP tree since the degree of their nodes is not kept the same as in the original tree. This condition is verified in the next add–drop that recovers the AP feasibility. Indeed, if, for example, a functional node has a single child in the initial tree, then after modification, it should have a single child in the resulting tree. □

3.3.3 Bound on Cardinality of AP-ETT(T)

To calculate the number of possible neighbors generated from a tree $T(r, F, V, A, d)$ when applying $ETT(T)$, we need to know the structure of T. With the same sets, F, and V, we can get more than one AP tree. Thus, we consider that the AP tree has the largest possible number of neighbors to obtain the upper bound on cardinality. In other words, all terminals need to have the same depth, and the degree of each functional node should be equal to 3. This kind of tree is called a perfect tree.

Proposition 1.4 *The cardinality of $N(T)$, where T is an AP perfect tree, is less than $(n - 1)(n - 2)$.*

Proof Let T be an AP tree, and let n_1 and n_2 denote the cardinality of F and V, respectively (F includes all functional nodes except the root). In a perfect tree, the following holds:

$$n_1 = \frac{n}{2} - 1, \quad n_2 = \frac{n}{2} + 1, \tag{6}$$

where $n = |F| + |V|$. Note that n should be an even number due to the fact that root is excluded from F and T is a perfect tree.

Let us first calculate the number of all possible added edges when both endpoints belong to F. It is obviously equal to $n_1(n_1+1)/2 - n_1 = n_1(n_1-1)/2$. As explained earlier, for each added edge from F, there is only one way to drop an edge from the cycle, but there are four possibilities to perform another add–drop move to recover feasibility. In fact, after steps 1 and 2 of the AP-ETT move (Section 3.3.2), at most, two different possibilities of transformations can be done. Then following step 3 and 4, for each transformation, we obtain two possible resulting trees. So, two multiplied by two, we get four possible tree neighbors when applying AP-ETT in a single iteration. Thus, the cardinality of this type of move is $4[n_1(n_1-1)/2] = 2n_1(n_1-1)$.

The number of added edges in the case where one endpoint is in F and the other in V, is obviously $n_1 * n_2$. As illustrated in the given example, there are just two options to recover feasibility in this case. Finally, we have

$$|N(T)| \leq 2n_1(n_1 - 1) + 2n_1n_2 = 2n_1(n_1 - 1 + n_2) = (n - 1)(n - 2),$$

after substituting n_1 and n_2 from (6). □

Comparing cardinalities of $N(T)$ from Proposition 1.2 and Proposition 1.4, the following statement is obvious.

Theorem 1.1 *The upper bound on the cardinality of ETT neighborhood in an AP tree is smaller than that of a tree into an undirected graph in the order of magnitude.*

It should be noted that the result of Theorem 1.1 holds in the worst case in both problems. If an undirected graph is sparse, then the number of possible added edges within the ETT neighborhood could be much smaller than $O(n^2)$.

The ETT-based local search is also presented in Algorithm 5. We employ the first improvement strategy: when an improving solution T' is found, the local search is stopped and T' is set to be the new incumbent solution. Note that the AP-ETT move, and consequently AP-ETT local search (AP-ETT-LS), can be applied to both the GP individual and the VNP solution.

4 VNP for Symbolic Regression

Symbolic regression problem (SRP) is usually used problem for testing a new proposed AP solution technique. SRP is the process of analyzing and modeling numeric data sets to find the formula as combination of symbols, variables, and coefficients that represent, in the best possible way, dependence of variables in data. In fact, it is an optimization problem that is solved by searching on the space of mathematical equations the optimum model (function structure) satisfying a set of fitness cases [26, 52]. SRP can also be considered as a machine learning problem.

Several techniques have been successfully applied to solve this problem, including Neural Network [18, 61], AP-based metaheuristics [4, 12, 50, 81], and hybridization between AP metaheuristic and machine learning techniques [45, 75, 76].

All the mentioned approaches are able to solve efficiently symbolic regression problems. However, we compare here our BVNP with two powerful recent metaheuristic-based approaches: Genetic Programming [81] and Artificial Bee Colony Programming [50]. In [81], Uy et al. proposed two new relations for the crossover step, called semantic similarity and semantic equivalence. Similarly, like GP and GA or VNP and VNS, the Artificial Bee Colony Programming (ABCP) extends the Artificial Bee Colony (ABC) algorithm by representing individuals using more complex structures (programs).

Neighborhood Structures Besides the new ETT move, we refer two neighborhood structures in Elleuch et al. [19]. In detail, the following three neighborhoods are used in our BVNP method:

– $N(T)$ denotes AP-ETT neighborhood structure used in the local search.
– $\mathcal{N}_1(T)$ or *changing a node value*. It conserves the shape of the tree and changes only one value of a functional or a terminal node. If we apply $\mathcal{N}_1(T)$ to the example of the program presented in Fig. 7, then we find an expression with the same shape of the original one but differing in just one value. The original expression is $\max\{\frac{3}{x_3*2.4}; x_1 - x_2\}$. If the functional set includes $\{*, -, +\}$, then, such a move can give the following expressions: $\max\{3 + x_3 * 2.4; x_1 - x_2\}$, $\max\{3 * x_3 * 2.4; x_1 - x_2\}$, etc. This neighborhood alters the terminal node values as well. If the terminal set H includes $\{3, 2.4, 0.66, x_1, x_2, x_3, x_4\}$, then, possible expressions generated after one move are: $\max\{\frac{2.4}{x_3*2.4}; x_1 - x_2\}$, $\max\{\frac{0.66}{x_3*2.4}; x_1 - x_2)\}$, $\max\{\frac{x_1}{x_3*2.4}; x_1 - x_2\}$, etc.
– $\mathcal{N}_2(T)$ *Swap operator*. In this neighborhood, a node has to be selected first from the current tree. Then, a new subtree is generated according to a chosen size. Finally, this new subtree is attached in the place of the subtree, corresponding to the selected node. More details are given in [19]. The cardinality of $\mathcal{N}_2(T)$ is larger than the cardinality of the other two neighborhoods. Swap operator affects the shape and the content of the current tree. In fact, starting from the tree illustrated in Fig. 7, an infinite number of trees can be generated. We give here some examples, based on the same functional and terminal sets chosen as in the previous neighborhood structure: $\max\{\frac{3}{x_1}; x_1 - x_2)\}$, $\max\{\frac{3}{x_3*2.4}; x_4 * 0.66\}$, $\max\{\frac{3}{x_3*2.4}; (x_4 + 3) * 2.4\}$, etc. We notice that in one move, one branch of the original tree is kept.

Shaking The shaking step is given in Algorithm 6. In this procedure, we obtain the kth neighbor of a tree T after k repetitions of one move. In other words, we first choose some neighborhood randomly (either *Node value change* or *Swap*) and then repeat the move k times using the selected neighborhood.

$s \leftarrow random\ value \in \{1, 2\}$
for $i = 1, k$ **do**
  Choose randomly $T' \in \mathcal{N}_s(T)$
  $T \leftarrow T'$
end for
return(T)

Algorithm 6: $Shake(T, k)$

Basic VNP Algorithm Algorithm 7 gives the Basic VNP procedure. The first program is generated using the grow initialization method [53]. The tree is generated according to a user-specified depth. The depth of a node is defined as the number of "traversed edges" starting from the root node until reaching this node.

repeat
  $k \leftarrow 1$
  while $k \leq k_{max}$ **do**
    $T' \leftarrow Shake(T, k)$
    $T" \leftarrow$ AP-ETT(T')
    Neighborhood_change$(T, T", k)$
  end while
until the stopping condition is met

Algorithm 7: BasicVNP$(T, k_{\max})$

To find the best representative model of a given problem, the optimization procedure should take into account a tree structure and the node values. Here, we propose just three neighborhood structures, although it is clear that many other can be included. We follow the principles of recently proposed *Less is more approach* (LIMA) [7, 14, 15, 65, 67]. In LIMA, we try to find the minimum number of ingredients that would make the new method, at least, as good as those from the state of the art. As a result, we removed some neighborhoods proposed in [19]. This LIMA approach is applied, in the next section, to the symbolic regression problem, where its effectiveness is demonstrated again.

As mentioned in introduction, many AP methods are applied to the symbolic regression problem. A large number of benchmark data sets can be found in the literature as well. This fact allows us to test the performance of our new approach. Symbolic regression is in fact a mathematical modeling method for analyzing numerical data in order to find an approximation of a mathematical function in the symbolic form [53]. Symbolic regression is a search for the mathematical expression that minimizes error metrics. There are several classical approaches in this field such as the regression by the Bernstein polynomial technique [8, 80] and the regression based on splines [17, 23]. Symbolic regression deals simultaneously with the search of the parameters such as linear or nonlinear regression and the shape and structure of the function.

4.1 Test Instances and Parameter Values

In this work, we evaluate the performance of our algorithm on ten real-valued symbolic regression benchmark data sets. We have studied the following problems: polynomial, trigonometric, square root, logarithmic, and bivariate functions. Table 1 shows the ten benchmark functions, which were used by Uy et al. in 2011 [81], where the largest set of issues in the symbolic regression. They are mostly taken from [42, 48, 51]. All the randomly generated points are uniformly distributed.

Uy et al. [81] defined two new relations called semantic similarity and semantic equivalence. They are based on the semantic distance between subtrees. Their purpose was to introduce two new crossover operators in GP that are semantic similarity-based crossover and semantics-aware crossover. The semantic similarity-based crossover extends the semantics-aware crossover by more controlling the semantic distance between subtrees. For solving an AP problem, Artificial Bee Colony Programming (ABCP) heuristic is proposed in [50] and compared with GP. The specific parameters of ABCP and GP are given in Table 2, whose adjustments are taken from [50].

To make a reliable and fair comparison, we chose the same parameter values for BVNP as in [50] and [81] and adapt the same fitness function (Table 2, Table 4). Here, in addition to GP and ABCP, we include in comparison the recent Reduced VNP as well.

Table 1 Symbolic regression benchmark functions

Benchmark functions	Instances
$F_1 = x^3 + x^2 + x$	20 Random points $\subseteq [-1, 1]$
$F_2 = x^4 + x^3 + x^2 + x$	20 Random points $\subseteq [-1, 1]$
$F_3 = x^5 + x^4 + x^3 + x^2 + x$	20 Random points $\subseteq [-1, 1]$
$F_4 = x^6 + x^5 + x^4 + x^3 + x^2 + x$	20 Random points $\subseteq [-1, 1]$
$F_5 = \sin(x^2)\cos(x) - 1$	20 Random points $\subseteq [-1, 1]$
$F_6 = \sin(x) + \sin(x + x^2)$	20 Random points $\subseteq [-1, 1]$
$F_7 = \log(x + 1) + \log(x^2 + 1)$	20 Random points $\subseteq [0, 2]$
$F_8 = \sqrt{x}$	20 Random points $\subseteq [0, 4]$
$F_9 = \sin(x) + \sin(y^2)$	100 Random points $\subseteq [-1, 1] \times [-1, 1]$
$F_{10} = 2\sin(x)\cos(y)$	100 Random points $\subseteq [-1, 1] \times [-1, 1]$

Table 2 Symbolic regression parameters adjustment of ABCP and GP

GP parameters	Value	ABCP parameters	Value
Population size	500	Colony size	500
#of node evaluations	$15 * 10^6$	#of node evaluations	$15 * 10^6$
Crossover	0.9	Limit	500
Mutation	0.05		
Tournament size	3		

Table 3 Results of the Basic VNP using different values of $k_{\max}$

Functions	$k_{\max} = 1$	$k_{\max} = 2$	$k_{\max} = 3$	$k_{\max} = 4$	$k_{\max} = 5$
F_1	0.8596	0.1012	0.0632	0.0012	0.0624
F_2	1.2911	0.7539	0.0615	0.0564	0.1798
F_3	0.2367	0.1479	0.0329	0.0428	0.0638
F_4	0.5333	0.1736	0.0953	0.0943	1.3694

Table 4 Reduced VNP and Basic VNP parameters adjustment

Parameter	Value
The functional set	$F = \{+, -, \times, /, \sin, \cos, \exp, \log\}$
The terminal set	$\{x_i \cup c\}, i \in [1, n]$, n is the number of variables, $c \in [-5, 5]$
Fitness function	Sum of absolute error on all fitness cases
#of node evaluations	$15 * 10^6$

VNP—Parameters For the shaking step of BNP, after a brief experimentation, we set $k_{\max} = 4$. The BVNP is applied to the different functions: F_1, F_2, F_3, and F_4 for $k_{\max}$ values: $k_{\max} = 1$, $k_{\max} = 2$, $k_{\max} = 3$, $k_{\max} = 4$, and $k_{\max} = 5$. Table 3 shows the results of the mean errors obtained in 20 runs. It is clear that when $k_{\max} = 4$, the algorithm provides the better outputs in most cases. For the function F_3, BVNP algorithm reaches the best fitness value when $k_{\max} = 3$.

GP and ABCP are population-based algorithms, while VNP is a trajectory-based algorithm. Thus, GP and ABCP cannot be compared with VNP using the population size parameter. However, we can use a measure based on the number *of function node* evaluations to limit the number of iterations of the old VNP (RVNP) and the Basic VNP (Tables 2 and 4).

This measure is not only employed by the different GP techniques [81] and ABCP method [50] but also by other GP investigations [43, 84]. Table 4 gives the functional and terminal sets. In general, the functional set is defined after the experimental study. In our case, we use the same function set as in [50, 81].

Uy et al.[81] presents various crossover operators of GP, where standard crossover (SC), semantics-aware crossover (SAC), no same mate (NSM) selection, soft brood selection (SBS), context-aware crossover (CAC), and semantic similarity-based crossover (SSC) are applied, respectively. The explanation and the details of these techniques are available in [81].

4.2 Comparison of BVNP with Other Methods

Both Basic VNP and Reduced VNP are coded in *Java* programming language and executed on Intel Core i3 CPU (330M/2.13 GHz). To compare the performance of Basic VNP with GP and ABCP based methods, we used the same evaluation criteria

Table 5 Number of successful runs out of 100, within $15 * 10^6$ node evaluations of each

Methods	Functions										Average % of success
	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	
ABCP	89	50	22	**12**	**57**	**87**	58	37	33	21	46.6
SC	48	22	7	4	20	35	35	16	7	18	21.2
NSM	48	16	4	4	19	36	40	28	4	17	21.6
SAC2	53	25	7	4	17	32	25	13	4	4	18.4
SAC3	56	19	6	2	21	23	25	12	3	8	17.5
SAC4	53	17	11	1	20	23	29	14	3	8	17.9
SAC5	53	17	11	1	19	27	30	12	3	8	18.1
CAC1	34	19	7	7	12	22	25	9	1	15	15.1
CAC2	34	20	7	7	13	23	25	9	2	16	15.6
CAC4	35	22	7	8	12	22	26	10	3	16	16.1
SBS31	43	15	9	6	31	28	31	17	13	33	22.6
SBS32	42	26	7	8	36	27	44	30	17	27	26.4
SBS34	51	21	10	9	34	33	46	25	26	33	28.8
SBS41	41	22	9	5	31	34	38	25	19	33	25.7
SBS42	50	22	17	10	41	32	51	24	24	33	30.4
SBS44	40	25	16	9	35	43	42	28	33	34	30.5
SSC8	66	28	22	10	48	56	59	21	25	47	38.2
SSC12	67	33	14	**12**	47	47	66	38	**37**	51	41.2
SSC16	55	39	20	11	46	44	**67**	29	30	**59**	40
SSC20	58	27	10	9	52	48	63	26	39	51	38.3
RVNP	87	42	38	11	33	50	43	46	26	19	39.5
BVNP	**91**	**63**	**40**	11	49	57	60	**66**	30	25	**49.2**

from [81] and [50]: (i) *the percentage of successful runs*, (ii) *the mean best fitness*, and (iii) *the best fitness*. In addition, we add (iv) *running time* criterion.

(i) **The percentage of the successful runs** for each approach is summarized in Table 5.

This measure represents the total number of successful runs. A run is considered to be successful when at least one individual fitness value is lower than the threshold value (hits criterion) for all instances. To ensure a fair comparison, we use the same hits criterion 0.01 as in [50, 81]. We can see that the Basic VNP algorithm has a large number of successful runs for the first three problems (F_1, F_2, F_3) and also for F_8. For F_4, SSC12 and ABCP score a better percentage of successful run. For F_5, F_6, and F_7,, the Basic VNP ranks the second. The last column of Table 5 clearly indicates the order of methods based on this criterion: BVNP (49.2% average success rate), ABSP (46.6%), RVNP (39.5%), SSC20 (38.3%), etc.

(ii) **The mean best fitness** metric is more meaningful when comparing the effectiveness of AP techniques. For each problem, the *mean best fitness* of all indicated methods is given in Table 6, where the best performance found for each function is boldfaced.

Table 6 Mean best fitness values in 100 trials, within $15 * 10^6$ node evaluations for each

Methods	Functions										Averg
	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	mean
ABCP	0.01	**0.05**	0.07	0.10	0.05	0.02	0.06	0.10	0.47	1.06	0.236
SC	0.18	0.26	0.39	0.41	0.21	0.22	0.13	0.26	5.54	2.26	1.156
NSM	0.16	0.29	0.34	0.40	0.19	0.17	0.11	0.19	5.44	2.16	1.105
SAC2	0.16	0.27	0.42	0.50	0.22	0.23	0.15	0.27	5.99	3.19	1.345
SAC3	0.13	0.27	0.42	0.48	0.18	0.23	0.15	0.27	5.77	3.13	1.303
SAC4	0.15	0.29	0.41	0.46	0.17	0.22	0.15	0.26	5.77	3.03	1.284
SAC5	0.15	0.29	0.40	0.46	0.17	0.21	0.15	0.26	5.77	2.98	1.276
CAC1	0.33	0.41	0.51	0.53	0.31	0.42	0.17	0.355	7.83	4.40	1.783
CAC2	0.32	0.41	0.52	0.53	0.31	0.42	0.17	0.35	7.38	4.30	1.716
CAC4	0.33	0.41	0.53	0.53	0.30	0.42	0.17	0.35	7.82	4.32	1.773
SBS31	0.18	0.29	0.30	0.36	0.17	0.30	0.15	0.19	4.78	2.75	1.105
SBS32	0.18	0.23	0.28	0.36	0.13	0.28	0.10	0.18	4.47	2.77	1.052
SBS34	0.16	0.23	0.31	0.33	0.13	0.21	0.11	0.19	4.17	2.90	1.024
SBS41	0.18	0.26	0.27	0.38	0.12	0.20	0.13	0.20	4.40	2.75	1.037
SBS42	0.12	0.24	0.29	0.30	0.12	0.18	0.10	0.16	3.95	2.76	0.964
SBS44	0.18	0.24	0.33	0.35	0.15	0.16	0.11	0.19	2.85	1.75	0.724
SSC8	0.09	0.15	0.19	0.29	0.10	0.09	0.07	0.15	3.91	1.53	0.776
SSC12	0.07	0.17	0.18	0.28	0.10	0.12	0.07	0.13	3.54	1.45	0.720
SSC16	0.10	0.15	0.23	0.26	0.10	0.10	0.06	0.14	3.11	1.22	0.640
SSC20	0.08	0.18	0.23	0.30	0.09	0.10	0.06	0.14	2.64	1.23	0.588
RVNP	0.037	0.069	0.052	0.17	0.028	0.016	0.03	0.088	0.43	0.68	0.183
BVNP	**0.0038**	**0.050**	**0.052**	**0.096**	**0.015**	**0.0071**	**0.0024**	**0.015**	**0.41**	**0.512**	**0.136**

Based on the results presented in Table 6, it is clear that the Basic VNP provides the best mean best fitness value in all cases. Therefore, it gives the most adequate function outputs in comparison with RVNP, GP, and ABCP. ABCP obtains the same value as the Basic VNP only for F_2. The last column of Table 6 clearly demonstrates the superiority of VNP based approach. The order of the methods is BVNP (0.136), RVNP (0.183), ABCP (0.236), and SSC20 (0.588).

(iii) **The best fitness value.** There is another way of comparing the methods based on the output functions of the individual, i.e., the solution with the best fitness value. This information is not available for GP. Thus, Table 7 illustrates the generated functions obtained by the ABCP and BVNP algorithms.

These two techniques generate seven functions equal to the original ones. For F_8, the output of VNP is $F_8 = e^{log(x)*0.499}$. However, it can be written as $F_8 \approx e^{\frac{log(x)}{2}}$, which is equal to the target function. VNP did not find the exact function F_3; however, as can be observed from Fig. 9, the generated function has the same shape as F_3.

ABCP and VNP generate a function different from function F_4. The two generated functions and the real function are given in Fig. 10. The curve shape of

Table 7 Functions generated by Basic VNP and by ABCP

Function	Generated function by Basic VNP	Generated function by ABCP
$x^3 + x^2 + x$	$x^3 + x^2 + x$	$x^3 + x^2 + x$
$x^4 + x^3 + x^2 + x$	$x^4 + x^3 + x^2 + x$	$x^4 + x^3 + x^2 + x$
$x^5 + x^4 + x^3 + x^2 + x$	$\frac{1.6x^3+1.54x^2}{-0.56x^2+1.34} + x$	$x^5 + x^4 + x^3 + x^2 + x$
$x^6 + x^5 + x^4 + x^3 + x^2 + x$	$1.6 * x^4 + 1.26 * x^3 + 0.16 * x^2 + \frac{1.19 * x}{1.41 - x}$	$x * e^{((\frac{(1-\cos(\sin(x)))}{\sin(1)}+1)*(x+1)-1)}$
$\sin(x^2)\cos(x) - 1$	$\cos(-3.15 + \sin(-1.5x - 3.15))$	$\sin(x^2)\cos(x) - 1$
$\sin(x) + \sin(x + x^2)$	$\sin(x) + \sin(x + x^2)$	$\sin(x) + \sin(x + x^2)$
$\log(x + 1) + \log(x^2 + 1)$	$\log(x + 1) + \log(x^2 + 1)$	$\frac{x}{e^h} \times$
$\sqrt{x}$	$e^{\log(x)*0.499}$	$e^{\frac{r\log(x)}{2}}$
$\sin(x) + \sin(y^2)$	$\sin(x) + \sin(y^2)$	$\sin(x) + \sin(y^2)$
$2\sin(x)\cos(y)$	$2\sin(x)\cos(y)$	$2\sin(x)\cos(y)$

Note: $*h = \left(1 + \sin(x) + \frac{((1-\cos(\sin(\cos(x))+1))*\cos(1-\sin(x))*\cos(x))}{\left(\frac{1}{\sin\left(e^{\cos(1)}\right)}+\sin(1)\right)}\right)^{-1}$

Table 8 Average running times in seconds for Reduced VNP and Basic VNP, within the same maximum number of node evaluations, on Intel Core i3 330M/2.13 GHz

Functions	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}
RVNP	0.14	0.13	0.29	0.057	7.10	4.88	6.06	3.62	7.23	6.41
BVNP	5.64	10.68	4.34	18.31	16.91	30.19	73.34	46.22	82.69	66.20

the function generated by VNP is closer to the curve of the target function than that by ABCP.

(iv) **Running times.** It is interesting to note that most AP techniques do not include running time in their computational results. Note that in our comparison, all the methods have the same stopping condition that is the maximum number of node evaluations. It is calculated by multiplying the number of fitness evaluations by the number of nodes of this tree. We use node evaluation instead of tree evaluation to estimate the computational cost for a uniform comparison. Using the same stopping rule, in Table 8, we report CPU times that RVNP and BVNP spent until their best values were obtained. Note that running times of the other methods were not available.

The results indicate that RVNP obtains the solutions fast. However, they cannot be improved by using additional CPU times.

In conclusion, the Basic VNP provides the best results for most studied problems from the literature under the same conditions. Our method is an effective way of improving the old VNP algorithm and creating a new research environment in the automatic programming field. The only limitation is the manual choice of

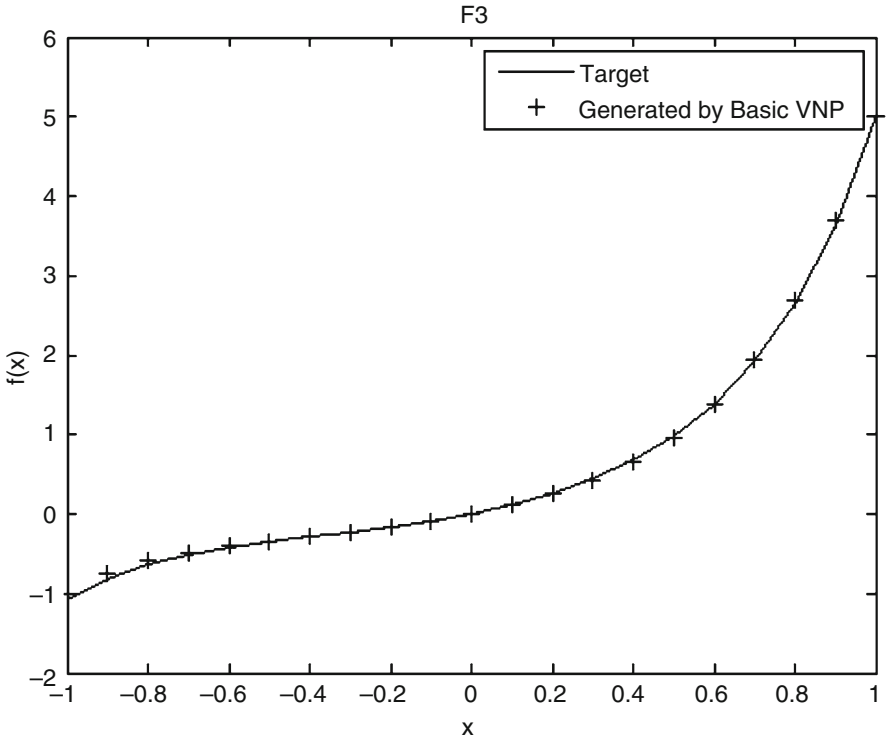


Fig. 9 Target and generated F3 functions obtained by Basic VNP

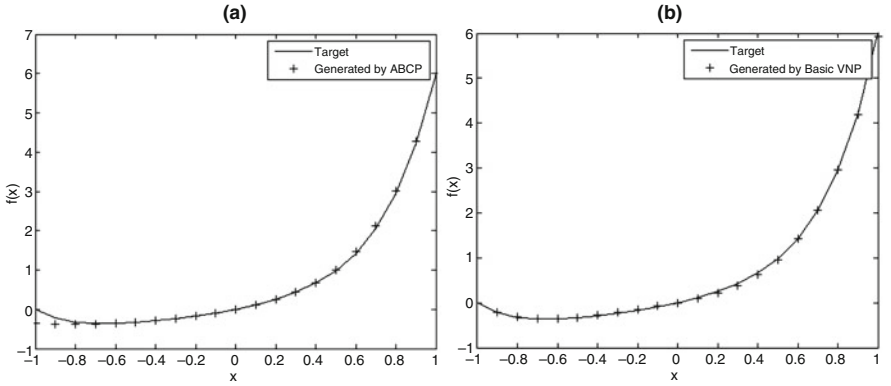


Fig. 10 Target and generated F4 functions obtained (**a**) by ABCP and (**b**) by Basic VNP

parameters and functional and terminal sets for both VNP and other automatic programming techniques.

5 Life Expectancy Estimation as a Symbolic Regression Problem Solved by VNP: Case Study on Russian Districts

Life expectancy is an indicator that reliably shows the general state of health in a society in a certain period of time. The public health model developed by Lalonde [55] recognizes that in addition to the health system, factors such as the environment, genetic characteristics, and behavior (determinants of health not related to the health system) affect the health of the population. The results of numerous studies using the approach of the production function of health and highlighting individual factors affecting the health of the population allow us to group these factors into large blocks: heredity, socio-economic factors, lifestyle, environment, and health care system. Factors affecting health are closely related. Modeling health indicators depending on individual factors determining it is a methodologically complex task.

The research topic we are considering in this paper appears to be important. At the "Our World in Data" site (https://ourworldindata.org/), it is included in the list of 297 world's largest problems (https://ourworldindata.org/grapher/life-expectancy-vs-health-expenditure). The web page purpose is expressed as *research and interactive data visualizations to understand the world's largest problems*. There, the life expectancy y is considered as a function of one variable, i.e., health care expenditure in each particular country. A data set of almost all countries is provided too at the site. A simple one-dimensional plot is presented for each country, showing mostly linear dependencies.

In [60], authors analyze the life expectancy of 70 years old people, as a function of their physical condition and health care money spent until they die, using the 1992–1998 *Medicare Current Beneficiary Survey* in the USA. They found that persons in better health had a longer life expectancy than those in poorer health but had similar cumulative health care expenditures until death.

To estimate life expectancy y, the input of the health care system is expressed by health care expenditures per capita (current US$) as a single attribute in [46]. The data are collected for 175 world countries, grouped according to the geographic position and income level, over 16 years (1995–2010). The authors applied a panel data analysis. The obtained results show a significant relationship between health expenditures and life expectancy. Country effects are significant and show the existence of important differences among the countries.

In 1995–2015, most OECD countries constantly increased their spending on health care, but to varying degrees and with different effects of increasing life expectancy [79]. This is well illustrated by the example of some countries with the highest level of GDP per capita (USA, Germany, Great Britain, Japan, Italy, France, Canada, Norway, Netherlands, Australia) [39]. During this period, life

expectancy at birth and the average per capita income for health care increased in all these countries, but not everywhere the level and increase in health expenditure were accompanied by an adequate increase in life expectancy. This may indicate that lifestyles and behavioral stereotypes play an important role in increasing life expectancy, as well as the effectiveness of the use of funds entering the health care system. The existence of a direct relationship between life expectancy and the level of health expenditure is also established on the basis of available data for OECD countries and partner countries in 2015 (or close to it) year [40]. The value of life expectancy at birth (2015) was the higher, the higher the proportion of government spending on health (in 2014) for large regions of the World Health Organization, with the exception of the African region [85].

There are many other studies regarding the life expectancy of people. They compare life expectancy between man and woman, different geographical regions, countries, continents, etc. However, most of them consider just one input variable separately in showing results. In this paper, we use three different input variables simultaneously, all three being kind of health care expenditure. Therefore, we use a multidimensional regression analysis to find functional dependencies more precisely. Moreover, for the first time, we applied an artificial intelligence approach in estimating the life expectancy. We applied a variable neighborhood programming technique, a recent automatic programming method for solving the symbolic regression problem. In addition, we collected the relevant data for all Russian geographical regions. Interesting results are derived from the final regression formulas. For example, some input expenditures are not relevant for the life expectancy, and also the functions are mostly linearly increasing.

In the next section, we define the estimation of life expectancy as a symbolic regression problem. In Sect. 3, we give a brief explanation of how Variable Neighborhood Programming (VNP) is working in solving the symbolic regression problem, while Sect. 4 provides details regarding case study on Russian geographical regions. Results and their analysis are provided in Sect. 5, while Sect. 6 concludes the paper.

5.1 Life Expectancy Estimation as a Machine Learning Problem

To estimate life expectancy, we use artificial intelligence and machine learning approach. More precisely, for each geographical region, we are trying to find the best analytic function $y = f(x)$, where $x = (x_1, x_2, x_3) \in R^3$.

Since we have a machine learning problem, the data set must be divided into learning and testing sets. Therefore, the method contains two steps:

– **Learning step:** The AI or machine learning algorithm is applied to the 2/3 of the data set of each district.

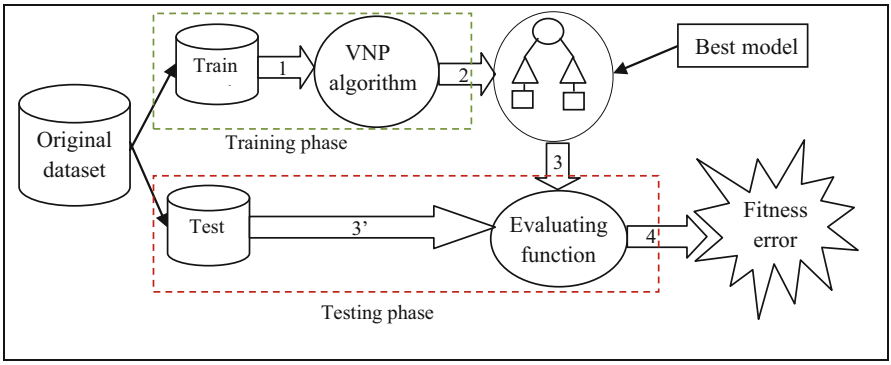


Fig. 11 Learning process schema

– **Testing step:** The AI algorithm is applied to the remainder of the data set of each district (see Fig. 11).

The symbolic regression problem consists of finding a mathematical relation in a symbolic form between the inputs and the outputs. Our economically based input variables are as follows:

x_1—expenditures of the consolidated budgets of constituent entities of the Russian Federation on health, million Roubles per 1 thousand people (Fig. 13);
x_2—expenditure of territorial funds of compulsory medical insurance of constituent entities of the Russian Federation, million roubles per 1 thousand people;
x_3—average per capita money income of the population, per month, in roubles.

We decided to eliminate the crude death as an attribute. Our only output variable y presents the life expectancy.

5.2 VNP for Estimating Life Expectancy Problem

To solve this problem, we apply the Variable Neighborhood Programming algorithm to the leaning data sets. Each data set concerns one district.

In our work, we use the Basic VNP variant [19, 20]. The solution is a program (function) that is presented by a tree. This tree graph includes two types of nodes: functional and terminal nodes, the latter one containing constants and variables. Used functional set F includes arithmetic operators that are $F = (+, -, *, /)$. The terminal set includes the problem's variables $\{x_1, x_2, x_3\}$ and constants that we decide to belong to interval $[-5, 5]$.

The first solution is generated using the grow initialization method.

The following three neighborhood structures are used in our VNP method:

– $N(T)$ denotes neighborhood structure used in the local search. Each possible edge is added to the tree T that represents the current regression formula, and then transformation is performed to remove an edge from the circle such obtained, in order to recover feasibility. Not all edges are allowed to be added or removed, as in usual tree with one node type (see [20] for details).
– $\mathcal{N}_1(T)$ or *Changing a node value*. It conserves the shape of the tree and changes only one value of a functional or a terminal node [19].
– $\mathcal{N}_2(T)$ *Swap operator*. In this neighborhood, a node has to be selected first from the current tree. Then, a new subtree is generated according to a chosen size. Finally, this new subtree is attached in the place of the subtree, corresponding to the selected node. More details are given in [19, 20].

Like the Basic VNS, the Basic VNP algorithm iterates *Shaking, Local search*, and *Neighborhood change* steps:

– **Shaking**. We use *Changing a node value* operator $\mathcal{N}_1(T)$ and the *Swap* operator $\mathcal{N}_2(T)$ that are randomly chosen for each k. Thus, the tree T' in k^{th} neighborhood of T is obtained by repeating k times random move either using $\mathcal{N}_\infty$ or $\mathcal{N}_2$.
– **Local search.** We use the adapted elementary tree transformation, i.e., $N_1(T)$ neighborhood is explored.
– **Neighborhood change.** If T'' represents better regression formula (with smaller error), then neighborhood parameter k is set to 1; otherwise, $k \leftarrow k + 1$.

5.3 Case Study at Russian Districts

In the last decade, Russia has been pursuing an active state policy in the field of health protection, increasing access to medical care. The average life expectancy at birth in the period from 2005 to 2018 increased by 7.5 years: from 65.37 to 72.91 [78]. However, the gap with the EU countries in this indicator in 2017 was 8.2 years (European Union represented by 28 countries had 80.9 years [21], while Russian Federation—72.7 years [78]).

There are various opinions of Russian experts on the impact of the health system on the life expectancy of the population: the Head of the Higher School of Organization and Management of Health Care, Guzel Ulumbekova, believes that the quality and availability of medical care in Russia only 30% determine the life expectancy of the population; by 37%, it depends on socio-economic factors, primarily on income, and by 33% on lifestyle, in particular, on alcohol and tobacco consumption; Sergei Shishkin, the Director of the Center for Health Policy at the Higher School of Economics, also believes that life expectancy is determined by socio-economic conditions, lifestyle, and quality of medical care, but in what proportion—there are no reliable assessment methods. The Head of the Laboratory for the Evaluation of Health Care Technologies of the Russian Academy of People's State and the State Service under the President of the Russian Federation, Vitaly

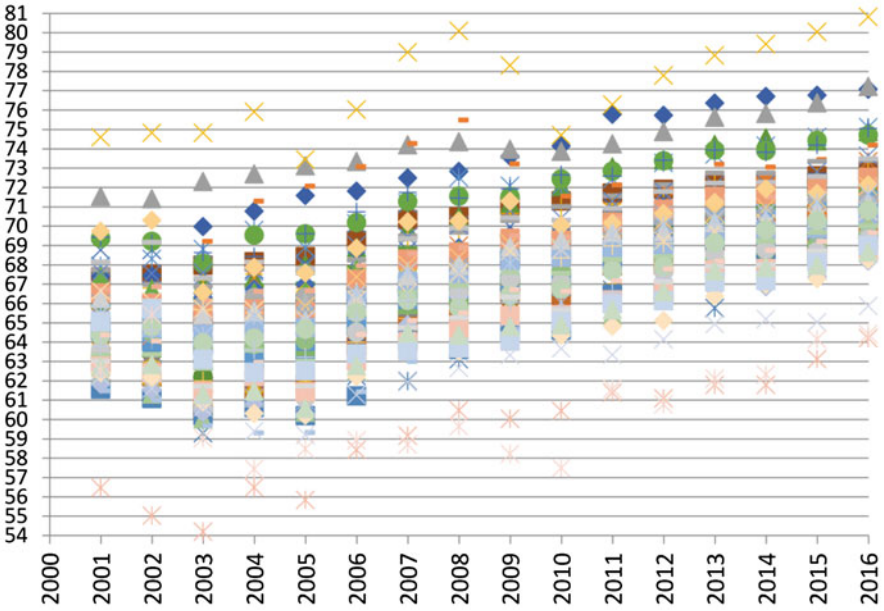


Fig. 12 Life expectancy at birth in 85 regions of Russia, 2001–2016

Omelyanovsky, believes that the world does not have a clear understanding of the quantitative contribution of medicine to the increase in life expectancy [59].

5.3.1 One-Attribute Analysis

To carry out the study on the basis of the official data of Rosstat, a database for 2001–2016 has been compiled for 85 regions [33–38]. We choose from statistical tables just 3 economic indicators as mentioned before as 3 variables for our symbolic regression model (x_1, x_2, x_3) and life expectancy y. Here, we first analyze each of those indicators separately. Then, we analyze the results of our symbolic regression approach.

Life expectancy y at birth in Russia from 2001 to 2016 increased by 6.58 years from 65.29 to 71.87 years (Fig. 12, where each color represents different districts). The minimum value increased from 56.48 to 64.21 years, and the maximum from 74.6 to 80.82 years. The gap between the minimum and maximum values during this period decreased from 1.3 to 1.26 times.

The main part of Russian citizens receives medical care for free [41] (according to the Russian Monitoring of the Economic Situation and Public Health of the Higher School of Economics in 2015) (Fig. 13).

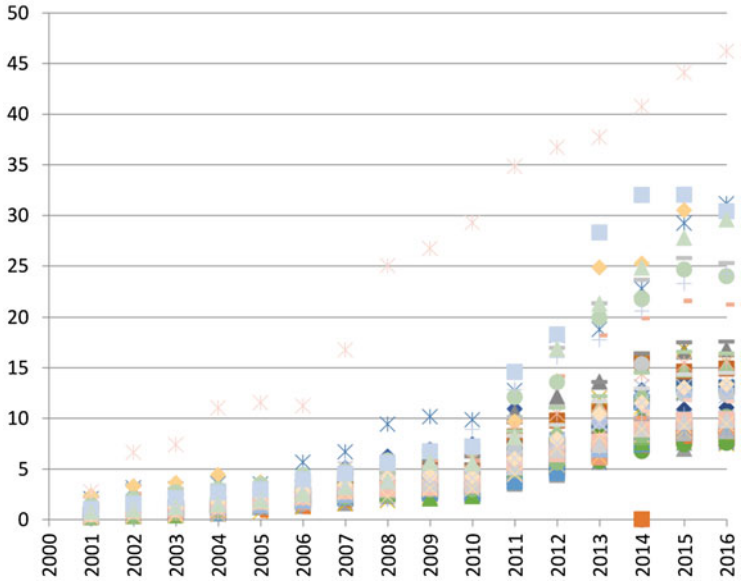


Fig. 13 Expenditure of territorial funds of compulsory medical insurance of constituent entities of the Russian Federation, million roubles per 1 thousand people (at current prices), 2001–2016

Expenditure of Territorial Funds x_2 of compulsory medical insurance of constituent entities of the Russian Federation, at current prices, on average in Russia, increased from 0.6 to 11.1 million roubles per 1 thousand people (Fig. 14). The minimum value has grown from 0.005 to 7.5 million roubles per 1 thousand people, and the maximum—from 2.8 to 46.3 million roubles per 1 thousand people. However, it is necessary to bear in mind that the data are given in current prices.

Average Per Capita Money Income x_3 of the population in current prices on average in Russia increased from 3062 roubles in the month to 30,744 roubles in a month (Fig. 15). The minimum value increased from 909 to 14,107 roubles in a month, and the maximum—from 10733 to 69,956 roubles per month. It must be borne in mind that the data are given in current prices.

5.3.2 Results and Discussion on 3-Attribute Data

We run VNP code to find formulas for each out of 85 districts. We start our analysis of the results obtained by VNP on the Central Federal District.

$$f(x) = \frac{25137x_2}{34889} + \frac{2777791212823}{42248834550}.$$

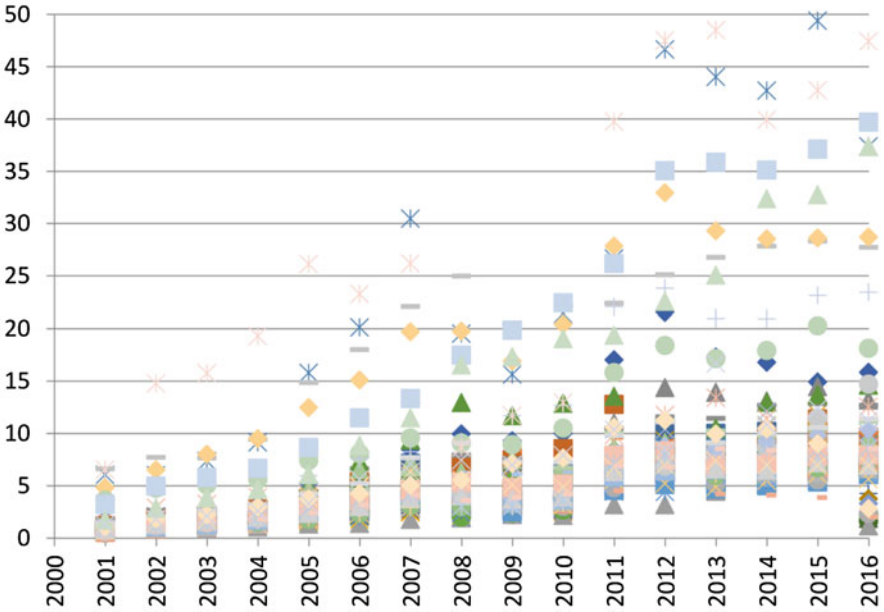


Fig. 14 Expenditures of the consolidated budgets of constituent entities of the Russian Federation on health, million roubles per 1 thousand people (at current prices), 2001–2016

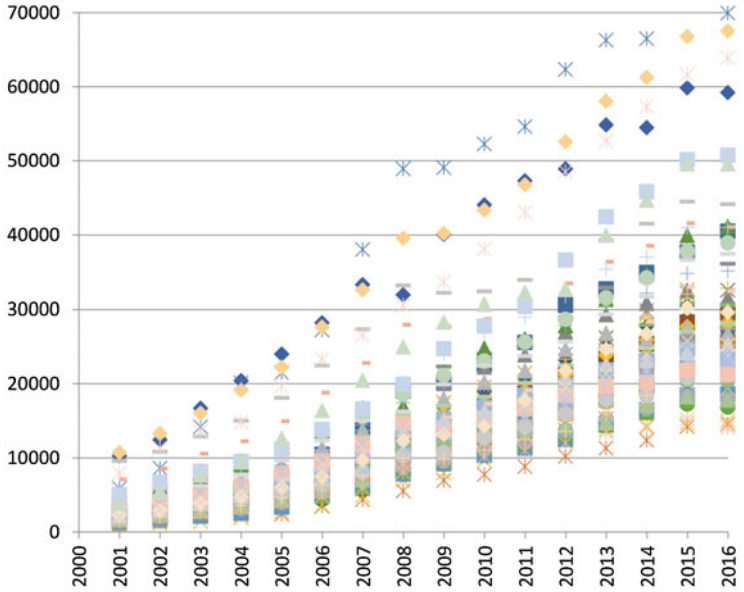


Fig. 15 Average per capita money income of population, in month, roubles (at current prices), 2001–2016

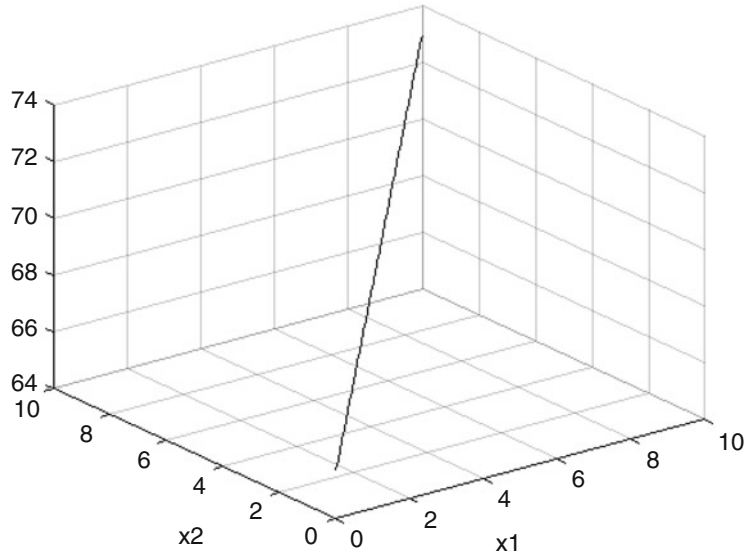


Fig. 16 Central Federal District output according to variables x_1 and x_2

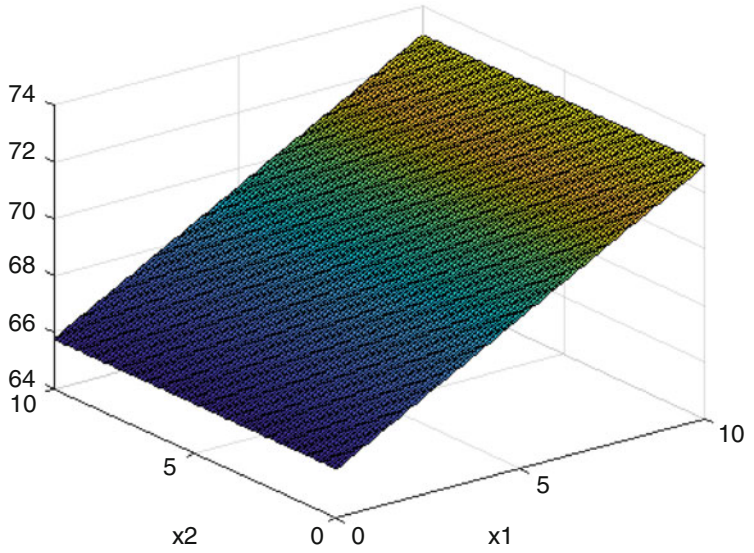


Fig. 17 Central Federal District output according to variables x_1 and x_3

Since we could not present four-dimensional figure, we separated obtained life expectancy into 2 figures: at Fig. 16, y is plotted as functions of x_1 and x_2; then at Fig. 17, we present $y = f(x_1, x_3)$;

It is clear that the final function $y = f(x_1, x_2, x_3)$ is almost linear. This observation is in accordance with other studies mentioned in Introduction.

The similar results are obtained by other 84 geographical districts. It is interesting to note that the output from our VNP code is sometimes very long and could be presented in reduced form, after elementary algebraic transformations. For example, for Ural Federal District, the obtained VNP output function is

$$f(x)=\frac{984067x_2}{50000\left(x_2+\frac{7249}{1300}\right)\left(-\frac{62500x_1}{578641x_2}+\frac{281}{100}+\frac{95677}{10000\left(-\frac{22883}{2000}-\frac{8074}{625\left(-\frac{98863}{15625}+\frac{3003}{1060x_2}\right)}\right)}\right)}+\frac{6236451}{100000}$$

that can be transformed into

$$f(x) = 62.36 + \frac{19.68x_2}{(x_2 + 5.58)(2.81 - 0.11\frac{x_1}{x_2} + \frac{270.76-604.67x_2}{59.39x_2-32.37})}$$

$$= 62.36 - \frac{1167.02x_2^3 - 637.04x_2^2}{438.04x_2^3+2264.46x_2^2+6.52x_1x_2^2+32.82x_1x_2-1003.28x_2-19.86x_1}.$$

Even the function is not linear, after plotting it in (x_1, x_2, y) space (see Fig. 18), we see that from small x_1 and x_2 the almost linear function has one slope, and another for larger investment. Note also that x_3 is not included again in the formula.

5.4 Conclusions

In this section, we addressed the life expectancy question and its relation with three expenditure types into a health care system. We propose symbolic regression, one machine learning approach, to find whether there is some simple analytic dependence of life expectancy from the three expenditure types. We tested our approach on 85 Russian geographical districts, taking into account data for 15 years. Our basic conclusions are: (1) for a small amount of investment, there are no clear functional dependencies of life expectancy and three investment types; (2) for a larger investment, there are almost linear dependencies in all 85 districts, i.e., the more expenditure, the larger life expectancy can be expected; (3) in some districts, one and sometimes even two types of investments have no influence on life expectancy. Most often variable x_3 did not have any influence on life expectancy.

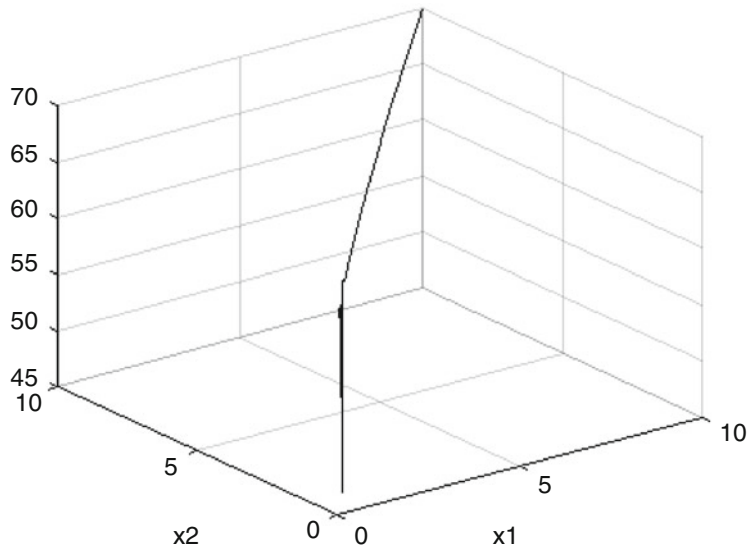


Fig. 18 Ural Federal District output according to variables x_1 and x_2

6 Preventive Maintenance in Railway Planning as a Machine Learning Problem

Railway transportation is becoming more and more important due to environmental reasons, e.g., CO_2 emission and low energy consumption requirements. To ensure that a railway system operates efficiently, the maintenance of the infrastructure must be carefully performed. Over the last decades, the number of railway travel lines has been increased. As a result, optimization of the maintenance planning is getting more complex and more important activity [62].

Railroad companies run an inspection for a determined period and record the characteristics of found defects in rail tracks. These defects can be classified into two classes, red and yellow. The red class includes the defects that violate Federal Railroad Administration (FRA) standards, and the yellow class includes the defects that meet FRA standards but violate the railroad's own standards. If a defect belongs to the red type, it must be repaired immediately; otherwise, the defect can be fixed after a specific time period. Each defect is characterized by many features, where the most important is the amplitude value that gives information about the status of the defect. The problem is that we cannot know the amplitude value in advance; hence, a step of prediction should be performed. Yet as the period of the maintenance planning is not constant, we need a powerful prediction algorithm. The third problem to tackle is to classify the defect based on its characteristics and the information provided by the prediction step.

The above problems can be solved by tools from the area of artificial intelligence, or more preciously, automatic programming (AP) techniques. An AP algorithm creates an intelligent process adapted to a specific problem. Among the traditional automatic programming methods, we cite, for example, inductive logic programming [68] and artificial neural networks [22, 27, 47, 54]. On the other hand, metaheuristics are a wide class of algorithms designed to solve approximately many optimization problems, and being flexible, they do not need much adaptation to a concrete problem. In recent years, these algorithms are emerging as good alternatives to classical AP methods. In general, metaheuristics are applied to problems characterized by their difficulty, where there is no specific and satisfactory algorithm to solve them. In the past three decades, the automatic generation of programs through metaheuristics like evolutionary computation and swarm intelligence has gained a significant widespread popularity. This success is mainly due to the research work of Koza [52] and the development of the Genetic Programming (GP) algorithm in 1992. GP starts from an initial population of programs and then applies genetic operators in each generation (iteration) of the method in order to reach a near-optimal solution or a near-optimal program. GP has been used in solving many problems [11, 53]. It was successfully compared with the artificial neural network on the classification problems in [70]. Immune Programming, another evolutionary-based AP technique, was proposed in [69]. It was inspired by the principles of the vertebrate immune system. Some swarm intelligence metaheuristics are implemented and adapted to solve AP problems as well, such as particle swarm intelligence [71], ant colony algorithm [77], and Artificial Bee Colony algorithm [50].

Despite the success of metaheuristics implemented in AP, there are several algorithms that are still not adapted for this area. The No Free Lunch theorems, developed by Wolpert [83], demonstrated that for the entire set of optimization problems, all metaheuristics have the same average of performance. However, on a particular class of problems, some algorithms perform better than others, and we think that it is the case with AP problems.

Notice that population-based AP metaheuristics have shown their effectiveness to solve complicated problems. However, when the target computer program is based on many features, the search strategy employed by population-based AP metaheuristics can suffer from good trouble balancing on its exploitation and exploration abilities. Actually, population-based algorithms explore at early generations a large number of computer programs. Nevertheless, some potential computer programs could not be considered because of their lack of early exploitation [86]. To overcome this, a Basic Variable Neighborhood Programming (BVNP) algorithm has been developed recently by [20] to solve forecasting problems. This algorithm is inspired by the Variable Neighborhood Search (VNS) that was introduced in 1997 [63]. VNS is based on a local search but does not follow a single trajectory. A local search is performed within neighborhood structures. In fact, the neighborhood of a solution is defined as the set of solutions obtained by applying one move. The neighborhood structures are designed to explore the vicinity of the current incumbent solution in the search space. VNS explores the search space in depth and conducts the process to

reach a good solution in a reasonable time [25]. VNS has many degrees of freedom, and as being very general, it can be applied to solve different problems. Besides, it implies several features such as simplicity, precision, effectiveness, and robustness [30]. VNP inherits all the characteristics of the VNS algorithm. However, it evolves a solution presented by a program. Therefore, a set of new neighborhood structures has to be defined allowing to explore neighbors in the search space of programs.

This paper proposes a decision support system for solving the preventive maintenance problem of railway infrastructure. For that purpose, the problem is first decomposed into two stages: prediction and classification. Both phases need two phases as well: a learning phase and testing phase. The solution technique for solving learning and testing problems is based on the recent Variable Neighborhood Programming (VNP) metaheuristic [19, 20]. For solving preventive maintenance problems, we have developed a reduced version of VNP (RVNP), i.e., there is no local search routine, and different neighborhood structures are implemented just in the shaking step of VNP.

Therefore, two main contributions have been made: (1) decomposing the original maintenance railway problem into two more elementary problems and (2) solving both sub-problems as automatic programming problems, using a new variant of the VNP algorithm. The computational results and the comparison with the GP algorithm approve the quality of our approach.

The rest of this work is organized in the following way. First, we give a literature review and an overview of the maintenance planning problem. Second, in Sect. 3, we describe the new Reduced VNP algorithm and the used neighborhood structure. Afterward, we explain the implementation of our system using a case study based on the 2015 RAS Problem Solving Competition data set (https://www.informs.org/Community/RAS/Problem-Solving-Competition). Results and comparison with GP are discussed in Sect. 5. The final section is reserved for conclusions.

6.1 Literature Review and Motivation

In the United States, 86,000 passengers use railroads every day. Moreover, railroads account for 40% of intercity freight volume [5]. Railway transportation is highly regulated by the state to protect travelers. Maintenance of railway infrastructure is an important task of railway transportation to keep freight and passenger trains moving safely. However, it is very expensive and very difficult to plan [62]. A survey in this area could be found in [58]. According to the Federal Railroad Administration (FRA), in 2012, nearly 33% of train accidents (about 577 accidents) were caused by geometric track defects, reported as 102.9 million dollars of damage [74].

Periodic inspection runs are planned every year by the North American railroads to address this problem. These inspections are accomplished using track geometry vehicles, and millions of dollars are spent on traveling across the network of the railroads to record 40 different rail defects [10] using Global Positioning System (GPS) and visual inspection technologies. The detected defects are classified

according to the severity level. If a defect does not meet FRA standards, then it is classified as a red class and it must be repaired immediately. Otherwise, the defect belongs to the yellow class, and it is not urgent to fix it. However, it is important to forecast when these yellow defects would become red tags.

The categories of maintenance that can be found in the literature are corrective maintenance and preventive maintenance [1, 32]. Corrective maintenance repairs the defects found in the railway infrastructure, whereas preventive maintenance is planned to avoid the possible future defects. In this area, the literature describes many works where preventive (planned) maintenance has high complexity and a large scale of operations [29, 56, 73, 88]. However, very few researches have tackled the problems of corrective (unplanned) maintenance [89]. In general, track degradation models are classified into two approaches: mechanistic and stochastic.

Mechanistic methods are mainly based on laboratory works to understand and explain the track degradation phenomena. Unfortunately, the mechanical properties and geometries are very difficult to quantify and differ from one place to another, resulting in considerable predictive errors [87].

Statistical approaches are based on probabilistic distributions and use recorded information in order to predict future defect [2, 3]. In fact, FRA sponsored projects study the preventive maintenance planning techniques, and their number increases with the development of the automatic track geometry measurement. On the other hand, artificial intelligence models are essentially developed based on Artificial Neural Networks, support vector regression, and neuro-fuzzy models. In fact, prediction about degradation in civil engineering that uses these approaches is quite modern. In 2014, Guler [28] have implemented an approach to calculate the track deterioration rate that is a non-constant function based on cumulative train loads, time, and other variables. This system is based on the Artificial Neural Networks technique. In the best of our knowledge, although automatic programming algorithms (as GP and VNP) are powerful techniques in artificial intelligence, they are not yet tested to solve preventive railway maintenance problems.

The purpose of this study was to develop an accurate system able to provide preventive maintenance planning. The inspection runs provide the history of the data describing the status of several track segments. The problem is to predict the defect color of a selected track segment in a predefined milepost value after a given period. Railroad companies distinguish many types of defects: surface, cross-level, and alignment. In this paper, we focus on three defects. Surface defect is detected by measuring the uniformity of the rail. These measures are recorded over a 62-foot chord for the right and left rails. Figure 19 shows an example of surface defect measurement.

Cross-level defect is measured by calculating the difference in height between the top of one rail to the top of the opposite rail. This difference is indicated in Fig. 20.

The third defect, Alignment, is also called Dip. It is calculated over a 31 foot. As illustrated in Fig. 21, Dip represents a fall in the track and the corresponding rise. The measured value is positive if the defect is a rise and is negative if the defect is a fall. We note that this defect is calculated in the center within a short distance.

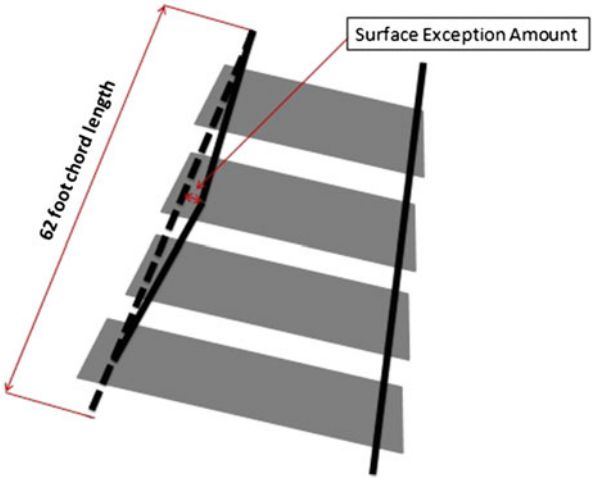


Fig. 19 Surface defect

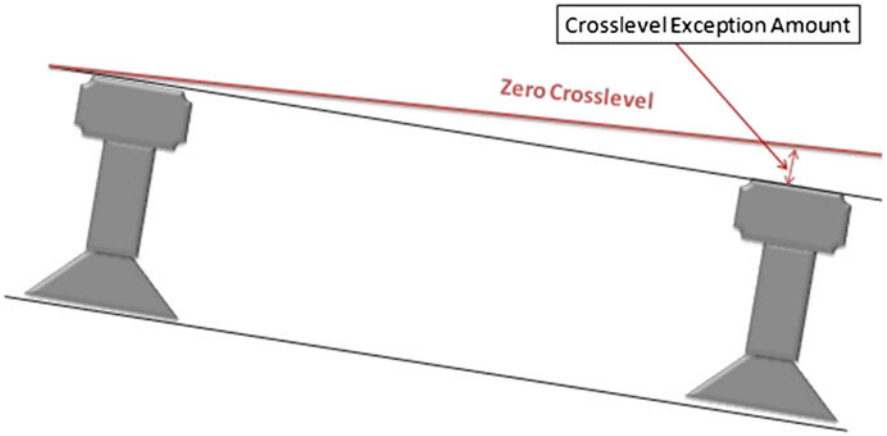


Fig. 20 Cross-level defect

6.2 Reduced VNP for Solving the Preventive Maintenance Planning of Railway Infrastructure

The preventive maintenance planning of railway infrastructure (PMPoRI) consists of identifying defects that have to be corrected immediately. The developed solution should predict the color of the defect after a given period.

Railway Application Section (RAS) provides a training data set and a testing data set. In the training data set, there are about twenty attributes (columns in the input matrix). The data set has around 36,000 defect samples (rows in the input matrix).

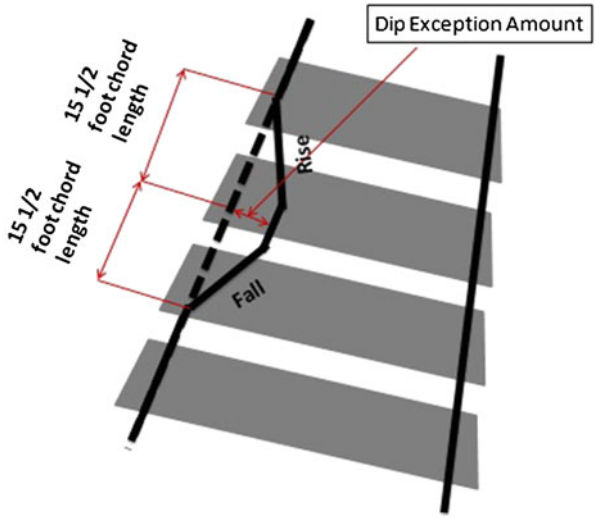


Fig. 21 Dip defect

In Table 9, the most important attributes with short descriptions, derived from the 2015 RAS training data set, are given (for more details, see the training data set available on the 2015 RAS competition website). As mentioned before, the measure of a defect depends on its type, which makes the problem more complicated and requires many analyses.

Each line in the file corresponds to one defect, and each defect is recorded in a known milepost value. The hardness of the problem is presented by the fact that participants have to assign a color to a defect after any given period of time. This period is fixed by organizers and differs across defects. To minimize the difficulty, we decide to divide the original problem into two different sub-problems, prediction, and classification (as we mentioned before). The task of the first one is the prediction of attribute values after a given period (defect color is not predicted). Then, the second goal is the classification of defects, based on data obtained by the prediction output and other variables.

Reduced VNP algorithm can be applied in both phases. In fact, the VNP solution can be presented in different complexities. Therefore, during the execution, a solution can change its size and its shape to satisfy the problem requirements. The next subsections illustrate the application of the proposed interwoven algorithm Reduced VNP in the two cited problems. In the two stages, we need a training step to extract statistical features and a testing step to evaluate the resulting model.

Table 9 Training data set columns

Column	Type	Characteristics
LINE_SEG_NBR	INT	Every track on the railroad has a unique identifying line segment number. It could be a single or a double track. By using line segment(LINE_SEG-NBR) and milepost(MILEPOST_START and MILEPOST_END), you can identify any location in the system
MILEPOST	DEC	A Milepost is a point on the track where the defect is detected
TRACK_SDTK_NBR	CHR	Distinguishes individual track segments. Mainline and branch numbers: 0 = SINGLE TRACK, 1–9 = MULTIPLE MAIN LINES (For example, 1 = NORTH MAIN, 2 = SOUTH MAIN)
TEST_DT	DATE	The date on which testing was performed
DEF_NBR	INT	Every defect is detected by a unique id defect number
GEO_CAR_NME	CHR	Geometry cars names
DEF_PRTY	CHR	Severity of the defect: Yellow or red
DEF_LGTH	INT	Length of a defect in feet DEF_LGTH is reported by the measurement car
DEF_AMPLTD	DEC	The defect amplitude—maximum size of a defect in inches or degrees within the defect length
TSC_CD	CHR	The track codes including tangent, spiral, and curve
CLASS	CHR	All tracks get a number between one and five. Each class represents operating speed limits for passenger and freight traffic. Class one has the lowest speed limit, and class five has the highest speed limit
TEST_FSPD	CHR	Operates speed for freight trains
TEST_PSPD	CHR	Operates speed for passenger trains. If the value = 0, means that it does not have passenger traffic
DFCT_TYPE	CHR	The defect type and the geometric defect type such as $Cross - level$, $Surface$, and Dip
TOT_DFLT_MGT	DEC	The sum of total gross tons traveling across the section

6.2.1 Learning for Stage 1: Prediction

The prediction in this work consists of finding a function that approximates sample data. Before starting this phase, we have to analyze and organize the data sample. When studying the available data, we distinguish three types of defects, mentioned previously (Surface, Cross-level, and Dip). Each defect differs from the other two in the amplitude measurements and the conditions that lead to the red level. Hence, we decided to study each defect individually.

Our goal in this stage is to select the attributes (columns from the training data set) that are responsible for the determination of the defect severity after a defined number of days and to create a prediction system able to update them. Actually, most columns in the data set are not time-dependent (see RAS training data set from https://www.informs.org/Community/RAS). When we examine the data set and the role of each column, we can note that the only column, which is updated after a specified time is the defect amplitude attributes "DEF_AMPLTD" (see Table 9).

This indicates that the prediction system output is the value of "DEF_AMPLTD" after a given period t_0. We also deduce that the value of "DEF_AMPLTD" in the instant $t-t_0$, and the number of days "PERIOD" between the instants t_0 and $t-t_0$ are the inputs. Now what remains is just to select the other prediction inputs. This step is accomplished by many tests. In each test, we select a set of attributes, and according to the training error, we decide to remove or add attributes to the input list. In fact, we can note that the sum of total gross tons traveling across the section column "TOT_DFLT_MGT" and the operating speed for freight trains and passenger train columns "TEST_FSPD" and "TEST_PSPD" seem to be good attributes to perform the prediction step and find a good updated defect amplitude value (Table 9 gives a definition of all the mentioned attributes).

After defining the inputs and output of our algorithm, we have to collect and filter data samples for the learning step. The problem here is that the inspection run to control the rail and to record data is not always done in the same exact milepost point. That is to say that the information of the defect amplitude value in a given milepost point after a certain period is not available. To overcome this, we have to consider that two milepost values having the same line segment number "LINE_SEG-NBR" and the same track segments number "TRACK_SDTK_NBR" and separated by a value less than 0.01 are, in fact, the same point. The attributes corresponding to the milepost recorded in an earlier date form the inputs; however, the defect amplitude value corresponding to the milepost recorded in the recent date forms the output (we need to remind that each entry in the training data set corresponds to the information about a defect in one milepost recorded in one inspection run). Another constraint must be respected; when selecting two records, the period between the first record and the second record is less or equal to 265 days. In addition, we tried to have 50% from the collected instances where the latest record is red. Table 11 shows how the data in Table 10 is used for prediction. Indeed, Table 10 gives the values of the attributes of instance 1 and instance 2, which represent information about the same defect "DIP" on two different inspection runs. These instances are chosen carefully to build one entry of the learning data for prediction. As we have explained, the milepost of instance 1 and the milepost of instance 2 can be considered the same point as they are separated by a value less than 0.01. Moreover, the two selected instances have the same line segment number "LINE_ SEG_ NBR" and the same track segments number "TRACK_ SDTK_ NBR." The "DEF_AMPLTD" value recorded on 18 August 2013 is considered as the input, and the "DEF_AMPLTD" value recorded on 17 September 2013 is the desired output since it is the most recent. "TOT_ DFLT_ MGT" presents the sum of total gross tons traveling across the section in one month. To complete the instance of Table 11, we also have to know the value of the input "TOT_ DFLT_ MGT" during the period between August and September. So we calculate the sum of "TOT_ DFLT_ MGT" value determined in August and "TOT_ DFLT_ MGT" value determined in September. The number of dates "PERIOD" is easily deduced. As we mentioned previously, "TEST_ FSPD" and "TEST_ PSPD" operate speeds for freight trains and passenger trains. They are set, respectively, on 70 and 79. These values present the maximum speeds of trains crossing the track rail between

Table 10 Example of the collected samples

Variable	Instance 1	Instance 2
LINE_ SEG_ NBR	2	2
MILEPOST	21.61269	21.62043
TRACK_ SDTK_ NBR	0	0
TEST_ DT	18 Aug 2013	17 Sep 2013
DEF_ AMPLTD	1.34	1.51
CLASS	5	5
TEST_ FSPD	70	70
TEST_ PSPD	79	79
DEF_ PRTY	*Dip*	*Dip*
TOT_ DFLT_ MGT	4.60	5.11

Table 11 Input and output instances in the learning data

Inputs					Output
C1	C2	C3	C4	C5	C6
1.34	70	79	$4.6 + 5.11$	31	1.51

C1: DEF_AMPLTD at $t - t_0$, C2: TEST_FSPD, C3: TEST_PSPD, C4: TOT_DFLT_MGT, C5: PERIOD, C6: DEF_AMPLTD at t_0

the two inspection runs. For each defect, we collect about 1000 samples to learn the algorithm in the prediction phase and 200 samples to test the resulting program.

6.2.2 Learning for Stage 2: Classification

Classification is the process of searching a set of models that distinguishes and characterizes data classes or concepts [49, 82]. In our work, a model is a program presented as a tree (see Fig. 2b). Indeed, a program is a mathematical function that fits the data points to be able to predict the class of objects whose class label is unknown. The derived model is based on the analysis of a set of training data. We can fix two different classes. The first gathers the milepost points having red defects, and the second gathers those having yellow defects.

In the present work, the classification is more complicated than the prediction. In fact, we aim, in this step, to extract the features of each class. At the beginning of our research, the initial algorithm did not give satisfactory results. After many tests, we note that for each defect, the extracted features depend on the class of tracks attribute "CLASS" (see Table 9). It represents operating speed limits for passenger and freight traffic. Therefore, we decided to divide the data set again, according to the defect type and to the class of each defect. An example of the program presentation is given in Fig. 22. The value of a program corresponds to the output. We fix the value "0" for the yellow class and "1" for the red class. When solving a classification problem with an automatic programming algorithm, the traditional threshold value used in the case of binary classification (two classes) is "0" [44].

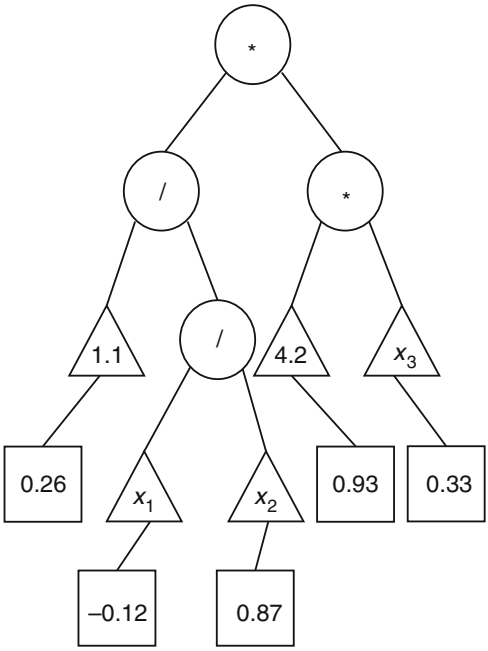


Fig. 22 Program presentation in classification problem; x_1 is the DEF_AMPLTD input, x_2 is the TEST_FSPD input, and x_3 is the TEST_PSPD input

However, the experimentation demonstrated that the value "0.5" is more suitable as a threshold. If the result given by the best program is less than "0.5," it means that the corresponding inputs belong to the yellow class. Otherwise, it belongs to the red class. The general model can be written as follows:

$$F(x_1, .., x_5) = \begin{cases} f_1(x_1, x_2, x_3) \text{ if } x_4 = 2 \text{ and } \quad x_5 = \texttt{"DIP"} \\ f_2(x_1, x_2, x_3) \text{ if } x_4 = 3 \text{ and } \quad x_5 = \texttt{"DIP"} \\ f_3(x_1, x_2, x_3) \text{ if } x_4 = 4 \text{ and } \quad x_5 = \texttt{"DIP"} \\ f_4(x_1, x_2, x_3) \text{ if } x_4 = 5 \text{ and } \quad x_5 = \texttt{"DIP"} \\ f_5(x_1, x_2, x_3) \text{ if } x_4 = 2 \text{ and } x_5 = \texttt{"SURAFCE"} \\ f_6(x_1, x_2, x_3) \text{ if } x_4 = 3 \text{ and } x_5 = \texttt{"SURAFCE"} \\ f_7(x_1, x_2, x_3) \text{ if } x_4 = 4 \text{ and } x_5 = \texttt{"SURAFCE"} \\ f_8(x_1, x_2, x_3) \text{ if } x_4 = 5 \text{ and } x_5 = \texttt{"SURAFCE"} \\ f_9(x_1, x_2, x_3) \text{ if } x_4 = 2 \text{ and } \ x_5 = \texttt{"XLEVEL"} \\ f_{10}(x_1, x_2, x_3) \text{ if } x_4 = 3 \text{ and } \ x_5 = \texttt{"XLEVEL"} \\ f_{11}(x_1, x_2, x_3) \text{ if } x_4 = 4 \text{ and } \ x_5 = \texttt{"XLEVEL"} \\ f_{12}(x_1, x_2, x_3) \text{ if } x_4 = 5 \text{ and } \ x_5 = \texttt{"XLEVEL"}. \end{cases} \tag{7}$$

In (7), x_1 denotes the DEF_AMPLTD, x_2 is the TEST_FSPD, x_3 is the TEST_PSPD, x_4 is the CLASS of speed, and x_5 is the type of defect. This function draws a curve separating the two classes, yellow and red. Obviously, the used $x_1 =$ DEF_AMPLTD value is the output of the prediction step. Indeed, the TSC_CD

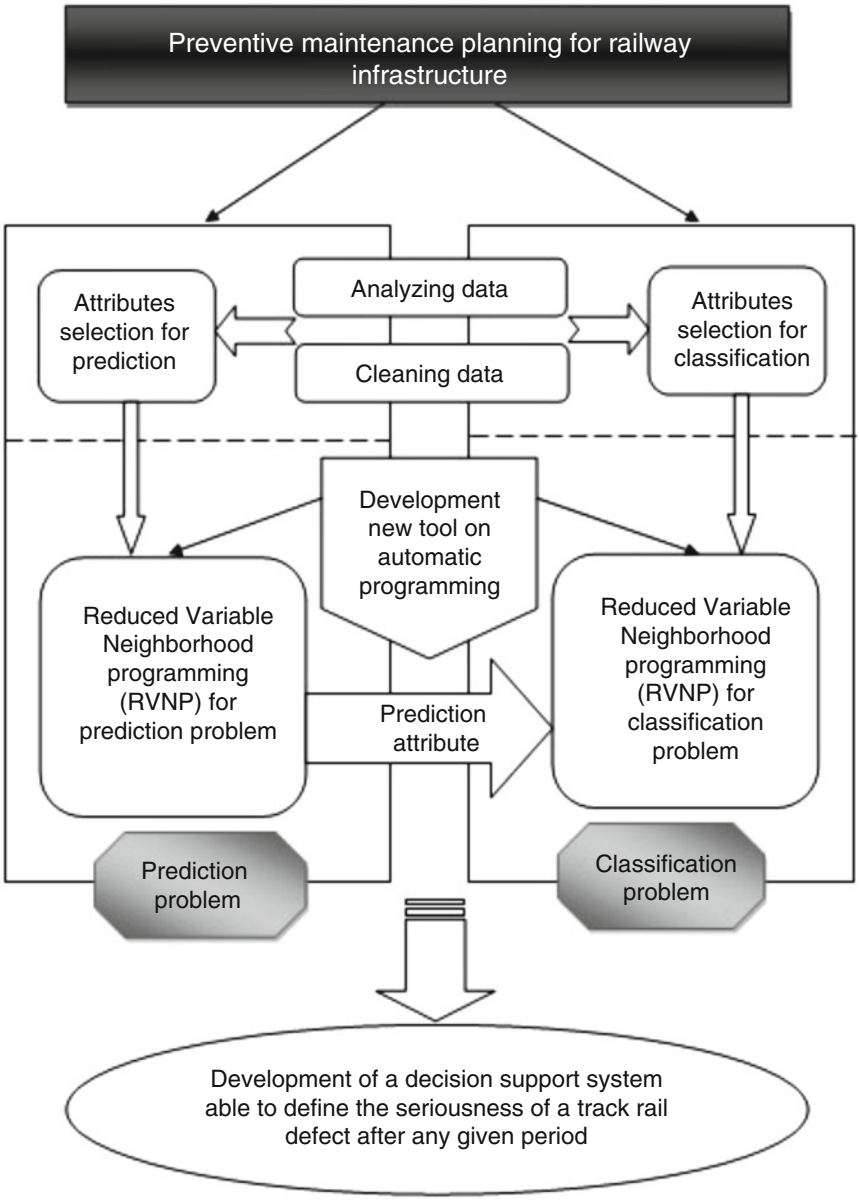


Fig. 23 Overall scheme of the proposed solution

attribute that denotes the track codes is used neither in the prediction step nor in the classification step because this value has not an influence in the convergence process. Figure 23 provides an illustration of the proposed method.

Table 12 Reduced VNP parameters adjustment for the prediction problem

The functional set	$F = \{+, -, \times, /\}$
The terminal set	{DEF_AMPLTD, TEST_FSPD, TEST_PSPD, TOT_dDFLT_MGT} $\cup\, c, c \in [-10, 10]$
The parameter set	$P = \alpha \in [-1, 1]$
Neighborhood structures	N_1, N_2, N_3
Minimum tree length	3 nodes
Maximum tree length	1500 nodes
Maximum iteration number	1000 iterations
Fitness function	MAE Error

6.3 Computation Results

6.3.1 Prediction

To apply the VNP algorithm, we have to follow four steps:

1. identification of *terminal* and *functional* sets;
2. selection of neighborhood structures;
3. adjustment of parameters;
4. definition of an adaptability or fitness function.

The functional set includes just arithmetic operators. Terminal set includes the input attributes of track segments and random real numbers between -10 and 10. The parameter set values serve to give a corresponding coefficient α_i to each terminal node i, thereby not giving the same importance to each of them. Therefore, the value of α_i, $\forall i$, is always between -1 and 1. We use all neighborhood structures presented in the previous section. Table 12 summarizes the Reduced VNP parameters adjustment. The learning step for each defect is run separately. During this process, the algorithm searches to minimize the error between the desired output and the output of our algorithm. The error used is the Mean Absolute Error (MAE), and it is calculated as follows:

$$\text{MAE} = \frac{1}{n} \sum_{j=1}^{n} \left| y_t^j - y_{\text{out}}^j \right|,$$

where n represents the total number of samples, and y_{out}^j and y_t^j are the VNP's given model, and the desired output of the sample number j, respectively.

Our algorithm is implemented in $Java$ language and executed by Intel core i3 processor (2.3 GHz) using Windows 7 Operating System. The average time of execution is 5 min for the learning process of each defect, and the MAE in this step is 7. The prediction provides a mathematical function that approximates the maximum points of data, and this function is used to update the amplitude defect

value after the period mentioned in the testing data set. The outputs of this phase are employed in the classification phase.

6.3.2 Classification

Before running the algorithm, we have to proceed as we did in the prediction step and fix the functional and terminal sets and the other parameters. Indeed, we choose the same values shown in Table 12, the only difference is in the terminal set, it includes:{DEF_AMPLTD, TEST_FSPD, TEST_PSPD} $\cup$ c, and c is selected uniformly at random from the range of [-10,10]. To measure the efficiency of our method in this classification problem, we employed another performance measure, named Accuracy (Acc). Accuracy presents the rate of the correctly classified instances. It is one of the most popular metrics in this area

$$Acc = \frac{TN + TP}{TP + FP + FN + TN} * 100, \tag{8}$$

where TP, TN, FP, and FN are the True Positive, True Negative, False Positive, and False Negative, respectively.

After several experiments, we reach an accuracy value equal to 99.7% in the training process, and we find a good representative model for each class in each defect. These models are just applied in the testing phase. The test data set includes 180 samples where the provided period varies between 8 and 265 days. RAS informed us that the application of our model in the testing data set gives an error equal to 23%. They appreciated the quality of our work and gave as an honorable mention for the originality of our idea. In this competition, we had the fourth place. In fact, among all participants, just four teams attained an error lower than 50%.

To evaluate more our approach, we apply the GP algorithm in place of the RVNP algorithm. A five-fold cross-validation procedure is carried out. Therefore, the data is five-fold partitioned, and each method is tested 5 times on each fold. One fold is treated as the validation set, and the algorithms are trained by the remaining four folds. The parameter adjustments of the GP algorithm are detailed in Table 13.

To fairly compare the AP algorithms, two different stopping criteria were employed in the training set: The algorithms were stopped, in one case, when a classifier with 100% accuracy was found or the number of function evaluations reached 10^8. The results presented in Table 14 are the mean of the different runs. We can observe that the most accurate results were obtained through the RVNP method. For the surface defect, the obtained accuracy values by the RNVP and the GP are very close. However, the difference between them in the average is more than 7%. Another advantage of the RVNP algorithm is its ability to produce trees smaller than those produced by GP thanks to the new individual representation and the used neighborhood structures. In fact, the new solution representation allows us to obtain solutions, which cannot be visited when using the old solution representation, by varying the coefficients (parameters). In addition, the selected neighborhood

Table 13 GP parameters adjustment for classification problem

The functional set	$F = \{+, -, \times, /\}$
The terminal set	{DEF_AMPLTD, TEST_FSPD, TEST_PSPD} $\cup\, c, c \in [-10, 10]$
Population size	100
Population crossover rate	0.9
Mutation rate per node	0.1
Tournament size	2
Minimum tree length	3 nodes
Maximum tree length	1500 nodes
Fitness function	Accuracy

Table 14 Overall accuracy results of GP and RVNP

Defect	GP	RVNP
Surface	79.96%	80.06%
Cross-level	68.12%	74.83%
Dip	65.41%	81.66%
Average	71.16%	78.85%

structures allow to discovery efficiently the research space and maintain a reasonable size of the tree.

6.4 Conclusions and Future Work

In this section, we propose a decision support system that is able to plan railway track maintenance. The purpose is to determine the color of a railway defect at a given date. This approach is based on the new automatic programming algorithm Reduced VNP. In fact, to solve the present problem, we divided it into a prediction problem and a classification problem. The prediction serves to update the important attributes after a number of days. The classification determines the color of a given defect based on the information provided by the prediction step. A new Reduced VNP algorithm was developed and applied to the real data published by the INFORMS Railway Application Section, and it gave good results in the training and testing stages. These results were also compared to the results of the GP algorithm using the cross-validation strategy. Our method proved to outperform the GP in all cases. Our proposed approach can be easily extended to forecast the severity of other types of defects and track, as long as there is enough track inspection information. In addition, future work can consider the other variants of VNP, such as General VNP, Mixed VNP, and Skewed VNP to solve the geometry track planning problem.

7 Conclusions

In this chapter, we first give the basic principles of Variable Neighborhood Programming (VNP), an artificial intelligence, and machine learning technique and then apply it to some classical problems with a large set of test instances, as well as on two practical real-life examples. It appears that our VNP is easy to understand and comparable to state-of-the-art AI and ML methods. Future work may include developing more sophisticated VNP variants and implementing VNP to other AI and ML problems.

Acknowledgments This publication is partially supported by the Khalifa University of Science and Technology under Award No. RC2 DSO. This research is also partially supported by the framework of Grant BR05236839, development of information technologies and systems for stimulation of personality's sustainable development as one of the bases of development of digital Kazakhstan.

References

1. Andersson, M.: Strategic planning of track maintenance – state of the art. Technical Report 02-035 (2002)
2. Andrade, A.R., Teixeira, P.F.: Hierarchical Bayesian modelling of rail track geometry degradation. Proc. Inst. Mech. Eng. F J. Rail Rapid Transit **227**(4), 364–375 (2013)
3. Andrade, A., Teixeira, P.: Statistical modelling of railway track geometry degradation using Hierarchical Bayesian models. Reliab. Eng. Syst. Saf. **142**, 169–183 (2015)
4. Arnaldo, I., Krawiec, K., O'Reilly, U.-M.: Multiple regression genetic programming. In: Proceedings of the 2014 Conference on Genetic and Evolutionary Computation – GECCO '14, pp. 879–886. ACM Press, New York (2014)
5. Association of American Railroads (AAR). https://www.aar.org/todays-railroads (2015)
6. Bouaziz, S., Dhahri, H., Alimi, A.M., Abraham, A.: A hybrid learning algorithm for evolving Flexible Beta Basis Function Neural Tree Model. Neurocomputing **117**, 107–117 (2013)
7. Brimberg, J., Mladenović, N., Todosijević, R., Urošević, D.: Less is more: solving the Max-Mean diversity problem with variable neighborhood search. Inf. Sci. **382–383**, 179–200 (2017)
8. Brown, B.M., Chen, S.X.: Beta-Bernstein smoothing for regression curves with compact support. Scand. J. Stat. **26**(1), 47–59 (1999)
9. Cai, W., Pacheco-Vega, A., Sen, M., Yang, K.: Heat transfer correlations by symbolic regression. Int. J. Heat Mass Trans. **49**(23–24), 4352–4359 (2006)
10. Cannon, D.F., Edel, K.-O., Grassie, S.L., Sawley, K.: Rail defects: an overview. Fatigue Fract. Eng. Mater. Struct. **26**(10), 865–886 (2003)
11. Castelli, M., Vanneschi, L., Silva, S.: Prediction of the unified Parkinson's disease rating scale assessment using a genetic programming system with geometric semantic genetic operators. Expert Syst. Appl. **41**(10), 4608–4616 (2014)
12. Castelli, M., Trujillo, L., Vanneschi, L.: Energy consumption forecasting using semantic-based genetic programming with local search optimizer. Comput. Intel. Neurosci. **2015**, 971908 (2015)
13. Choi, W.-J., Choi, T.-S.: Genetic programming-based feature transform and classification for the automatic detection of pulmonary nodules on computed tomography images. Inf. Sci. **212**, 57–78 (2012)

14. Gonçalves-de-Silva, K., Aloise, D., Xavier-de-Souza, S., Mladenovic, N.: Less is more: simplified Nelder-Mead method for large unconstrained optimization. Yugosl. J. Oper. Res. **28**, 153–169 (2018)
15. Costa, L.R., Aloise, D., Mladenović, N.: Less is more: basic variable neighborhood search heuristic for balanced minimum sum-of-squares clustering. Inf. Sci. **415-416**, 247–253 (2017)
16. de Arruda Pereira, M., Davis Júnior, C.A., Gontijo Carrano, E., de Vasconcelos, J.A.A.: A niching genetic programming-based multi-objective algorithm for hybrid data classification. Neurocomputing **133**, 342–357 (2014)
17. De Boor, C.: A Practical Guide to Splines: With 32 Figures. Springer, Berlin (2001)
18. Deklel, A.K., Saleh, M.A., Hamdy, A.M., Saad, E.M.: Transfer learning with long term artificial neural network memory (LTANN-MEM) and neural symbolization algorithm (NSA) for solving high dimensional multi-objective symbolic regression problems. In: 2017 34th National Radio Science Conference (NRSC), pp. 343–352. IEEE, Piscataway (2017)
19. Elleuch, S., Jarboui, B., Mladenovic, N.: Variable neighborhood programming – a new automatic programming method in artificial intelligence. Technical report, G-2016-92, GERAD, Montreal (2016)
20. Elleuch, S., Hansen, P., Jarboui, B., Mladenović, N.: New VNP for automatic programming. Elect. Notes Discrete Math. **58**, 191–198 (2017)
21. Eurostat. https://ec.europa.eu/eurostat/data/database
22. Fernandez de Canete, J., Del Saz-Orozco, P., Baratti, R., Mulas, M., Ruano, A., Garcia-Cerezo, A.: Soft-sensing estimation of plant effluent concentrations in a biological wastewater treatment plant using an optimal neural network. Expert Syst. Appl. **63**, 8–19 (2016)
23. Friedman, J. H.: Multivariate adaptive regression splines. Annal. Stat. **19**(1), 1–67 (1991)
24. Galar, M., Fernández, A., Barrenechea, E., Bustince, H., Herrera, F.: An overview of ensemble methods for binary classifiers in multi-class problems: experimental study on one-vs-one and one-vs-all schemes. Patt. Recog. **44**(8), 1761–1776 (2011)
25. García-Torres, M., Gómez-Vela, F., Melián-Batista, B., Moreno-Vega, J.M.: High-dimensional feature selection via feature grouping: a variable neighborhood search approach. Inf. Sci. **326**, 102–118 (2016)
26. Ghaddar, B., Sakr, N., Asiedu, Y.: Spare parts stocking analysis using genetic programming. Europ. J. Oper. Res. **252**(1), 136–144 (2016)
27. Graupe, D.: Principles of Artificial Neural Networks. World Scientific, Singapore (2007)
28. Guler, H.: Prediction of railway track geometry deterioration using artificial neural networks: a case study for Turkish state railways. Struct. Infrastruct. Eng. **10**(5), 614–626 (2014)
29. Gustavsson, E., Patriksson, M., Strömberg, A.-B., Wojciechowski, A., Önnheim, M.: Preventive maintenance scheduling of multi-component systems with interval costs. Comput. Ind. Eng. **76**, 390–400 (2014)
30. Hansen, P., Mladenović, N.: Variable neighborhood search. In: Search Methodologies, pp. 211–238. Springer, Boston (2005)
31. Hansen, P., Mladenović, N., Pérez, JAM.: Variable neighbourhood search: methods and applications. Ann. Oper. Res. **175**(1), 367–407
32. He, Q., Li, H., Bhattacharjya, D., Parikh, D.P., Hampapur, A.: Track geometry defect rectification based on track deterioration modelling and derailment risk assessment. J. Oper. Res. Soc. **66**(3), 392–404 (2015)
33. Healthcare in Russia. Stat. book./Rosstat (2006)
34. Healthcare in Russia. Stat. book./Rosstat (2007)
35. Healthcare in Russia. Stat. book./Rosstat (2009)
36. Healthcare in Russia. Stat. book./Rosstat (2011)
37. Healthcare in Russia. Stat. book./Rosstat (2015)
38. Healthcare in Russia. Stat. book./Rosstat (2017)
39. Health at a Glance 2017: OECD indicators. http://dx.doi.org/10.1787/888933602215
40. Health at a Glance 2017: OECD indicators. http://dx.doi.org/10.1787/888933602272
41. Health care: current status and possible development scenarios. In: Dokl. to the 18th April International Scientific conference on the Problems of Economic and Social Development, Moscow, April 11–14, 2017. House of the Higher School of Economics (2017)

42. Hoai, N., McKay, R., Essam, D., Chau, R.: Solving the symbolic regression problem with tree-adjunct grammar guided genetic programming: the comparative results. In: Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No. 02TH8600), vol. 2, pp. 1326–1331. IEEE, Piscataway (2002)
43. Hoang, T.-H., Essam, D., McKay, B., Hoai, N.-X.: Building on success in genetic programming: adaptive variation and developmental evaluation. In: Advances in Computation and Intelligence, pp. 137–146. Springer, Berlin (2007)
44. Howard, D., Roberts, S., Brankin, R.: Target detection in SAR imagery by genetic programming. Adv. Eng. Softw. **30**(5), 303–311 (1999)
45. Icke, I., Bongard, J.C.: Improving genetic programming based symbolic regression using deterministic machine learning. In: 2013 IEEE Congress on Evolutionary Computation, pp. 1763–1770. IEEE, Piscataway (2013)
46. Jaba, E., Balan, C.B., Robu, I.-B.: The relationship between life expectancy at birth and health expenditures estimated by a cross-country and time-series analysis. Proc. Eco. Finance **15**, 108–114 (2014). Emerging Markets Queries in Finance and Business (EMQ 2013).
47. Jiaqiu, W., Ioannis, T., Chen, Z.: A space–time delay neural network model for travel time prediction. Eng. Appl. Artif. Intell. **52**, 145–160 (2016)
48. Johnson, C.G.: Genetic Programming Crossover: Does It Cross over? pp. 97–108. Springer, Berlin (2009)
49. Kantardzic, M.: Data Mining Concepts, Models, Methods, and Algorithms. Wiley-IEEE Press, Hoboken (2011)
50. Karaboga, D., Ozturk, C., Karaboga, N., Gorkemli, B.: Artificial bee colony programming for symbolic regression. Inf. Sci. **209**, 1–15 (2012)
51. Keijzer, M.: Improving Symbolic Regression with Interval Arithmetic and Linear Scaling, pp. 70–82. Springer, Berlin (2003)
52. Koza, J.R.: Genetic Programming: On the Programming of Computers by Means of Natural Selection. MIT Press, Cambridge (1992)
53. Koza, J.R.: Genetic Programming II: Automatic Discovery of Reusable Programs. MIT Press, Cambridge (1994)
54. Kristjanpoller, W., Minutolo, M.C.: Forecasting volatility of oil price using an artificial neural network-GARCH model. Expert Syst. Appl. **65**, 233–241 (2016)
55. Lalonde, M.: A new perspective on the health of Canadians. Technical report (1994)
56. Lamson, S.T., Hastings, N.A.J., Willis, R.J.: Minimum cost maintenance in heavy haul rail track. J. Oper. Res. Soc. **34**(3), 211 (1983)
57. Lane, F., Azad, R., Ryan, C.: On effective and inexpensive local search techniques in genetic programming regression. In: Parallel Problem Solving from Nature – PPSN XIII, vol. 8672. Lecture Notes in Computer Science. Springer International Publishing, Berlin (2014)
58. Lidén, T.: Railway infrastructure maintenance – a survey of planning problems and conducted research. Trans. Res. Proc. **10**, 574–583 (2015)
59. Life expectancy increasing in Russia, experts claim. https://www.vedomosti.ru/economics/articles/2018/05/29/770996-rosta-prodolzhitelnosti-zhizni
60. Lubitz, J., Cai, L., Kramarow, E., Lentzner, H.: Health, life expectancy, and health care spending among the elderly. N. Engl. J. Med. **349**(11), 1048–1055 (2003). PMID: 12968089
61. Ly, D.L., Lipson, H.: Learning symbolic representations of hybrid dynamical systems. J. Mach. Learn. Res. **13**(Dec), 3585–3618 (2012)
62. Macchi, M., Garetti, M., Centrone, D., Fumagalli, L., Piero Pavirani, G.: Maintenance management of railway infrastructures based on reliability analysis. Reliab. Eng. Syst. Saf. **104**, 71–83 (2012)
63. Mladenović, N., Hansen, P.: Variable neighborhood search. Comput. Oper. Res. **24**(11), 1097–1100 (1997)
64. Mladenović, N., Urošević, D.: Variable Neighborhood Search for the K-Cardinality Tree. Metaheuristics: Computer Decision-Making, Applied Optimization. Springer, Boston (2003)
65. Mladenović, N., Todosijević, R., Urošević, D.: Less is more: basic variable neighborhood search for minimum differential dispersion problem. Inf. Sci. **326**, 160–171 (2016)

66. Mladenović, M., Delot, T., Laporte, G., Wilbaut, C.: The parking allocation problem for connected vehicles. J. Heuristics **26**, 377–399 (2020)
67. Mladenović, N., Alkandari, A., Pei, J., Todosijević, R., Pardalos, P.M.: Less is more approach: basic variable neighborhood search for the obnoxious *p*-median problem. Int. Trans. Oper. Res. **27**(1), 480–493 (2020)
68. Muggleton, S., de Raedt, L.: Inductive logic programming: theory and methods. J. Logic Program. **19–20**, 629–679 (1994)
69. Musilek, P., Lau, A., Reformat, M., Wyardscott, L.: Immune programming. Inf. Sci. **176**(8), 972–1002 (2006)
70. Nguyen, S., Zhang, M., Member, S., Johnston, M., Tan, K.C.: Automatic programming via iterated local search for dynamic job shop scheduling. IEEE Trans. Cybern. **45**(1), 1–14 (2015)
71. O'Neill, M., Brabazon, A.: Grammatical swarm. In: Genetic and Evolutionary Computation Conference (GECCO), pp. 163–174. Springer, Berlin (2004)
72. Pei, J., Mladenović, N., Urošević, D., Brimberg, J., Liu, X.: Solving the traveling repairman problem with profits: a novel variable neighborhood search approach. Inf. Sci. **507**, 108–123 (2020)
73. Peng, F., Kang, S., Li, X., Ouyang, Y., Somani, K., Acharya, D.: A heuristic approach to the railroad track maintenance scheduling problem. Comput. Aided Civ. Inf. Eng. **26**(2), 129–145 (2011)
74. Peng, F., Ouyang, Y., Somani, K.: Optimal routing and scheduling of periodic inspections in large-scale railroad networks. J. Rail Transp. Plann. Manage. **3**(4), 163–171 (2013)
75. Peng, Y., Yuan, C., Qin, X., Huang, J., Shi, Y.: An improved gene expression programming approach for symbolic regression problems. Neurocomputing **137**, 293–301 (2014)
76. Rad, H.I., Feng, J., Iba, H.: GP-RVM: genetic programming-based symbolic regression using relevance vector machine. ArXiv: 1806,02502v (2018)
77. Roux, O., Cyril, F.: Ant programming: or how to use ants for automatic programming. In: International Conference on Swarm Intelligence, pp. 121–129 (2000)
78. Russian statistical yearbook. Rosstat (2018)
79. Shcherbakova, E.: Life expectancy and health care in OECD countries. Technical Report, Demoscope weekly (2018)
80. Stadtmüller, U.: Asymptotic properties of nonparametric curve estimates. Period. Math. Hung. **17**(2), 83–108 (1986)
81. Uy, N.Q., Hoai, N.X., O'Neill, M., McKay, R.I., Galván-López, E.: Semantically-based crossover in genetic programming: application to real-valued symbolic regression. Genet. Program Evolvable Mach. **12**(2), 91–119 (2011)
82. Witten, I.H., Frank, E., Hall, M.A.: Data Mining: Practical Machine Learning Tools and Techniques. Morgan Kaufmann, Burlington (2011)
83. Wolpert, D., Macready, W.: No free lunch theorems for optimization. IEEE Trans. Evol. Comput. **1**(1), 67–82 (1997)
84. Wong, P., Zhang, M.: SCHEME: caching subtrees in genetic programming. In: 2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence), pp. 2678–2685. IEEE, Piscataway (2008)
85. World health statistics 2017: monitoring health for the SDGs, Sustainable Development Goals. Technical Report, World Health Organization (2017)
86. Yao, X., Liu, Y., Lin, G.: Evolutionary programming made faster. IEEE Trans. Evol. Comput. **3**(2), 82–102 (1999)
87. Yousefikia, M., Moridpour, S., Setunge, S., Mazloumi, E.: Modeling degradation of tracks for maintenance planning on a tram line. J. Traffic Logist. Eng. **2**(2), 86–91 (2014)
88. Zhao, J., Chan, A.H.C., Stirling, A.B., Madelin, K.B.: Optimizing policies of railway ballast tamping and renewal. Trans. Res. Record J. Trans. Res. Board **1943**(1), 50–56 (2006)
89. Zhao, J., Chan, A.H.C., Burrow, M.P.N.: Reliability analysis and maintenance decision for railway sleepers using track condition information. J. Oper. Res. Soc. **58**(8), 1047–1055 (2007)

Non-lattice Covering and Quantization of High Dimensional Sets



Jack Noonan and Anatoly Zhigljavsky

1 Introduction

The problem of the main importance in this paper is the following problem of covering a cube $[-1,1]^d$ by n balls. Let $Z_1,\dots,Z_n$ be a collection of points in $\mathbb{R}^d$ and $\mathcal{B}_d(Z_j,r)=\{Z:\|Z-Z_j\|\le r\}$ be the Euclidean balls of radius r centered at Z_j $(j=1,\dots,n)$. The dimension d, the number of balls n, and their radius r could be arbitrary.

We are interested in choosing the locations of the centers of the balls $Z_1,\dots,Z_n$ so that the union of the balls $\cup_j\mathcal{B}_d(Z_j,r)$ covers the largest possible proportion of the cube $[-1,1]^d$. More precisely, we are interested in choosing a collection of points (called "design") $\mathbb{Z}_n=\{Z_1,\dots,Z_n\}$ so that

$$C_d(\mathbb{Z}_n,r):=\mathrm{vol}([-1,1]^d\cap\mathcal{B}_d(\mathbb{Z}_n,r))/2^d \tag{1}$$

is as large as possible (given n, r, and the freedom, we are able to use in choosing $Z_1,\dots,Z_n$). Here, $\mathcal{B}_d(\mathbb{Z}_n,r)$ is the union of the balls

$$\mathcal{B}_d(\mathbb{Z}_n,r)=\bigcup_{j=1}^{n}\mathcal{B}_d(Z_j,r), \tag{2}$$

and $C_d(\mathbb{Z}_n,r)$ is the proportion of the cube $[-1,1]^d$ covered by $\mathcal{B}_d(\mathbb{Z}_n,r)$. If $Z_j\in\mathbb{Z}_n$ are random, then we shall consider $\mathbb{E}_{\mathbb{Z}_n}C_d(\mathbb{Z}_n,r)$, the expected value

J. Noonan · A. Zhigljavsky (✉)
School of Mathematics, Cardiff University, Cardiff, UK
e-mail: Noonanj1@cardiff.ac.uk; ZhigljavskyAA@cardiff.ac.uk

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_10

of the proportion (1); for simplicity of notation, we will drop $\mathbb{E}_{\mathbb{Z}_n}$ while referring to $\mathbb{E}_{\mathbb{Z}_n} C_d(\mathbb{Z}_n, r)$.

For a design $\mathbb{Z}_n$, its covering radius is defined by $\mathrm{CR}(\mathbb{Z}_n) = \max_{X \in C_d} \min_{Z_j \in \mathbb{Z}_n} \|X - Z_j\|$. In computer experiments, covering radius is called minimax-distance criterion, see [2] and [9]; in the theory of low-discrepancy sequences, covering radius is called dispersion, see [3, Ch. 6].

The problem of optimal covering of a cube by n balls has very high importance for the theory of global optimization and many branches of numerical mathematics. In particular, the n-point designs $\mathbb{Z}_n$ with smallest CR provide the following: (a) the n-point min–max optimal quadratures, see [10, Ch.3,Th.1.1], (b) min–max n-point global optimization methods in the set of all adaptive n-point optimization strategies, see [10, Ch.4,Th.2.1], and (c) worst-case n-point multi-objective global optimization methods in the set of all adaptive n-point algorithms, see [14]. In all three cases, the class of (objective) functions is the class of Lipschitz functions, where the Lipschitz constant may be unknown. The results (a) and (b) are the celebrated results of A. G. Sukharev obtained in the late nineteen-sixties, see, e.g., [11], and (c) is a recent result of A. Žilinskas.

If d is not small (say, $d > 5$), then computation of the covering radius $\mathrm{CR}(\mathbb{Z}_n)$ for any non-trivial design $\mathbb{Z}_n$ is a very difficult computational problem. This explains why the problem of construction of optimal n-point designs with smallest covering radius is notoriously difficult, see, for example, recent surveys [12, 13]. If $r = \mathrm{CR}(\mathbb{Z}_n)$, then $C_d(\mathbb{Z}_n, r)$ defined in (1) is equal to 1, and the whole cube C_d gets covered by the balls. However, we are only interested in reaching the values like 0.95 or 0.99, when only a large part of the ball is covered.

We will say that $\mathcal{B}_d(\mathbb{Z}_n, r)$ makes a $(1-\gamma)$-covering of $[-1, 1]^d$ if

$$C_d(\mathbb{Z}_n, r) = 1 - \gamma\,; \tag{3}$$

the corresponding value of r will be called $(1-\gamma)$-covering radius and denoted as $r_{1-\gamma}$ or $r_{1-\gamma}(\mathbb{Z}_n)$. If $\gamma = 0$, then the $(1-\gamma)$-covering becomes the full covering and 1-covering radius $r_1(\mathbb{Z}_n)$ becomes the covering radius $\mathrm{CR}(\mathbb{Z}_n)$. The problem of construction of efficient designs with smallest possible $(1-\gamma)$-covering radius (with some small $\gamma > 0$) will be referred to as the problem of weak covering.

Let us give two strong arguments why the problem of weak covering could be even more practically important than the problem of full covering.

- Numerical checking of weak covering (with an approximate value of γ) is straightforward, while numerical checking of the full covering is practically impossible, if d is large enough.
- For a given design $\mathbb{Z}_n$, $C_d(\mathbb{Z}_n, r)$ defined in (1) and considered as a function of r, is a cumulative distribution function (c.d.f.) of the random variable (r.v.) $\varrho(U, \mathbb{Z}_n) = \min_{Z_i \in \mathbb{Z}_n} \|U - Z_i\|$, where U is a random vector uniformly distributed on $[-1, 1]^d$, see (29) below. The covering radius $\mathrm{CR}(\mathbb{Z}_n)$ is the upper bound of this r.v., while in view of (3), $r_{1-\gamma}(\mathbb{Z}_n)$ is the $(1-\gamma)$-quantile. Many practically important characteristics of designs such as quantization error

considered in Sect. 7 are expressed in terms of the whole c.d.f. $C_d(\mathbb{Z}_n, r)$ and their dependence on the upper bound $\mathrm{CR}(\mathbb{Z}_n)$ are marginal. As shown in Sect. 7.5, the numerical studies indicate that comparison of designs on the base of their weak coverage properties is very similar to quantization error comparisons, but this may not be true for comparisons with respect to $\mathrm{CR}(\mathbb{Z}_n)$. This phenomenon is similar to the well-known fact in the theory of space covering by lattices (see an excellent book [1] and surveys [12, 13]), where best lattice coverings of space are often poor quantizers and vice versa. Moreover, Figs. 1 and 2 below show that $\mathrm{CR}(\mathbb{Z}_n)$ may give a totally inadequate impression about the c.d.f. $C_d(\mathbb{Z}_n, r)$ and could be much larger than $r_{1-\gamma}(\mathbb{Z}_n)$ with very small $\gamma > 0$.

In Figs. 1, 2, we consider two simple designs for which we plot their c.d.f. $C_d(\cdot, r)$, black line, and also indicate the location of the r_1=CR and $r_{0.999}$ by vertical red and green lines, respectively. In Fig. 1, we take $d = 10$, $n = 512$ and use a 2^{d-1} design of maximum resolution concentrated at the points[1] $(\pm 1/2, \ldots, \pm 1/2) \in \mathbb{R}^d$ as design $\mathbb{Z}_n$; this design is a particular case of Design 4 of Sect. 8 and can be defined for any $d > 2$. In Fig. 2, we keep $d = 10$ but take the full factorial 2^d design with $m = 2^d$ points, again concentrated at the points $(\pm 1/2, \ldots, \pm 1/2)$; denote this design by $\mathbb{Z}'_m$.

For both designs, it is very easy to analytically compute their covering radii (for any $d > 2$): $\mathrm{CR}(\mathbb{Z}_n) = \sqrt{d+8}/2$ and $\mathrm{CR}(\mathbb{Z}'_m) = \sqrt{d}/2$; for $d = 10$, this gives $\mathrm{CR}(\mathbb{Z}_n) \simeq 2.1213$ and $\mathrm{CR}(\mathbb{Z}'_m) \simeq 1.58114$. The values of $r_{0.999}$ are: $r_{0.999}(\mathbb{Z}_n) \simeq 1.3465$ and $r_{0.999}(\mathbb{Z}'_m) \simeq 1.2708$. Their values have been computed using very accurate approximations developed in [5]; we claim 3 correct decimal places in both values of $r_{0.999}$. We will return to this example in Sect. 2.1.

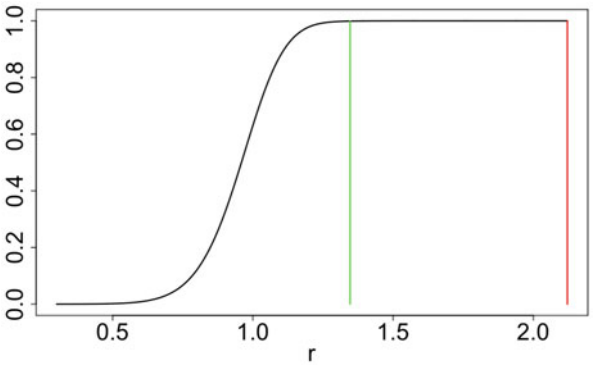


Fig. 1 $C_d(\mathbb{Z}_n, r)$ with $r_{0.999}$ and r_1: $d = 10$, $\mathbb{Z}_n$ is a 2^{d-1}-factorial design with $n = 2^{d-1}$

[1] For simplicity of notation, vectors in $\mathbb{R}^d$ are represented as rows.

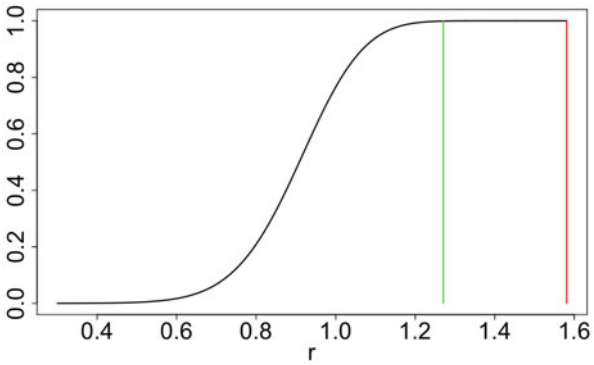


Fig. 2 $C_d(\mathbb{Z}'_m, r)$ with $r_{0.999}$ and r_1: $d = 10$, $\mathbb{Z}_m$ is a 2^d-factorial design

Of course, for any $\mathbb{Z}_n = \{Z_1, \ldots, Z_n\}$, we can reach $C_d(\mathbb{Z}_n, r) = 1$ by means of increase of r. Likewise, for any given r, we can reach $C_d(\mathbb{Z}_n, r) = 1$ by sending $n \to \infty$. However, we are not interested in very large values of n and try to get the coverage of the most part of the cube C_d with the radius r as small as possible. We will keep in mind the following typical values of d and n that we will use for illustrating our results: $d = 5, 10, 20, 50$; $n = 2^k$ with $k = 6, \ldots, 11$ (we have chosen n as a power of 2 since this a favorable number for Sobol's sequence (Design 3) as well as Design 4 defined in Sect. 8).

The structure of the rest of the paper is as follows. In Sect. 2, we discuss the concept of weak covering in more detail and introduce three generic designs that we will concentrate our attention on. In Sects. 3, 4, and 5, we derive approximations for the expected volume of intersection of the cube $[-1, 1]^d$ with n balls centered at the points of these designs. In Sect. 6, we provide numerical results showing that the developed approximations are very accurate. In Sect. 7, we derive approximations for the mean squared quantization error for chosen families of designs and numerically demonstrate that the developed approximations are very accurate. In Sect. 8, we numerically compare covering and quantization properties of different designs including scaled Sobol's sequence and a family of very efficient designs defined only for very specific values of n. In Sect. 9, we try to answer the question raised by Michael Vrahatis by numerically investigating the importance of the effect of scaling points away from the boundary (we call it δ-effect) for covering and quantization in a d-dimensional simplex. In Appendix, Sect. 10, we formulate a simple but important lemma about the distribution and moments of a certain random variable.

Our main theoretical contributions in this paper are:

- derivation of accurate approximations (16) and (22) for the probability $P_{U,\delta,\alpha,r}$ defined in (9);

- derivation of accurate approximations (18), (24), and (27) for the expected volume of intersection of the cube $[-1, 1]^d$ with n balls centered at the points of the selected designs;
- derivation of accurate approximations (32), (34) and (35) for the mean squared quantization error for the selected designs.

We have performed a large-scale numerical study and provided a number of figures and tables. The following are the key messages containing in these figures and tables.

- Figures 1 and 2: Weak covering could be much more practically useful than the full covering.
- Figures 3–14: Developed approximations for the probability $P_{U,\delta,\alpha,r}$ defined in (9) are very accurate.
- Figures 15–28: (a) developed approximations for $C_d(\mathbb{Z}_n, r)$ are very accurate, (b) there is a very strong δ-effect for all three types of designs, and (c) this δ-effect gets stronger as d increases.
- Tables 1 and 2 and Figs. 29 and 30: Smaller values of α are beneficial in Design 1 but Design 2 (where $\alpha = 0$) becomes inefficient when n gets close to 2^d.
- Figures 31–44: Developed approximations for the quantization error are very accurate, and there is a very strong δ-effect for all three types of designs used for quantization.
- Tables 3 and 4 and Figs. 45 and 46: (a) Designs 2a and especially 2b provide very high quality coverage for suitable n, (b) properly δ-tuned deterministic non-nested Design 4 provides superior covering, (c) coverage properties of δ-tuned low-discrepancy sequences are much better than those of the original low-discrepancy sequences, and (d) coverage properties of unadjusted low-discrepancy sequences are very low, if dimension d is not small.
- Tables 5 and 6, Figs. 47 and 48: Very similar conclusions to the above but made with respect to the quantization error.
- Figures 51–62: The δ-effect for covering and quantization schemes in a simplex is definitely present (this effect is more apparent in quantization), but it is much weaker than that in a cube.

2 Weak Covering

In this section, we consider the problem of weak covering defined and discussed in Sect. 1. The main characteristic of interest will be $C_d(\mathbb{Z}_n, r)$, the proportion of the cube covered by the union of balls $\mathcal{B}_d(\mathbb{Z}_n, r)$; it is defined in (1). We start the section with short discussion on comparison of designs based on their covering properties.

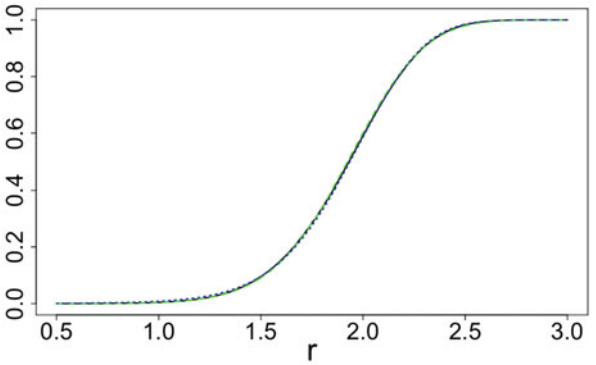


Fig. 3 $P_{U,\delta,\alpha,r}$ and approximations: $d = 10$, $\alpha = 0.5$

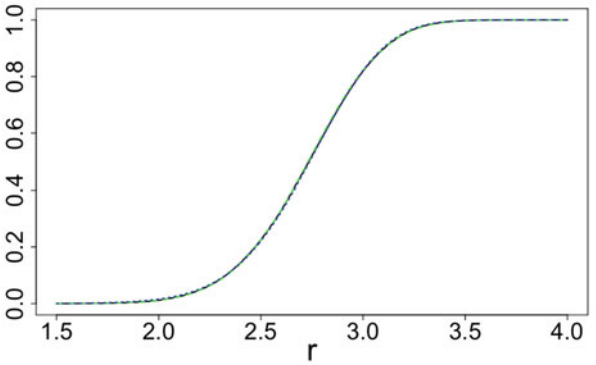


Fig. 4 $P_{U,\delta,\alpha,r}$ and approximations: $d = 20$, $\alpha = 0.5$

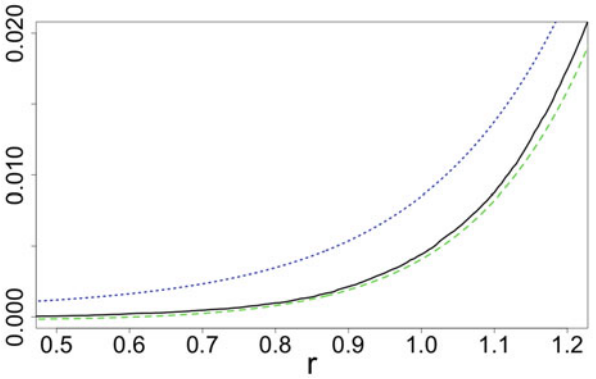


Fig. 5 $P_{U,\delta,\alpha,r}$ and approximations: $d = 10$, $\alpha = 0.5$

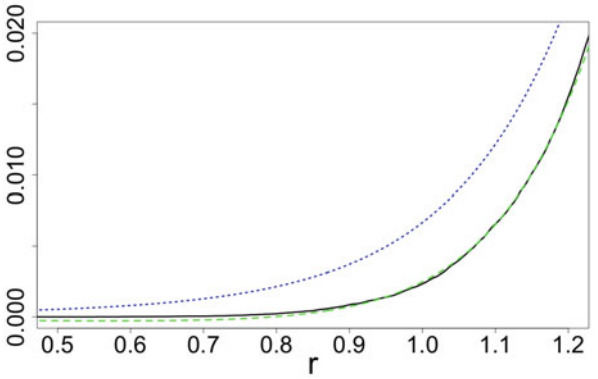


Fig. 6 $P_{U,\delta,\alpha,r}$ and approximations: $d = 10$, $\alpha = 1$

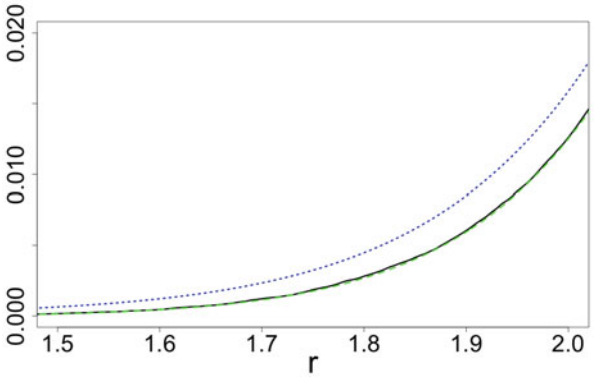


Fig. 7 $P_{U,\delta,\alpha,r}$ and approximations: $d = 20$, $\alpha = 0.5$

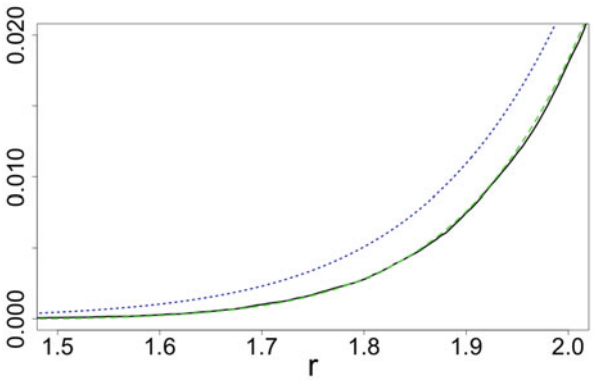


Fig. 8 $P_{U,\delta,\alpha,r}$ and approximations: $d = 20$, $\alpha = 1$

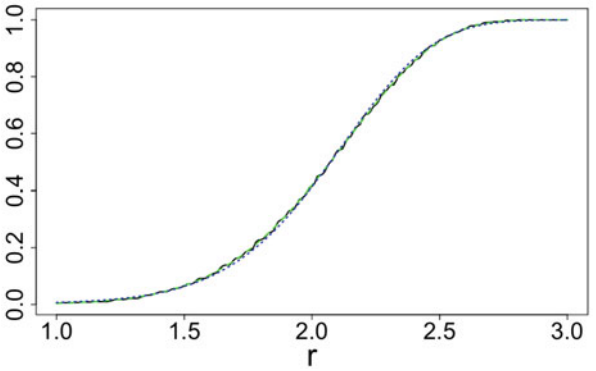


Fig. 9 $P_{U,\delta,0,r}$ and approximations: $d = 10$, $seed = 10$

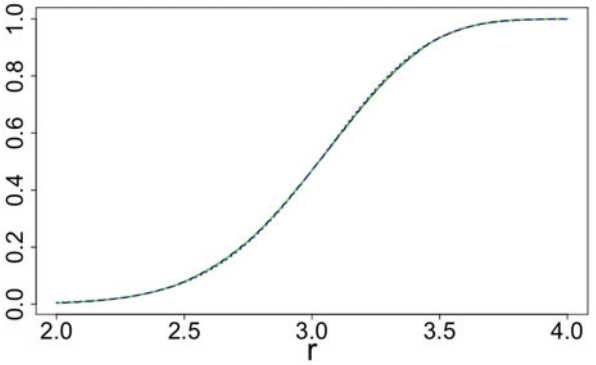


Fig. 10 $P_{U,\delta,0,r}$ and approximations: $d = 20$, $seed = 10$

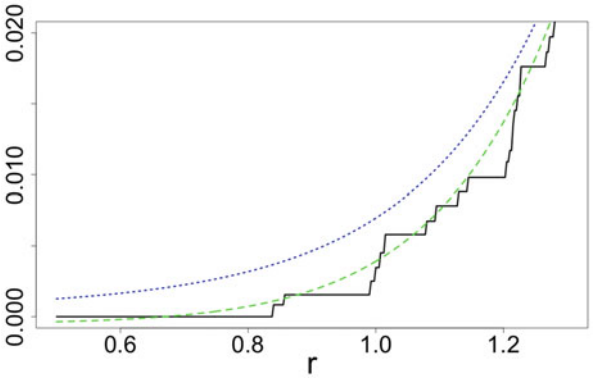


Fig. 11 $P_{U,\delta,0,r}$ and approximations: $d = 10$, $seed = 10$

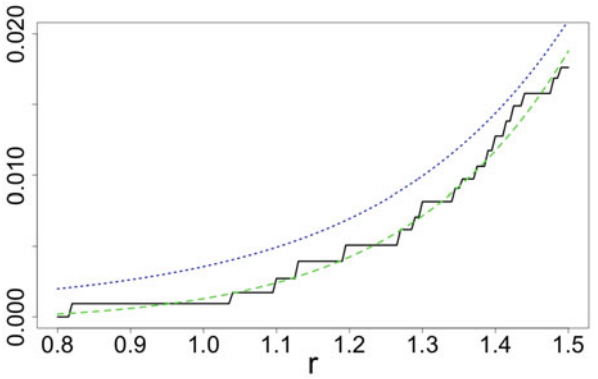


Fig. 12 $P_{U,\delta,0,r}$ and approximations: $d = 10$, $seed = 15$

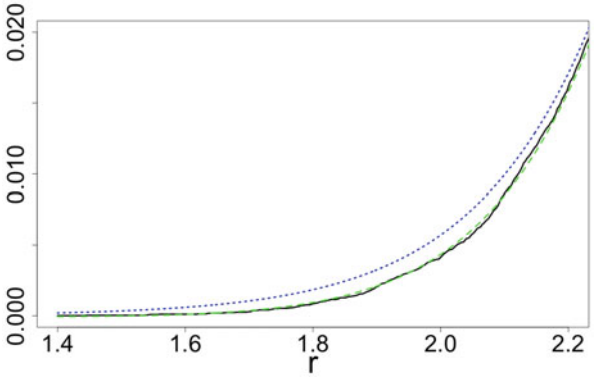


Fig. 13 $P_{U,\delta,0,r}$ and approximations: $d = 20$, $seed = 10$

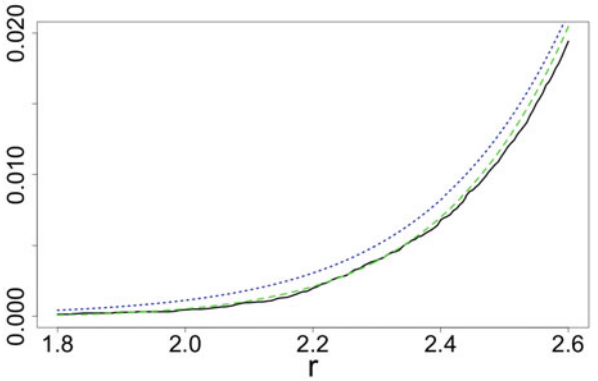


Fig. 14 $P_{U,\delta,0,r}$ and approximations: $d = 20$, $seed = 15$

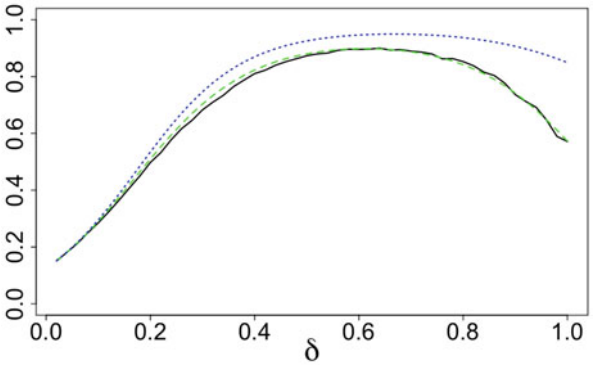


Fig. 15 Design 1: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 10, \alpha = 0.5, n = 128$

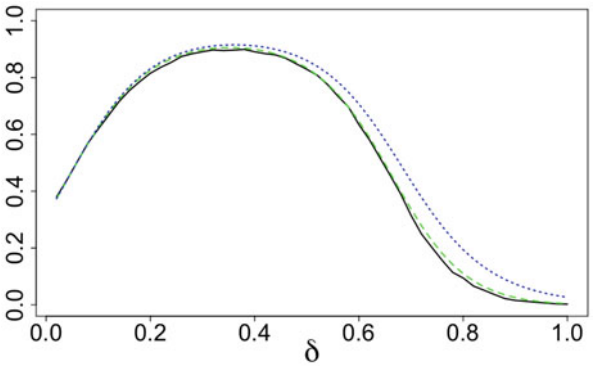


Fig. 16 Design 1: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 20, \alpha = 0.1, n = 128$

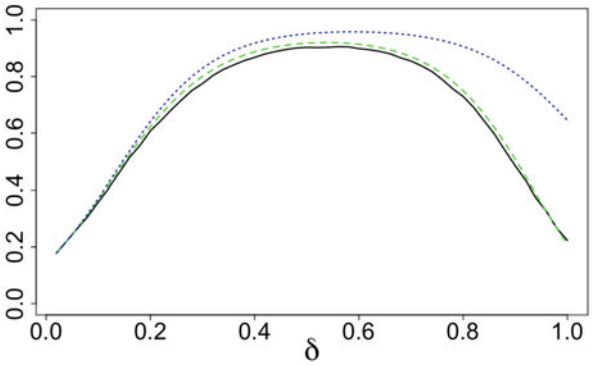


Fig. 17 Design 1: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 20, \alpha = 0.5, n = 512$

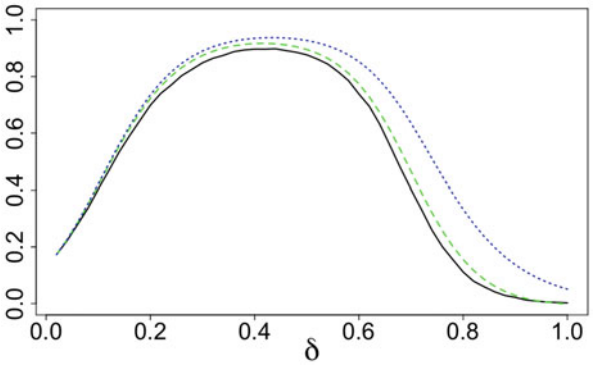


Fig. 18 Design 1: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 20$, $\alpha = 0.1$, $n = 512$

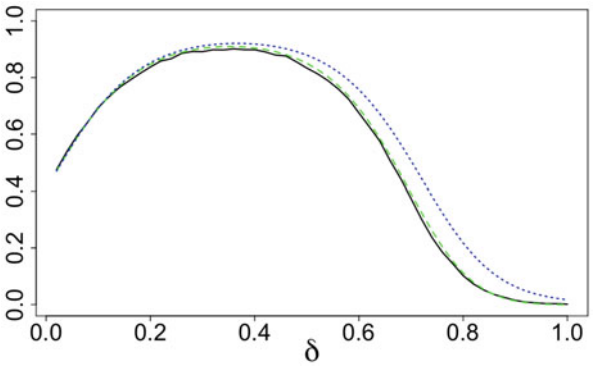


Fig. 19 Design 1: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 50$, $\alpha = 0.5$, $n = 512$

2.1 Comparison of Designs from the View Point of Weak Covering

Two different designs will be differentiated in terms of covering performance as follows. Fix d and let $\mathbb{Z}_n$ and $\mathbb{Z}'_n$ be two n-point designs. For $(1-\gamma)$-covering with $\gamma \geq 0$, if $C_d(\mathbb{Z}_n, r) = C_d(\mathbb{Z}'_n, r') = 1-\gamma$ and $r < r'$, then the design $\mathbb{Z}_n$ provides a more efficient $(1-\gamma)$-covering and is therefore preferable. Moreover, the natural scaling for the radius is $r_n = n^{1/d} r$, and therefore, we can compare an n-point design $\mathbb{Z}_n$ with an m-point design $\mathbb{Z}'_m$ as follows: if $C_d(\mathbb{Z}_n, r) = C_d(\mathbb{Z}'_m, r') = 1-\gamma$ and $n^{1/d} r < m^{1/d} r'$, then we say that the design $\mathbb{Z}_n$ provides a more efficient $(1-\gamma)$-covering than the design $\mathbb{Z}'_m$.

As an example, consider the designs used for plotting Figs. 1 and 2 in Sect. 1: $\mathbb{Z}_n$ with $n = 2^{d-1}$ and $\mathbb{Z}'_m$ with $m = 2^d$. For the full covering, we have for any d:

$$n^{1/d} r_1(\mathbb{Z}_n) = 2^{-1/d} \sqrt{d+8} > \sqrt{d} = r_1(\mathbb{Z}'_m) m^{1/d},$$

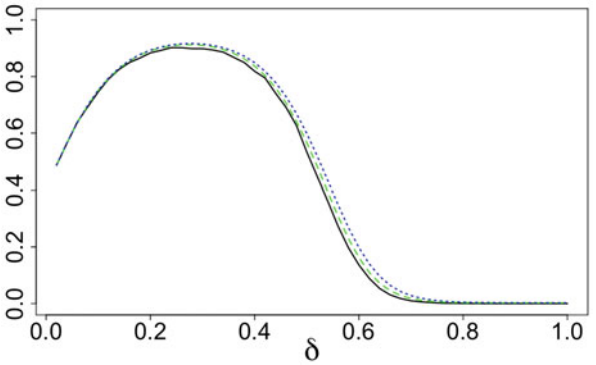


Fig. 20 Design 1: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 50, \alpha = 0.1, n = 512$

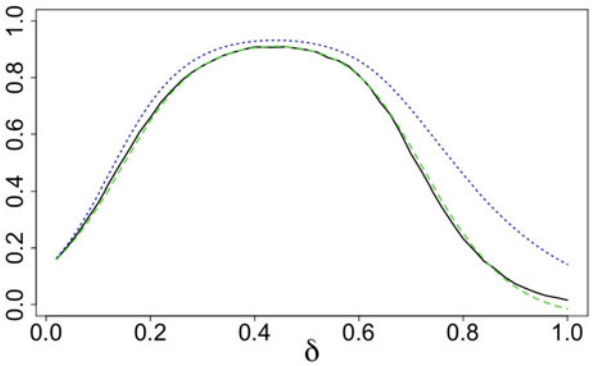


Fig. 21 Design 2a: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 10, \alpha = 0, n = 128$

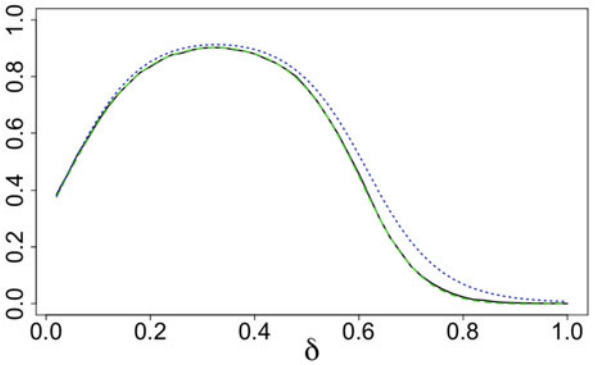


Fig. 22 Design 2a: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 20, \alpha = 0, n = 128$

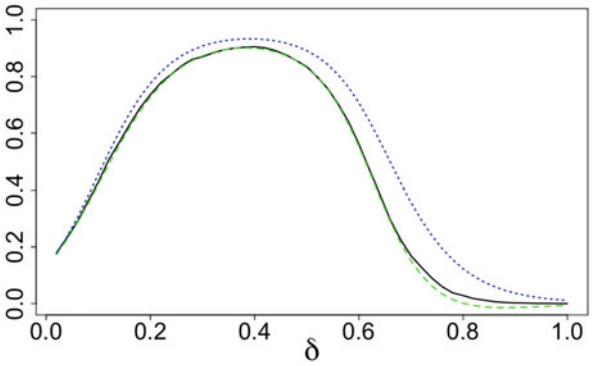


Fig. 23 Design 2a: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 20, \alpha = 0, n = 512$

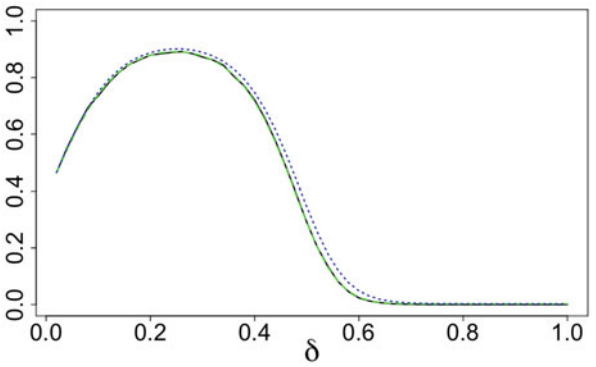


Fig. 24 Design 2a: $C_d(\mathbb{Z}_n, r)$ and approximations; $d = 50, \alpha = 0, n = 512$

so that the design $\mathbb{Z}'_m$ is better than $\mathbb{Z}_n$ for the full covering for any d and the difference between normalized covering radii is quite significant. For example, for $d = 10$, we have

$$n^{1/d} r_1(\mathbb{Z}_n) \simeq 3.9585 \quad \text{and} \quad r_1(\mathbb{Z}'_m) m^{1/d} \simeq 3.1623.$$

For 0.999 covering, however, the situation is reverse, at least for $d = 10$, where we have:

$$n^{1/d} r_{0.999}(\mathbb{Z}_n) \simeq 2.5126 < 2.5416 \simeq r_1(\mathbb{Z}'_m) m^{1/d},$$

and therefore, the design $\mathbb{Z}_n$ is better for 0.999 covering than the design $\mathbb{Z}'_m$ for $d = 10$.

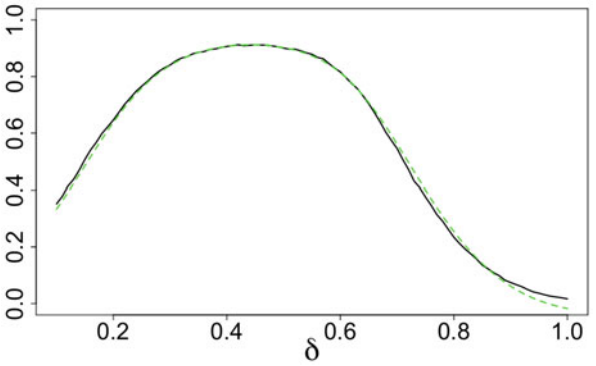


Fig. 25 Design 2b: $C_d(\mathbb{Z}_n, r)$ and approximation (27); $d = 10, n = 128$

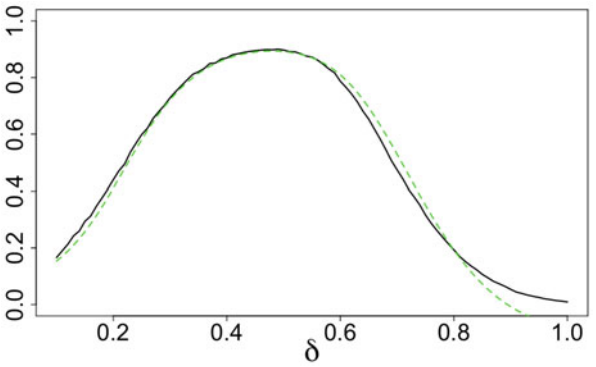


Fig. 26 Design 2b: $C_d(\mathbb{Z}_n, r)$ and approximation (27); $d = 10, n = 256$

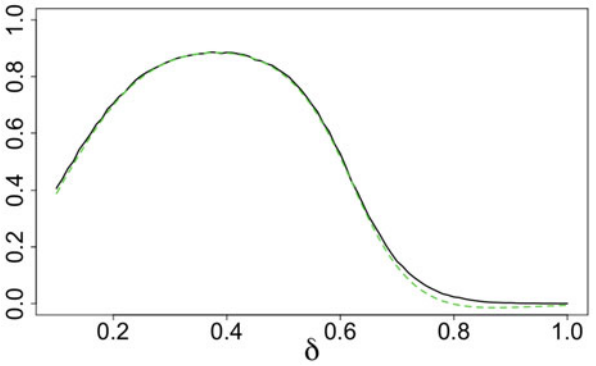


Fig. 27 Design 2b: $C_d(\mathbb{Z}_n, r)$ and approximation (27); $d = 20, n = 512$

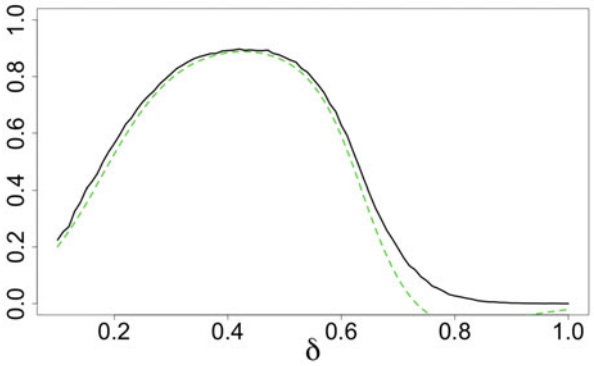


Fig. 28 Design 2b: $C_d(\mathbb{Z}_n, r)$ and approximation (27); $d = 20, n = 2048$

2.2 *Reduction to the Probability of Covering a Point by One Ball*

In the designs $\mathbb{Z}_n$, which are of most interest to us, the points $Z_j \in \mathbb{Z}_n$ are i.i.d. random vectors in $\mathbb{R}^d$ with a specified distribution. Let us show that for these designs, we can reduce computation of $C_d(\mathbb{Z}_n, r)$ to the probability of covering $[-1, 1]^d$ by one ball.

Let $Z_1, \ldots, Z_n$ be i.i.d. random vectors in $\mathbb{R}^d$ and $\mathcal{B}_d(\mathbb{Z}_n, r)$ be as defined in (2). Then, for given $U = (u_1, \ldots, u_d) \in \mathbb{R}^d$,

$$\begin{aligned}\mathbb{P}\{U \in \mathcal{B}_d(\mathbb{Z}_n, r)\} &= 1 - \prod_{j=1}^{n} \mathbb{P}\left\{U \notin \mathcal{B}_d(Z_j, r)\right\} \\ &= 1 - \prod_{j=1}^{n} \left(1 - \mathbb{P}\left\{U \in \mathcal{B}_d(Z_j, r)\right\}\right) \\ &= 1 - \left(1 - \mathbb{P}_Z\{\|U - Z\| \le r\}\right)^n . \end{aligned} \tag{4}$$

$C_d(\mathbb{Z}_n, r)$, defined in (1), is simply

$$C_d(\mathbb{Z}_n, r) = \mathbb{E}_U \mathbb{P}\{U \in \mathcal{B}_d(\mathbb{Z}_n, r)\} , \tag{5}$$

where the expectation is taken with respect to the uniformly distributed $U \in [-1, 1]^d$. For numerical convenience, we shall simplify the expression (4) by using the approximation

$$(1 - t)^n \simeq e^{-nt} , \tag{6}$$

Table 1 Values of r and δ (in brackets) to achieve 0.9 coverage for $d = 5$

$d = 5$				
	$n = 25$	$n = 50$	$n = 100$	$n = 500$
Design 2a ($\alpha = 0$)	1.051 (0.44)	0.885 (0.50)	0.812 (0.50)	0.798 (0.50)
Design 1, $\alpha = 0.5$	1.072 (0.68)	0.905 (0.78)	0.770 (0.78)	0.540 (0.80)
Design 1, $\alpha = 1$	1.072 (0.78)	0.931 (0.86)	0.798 (0.98)	0.555 (1.00)
Design 1, $\alpha = 1.5$	1.091 (0.92)	0.950 (0.96)	0.820 (0.98)	0.589 (1.00)

Table 2 Values of r and δ (in brackets) to achieve 0.9 coverage for $d = 10$

$d = 10$				
	$n = 500$	$n = 1000$	$n = 5000$	$n = 10000$
Design 2a ($\alpha = 0$)	1.228 (0.50)	1.135 (0.50)	1.073 (0.50)	1.071 (0.50)
Design 1, $\alpha = 0.5$	1.271 (0.69)	1.165 (0.73)	0.954 (0.76)	0.886 (0.78)
Design 1, $\alpha = 1$	1.297 (0.87)	1.194 (0.90)	0.992 (0.93)	0.917 (0.95)
Design 1, $\alpha = 1.5$	1.320 (1.00)	1.220 (1.00)	1.032 (1.00)	0.953 (1.00)

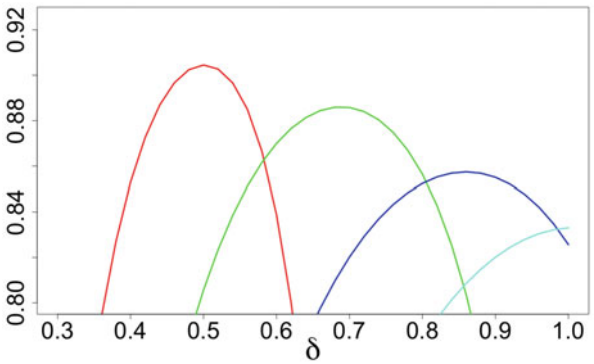


Fig. 29 $d = 10, n = 512, r = 1.228$

where $t = \mathbb{P}_Z\{\|U - Z\| \le r\}$. This approximation is very accurate for small values of t and moderate values of nt, which is always the case of our interest. Combining (4), (5), and (6), we obtain the approximation

$$C_d(\mathbb{Z}_n, r) \simeq 1 - \mathbb{E}_U \exp(-n \cdot \mathbb{P}_Z\{\|U - Z\| \le r\}). \tag{7}$$

In the next section, we will formulate three schemes that will be of theoretical interest in this paper. For each scheme and hence different distribution of Z, we shall derive accurate approximations for $\mathbb{P}_Z\{\|U - Z\| \le r\}$ and therefore, using (7), for $C_d(\mathbb{Z}_n, r)$.

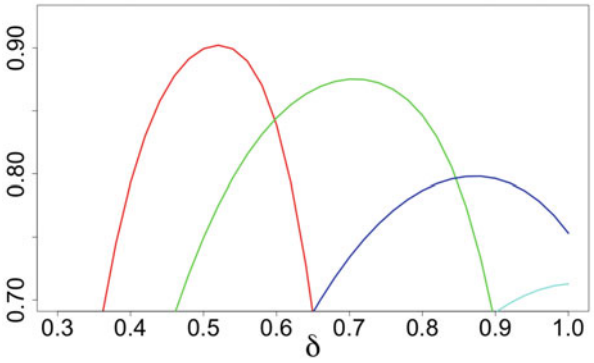


Fig. 30 $d = 10, n = 1024, r = 1.13$

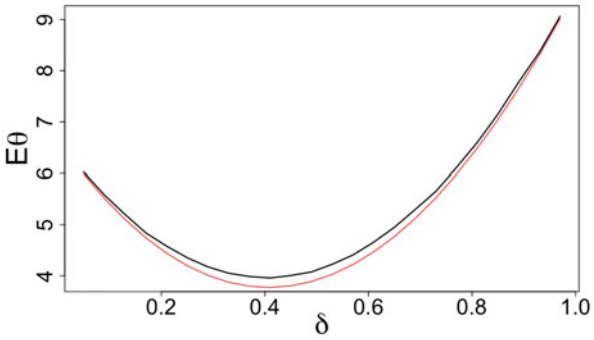


Fig. 31 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (32): $d = 20, \alpha = 1, n = 500$

2.3 Designs of Theoretical Interest

The three designs that will be the focus of theoretical investigation in this paper are as follows.

Design 1 *$Z_1, \ldots, Z_n \in \mathbb{Z}_n$ are i.i.d. random vectors on $[-\delta, \delta]^d$ with independent components distributed according to the following $Beta_\delta(\alpha, \alpha)$ distribution with density:*

$$p_{\alpha,\delta}(t) = \frac{(2\delta)^{1-2\alpha}}{Beta(\alpha, \alpha)}[\delta^2 - t^2]^{\alpha-1}, \quad -\delta < t < \delta, \text{ for some } \alpha > 0 \text{ and } 0 \le \delta \le 1. \tag{8}$$

Design 2a *$Z_1, \ldots, Z_n \in \mathbb{Z}_n$ are i.i.d. random vectors obtained by sampling with replacement from the vertices of the cube $[-\delta, \delta]^d$.*

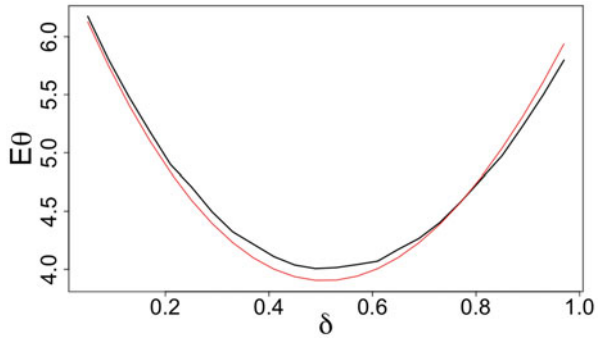


Fig. 32 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (32): $d = 20, \alpha = 0.5, n = 500$

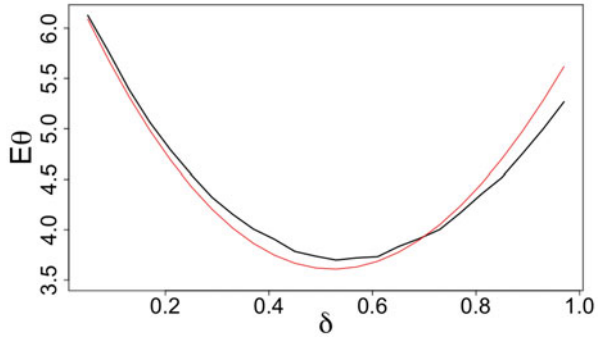


Fig. 33 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (32): $d = 20, \alpha = 0.5, n = 1000$

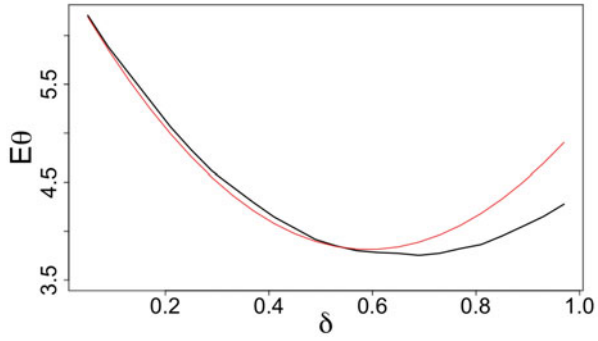


Fig. 34 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (32): $d = 20, \alpha = 1, n = 1000$

Design 2b *$Z_1, \ldots, Z_n \in \mathbb{Z}_n$ are random vectors obtained by sampling without replacement from the vertices of the cube $[-\delta, \delta]^d$.*

All three designs above are nested so that $\mathbb{Z}_n \subset \mathbb{Z}_{n+1}$ for all eligible n. Designs 1 and 2a are defined for all $n = 1, 2, \ldots$, whereas Design 2b is defined for $n =$

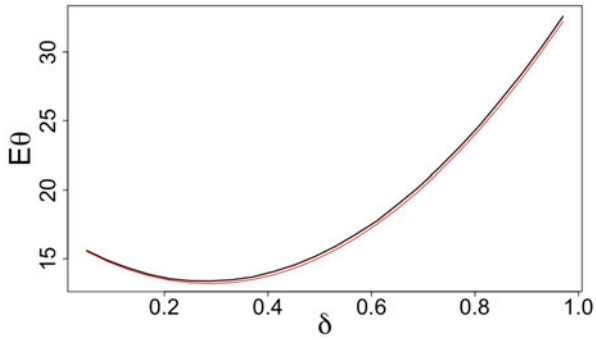


Fig. 35 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (32): $d = 50, \alpha = 0.1, n = 1000$

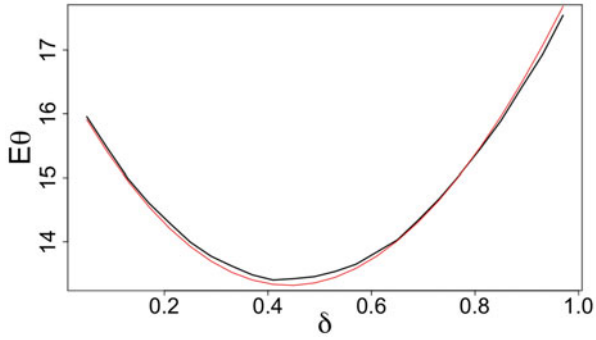


Fig. 36 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (32): $d = 50, \alpha = 1, n = 1000$

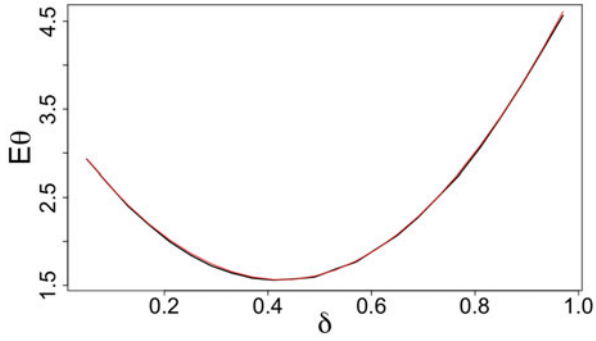


Fig. 37 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (34): $d = 10, \alpha = 0, n = 100$

$1, 2, \ldots, 2^d$. The appealing property of any design whose points Z_i are i.i.d. is the possibility of using (4); this is the case of Designs 1 and 2a. For Design 2b, we will need to make some adjustments, see Sect. 5.

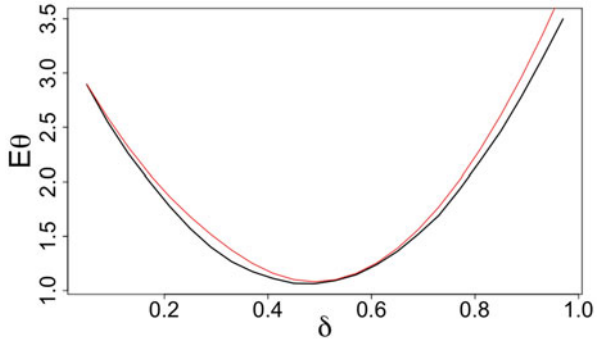


Fig. 38 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (34): $d = 10, \alpha = 0, n = 500$

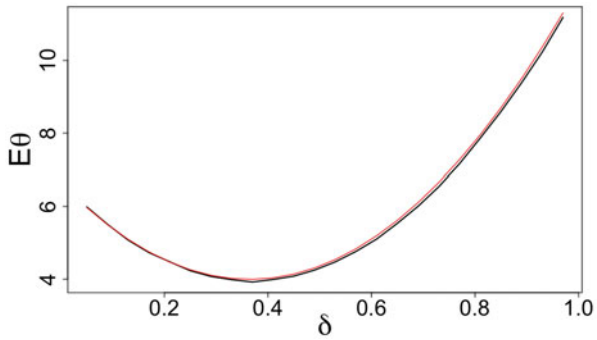


Fig. 39 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (34): $d = 20, \alpha = 0, n = 500$

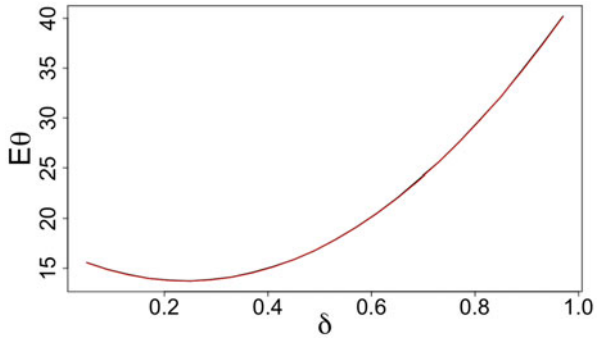


Fig. 40 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (34): $d = 50, \alpha = 0, n = 500$

In the case of $\alpha = 1$ in Design 1, the distribution $\text{Beta}_\delta(\alpha, \alpha)$ becomes uniform on $[-\delta, \delta]^d$. This case has been comprehensively studied in [4] with a number of approximations for $C_d(\mathbb{Z}_n, r)$ being developed. The approximations developed in Sect. 3 are generalizations of the approximations of [4]. Numerical results of

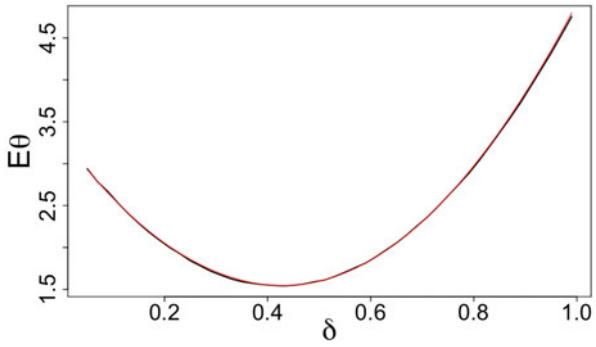


Fig. 41 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (35): $d = 10, n = 100$

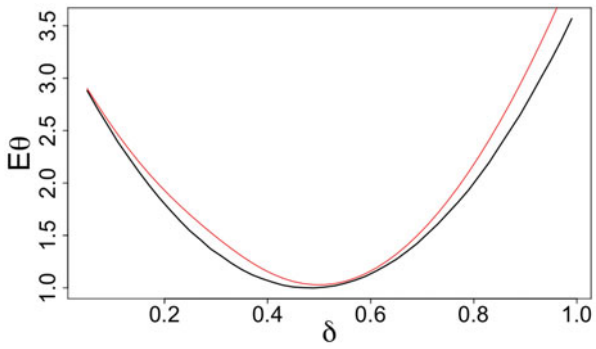


Fig. 42 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (35): $d = 10, n = 500$

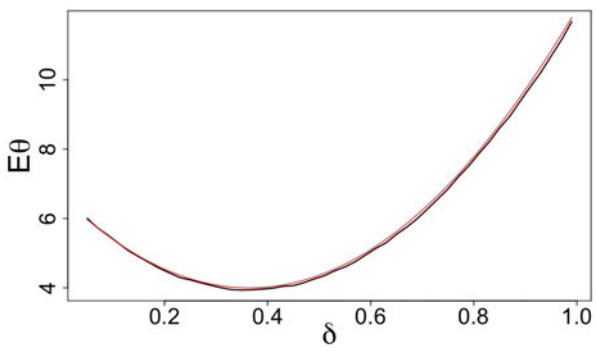


Fig. 43 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (35): $d = 20, n = 500$

[4] indicated that Beta-distribution with $\alpha < 1$ provides more efficient covering schemes; this explains the importance of the approximations of Sect. 3. Design 2a is the limiting form of Design 1 as $\alpha \to 0$. Theoretical approximations developed

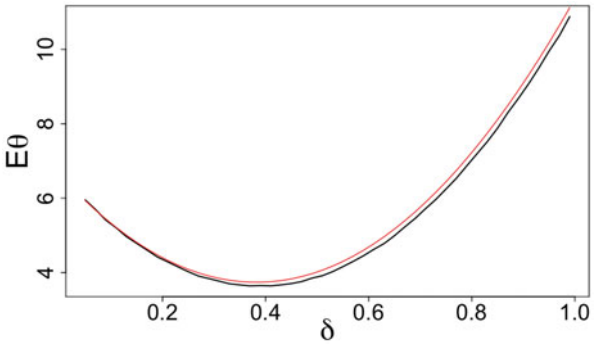


Fig. 44 $\mathbb{E}\theta(\mathbb{Z}_n)$ and approximation (35): $d = 20$, $n = 1000$

Table 3 Values of r and δ (in brackets) to achieve 0.9 coverage for $d = 10$

$d = 10$				
	$n = 64$	$n = 128$	$n = 512$	$n = 1024$
Design 1, $\alpha = 0.5$	1.629 (0.58)	1.505 (0.65)	1.270 (0.72)	1.165 (0.75)
Design 1, $\alpha = 1.5$	1.635 (0.80)	1.525 (0.88)	1.310 (1.00)	1.210 (1.00)
Design 2a	1.610 (0.38)	1.490 (0.46)	1.228 (0.50)	1.132 (0.50)
Design 2b	1.609 (0.41)	1.475 (0.43)	1.178 (0.49)	1.075 (0.50)
Design 3	1.595 (0.72)	1.485 (0.80)	1.280 (0.85)	1.170 (0.88)
Design 3, $\delta = 1$	1.678 (1.00)	1.534 (1.00)	1.305 (1.00)	1.187 (1.00)
Design 4	1.530 (0.44)	1.395 (0.48)	1.115 (0.50)	1.075 (0.50)

Table 4 Values of r and δ (in brackets) to achieve 0.9 coverage for $d = 20$

$d = 20$				
	$n = 64$	$n = 128$	$n = 512$	$n = 1024$
Design 1, $\alpha = 0.5$	2.540 (0.44)	2.455 (0.48)	2.285 (0.55)	2.220 (0.60)
Design 1, $\alpha = 1.5$	2.545 (0.60)	2.460 (0.65)	2.290 (0.76)	2.215 (0.84)
Design 2a	2.538 (0.28)	2.445 (0.30)	2.270 (0.36)	2.180 (0.42)
Design 2b	2.538 (0.29)	2.445 (0.30)	2.253 (0.37)	2.173 (0.42)
Design 3	2.520 (0.50)	2.445 (0.60)	2.285 (0.68)	2.196 (0.72)
Design 3, $\delta = 1$	2.750 (1.00)	2.656 (1.00)	2.435 (1.00)	2.325 (1.00)
Design 4	2.490 (0.32)	2.410 (0.35)	2.220 (0.40)	2.125 (0.44)

below for $C_d(\mathbb{Z}_n, r)$ for Design 2a are, however, more precise than the limiting cases of approximations obtained for $C_d(\mathbb{Z}_n, r)$ in case of Design 1. For numerical comparison, in Sect. 6, we shall also consider several other designs.

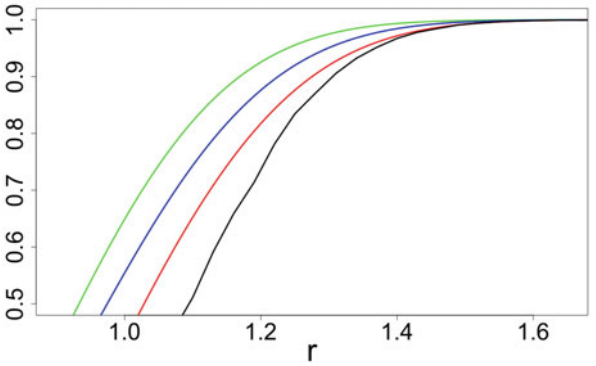


Fig. 45 $C_d(\mathbb{Z}_n, r)$ as a function of r for several designs: $d = 10$, $n = 512$

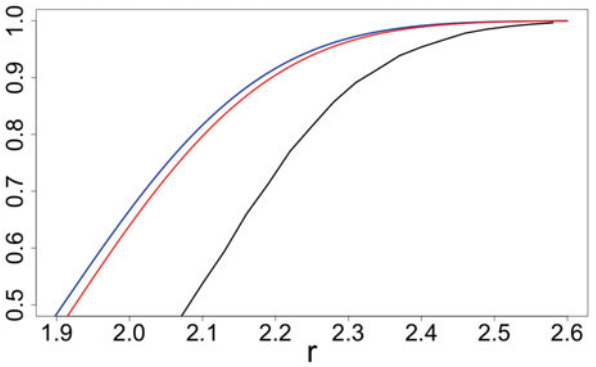


Fig. 46 $C_d(\mathbb{Z}_n, r)$ as a function of r for several designs: $d = 20$, $n = 1024$

Table 5 Minimum value of $n^{2/d}\mathbb{E}\theta_n$ and δ (in brackets) across selected designs; $d = 10$

$d = 10$				
	$n = 64$	$n = 128$	$n = 512$	$n = 1024$
Design 1, $\alpha = 0.5$	4.072 (0.56)	4.013 (0.60)	3.839 (0.68)	3.770 (0.69)
Design 1, $\alpha = 1$	4.153 (0.68)	4.105 (0.72)	3.992 (0.80)	3.925 (0.84)
Design 1, $\alpha = 1.5$	4.164 (0.80)	4.137 (0.86)	4.069 (0.96)	4.026 (0.98)
Design 2a	3.971 (0.38)	3.866 (0.44)	3.670 (0.48)	3.704 (0.50)
Design 2b	3.955 (0.40)	3.798 (0.44)	3.453 (0.48)	3.348 (0.50)
Design 3	3.998 (0.68)	3.973 (0.76)	3.936 (0.80)	3.834 (0.82)
Design 3, $\delta = 1$	4.569 (1.00)	4.425 (1.00)	4.239 (1.00)	4.094 (1.00)
Design 4	3.663 (0.40)	3.548 (0.44)	3.221 (0.48)	3.348 (0.50)

Table 6 Minimum value of $n^{2/d}\mathbb{E}\theta_n$ and δ (in brackets) across selected designs; $d = 20$

$d = 20$				
	$n = 64$	$n = 128$	$n = 512$	$n = 1024$
Design 1, $\alpha = 0.5$	7.541 (0.40)	7.515 (0.44)	7.457 (0.52)	7.421 (0.54)
Design 1, $\alpha = 1$	7.552 (0.52)	7.563 (0.56)	7.528 (0.64)	7.484 (0.68)
Design 1, $\alpha = 1.5$	7.561 (0.60)	7.571 (0.64)	7.556 (0.74)	7.527 (0.78)
Design 2a	7.488 (0.30)	7.461 (0.33)	7.346 (0.35)	7.248 (0.39)
Design 2b	7.487 (0.29)	7.458 (0.34)	7.345 (0.36)	7.234 (0.40)
Design 3	7.445 (0.48)	7.464 (0.56)	7.487 (0.64)	7.453 (0.66)
Design 3, $\delta = 1$	9.089 (1.00)	9.133 (1.00)	8.871 (1.00)	8.681 (1.00)
Design 4	7.298 (0.32)	7.270 (0.33)	7.133 (0.36)	7.016 (0.40)

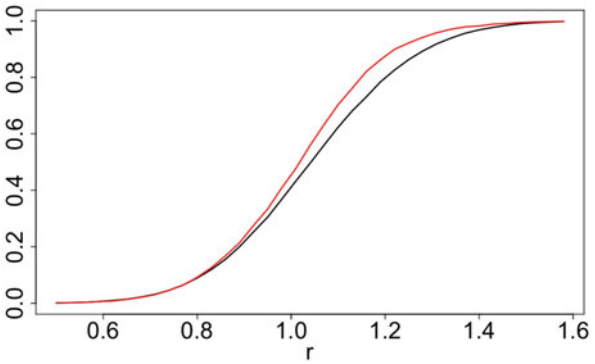


Fig. 47 $d = 10, n = 512$: Design 2a with $\delta = 0.5$ stochastically dominates Design 3 with $\delta = 0.8$

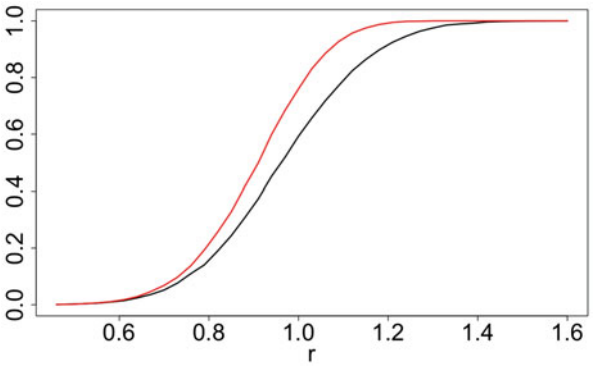


Fig. 48 $d = 10, n = 1024$: Design 2b with $\delta = 0.5$ stochastically dominates Design 3 with $\delta = 0.82$

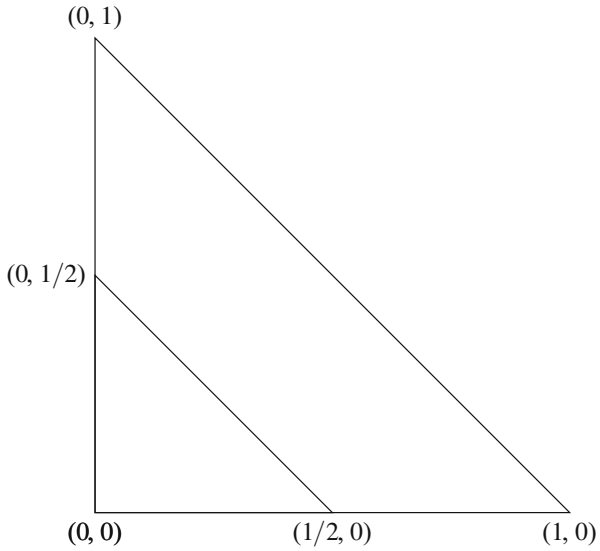


Fig. 49 $\mathcal{S}_d$ and $\mathcal{S}_{d,1}^{(\delta)}$ with $d = 2$ and $\delta = 0.5$

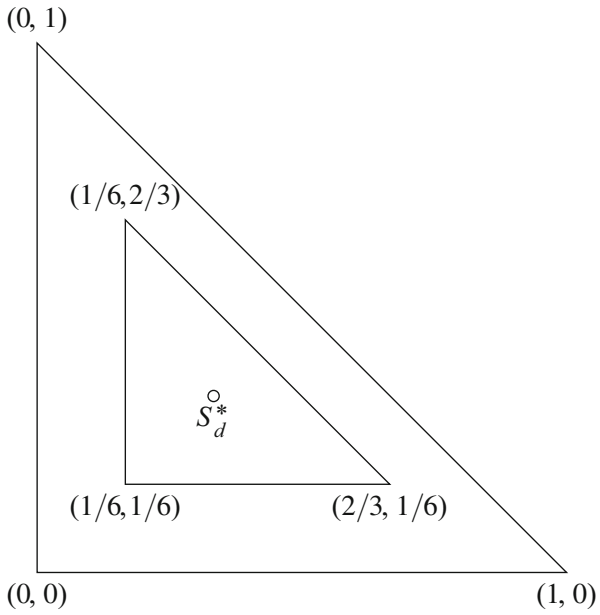


Fig. 50 $\mathcal{S}_d$ and $\mathcal{S}_{d,2}^{(\delta)}$ with $d = 2$ and $\delta = 0.5$

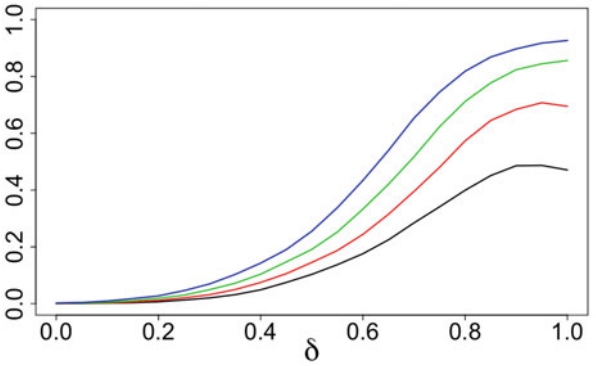


Fig. 51 $C_d(\mathbb{Z}_n, r)$ for Design S1: $d = 5$, $n = 128$, r from 0.11 to 0.17 increasing by 0.02

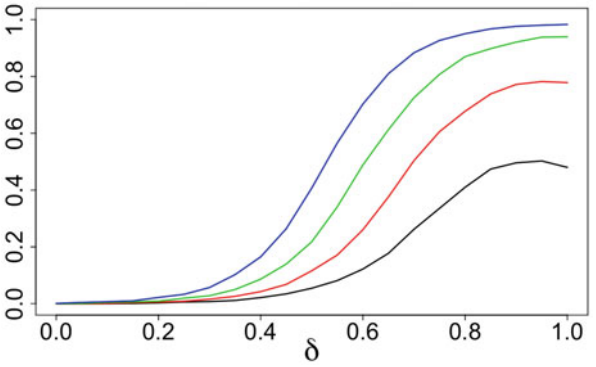


Fig. 52 $C_d(\mathbb{Z}_n, r)$ for Design S1: $d = 10$, $n = 512$, r from 0.13 to 0.19 increasing by 0.02

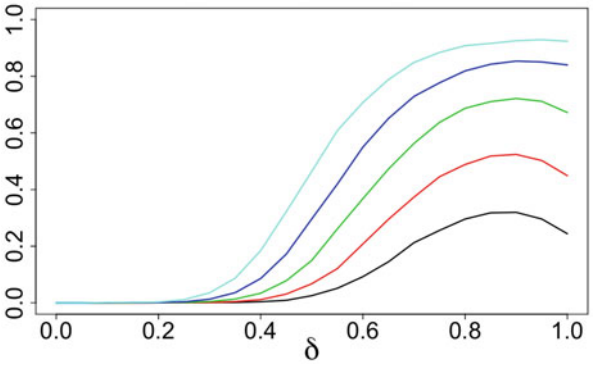


Fig. 53 $C_d(\mathbb{Z}_n, r)$ for Design S1: $d = 20$, $n = 1024$, r from 0.13 to 0.17 increasing by 0.01

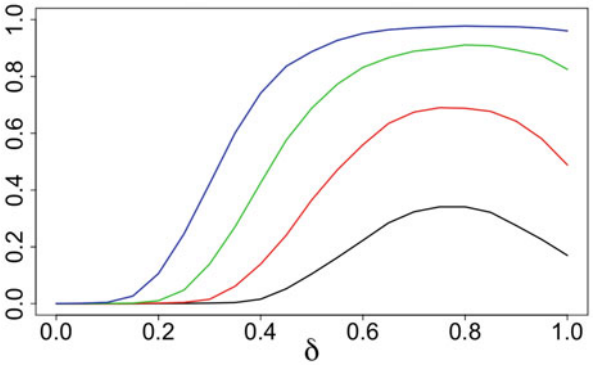


Fig. 54 $C_d(\mathbb{Z}_n, r)$ for Design S1: $d = 50$, $n = 1024$, r from 0.12 to 0.15 increasing by 0.01

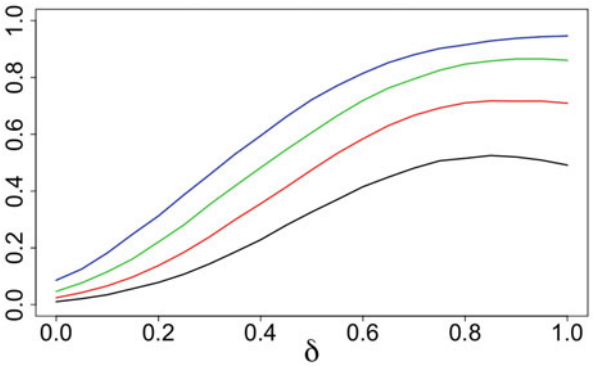


Fig. 55 $C_d(\mathbb{Z}_n, r)$ for Design S2: $d = 5$, $n = 128$, r from 0.11 to 0.17 increasing by 0.02

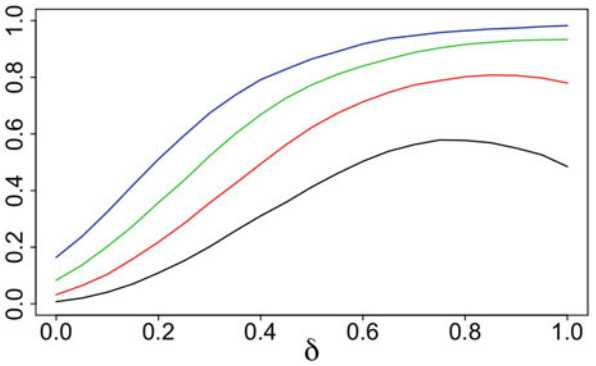


Fig. 56 $C_d(\mathbb{Z}_n, r)$ for Design S2: $d = 10$, $n = 512$, r from 0.13 to 0.19 increasing by 0.02

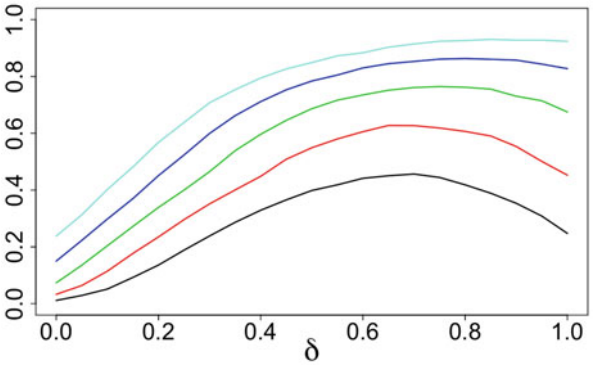


Fig. 57 $C_d(\mathbb{Z}_n, r)$ for Design S2: $d = 20$, $n = 1024$, r from 0.13 to 0.17 increasing by 0.01

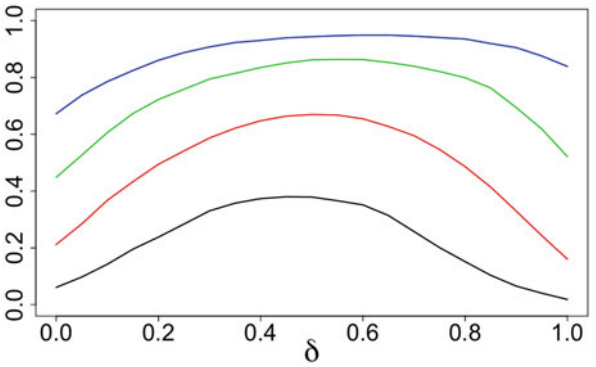


Fig. 58 $C_d(\mathbb{Z}_n, r)$ for Design S2: $d = 50$, $n = 1024$, r from 0.11 to 0.14 increasing by 0.01

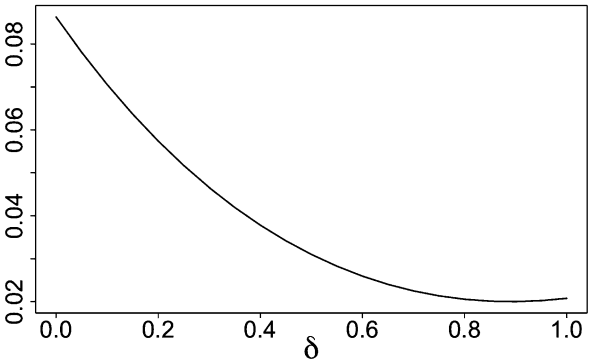


Fig. 59 $\mathbb{E}\theta(\mathbb{Z}_n)$ for Design S1: $d = 20$, $n = 1024$

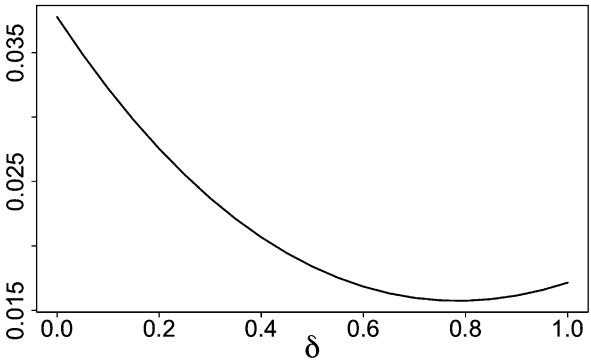


Fig. 60 $\mathbb{E}\theta(\mathbb{Z}_n)$ for Design S1: $d = 50, n = 1024$

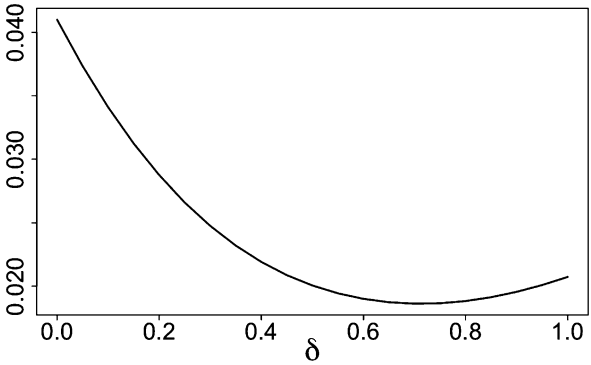


Fig. 61 $\mathbb{E}\theta(\mathbb{Z}_n)$ for Design S2: $d = 20, n = 1024$

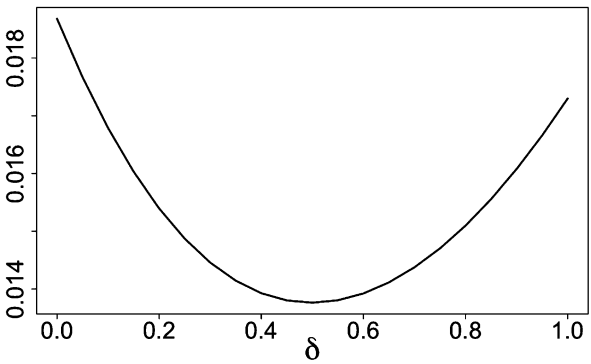


Fig. 62 $\mathbb{E}\theta(\mathbb{Z}_n)$ for Design S2: $d = 50, n = 1024$

3 Approximation of $C_d(\mathbb{Z}_n, r)$ for Design 1

As a result of (7), our main quantity of interest in this section will be the probability

$$P_{U,\delta,\alpha,r} := \mathbb{P}_Z\{\|U-Z\| \le r\} = \mathbb{P}_Z\left\{\|U-Z\|^2 \le r^2\right\} = \mathbb{P}\left\{\sum_{j=1}^{d}(u_j - z_j)^2 \le r^2\right\}, \quad (9)$$

in the case when Z has the Beta-distribution with Density (8). We shall develop a simple approximation based on the Central Limit Theorem (CLT) and then subsequently refine it using the general expansion in the CLT for sums of independent non-identical r.v.

3.1 Normal Approximation for $P_{U,\delta,\alpha,r}$

Let $\eta_{u,\delta,\alpha} = (z-u)^2$, where z has Density (8). In view of Lemma 1, the r.v. $\eta_{u,\delta,\alpha}$ is concentrated on the interval $[(\max(0, \delta - |u|))^2, (\delta + |u|)^2]$, and its first three central moments are

$$\mu_u^{(1)} = \mathbb{E}\eta_{u,\delta,\alpha} = u^2 + \frac{\delta^2}{2\alpha+1}, \quad (10)$$

$$\mu_u^{(2)} = \mathrm{var}(\eta_{u,\delta,\alpha}) = \frac{4\delta^2}{2\alpha+1}\left[u^2 + \frac{\delta^2\alpha}{(2\alpha+1)(2\alpha+3)}\right], \quad (11)$$

$$\mu_u^{(3)} = \mathbb{E}\left[\eta_{u,\delta,\alpha} - \mu_u^{(1)}\right]^3 = \frac{48\alpha\,\delta^4}{(2\alpha+1)^2(2\alpha+3)}\left[u^2 + \frac{\delta^2(2\alpha-1)}{3(2\alpha+5)(2\alpha+1)}\right]. \quad (12)$$

For a given $U = (u_1, \ldots, u_d) \in \mathbb{R}^d$, consider the r.v.

$$\|U - Z\|^2 = \sum_{i=1}^{d}\eta_{u_i,\delta,\alpha} = \sum_{j=1}^{d}(u_j - z_j)^2,$$

where we assume that $Z = (z_1, \ldots, z_d)$ is a random vector with i.i.d. components z_i with Density (8). From (10), its mean is

$$\mu = \mu_{d,\delta,\alpha,U} := \mathbb{E}\|U - Z\|^2 = \|U\|^2 + \frac{d\delta^2}{2\alpha+1}.$$

Using independence of $z_1, \ldots, z_d$ and (11), we obtain

$$\sigma^2_{d,\delta,\alpha,U} := \mathrm{var}(\|U - Z\|^2) = \frac{4\delta^2}{2\alpha+1}\left[\|U\|^2 + \frac{d\delta^2\alpha}{(2\alpha+1)(2\alpha+3)}\right],$$

and from independence of $z_1, \ldots, z_d$ and (12), we get

$$\mu^{(3)}_{d,\delta,\alpha,U} := \mathbb{E}\left[\|U - Z\|^2 - \mu\right]^3 = \sum_{j=1}^{d} \mu^{(3)}_{u_j} = \frac{48\alpha\,\delta^4}{(2\alpha+1)^2(2\alpha+3)}$$
$$\times\left[\|U\|^2 + \frac{d\delta^2(2\alpha-1)}{3(2\alpha+5)(2\alpha+1)}\right]..$$

If d is large enough, then the conditions of the CLT for $\|U - Z\|^2$ are approximately met and the distribution of $\|U - Z\|^2$ is approximately normal with mean $\mu_{d,\delta,\alpha,U}$ and variance $\sigma^2_{d,\delta,\alpha,U}$. That is, we can approximate the probability $P_{U,\delta,\alpha,r} = \mathbb{P}_Z\{\|U-Z\| \le r\}$ by

$$P_{U,\delta,\alpha,r} \cong \Phi\left(\frac{r^2 - \mu_{d,\delta,\alpha,U}}{\sigma_{d,\delta,\alpha,U}}\right), \tag{13}$$

where $\Phi(\cdot)$ is the c.d.f. of the standard normal distribution

$$\Phi(t) = \int_{-\infty}^{t} \varphi(v)dv \quad \text{with} \quad \varphi(v) = \frac{1}{\sqrt{2\pi}} e^{-v^2/2}.$$

The approximation (13) has acceptable accuracy if the probability $P_{U,\delta,\alpha,r}$ is not very small; for example, it falls inside a 2σ-confidence interval generated by the standard normal distribution. In the next section, we improve approximations (13) by using an Edgeworth-type expansion in the CLT for sums of independent non-identically distributed r.v.

3.2 *Refined Approximation for* $P_{U,\delta,\alpha,r}$

General expansion in the Central Limit Theorem for sums of independent non-identical r.v. has been derived by V. Petrov, see Theorem 7 in Chapter 6 in [6], see also Proposition 1.5.7 in [8]. The first three terms of this expansion have been specialized by V.Petrov in Section 5.6 in [7]. By using only the first term in this expansion, we obtain the following approximation for the distribution function of $\|U - Z\|^2$:

$$\mathbb{P}\left(\frac{\|U - Z\|^2 - \mu_{d,\delta,\alpha,U}}{\sigma_{d,\delta,\alpha,U}} \le x\right) \cong \Phi(x) + \frac{\mu^{(3)}_{d,\delta,\alpha,U}}{6\sigma^3_{d,\delta,\alpha,U}}(1 - x^2)\varphi(x), \tag{14}$$

leading to the following improved form of (13):

$$P_{U,\delta,\alpha,r} \cong \Phi(t) + \frac{\alpha\delta\left[\|U\|^2 + \frac{d\delta^2(2\alpha-1)}{3(2\alpha+5)(2\alpha+1)}\right]}{(2\alpha+3)(2\alpha+1)^{1/2}\left[\|U\|^2 + \frac{d\delta^2\alpha}{(2\alpha+1)(2\alpha+3)}\right]^{3/2}}(1-t^2)\varphi(t)\,, \tag{15}$$

where

$$t := \frac{r^2 - \mu_{d,\delta,\alpha,U}}{\sigma_{d,\delta,\alpha,U}} = \frac{\sqrt{2\alpha+1}(r^2 - \|U\|^2 - \frac{d\delta^2}{2\alpha+1})}{2\delta\sqrt{\|U\|^2 + \frac{d\delta^2\alpha}{(2\alpha+1)(2\alpha+3)}}}\,.$$

For $\alpha = 1$, we obtain

$$P_{U,\delta,\alpha,r} \cong \Phi(t) + \frac{\delta\left[\|U\|^2 + d\delta^2/63\right]}{5\sqrt{3}\left[\|U\|^2 + d\delta^2/15\right]^{3/2}}(1-t^2)\varphi(t)$$

$$\text{with} \quad t = \frac{\sqrt{3}(r^2 - \|U\|^2 - d\delta^2/3)}{2\delta\sqrt{\|U\|^2 + d\delta^2/15}}\,,$$

which coincides with formula (16) of [4].

A very attractive feature of the approximations (13) and (15) is their dependence on U through $\|U\|$ only. We could have specialized for our case the next terms in Petrov's approximation, but these terms no longer depend on $\|U\|$ only, and hence, the next terms are much more complicated. Moreover, adding one or two extra terms from Petrov's expansion to the approximation (15) does not fix the problem entirely for all U, δ, α, and r. Instead, we propose a slight adjustment to the r.h.s. of (15) to improve this approximation, especially for small dimensions. Specifically, we suggest the approximation

$$P_{U,\delta,\alpha,r} \cong \Phi(t) + c_{d,\alpha}\frac{\alpha\delta\left[\|U\|^2 + \frac{d\delta^2(2\alpha-1)}{3(2\alpha+5)(2\alpha+1)}\right]}{(2\alpha+3)(2\alpha+1)^{1/2}\left[\|U\|^2 + \frac{d\delta^2\alpha}{(2\alpha+1)(2\alpha+3)}\right]^{3/2}}(1-t^2)\varphi(t)\,, \tag{16}$$

where $c_{d,\alpha} = 1 + 3/(\alpha d)$.

Below, there are figures of two types. In Figs. 3 and 4, we plot $P_{U,\delta,\alpha,r}$ over a wide range of r ensuring that values of $P_{U,\delta,\alpha,r}$ lie in the whole range $[0, 1]$. In Figs. 5–8, we plot $P_{U,\delta,\alpha,r}$ over a much smaller range of r with $P_{U,\delta,\alpha,r}$ lying roughly in the range $[0, 0.02]$. For the purpose of using formula (4), we need to

assess the accuracy of all approximations for smaller values of $P_{U,\delta,\alpha,r}$, and hence, the second type of plots is more useful. In these figures, the solid black line depicts $P_{U,\delta,\alpha,r}$ obtained via Monte Carlo methods where for simplicity we have set $U = (1/2, 1/2, \ldots, 1/2)$ and $\delta = 1/2$. Approximations (13) and (16) are depicted with dotted blue and dash green lines, respectively. From numerous simulations and these figures, we can conclude the following. While the basic normal approximation (13) seems adequate in the whole range of values of r, for particularly small probabilities, which we are most interested in, approximation (16) is much superior and appears to be very accurate for all values of α.

3.3 Approximation for $C_d(\mathbb{Z}_n, r)$ for Design 1

Consider now $C_d(\mathbb{Z}_n, r)$ for Design 1, as expressed via $P_{U,\delta,\alpha,r}$ in (7). As U is uniform on $[-1, 1]^d$, $\mathbb{E}\|U\|^2 = d/3$ and $\mathrm{var}(\|U\|^2) = 4d/45$. Moreover, if d is large enough, then $\|U\|^2 = \sum_{j=1}^{d} u_j^2$ is approximately normal.

We will combine the expressions (7) with approximations (13) and (16) as well as with the normal approximation for the distribution of $\|U\|^2$, to arrive at two final approximations for $C_d(\mathbb{Z}_n, r)$ that differ in complexity. If the original normal approximation (13) of $P_{U,\delta,\alpha,r}$ is used, then we obtain

$$C_d(\mathbb{Z}_n, r) \simeq 1 - \int_{-\infty}^{\infty} \psi_{1,\alpha}(s)\varphi(s)ds \tag{17}$$

with

$$\psi_{1,\alpha}(s) = \exp\{-n\Phi(c_s)\}, \quad c_s = \frac{(2\alpha+1)^{1/2}\left(r^2 - s' - \frac{d\delta^2}{2\alpha+1}\right)}{2\delta\sqrt{s'+\kappa}},$$

$$s' = s\sqrt{\frac{4d}{45}} + d/3, \quad \kappa = \frac{d\delta^2\alpha}{(2\alpha+1)(2\alpha+3)}.$$

If the approximation (16) is used, we obtain

$$C_d(\mathbb{Z}_n, r) \simeq 1 - \int_{-\infty}^{\infty} \psi_{2,\alpha}(s)\varphi(s)ds, \tag{18}$$

with

$$\psi_{2,\alpha}(s) = \exp\left\{-n\left(\Phi(c_s) + c_{d,\alpha}\frac{\alpha\delta\left[s' + \frac{d\delta^2(2\alpha-1)}{3(2\alpha+5)(2\alpha+1)}\right]}{(2\alpha+3)(2\alpha+1)^{1/2}\,[s'+\kappa]^{3/2}}(1-c_s^2)\varphi(c_s)\right)\right\}.$$

For $\alpha = 1$, we get

$$\psi_{2,1}(s) = \exp\left\{-n\left(\Phi(c_s) + c_{d,\alpha}\frac{\delta\left[s' + \frac{d\delta^2}{63}\right]}{5\sqrt{3}\left[s' + \frac{d\delta^2}{15}\right]^{3/2}}(1-c_s^2)\varphi(c_s)\right)\right\}, \quad (19)$$

and the approximation (18) coincides with the approximation (26) in [4]. The accuracy of approximations (17) and (18) will be assessed in Sect. 6.1.

4 Approximating $C_d(\mathbb{Z}_n, r)$ for Design 2a

Our main quantity of interest in this section will be the probability $P_{U,\delta,0,r}$ defined in (9) in the case where components z_i of the vector $Z = (z_1, \ldots, z_d) \in \mathbb{R}^d$ are i.i.d.r.v with $\Pr(z_i = \delta) = \Pr(z_i = -\delta) = 1/2$; this is a limiting case of $P_{U,\delta,\alpha,r}$ as $\alpha \to 0$.

4.1 Normal Approximation for $P_{U,\delta,0,r}$

Using the same approach that led to approximation (13) in Sect. 3.1, the initial normal approximation for $P_{U,\delta,0,r}$ is:

$$P_{U,\delta,0,r} \cong \Phi\left(\frac{r^2 - \mu_{d,\delta,U}}{\sigma_{d,\delta,U}}\right), \quad (20)$$

where, from Lemma 1, we have

$$\mu_{d,\delta,U} = \|U\|^2 + d\delta^2 \quad \text{and} \quad \sigma^2_{d,\delta,U} = 4\delta^2\|U\|^2.$$

4.2 Refined Approximation for $P_{U,\delta,0,r}$

From (38), we have $\mu^{(3)}_{d,\delta,\alpha,U} = 0$ and therefore the last term in the rhs of (14) with $\alpha = 0$ is no longer present. By taking an additional term in the general expansion, see V. Petrov in Section 5.6 in [7], we obtain the following approximation for the distribution function of $\|U - Z\|^2$:

$$\mathbb{P}\left(\frac{\|U-Z\|^2 - \mu_{d,\delta,U}}{\sigma_{d,\delta,U}} \le x\right) \cong \Phi(x) - (x^3 - 3x)\frac{\kappa^{(4)}_{d,\delta,0,U}}{24\sigma^4_{d,\delta,0,U}}\varphi(x), \quad (21)$$

where $\kappa^{(4)}_{d,\delta,0,U}$ is the sum of d fourth cumulants of the centered r.v. $(z-u)^2$, where z is concentrated at two points $\pm\delta$ with $\Pr(z=\pm\delta)=1/2$. From (38),

$$\kappa^{(4)}_{d,\delta,0,U} := \sum_{j=1}^{d}(\mu^{(4)}_{u_j} - 3[\mu^{(2)}_{u_j}]^2) = -32\delta^4 \sum_{i=1}^{d} u_i^4 .$$

Unlike (14), the rhs of (21) does not depends solely on $\|U\|^2$. However, the quantities $\|U\|^2$ and $\sum_{i=1}^{d} u_i^4$ are strongly correlated; one can show that for all d

$$\operatorname{corr}\left(\|U\|^2, \sum_{i=1}^{d} u_i^4\right) = \frac{3\sqrt{5}}{7} \cong 0.958 .$$

This suggests (by rounding the correlation above to 1) the following approximation:

$$\sum_{i=1}^{d} u_i^4 \cong \frac{4\sqrt{d}}{15}\left(\frac{\|U\|^2 - d/3}{\sqrt{\frac{4d}{45}}}\right) + \frac{d}{5} .$$

With this approximation, the r.h.s. of (21) depends only on $\|U\|^2$. As a result, the following refined form of (20) is:

$$P_{U,\delta,0,r} \cong \Phi(t) + (t^3 - 3t)\frac{2(\|U\|^2 - d/3)/\sqrt{5} + d/5}{12\|U\|^4}\varphi(t),$$

where

$$t := \frac{r^2 - \mu_{d,\delta,0,U}}{\sigma_{d,\delta,0,U}} = \frac{(r^2 - \|U\|^2 - d\delta^2)}{2\delta\|U\|} .$$

Similarly to approximation (16), we propose a slight adjustment to the r.h.s. of the approximation above:

$$P_{U,\delta,0,r} \cong \Phi(t) + \left(1 + \frac{3}{d}\right)(t^3 - 3t)\frac{2(\|U\|^2 - d/3)/\sqrt{5} + d/5}{12\|U\|^4}\varphi(t). \quad (22)$$

In the same style as at the end of Sect. 3.2, below there are figures of two types. In Figs. 9 and 10, we plot $P_{U,\delta,0,r}$ over a wide range of r ensuring that values of $P_{U,\delta,0,r}$ lie in the range [0, 1]. In Figs. 11–14, we plot $P_{U,\delta,0,r}$ over a much smaller range of r with $P_{U,\delta,0,r}$ lying in the range [0, 0.02]. In these figures, the solid black line depicts $P_{U,\delta,\alpha,r}$ obtained via Monte Carlo methods where we have set $\delta = 1/2$ and U is a point sampled uniformly on $[-1, 1]^d$; for reproducibility, in the caption of each figure we state the random seed used in R. Approximations (20) and (22)

are depicted with a dotted blue and dash green line respectively. From these figures, we can conclude the same outcome as in Sect. 3.2. Whilst the approximation (20) is rather good overall, for small probabilities the approximation (22) is much superior and is very accurate. Note that since random vectors Z_j are taking values on a finite set, which is the set of points $(\pm\delta, \dots, \pm\delta)$, the probability $P_{U,\delta,0,r}$ considered as a function of r, is a piece-wise constant function.

4.3 Approximation for $C_d(\mathbb{Z}_n, r)$

Consider now $C_d(\mathbb{Z}_n, r)$ for Design 2a, as expressed via $P_{U,\delta,\alpha,r}$ in (7). Using the normal approximation for $\|U\|^2$ as made in the beginning of Sect. 3.3, we will combine the expressions (7) with approximations (20) and (22) to arrive at two approximations for $C_d(\mathbb{Z}_n, r)$ that differ in complexity.

If the original normal approximation (20) of $P_{U,\delta,0,r}$ is used then we obtain:

$$C_d(\mathbb{Z}_n, r) \simeq 1 - \int_{-\infty}^{\infty} \psi_{3,n}(s)\varphi(s)ds, \tag{23}$$

with

$$\psi_{3,n}(s) = \exp\{-n\Phi(c_s)\}\ , \quad c_s = \frac{\left(r^2 - s' - d\delta^2\right)}{2\delta\sqrt{s'}},\ s' = s\sqrt{\frac{4d}{45}} + d/3\ .$$

If the approximation (22) is used, we obtain:

$$C_d(\mathbb{Z}_n, r) \simeq 1 - \int_{-\infty}^{\infty} \psi_{4,n}(s)\varphi(s)ds, \tag{24}$$

with

$$\psi_{4,n}(s) = \exp\left\{-n\left(\Phi(c_s) + \left(1+\frac{3}{d}\right)(c_s^3 - 3c_s)\frac{2(s' - d/3)/\sqrt{5} + d/5}{12(s')^2}\varphi(c_s)\right)\right\}. \tag{25}$$

and

$$c_s = \frac{\left(r^2 - s' - d\delta^2\right)}{2\delta\sqrt{s'}},\ s' = s\sqrt{\frac{4d}{45}} + d/3\ .$$

The accuracy of approximations (23) and (24) will be assessed in Sect. 6.1.

5 Approximating $C_d(\mathbb{Z}_n, r)$ for Design 2b

Designs whose points Z_i have been sampled from a finite discrete set without replacement have dependence, for example, Design 2b, and therefore, formula (4) cannot be used.

In this section, we suggest a way of modifying the approximations developed in Sect. 4 for Design 2a. This will amount to approximating sampling without replacement by a suitable sampling with replacement.

5.1 Establishing a Connection Between Sampling with and Without Replacement: General Case

Let $\mathcal{S}$ be a discrete set with k distinct elements, where k is reasonably large. In case of Design 2b, the set $\mathcal{S}$ consists of $k = 2^d$ vertices of the cube $[-\delta, \delta]^d$. Let $\mathbb{Z}_n = \{Z_1, \ldots, Z_n\}$ denote an n-point design whose points Z_i have been sampled without replacement from $\mathcal{S}$; $n < k$. Also, let $\mathbb{Z}'_m = \{Z'_1, \ldots, Z'_m\}$ denote an associated m-point design whose points Z'_i are sampled with replacement from the same discrete set $\mathcal{S}$; $Z'_1, \ldots, Z'_m$ are i.i.d. random vectors with values in $\mathcal{S}$. Our aim in this section is to establish an approximate correspondence between n and m.

When sampling m times with replacement, denote by X_i the number of times the ith element of $\mathcal{S}$ appears. Then, the vector $(X_1, X_2, \ldots, X_k)$ has the multinomial distribution with number of trials m and event probabilities $(1/k, 1/k, \ldots, 1/k)$ with each individual X_i having the Binomial distribution $Binomial(m, 1/k)$. Since $\text{corr}(X_i, X_j) = -1/k^2$ when $i \neq j$, for large k, the correlation between random variables $X_1, X_2, \ldots, X_k$ is very small and will be neglected. Introduce the random variables

$$Y_i = \begin{cases} 1, & \text{if } X_i = 0 \\ 0, & \text{if } X_i > 0. \end{cases}$$

Then, the random variable $N_0 = \sum_{i=1}^{k} Y_i$ represents the number of elements of $\mathcal{S}$ not selected. Given the weak correlation between X_i, we approximately have $N_0 \sim Binomial(k, P(X_1 = 0))$. Using the fact $P(X_1 = 0) = (1 - 1/k)^m$, the expected number of unselected elements when sampling with replacement is approximately $\mathbb{E}N_0 \cong k(1 - 1/k)^m$. Since, when sampling without replacement from $\mathcal{S}$, we have chosen $N_0 = k - n$ elements, to choose the value of m, we equate $\mathbb{E}N_0$ to $k - n$. By solving the equation

$$k - n = k\left(1 - \frac{1}{k}\right)^m \quad [\cong \mathbb{E}N_0]$$

for m, we obtain

$$m = \frac{\log(k-n) - \log(k)}{\log(k-1) - \log(k)}. \tag{26}$$

5.2 Approximation of $C_d(\mathbb{Z}_n, r)$ for Design 2b.

Consider now $C_d(\mathbb{Z}_n, r)$ for Design 2b. By applying the approximation developed in the previous section, the quantity $C_d(\mathbb{Z}_n, r)$ can be approximated by $C_d(\mathbb{Z}_m, r)$ for Design 2a with m given in (26).

Approximation of $C_d(\mathbb{Z}_n, r)$ for Design 2b *We approximate it by $C_d(\mathbb{Z}_m, r)$, where m is given in (26), and $C_d(\mathbb{Z}_m, r)$ is approximated by (24) with n substituted by m from (26).*

Specifying this, we obtain

$$C_d(\mathbb{Z}_n, r) \simeq 1 - \int_{-\infty}^{\infty} \psi_{4,m}(s)\varphi(s)ds, \tag{27}$$

where

$$m = m_{n,d} = \frac{\log(2^d - n) - d\log(2)}{\log(2^d - 1) - d\log(2)}, \tag{28}$$

and the function $\psi_{4,\cdot}(\cdot)$ is defined in (25). The accuracy of the approximation (27) will be assessed in Sect. 6.1.

6 Numerical Study

6.1 Assessing Accuracy of Approximations of $C_d(\mathbb{Z}_n, r)$ and Studying Their Dependence on δ

In this section, we present the results of a large-scale numerical study assessing the accuracy of approximations (17), (18), (23), (24), and (27). In Figs. 15–28, by using a solid black line, we depict $C_d(\mathbb{Z}_n, r)$ obtained by Monte Carlo methods, where the value of r has been chosen such that the maximum coverage across δ is approximately 0.9. In Figs. 15–20, dealing with Design 1, approximations (17) and (18) are depicted with dotted blue and dashed green lines, respectively. In Figs. 21–24 (Design 2a), approximations (23) and (24) are illustrated with dotted blue and dashed green lines, respectively. In Figs. 25–28 (Design 2b) the dashed green line depicts approximation (27). From these figures, we can draw the following conclusions.

- Approximations (18) and (24) are very accurate across all values of δ and α. This is particularly evident for $d = 20, 50$.
- Approximations (17) and (23) are accurate only for very large values of d, like $d = 50$.
- Approximation (24) is generally accurate. For δ close to one (for such values of δ, the covering is very poor) and n close to 2^d, this approximation begins to worsen, see Figs. 26 and 28.
- A sensible choice of δ can dramatically increase the coverage proportion $C_d(\mathbb{Z}_n, r)$. This effect, which we call "δ-effect," is evident in all figures and is very important. It gets much stronger as d increases.

6.2 Comparison Across α

In Table 1, for Design 2a and Design 1 with $\alpha = 0.5, 1, 1.5$, we present the smallest values of r required to achieve the 0.9 coverage on average. For these schemes, the value inside the brackets shows the average value of δ required to obtain this 0.9 coverage. Design 2b is not used as d is too small (for this design, we must have $n < 2^d$, and in these cases, Design 2b provides better coverings than the other designs considered).

From Tables 1 and 2, we can make the following conclusions:

- For small n ($n < 2^d$ or $n \simeq 2^d$), Design 2a provides a more efficient covering than other three other schemes, and hence, smaller values of α are better.
- For $n > 2^d$, Design 2a begins to become impractical since a large proportion of points duplicate. This is reflected in Table 1 by comparing $n = 100$ and $n = 500$ for Design 2a; there is only a small reduction in r despite a large increase in n. Moreover, for values of $n >> 2^d$, Design 2a provides a very inefficient covering.
- For $n >> 2^d$, from looking at Design 1 with $\alpha = 0.5$ and $n = 500$, it would appear beneficial to choose $\alpha \in (0, 1)$ rather than $\alpha > 1$ or $\alpha = 0$.

Using approximations (18) and (24), in Figs. 29 and 30, we depict $C_d(\mathbb{Z}_n, r)$ across δ for different choices of α. In Figs. 29 and 30, the red line, green line, blue line, and cyan line depict approximation (24) ($\alpha = 0$) and approximation (18) with $\alpha = 0.5$, $\alpha = 1$, and $\alpha = 1.5$, respectively. These figures demonstrate the clear benefit of choosing a smaller α, at least for these values of n and d.

7 Quantization in a Cube

7.1 Quantization Error and Its Relation to Weak Covering

In this section, we will study the following characteristic of a design $\mathbb{Z}_n$.

Quantization Error Let $U = (u_1, \dots, u_d)$ be uniform random vector on $[-1, 1]^d$. The mean squared quantization error for a design $\mathbb{Z}_n = \{Z_1, \dots, Z_n\} \subset \mathbb{R}^d$ is defined by

$$\theta(\mathbb{Z}_n) = \mathbb{E}_U \varrho^2(U, \mathbb{Z}_n)\,, \quad \text{where} \quad \varrho(U, \mathbb{Z}_n) = \min_{Z_i \in \mathbb{Z}_n} \|U - Z_i\|\,. \tag{29}$$

If the design $\mathbb{Z}_n$ is randomized, then we consider the expected value $\mathbb{E}_{\mathbb{Z}_n}\theta(\mathbb{Z}_n)$ of $\theta(\mathbb{Z}_n)$ as the main characteristic without stressing this.

The mean squared quantization error $\theta(\mathbb{Z}_n)$ is related to our main quantity $C_d(\mathbb{Z}_n, r)$ defined in (1): indeed, $C_d(\mathbb{Z}_n, r)$, as a function of $r \geq 0$, is the c.d.f. of the r.v. $\varrho(U, \mathbb{Z}_n)$, while $\theta(\mathbb{Z}_n)$ is the second moment of the distribution with this c.d.f.

$$\theta(\mathbb{Z}_n) = \int_{r \geq 0} r^2 dC_d(\mathbb{Z}_n, r)\,. \tag{30}$$

This relation will allow us to use the approximations derived above for $C_d(\mathbb{Z}_n, r)$ in order to construct approximations for the quantization error $\theta(\mathbb{Z}_n)$.

7.2 Quantization Error for Design 1

Using approximation (18) for the quantity $C_d(\mathbb{Z}_n, r)$, we obtain

$$\begin{aligned}\frac{d}{dr}(C_d(\mathbb{Z}_n, r)) \cong f_{\alpha,\delta}(r) := \frac{n \cdot r}{\delta} \int_{-\infty}^{\infty} \varphi(s)\varphi(c_s)\psi_{2,\alpha}(s) \\ \times \left[\frac{\sqrt{2\alpha+1}}{\sqrt{s'+k}} + c_{d,\alpha} \frac{\alpha\left(s' + \frac{d\delta^2(2\alpha-1)}{3(2\alpha+5)(2\alpha+1)}\right)}{(2\alpha+3)\,(s'+k)^2} \right. \\ \left. \times \left\{ \delta(c_s^3 - c_s) - \frac{\sqrt{2\alpha+1}(r^2 - \frac{d\delta^2}{2\alpha+1} - s')}{\sqrt{s'+k}} \right\} \right] ds\,.\end{aligned} \tag{31}$$

By then using relation (30), we obtain the following approximation for the mean squared quantization error with Design 1:

$$\theta(\mathbb{Z}_n) \cong \int_0^{\infty} r^2 f_{\alpha,\delta}(r) dr\,. \tag{32}$$

By taking $\alpha = 1$ in (31), we obtain

$$f_{1,\delta}(r) := \frac{n \cdot r}{\delta} \int_{-\infty}^{\infty} \varphi(s)\varphi(c_s)\psi_{2,1}(s) \left[\frac{\sqrt{3}}{\sqrt{s'+k}} + c_{d,1} \frac{\left(s' + \frac{d\delta^2}{63}\right)}{5\,(s'+k)^2} \right.$$
$$\left. \times \left\{ \delta(c_s^3 - c_s) - \frac{\sqrt{3}(r^2 - \frac{d\delta^2}{3} - s')}{\sqrt{s'+k}} \right\} \right] ds\,.$$

with $\psi_{2,1}$ defined in (19). The resulting approximation

$$\theta(\mathbb{Z}_n) \cong \int_0^{\infty} r^2 f_{1,\delta}(r) dr$$

coincides with [4, formula 31].

7.3 Quantization Error for Design 2a

Using approximation (24) for the quantity $C_d(\mathbb{Z}_n, r)$, we have

$$\frac{d}{dr}(C_d(\mathbb{Z}_n, r)) \cong f_{0,\delta;n}(r) := \frac{n \cdot r}{\delta} \int_{-\infty}^{\infty} \frac{\varphi(s)\varphi(c_s)\psi_{4,n}(s)}{\sqrt{s'}}$$
$$\times \left[1 + \left(1 + \frac{3}{d}\right) \frac{(2(s' - d/3)/\sqrt{5} + d/5)(6c_s^2 - c_s^4 - 3)}{12(s')^2} \right] ds, \;, \tag{33}$$

where $\psi_{4,n}(\cdot)$ is defined in (25). From (30), we then obtain the following approximation for the mean squared quantization error with Design 2a:

$$\theta(\mathbb{Z}_n) \cong \int_0^{\infty} r^2 f_{0,\delta;n}(r) dr. \tag{34}$$

7.4 Quantization Error for Design 2b

Similarly to (34), for Design 2b, we use the approximation

$$\theta(\mathbb{Z}_n) \cong \int_0^{\infty} r^2 f_{0,\delta;m}(r) dr\,. \tag{35}$$

where $f_{0,\delta,m}(r)$ is defined by (33) and $m = m_{n,d}$ is defined in (28).

7.5 Accuracy of Approximations for Quantization Error and the δ-Effect

In this section, we assess the accuracy of approximations (32), (34), and (35). Using a black line, we depict $\mathbb{E}_{\mathbb{Z}_n}\theta(\mathbb{Z}_n)$ obtained via Monte Carlo simulations. Depending on the value of α, in Figs. 31–36, approximation (32) or (34) is shown using a red line. In Figs. 41–44, approximation (35) is depicted with a red line. From the figures below, we can see that all approximations are generally very accurate. Approximation (34) is much more accurate than approximation (32) across all choices of δ and n, and this can be explained by the additional term taken in the general expansion; see Sect. 4.2. This high accuracy is also seen with approximation (35). The accuracy of approximation (32) seems to worsen for large δ, n, and d not too large like $d = 20$, see Figs. 33 and 34. For $d = 50$, all approximations are extremely accurate for all choices of δ and n. Figs. 31–36 very clearly demonstrate the δ-effect implying that a sensible choice of δ is crucial for good quantization.

8 Comparative Numerical Studies of Covering Properties for Several Designs

Let us extend the range of designs considered above by adding the following two designs.

Design 3 *$Z_1, \dots, Z_n$ are taken from a low-discrepancy Sobol's sequence on the cube $[-\delta, \delta]^d$.*

Design 4 *$Z_1, \dots, Z_n$ are taken from the minimum-aberration 2^{d-k} fractional factorial design on the vertices of the cube $[-\delta, \delta]^d$.*

Unlike Designs 1, 2a, 2b, and 3, Design 4 is non-adaptive and defined only for a particular n of the form $n = 2^{d-k}$ with some $k \geq 0$. We have included this design into the list of all designs as "the golden standard." In view of the numerical study in [4] and theoretical arguments in [5], Design 4 with $k = 1$ and optimal δ provides the best quantization we were able to find; moreover, we have conjectured in [5] that Design 4 with $k = 1$ and optimal δ provides minimal normalized mean squared quantization error for all designs with $n \leq 2^d$. We repeat that Design 4 is defined for one particular value of n only.

8.1 Covering Comparisons

In Tables 3 and 4, we present results of Monte Carlo simulations where we have computed the smallest values of r required to achieve the 0.9 coverage on average

(on average, for Designs 1, 2a, 2b). The value inside the brackets shows the value of δ required to obtain the 0.9 coverage.

From Tables 3 and 4, we draw the following conclusions:

- Designs 2a and especially 2b provide very high quality coverage (on average) while being online procedures (i.e., nested designs);
- Design 2b has significant benefits over Design 2a for values of n close to 2^d.
- Properly δ-tuned deterministic non-nested Design 4 provides superior covering.
- Coverage properties of δ-tuned low-discrepancy sequences are much better than those of the original low-discrepancy sequences.
- Coverage of an unadjusted low-discrepancy sequence is poor.

In Figs. 45 and 46, after fixing n and δ, we plot $C_d(\mathbb{Z}_n, r)$ as a function of r for the following designs: Design 1 with $\alpha = 1$ (red line), Design 2a (blue line), Design 2b (green line), and Design 3 with $\delta = 1$ (black line). For Design 1 with $\alpha = 1$, Design 2a, and Design 2b, we have used approximations (19), (25), and (27), respectively, to depict $C_d(\mathbb{Z}_n, r)$, whereas for Design 3, we have used Monte Carlo simulations. For the first three designs, depending of the choice of n, the value of δ has been fixed based on the optimal value for quantization; these are the values inside the brackets in Tables 5 and 6.

From Fig. 45, we see that Design 2b is superior and uniformly dominates all other designs for this choice of d and n (at least when the level of coverage is greater than 1/2). In Fig. 46, since $n << 2^d$, the values of $C_d(\mathbb{Z}_n, r)$ for Designs 2a and 2b practically coincide and the green line hides under the blue. In both figures, we see that Design 3 with an unadjusted δ provides a very inefficient covering.

8.2 *Quantization Comparisons*

As follows from the results of [3, Ch.6], for efficient covering schemes, the order of convergence of the covering radius to 0 as $n \to \infty$ is $n^{-1/d}$. Therefore, for the mean squared distance (which is the quantization error), we should expect the order $n^{-2/d}$ as $n \to \infty$. Therefore, for sake of comparison of quantization errors θ_n across n, we renormalize this error from $\mathbb{E}\theta_n$ to $n^{2/d}\mathbb{E}\theta_n$.

In Table 5 and Table 6, we present the minimum value of $n^{2/d}\mathbb{E}\theta_n$ for a selection of designs. In these tables, the value within the brackets corresponds to the value of δ, where the minimum of $n^{2/d}\mathbb{E}\theta_n$ was obtained.

In Fig. 47, we depict the c.d.f.'s for the distance $\varrho(X, \mathbb{Z}_n)$ for Design 2a with $\delta = 0.5$ (in red) and Design 3 with $\delta = 0.8$ (in black). We can see that for $d = 10$ and $n = 512$, Design 2a stochastically dominates Design 3. The style of Fig. 48 is the same as that of Fig. 47; however, we set $n = 1024$, and Design 2a is replaced with Design 2b with $\delta = 0.5$ (we also set $\delta = 0.82$ for Design 3). Here, we see a very clear stochastic dominance of Design 2b over Design 4. All findings are consistent with Tables 5 and 6. In Figs. 47 and 48, the values of the parameter δ for all designs are chosen as numerically optimal, in accordance with Table 5.

We make the following conclusions from analyzing results of this section.

- Designs 2a and 2b provide very good quantization per point. As expected, Design 2b is superior over Design 2a when n is close to 2^d; see Table 5.
- Properly δ-tuned non-nested Design 4 provides the best quantization per point of all designs considered.
- Properly δ-tuned Design 3 is comparable in performance to Design 1, but it is not as efficient as Designs 2a, 2b, and 4.

9 Covering and Quantization in the d-Simplex

9.1 Characteristics of Interest

Consider the standard orthogonal d-simplex

$$\mathcal{S}_d := \left\{ (u_1, u_2, \dots, u_d) \in \mathbb{R}^d \,\middle|\, \sum_{i=1}^{d} u_i \le 1 \text{ and } u_i \ge 0 \text{ for all } i \right\}$$

with $\mathrm{vol}(\mathcal{S}_d) = 1/d!$. For a design $\mathbb{Z}_n = \{Z_1, \dots, Z_n\}$, consider the following two characteristics:

(a) the proportion of the simplex $\mathcal{S}_d$ covered by $\mathcal{B}_d(\mathbb{Z}_n, r)$:

$$C_d(\mathbb{Z}_n, r) := d!\,\mathrm{vol}(\mathcal{S}_d \cap \mathcal{B}_d(\mathbb{Z}_n, r))\,, \tag{36}$$

(b) $\theta(\mathbb{Z}_n) = \mathbb{E}_U \min_{i=1,\dots,n} \|U - Z_i\|^2$, the mean squared quantization error for $\mathbb{Z}_n$, where $U = (u_1, \dots, u_d)$ is a random vector uniformly distributed in $\mathcal{S}_d$.

In this section, we investigate whether the δ-effect seen in Sects. 6, 7.5, and 8 for the cube is present for the simplex $\mathcal{S}_d$. We will consider two possible ways of scaling points in $\mathcal{S}_d$. Define the two δ-simplices $\mathcal{S}_{d,1}^{(\delta)}$ and $\mathcal{S}_{d,2}^{(\delta)}$ as follows:

$$\mathcal{S}_{d,1}^{(\delta)} := \delta \cdot \mathcal{S}_d\,,$$

$$\mathcal{S}_{d,2}^{(\delta)} := \left\{ (u_1, u_2, \dots, u_d) \in \mathbb{R}^d \,\middle|\, \sum_{i=1}^{d} u_i \le \frac{d+\delta}{d+1} \text{ and } u_i \ge \frac{1-\delta}{d+1} \text{ for all } i \right\}.$$

By construction, the value of δ in $\mathcal{S}_{d,2}^{(\delta)}$ scales the simplex around its centroid $S_d^* = \left(\frac{1}{d+1}, \frac{1}{d+1}, \dots, \frac{1}{d+1}\right)$, where for $\delta = 1$, we have $\mathcal{S}_{d,2}^{(\delta)} = \mathcal{S}_d$. Simple depictions of $\mathcal{S}_{d,1}^{(\delta)}$ and $\mathcal{S}_{d,2}^{(\delta)}$ are given in Figs. 49 and 50.

We will numerically assess covering and quantization characteristics for the following two designs.

Design S1 *$Z_1, \ldots, Z_n$ are i.i.d. random vectors uniformly distributed in the δ-scaled simplex $\mathcal{S}_{d,1}^{(\delta)}$, where $\delta \in [0, 1]$ is a parameter.*

Design S2 *$Z_1, \ldots, Z_n$ are i.i.d. random vectors uniformly distributed in the δ-scaled simplex $\mathcal{S}_{d,2}^{(\delta)}$, where $\delta \in [0, 1]$ is a parameter.*

To simulate points Y uniformly distributed in the simplex $\mathcal{S}_d$, we can simply generate d i.i.d. uniformly distributed points in $[0, 1]$, add 0 and 1 to the collection of points and take the first d spacings (out of the total number $d + 1$ of these spacings). Points $Y' = \delta Y$ and $Y'' = \delta \cdot (Y - S_d^*) + S_d^*$ are then uniform in $\mathcal{S}_{d,1}^{(\delta)}$ and $\mathcal{S}_{d,2}^{(\delta)}$, respectively. This procedure can be easily performed in R using the package "uniformly."

9.2 Numerical Investigation of the δ-Effect for d-Simplex

Using the above procedure, we numerically study characteristics of Designs S1 and S2. In Figs. 51–54, we plot $C_d(\mathbb{Z}_n, r)$ as functions of $\delta \in [0, 1]$ across n, r and d for Design S1. The corresponding results for Design S2 are given in Figs. 55–58. In Figs. 59–62, we depict $\mathbb{E}\theta(\mathbb{Z}_n)$ for Designs S1 and S2, respectively, for different n and d. In each figure, we plot values of $\mathbb{E}\theta(\mathbb{Z}_n)$ for different values of r; a step in r increase gives the next curve up.

From the above figures, we arrive at the following conclusions:

- The δ-effect for the simplex is much less prominent than that for the cube.
- Between Designs S1 and S2, the δ-effect is more apparent for Design S2; for example, compare Fig. 60 with Fig. 62.

10 Appendix: An Auxiliary Lemma

Lemma 1 *Let $\delta > 0$, $u \in \mathbb{R}$ and $\eta_{u,\delta}$ be a r.v. $\eta_{u,\delta} = (\xi - u)^2$, where r.v. $\xi \in [-\delta, \delta]$ has $Beta_\delta(\alpha, \alpha)$ distribution with density*

$$p_{\alpha,\delta}(t) = \frac{(2\delta)^{1-2\alpha}}{Beta(\alpha, \alpha)}[\delta^2 - t^2]^{\alpha-1}, \quad -\delta < t < \delta, \alpha > 0. \tag{37}$$

$Beta(\cdot, \cdot)$ is the Beta-function. The r.v. $\eta_{u,\delta}$ is concentrated on the interval $[(\max(0, \delta - |u|))^2, (\delta + |u|)^2]$. Its first three central moments are:

$$\mu_{u,\delta}^{(1)} = E\eta_{u,\delta} = u^2 + \frac{\delta^2}{2\alpha+1},$$

$$\mu_{u,\delta}^{(2)} = \mathrm{var}(\eta_{u,\delta}) = \frac{4\delta^2}{2\alpha+1}\left[u^2 + \frac{\delta^2\alpha}{(2\alpha+1)(2\alpha+3)}\right],$$

$$\mu_{u,\delta}^{(3)} = E\left[\eta_{u,\delta} - E\eta_{u,\delta}\right]^3 = \frac{48\alpha\,\delta^4}{(2\alpha+1)^2(2\alpha+3)}\left[u^2 + \frac{\delta^2(2\alpha-1)}{3(2\alpha+5)(2\alpha+1)}\right].$$

In the limiting case $\alpha = 0$, where the r.v. ξ is concentrated at two points $\pm\delta$ with equal weights, we obtain $\mu_{u,\delta}^{(1)} = E\eta_{u,\delta} = u^2 + \delta^2$ and

$$\mu_{u,\delta}^{(2k)} = [2\delta u]^{2k}, \quad \mu_{u,\delta}^{(2k+1)} = 0, \quad \text{for } k = 1, 2, \ldots. \tag{38}$$

References

1. Conway, J., Sloane, N.: Sphere Packings, Lattices and Groups. Springer, Berlin (2013)
2. Johnson, M.E., Moore, L.M., Ylvisaker, D.: Minimax and maximin distance designs. J. Stat. Plan. Inference **26**(2), 131–148 (1990)
3. Niederreiter, H.: Random Number Generation and Quasi-Monte Carlo Methods. SIAM, Philadelphia (1992)
4. Noonan, J., Zhigljavsky, A.: Covering of high-dimensional cubes and quantization. SN Oper. Res. Forum (2020). Preprint arXiv:2002.06118
5. Noonan, J., Zhigljavsky, A.: Efficient quantization and weak covering of high dimensional cubes (2020). arXiv preprint arXiv:2005.07938
6. Petrov, V.V.: Sums of Independent Random Variables. Springer, Berlin (1975)
7. Petrov, V.V.: Limit Theorems of Probability Theory: Sequences of Independent Random Variables. Oxford Science Publications, Oxford (1995)
8. Prakasa Rao, B.L.S.: Asymptotic Theory of Statistical Inference. Wiley, London (1987)
9. Pronzato, L., Müller, W.G.: Design of computer experiments: space filling and beyond. Stat. Comput. **22**(3), 681–701 (2012)
10. Sukharev, A: Minimax Models in the Theory of Numerical Methods. Springer, Berlin (1992)
11. Sukharev, A.G.: Optimal strategies of the search for an extremum. USSR Comput. Math. Math. Phys. **11**(4), 119–137 (1971)
12. Tóth, G.F.: Packing and covering. In Handbook of Discrete and Computational Geometry, pp. 27–66. Chapman and Hall/CRC, London (2017)
13. Tóth, G.F., Kuperberg, W.: Packing and covering with convex sets. In: Handbook of Convex Geometry, pp. 799–860. Elsevier, Amsterdam (1993)
14. Žilinskas, A.: On the worst-case optimal multi-objective global optimization. Optim. Lett. **7**(8), 1921–1928 (2013)

Finding Effective SAT Partitionings Via Black-Box Optimization



Alexander Semenov, Oleg Zaikin, and Stepan Kochemazov

1 Introduction

In modern world, many different concepts are used to tackle hard combinatorial problems. The rapid development of computational hardware in the last few decades puts a special emphasis on the methods that are able to make use of massive amount of computational processes provided by today's computers and supercomputers. One of the most straightforward approaches of this kind consists in partitioning a hard problem into a (possibly very large) family of subproblems which are much easier to solve. However, the question remains, how to construct such partitionings, and how to distinguish which of the many partitionings is better than the others.

In the present chapter we focus on just such questions that arise when considering hard instances of the Boolean satisfiability problem (SAT) [6]. The goal of SAT is for an arbitrary Boolean formula to answer the question whether there exists an assignment of its variables that satisfies this formula. Despite the fact that SAT is NP-complete [25], the progress in practical SAT solving in the recent few decades is nothing short of spectacular. Today, SAT solvers are routinely used to deal with many problems arising in a plethora of different areas, such as hardware verification, model checking, bioinformatics, and cryptanalysis. The major disadvantage of the state-of-the-art SAT solvers is that it is impossible to predict how long will it take a solver to tackle any particular SAT instance given to it as an input, because in the worst-case scenario its runtime will be exponential in the number of variables of the input formula. And while the SAT solvers often work exceptionally well even with SAT instances over hundreds of thousands of variables, there remain hard SAT instances that are seemingly impossible to solve at the current level of technology.

A. Semenov · O. Zaikin (✉) · S. Kochemazov
Matrosov Institute for System Dynamics and Control Theory SB RAS, Irkutsk, Russia
e-mail: zaikin.icc@gmail.com

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_11

In the present chapter we consider the so-called Divide-and-conquer approach to solving hard SAT instances, see, e.g., [76]. It consists in partitioning an instance into a family of subproblems and tackling these subproblems individually, with the possibility to solve them independently in parallel. There exist many ways to partition a SAT instances, see [34]. In the terminology of [34], we focus on the *plain partitioning* variant that uses a subset of variables of a SAT instance to split it into a family of simpler subproblems. We formally show that it is possible to use the Monte Carlo method [51] to estimate the time required by any deterministic complete SAT solving algorithm to solve all subproblems from such a family. This fact allows one to formulate the problem of finding a set of variables that yields a SAT partitioning with the smallest runtime estimation as the problem of optimizing a black-box function that takes as an input a set of variables of a SAT instance and outputs the corresponding runtime estimation. We consider three different objective functions of this type that differ in the way the problems from the SAT partitioning are tackled. Each of them is a pseudo-Boolean black-box costly stochastic function. Therefore, the range of suitable optimization algorithms is very limited. In particular, in the context of the chapter we consider several black-box optimization algorithms that rely only on direct calculations of an objective function (see, e.g., [41]).

In the computational experiments for several SAT-based cryptanalysis instances we aim to construct good runtime estimations and find the corresponding effective SAT partitionings. The results of experiments show that having a portfolio of optimization algorithms often helps since there is no algorithm that works better than the others on all possible inputs. We also checked that for the SAT instances for which the constructed runtime estimation is not too large, the time required to solve the corresponding SAT instances via the found partitionings agrees well with the constructed runtime estimations.

The chapter is organized as follows. In the next section, we briefly provide the basic notation regarding SAT and SAT-based cryptanalysis. Section 3 considers the technique employed to construct SAT partitionings, i.e., how an original problem is split into a family of subproblems. Then it proceeds with describing two main approaches to estimating the time required to solve the corresponding subproblems. In particular, we provide strict formal justifications showing that the Monte Carlo method in its original formulation can be applied to the problems at hand. The main contribution of Sect. 4 is formed by defining three pseudo-Boolean black-box functions that evaluate the effectiveness of SAT partitionings of the considered kind. Also, it describes several heuristic improvements that make use of the peculiarities of hard SAT instances, which encode cryptanalysis problems, and the state-of-the-art SAT solving techniques. Section 5 describes the optimization algorithms that are further used to minimize the objective functions. Finally, in Sect. 6 we consider several hard optimization problems that consist in finding good runtime estimations for relevant cryptanalysis problems, present the results of the corresponding computational experiments and discuss them.

2 Preliminaries

Binary words are the words over the alphabet $\{0, 1\}$. Let us denote by $\{0, 1\}^k$ the set of all possible binary words of length k, $k \in \mathbb{N}^+$. The variables that take values from the set $\{0, 1\}$ are called *Boolean*, thus, in some sources, the elements from $\{0, 1\}^k$ are often called *Boolean vectors* of length k. The set of all possible binary words of an arbitrary finite length is denoted as

$$\{0, 1\}^+ = \bigcup_{k=1}^{\infty} \{0, 1\}^k.$$

By *Boolean formula* over a set of variables $X = \{x_1, \ldots, x_k\}$ we mean an expression that is constructed using specific rules over a finite alphabet which includes the variables from X, braces, and special auxiliary symbols called Boolean connectives. Usually, it is implied that the Boolean connectives form a *complete basis* [70]. Hereinafter, we consider Boolean formulas over complete bases $\{\wedge, \vee, \neg\}$ or $\{\wedge, \neg\}$, where $\wedge$ is conjunction, $\vee$ is disjunction, and $\neg$ is negation. The formulas of the kind x and $\neg x$, $x \in X$ are called *literals* (over X). A pair of literals $(x, \neg x)$ is called a *complementary pair*. A *clause* is an arbitrary disjunction of different literals among which no pair is complementary. A *Conjunctive Normal Form* (CNF) is an arbitrary conjunction of different clauses.

2.1 Boolean Satisfiability Problem (SAT)

Let F be an arbitrary Boolean formula over $X = \{x_1, \ldots, x_k\}$. One can naturally associate with F a Boolean function $f_F : \{0, 1\}^k \to \{0, 1\}$. An arbitrary Boolean vector of length k can then be viewed as an assignment of variables from X. Formula F is called *satisfiable* if there exists such $\alpha \in \{0, 1\}^k$ that $f_F(\alpha) = 1$. Such an α is referred to as a *satisfying assignment* of F. If there are no assignments of variables that satisfy F, then the formula is called *unsatisfiable*. The *Boolean satisfiability problem (SAT)* is to determine the satisfiability of an arbitrary formula F [24]. Using the transformations described in [69], SAT for an arbitrary Boolean formula can be reduced to SAT for a formula in CNF in polynomial time. Thus, without the loss of generality, it is possible to consider SAT only in application to CNFs.

SAT is the historically first NP-complete problem. It is clear that the following problem is NP-hard [25]: for an arbitrary CNF C to find an assignment satisfying C or to prove that C is unsatisfiable. This problem is also denoted as SAT. Despite the NP-hardness, there are many subclasses and special cases of SAT for which it is possible to solve the corresponding instances in reasonable time. These facts led to the intensive development of computational algorithms for solving SAT. The resulting algorithms have been successfully applied to various problems [6].

In the present study we use only the SAT solving algorithms that are based on the *Conflict-Driven Clause Learning (CDCL)* concept [46, 47]. Today, they perform best overall in application to wide spectrum of problems from different areas. Informally, a CDCL algorithm traverses a binary tree representing the process of finding satisfying assignments of an input Boolean formula in CNF. During this process, it encounters the so-called *conflicts*, meaning that some branches of the search tree resulted in contradictions. CDCL solvers store the information about refuted branches in form of the *learnt clauses* [47]. As it follows from their name, they use the information about conflicts to direct the further traversal of the search tree. The distinctive feature of CDCL is that the algorithm is complete, therefore not only can it find an assignment of variables that satisfies the input formula but it also can prove that such an assignment does not exist. An important fact for the further constructions consists in that due to its completeness, the runtime of CDCL is finite on any input SAT instance.

The practical implementations of CDCL usually employ many different heuristic techniques that allow them to cope with large industrial problems. Historically, one of the most successful solvers is the MINISAT solver [20] first presented in 2003. It introduced a simple framework that is easy to improve and experiment with. Even almost 20 years later, the improved versions of MINISAT are still considered to be among the best performing solvers.

When it comes to hard SAT instances where standard sequential SAT solvers might not be enough, there exist two main approaches to solve SAT in parallel. The first one is called the *portfolio* approach [29] that consists in launching different SAT solving algorithms (or the same algorithm with different parameters) on the same SAT instance in parallel. The motivation here is that since CDCL is a complete algorithm then at some point at least one of the algorithms will manage to solve the problem. However, in all but the most simple cases it is impossible to give any predictions when the problem will be solved. The second approach is called the *partitioning* approach and it implements the Divide-and-conquer strategy [76]. It consists in splitting the original problem into several (possibly very many) disjoint subproblems and solving them in parallel. In that case intuitively each subproblem should be easier to solve than the original one.

The existing parallel CDCL solvers such as PLINGELING, TREENGELING, PAINLESS, CRYPTOMINISAT, and others usually implement a mix of portfolio and partitioning approaches [3]. In addition they use some specific parallel heuristics. However, they achieved only a moderate success since the speedup from the parallelization is usually far from linear. In addition to that, the parallel solvers are often not deterministic meaning that several launches of the same solver on the same problem may yield the answer in drastically different times.

2.2 SAT-Based Cryptanalysis

To stimulate the progress, there is always a need for difficult benchmarks that act as challenges for new generations of SAT solving algorithms. One of the most fruitful areas that produces a lot of such benchmarks is combinatorics, including various graph related problems, algebraic problems, etc. Another prominent area that allows one to construct exceptionally difficult benchmarks is cryptanalysis [15]. In the present chapter we consider hard SAT instances that are related to the so-called *SAT-based cryptanalysis*, which is a subarea of the algebraic cryptanalysis [4].

Let us briefly consider the procedures used to transform cryptanalysis instances into SAT instances. One can view a large part of such problems in the context of a general problem to which we will refer as to *problem of inversion of a discrete function* [66].

Consider a total function:

$$f : \{0, 1\}^+ \to \{0, 1\}^+, \tag{1}$$

defined by some algorithm $A(f)$. Such an algorithm naturally defines a family of functions of the kind

$$f_n : \{0, 1\}^n \to \{0, 1\}^+, n \in \mathbb{N}^+.$$

Hereinafter, assume that for a particular n the result of $A(f)$ on an arbitrary word $\alpha \in \{0, 1\}^n$ is a binary word of length m, meaning the functions of the following kind:

$$f_n : \{0, 1\}^n \to \{0, 1\}^m. \tag{2}$$

We will refer to functions (1) and (2) as to *discrete functions*. Let us additionally assume that the complexity of the algorithm $A(f)$ is bounded by a polynomial in n. Then the problem of inversion of a function f is formulated as follows: for known $A(f)$, n and an arbitrary $\gamma \in Range\ f_n \subseteq \{0, 1\}^m$ to find such $\alpha \in \{0, 1\}^n$ that $f_n(\alpha) = \gamma$.

If we view $A(f)$ as a program for a Turing machine that works with binary data [25], then it is possible to show that there exists a procedure with a complexity bounded by a polynomial in n, that given a program $A(f)$ and an arbitrary n as an input, outputs a circuit $S(f_n)$ over a basis $\{\neg, \wedge\}$ that defines function f_n. This fact is a reformulation of the Cook-Levin theorem [14, 44] in the context of a problem of inversion of a function of the kind (1). By applying to circuit $S(f_n)$ the Tseitin transformations [69] it is possible to construct a CNF which we denote as $C(f_n)$. Let us refer to $C(f_n)$ as to *template CNF* for f_n [66].

CNF $C(f_n)$ has an important property that we will frequently use below. This property is based on the well-known *Unit Propagation rule* [19, 47]. Essentially, it is a variant of the resolution method [58], when one of the two clauses used to

construct a resolvent consists of a single literal. It works as follows. Let C be an arbitrary CNF over X and l be some literal over X. Consider CNF $C' = l \wedge C$. First, remove from C' all clauses that contain literal l except the unit clause l. Then from each clause in C' containing literal $\neg l$ remove this literal. The resulting CNF C'' is equivalent to C'. The described transformation represents one iteration of Unit Propagation.

Assume that C is some CNF over X. For an arbitrary $x \in X$ and $\delta \in \{0, 1\}$ let us define the result of the substitution of $x = \delta$ in CNF C as a CNF $C|_{x=\delta}$ constructed by replacing all occurrences of x to δ and performing all possible elementary transformations [12]. Let $l_\delta(x)$, $\delta \in \{0, 1\}$ be literal $\neg x$ when $\delta = 0$ and x when $\delta = 1$. It is easy to see that CNF $l_\delta(x) \wedge C$ is satisfiable if and only if CNF $C|_{x=\delta}$ is satisfiable. Thus, the application of Unit Propagation to $l_\delta(x) \wedge C$ can be interpreted as substitution of $x = \delta$ to C. During Unit Propagation there can appear new unit clauses of the kind $l_{\delta'}(x')$. Taking into account all of the above, we say that in this case the value δ' of variable x' *is derived* from a corresponding CNF via Unit Propagation. The property of the template CNFs we mentioned earlier consists in the following.

Let f_n be a discrete function of the kind (2). Assume that $S(f_n)$ is a Boolean circuit that specifies f_n, $C(f_n)$ is the corresponding template CNF, and X is a set of Boolean variables from $C(f_n)$. Let us outline in X the sets $X^{in} = \{x_1, \ldots, x_n\}$ and $Y = \{y_1, \ldots, y_m\}$ formed by the variables associated with the inputs and outputs of circuit $S(f_n)$, respectively.

Lemma 1 *Suppose that $\alpha = (\alpha_1, \ldots, \alpha_n)$ is an arbitrary assignment of variables from X^{in}. Consider a CNF*

$$l_{\alpha_1}(x_1) \wedge \ldots \wedge l_{\alpha_n}(x_n) \wedge C(f_n). \tag{3}$$

The iterative application of Unit Propagation to (3) will result in the derivation of all variables from X. In particular, it means that the values $y_1 = \gamma_1, \ldots, y_m = \gamma_m$ will be derived such that $f_n(\alpha) = \gamma$, $\gamma = (\gamma_1, \ldots, \gamma_m)$.

The statements which are close to Lemma 1 can be found in many papers such as [5, 37, 38, 60].

It was shown in [66] that if $\gamma = (\gamma_1, \ldots, \gamma_m) : \gamma \in Range\ f_n$, then CNF

$$C(f_n, \gamma) = l_{\gamma_1}(y_1) \wedge \ldots \wedge l_{\gamma_m}(y_m) \wedge C(f_n) \tag{4}$$

is satisfiable and from its satisfying assignment one can extract the assignment α of variables from X^{in} such that $f_n(\alpha) = \gamma$.

The transition from the inversion problem for a function of the kind (2) to SAT for some CNF (4) is an essential first step of any attempt at solving cryptanalysis problems with SAT solvers. In practice it is possible to use various software tools to perform this step: CBMC [13]; URSA [36]; SAW [11]; CryptoSAT [43]; Grain of Salt [67]. In the present chapter we use SAT encodings constructed via the TRANSALG software tool [53], which takes into account many features that are

specific to cryptographic functions. The detailed comparison of the tools together with a more detailed description of the SAT-based cryptanalysis method can be found in [66].

3 Decomposition Sets and Backdoors in SAT with Application to Inversion of Discrete Functions

In the present section we describe the method that we use to partition a hard SAT instance into a family of simpler subproblems. Using the notation from [34], we employ the so-called *partitioning approach*, which can be viewed as a special case of data parallelism.

Definition 1 ([34, 35]) Let C be an arbitrary CNF over a set X of Boolean variables. A *plain partitioning* of C is a set of formulas

$$G_j \wedge C, j \in \{1, \ldots, r\}$$

such that for any $i, j : i \neq j$ formula $G_i \wedge G_j \wedge C$ is unsatisfiable and

$$C \equiv G_1 \wedge C \vee \ldots \vee G_r \wedge C,$$

where $\equiv$ stands for logical equivalence.

Obviously, when one has a plain partitioning of an original SAT instance, SAT for formulas $G_j \wedge C, j \in \{1, \ldots, r\}$ can be solved independently in parallel. There exist various partitioning techniques. For example, one can construct $\{G_j\}, j = 1, \ldots, r$ using the so-called scattering procedure, a guiding path solver, look-ahead solver, or a number of other techniques described in [34].

The idea to use look-ahead strategy to construct SAT partitionings, first expressed in [35], was later developed in [32], which presented a SAT solver that combines the features of CDCL and look-ahead concepts. In more detail, in [32] it was proposed to use a look-ahead solver as an external procedure that constructs some partitioning tree. If during this process the look-ahead solver refutes some branch, then this branch is not considered later. Otherwise, the look-ahead solver uses the special cutoff heuristics to terminate the construction of a tree along some branch. This branch represents the so-called *cube*, i.e., a conjunction of several literals. The resulting set of r cubes corresponding to cutoff branches of partitioning tree forms a set of formulas of the kind $\{G_j\}_{j=1}^{r}$ in the context of the plain partitioning strategy from [35]. The strategy described in [32] was called *Cube and Conquer*. In the recent years, Cube and Conquer SAT solvers were used to successfully solve several hard combinatorial problems related to the Ramsey theory. One of the most significant results in this area consists in solving the Boolean Pythagorean Triples Problem [33].

3.1 On Interconnection Between Plain Partitionings and Cryptographic Attacks

Another area for which the construction of SAT partitionings appears to be relevant is algebraic cryptanalysis. Below we show that a good plain partitioning of a SAT instance that encodes some cryptanalysis problem in fact may yield a cryptographic attack which is significantly better than brute force.

In the previous section we noted that a number of cryptanalysis problems can be viewed in the general context of the problem of finding preimages for functions of the kind (2). Assume that f_n is an arbitrary function of the kind (2) defined by some cipher. For example, f_n can correspond to some *keystream generator* [50] that uses an input sequence α of length n (it corresponds to either a secret key or some intermediate state of the registers) to produce a keystream fragment of length m. The cryptanalysis problem looks as follows: for a given $\gamma \in Range\ f_n$ to find $\alpha \in \{0,1\}^n$ such that $f_n(\alpha) = \gamma$. The formulated variant corresponds to the so-called *Known plaintext attack* on generator f_n [50]. Suppose that we reduced it to a problem of finding a satisfying assignment for a CNF of the kind (4). It is entirely possible that the resulting SAT instance is too hard even for the most cutting edge SAT solvers. It holds true, for example, in case of such keystream generators as Trivium [10] or Grain [31]. On the other hand, Lemma 1 states that for an arbitrary $\alpha' = (\alpha'_1, \ldots, \alpha'_n)$, $\alpha' \in \{0,1\}^n$ we can consider a CNF of the kind (3), use Unit Propagation to derive from it the corresponding $\gamma' = (\gamma'_1, \ldots, \gamma'_m)$, such that $f_n(\alpha') = \gamma'$, and check whether γ' is equal to γ. If yes, then α' is the sought key. Otherwise, we check the next α'. The described method essentially represents a variant of *brute force attack*, and its complexity is $2^n \times T_0$, where T_0 is the time required to check one key candidate. For some ciphers it is possible to find a set $B \subset X$ with the following properties:

1. $|B| = s, s < n$,
2. $2^s \cdot \widehat{T}_{\mathcal{A}} << 2^n \cdot T_0$.

Here by $\widehat{T}_{\mathcal{A}}$ we mean an upper bound on the runtime of some algorithm $\mathcal{A}$ that solves the SAT instances obtained by substituting the values of variables from B to CNF $C(f_n, \gamma)$. If a set B is found such that properties 1–2 hold for the majority of $\gamma \in Range\ f_n$, then it can be said that there exists a non-trivial *guess-and-determine attack based on guessed bits set B on function* f_n.

The guess-and-determine attacks form one of the most voluminous classes of attacks in algebraic cryptanalysis [4]. It is important to note that the problems of inversion of functions of the kind (2) can be effectively reduced to solving systems of algebraic equations over a finite field (usually, $GF(2)$), instead of SAT. It does not change the general concept of a guess-and-determine attack.

Let B be some guessed bits set. From the point of view of the notation introduced earlier, we can associate with an arbitrary set $B = \{b_1, \ldots, b_s\}$, $B \subseteq X$ and an arbitrary $\gamma \in Range\ f_n$ a plain partitioning $P(C(f_n, \gamma), B)$ formed by the formulas of the kind

$$G(\beta) \wedge C(f_n, \gamma) \tag{5}$$

over all possible $\beta \in \{0,1\}^s$, $\beta = (\beta_1, \ldots, \beta_s)$, where

$$G(\beta) = l_{\beta_1}(b_1) \wedge \ldots \wedge l_{\beta_s}(b_s).$$

Let us apply to an arbitrary formula of the kind (5) some complete CDCL SAT solver. If the properties listed above are satisfied for a considered B, then we have a non-trivial guess-and-determine attacks based on B on function f_n, which uses a SAT solver in the role of the algorithm $\mathcal{A}$. Note that in terms of the notation from [34], formula $G(\beta)$, $\beta \in \{0,1\}^s$ in (5) is a cube over set B.

We have just described a simple method for constructing SAT partitionings: for an arbitrary set $B \subseteq X$ construct a set of formulas of the kind (5) over all possible $\beta \in \{0,1\}^{|B|}$ (hereinafter by $\{0,1\}^{|B|}$ we denote the set of all possible assignments of variables from B). It is assumed that after this one applies some complete SAT solving algorithm to CNFs (5). The above strongly correlates with a well-known notion of strong backdoor sets introduced in [71].

Let C be an arbitrary CNF over X, and B, $|B| = s$, be an arbitrary subset of X. Let us use the following notation:

$$C[\beta/B] = G(\beta) \wedge C, \tag{6}$$

where $\beta = (\beta_1, \ldots, \beta_s)$, $\beta \in \{0,1\}^{|B|}$.

Definition 2 Let C be an arbitrary CNF over a set of variables X and B, $B \subseteq X$ be an arbitrary nonempty set. Let us refer to the set B as *decomposition set* for CNF C. The set of formulas of the kind (5) for all possible $\beta \in \{0,1\}^{|B|}$ is called a *SAT partitioning of C generated by decomposition set B*.

Definition 3 ([71]) Let C be an arbitrary CNF over a set of variables X, and let $\mathcal{A}$ be a polynomial time algorithm. A nonempty set B, $B \subseteq X$, is a *strong backdoor set* for C w.r.t. algorithm $\mathcal{A}$ if for each $\beta \in \{0,1\}^{|B|}$ algorithm $\mathcal{A}$ decides formula $C[\beta/B]$.

Thus, a strong backdoor set is such a decomposition set $B \subseteq X$ that SAT for an arbitrary CNF of the kind (6) can be solved in polynomial time. It is important to note that from Lemma 1 it follows that when f_n is an arbitrary discrete function of the kind (2), then in CNFs $C(f_n)$, $C(f_n, \gamma)$ the set X^{in} is the strong backdoor set w.r.t. the Unit Propagation rule. Below let us refer to such a set as *strong Unit Propagation backdoor set (SUPBS)*.

Further we show how to relinquish the requirement that $\mathcal{A}$ must have a polynomial time complexity and employ in the role of $\mathcal{A}$ a CDCL SAT solver with finite runtime on an arbitrary input. Taking all this into account it is possible to define another subclass of decomposition sets.

Definition 4 ([65]) Assume that $\mathcal{A}$ is an arbitrary complete SAT solving algorithm, and C is an arbitrary CNF over the set X of Boolean variables. Denote by $t(\mathcal{A}, C)$

the runtime of $\mathcal{A}$ on C. A nonempty set $B \subseteq X$ is called a *Non-Deterministic Oracle Backdoor Set* (NOBS) [65] for CNF C w.r.t $\mathcal{A}$ if

$$\sum_{\beta \in \{0,1\}^{|B|}} t(\mathcal{A}, C[\beta/B]) < t(\mathcal{A}, C).$$

In other words NOBS is such a decomposition set $B \subseteq X$ that the total runtime of $\mathcal{A}$ over all SAT instances in the SAT partitioning $P(C, B)$ generated by B is lower than the runtime of $\mathcal{A}$ on the original CNF C.

Hereinafter we study only the SAT partitionings generated by decomposition sets in the context of Definition 2. In the next subsection we show that for SAT partitionings from this class it is possible to naturally define the measures of their effectiveness. In the role of such measures we use the estimations of the time required to solve all SAT instances from the corresponding SAT partitioning. We say that a SAT partitioning P_1 is more effective than SAT partitioning P_2 if the value of the introduced measure for P_1 is lower than that for P_2. When considering problems of inversion of cryptographic functions we can use effective SAT partitionings to construct non-trivial guess-and-determine attacks. The set which generates the corresponding SAT partitioning in that cases acts as a set of guessed bits.

To define the measures of SAT partitionings' effectiveness we employ the Monte Carlo method. We would like to note that this term is often used without any strict formal justifications, when it basically implies the use of random sampling. Below we present the formal basis that allows one to use the Monte Carlo method in its classical form [51] to evaluate the effectiveness of SAT partitionings. For the first time the corresponding results were presented in papers [61, 63], however, similar ideas, albeit without strict justification, were employed in earlier works, e.g., [22, 49, 68].

3.2 *Using Monte Carlo Method to Estimate Runtime of SAT-Based Guess-and-Determine Attacks*

So, let us consider the inversion problem for a function of the kind (2) for a known image $\gamma \in Range\ f_n$. First reduce it to the problem of finding a satisfying assignment of CNF $C(f_n, \gamma)$. Assume that X is the set of variables from $C(f_n, \gamma)$. To solve SAT we will use a complete deterministic SAT solver $\mathcal{A}$.

Consider an arbitrary decomposition set $B \subseteq X$, $B = \{b_1, \ldots, b_s\}$. Define a uniform distribution over set $\{0, 1\}^s$. With an arbitrary $\beta = (\beta_1, \ldots, \beta_s)$, randomly chosen from $\{0, 1\}^s$, let us associate the value of random variable ξ which is equal to runtime of $\mathcal{A}$ on the corresponding CNF of the kind (5). Denote the spectrum of values of ξ over all possible CNFs of the kind (5) as $Sp(\xi) = \{\xi_1, \ldots, \xi_Q\}$. It is important to note that ξ has a finite expected value $E[\xi]$ and finite variance

$Var(\xi)$ due to the fact that $\mathcal{A}$ has finite runtime on any CNF. The following fact was established in [63].

Theorem 1 *Let $C = C(f_n, \gamma)$ be a CNF of the kind (4) over a set of Boolean variables X. For an arbitrary $B \subseteq X, |B| = s$ consider the SAT partitioning $P(C, B)$ formed by the formulas of the kind (5). Assume that $\mathcal{A}$ is a complete deterministic SAT solving algorithm, ξ is the random variable introduced above and $T(\mathcal{A}, C, B)$ is the total runtime of $\mathcal{A}$ on all SAT instances from $P(C, B)$. Then*

$$T(\mathcal{A}, C, B) = 2^{|B|} \cdot E[\xi]. \tag{7}$$

Proof Assume that all conditions of the theorem are satisfied. Consider random variable ξ and its spectrum $Sp(\xi) = \{\xi_1, \ldots, \xi_Q\}$. Note that $Sp(\xi)$ is formed by finite positive real numbers. Let us associate with ξ the possibility space Ω, the elements of which are Boolean vectors from $\{0, 1\}^s$. With each $\xi_j \in Sp(\xi)$, $j \in \{1, \ldots, Q\}$ we link a set $\Omega_j, \Omega_j \subseteq \Omega$, formed by Boolean vectors $\beta = (\beta_1, \ldots, \beta_s)$ such that the runtime of $\mathcal{A}$ on each CNF of the kind (5) is ξ_j. Let us introduce the notation $\#\xi_j = |\Omega_j|$. Consider the set of numbers

$$P(\xi) = \left\{ \frac{\#\xi_1}{2^s}, \ldots, \frac{\#\xi_Q}{2^s} \right\}.$$

It is clear that the numbers from $P(\xi)$ can be viewed as the probabilities with which random variable ξ takes corresponding values from its spectrum. Thus, $P(\xi)$ is a probability distribution of random variable ξ. Taking all this into account it holds that:

$$T(\mathcal{A}, C, B) = \sum_{j=1}^{Q} \#\xi_j \cdot \xi_j = 2^s \cdot \sum_{j=1}^{Q} \frac{\#\xi_j}{2^s} \cdot \xi_j = 2^s \cdot E[\xi].$$

Thus, (7) holds. □

Formula (7) justifies the use of the Monte Carlo method for estimating the value of $T(\mathcal{A}, C, B)$. Indeed, let us denote by $\xi^1, \ldots, \xi^N$ the values of ξ observed in N independent probabilistic experiments. We assume that in each of the latter the vector β is chosen from $\{0, 1\}^s$ with respect to the uniform distribution. Since $\mathcal{A}$ is a complete deterministic algorithm for solving SAT, it means that $\xi^1, \ldots, \xi^N$ can be viewed as a single observation of N independent random variables with the same distribution, expected value $E[\xi]$ and finite variance $Var(\xi)$. Let us consider a random variable

$$\frac{1}{N} \cdot \sum_{j=1}^{N} \xi^j. \tag{8}$$

We apply to (8) the Chebyshev's inequality [23]:

$$Pr\left\{\left|\frac{1}{N}\cdot\sum_{j=1}^{N}\xi^{j}-E[\xi]\right|\leq\epsilon\right\}\geq 1-\frac{Var(\xi)}{\epsilon^{2}\cdot N}. \quad (9)$$

From (9) it follows that the value of $E[\xi]$ can be estimated by the values of (8) with an arbitrary tolerance $\epsilon > 0$ by increasing the number of observations N. However, when constructing Monte Carlo estimations for functions (7) in application to practical problems, it is important to take into account several issues. The first issue is that we can construct an accurate estimation only if the variance $Var(\xi)$ is relatively small. Unfortunately, in practice it takes place quite rarely. For example, in [62] it was noted that when calculating the values of (8) for cryptanalysis of the Bivium cipher [10], the random samples of size $N < 10^4$ result in overly optimistic estimations. It happens because hard SAT instances form relatively small portion of the SAT partitioning and therefore they are often absent in samples of small sizes. This fact leads to the significant growth of sample variance with the increase of the sample size. The conclusion of [62] was that such effects are caused by the phenomenon known as heavy-tailed behavior of complete SAT solvers [27]. Another issue that has to be taken into account is that if we apply the described method to cryptographic functions, then even when $Var(\xi)$ is small, we construct the estimation of an attack runtime only for a single specific $\gamma \in Rangef_n$. Nevertheless, the construction of Monte Carlo estimations for (7) and similar functions is of particular interest not only in the context of cryptanalysis but also for the problems in which the goal is to prove the unsatisfiability of some CNF (e.g., in verification problems [42]).

Another approach to estimating the runtime of guess-and-determine attacks on functions of the kind (2) was described in [65]. Once again, consider the problem of inversion of an arbitrary function of the kind (2). However, this time our goal is to construct a guess-and-determine attack on f_n, for which the runtime estimation does not depend on particular $\gamma \in Range\ f_n$.

Assume that there is an algorithm $A(f)$ that defines f_n, a circuit $S(f_n)$, and the corresponding template CNF $C = C(f_n)$ over a set X of Boolean variables. Let us outline in X the subset $X^{in} = \{x_1, \ldots, x_n\}$ formed by the variables corresponding to the inputs of circuit $S(f_n)$, and the subset $Y = \{y_1, \ldots, y_m\}$ of variables corresponding to $S(f_n)$ outputs. The notations $C[\alpha/X^{in}]$ and $C[\gamma/Y]$ (for arbitrary $\alpha \in \{0,1\}^{|X^{in}|}$ and $\gamma \in \{0,1\}^{|Y|}$, respectively) that we use below are similar in spirit to (6).

The main distinction between the guess-and-determine attack described further from the one outlined above consists in the following: we limit the runtime of SAT solver $\mathcal{A}$ by some τ on CNFs from a SAT partitioning generated by B. If the runtime of $\mathcal{A}$ on an arbitrary CNF from the partitioning exceeded τ, then $\mathcal{A}$ is interrupted.

The notions we introduce further are based on the property formulated in Lemma 1. Recall that according to this property, for an arbitrary $\alpha \in \{0,1\}^{|X^{in}|}$ the application of Unit Propagation to CNF $C[\alpha/X^{in}]$ results in derivation of values of all variables from X including the value of f_n. For an arbitrary set $B \subseteq X$ let us

refer to the values of its variables derived by Unit Propagation from CNF (3) as to an *assignment induced by* α.

Define a uniform distribution over $\{0,1\}^n$. Fix an arbitrary $B \subseteq X \backslash Y$. With a randomly chosen $\alpha \in \{0,1\}^n$ associate the assignments $\beta(\alpha)$ and $\gamma(\alpha) = f_n(\alpha)$ of variables from B and Y, respectively, which were induced by α from CNF (3). Consider CNF $C[\gamma(\alpha)/Y, \beta(\alpha)/B]$ where $C = C(f_n)$. Let us denote by $t(\mathcal{A}, C, B, \alpha)$ the runtime of $\mathcal{A}$ on the input $C[\gamma(\alpha)/Y, \beta(\alpha)/B]$. Assume that the value $\tau > 0$ is fixed to some constant. Then consider the following value:

$$p_B(\tau) = \frac{\#\{\alpha \in \{0,1\}^n : t(\mathcal{A}, C, B, \alpha) \leq \tau\}}{2^n}. \tag{10}$$

The numerator of (10) represents the number of such $\alpha \in \{0,1\}^n$ for which CNF $C[\gamma(\alpha)/Y, \beta(\alpha)/B]$ is decided by $\mathcal{A}$ in time $\leq \tau$. The denominator of (10) is the number of all α. Therefore, (10) is essentially the probability of the following event: that a randomly selected $\alpha \in \{0,1\}^n$ induces such assignments $\beta(\alpha)$ and $\gamma(\alpha)$ that $\mathcal{A}$ decides $C[\gamma(\alpha)/Y, \beta(\alpha)/B]$ in time $\leq \tau$.

Definition 5 ([65]) An arbitrary nonempty decomposition set B, $B \subseteq X \backslash Y$, $|B| = s$, with properties described above is called an *Inverse Backdoor Set* (IBS) with parameters $(s, \tau, p_B(\tau))$ for $C(f_n)$ w.r.t. algorithm $\mathcal{A}$.

In [65] there was introduced a new class of guess-and-determine attacks. They are based on the following two definitions.

Definition 6 (IBS-Based Elementary Guess-and-Determine Attack, [65]) Consider the inversion problem for a function f_n of the kind (2). Assume that we have an arbitrary $\gamma \in Range\ f_n$ that corresponds to some $\alpha \in \{0,1\}^n$ (i.e., $\gamma = f_n(\alpha)$).

1. Let B be an IBS with parameters $(s, \tau, p_B(\tau))$ and $\beta \in \{0,1\}^s$ be an assignment to variables of B.
2. Construct CNF $C[\gamma/Y, \beta/B]$ and run SAT solver $\mathcal{A}$ on it.
3. If the runtime of $\mathcal{A}$ on this SAT instance exceeds τ, interrupt the solving process and move to another $\beta \in \{0,1\}^{|B|}$.
4. For $\beta = \beta(\alpha)$ algorithm $\mathcal{A}$ will find a satisfying assignment for $C[\gamma/Y, \beta/B]$ in time $\leq \tau$ with probability $p_B(\tau)$. This means that in this case $\mathcal{A}$ will compute α s.t. $f_n(\alpha) = \gamma$.

It is possible that the analysis of some γ leads to no result because for each formula $C[\gamma/Y, \beta/B]$, $\beta \in \{0,1\}^{|B|}$ either the runtime of $\mathcal{A}$ exceeded τ or the unsatisfiability was proved. In that case due to the cryptographic context we can consider another element from $Range\ f_n$ that is different from γ. For example, if f_n is some keystream generator, then we can consider a fragment of keystream of size m that follows after γ. Thus, we have the following iterative guess-and-determine attack.

Definition 7 (IBS-Based Guess-and-Determine Attack, [65]) Consider the inversion problem for a function f_n of the kind (2).

1. Let $\gamma^1, \ldots, \gamma^r$ be observed outputs of function f_n. These outputs correspond to inputs $\alpha^1, \ldots, \alpha^r$.
2. Let B be some IBS with parameters $(s, \tau, p_B(\tau))$, $p_B(\tau) > 0$.
3. A guess-and-determine attack based on IBS B consists in successive application of elementary attack, as described in Definition 6, to outputs $\gamma^1, \ldots, \gamma^r$.
4. The attack is said to be *successful* if for at least one $j \in \{1, \ldots, r\}$ the corresponding inversion problem for f_n is solved.

For an arbitrary function of the kind (2) by the runtime of a corresponding guess-and-determine attack we mean the time required by this attack until at least one image of function f_n is inverted.

Definition 8 Let P_r^* be the probability of the event that a guess-and-determine attack from Definition 7 is successful. Let us say that this attack is *statistically significant* if $P_r^* \geq 0.95$.

Let us denote the runtime of a statistically significant guess-and-determine attack as $T(\mathcal{A}, C, B, \tau)$, where $C = C(f_n)$. The following result was implicitly presented in [65].

Theorem 2 *Consider an inversion problem for a function f_n of the kind (2). Assume that in the context of this problem B is an arbitrary IBS with parameters $(s, \tau, p_B(\tau))$, $p_B(\tau) > 0$. Then there exists a statistically significant guess-and-determine attack based on IBS B with runtime*

$$T(\mathcal{A}, C, B, \tau) = 2^s \cdot \tau \cdot \left\lceil \frac{3}{p_B(\tau)} \right\rceil. \tag{11}$$

Proof Assume that the conditions of the theorem are satisfied. In accordance with Definition 6 the $p_B(\tau)$ is the probability that $\mathcal{A}$ will invert an arbitrary $\gamma \in Range\ f_n$ after traversing all the $\beta \in \{0,1\}^{|B|}$, and spending on each β the time $\leq \tau$. If we consider the inputs $\alpha_1, \ldots, \alpha_r$ independently chosen from $\{0,1\}^n$ and the corresponding outputs $\gamma_1, \ldots, \gamma_r$ of f_n, then it is easy to see that

$$P_r^* = 1 - (1 - p_B(\tau))^r.$$

Assume that $r = \left\lceil \frac{3}{p_B(\tau)} \right\rceil$ and consider the value

$$(1 - p_B(\tau))^{\left\lceil \frac{3}{p_B(\tau)} \right\rceil} \leq (1 - p_B(\tau))^{\frac{3}{p_B(\tau)}}.$$

The value in the right hand side of the latter inequality monotonically increases with the decrease of $p_B(\tau)$ and does not exceed $e^{-3} \approx 0.04978$. □

We would like to additionally note that the value $p_B(\tau)$ does not depend on a specific output of f_n, and thus (11) characterizes the runtime of the described attack for $\left\lceil \frac{3}{p_B(\tau)} \right\rceil$ arbitrary outputs of f_n induced by random inputs.

Similarly to function (7) the values of function (11) can be estimated using the Monte Carlo method. In more detail, we can use this method to estimate the probability $p_B(\tau)$. For this purpose let us associate with the possibility space $\Omega = \{0, 1\}^n$ the random variable ζ that takes values from the set $\{0, 1\}$. If for an arbitrary $\alpha \in \{0, 1\}^n$ the algorithm $\mathcal{A}$ decides the satisfiability of CNF $C[\gamma(\alpha)/Y, \beta(\alpha)/B]$ in time $\leq \tau$, then we say that $\zeta = 1$, otherwise $\zeta = 0$. It is clear that ζ is a random variable with the spectrum $Sp(\zeta) = \{1, 0\}$ and probability distribution $\{p_B(\tau), 1 - p_B(\tau)\}$. Thus, ζ is the Bernoulli random variable with success probability $p_B(\tau)$. Therefore, $E[\zeta] = p_B(\tau)$, $Var(\zeta) = p_B(\tau) \cdot (1 - p_B(\tau))$. Taking this into account let us apply the Chebyshev's inequality to produce the following statement (similarly to (9)):

$$Pr\left\{\left|\frac{1}{N}\cdot\sum_{j=1}^{N}\zeta^j - p_B(\tau)\right| \leq \epsilon\right\} \geq 1 - \frac{1}{4\cdot\epsilon^2\cdot N}. \tag{12}$$

The right hand side of (12) makes use of the fact that the function $x \cdot (1-x)$ achieves its maximum over $[0, 1]$ in $x = 0.5$.

Note that (12) looks beneficially compared to (9) because it lacks the unknown variance. Thus, in the IBS-based attacks we can guarantee the precision of the constructed estimations of their runtime by increasing the size N of a random sample. From the other hand, the method presented in [61, 63] makes it possible to construct runtime estimations for arbitrary SAT partitionings, including that for unsatisfiable CNFs, while IBS-based approach cannot be applied to the latter.

4 Practical Aspects of Evaluating Effectiveness of SAT Partitionings

In this section we describe how we can represent the problem of finding an effective SAT partitioning as a black-box optimization problem. We also consider several implementation level details that make it possible to sometimes substantially improve the performance of the proposed approach in practice.

First, let us briefly recapture the results of the previous section. It was shown that given a CNF C over a set of Boolean variables X and some subset $B \subseteq X$ it is possible to use this subset to partition C into a family of (usually) easier subproblems. Due to peculiar features of SAT, it is entirely possible that for a given SAT solving algorithm $\mathcal{A}$ its runtime on an original problem can be significantly higher than its total runtime on all subproblems from a constructed SAT partitioning [40]. In the previous section we introduced two main ways for evaluating the effectiveness of SAT partitionings. The functions (7) and (11) represent the essence of these approaches. Regardless of the approach, we want to find a SAT partitioning that has the best effectiveness.

In practice, it is infeasible to compute the exact values of (7) and (11) because it is equivalent to solving the original problem. However, as it was noted above, the Monte Carlo method makes it possible to construct probabilistic estimations of (7) and (11) that can often be computed in realistic time. The corresponding functions for (7) and (11), respectively, look as follows:

$$\Phi_{\mathcal{A},C,N}(B) = \frac{2^s}{N} \cdot \sum_{j=1}^{N} \xi^j, \tag{13}$$

$$\Psi_{\mathcal{A},C,N,\tau}(B) = 2^s \cdot \tau \cdot \frac{3}{\sum_{j=1}^{N} \zeta^j}. \tag{14}$$

It should be noted that for (14) if the value in the denominator is equal to zero, then we assume that the function's value is $+\infty$. Also, $C = C(f_n, \gamma)$ in (13) and $C = C(f_n)$ in (14).

Let us consider functions (13) and (14) in more detail. Note that when computing the values of (13) and (14) one needs to generate a random sample of size N, observe N values of a corresponding random variable, and then perform simple arithmetic operations. The N observations of random variable in case of (13) mean measuring the runtime of a SAT solving algorithm $\mathcal{A}$ on the N problems forming the sample. For (14) it is necessary to not only measure the runtime of $\mathcal{A}$ but to also interrupt it once the time limit τ is exceeded. It means that the functions (13) and (14) are not defined analytically and thus can be viewed as black-box functions. Therefore, taking into account all of the above we can move from the problem of finding a SAT partitioning with the best value of (7) or (11) to the problem of finding a set B that yields a minimum of the function (13) or (14), respectively, over some finite search space (ideally, over 2^X) for fixed $\mathcal{A}$, C, N, τ. Thus the problem of finding an effective SAT partitioning is considered a black-box optimization problem.

4.1 Narrowing Search Space to SUPBS

As it is the case for all optimization algorithms, if there is a possibility to reduce the search space—it must be used. Fortunately, in the case of functions (13) and (14), Lemma 1 provides just the opportunity to do so, at least for major classes of problems. Essentially, since from Lemma 1 it follows that the variables from X^{in} form a SUPBS (see Sect. 3.1) for both $C(f_n)$ and $C(f_n, \gamma)$, then they can be viewed as much more important than the other variables in C. This fact fits nicely with the cryptographic specifics of the problems considered in later sections. Thus, we can narrow down the search space as follows: we choose a set B only from the set of subsets of a set X^{in}. Note that from Lemma 1 it follows that for $B = X^{in}$ the values

of (13) and (14) can be computed effectively in polynomial time in the size of binary description of CNF C.

So, assume that the set of possible alternatives of B is $2^{X^{in}}$, $X^{in} = \{x_1, \ldots, x_n\}$. Then we can represent any $B \subseteq X^{in}$ by a Boolean vector $\lambda = (\lambda_1, \ldots, \lambda_n)$, assuming that $\lambda_i = 1$, $i \in \{1, \ldots, n\}$ if and only if $x_i \in B$. Otherwise, $\lambda_i = 0$. Taking this into account, (13) and (14) are pseudo-Boolean functions [8]:

$$\Phi_{\mathcal{A},C,N} : \{0,1\}^n \to \mathbb{R}, \Psi_{\mathcal{A},C,N,\tau} : \{0,1\}^n \to \mathbb{R}.$$

For an arbitrary $\lambda \in \{0,1\}^n$ the value of, say, $\Phi_{\mathcal{A},C,N}(\lambda)$ is computed as follows: first we construct a set B corresponding to λ. Then for this B we consider N independent observations of random variable $\xi : \xi^1, \ldots, \xi^N$ and compute (8). The value of $\Psi_{\mathcal{A},C,N,\tau}(\lambda)$ is computed in a similar fashion according to (14).

4.2 Applications of Incremental SAT Solving

It is quite clear that when computing values of (13) it is necessary to solve many similar SAT instances. As it happens, there is a special SAT solving technique that makes it possible to sometimes increase the performance of SAT solvers on such tasks. It is called *incremental SAT solving* and was described, for example, in [21]. Below let us briefly recall one of its variants that we can use in practice when computing the function (13).

Consider SAT for an arbitrary CNF C over the set X of Boolean variables. Let $B = \{b_1, \ldots, b_s\}$, $B \subseteq X$ be a decomposition set that generates SAT partitioning $P(C, B)$. Assume that there is a sequence $\rho^1, \ldots, \rho^k$, $k \geq 2$, $\rho^i = (\beta_1^i, \ldots, \beta_s^i)$, $\rho^i \in \{0,1\}^{|B|}$, $i = 1, \ldots, k$ of assignments of variables from B, and our goal is to solve SAT for CNFs

$$\begin{gathered} G(\rho^1) \wedge C \\ \ldots \\ G(\rho^k) \wedge C, \end{gathered}$$

where $G(\rho^i) = l_{\beta_1^i}(b_1) \wedge \ldots \wedge l_{\beta_s^i}(b_s)$. The essence of the incremental SAT solving technique consists in the augmentation of the standard CDCL algorithm that allows it to process a sequence of CNFs $G(\rho^1) \wedge C, G(\rho^2) \wedge C, \ldots, G(\rho^k) \wedge C$, in a single launch of SAT solver. For this purpose the literals of the kind $l_{\beta_1^i}(b_1), \ldots, l_{\beta_s^i}(b_s)$ are not added to C as unit clauses. Instead, the SAT solver uses them as the so-called *assumptions* [21]. It means that it treats $\beta_1^i, \ldots, \beta_s^i$ as guessed values of the corresponding variables. Thus, it assigns the value β_1^i to b_1 the same way it would do it when following an arbitrary path in the search tree. As a result of this trick, any learnt clause that was derived while processing $G(\rho^i) \wedge C$ can be reused when solving $G(\rho^j) \wedge C$, $i \neq j$ or even C. Observe that $G(\rho^i)$ naturally corresponds to

the elements of a random sample used when computing (13). It is currently unclear whether the incremental processing can be naturally applied to computing (14).

In practice the incremental SAT solving has several specific features that should be taken into account. As it was already noted, the learnt clauses generated for one assignment of variables from B can be reused while solving SAT for another. It means that we can solve not individual instances, but blocks of k instances per single launch of the SAT solver, $k > 1$. The value of k has a significant impact on the performance of the incremental SAT solver and is usually chosen to be relatively small, e.g., $10 \leq k \leq 100$. Since one of the benefits of incremental SAT solving consists in not having to initialize the solver for each particular instance, it means that the effect tends to be better when individual instances are relatively easy and can be solved in tens of seconds at most. Otherwise, it is often the case that the state-of-the-art preprocessing and inprocessing methods yield better results when solving $G(\rho^i) \wedge C$ individually.

4.3 Finding Partitionings via Incremental SAT

As it follows from the previous subsection, using incremental SAT we can decrease the time required by a solver to compute $\Phi_{\mathcal{A},C,N}(B)$ especially for larger N. It appears to be extremely worthwhile in the case when we use B to solve SAT for C, i.e., when we need to process all $N{=}2^{|B|}$ subproblems from a partitioning.

The goal of the function introduced in [75] is to evaluate the effectiveness of SAT partitionings using state-of-the-art incremental SAT solvers. Consider a Boolean hypercube $\{0,1\}^{|B|}$, $|B| = s$. With an arbitrary $i \in \{1,\ldots,2^s\}$ associate an s-bit vector ρ^i which represents the number $i-1$ in binary. Consider $\{0,1\}^{|B|}$ as a set of vectors of the kind ρ^i over all possible $i \in \{1,\ldots,2^s\}$. Fix an arbitrary $k : 2 \leq k < 2^s$ and split $\{0,1\}^{|B|}$ into distinct subsets of the following kind:

$$\begin{array}{c} I_1 = \{\rho^1,\ldots,\rho^k\}, I_2 = \{\rho^{k+1},\ldots,\rho^{2k}\},\ldots, \\ I_{U-1} = \{\rho^{(U-2)\times k+1},\ldots,\rho^{(U-1)\times k}\}, I_U = \{\rho^{(U-1)\times k+1},\ldots,\rho^{2^s}\}. \end{array} \tag{15}$$

In (15), $U = \lceil \frac{2^s}{k} \rceil$, $2^s = k \cdot \left(\lceil \frac{2^s}{k} \rceil - 1 \right) + |I_U|$, $|I_U| \in \{1,\ldots,k\}$. Further let us use the following notation.

$$I^B = \{I_1,\ldots,I_U\}.$$

We will refer to I^B as to an *interval partition of rank k* of hypercube $\{0,1\}^{|B|}$.

The basic idea of the approach presented in [75] is to allow a SAT solver to process all vectors from an arbitrary interval $I \in I^B$ incrementally when solving the subproblems from a SAT partitioning $P(C,B)$.

It is possible to define the effectiveness of $P(C,B)$ with respect to a complete deterministic incremental SAT solver $\mathcal{A}$ in a similar way as it was done for (7).

Associate with $\{0, 1\}^{|B|}$ and its interval partition I^B of the kind (15) a value η, the spectrum of which is formed by all possible runtimes of $\mathcal{A}$ on intervals from I^B. The possibility space associated with η is I^B. Define a uniform distribution over I^B. Let us denote by $T(\mathcal{A}, C, B, I^B)$ the time required to solve all problems from a SAT partitioning $P(C, B)$ by incremental SAT solver $\mathcal{A}$ that employs the interval partition I^B of the hypercube $\{0, 1\}^{|B|}$. Then, similar to (7) it holds that

$$T(\mathcal{A}, C, B, I^B) = U \cdot E[\eta]. \tag{16}$$

Now let us define a pseudo-Boolean function the values of which will estimate (16) for various B and the corresponding interval partitions (15) with fixed $\mathcal{A}$, C, and N:

$$\Upsilon_{\mathcal{A},C,N}(B, I^B) = \frac{U}{N} \cdot \sum_{j=1}^{N} \eta^j. \tag{17}$$

In (17) by η^j, $j \in \{1, \ldots, N\}$ we denote the values of random variable η observed in N independent observations. In each experiment an interval I is first randomly chosen from I^B according to the uniform distribution. Then an incremental SAT solver $\mathcal{A}$ is launched on I. The observed value of η is the total runtime of $\mathcal{A}$ over all problems from I.

5 Employed Optimization Algorithms

The objective functions (13), (14), and (17) proposed in two previous sections are pseudo-Boolean black-box costly stochastic functions. Therefore they can be minimized by pseudo-Boolean black-box optimization algorithms based on direct calculations (see, e.g., [1, 41, 45, 57]). In this section several such algorithms are described that are further used in Sect. 6. The list is presented below.

1. steepest ascent hill climbing (SAHC);
2. first-choice hill climbing (FCHC);
3. first-choice hill climbing with variables-based jump (FCHCVJ);
4. tabu search (TS);
5. tabu search with activity-based escape (TSAE);
6. simulated annealing (SA);
7. (1+1) evolutionary algorithm ((1+1)-EA);
8. genetic algorithm (GA).

Some features of the considered algorithms are outlined in Table 1. Note that all three objective functions are extremely costly, that is why in almost all employed optimization algorithms (except SAHC and FCHC) a *tabu list* is used to prevent calculating an objective function's value at any point more than once. If not mentioned otherwise, it is implied that the tabu list stores all processed points. In

Table 1 Some features of the considered optimization algorithms

Algorithm	Stochastic	Tabu lists	Trajectory-based	Escaping local minima
SAHC	No	No	Yes	No
FCHC	Yes	No	Yes	No
FCHCVJ	Yes	Yes	Yes	Yes
TS	No	Yes	Yes	Yes
TSAE	No	Yes	Yes	Yes
SA	Yes	Yes	Yes	Yes
(1+1)-EA	Yes	Yes	No	–
GA	Yes	Yes	No	–

trajectory-based algorithms a successor solution is sought in a neighborhood of a current solution. *Escaping local minima* means that a trajectory-based algorithm uses a specific heuristic to move out of points with neighborhoods in which it could not improve a function's value.

Since the functions (13), (14), and (17) are pseudo-Boolean, it means that they take a Boolean vector $\lambda = (\lambda_1, \ldots, \lambda_n)$ as an input. Such a vector is considered as a point in the search space $\{0, 1\}^n$. Thus, it is quite natural to operate with the Hamming distance [30] between points to form a neighborhood. To be more specific, we assume that a neighborhood of a point λ contains all points that are at Hamming distance at most H from λ. Hereinafter we refer to a point for which an objective function value has already been calculated as *processed point*. Similarly, a neighborhood is called *processed* if all its points are processed, otherwise it is called *unprocessed*.

Let us describe the mentioned optimization algorithms in more detail. In the corresponding pseudocodes, g stands for an objective function ((13), (14), or (17)), g_{best} and λ_{best}—for the *best known value* (BKV) of the objective function and the corresponding point, respectively. Every algorithm is given a starting point λ_{start}. It goes without saying that several parameters are common for all functions: CNF C, SAT solving algorithm $\mathcal{A}$, random sample size N (see Sect. 3). Some of the objective functions require additional parameters, but for brevity we do not mention them in pseudocodes. Instead, it is implied that all the required ones are given.

The *steepest ascent hill climbing* algorithm (see, e.g., [59]) calculates objective function values for all points from a current neighborhood and chooses the best candidate from them (in our case—the one with the lowest objective function value). If the best candidate is better than the current λ_{best}, then both λ_{best} and g_{best} are updated. If all points from the neighborhood are worse than the current λ_{best}, then a local minimum is reached and the algorithm stops. The pseudocode is shown in Algorithm 1. For a given point λ, the function `getRandOrdNeighb(λ,H)` returns the list of randomly ordered points that are at Hamming distance at most H from λ. Note that for this algorithm any ordering can be used, but random ordering is required for the first-choice hill climbing that is described further.

Input: Starting point λ_{start}, Hamming distance H, time limit t
1 $\langle \lambda_{best}, g_{best} \rangle \leftarrow \langle \lambda_{start}, g(\lambda_{start}) \rangle$
2 **repeat**
3 BestValueUpdated $\leftarrow$ false
4 CurNeigh $\leftarrow$ `getRandOrdNeighb`(λ_{best}, H)
5 **for** `each` λ `in` CurNeigh **do**
6 NewFuncValue $\leftarrow g(\lambda)$
7 **if** NewFuncValue $< g_{best}$ **then**
8 $\langle \lambda_{best}, g_{best} \rangle \leftarrow \langle \lambda$, NewFuncValue$\rangle$
9 BestValueUpdated $\leftarrow$ true
10 **until** `timeExceeded`(t) `or not` BestValueUpdated
11 **return** $\langle \lambda_{best}, g_{best} \rangle$

Algorithm 1: Steepest ascent hill climbing

In opposite to steepest ascent hill climbing, the *first-choice hill climbing* algorithm (see, e.g., [59]) immediately stops processing the current neighborhood as soon as λ_{best} is updated. Thus the pseudocode of first-choice hill climbing can be easily made by adding the line *Break* after line number 9 in Algorithm 1. Since both steepest ascent hill climbing and first-choice hill climbing do not have heuristics for escaping local minima, they are not used in experiments described in Sect. 6. However, the next four algorithms that we actually use for this purpose are based on either SAHC or FCHC.

Input: Starting point λ_{start}, Hamming distance H, time limit t
1 $\langle \lambda_{best}, g_{best} \rangle \leftarrow \langle \lambda_{start}, g(\lambda_{start}) \rangle$
2 $\lambda_{center} \leftarrow \lambda_{start}$
3 TabuList $\leftarrow \{\lambda_{start}\}$
4 **repeat**
5 NonTabuPoints $\leftarrow$ `getNonTabuNeighbors`$(\lambda_{center}, H,$ TabuList$)$
6 **if** NonTabuPoints $= \emptyset$ **then**
7 `Break`
8 $g_{candidate} \leftarrow \infty$
9 **for** `each` λ `in` NonTabuPoints **do**
10 NewFuncValue $\leftarrow g(\lambda)$
11 **if** NewFuncValue $< g_{candidate}$ **then**
12 $\langle \lambda_{candidate}, g_{candidate} \rangle \leftarrow \langle \lambda$, NewFuncValue$\rangle$
13 **if** $g_{candidate} < g_{best}$ **then**
14 $\langle \lambda_{best}, g_{best} \rangle \leftarrow \langle \lambda_{candidate}, g_{candidate} \rangle$
15 `updateTabuList`$(\lambda_{candidate},$ TabuList$)$
16 $\lambda_{center} \leftarrow \lambda_{candidate}$
17 **until** `timeExceeded`(t)
18 **return** $\langle \lambda_{best}, g_{best} \rangle$

Algorithm 2: The tabu search algorithm

Tabu search was proposed in [26]. We implemented a tabu search algorithm on the basis of steepest ascent hill climbing. A tabu list is used where the last 1000 best points from the processed neighborhoods are stored. In opposite to first-choice hill climbing and steepest ascent hill climbing, if a local minimum is reached, the algorithm does not stop. Instead, the best point from the neighborhood is chosen, and the processing of its neighborhood is started. Algorithm 2 shows the corresponding pseudocode. The function `getNonTabuNeighbors(λ,H,TabuList)` returns randomly ordered points from the neighborhood of λ with Hamming distance at most H that are not in the tabu list. The function `updateTabuList()` adds a given point to the top of the tabu list. If the tabu list is full, the last point is removed from it before adding a new one. It means that repeated calculations are possible in this algorithm. By $\lambda_{candidate}$ and $g_{candidate}$ we denote the best non-tabu point from the current neighborhood and the objective function value for it, respectively.

Tabu search with activity-based escape was proposed in [63]. Unlike the tabu search algorithm described above, it uses another heuristic for escaping local minima, and stores all processed points. More specifically, the information about processed points is stored in two tabu lists L_1 and L_2. L_1 contains processed points with processed neighborhoods, while L_2 contains processed points with unprocessed neighborhoods. In particular, each point in L_2 is stored along with a Boolean vector that specifies which points from the corresponding neighborhood have not been processed yet. The pseudocode is presented in Algorithm 3. The list of points that have not been processed so far according to both tabu lists is formed by the function `getNonTabuNeighbors2`. The function `markPointInTabuLists`(λ, L_1, L_2) adds the point λ to L_2 and then marks λ as processed in all neighborhoods of points from L_2 that contain λ. If as a result the neighborhood of some point λ becomes processed, the point is removed from L_2 and is added to L_1. If all points in the neighborhood of λ_{center} have been processed but g_{best} has not been improved, then some point from L_2 is chosen as λ_{center} to escape the reached local minimum. It is done via the function `getNewCenter(`L_2`)` that chooses from L_2 a point with the largest total conflict activity of Boolean variables from the corresponding decomposition set. This activity is taken from a CDCL-based [47] SAT solving algorithm $\mathcal{A}$.

Input: Starting point λ_{start}, Hamming distance H, time limit t

```
1  ⟨λ_best, g_best⟩ ← ⟨λ_start, g(λ_start)⟩
2  λ_center ← λ_start
3  L_1 ← ∅
4  L_2 ← {λ_start}
5  repeat
6  |  BestValueUpdated ← false
7  |  NonTabuPoints ← getNonTabuNeighbors2(λ_center, H, L_1, L_2)
8  |  for each λ in NonTabuPoints do
9  |  |  NewFuncValue ← g(λ)
10 |  |  markPointInTabuLists(λ, L_1, L_2)     // update tabu
   |  |   lists
11 |  |  if NewFuncValue < g_best then
12 |  |  |  ⟨λ_best, g_best⟩ ← ⟨λ, NewFuncValue⟩
13 |  |  |  BestValueUpdated ← true
14 |  if BestValueUpdated then
15 |  |  λ_center ← λ_best
16 |  else
17 |  |  λ_center ← getNewCenter(L_2)
18 until timeExceeded(t)
19 return ⟨λ_best, g_best⟩
```

Algorithm 3: Tabu search with activity-based escape

First-choice hill climbing with variables-based jump was proposed in [40, 74]. It is a first-choice hill climbing improved by a memory-based heuristic for escaping local minima. This heuristic employs two arrays of counters: $U^1, |U^1| = n$ and $U^2, |U^2| = n$. All elements of both arrays are initialized with zeros. If an objective function is calculated in a point λ, then for all i such that $\lambda_i = 1$ the counter U^1_i is increased by one. The second vector of counters is updated in the same manner, but only for such points in which g_{best} has been updated. If a local minimum is reached in a neighborhood of a current λ_{best}, then a new starting point is constructed by changing $2m$ elements of λ_{best} from 0 to 1. The elements to flip are picked as follows: m of them correspond to m lowest values in U^1 and the other m to the m highest values in U^2. The motivation here is to move to a point that simultaneously contains the variables that often resulted in an improvement of function's value and the variables that were rarely added in the process of the search so far to balance the exploitation and exploration. In all experiments described in Sect. 6, m was equal to 4. The pseudocode is shown in Algorithm 4. Functions `updateFirstCounter()` and `updateSecondCounter()` update U^1 and U^2, respectively. The function `getJumpPoint(λ,m)` constructs a new point by changing $2m$ elements of a given point λ from 0 to 1 as it was described above.

Input: Starting point λ_{start}, escape parameter m, Hamming distance H, time limit t

```
⟨λ_best, g_best⟩ ← ⟨λ_start, g(λ_start)⟩
λ_center ← λ_start
TabuList ← {λ_start}
U^1 = U^2 = 0                    // initialize with zero vectors
repeat
    NonTabuPoints ← getNonTabuNeighbors(λ_center, H, TabuList)
    BestValueUpdated ← false
    for each λ in NonTabuPoints do
        NewFuncValue ← g(λ)
        addPointTabuList(λ, TabuList)
        updateFirstCounter(λ, U^1)
        if NewFuncValue < g_best then
            ⟨λ_best, g_best⟩ ← ⟨λ, NewFuncValue⟩
            updateSecondCounter(λ_best, U^2)
            BestValueUpdated ← true
            Break
    if BestValueUpdated = true then
        λ_center ← λ_best
    else
        λ_center ← getJumpPoint(λ_best, m)
until timeExceeded(t)
return ⟨λ_best, g_best⟩
```

Algorithm 4: First-choice hill climbing with variables-based jump

The simulated annealing algorithm was proposed in [39]. Below we describe its variant from [63] aimed at minimizing objective functions of the considered type. It is based on steepest ascent hill climbing. A point λ from a current neighborhood becomes λ_{center} with the probability

$$Pr\{\lambda \to \lambda_{center}\} = \begin{cases} 1, & if\ g(\lambda) < g(\lambda_{best}) \\ \exp(-\frac{g(\lambda)-g(\lambda_{best}))}{T}), & if\ g(\lambda) \geq g(\lambda_{best}) \end{cases} \tag{18}$$

Here T corresponds to the "temperature of the environment" [39]. First, T is assigned a large value T_0. At each calculation of an objective function, T is decreased: $T = Q \cdot T$, where $Q \in (0, 1)$. Algorithm stops when T drops below a threshold value T_{thresh}. In [63] the following values were used: $T_0 = \frac{g(\lambda_{start})}{10}$, $Q = 0.9$, $T_{thresh} = 20$. The pseudocode is shown in Algorithm 5. The function `pointAccepted`$(g(\lambda), g_{best}, T)$ employs formula (18) to test if g becomes g_{best}.

Input: Starting point λ_{start}, Hamming distance H, time limit t, temperature decrease parameter Q, temperature threshold T_{thresh}

$\langle \lambda_{best}, g_{best} \rangle \leftarrow \langle \lambda_{start}, g(\lambda_{start}) \rangle$
$T = \frac{g(\lambda_{start})}{10}$
TabuList $\leftarrow \{\lambda_{start}\}$
repeat
  NonTabuPoints $\leftarrow$ `getNonTabuNeighbors`(λ_{best}, H, TabuList)
  if NonTabuPoints $= \emptyset$ **then**
    `Break`
  for `each` λ `in` NonTabuPoints **do**
    NewFuncValue $\leftarrow g(\lambda)$
    `addPointTabuList`(λ, TabuList)
    if `pointAccepted` (NewFuncValue, g_{best}, T) **then**
      $\langle \lambda_{best}, g_{best} \rangle \leftarrow \langle \lambda$, NewFuncValue$\rangle$
    $T = T \cdot Q$
until `timeExceeded`(t) `or` $T < T_{thresh}$
return $\langle \lambda_{best}, g_{best} \rangle$

Algorithm 5: The simulated annealing algorithm

The *(1+1) evolutionary* algorithm is described, e.g., in [52]. Below we employ this algorithm in the form that was used in [55] for minimization of the objective function (15). Unlike the original algorithm, it starts from a given point. Algorithm 6 shows the pseudocode. The function `mutate`($\lambda, \frac{1}{n}$) flips independently each element of a given point λ (which in turn is a Boolean vector of size n) with probability $\frac{1}{n}$. This function is run until a non-tabu point is obtained.

Input: Starting point λ_{start}, time limit t

$\langle \lambda_{best}, g_{best} \rangle \leftarrow \langle \lambda_{start}, g(\lambda_{start}) \rangle$
TabuList $\leftarrow \{\lambda_{start}\}$
repeat
  repeat
    $\lambda_{flipped} \leftarrow$ `mutate`($\lambda_{best}, \frac{1}{n}$)
  until $\lambda_{flipped}$ `not in` TabuList
  NewFuncValue $\leftarrow g(\lambda_{flipped})$
  `addPointTabuList`(λ, TabuList)
  if NewFuncValue $< g_{best}$ **then**
    $\langle \lambda_{best}, g_{best} \rangle \leftarrow \langle \lambda_{flipped}$, NewFuncValue$\rangle$
until `timeExceeded`(t)
return $\langle \lambda_{best}, g_{best} \rangle$

Algorithm 6: The (1+1) evolutionary algorithm

Finally, we<?pag ?> present a variant of *genetic algorithm* proposed in [55]. Each point $\lambda \in \{0, 1\}^n$ is considered as an *individual* (see, e.g., [45]), the fitness of which is determined by the value of an objective function in the corresponding point. Algorithm 7 shows the pseudocode. The sizes of all populations are fixed to K. The starting population is initialized via the function `initPopulation`(λ_{start}, K)

that constructs K replicas of λ_{start}. The key function of the genetic algorithm is the one used to move from the current population to the next. In Algorithm 7 it is `getNextPopulation`(P_{curr}, E, M, R). Assume that $P_{curr} = \{I_1, \ldots, I_K\}$ and $P_{next}, |P_{next}| = K$ are a current and a new population, respectively. P_{next} is formed by $E + M + R = K$ points that are chosen as follows. First E are chosen as the top E individuals with the best values of the objective function from P_{curr}. It is done in accordance with the so-called *elitism* principle (see, e.g., [45]). Next, we compute auxiliary values

$$p_j = \frac{\frac{1}{g(I_j)}}{\sum_{u=1}^{K} \frac{1}{g(I_u)}}, j = 1, \ldots, K$$

and use them to form probability distribution $D_{curr} = \{p_1, \ldots, p_K\}$. Then M individuals are chosen randomly from P_{curr} according to D_{curr}, undergo standard (1+1)-EA mutation and go into P_{next}. Finally, R pairs of individuals are chosen from P_{curr} according to D_{curr} to perform the standard two-point crossover. The result of crossover goes to P_{next}. The situation when the algorithm fails to improve the BKV during one generation is called *stagnation*. If the number of stagnations exceeds a given limit max_stag, then the algorithm restarts from the initial population based on K replicas of λ_{start}. Note that in this case the tabu list is not cleared. In [55], max_stag was varied from 100 to 300, while the constants were set to $K = 10, E = 2, M = R = 4$.

Input: Starting point λ_{start}, time limit t, evolution parameters K, E, M, R, max_stag

```
1  P ← initPopulation(λ_start, K)
2  ⟨λ_best, g_best⟩ ← ⟨λ_start, g(λ_start)⟩
3  TabuList ← {λ_start}
4  stag_num ← 0
5  repeat
6      stag_num ← stag_num + 1
7      for each λ in P do
8          if λ not in TabuList then
9              NewFuncValue ← g(λ)
10             addPointTabuList(λ, TabuList)
11             if NewFuncValue < g_best then
12                 ⟨λ_best, g_best⟩ ← ⟨λ, NewFuncValue⟩
13                 stag_num ← 0
14     if stag_num < max_stag then
15         P ← getNextPopulation(P, E, M, R)
16     else
17         stag_num ← 0
18         P ← initPopulation(λ_start, K)
19 until timeExceeded(t)
20 return ⟨λ_best, g_best⟩
```

Algorithm 7: The genetic algorithm

6 Experimental Results

In this section we apply the optimization algorithms described in the previous section to minimization of objective functions introduced in the chapter. As benchmarks we use the problems of finding effective SAT partitionings for several instances of SAT-based cryptanalysis of certain symmetric-key algorithms. In particular, each found BKV corresponds to a guess-and-determine attack on the considered symmetric-key algorithm (see Sect. 3.1). Below we first briefly describe the considered cryptanalysis problems, then say a few words about the implementations of objective functions employed in experiments and finally present the results of experimental evaluation.

6.1 Considered Problems

We consider SAT-based cryptanalysis of several symmetric-key cryptographic algorithms listed in Table 2. In particular, we consider both keystream generators and block ciphers. The keystream generators include: the A5/1 generator (see, e.g., [7]) that is still being used to encrypt traffic in the GSM standard in some countries; the alternating step generator (ASG) characterized by high encryption speed and good statistical properties [28]; the Trivium [10] and Grain_v1 [31] generators that were the finalists of the eSTREAM project aimed at identifying new fast and resistant stream ciphers. The block ciphers include the 2.5-round version of the Advanced Encryption Standard (AES [18]) with 128-bit key, the full (10-round) version of which is one of the most resistant block ciphers used today. Following the notation of [9], "x.5r" means x full rounds and the final round. Another block cipher we study is the 8-round version of the Magma block cipher (also known as GOST 28147-89, see, e.g., [16]). This very weakened version of Magma was studied by SAT-based cryptanalysis in [2, 17]. The full (32-round) version of Magma was used in USSR and Russian cryptographic standards from 1989 to 2019.

Recall that the *known plaintext scenario* [50] in application to a keystream generator means that the attacker has access to a keystream fragment of size m and tries to find the registers' state of size n that produced this fragment. In some cases, the registers' state coincides with the secret key, in other cases it does not, but the secret key can be efficiently computed based on the corresponding state (see, e.g., [48]). Block ciphers construct one block of a ciphertext of a fixed size based on a fixed secret key and one block of a plaintext (as a rule, of the same size as a block of a ciphertext). Known plaintext scenario for block ciphers [50] implies that the attacker has access to known plaintexts and the corresponding ciphertexts, and tries to find the secret key.

It should be noted that the considered objective functions were applied to SAT-based cryptanalysis of several other keystream generators: Bivium (a simplified version of Trivium) and Grain_v0 [63]; Rabbit and Mickey [75]; Salsa20 [74];

Table 2 Features of studied cryptographic algorithms. Sizes are given in bits

Algorithm	Registers' state	Keystream
Keystream generators		
A5/1	64	114
AES-96	96	112
Trivium	288	300
Grain_v1	160	200
Algorithm	Secret key	Plaintext
Block ciphers		
AES-128-2.5r	128	384
Magma-8r	256	786

Shrinking and Self-shrinking [72]; Bivium and two other simplified versions of Trivium (Trivium64 and Trivium96) [54]. For brevity, in this study none of them is considered.

All considered cryptanalysis problems were reduced to SAT via the TRANSALG tool [66]. Note that the set X^{in} that is given to some optimization algorithms as the starting point is constructed differently for keystream generators and for block ciphers. More specifically, for keystream generators X^{in} consists of the Boolean variables that encode starting registers' state that has to be found. Recall that a block cipher takes as an input a pair (secret key, plaintext). Let S_1 be a Boolean circuit that specifies a function of the kind $h : \{0, 1\}^{n+m} \to \{0, 1\}^m$ that corresponds to the considered block cipher. This circuit has $n + m$ inputs, where n is the secret key size in bits, and m—analyzed plaintext size in bits (i.e., m can correspond to several known plaintexts). Note that in the known plaintext scenario the corresponding plaintext is known. Then, following [66] it can be shown that for this case one can efficiently transit from the circuit S_1 to a circuit S_2 that has n inputs and m outputs. The circuit S_2 specifies some function of the kind (2). If an inversion problem is solved for this function, then the corresponding secret key is found. In this case in the template CNF constructed for S_2 the set X^{in} contains Boolean variables that encode a secret key.

As it was already stated in Sect. 4, in each studied SAT-based cryptanalysis problem X^{in} is a SUPBS for the corresponding SAT instance. This allows us to significantly reduce the search space of the optimization problem. Thus, in the computational experiments the objective functions are minimized over the search spaces of the following sizes: 2^{64} for A5/1; 2^{96} for ASG-96; 2^{288} for Trivium; 2^{160} for Grain_v1; 2^{128} for AES-128-2.5r; 2^{256} for Magma-8r. Therefore, we have 6 hard pseudo-Boolean black-box optimization problems.

Table 3 Information on software tools implementing the proposed approach

Tool	Functions	Algorithms
PDSAT	(13), (14), (17)	TSAE, SA
ALIAS	(17)	FCHCVJ, TS, (1+1)-EA
CRYPTOEV	(13),(14)	(1+1)-EA, GA

6.2 Implementations of Objective Functions

The objective functions (13), (14), and (17) have been implemented in three software tools: PDSAT[1] [63], ALIAS[2] [40], and CRYPTOEV[3] [55]. They employ different combinations of the objective functions and the algorithms for their minimization. Table 3 shows the optimization algorithms (see Sect. 5) implemented in these tools.

The PDSAT tool is implemented in C++, the other two use Python. All these tools use CDCL solvers [47] to solve SAT instances from a SAT partitioning. In the following subsections, the results of all experiments have been obtained using one of the three tools mentioned in Table 3.

6.3 Finding Effective SAT Partitionings

The experiments aimed at minimization of the objective functions (13), (14), and (17) on considered benchmarks were performed in different papers at different times. For these reasons, we are not able to provide the results of each optimization algorithm for minimization of each problem. Thus we will limit the presentation to only published results accompanied by the corresponding citations.

Recall that for each problem all optimization algorithms were given the corresponding Boolean representation of X^{in} as a starting point λ_{start}. For instance, for A5/1 $X^{in} = \{x_1, x_2, \ldots, x_{64}\}$, so $\lambda_{start} = (1, 1, \ldots, 1)$, $|\lambda_{start}| = 64$. Since all studied objective functions are extremely costly, in all trajectory-based optimization algorithms the Hamming distance H was set to 1.

The experiments were conducted on the computing cluster "Academician V.M. Matrosov".[4] This cluster has nodes of two types—the first one is equipped with 2 × 16-core AMD Opteron 6276 CPUs (32 CPU cores in total) and 64 Gb of RAM, while the second has 2 × 18-core Intel Xeon E5-2695 CPUs and 128 Gb of RAM. Table 4 shows parameters of the conducted experiments. Here "Func." stands for objective function and "Cores" for the number of CPU cores. Note that in

[1] https://github.com/Nauchnik/pdsat.

[2] https://github.com/Nauchnik/alias.

[3] https://github.com/lytr777/CryptoEv.

[4] Irkutsk Supercomputer Center of SB RAS, http://hpc.icc.ru.

Table 4 Parameters of the performed experiments

Problem	Func.	Source	Tool	Node type	Cores	Time limit	Sample size (N)
A5/1	(13)	[63]	PDSAT	AMD	160	24 h	10,000
ASG-96	(14)	[55]	CryptoEv	Intel	180	24 h	from 10 to 800
	(17)	[73]	PDSAT	Intel	360	12 h	1000
Trivium	(14)	[65]	PDSAT	Intel	360	24 h	1000
	(17)	[75]	ALIAS	Intel	36	3 × 24 h	100
Grain_v1	(17)	[75]	ALIAS	Intel	36	3 × 24 h	100
AES-128-2.5	(14)	[65]	PDSAT	Intel	360	24 h	1000
Magma-8r	(14)	[65]	PDSAT	Intel	360	24 h	1000

Table 5 Optimization algorithms (and the corresponding found BKVs) applied to each pair (problem, objective function). "-" means that the problem has not been analyzed via the corresponding objective function. For each problem the BKV (and the corresponding algorithm) among all applied objective functions is marked with bold

Problem	BKV for (13)		BKV for (14)		BKV for (17)	
A5/1	**4.64e+08**	**TSAE** [63]	-	-	-	-
	4.78e+08	SA [63]				
ASG-96	-	-	3.72e+06	GA [55]	**1.06e+06**	**TSAE** [73]
			6.76e+06	(1+1)-EA [55]		
Trivium	-	-	2.04e+41	TSAE [65]	4.46e+43	TS [75]
					2.46e+41	FCHCVJ [75]
					7.15e+40	**(1+1)-EA** [75]
Grain_v1	-	-	-	-	**2.85e+30**	**TS** [75]
					4.07e+30	FCHCVJ [75]
					4.69e+30	(1+1)-EA [75]
AES-128-2.5r	-	-	**1.45e+15**	**TSAE** [65]	-	-
Magma-8r	-	-	**3.55e+22**	**TSAE** [65]	-	-

CryptoEv the sample size was varied from 10 to 800 [55], while in all other cases it was constant. In the case of ALIAS, three runs of each optimization algorithm were performed, then the best result out of them was chosen. In other cases, there was only one run.

Table 5 shows optimization algorithms employed for every pair (problem, objective function), as well as the corresponding BKVs. Figure 1 shows how the objective function (14) was minimized via (1+1)-EA and GA on ASG-96, while Fig. 2 shows how (17) was minimized via TS, FCHCVJ, and (1+1)-EA on Trivium and Grain_v1. In all these figures, x-axis corresponds to the time in seconds elapsed since the algorithm start, while y-axis corresponds to the objective function's values for λ_{best} in logarithmic scale.

It is not possible to directly compare all employed optimization algorithms since their different combinations were applied to different problems. However, some

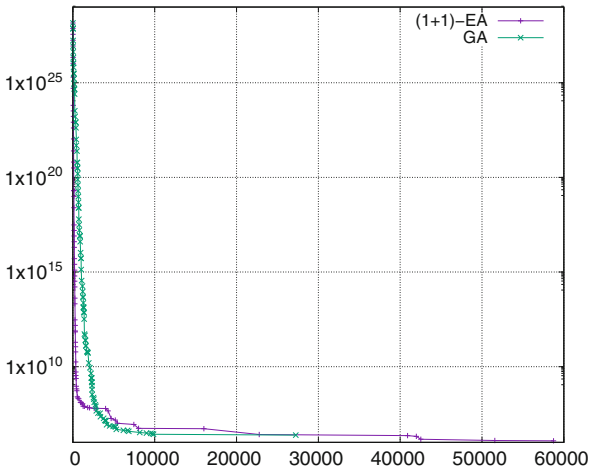


Fig. 1 Minimization of the objective function (14) on ASG-96 by (1+1)-EA and GA

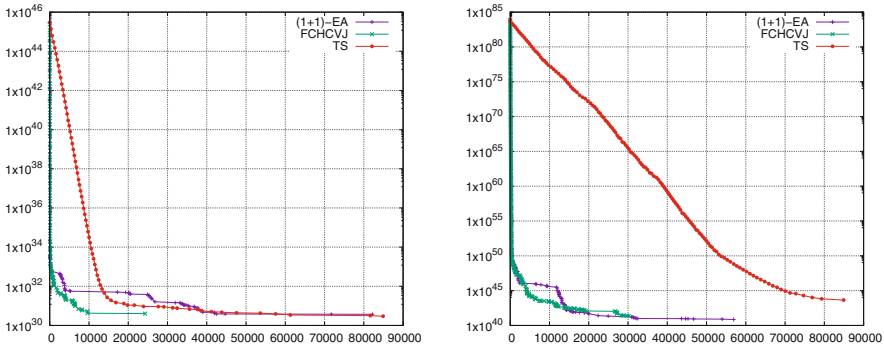


Fig. 2 Minimization of the objective function (17) on Trivium and Grain_v1 by (1+1)-EA, FCHCVJ, and TS. (**a**) Grain_v1. (**b**) Trivium

conclusions can be made. From the presented results it follows that the algorithms' diversity makes sense for the problems of the considered type. When minimizing the objective function (17) on Grain_v1, the TS, (1+1)-EA, and FCHCVJ algorithms showed comparable results, but TS outperformed its competitors. On Trivium, (1+1)-EA is slightly better than FCHCVJ, while TS showed extremely bad results. The main reason is that TS thoroughly traverses neighborhoods (see Sect. 5), and in the case of Trivium the neighborhoods are quite large. When minimizing the objective function (14) on ASG-96, GA slightly outperformed (1+1)-EA. On A5/1, TSAE showed better results than SA when minimizing the objective function (13). As for AES-128-2.5r and Magma-8r, only TSAE was employed for each of them.

6.4 Solving Hard SAT Instances via Found Partitionings

To make sure that the constructed runtime estimations are accurate, we solved several SAT-based cryptanalysis instances via the constructed SAT partitionings. Recall that each SAT partitioning is constructed using a decomposition set that is defined by some λ (see Sect. 3). We used λ_{best} found for A5/1 and ASG-96 (see the best results in Table 5) to implement SAT-based guess-and-determine attacks on these keystream generators. This choice was motivated by the fact that they correspond to the smallest runtime estimations among all presented in Table 5.

For the first time SAT-based cryptanalysis of A5/1 was considered in [64]. In that paper, the decomposition set consisting of 31 variables (out of 64) was constructed manually based on the analysis of A5/1 algorithmic features. Using this partitioning, 10 SAT-based cryptanalysis instances for A5/1 were solved [63] in the volunteer computing project SAT@home [56]. Also, in SAT@home the same 10 instances were solved by the decomposition set found via TSAE when minimizing the objective function (13) [63]. Note that this decomposition set consists of 32 variables. In both experiments the average runtime on simplified instances from the corresponding SAT partitionings was comparable.

As for ASG-96, 20 SAT-based cryptanalysis instances were solved on the computing cluster "Academician V.M. Matrosov" via the decomposition set found by the TSAE optimization algorithm when minimizing (17) [73]. The decomposition set consisted of 30 variables (out of 96). Is should be noted, that the corresponding SAT partitioning significantly outperformed by average runtime the one constructed manually [73].

According to the best obtained estimations, Trivium, Grain_v1, AES-128-2.5r, and Magma-8r are way too hard for processing the corresponding SAT partitionings in reasonable time. Following [63, 66], we studied weakened SAT-based cryptanalysis problems for Trivium and Grain_v1 by assigning correct values to a portion of variables from X^{in} [75]. In particular, we assigned values to k (out of n) last variables from X^{in}. In such a weakened variant, the first $n-k$ variables from X^{in} are unknown, thus, λ_{best} is picked from the set of all possible subsets of $\{x_1, \ldots, x_{n-k}\}$. The following weakened variants were considered: $k = 96$ for Grain_v1 and $k = 134$ for Trivium, so λ_{best} was picked from subsets of $\{x_1, \ldots, x_{64}\}$ and $\{x_1, \ldots, x_{154}\}$, respectively. (1+1)-EA was run on both weakened problems for 1 h on one cluster node. As a result, for Grain_v1 (Trivium) λ_{best} consisting of 22 (26) variables was found. By processing the corresponding SAT partitionings, for both Grain_v1 and Trivium three randomly generated weakened SAT-based cryptanalysis problems were successfully solved on one cluster node.

It turned out, that for each solved problem the runtime estimation is accurate since it is quite close to the corresponding real solving time.

7 Conclusion

In this chapter we discussed how the black-box optimization methods can be applied to finding good partitionings for hard instances of the Boolean satisfiability problem. We narrow this area to the cases when a special partitioning method can be applied, since it allows one to formulate the problem of finding effective partitionings as a black-box optimization problem. The presented approach works quite well with cryptanalysis problems in SAT form, and sometimes leads to the state-of-the-art results. In contrast to the majority of cryptographic attacks that require in-depth analysis of the underlying cipher and a lot of manual tinkering with peculiarities of a particular construction, the approach presented in the chapter is completely automatic. We give a problem formulation to a black-box optimization algorithm, launch it and wait while it finds a minimum. While it is by no means a silver bullet, and the approach has some pitfalls, the results of experiments appear to be quite promising. In combination with the relevant research in neighboring areas, the presented results make us hope that the potential of SAT in the context of solving hard combinatorial problems is far from being exhausted.

Acknowledgments The research was funded by Russian Science Foundation (project No. 16-11-10046). Stepan Kochemazov is additionally supported by the Council for Grants of the President of the Russian Federation (stipend SP-2017.2019.5).

References

1. Audet, C., Hare, W.: Derivative-Free and Blackbox Optimization. Springer Series in Operations Research and Financial Engineering, Springer, Berlin (2017). https://doi.org/10.1007/978-3-319-68913-5
2. Babenko, L.K., Maro, E.A., Anikeev, M.V.: Application of algebraic cryptanalysis to MAGMA and PRESENT block encryption standards. In: Proceedings of IEEE 11th International Conference on Application of Information and Communication Technologies (AICT), pp. 1–7 (2017). https://doi.org/10.1109/ICAICT.2017.8686954
3. Balyo, T., Sinz, C.: Parallel satisfiability. In: Hamadi, Y., Sais, L. (eds.) Handbook of Parallel Constraint Reasoning, pp. 3–29. Springer, Berlin (2018). https://doi.org/10.1007/978-3-319-63516-3_1
4. Bard, G.V.: Algebraic Cryptanalysis, 1st edn. Springer, Berlin (2009)
5. Bessiere, C., Katsirelos, G., Narodytska, N., Walsh, T.: Circuit complexity and decompositions of global constraints. In: Proceedings of the 21st International Joint Conference on Artificial Intelligence - IJCAI'09, pp. 412–418 (2009)
6. Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.): Handbook of Satisfiability. Frontiers in Artificial Intelligence and Applications, vol. 185. IOS Press, Amsterdam (2009)
7. Biryukov, A., Shamir, A., Wagner, D.A.: Real time cryptanalysis of A5/1 on a PC. In: Schneier, B. (ed.) Fast Software Encryption, 7th International Workshop, FSE 2000. Lecture Notes in Computer Science, vol. 1978, pp. 1–18. Springer, Berlin (2000). https://doi.org/10.1007/3-540-44706-7_1
8. Boros, E., Hammer, P.L.: Pseudo-Boolean optimization. Discrete Appl. Math. **123**(1–3), 155–225 (2002)

9. Bouillaguet, C., Derbez, P., Fouque, P.: Automatic search of attacks on round-reduced AES and applications. In: Rogaway, P. (ed.) Advances in Cryptology - CRYPTO 2011 - 31st Annual Cryptology Conference. Lecture Notes in Computer Science, vol. 6841, pp. 169–187. Springer, Berlin (2011). https://doi.org/10.1007/978-3-642-22792-9_10
10. Cannière, C.D., Preneel, B.: Trivium. In: Robshaw, M.J.B., Billet, O. (eds.) New Stream Cipher Designs - The eSTREAM Finalists. Lecture Notes in Computer Science, vol. 4986, pp. 244–266. Springer, Berlin (2008)
11. Carter, K., Foltzer, A., Hendrix, J., Huffman, B., Tomb, A.: SAW: the software analysis workbench. In: Boleng, J., Taft, S.T. (eds.) Proceedings of the 2013 ACM SIGAda Annual Conference on High Integrity Language Technology, HILT, pp. 15–18. ACM, New York (2013). https://doi.org/10.1145/2527269.2527277
12. Chang, C.L., Lee, R.C.T.: Symbolic Logic and Mechanical Theorem Proving, 1st edn. Academic Press, Cambridge (1997)
13. Clarke, E.M., Kroening, D., Lerda, F.: A tool for checking ANSI-C programs. In: Jensen, K., Podelski, A. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, 10th International Conference, TACAS 2004. Lecture Notes in Computer Science, vol. 2988, pp. 168–176. Springer, Berlin (2004). https://doi.org/10.1007/978-3-540-24730-2_15
14. Cook, S.A.: The complexity of theorem-proving procedures. In: Proceedings of the 3rd Annual ACM Symposium on Theory of Computing, pp. 151–158 (1971)
15. Cook, S.A., Mitchell, D.G.: Finding hard instances of the satisfiability problem: a survey. In: Satisfiability Problem: Theory and Applications. DIMACS Series in Discrete Mathematics and Theoretical Computer Science, vol. 35, pp. 1–18. American Mathematical Society, Providence (1996)
16. Courtois, N.T.: Algebraic complexity reduction and cryptanalysis of GOST. IACR Cryptol. ePrint Arch. **2011**, 626 (2011). http://eprint.iacr.org/2011/626
17. Courtois, N.T., Gawinecki, J.A., Song, G.: Contradiction immunity and guess-then-determine attacks on GOST. Tatra Mt. Math. Publ. **53**(1), 2–13 (2012)
18. Daemen, J., Rijmen, V.: The Design of Rijndael: AES - The Advanced Encryption Standard. Information Security and Cryptography. Springer, Berlin (2002). https://doi.org/10.1007/978-3-662-04722-4
19. Dowling, W.F., Gallier, J.H.: Linear-time algorithms for testing the satisfiability of propositional horn formulae. J. Log. Program. **1**(3), 267–284 (1984)
20. Eén, N., Sörensson, N.: An extensible SAT-solver. In: Giunchiglia, E., Tacchella, A. (eds.) Theory and Applications of Satisfiability Testing, 6th International Conference, SAT 2003. Selected Revised Papers. Lecture Notes in Computer Science, vol. 2919, pp. 502–518. Springer, Berlin (2003). https://doi.org/10.1007/978-3-540-24605-3_37
21. Eén, N., Sörensson, N.: Temporal induction by incremental SAT solving. Electr. Notes Theor. Comput. Sci. **89**(4), 543–560 (2003)
22. Eibach, T., Pilz, E., Völkel, G.: Attacking Bivium using SAT solvers. In: Büning, H.K., Zhao, X. (eds.) Theory and Applications of Satisfiability Testing - SAT 2008, 11th International Conference, SAT 2008. Lecture Notes in Computer Science, vol. 4996, pp. 63–76. Springer, Berlin (2008). https://doi.org/10.1007/978-3-540-79719-7_7
23. Feller, W.: An Introduction to Probability Theory and Its Applications, Volume II. Wiley, New York (1971)
24. Franco, J., Martin, J.: A history of satisfiability. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability. Frontiers in Artificial Intelligence and Applications, vol. 185, pp. 3–74. IOS Press, Amsterdam (2009)
25. Garey, M.R., Johnson, D.S.: Computers and Intractability, vol. 174. Freeman, New York (1979)
26. Glover, F.: Future paths for integer programming and links to artificial intelligence. Comput. OR **13**(5), 533–549 (1986)
27. Gomes, C.P., Sabharwal, A.: Exploiting runtime variation in complete solvers. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability. Frontiers in Artificial Intelligence and Applications, vol. 185, pp. 271–288. IOS Press, Amsterdam (2009)

28. Günther, C.G.: Alternating step generators controlled by de Bruijn sequences. In: Chaum, D., Price, W.L. (eds.) Advances in Cryptology - EUROCRYPT'87, Workshop on the Theory and Application of Cryptographic Techniques. Lecture Notes in Computer Science, vol. 304, pp. 5–14. Springer, Berlin (1987). https://doi.org/10.1007/3-540-39118-5_2
29. Hamadi, Y., Jabbour, S., Sais, L.: Manysat: a parallel SAT solver. J. Satisf. Boolean Model. Comput. **6**(4), 245–262 (2009)
30. Hamming, R.W.: Error detecting and error correcting codes. Bell Syst. Tech. J. **29**(2), 147–160 (1950). https://doi.org/10.1002/j.1538-7305.1950.tb00463.x
31. Hell, M., Johansson, T., Maximov, A., Meier, W.: The Grain family of stream ciphers. In: Robshaw, M.J.B., Billet, O. (eds.) New Stream Cipher Designs - The eSTREAM Finalists. Lecture Notes in Computer Science, vol. 4986, pp. 179–190. Springer, Berlin (2008)
32. Heule, M., Kullmann, O., Wieringa, S., Biere, A.: Cube and conquer: guiding CDCL SAT solvers by lookaheads. In: Eder, K., Lourenço, J., Shehory, O. (eds.) Hardware and Software: Verification and Testing - 7th International Haifa Verification Conference, HVC 2011. Lecture Notes in Computer Science, vol. 7261, pp. 50–65. Springer, Berlin (2011). https://doi.org/10.1007/978-3-642-34188-5_8
33. Heule, M.J.H., Kullmann, O., Marek, V.W.: Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer. In: Creignou, N., Le Berre, D. (eds.) Theory and Applications of Satisfiability Testing – SAT 2016. Lecture Notes in Computer Science, vol. 9710, pp. 228–245. Springer, Berlin (2016)
34. Hyvärinen, A.E.J.: Grid based propositional satisfiability solving. Ph.D. Thesis, Aalto University (2011)
35. Hyvärinen, A.E.J., Junttila, T.A., Niemelä, I.: Partitioning SAT instances for distributed solving. In: Fermüller, C.G., Voronkov, A. (eds.) Logic for Programming, Artificial Intelligence, and Reasoning, LPAR-17, pp. 372–386. Springer, Berlin (2010). https://doi.org/10.1007/978-3-642-16242-8_27
36. Janicic, P.: URSA: a system for uniform reduction to SAT. Log. Meth. Comput. Sci. **8**(3), 1–39 (2012)
37. Järvisalo, M., Junttila, T.: Limitations of restricted branching in clause learning. Constraints **14**(3), 325–356 (2009)
38. Järvisalo, M., Biere, A., Heule, M.: Simulating circuit-level simplifications on CNF. J. Autom. Reason. **49**(4), 583–619 (2012)
39. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983)
40. Kochemazov, S., Zaikin, O.: ALIAS: a modular tool for finding backdoors for SAT. In: Beyersdorff, O., Wintersteiger, C.M. (eds.) Theory and Applications of Satisfiability Testing - 21st International Conference, SAT 2018. Lecture Notes in Computer Science, vol. 10929, pp. 419–427. Springer, Berlin (2018). https://doi.org/10.1007/978-3-319-94144-8_25
41. Kolda, T.G., Lewis, R.M., Torczon, V.: Optimization by direct search: new perspectives on some classical and modern methods. SIAM Rev. **45**(3), 385–482 (2003)
42. Kroening, D.: Software verification. In: Biere et al. [6], pp. 505–532
43. Lafitte, F.: Cryptosat: a tool for SAT-based cryptanalysis. IET Inf. Secur. **12**(6), 463–474 (2018). https://doi.org/10.1049/iet-ifs.2017.0176
44. Levin, L.: Universal sequential search problems. Probl. Inf. Transm. **9**, 265–266 (1973)
45. Luke, S.: Essentials of Metaheuristics, 2nd edn. Lulu, Morrisville (2013). http://cs.gmu.edu/~sean/book/metaheuristics/
46. Marques-Silva, J.P., Sakallah, K.A.: GRASP - a new search algorithm for satisfiability. In: Rutenbar, R.A., Otten, R.H.J.M. (eds.) Proceedings of the 1996 IEEE/ACM International Conference on Computer-Aided Design, ICCAD 1996, pp. 220–227. IEEE Computer Society/ACM, New York (1996). https://doi.org/10.1109/ICCAD.1996.569607
47. Marques-Silva, J.P., Lynce, I., Malik, S.: Conflict-driven clause learning SAT solvers. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability. Frontiers in Artificial Intelligence and Applications, vol. 185, pp. 131–153. IOS Press, Amsterdam (2009)

48. Maximov, A., Biryukov, A.: Two trivial attacks on trivium. In: Adams, C.M., Miri, A., Wiener, M.J. (eds.) Selected Areas in Cryptography, 14th International Workshop, SAC 2007, Revised Selected Papers. Lecture Notes in Computer Science, vol. 4876, pp. 36–55. Springer, Berlin (2007). https://doi.org/10.1007/978-3-540-77360-3_3
49. Mcdonald, C., Charnes, C., Pieprzyk, J.: Attacking Bivium with MiniSat. Tech. Rep. 2007/040, ECRYPT Stream Cipher Project (2007)
50. Menezes, A.J., Vanstone, S.A., Oorschot, P.C.V.: Handbook of Applied Cryptography, 1st edn. CRC Press, Boca Raton (1996)
51. Metropolis, N., Ulam, S.: The Monte Carlo Method. J. Am. Stat. Assoc. **44**(247), 335–341 (1949)
52. Mühlenbein, H.: How genetic algorithms really work: mutation and hillclimbing. In: Männer, R., Manderick, B. (eds.) Parallel Problem Solving from Nature 2, PPSN-II, pp. 15–26. Elsevier, Amsterdam (1992)
53. Otpuschennikov, I.V., Semenov, A.A., Gribanova, I., Zaikin, O., Kochemazov, S.: Encoding cryptographic functions to SAT using TRANSALG system. In: Kaminka, G.A., Fox, M., Bouquet, P., Hüllermeier, E., Dignum, V., Dignum, F., van Harmelen, F. (eds.) ECAI 2016 - 22nd European Conference on Artificial Intelligence. Frontiers in Artificial Intelligence and Applications, vol. 285, pp. 1594–1595. IOS Press, Amsterdam (2016). https://doi.org/10.3233/978-1-61499-672-9-1594
54. Pavlenko, A., Buzdalov, M., Ulyantsev, V.: Fitness comparison by statistical testing in construction of SAT-based guess-and-determine cryptographic attacks. In: Auger, A., Stützle, T. (eds.) Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2019, pp. 312–320 (2019). https://doi.org/10.1145/3321707.3321847
55. Pavlenko, A., Semenov, A.A., Ulyantsev, V.: Evolutionary computation techniques for constructing SAT-based attacks in algebraic cryptanalysis. In: Kaufmann, P., Castillo, P.A. (eds.) Applications of Evolutionary Computation - 22nd International Conference, EvoApplications 2019. Lecture Notes in Computer Science, vol. 11454, pp. 237–253. Springer, Berlin (2019). https://doi.org/10.1007/978-3-030-16692-2_16
56. Posypkin, M., Semenov, A.A., Zaikin, O.: Using BOINC desktop grid to solve large scale SAT problems. Comput. Sci. (AGH) **13**(1), 25–34 (2012)
57. Rios, L., Sahinidis, N.: Derivative-free optimization: a review of algorithms and comparison of software implementations. J. Global Optim. **56**, 1247–1293 (2013). https://doi.org/10.1007/s10898-012-9951-y
58. Robinson, J.A.: A machine-oriented logic based on the resolution principle. J. ACM **12**(1), 23–41 (1965). https://doi.org/10.1145/321250.321253
59. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach, 3rd edn. Prentice Hall, Upper Saddle River (2009)
60. Semenov, A.: Decomposition representations of logical equations in problems of inversion of discrete functions. J. Comput. Syst. Sci. Int. **48**, 718–731 (2009)
61. Semenov, A.A., Zaikin, O.: Using Monte Carlo method for searching partitionings of hard variants of Boolean satisfiability problem. In: Malyshkin, V. (ed.) Parallel Computing Technologies - 13th International Conference, PaCT 2015. Lecture Notes in Computer Science, vol. 9251, pp. 222–230. Springer, Berlin (2015). https://doi.org/10.1007/978-3-319-21909-7_21
62. Semenov, A.A., Zaikin, O.: On the accuracy of statistical estimations of SAT partitionings effectiveness in application to discrete function inversion problems. In: Kononov, A.V., Bykadorov, I.A., Khamisov, O.V., Davydov, I.A., Kononova, P.A. (eds.) Supplementary Proceedings of the 9th International Conference on Discrete Optimization and Operations Research and Scientific School (DOOR 2016). CEUR Workshop Proceedings, vol. 1623, pp. 261–275. CEUR-WS.org (2016)
63. Semenov, A., Zaikin, O.: Algorithm for finding partitionings of hard variants of Boolean satisfiability problem with application to inversion of some cryptographic functions. SpringerPlus **5**(1), 1–16 (2016)

64. Semenov, A.A., Zaikin, O., Bespalov, D., Posypkin, M.: Parallel logical cryptanalysis of the generator A5/1 in BNB-grid system. In: Malyshkin, V. (ed.) Parallel Computing Technologies - 11th International Conference, PaCT 2011. Lecture Notes in Computer Science, vol. 6873, pp. 473–483. Springer, Berlin (2011). https://doi.org/10.1007/978-3-642-23178-0_43
65. Semenov, A.A., Zaikin, O., Otpuschennikov, I.V., Kochemazov, S., Ignatiev, A.: On cryptographic attacks using backdoors for SAT. In: McIlraith, S.A., Weinberger, K.Q. (eds.) Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18), pp. 6641–6648. AAAI Press, Palo Alto (2018)
66. Semenov, A., Otpuschennikov, I., Gribanova, I., Zaikin, O., Kochemazov, S.: Translation of algorithmic descriptions of discrete functions to SAT with applications to cryptanalysis problems. Log. Meth. Comput. Sci. **16**, 29:1–29:42 (2020)
67. Soos, M.: Grain of Salt - an automated way to test stream ciphers through SAT solvers. In: Tools'10: Proceedings of the Workshop on Tools for Cryptanalysis, pp. 131–144 (2010)
68. Soos, M., Nohl, K., Castelluccia, C.: Extending SAT solvers to cryptographic problems. In: Kullmann, O. (ed.) Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009. Lecture Notes in Computer Science, vol. 5584, pp. 244–257. Springer, Berlin (2009). https://doi.org/10.1007/978-3-642-02777-2_24
69. Tseitin, G.S.: On the complexity of derivation in propositional calculus. In: A.O. Slisenko (ed.) Studies in Mathematics and Mathematical Logic, Part II, pp. 115–125. Steklov Mathematical Institute, Moscow (1968)
70. Wegener, I.: The Complexity of Boolean Functions. Wiley, Hoboken (1987)
71. Williams, R., Gomes, C.P., Selman, B.: Backdoors to typical case complexity. In: Gottlob, G., Walsh, T. (eds.) Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence, IJCAI-03, pp. 1173–1178. Morgan Kaufmann, Burlington (2003)
72. Zaikin, O.: SAT-based cryptanalysis: from parallel computing to volunteer computing. In: Voevodin, V.V., Sobolev, S. (eds.) Supercomputing - 5th Russian Supercomputing Days, RuSCDays 2019. Communications in Computer and Information Science, vol. 1129, pp. 701–712. Springer, Berlin (2019). https://doi.org/10.1007/978-3-030-36592-9_57
73. Zaikin, O., Kochemazov, S.: An improved SAT-based guess-and-determine attack on the alternating step generator. In: Nguyen, P.Q., Zhou, J. (eds.) Information Security - 20th International Conference, ISC 2017. Lecture Notes in Computer Science, vol. 10599, pp. 21–38. Springer, Berlin (2017). https://doi.org/10.1007/978-3-319-69659-1_2
74. Zaikin, O., Kochemazov, S.: Pseudo-boolean black-box optimization methods in the context of divide-and-conquer approach to solving hard SAT instances. In: OPTIMA 2018 (Supplementary Volume), pp. 76–87. DEStech Publications, Lancaster (2018)
75. Zaikin, O., Kochemazov, S.: On black-box optimization in divide-and-conquer SAT solving. Optimization Methods and Software pp. 1–25 (2019). https://doi.org/10.1080/10556788.2019.1685993
76. Zhang, H., Bonacina, M.P., Hsiang, J.: PSATO: a distributed propositional prover and its application to quasigroup problems. J. Symb. Comput. **21**(4), 543–560 (1996). https://doi.org/10.1006/jsco.1996.0030

The No Free Lunch Theorem: What Are its Main Implications for the Optimization Practice?



Loris Serafino

1 Overview

This chapter will reconsider the meaning and role of the *No Free Lunch Theorem* (henceforth NFLT) with the objective of stressing its implication for the practitioners, Engineers, and scientist, that face black-box optimization problems in particular in the industrial design practice. The increasing importance of optimization techniques in product design is obvious, and this is reflected by the annual amount of publications dedicated. The NFLT theorem for optimization and search has been introduced in the 1990s of the last century in a seminal series of work by Wolpert and Macready [1]. Roughly it states that any two algorithms (deterministic or stochastic) have the same average performance when compared on all the possible optimization problems. What does this mean for the Engineer who needs to optimize the performance of, just to mention, a novel telecommunication device or a turbofan engine? This chapter will try to answer this question.

The NFLT is a result that stands at the very core of the optimization practice, but there is more. We believe that the NFLT, black-box optimization, metaheuristics, and the underlying Bayesian interpretation that will be discussed in this chapter are so strongly interrelated that they are basically the same thing just observed from a different perspective. Black-box optimization is about *learning*[1] the geometry of an objective function knowing only the values at some points. Metaheuristics are

[1] With some abuse of language, in this chapter we will use the terms *learning* and *inferring* interchangeably.

L. Serafino (✉)
Australian College of Kuwait, Kuwait
e-mail: l.serafino@ack.edu.kw

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_12

implementations of this learning process in action. The NFLT is basically about the limits of this learning process. Given that the entry point for a discussion about the NFLT is somehow arbitrary, we start with an overview and basic terminology of black-box optimization and metaheuristics.

2 Black-Box Optimization as a Machine Learning Challenge

For each type of design problem there can be numerous if not an infinite number of alternative configurations or solutions. Optimization involves the attempt to identify the best of the alternatives.

A maximization problem with an explicit objective can in general be expressed in the following mathematical form: finding the value

$$\arg\max_{x \in \mathcal{H}} f(x), \tag{1}$$

where x is a given vector in a generic multidimensional space $\mathcal{H}$ and $f : \mathcal{H} \to \mathbb{R}$ is a function (called the *objective function*[2]) of the vector x and $\mathcal{H} \subset \mathbb{R}^n$is a (discrete or continuous, but here the focus will be on continuous) subset of the multidimensional real Euclidean space. From now on we will refer to $\mathcal{H}$ as the *search* or *sample space*.[3]In the black-box scenario, no analytic expression exists for accurately evaluating the response of a candidate solution. In this case what is known about the objective function consists just in a set of pairs of input and output gathered from a computational simulation or an experiment:

$$\mathcal{D} = \{(x_1, f(x_1)) \ldots, (x_n, f(x_n))\} \tag{2}$$

This chapter will conceptualize black-box optimization as a learning problem where a response landscape is to be inferred given $\mathcal{D}$: drawing samples from the unknown underlying function $f(x)$ and then move to the next most promising area where the maximum can be located (Fig. 1).

Since black-box optimization is a field which cuts across a variety of disciplines (from applied mathematics to engineering, etc.) it is important to agree at the outset about basic terms and definitions [2].

Definition 1 A *heuristic* is any technique (consisting of a rule or a set of rules) which seeks (and hopefully finds) good solutions at a reasonable computational cost.

[2]In metaheuristics optimization the objective function is often referred to as *fitness* function or simply *fitness*.

[3]Even if there is no explicit mention of constrains here the formulation is nonetheless enough general since they can be incorporated through an appropriate definition of the search space $\mathcal{H}$.

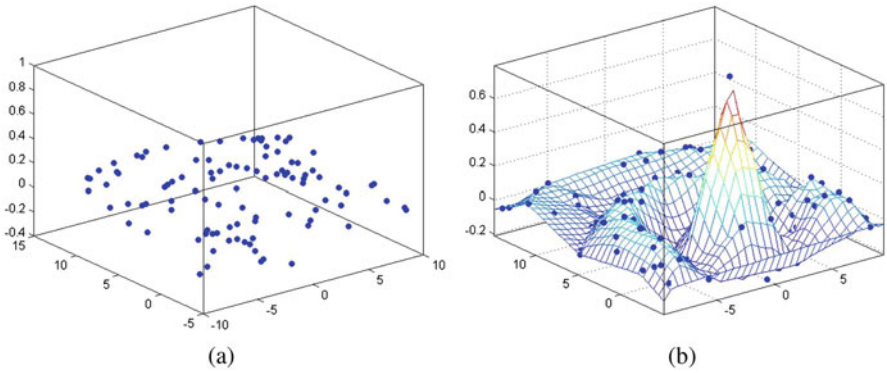


Fig. 1 In black-box optimization, the challenge is to infer promising regions for the maximum given only a discrete collection of values (**a**). The underlying analytic expression for the objective function (**b**) is unknown

A heuristic is approximate in the sense that it provides (hopefully) a good solution for relatively little effort, but it does not guarantee optimality. A particular type of heuristic approach is called *metaheuristic*, which is defined as follows:

Definition 2 A *metaheuristic* (or meta-heuristic) is the mechanism, or working logic, that guides the process by which a heuristics produces its solution.

The guiding process is a rule about possible local or global moves given the information collected in a sample of points.

Definition 3 An *algorithm* is the description of the logical steps by which a sequence of points from the search space is generated. This in general can depend on the previous points, the value of the objective function in those points and some other factors. In formulas:

$$\left\{(x_1^i, \ldots, x_n^i, f\left(x_1^i\right), \ldots, f\left(x_n^i\right\}, \Theta\right\} \xrightarrow[\text{algorithm}]{} \left\{x_1^{i+1}, \ldots x_m^{i+1}\right\} \tag{3}$$

where Θ is a random variable and index i is the iteration counter. An optimization algorithm is an algorithm that aims to generate a sequence of points that converges to a local or global maximum or minimum.

Another useful concept that will be used in this chapter is that of *response landscape* or *fitness landscape*:

Definition 4 The *response* (or *fitness*) *landscape* is the (hyper)surface obtained by applying the objective function to every point in the search space over which search is being executed.

This is a commonly used metaphor. It allows interpreting the search in terms of well known topographical objects such as peaks, valleys, mesas, etc. (Fig. 2).

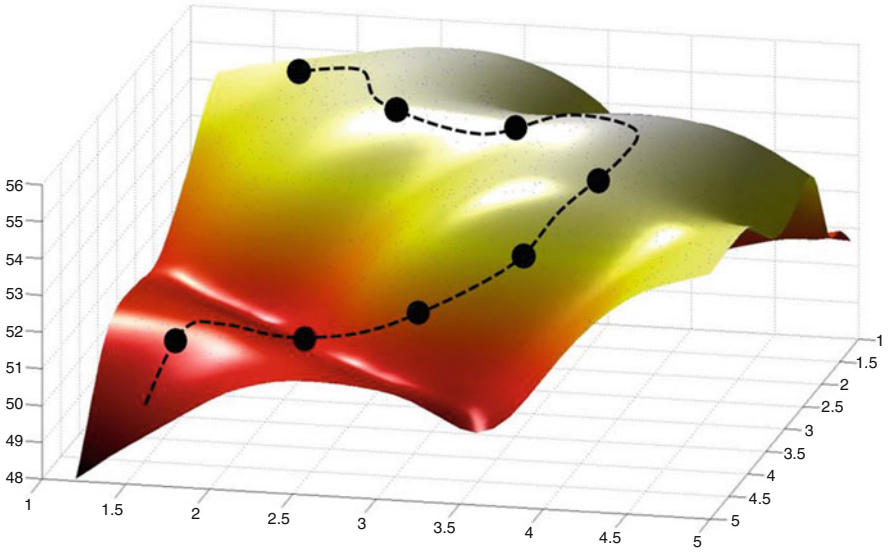


Fig. 2 Metaheuristics can be viewed as strategies for navigating a geometry using the information provided by the guiding fitness function

Differences among different optimization algorithms come from the specific metaheuristic adopted i.e. the mechanism used to generate and accept new samples and in so doing alternating an exploration phase (that scans the search space globally) with an exploitation—or local—phase.

For example just to mention one class, *nature-inspired* algorithms, like the well known *genetic algorithm* (henceforth GA), are based on the idea of mimicking some natural phenomena that leads to the maximization of some defined quantity [3]. Starting with a randomly generated population of candidate solutions (called chromosomes in usual terminology), a genetic algorithm carries out a process of fitness-based selection and recombination to produce a successor population, the next generation. During recombination, *parent* solutions are selected and their genetic material is recombined to produce child chromosomes. After this step, in practical implementation a mutation operator is applied. Mutation perturbs the recombined solutions slightly to explore their immediate neighborhood. These then pass into the successor population. As this process is iterated, a sequence of successive generations evolves and the average fitness of the chromosomes tends to increase until some stopping criterion is reached. In this way, a GA evolves to a best solution for a given problem. Through the process of evolution, one population is replaced by another and so on (for an overview on GAs see [4, 5]). Many other search metaheuristics are used nowadays: Simulated Annealing, Particle Swarm, Taboo Search, and many other families [6, 7].

It is important to understand that at its very basic, any metaheuristic is substantially "just" a balance, or combination, of global search (exploration) and local

search (exploitation). In an illuminating work, Yang (in [8]) synthesized the essence of optimization metaheuristics in the concept of *Generalized Evolutionary Walk Algorithm* (GEWA). GEWA combines all the basic features of any optimization method:

(a) global exploration through randomization:

$$\mathbf{x}_{new} = \mathbf{L} + (\mathbf{U} - \mathbf{L}) * \epsilon_u \tag{4}$$

where $\mathbf{L}$ and $\mathbf{U}$ are the lower bound and upper bound, respectively. ϵ_u is a uniformly distributed random variable in [0, 1].

(b) intensive local search by random walk:

$$\mathbf{x}_{t+1} = \mathbf{g}_* + \mathbf{w}, \;\; \mathbf{w} = \epsilon \mathbf{d} \tag{5}$$

where ε is drawn from a Gaussian distribution or normal distribution $\mathrm{N}(0, \sigma^2)$, and $\mathbf{d}$ is the step length vector which should be related to the actual scales of independent variables.

(c) the selection of the best with some elitism.

The GEWA goes at the essence of a metaheuristic. Tuning the parameters in the algorithm can generate different combination of exploration-exploitation trade-off. All metaheuristics, in a way or another, alternate a global with a local search.

3 The No Free Lunch Theorem: Mathematics and Meaning

There are many formulations of the result in the literature, with different emphasis. The original formulation from [1] asserts that:

> Any two search or optimization algorithms are equivalent when their performance is averaged across all possible problems.

Wolpert and Macready's result considered a finite search space $\mathcal{X}$ and the space of all the possible objective functions $f : \mathcal{X} \mapsto \mathcal{Y}$ defined on it called $\mathcal{F} = \mathcal{Y}^{\mathcal{X}}$. They defined with $P(y_k|f, k, Alg)$ the conditional probability of finding a value $y_k \in \mathcal{Y}$ given a function f, after k iteration with algorithm Alg. This can be seen as a performance measure of the algorithm, its ability to locate a given function value after a given amount of iterations. Under some quite general conditions the theorem states that, for any pair of algorithms Alg_1 and Alg_2:

$$\sum_f P(y_k|f, k, Alg_1) = \sum_f P(y_k|f, k, Alg_2) \tag{6}$$

where the sum is carried out over the set of all the possible functions $\mathcal{F}$. What is the meaning of (6)? According to the most common understanding the NFLT

implies that there is no optimization method superior to others for all possible optimization problems. For some functions Alg_1 will be able to locate the maximum faster than Alg_2; for some other functions it will be the opposite. Averaging over the whole space $\mathcal{F}$ the performance will be the same. Wolpert and Macready adopted a probabilistic framework. Their result holds if we assume a uniform distribution over $\mathcal{F}$, i.e. any functional form is uniformly admissible, and then they prove it by induction over the index k.

The NFLT has been also stated in equivalent forms. Most common formulations are

> No algorithm performs better than any other when their performance is averaged over all possible problems;

or

> On average, no algorithm is better than pure random search in locating the global optimum.

An optimization algorithm is *better* if, for a given problem, is able to locate the optimum with fewer iterations compared to other algorithms. This ability comes from the *matching* between the searching logic of the algorithm and the geometry of the objective function landscape: new samples are generated in some given areas and not in others according to the exploration-exploitation mechanism that is specific to each algorithm. If an algorithm tends to explore promising areas for some class of landscapes, the same algorithm will suffer on different geometries. It is in this matching between an algorithm and a problem landscape that lies the effectiveness of an algorithm. The NFLT is a mathematical way of looking at this fact.

To understand the practical implication of this theorem for black-box optimization problems, let us re-state it adopting a different perspective that takes into account the search mechanism implicit in any metaheuristics. Substantially the NFLT can be reformulated in the following way [9]:

> With no prior knowledge about the function $f : \mathcal{X} \mapsto \mathcal{Y}$, in a situation where any functional form is uniformly admissible, the information provided by the value of the function in some points of the domain will not say anything about the values of the function in other regions of the domain.

This interpretation of the NFLT is pictured in Fig. 3. So in this scenario the information collected with the data sample $\mathcal{D}$ is not helpful in guiding the search in which direction is better to explore next. In this sense, there is no optimization method superior to others for all possible optimization problems. Of course every function has its own structure, the problem is when prior knowledge about the functional form is not available because no rationale can guide an optimization strategy i.e. to decide which optimization method to use, which set of parameters, and so on.

In the NFLT scenario, where nothing is known in literature about the structure of the problem at hand, practitioners tend to decide the optimization method according to their background knowledge, practical availability of code, simulation software, and so on. In a different scenario where something is known about the function, like a lower bound on the function value or some information about the response

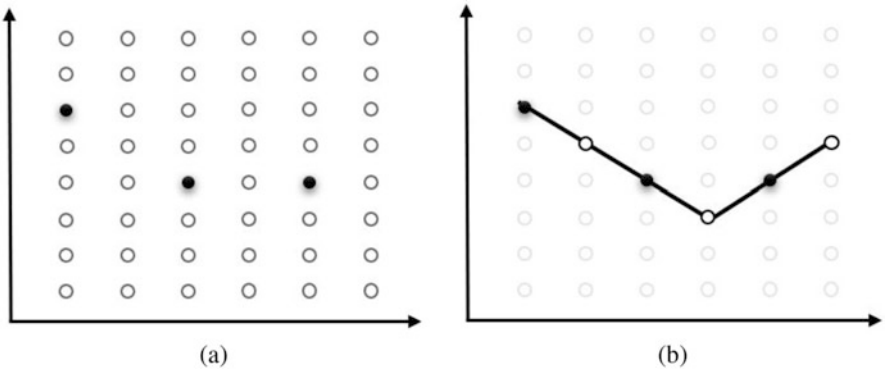


Fig. 3 The sample of points collected so far will not say anything about the values of the function in other regions (case **a**). For a given subclass of functions, for example convex (Case **b**), those algorithms that can take advantage of this structure will perform better than others

landscape, this information must be used to tailor the algorithm and the optimization strategy accordingly. Knowledge of the objective function structure is a key to adjust effective algorithms in terms of a better trade-off between exploration and exploitation.

Here we have a first practical implication of the NFLT. Of course every function has its own structure, the problem is when prior knowledge about the functional form is not available because in this case no rationale can guide an optimization strategy i.e. to decide which optimization method to use, which set of parameters, and so on.

The lesson that has to be learned from NFLT lies in the implications for a rational optimization strategy able to tackle black-box optimization problems. It is clear now that for the practitioner the correct question is not *which algorithm I have to use* is strictly related to *what is the geometry of the objective function*: the optimization problem is equivalent to a learning or an inferential problem as it will be clear in the discussion below. Prior information about the structure of the objective landscape makes (theoretically) possible to properly tune an algorithm in terms of a trade-off between local search and global search. *Learning* the objective function and optimizing it are activities that go hand in hand. Knowledge of the objective function structure is a key to adjust effective algorithms in terms of a better trade-off between exploration and exploitation.

In a scenario where something is known about the function, like the general functional structure (smoothness, periodicity, etc.), bounds on the function value, and in general some information about the response landscape, this knowledge must be used to tailor the algorithm and the optimization strategy accordingly. In the case where nothing or very little is known about the structure of the problem at hand (lack of studies in the specialized literature combined with lack of practical experience on that particular application), practitioners tend to decide the optimization method

according to their background expertise, availability of code, simulation software used, and so on. This latter critical case will be discussed in the final section.

4 An Interpretation in Terms of Bayesian Probability Theory

In this section we will consider the interpretation of the NFLT in terms of Bayesian probability theory. At its very core, the NFLT is about the possibility and limits of learning and as such it is not necessarily limited to optimization but concerns any broader issues related to machine learning.

In the black-box context only a finite sample data set concerning the functional response is available: $\mathcal{D} = \{(x_1, f(x_1)) \ldots, (x_n, f(x_n))\}$. In this case locating a function value (the optimum) becomes an inferential problem that can be better understood through the lens of Bayesian interpretation of probability. The rationale of Bayesian probability theory is briefly summarized here in the context of optimization. The problem is to construct a model of the function we need to optimize. In the Bayesian framework there are two ingredients: a data set and a prior. A prior distribution $P(f)$ over the space of functions is combined with the likelihood $P(\mathcal{D}|f)$ to generate a posterior:

$$P(f|\mathcal{D}) \propto P(\mathcal{D}|f)P(f) \tag{7}$$

that takes into account the information given by the data set $\mathcal{D}$. In terms of modeling a function, if we assume as a prior every possible function uniformly (this corresponds to the case where there is no knowledge whatsoever about the objective function geometry) we fall exactly in the NFLT case: no useful inferential information can be extracted from the data set. If the prior can be restricted to a proper class of function the inference about the model realizing the data set will be more accurate. As said about the implications of the NFLT, if it is possible to restrict the problem to a given subclass of function, a proper optimization algorithm choice can produce good results.

> For an optimization algorithm, given $\mathcal{D}$,[4] its ability to direct the search in promising areas where the optimum can be located is equivalent to say that a (posterior) refined model of the objective function has been inferred in Bayesian terms.

With the exception of *Pure random search* that does not assume any prior, all metaheuristics adopt—implicitly or explicitly—a model class of the objective function (the Bayesian prior). During the search they combine it with the sampled points at a given iteration to direct the search in the most promising directions. If the prior matches the fitness landscape they will work well, if not their performance will be poor. Averaging metaheuristics over all the possible geometries, the performance

[4] Of course the sample must be properly generated. For Design of Experiments techniques see [10].

will be the same: **do not forget, the NFLT is about the average behavior of metaheuristics**.

For example *genetic algorithms*, as well as other optimizers, assume (and they work well if there is) *strong causality*: small causes produce small effects. For function with low causality they tend to suffer [11]. In metaheuristics the prior about the function is implicitly defined by the search operators used (*crossover*, *mutation*, etc.) and the set of internal working parameters. Many automated self-adapting strategies able to change the internal parameters values according to partial results obtained during the search process have been studied in recent years [12]. What these (so-called *smart*) optimization algorithms do is to incorporate a more explicit Bayesian operating logic. They generate a posterior model using the sampled points collected at a given iteration and some assumptions about the function structure (prior). Iteration after iteration, they adjust the set of internal parameters to better tailor the function model and to foster the search in more promising regions (according to the *posterior* model). This is for example the case of in the *CMA-ES* class of algorithms [13].

5 A Reconsideration of Model-Assisted Optimization

The discussion above about the probabilistic interpretation of NFLT can be useful for understanding the class of so-called *surrogate-based* optimization techniques. In the context of black-box optimization there is a large literature about methods for objective function approximation (also called *meta-models* or *surrogates*) as ways to generate functional models that are computationally efficient and that approximate the true function. All of them *assume* a prior in terms of a restricted class of modeling function. Functional surrogate models can be algebraic representations of the true problem functions. The most popular ones are polynomials, in a method often known in the statistical literature as *response surface methodology*. Several related methods are now commonly used for function approximation: Neural Networks, Kriging model, radial-basis-function networks, and Support Vector Machines (for a general reference, see [10, 14]). After the model has been developed some classical derivative-based or derivative-free methods can be applied [15]. *Bayesian optimization* is briefly discussed here as a tool that is getting relevance in optimization practice. This technique conceptually combines surrogate estimation with optimum localization (for a good introduction see [16–18]). It applies the Bayesian rationale to infer about a starting function model. The second step is to infill new points of the search space in those regions where a maximum improvement is expected according to the current best value and the predictive variance (Fig. 4). Then the Bayesian procedure is updated. Usually in literature the prior is supposed to be a realization of a **Gaussian process** which means a distribution over the *space of function* modeled as a multivariate Gaussian distribution, so one can think the Gaussian process as defining a distribution over the space of functions:

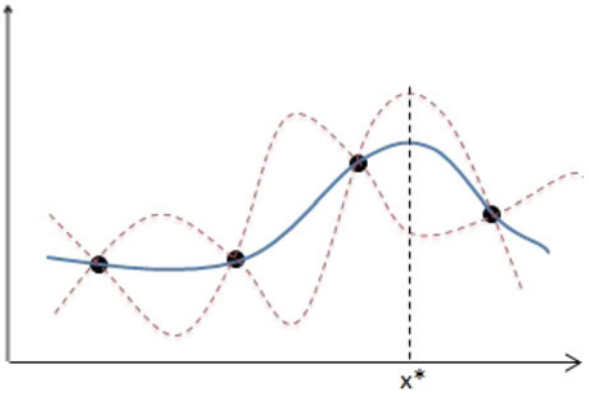


Fig. 4 Bayesian optimization. Data points are shown as black dots. The GP modeled distribution has a mean shown by the blue line that fits the data, and a standard deviation represented here by the red dotted lines. Here $x*$ represent the next point to explore since maximum improvement is expected

$$f(x) \sim \mathrm{GP}(m(x), k(x_i, x_j)) \tag{8}$$

meaning that the function f is distributed as a GP with mean function $m(x)$ and covariance function $k(x_i, x_j)$. A common choice for the covariance function is defined by

$$k(x_i, x_j) = \exp\big(-\tfrac{1}{2}(x_i - x_j)^T \,\mathrm{diag}(\phi)^{-2}(x_i - x_j)\big) \tag{9}$$

where ϕ represents a family of hyper-parameters that have to be properly estimated.

It has been stressed in the literature that Bayesian optimization techniques are some of the most efficient approaches *in terms of the number of function evaluations required* [16]. These statements need a careful re-examination in the light of the previous discussion about the NFLT.

Simply put, as for any other optimization algorithm, Bayesian optimization cannot escape the limitations given by the NFLT. The key point in the Bayesian rationale is given by the choice of the prior, i.e. the non-uniform distribution over $\mathcal{F}$ in the NFLT terminology. The choice of the prior as a *Gaussian process* implies that the function is supposed to be in the class of smooth well-behaving functions and this can be an arbitrary hypothesis in some industrial applications. For objective functions with discontinuities or jumps this prior will not be able to provide very good results. Further, the construction of the surrogate model is essentially an exercise of design space exploration exactly in the same way any optimization algorithm result in essence is. As the dimension of the search space increases (the exact number depending on hardware's computational power) the ability to sample promising regions becomes harder and harder. Bayesian optimization cannot avoid

the *curse of dimensionality*: the number of candidate solutions grows exponentially with increasing dimensionality [19].

Summarizing, the ability of every optimization method is connected to the following points: a) how much the adopted internal prior model fits the geometry of the problem at hand and b) on the other side, the possibility to explore the sample space with experimental design techniques: $\mathcal{D} = \{(x_1, f(x_1)) \ldots, (x_n, f(x_n))\}$ of $\mathcal{H} \times \mathrm{Im}(f)$. This can be a problem for computational expensive functions and/or for high-dimensional search spaces. As we have said, the curse of dimensionality affects the ability to generate a valid posterior since with increasing dimensions the needed number of sampled points able to say something usefully in the inferential process increases exponentially.

6 The NFLT and the Optimization Practice

It has been sharply pointed out how in recent years, the field of optimization has witnessed a true tsunami of metaheuristics methods, most of them based on a metaphor of some natural or man-made process [20]. How can we make sense of this proliferation of optimization metaheuristics in light of the discussion about the NFLT made so far? Even if we can agree that in most of this proliferation the "publish or perish" working logic of Academia plays a big role, we believe that in part this depends on a not complete understanding of the practical meaning of the NFLT. Here we should distinguish two cases:

(a) One case is to look at the problem from the perspective of the researcher working on the development of new optimization algorithms.
(b) The second case is to consider it from the point of view of those who need to apply those methods in industrial, professional applications.

There is no space here to fully consider case (a), but few basic considerations are in order. The NFLT teaches that there is no magic one-size-fits-all optimization method. Research efforts should be dedicated to elucidate for which *class of functions* a give method works better than others. There is little consideration of this problem in most of publication in the field. I think some research efforts should be directed in this direction. Here I give an interesting example. Molecular biology is nowadays a source of very interesting mathematical challenges. In the so called multiple-minima problem, the aim is to locate the global minimum of the potential energy of structure a polypeptide chain and so to be able to predict the native three dimensional structure. The solution of this problem can have a relevant impact in biomedical research in application like drug design and the study of various disease states caused by incorrect folding. The folding of a protein depends on the one dimensional linear sequence of amino-acids and it is described by a set of angular degrees of freedom between atoms that represent the input parameters whereas the molecular potential energy represents the output of the function to be minimized. The number of input variables involved can be extremely high. The class of funnel

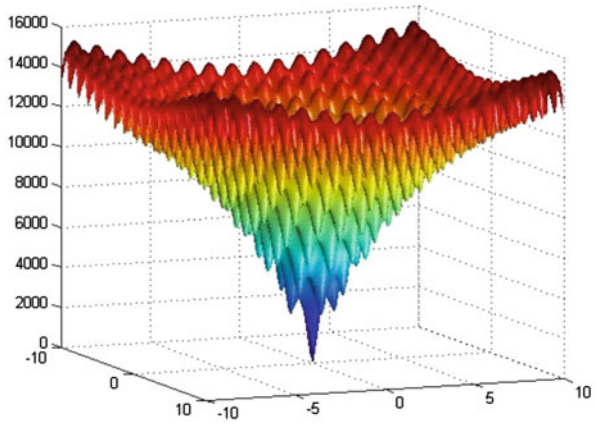


Fig. 5 Polypeptide molecular potential energy has a funnel-like geometry. Here is a simplified 3D example

functions (Fig. 5 shows an example), more or less *rugged*, are supposed to play an important role in describing protein folding energy landscapes [21]. An important research project can be the development of a metaheuristic that best fits this kind of landscapes. To match the best metaheuristic with a specific geometry is still an open and important research area both in theoretical and practical terms.

Going to point (b), from the perspective of the practitioner, the understanding of the NFLT given above must be critically evaluated. The implication of the NFLT is of practical concern whereas there is no prior knowledge about the objective function landscape with no reference in the literature or previous experiences. This can be a reality in cutting edge technology where a novel device or an innovative component has not been assessed before or there is no technical documentation about optimization approaches. What the NFLT implies in this scenario is *not* that any metaheuristic is equivalent *for this specific problem* but that not necessarily the *last published algorithm*—or the one hyped up at that moment—will be the best choice. This said, the lack of knowledge about the fitness landscape is not necessarily a problem. Given the computational power of modern computers, different metaheuristics can provide good results in many situations. This lack of knowledge about the fitness landscape can be a real problem only if it joined with other factors. Most common of them are:

- **High number of independent input variables**. We have already introduced the phenomenon named *the curse of dimensionality* before. The large number of candidate solutions to an optimization problem makes it computationally very hard to be attacked by evolutionary algorithms because the number of candidate solutions grows exponentially with increasing dimensionality. This is well known by practitioners that have to handle problems with hundreds of variables [19].

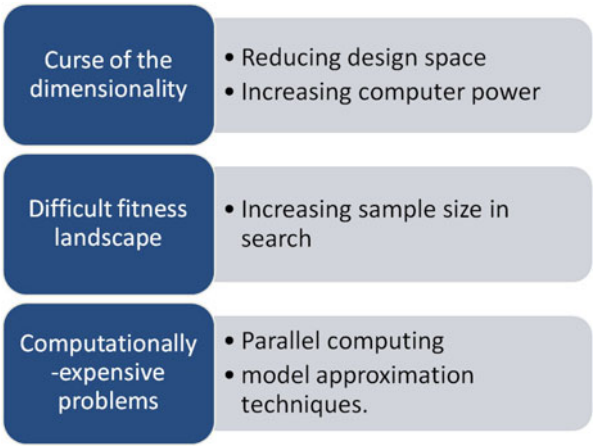


Fig. 6 Some basic strategies for hard optimization [25]

- **Difficult fitness landscape**. In landscape surface with weak (low) causality, small changes in the candidate solutions often lead to large changes in the objective values, i.e. ruggedness [22]. A fitness landscape is rugged if there are many local optima concentrated in any constrained region of the space. At the opposite side, even *neutral* landscape where the optimum is a narrow peak in between regions with similar fitness values can be very hard to optimize since no guiding information can be extracted from neutral areas.
- **Computational expensive fitness evaluation**. Simulation can be very time consuming in the order of many minutes or even hours for each single fitness call.

In hard black-box optimization many issues are at stake. Some good reviews are [11, 23, 24]. In Fig. 6 are presented some practical hints derived from literature. It is worth noting that increasing sample size as countermeasure for difficult landscapes competes with the need of reducing the number of fitness calls in computational expensive problems. Further, a number of engineering design tasks as well in other context are modeled as multi-objective problems and this makes the optimization process even harder.

Combined with this difficulty together with a lack of priors about the objective function, makes the optimization task a real challenge. There are no magic recipes here, but some practical suggestion can be given.

Despite the number of optimization techniques available nowadays, and despite the fact that are considered somehow "out of fashion" nowadays, I would argue here in favor of genetic algorithms as a first line optimization strategy. GAs still can play a central role in coping with a wide variety of optimization problems without any a priori assumptions about their continuity and differentiability. Even if it they still lack a strong theoretical foundation able to understand or predict the dynamics other than a superficial level, the biological analogy is conceptually easy

to understand and it is relatively simple to develop program code for any kind of application. They are the oldest and best known metaheuristic and examples of application in different areas of sciences and engineering are countless and well reported in literature (for example see [26] for the field of material engineering). Similar good reviews can be found on internet in virtually every field of Engineering and Science. There are a number of resources freely available on Internet and optimization toolboxes implementing genetic strategies are available in most of common scientific software packages like MATLAB®. A multi-objective GA code like NSGA-II can also be used as a pre-optimization tool to explore the surface response of a given mono-objective fitness function. GAs have demonstrated their potential in numberless projects since they are easy to use. They are problem independent, i.e. they neither need a special representation for candidate solutions nor put any other restricting requirements on the problems to be tackled. Thus they can be viewed as universally applicable. However, practice shows that any successful application depends on careful tuning of operators, parameters, and problem-dependent features. As repeatedly said throughout this chapter, every metaheuristic can be imagined as a trade-off between global search and local search. In GAs this trade-off is managed acting on population size, mutation, and crossover rate.

The information collected about the optimization problem and the response obtained from algorithms will allow updating both the GA parameters configuration of the single algorithm, the choice of the algorithm class and the overall strategy itself. If the GA option does not provide a satisfactory response, other metaheuristic must be tested such as particle swarm optimization, simulated annealing, cross-entropy, MCMC, or others and so continuing with methods of different working logic. A very rugged fitness landscape will lead to a degradation of the performance of the optimizer. Using large population sizes and applying methods that increase the exploration capabilities of the algorithm can decrease the influence of ruggedness, at least up to a certain degree [22]. Furthermore, experimental results show that GAs (initialized with random values) are better suited for the exploration of the problem space than for its exploitation: it is known that GA evolution makes extremely rapid progress at first, as the diverse elements in the initial population are combined and tested. Over time, the population begins to converge, with the separate individuals resembling each other more and more. Effectively this results in the GA narrowing its search in the objective space and reducing the size of any changes made by evolution until eventually the population converges to a single solution. In contrast, hill climbing or gradient ascent algorithms, which initialize at some point and then move greedily toward the nearest local optima, are maximally exploitative, since they make no attempt to explore alternative solutions away from its current trajectory. This discussion justifies why hybrid strategies are highly recommended [27]. The application of a GA followed by a local search method (being it gradient based or stochastic according to the fitness properties) can be a good strategy if good accuracy in the optimal solution is needed. In hybrid algorithms, a GA with good exploration capacities are often used to locate some promising zones within the wide search space, while the local optimization methods exploit the located

promising zones to achieve the best solution quickly and accurately. Of course GAs must be combined with fitness approximation techniques in computational expensive problems [28].

References

1. Wolpert, D.H., Macready, W.G.: No free lunch theorems for optimization. IEEE Trans. Evol. Comput. **1**(1), 67–82 (1997)
2. Schöffler, S.: Global Optimization A Stochastic Approach. Springer, New York (2012)
3. Holland, J.H.: Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence. The MIT Press, Cambridge (1992)
4. Mitchell, M.: An Introduction to Genetic Algorithms MIT Press, Cambridge (1998)
5. Haupt, S.E., Haupt, R.L.: Practical Genetic Algorithms, vol. 100. Wiley, Hoboken (2004)
6. Simon, D.: Evolutionary Optimization Algorithms. Wiley, Hoboken (2013)
7. Luke, S.: Essentials of metaheuristics. https://cs.gmu.edu/~sean/book/metaheuristics/ (2009)
8. Yang, X.S.: Review of metaheuristics and generalized evolutionary walk algorithm. J. Bio-Inspired Comput. **3**(2), 77–84 (2011)
9. Serafino, L.: Optimizing without derivatives: what does the no free lunch theorem actually says? Not. AMS **61**, 750–755 (2014)
10. Sudjianto, A., Fang, K.T., Li, R.: Design and Modeling for Computer Experiments. Computer Science & Data Analysis. Chapman & Hall/CRC, London (2005)
11. Weise, T.: Global Optimization Algorithms. Theory and Application (2011). http://www.it-weise.de/projects/bookNew.pdf
12. Eiben, A.E., Hinterding, R., Michalewicz, Z.: Parameter control in evolutionary algorithms. IEEE Trans. Evol. Comput. **3**, 124–141 (2000)
13. Hansen, N.: The CMA evolution strategy: a comparing review. In: Towards a New Evolutionary Computation. Advances on Estimation of Distribution Algorithms, pp. 1769–1776. Springer, Berlin (2006)
14. Jin, Y.: A comprehensive survey of fitness approximation in evolutionary computation. Soft Comput. **9**(1), 3–12 (2005)
15. Conn, A.R., Scheinberg, K., Vicente, L.N.: Introduction to Derivative-Free Optimization. Society for Industrial and Applied Mathematics, Philadelphia (2009)
16. Brochu, E., Cora, V.M., de Freitas, N.: A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. CoRR abs/1012.2599 (2010)
17. Alpcan, T.: A framework for optimization under limited information. J. Global Optim. **55**, 681–706 (2013)
18. Hoffman, M., Brochu, E., de Freitas, N.: Portfolio allocation for Bayesian optimization. UAI, arXiv:1009.5419v2 (2011)
19. Hastie, T., Tibshirani, R., Friedman, J.: The Elements of Statistical Learning: Data Mining, Inference, and Prediction, corrected edn. Springer, Berlin (2003)
20. Sørensen, K.: Metaheuristics–the metaphor exposed. Intl. Trans. Oper. Res. **22**, 3–18 (2015)
21. Stefani, M.: Protein folding and misfolding on surfaces. Int. J. Mol. Sci. **9**, 2515–2542 (2008)
22. Weise, T., Zapf, M., Chiong, R., Nebro Urbaneja, A.J.: Why is optimization difficult? In: Chiong, R. (ed.), Nature-Inspired Algorithms for Optimisation, pp. 1–50. Springer, Berlin (2009)
23. Shan, S., Gary Wang, G.: Survey of modeling and optimization strategies to solve high-dimensional design problems with computationally-expensive black-box functions. Struct. Multidiscip. Optim. **41**(2), 219–241 (2010)

24. Tennem, Y.: A computational intelligence algorithm for expensive engineering optimization problems. Eng. Appl. Artif. Intell. **25**(5), 1009–1021 (2012)
25. Serafino, L.: No free lunch theorem and Bayesian probability theory: two sides of the same coin. Some implications for black-box optimization and metaheuristics (2013). arXiv:cs.LG/1311.6041
26. Alander, J.T.: An indexed bibliography of genetic algorithms in materials science and engineering (2008). http://lipas.uwasa.fi/TAU/report94-1/gaMSEbib.pdf
27. El-Mihoub, T.A., Nolle, L., Battersby, A., Hopgood, A.A.: Hybrid genetic algorithms: a review. Eng. Lett. 13(2), 103591 (2006)
28. Garcia-Martinez, C., Rodriguez, F.J., Lozano, M.: Arbitrary function optimisation with metaheuristics. Soft Comput. **16**(12), 2115–2133 (2012)

What Is Important About the No Free Lunch Theorems?



David H. Wolpert

1 Introduction

The first of what are now called the No Free Lunch (NFL) theorems were published in [1]. Soon after publication, they were popularized in [2], building on a preprint version of [1]. Those first theorems focused on (supervised) machine learning. Loosely speaking, they can be viewed as a formalization and elaboration of informal concerns about the legitimacy of inductive inference that date back to David Hume (if not earlier). Shortly after these original theorems were published, additional NFL theorems that apply to search were introduced in [3]. Broadly speaking, the NFL theorems say that under a uniform distribution over problems (be they supervised learning problems or search problems), all algorithms perform equally.

The NFL theorems have stimulated a huge amount of research, with over 10,000 citations of [3] alone by summer 2020, according to Google Scholar. Much of the early work focused on finding other prior distributions over problems besides a uniform distribution that still results in all algorithms having the same expected performance [4, 5]. Other, more recent work has extended NFL to other domains, beyond (classical physics based) supervised learning [6], and beyond learning and search entirely [7].

However, as stated in [3], perhaps the primary significance of the NFL theorems for search is what they tell us about "the underlying mathematical 'skeleton' of optimization theory *before the 'flesh' of the probability distributions of a particular context and set of optimization problems are imposed*." So in particular, while the NFL theorems have strong implications *if* one believes in a uniform distribution over optimization problems, in no sense should they be interpreted as advocating

D. H. Wolpert (✉)
Santa Fe Institute, Santa Fe, NM, USA

© Springer Nature Switzerland AG 2021

P. M. Pardalos et al. (eds.), *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, Springer Optimization and Its Applications 170,
https://doi.org/10.1007/978-3-030-66515-9_13

such a distribution. Rather such a distribution is used as a tool, to prove results concerning *nonuniform* distributions, and in addition to comparing different search algorithms, without any direct assumption about the distribution over problems at all.

In this chapter I describe these aspects of the NFL theorems. After presenting the inner product formula that determines the performance of any search algorithm, I then expand on the inner product formula to present the NFL theorems for search and supervised learning. As an example of the true significance of the NFL theorems, I consider "anti-cross-validation." This is the meta-algorithm that chooses among a candidate set of algorithms based on which has the worst out-of-sample performance on a given data set. (In contrast, standard cross-validation chooses the algorithm with the best such performance.) As I discuss, the NFL theorems mean that anti-cross-validation outperforms cross-validation as often as vice versa, over the set of all objective functions. So without making some assumption about the relationship between the candidate algorithms and the distribution over optimization problems, one cannot even justify using cross-validation.

Following up on this, I briefly discuss how the NFL theorems are consistent with the (very) many proofs in the literature that provide lower bounds on the performance of particular algorithms without making any assumptions about the distribution over problems that are fed to those algorithms. I also point out the implications of the NFL theorems for the entire scientific enterprise, i.e., for philosophy of science [8].

I then discuss how the fact that there are NFL theorems for both search and supervised learning is symptomatic of the deep formal relationship between those two fields. Once that relationship is disentangled, it suggests many ways that we can exploit practical techniques that were first developed in supervised learning to help us do search. I summarize some experiments that confirm the power of search algorithms developed in this way.

After this, I briefly discuss the various *free* lunch theorems that have been derived, which establish a priori benefits for using one algorithm rather than another. I end by discussing possible directions for future research.

2 The Inner Product at the Heart of All Search

Let X be a countable **search space**, and specify an **objective function** $f : X \to Y$, where $Y \subset \mathbb{R}$ is a countable set. Sometimes an objective function is instead called a "search problem," "fitness function," "cost function," etc. I use the term **data set** to mean any set of m separate pairs $(x \in X, f(x))$, written as $d^m = \{d_X^m, d_Y^m\}$. A **search algorithm** is a function $\mathscr{A}$ that maps any d^m for any $m \in \{0, 1, \ldots\}$ to an $x\texttt{not} \in d_X^m$. Examples range from simulated annealing to genetic algorithms to hill descending. By iteratively running a search algorithm to produce a new sample point x and then evaluating $f(x)$, we can build successively larger data sets: $d^{m+1} = d^m \cup (A(d_X^m), f[A(d_X^m)])$ for all $m \geq 0$.

Suppose we are given an arbitrary **performance measure** $\Phi : d_Y^m \rightarrow \mathbb{R}$. Then we can evaluate how the performance of a given search algorithm on a given objective function changes as it is run on that function. Note that to "normalize" different search algorithms, we only consider their behavior in terms of generating new points at which to sample the objective function that are not yet in the data set. (Equivalently, if an algorithm chooses a new point to sample that is already in its data set, we allow it to "try again.") This is crucial; we are only interested in *off-data-set* behavior.

For simplicity, from now on I restrict attention to deterministic search algorithms and deterministic objective functions neither of which varies from one iteration of the search algorithm to the next. However, everything presented in this chapter can be extended in a straightforward way to the case of a stochastic search algorithm, stochastic objective function, time-varying objective function, etc.

In practice, often one does not know f explicitly. This is the case whenever f is a "blackbox," or an "oracle," that one can sample at a particular x but does not know in closed form. Moreover, often even if a practitioner does explicitly know f, they *act* as though they do not know it, for example, when they choose what search algorithm to use on f. For example, often someone trying to solve a particular instance of the Traveling Salesman Problem (TSP) will use the same search algorithm that they would use on any other instance of TSP. In such a case, they are behaving exactly as they would if they only knew that the objective function is a TSP, without knowing specifically which one it is.

These kinds of uncertainty about the precise f being searched can be expressed as a distribution $P(f)$. Say we are given such a $P(f)$, along with a search algorithm, and a real-valued measure of the performance of that algorithm when it is run on any objective function f. Then we can solve for the probability that the algorithm results in a performance value ϕ. The result is an inner product of two real-valued vectors each indexed by f. (See Appendix.) The first of those vectors gives all the details of how the search algorithm operates, but nothing concerning the world in which one deploys that search algorithm. The second vector is $P(f)$. All the details of the world in which one deploys that search algorithm are specified in this vector, but nothing concerning the search algorithm itself.

This result tells us that at root, how well any search algorithm performs is determined by how well it is "aligned" with the distribution $P(f)$ that governs the problems on which that algorithm is run. For example, it means that the (tens of?) thousands of person-years of research into the TSP have (presumably) resulted in algorithms aligned with the implicit $P(f)$ describing traveling salesman problems of interest to TSP researchers.

3 The No Free Lunch Theorems for Search

The inner product result governs how well any particular search algorithm does in practice. Therefore, either explicitly or implicitly, it serves as the basis for any practitioner who chooses a search algorithm to use in a given scenario. More

precisely, the designer of any search algorithm first specifies a $P(f)$ (usually implicitly, e.g., by restricting attention to a class of optimization problems). Then they specify a performance measure Φ (sometimes explicitly). Properly speaking, they should then solve for the search algorithm that the inner product result tells us will have the best distribution of values of that performance measure, for that $P(f)$. In practice though, instead informal arguments are often used to motivate the search algorithm.

In addition to governing both how a practitioner should design their search algorithm and how well the actual algorithm they use performs, the inner product result can be used to make more general statements about search, results that hold for all $P(f)$'s. It does this by allowing us to compare the performance of a given search algorithm on different subsets of the set of all objective functions. The result is the NFL theorem for search. It tells us that if any search algorithm performs particularly well on one set of objective functions, it must perform correspondingly *poorly* on all other objective functions.

This implication is the primary significance of the NFL theorem for search. To illustrate it, choose the first set to be the set of objective functions on which your favorite search algorithm performs better than the *purely random search algorithm*, which chooses the next sample point randomly. Then the NFL for search theorem says that compared to random search, your favorite search algorithm "loses on as many" objective functions as it wins (if one weights wins/losses by the amount of the win/loss). This is true no matter what performance measure you use.

As another example, say that your performance measure prefers low values of the objective function to high values, i.e., that your goal is to find low values of the objective rather than high ones. Then we can use the NFL theorem for search to compare a hill-descending algorithm to a hill-*ascending* algorithm, i.e., to an algorithm that "tries" to do as poorly as possible according to the objective function. The conclusion is that the hill-descending algorithm "loses to the hill-ascending algorithm on as many objective functions as it wins." The lesson is that without arguing for a particular $P(f)$ that is biased toward the objective functions on which one's favorite search algorithm performs well, one has no formal justification that that algorithm has good performance.

A secondary implication of the NFL theorem for search is that *if* it so happens that you assume/believe that $P(f)$ is uniform, then the average over f's used in the NFL for search theorem is the same as $P(f)$. In this case, you must conclude that all search algorithms perform equally well for your assumed $P(f)$. This conclusion is only as legitimate as is the assumption for $P(f)$ it is based on. Once other $P(f)$'s are allowed, the conclusion need not hold.

An important point in this regard is that simply allowing $P(f)$ to be nonuniform, *by itself*, does not invalidate the NFL theorem for search. Arguments that $P(f)$ is nonuniform in the real world do not, by themselves, establish anything whatsoever about what search algorithm to use in the real world.

In fact, allowing $P(f)$'s to vary provides us with a new NFL theorem. In this new theorem, rather than compare the performance of two search algorithms over all f's, we compare them over all $P(f)$'s. The result is what one might expect: If

any given search algorithm performs better than another over a given set of $P(f)$'s, then it must perform corresponding worse on all other $P(f)$'s. (See Appendix for proof.)

4 The Supervised Learning No Free Lunch Theorems

The discussion above tells us that if we only knew and properly exploited $P(f)$, we would be able to design an associated search algorithm that performs better than random. This suggests that we try to use a search process itself to learn something about the real world's $P(f)$, or at least about how well one or more search algorithms perform on that $P(f)$. For example, we could do this by recording the results of running a particular search algorithm on a set of (randomly chosen) real-world search problems, and using those results as a "training set" for a supervised machine learning algorithm that models how those algorithms compare to one another on such search problems. The hope would be that by doing this, we can give ourselves formal assurances that one search algorithm should be used rather than another, for the $P(f)$ that governs the real world.

The precise details of how well such an approach would perform depend on the precise way that it is formalized. However two broadly applicable restrictions on its performance are given by an inner product formula for supervised learning and an associated NFL theorem for supervised learning.

Just like search, supervised learning involves an input space X, an output space Y, a function f relating the two, and a data set of (x, y) pairs. The goal in supervised learning though is not to iteratively augment the data to find what x minimizes the "target function" $f(x)$. Rather it is to take a fixed data set and estimate the entire function f. Such a function mapping a data set to an estimate of f (or more generally an estimate of a distribution over f's) is called a **learning algorithm**. We then refer to the accuracy of the estimate for x's that do not occur in the data set as **off-training set error**. More precisely, in supervised learning we are concerned with the expected value of a "loss function" over points outside of the training set, which plays the same role as the performance measure Φ does in search.

The supervised learning inner product formula tells us that the performance of any supervised learning algorithm is governed by an inner product between two vectors, both indexed by the set of all target functions. In particular, as long as the loss function is symmetric, it tells us that how "aligned" the supervised learning algorithm is with the real world (i.e., with the posterior distribution of target functions conditioned on a training set) determines how well that algorithm will generalize from any training set to a separate test set. (See appendix.) This supervised learning inner product formula results in a set of NFL theorems for supervised learning, applicable when some additional common conditions concerning the loss function hold. In some ways these theorems are even more striking than the NFL for search theorems.

As an example, let Θ be a set of the favorite supervised learning algorithms of some scientist $\mathscr{A}$. So when given a training set d, scientist $\mathscr{A}$ estimates what f produced that training set the following way. First they run cross-validation on d to compare the algorithms in Θ. They then choose the algorithm $\theta \in \Theta$ with the lowest such cross-validation error. As a final step, they run that algorithm on all of d. In this way $\mathscr{A}$ generates their final hypothesis h to generalize from d.

Next suppose that scientist $\mathscr{B}$ has the same set of favorite learning algorithms. So they decide how to generalize from a given data set the same way as $\mathscr{A}$ does—but with a twist. For some reason, $\mathscr{B}$ uses *anti*-cross-validation rather than cross-validation. So the the algorithm they choose to train on all of d is the element of Θ with the *greatest* cross-validation error on d, not the one with the smallest such error.

Note that since Θ is fixed, the procedure run by scientist $\mathscr{A}$ is simply a rule that maps any arbitrary training set d to an estimate of the target function for x outside of that training set. In other words, $\mathscr{A}$ themselves constitute a supervised learning algorithm. Similarly, since Θ is fixed, $\mathscr{B}$ is a (different) supervised learning algorithm. So by the NFL theorems for supervised learning, we have no a priori basis for preferring scientist $\mathscr{A}$'s hypothesis to scientist $\mathscr{B}$'s. Although it is difficult to actually produce such f's in which $\mathscr{B}$ beats $\mathscr{A}$, by the NFL for supervised learning theorem we know that there must be "as many" of them (weighted by performance) as there are f's for which $\mathscr{A}$ beats $\mathscr{B}$. In other words, anti-cross-validation beats cross-validation as often as the reverse.

Despite this lack of formal guarantees behind cross-validation in supervised learning, it is hard to imagine any scientist who would not prefer to use it to using anti-cross-validation. Indeed, one can view cross-validation (or more generally "out of sample" techniques) as a formalization of the scientific method: choose among a set of scientific theories that were formulated in response to some initial experimental data according to which of those theories better fits new experimental data that is generated after the theory was formulated, and then use the theory chosen this way to make predictions for subsequent experiments. By the inner product formula for supervised learning, this bias of the scientific community in favor of using out-of-sample techniques in general, and cross-validation in particular, must correspond somehow to a bias in favor of a particular $P(f)$. This implicit prior $P(f)$ is quite difficult to express mathematically. Yet almost every conventional supervised learning prior (e.g., in favor of smooth targets) or non-Bayesian bias favoring some learning algorithms over others (e.g., a bias in favor of having few degrees of freedom in a hypothesis class, in favor of generating a hypothesis with low algorithmic complexity, etc.) is often debated by members of the scientific community. In contrast, nobody debates the "prior" implicit in out-of-sample techniques. Indeed, it is exactly this prior that justifies the ubiquitous use of contests involving hidden test data sets to judge which of a set of learning algorithms are best.

5 Implications of NFL for Other Formal Results Concerning Inference, and for Philosophy of Science

It is worth taking a moment to describe how the NFL theorems can be reconciled with the many proofs in the literature of lower bounds on generalization error that would appear to provide *a priori* reason to prefer one algorithm over another. Briefly, there are two problematic aspects to those proofs. First, the NFL theorems all concern the conditional distribution

$$P(\Phi \mid d, \mathscr{A}), \tag{1}$$

where Φ is the random variable giving the expected loss of the prediction made by algorithm $\mathscr{A}$ for test points outside of the training set data d. In particular, they tell us that

$$\mathbb{E}(\Phi \mid d, \mathscr{A}) = \mathbb{E}(\Phi \mid d, \mathscr{B}) \tag{2}$$

for any two algorithms $\mathscr{A}$ and $\mathscr{B}$.

This equation concerns the posterior expected (off-training set) loss, conditioned on d, which according to Bayesian decision theory, should guide our decision of which algorithm to use, $\mathscr{A}$ or $\mathscr{B}$. One can average over d's (which are produced by sampling the implicit prior $P(f)$ and likelihood function $P(d|f)$), to see that

$$\mathbb{E}(\Phi \mid m, \mathscr{A}) = \mathbb{E}(\Phi \mid m, \mathscr{B}) \tag{3}$$

i.e., the expected off-training set loss conditioned only on m, the size of the data set d, is the same for both algorithms.

Next, let Φ' be the expected loss over the entire space X, not just the portion of X outside of d_X. There are many formal results in the literature that concern Φ', not Φ. In addition, many of these results do not explicitly specify what the conditioning event is for the distributions they calculate, simply writing them without any conditioning event at all. However when you dig into the proofs, you often find that the results concern conditional distributions like

$$P(\Phi' \mid m, \mathscr{A}, f). \tag{4}$$

Note that in a formal sense, this conditional distribution is "backward"—it conditions on f, which is what is unknown, and averages over d's, even though the actual d *is* known. (This criticism of the choice of conditioning event is, of course, a central issue in the age-old controversy between Bayesian statistics and non-Bayesian statistics.)

Often these results are independent of f, which sometimes leads researchers to interpret them as meaning that some algorithm $\mathscr{A}$ should be used rather than some other algorithm $\mathscr{B}$, "no matter what the distribution over f's is." In particular,

the argument is often made that so long as m is far smaller than the size of the space X, Φ' will approximate Φ arbitrarily well with arbitrarily high probability, and therefore a result like $\mathbb{E}(\Phi' \mid m, \mathscr{A}, f) < \mathbb{E}(\Phi' \mid m, \mathscr{B}, f)\ \forall f$ means that algorithm $\mathscr{A}$ is better than $\mathscr{B}$ on off-training set error, no matter what $P(f)$ is. That reasoning is simply wrong though; if one tries to use the standard rules of probability theory to convert results like $\mathbb{E}(\Phi' \mid m, \mathscr{A}, f) < \mathbb{E}(\Phi' \mid m, \mathscr{B}, f)$ to results like $\mathbb{E}(\Phi \mid m, \mathscr{A}) < \mathbb{E}(\Phi \mid m, \mathscr{B})$, one fails (unless one makes an assumption for $P(f)$). This is true no matter how big X is compared to m. In fact, it is not trivial to make the transition from results concerning $\mathbb{E}(\Phi' \mid m, \mathscr{A}, f)$ to results concerning $\mathbb{E}(\Phi' \mid d, \mathscr{A})$ [9, 10].

There has also been work that has claimed to derive a "Bayesian Occam's razor" using Bayes factors, where the analysis is over (ultimately arbitrarily defined) *models* of problems, rather than over individual problems directly [11–14]. However, NFL tells us that such approaches must, ultimately, simply be hiding their assumption concerning the prior over problems. Indeed, as a *reductio ad absurdum*, one can use the kind of reasoning promoted in these papers to imply that *any* algorithm is superior to any other, by appropriately redefining the models [15]. There has also been work that claims to use algorithmic information theory [16] to refute NFL [17]. However, ultimately this work simply makes an assumption for a particular prior and then shows that there are a priori distinctions between algorithms for that prior (in this case, the prior of algorithmic information theory, which explicitly prefers hypotheses that can be encoded in shorter programs). Of course, this is completely consistent with NFL. More broadly, other work has shown that all one needs to assume is that as one's algorithm is fed more and more data it performs better (on average), in order to justify a particular form of Occam's razor [18]. This too is consistent with NFL.

The implications of NFL for the entire scientific enterprise are also wide ranging. In particular, we can let X to be the specification of how to configure an experimental apparatus, and Y the outcome of the associated experiment. So f is the relevant physical law determining the results of any such experiment, i.e., they are a specification of a universe. In addition, d is a set of such experiments, and the function h produced by the "learning algorithm" is a theory that tries to explain that experimental data ($P(h \mid d)$ being the distribution that embodies the scientist who generates that theory). Under this interpretation, off-training set error quantifies how well any theory produced by a particular scientist predicts the results of experiments not yet conducted. So roughly speaking, according to the NFL theorems for search, if scientist $\mathscr{A}$ does a better job than scientist $\mathscr{B}$ of producing accurate theories from data for one set of universes, scientist $\mathscr{B}$ will do a better job on the remaining set of universes. This is true even if both universes produced the exact same set of scientific data that the scientists use to construct their theories—in which case, it is theoretically impossible for the scientists to use any of the experimental data they have ever seen *in any way whatsoever* to determine which set of universes they are in.

As another implication of NFL for supervised learning, take $x \in X$ to be the specification of an objective function, and say we have two professors, Smith and

Jones, each of whom when given any such x will produce a search algorithm to run on x. Let $y \in Y$ be the bit that equals 1 iff the performance of the search algorithm produced by Prof. Smith is better than the performance of the search algorithm produced by Prof. Jones.[1] So any training set d is a set of objective functions, together with the bit of which of (the search algorithms produced by) the two professors on those objective functions performed better.

Next, let the learning algorithm $\mathscr{C}$ be the simple rule that we predict y for all $x \texttt{not} \in d_X^m$ to be 1 iff the majority of the values in d_Y^m is 1, and the learning algorithm $\mathscr{D}$ to be the rule that we predict y to be -1 iff the majority of the values in d_Y^m is 1. So $\mathscr{C}$ is saying that if Professor Smith's choice of search algorithm outperformed the choice by Professor Jones the majority of times in the past and then predict that they will continue to outperform Professor Jones in the future. In contrast, $\mathscr{D}$ is saying that there will be a magical flipping of relative performance, in which suddenly Professor Jones is doing better in the future, if and only if they did worse in the past.

The NFL for supervised learning theorem tells us that there are as many universes in which algorithm $\mathscr{C}$ will perform worse than algorithm $\mathscr{D}$—so that Professor Jones magically starts performing worse than Professor Smith—as there are universes the other way around. This is true even if Professor Jones produces the random search algorithm no matter what the value of x is (i.e., no matter what objective function they are searching). In other words, just because Professor Smith produces search algorithms that outperform random search in the past, without making some assumption about the probability distribution over universes, we cannot conclude that they are likely to continue to do so in the future.

The possible implications for how tenure decisions and grant awards are made will not be considered here.

6 Exploiting the Relation Between Supervised Learning and Search to Improve Search

Given the preceding discussion, it seems that supervised learning is closely analogous to search, if one replaces the "search algorithm" with a "learning algorithm" and the "objective function" with a "target function." So it should not be too surprising that the inner product formula and NFL theorem for search have analogs in supervised learning. This close formal relationship between search and supervised learning means that techniques developed in one field can often be "translated" to apply directly to the other field.

[1] Note that as a special case, we could have each of the two professors always produce the exact same search algorithm for any objective function they are presented. In this case comparing the performance of the two professors just amounts to comparing the performance of the two associated search algorithms.

A particularly pronounced example of this occurs in the simplest (greedy) form of the Monte Carlo optimization (MCO) approach to search [19]. In that form of MCO, one uses a data set d to form a distribution $q(x \in X)$ rather than (as in most conventional search algorithms) directly forming a new x. That q is chosen so that that one expects the expected value of the objective function, $\sum_x q(x) f(x)$ to have a low value, i.e., so that one expects a sample of $q(.)$ to produce an x with a good value of the objective function. One then forms a sample x of that $q(.)$ and evaluates $f(x)$. This provides a new pair $(x, f(x))$ that gets added to the data set d, and the process repeats.

MCO algorithms can be viewed as variants of random search algorithms like genetic algorithms and simulated annealing, in which the random distribution governing which point to sample next is explicitly expressed and controlled, rather than be implicit and only manipulated indirectly. Several other algorithms can be cast as forms of MCO (e.g., the cross-entropy method [20], the MIMIC algorithm [21]). MCO algorithms differ from one another in how they form the distribution q for what point next to sample, with some not trying directly to optimize $\sum_x q(x) f(x)$ but instead using some other optimization goal.

It turns out that the problem of how best to choose a next q in MCO is formally identical to the supervised learning problem of how best to choose a hypothesis h based on a training set d [22, 23]. If one simply re-interprets all MCO variables as appropriate supervised learning variables, one transforms any MCO problem into a supervised learning problem (and vice versa). The rule for this re-interpretation is effectively a dictionary that allows us to transform any technique that has been developed for supervised learning into a technique for (MCO-based) search. Regularization, bagging, boosting, cross-validation, stacking, etc., can all be transformed this way into techniques to improve search.

As an illustration, we can use the dictionary to translate the use of cross-validation to choose a hyperparameter from the domain of supervised learning into the domain of search. Training sets become data sets, and the hyperparameters of a supervised learning algorithm become the parameters of an MCO-based search algorithm. For example, a regularization constant in supervised learning gets transformed into the temperature parameter of (a form of MCO that is a small variant of) simulated annealing. In this way using the dictionary to translate cross-validation into the search domain shows us how to use it on one's data set in search to dynamically update the temperature in the temperature-based MCO search algorithm. That updating process proceeds by running the MCO algorithm repeatedly on subsets of one's *already existing* data set d. (No new samples of the objective function f beyond those already in d are involved in this use of cross-validation for search, just like no new samples are involved in the use of cross-validation in supervised learning.)

Experimental tests of MCO search algorithms designed by using the dictionary have established that they work quite well in practice [22, 23]. Applying the dictionary to create analogs of bagging and stacking in the context of search, in addition to creating analogs of cross-validation, have all been found to transform an initially powerful search algorithm into a new one with improved search performance.

Of course, these experimental results do not mean that there is any formal justification for these kinds of MCO search algorithms; NFL for search cannot be circumvented.

7 Free Lunches and Future Research

There are many avenues of research related to the NFL theorems that have not yet been properly explored. Some of these involve *free lunch* theorems that concern fields closely related to search, e.g., co-evolution [24]. Other free lunches arise in supervised learning, e.g., when the loss function does not obey the conditions that were alluded to above [10].

However it is important to realize that none of these (no) free lunch theorems concern the *covariational* behavior of search and/or learning algorithms. For example, despite the NFL for search theorems, there are scenarios where, for some f's, $\mathbb{E}(\Phi \mid f, m, \mathscr{A}) - \mathbb{E}(\Phi \mid f, m, \mathscr{B}) = k$ (using the notation of the appendix), but there are no f's for which the reverse is true, i.e., for which the difference $\mathbb{E}(\Phi \mid f, m, \mathscr{B}) - \mathbb{E}(\Phi \mid f, m, \mathscr{A}) = k$. It is interesting to speculate that such "head-to-head" distinctions might ultimately provide a rationale for using many almost universally applied heuristics, in particular for using cross-validation rather than anti-cross-validation in both supervised learning and search.

There are other results where, in contrast to the NFL for search theorem, one does not consider fixed search algorithms and averages over f's but rather fixes f and averages over algorithms. These results allow us to compare how intrinsically hard it is to search over a particular f. They do this by allowing us to compare two f's based on the sizes of the sets of algorithms that do better than the random algorithm does on those f's [25]. While there are presumably analogous results for supervised learning, which would allow us to measure how intrinsically hard it is to learn a given f, nobody currently knows. All of these issues are the subject of future research.

Appendix

A.1 NFL and Inner Product Formulas for Search

To begin, expand the performance probability distribution:

$$\begin{aligned} P(\phi \mid \mathscr{A}, m) &= \sum_{d_Y^m} P(d_Y^m \mid \mathscr{A}, m) P(\phi \mid d_Y^m, \mathscr{A}, m) \\ &= \sum_{d_Y^m} P(d_Y^m \mid \mathscr{A}, m) \delta(\phi, \Phi(d_m^Y)), \end{aligned} \tag{5}$$

where the delta function equals 1 if its two arguments are equal, and 0 otherwise. The choice of search algorithm affects performance only through the term $P(d_Y^m \mid \mathscr{A}, m)$. In turn, this probability of d_Y^m under $\mathscr{A}$ is given by

$$P(d_Y^m \mid \mathscr{A}, m) = \sum_f P(d_Y^m \mid f, m, \mathscr{A}) P(f \mid m, \mathscr{A})$$
$$= \sum_f P(d_Y^m \mid f, m, \mathscr{A}) P(f). \tag{6}$$

Plugging in gives

$$P(\phi \mid \mathscr{A}, m) = \sum_f P(f) D(f; d_Y^m, \mathscr{A}, m), \tag{7}$$

where

$$D(f; d_{\cdot}^m \mathscr{A}, m) := \sum_{d_Y^m} P(d_Y^m \mid f, \mathscr{A}, m) \delta(\phi, \Phi(d_m^Y)). \tag{8}$$

So for any fixed ϕ, $P(\phi \mid \mathscr{A}, m)$ is an inner product of two real-valued vectors each indexed by f: $D(f; d_Y^m, \mathscr{A}, m)$ and $P(f)$. Note that all the details of how the search algorithm operates are embodied in the first of those vectors. In contrast, the second one is completely independent of the search algorithm.

This notation also allows us to state the NFL for search theorem formally. Let B be any subset of the set of all objective functions, Y^X. Then Eq. (7) allows us to express the expected performance for functions inside B in terms of expected performance outside of B:

$$\sum_{f \in B} \mathbb{E}(\Phi \mid f, m, \mathscr{A}) = constant - \sum_{f \in Y^X \setminus B} \mathbb{E}(\Phi \mid f, m, \mathscr{A}), \tag{9}$$

where the constant on the right-hand side depends on the performance measure $\Phi(.)$ but is independent of both $\mathscr{A}$ and B [3]. Expressed differently, Eq. (9) says that $\sum_f E(\Phi \mid f, m, \mathscr{A})$ is independent of $\mathscr{A}$. This is the core of the NFL for search, as elaborated in the next section.

A.2 NFL for Search When We Average over $P(f)$'s

To derive the NFL theorem that applies when we vary over $P(f)$'s, first recall our simplifying assumption that both X and Y are finite (as they will be when doing search on any digital computer). Due to this, any $P(f)$ is a finite-dimensional real-valued vector living on a simplex Ω. Let π refer to a generic element of Ω. So

$\int_\Omega d\pi \; P(f \mid \pi)$ is the average probability of any one particular f, if one uniformly averages over all distributions on f's. By symmetry, this integral must be a constant, independent of f. In addition, as mentioned above, Eq. (9) tells us that $\sum_{f \in B} \mathbb{E}(\Phi \mid f, m, \mathscr{A})$ is independent of $\mathscr{A}$. Therefore for any two search algorithms $\mathscr{A}$ and $\mathscr{B}$,

$$\sum_f \mathbb{E}(\Phi \mid f, m, \mathscr{A}) = \sum_f \mathbb{E}(\Phi \mid f, m, \mathscr{B}),$$

$$\sum_f \mathbb{E}(\Phi \mid f, m, \mathscr{A}) \left[\int_\Omega d\pi \; P(f \mid \pi) \right] = \sum_f \mathbb{E}(\Phi \mid f, m, \mathscr{B}) \left[\int_\Omega d\pi \; P(f \mid \pi) \right],$$

$$\sum_f \mathbb{E}(\Phi \mid f, m, \mathscr{A}) \left[\int_\Omega d\pi \; \pi(f) \right] = \sum_f \mathbb{E}(\Phi \mid f, m, \mathscr{B}) \left[\int_\Omega d\pi \; \pi(f) \right],$$

$$\int_\Omega d\pi \; \sum_f \mathbb{E}(\Phi \mid f, m, \mathscr{A}) \pi(f) = \int_\Omega d\pi \; \sum_f \mathbb{E}(\Phi \mid f, m, \mathscr{B}) \pi(f), \quad (10)$$

that is,

$$\int_\Omega d\pi \; \mathbb{E}_\pi(\Phi \mid m, \mathscr{A}) = \int_\Omega d\pi \; \mathbb{E}_\pi(\Phi \mid m, \mathscr{B}). \quad (11)$$

We can re-express this result as the statement that $\int_\Omega d\pi \; \mathbb{E}_\pi(\Phi \mid m, \mathscr{A})$ is independent of $\mathscr{A}$.

Next, let Π be any subset of Ω. Then our result that $\int_\Omega d\pi \; \mathbb{E}_\pi(\Phi \mid m, \mathscr{A})$ is independent of $\mathscr{A}$ implies

$$\int_{\pi \in \Pi} d\pi \; \mathbb{E}_\pi(\Phi \mid m, \mathscr{A}) = constant - \int_{\pi \in \Omega \setminus \Pi} d\pi \; \mathbb{E}_\pi(\Phi \mid m, \mathscr{A}), \quad (12)$$

where the constant depends on Φ but is independent of both $\mathscr{A}$ and Π. So if any search algorithm performs particularly well for one set of $P(f)$'s, Π, it must perform correspondingly *poorly* on all other $P(f)$'s. This is the NFL theorem for search when $P(f)$'s vary.

A.3 NFL and Inner Product Formulas for Supervised Learning

To state the supervised learning inner product and NFL theorems requires introducing some more notation. Conventionally, these theorems are presented in the version where both the learning algorithm and the target function are stochastic. (In contrast, the restrictions for search—presented above—conventionally involve a

deterministic search algorithm and deterministic objective function.) This makes the statement of the restrictions for supervised learning intrinsically more complicated.

Let X be a finite **input** space, Y a finite **output** space, and say we have a **target distribution** $f(y_f \in Y \mid x \in X)$, along with a **training set** $d = (d_X^m, d_Y^m)$ of m pairs $\{(d_X^m(i) \in X, d_Y^m(i) \in Y)\}$, which is stochastically generated according to a distribution $P(d \mid f)$ (conventionally called a **likelihood**, or "data-generation process"). Assume that based on d we have a **hypothesis distribution** $h(y_h \in Y \mid x \in X)$. (The creation of h from d—specified *in toto* by the distribution $P(h \mid d)$—is conventionally called the **learning algorithm**.) In addition, let $L(y_h, y_f)$ be a **loss function** taking $Y \times Y \to \mathbb{R}$. Finally, let $C(f, h, d)$ be an **off-training set cost function**,[2]

$$C(f, h, d) \propto \sum_{y_f \in Y, y_h \in Y} \sum_{q \in X \setminus d_X^m} P(q) L(y_f, y_h) f(y_f \mid q) h(y_h \mid q), \tag{13}$$

where $P(q)$ is some probability distribution over X assigning nonzero measure to $X \setminus d_X^m$.

All aspects of any supervised learning scenario—including the prior, the learning algorithm, the data likelihood function, etc.—are given by the joint distribution $P(f, h, d, c)$ (where c is the value of the cost function) and its marginals. In particular, in [9] it is proven that the probability of a particular cost value c is given by

$$P(c \mid d) = \int df dh \; P(h \mid d) P(f \mid d) M_{c,d}(f, h) \tag{14}$$

for a matrix $M_{c,d}$ that is symmetric in its arguments so long as the loss function is. $P(f \mid d) \propto P(d \mid f) P(f)$ is the posterior probability that the real world has produced a target f for you to try to learn, given that you only know d. It has nothing to do with your learning algorithm. In contrast, $P(h \mid d)$ is the specification of your learning algorithm. It has nothing to do with the distribution of targets f in the real world. So Eq. (14) tells us that as long as the loss function is symmetric, how "aligned" you (the learning algorithm) are with the real world (the posterior) determines how well you will generalize.

This supervised learning inner product formula results in a set of NFL theorems for supervised learning, once one imposes some additional conditions on the loss function. See [9] for details.

[2]The choice to use an off-training set cost function for the analysis of supervised learning is the analog of the choice in the analysis of search to use a search algorithm that only searches over points not yet sampled. In both the cases, the goal is to "mod out" aspects of the problem that are typically not of interest and might result in misleading results: ability of the learning algorithm to reproduce a training set in the case of supervised learning, and ability to revisit points already sampled with a good objective value in the case of search.

References

1. Wolpert, D.H.: The lack of a prior distinctions between learning algorithms and the existence of a priori distinctions between learning algorithms. Neural Comput. **8**, 1341–1390, 1391–1421 (1996)
2. Schaffer, C.: A conservation law for generalization performance. In: International Conference on Machine Learning, pp. 295–265. Morgan Kaufmann, San Mateo (1994)
3. Wolpert, D.H., Macready, W.G.: No free lunch theorems for optimization. IEEE Trans. Evol. Comput. **1**(1), 67–82 (1997)
4. Whitley, D., Rowe, J.: A "no free lunch" tutorial: sharpened and focused no free lunch. In: Theory of Randomized Search Heuristics: Foundations and Recent Developments, pp. 255–287. World Scientific, Singapore (2011)
5. Igel, C., Toussaint, M.: A no-free-lunch theorem for non-uniform distributions of target functions. J. Math. Model. Algorithms **3**(4), 313–322 (2005)
6. Poland, K., Beer, K., Osborne, T.J.: No free lunch for quantum machine learning (2020). Preprint, arXiv:2003.14103
7. Peel, L., Larremore, D.B., Clauset, A.: The ground truth about metadata and community detection in networks. Sci. Adv. **3**(5), e1602548 (2017)
8. Godfrey-Smith, P.: Theory and Reality: An Introduction to the Philosophy of Science. University of Chicago Press, Chicago (2009)
9. Wolpert, D.H.: The relationship between PAC, the statistical physics framework, the Bayesian framework, and the VC framework. In: The Mathematics of Generalization, pp. 117–215. Addison-Wesley, Reading (1995)
10. Wolpert, D.H.: On bias plus variance. Neural Comput. **9**(6), 1211–1243 (1997)
11. Mackay, D.J.C.: Information Theory, Inference, and Learning Algorithms. Cambridge University Press, Cambridge (2003)
12. Jefferys, W.H., Berger, J.O.: Ockham's razor and Bayesian analysis. Am. Sci. **80**(1), 64–72 (1992)
13. Loredo, T.J.: From Laplace to SN 1987a: Bayesian inference in astrophysics. In: Maximum Entropy and Bayesian Methods, pp. 81–142. Kluwer Academic, Dordrecht (1990)
14. Gull, S.F.: Bayesian inductive inference and maximum entropy. In: Maximum Entropy and Bayesian Methods, pp. 53–74. Kluwer Academic, Dordrecht (1988)
15. Wolpert, D.H.: On the Bayesian "Occam factors" argument for Occam's razor. In: Petsche T., et al. (eds.) Computational Learning Theory and Natural Learning Systems III. MIT Press, Cambridge (1995)
16. Li, M., Vitanyi, P.: An Introduction to Kolmogorov Complexity and Its Applications. Springer, Berlin (2008)
17. Lattimore, T., Hutter, M.: No free lunch versus Occam's razor in supervised learning. In: Algorithmic Probability and Friends. Bayesian Prediction and Artificial Intelligence, pp. 223–235. Springer, Berlin (2013)
18. Wolpert, D.H.: The relationship between Occam's razor and convergent guessing. Complex Syst. **4**, 319–368 (1990)
19. Ermoliev, Y.M., Norkin, V.I.: Monte Carlo optimization and path dependent nonstationary laws of large numbers. Technical Report IR-98-009, International Institute for Applied Systems Analysis, March 1998
20. Rubinstein, R., Kroese, D.: The Cross-Entropy Method. Springer, Berlin (2004)
21. De Bonet, J.S., Isbell, C.L. Jr., Viola, P.: Mimic: Finding optima by estimating probability densities. In: Advances in Neural Information Processing Systems - 9. MIT Press, Cambridge (1997)
22. Rajnarayan, D., Wolpert, D.H.: Exploiting parametric learning to improve black-box optimization. In: Jost, J. (ed.) Proceedings of ECCS 2007 (2007)

23. Rajnarayan, D., Wolpert, D.H.: Bias-variance techniques for Monte Carlo optimization: cross-validation for the CE method (2008). arXiv:0810.0877v1
24. Wolpert, D.H., Macready, W.: Coevolutionary free lunches. Trans. Evol. Comput. **9**, 721–735 (2005)
25. Macready, W.G., Wolpert, D.H.: What makes an optimization problem hard? Complexity **1**, 40–46 (1995)